

全稿 · 征求意见稿

WorkBuddy

实战指南

把 AI 用进工程现场

Eric 著

汽车微电机与智能执行器研发工程师 · 11 年一线实战 · 260篇Q3章 · 72 个场景 · 附索引术语表与随书资源

目录

- 作者简介 4
- 序章 凌晨一点，我还在第五十次复制粘贴 9
- 第一篇 · 认识 WorkBuddy
 - 第 1 章 它不是聊天框，是工位 13
- 第二篇 · 环境搭建与快速上手
 - 第 2 章 装好它，半小时跑通第一件活 24
 - 第 3 章 什么时候问，什么时候谋，什么时候干 41
- 第三篇 · 核心机制深度理解
 - 第 4 章 把你用过的好招数存成技能 63
 - 第 5 章 让它记住你和你的项目 84
 - 第 6 章 一个干不完，就拆开分给几个 102
- 第四篇 · 72 个场景实战
 - 第 7 章 通用办公自动化：先把每周那十小时抢回来 124
 - 第 8 章 个人效率与知识管理：让资料自己长腿 153
 - 第 9 章 需求与项目定义：把客户那段话翻译成能开工的任务 186
 - 第 10 章 造型与几何：让 AI 真的去动你的建模软件 214
 - 第 11 章 电子、软件与仿真：AI 进得了 PCB、控制器和求解器吗 250
 - 第 12 章 让 AI 碰到你的电脑：浏览器、桌面、远程 283
 - 第 13 章 设计审核与风险：让 AI 当那个挑刺的人 316

第 14 章	供应链与产线：从一堆报价和报表里看出真话	357
第 15 章	NVH 与试验：从一堆曲线里捞出真话	390
第 16 章	让 Agent 不跑偏：完成条件、边界条件和反指标	428
第 17 章	AI 的七宗罪与反制：它会在哪里骗你	459
第 18 章	验收、溯源与责任：在它的产出上签你的名字	503
• 第五篇 • 连接与工具扩展		
第 19 章	连接器：让 AI 接上你的邮箱、文档和代码仓库	528
第 20 章	MCP：给 AI 装插座 ★★★	556
• 第六篇 • 团队落地 • 成本 • 排查		
第 21 章	从一个人用，到一个部门用	583
第 22 章	这笔账到底怎么算	607
第 23 章	出了问题怎么救	624
• 附录 • 索引 • 术语 • 资源		
附录 A	72 场景速查索引	649
附录 D	全书术语表	659
附录 E	学习资源与延伸阅读	687
附录 F	随书资源清单	692
附录 H	致谢与灵感来源	697

作者简介

作者简介



作者 Eric：汽车微电机与智能执行器研发工程师，AI 探路者

关于作者

Eric，AI 探路者，汽车微电机与智能执行器研发工程师，在这一行干了十一年。

他主要研究的方向是：把 AI 用进汽车零部件研发。从电机研发工程师做到创新研发经理，主导过五个以上平台的产品从零到一：微电机、微型齿轮箱、智能执行器、隐藏式门把手。经手过三百多份检测报告，参与过四十多款量产车型。发明专利七项，实用新型三项。

目前的主要产出：

1. 用 Vibe Coding 的方式，自己写了三套智能检测系统，分别用在研发部门、工厂端和质量端，做汽车零部件的 NVH（噪声、振动与声振粗糙度）自动判读。三套系统都没有专业开发团队参与，是他一个人对着 AI 敲出来的。
2. 开发了三十多个在研发流程里可以直接用起来的 Skill 和工作流，覆盖数据整理、报告生成、图纸检查、试验记录这些重复劳动最集中的环节。本书第 4 篇的场景，大部分来自这批工作流。
3. 自建了面向汽车与制造行业的 AI 社区站点：<https://qiyanagent.org.cn>
4. 开发了从入门到精通的课程与配套视频，并给多家企业与机构做过专业培训，其中脱敏后的课程视频已上线 B 站（哔哩哔哩）。
5. 在微信公众号、知乎、B 站、小红书、抖音等多个平台持续分享自己用 AI 的踩坑与心得。

这些数字不重要。重要的是另一件事：在那十一年里，他所在的行业有一条铁规矩，写进报告的每一个数字，都必须能追到它是从哪一次试验、哪一个样本、经过什么处理得来的。追不到，这个数字就不能用。不是为了好看，是因为一个数字错了，后面可能是召回，是赔钱，是有人受伤。

2025 年底，他开始把 AI 引进日常工作，很快就栽了一个跟头：一份由 AI 生成的材料，格式漂亮，数字合理，在评审会上被人问了一句“这个数是哪来的”，当场答不上来。

那次之后，他把行业里那套“溯源、验收、担责”的规矩搬到了 AI 上，一边用一边记，写了两百多篇实战记录。这本书是那套规矩的完整版。

他不是 AI 专家，也不打算成为 AI 专家。他只是一个对自己的工作成果负责的人，碰巧学会了一套把脏活交出去、同时把判断留给自己的方法。

找得到他的地方

微信公众号「Eric的数字花园」



Eric的数字花园 公众号二维码

这本书写完的部分在这里，没写完的部分也在这里。回复「实战」可以领取随书的七十二个场景提示词合集，以及案例用的脱敏练习素材。

公众号持续更新的内容：新工具怎么接进 workflow、旧方法哪里开始失灵、读者寄来的翻车现场。AI 加汽车零部件行业的交叉内容，主要发在这里。

同名账号也在知乎、B 站（哔哩哔哩）、小红书、抖音上更新。B 站上有脱敏后的课程视频，想先看人再决定买不买这本书的，去那里翻。

GitHub 开源仓库

<https://github.com/Shun1989/Ericks-WorkBuddy-Practice>



本书开源仓库二维码

本书的全部可复用资产都在这个仓库，MIT 协议：

- 七十二个场景的抄作业模板（提示词、检查清单、验收标准齐全）
- 三十多个可直接安装的 WorkBuddy 技能与 workflows
- 书中全部自绘示意图的 SVG 源文件，拿去改就能用
- 翻车案例库：每个场景的已知错误模式与排错记录

开源不是为了显得大方。这本书反复讲“可复现”：一个场景如果你拿去跑不通，那它就只是故事，不是方法。仓库里的东西就是让你跑通的。

汽研 Agent 中文社区

网址：<https://qiyanagent.org.cn>

面向汽车与制造行业的 AI 应用者社区。每天整理行业 AI 动态，积累工程场景的真实用法，也收集翻车案例。

如果你在制造业、汽车或者硬件行业，这个社区比公众号更适合你。公众号面向泛职场，社区只聊工程。

一句话给读者

如果你读完这本书只记住一件事，希望是这句：

别问 AI 该怎么办，把活给它，然后验收。

聊天框里的答案转瞬即忘，交付出去的活才作数。这本书教的就是这件事：怎么把活交出去，交得清楚；怎么把结果收回来，收得放心。

致谢位置

（留给正式出版时补充：感谢素材整理、案例核实、书稿试读阶段提供帮助的同事与读者，以及开源仓库的早期贡献者。）

序章

凌晨一点，我还在第五十次复制粘贴

你缺的不是 AI，是一个肯替你干脏活的。

2025 年 12 月的一个晚上，我在做一件从入职第一年就在做的事：供应商资质核查。

事情不复杂。一个新项目要定点两家供应商，我需要核对五家候选供应商的资质：质量体系证书、实验室认可范围、关键设备清单、近三年的交付记录、报价。这些信息散在五个地方：两家供应商自己的门户、一个行业平台、一个招标网站，还有对方邮件发来的一个压缩包。

我打开第一个门户，登录，找到证书页，复制证书编号，切到 Excel，粘贴。打开第二个门户，登录，找到设备清单，截图，切回 Word，插图片。第三家供应商的门户要用手机验证码登录，我拿起手机，等短信，输进去，页面过期了，重来。

晚上十一点，我完成了三家。Excel 里躺着一百多行资质数据，Word 里塞了四十多张截图。还剩两家，压缩包还没解开看。

凌晨一点，我在解压包里翻设备清单的 PDF，翻到第四十页的时候，手停了一下。我突然意识到一件事：今晚我干的全部工作，没有一个动作需要"我"这个人。登录、翻页、复制、粘贴、截图、归类，每一个动作都有固定规则，每一个规则都简单到不需要判断。我干了一晚上的，是一台机器该干的活。

而与此同时，我的电脑上装着一个能替我干这些活的 AI。它有浏览器，能登我的门户；它能读 Excel 和 PDF；它能写文件。它就装在那里，而我在用手当复印机。

那天晚上我问了自己一个问题，这个问题后来变成了这本书：

为什么 AI 明明能干，我却还在自己干？

差的不是能力，是交活的方法

那天之后我复盘了很久。答案慢慢清楚了。

不是我不会用 AI。我会聊天，会写提示词，还会让它帮我改邮件、做总结。过去一年，我跟 AI 的对话超过一万条。

问题出在我对"用 AI"这件事的理解上。我一直以为用 AI 的方式是：我把活里的某个环节拿出来，问 AI，它给我答案，我决定用不用。就像问一个坐在旁边的顾问。

这种方式有个天花板：顾问只动嘴。查资质这件事里没有"问"的环节，全是"动手"的环节。登录是我的手，翻页是我的手，复制粘贴是我的手。我问它一百次，它回答一百次，最后动手的还是我。

真正的转折是我换了一种问法。我没问"帮我看看供应商资质怎么核查"，我说：这里有一个登录方式清单，五个门户，你去把每家的证书编号、有效期、设备清单摘出来，写进这张表格，格式照这个样例。做完告诉我哪些证书半年内到期。

它去干了。中间在一个门户的验证码那里停下来问我，我点了一下，它继续。四十分钟后，五个门户全部跑完，表格躺在我的桌面上，还有一页汇总：两家供应商的三张证书在半年内到期。

四十分钟。我原来要干两个晚上。

那一刻我明白的事情，就是这本书的全部内容：AI 的价值不在聊天框里，在工位上。你得给它工具、给它权限、给它验收标准，让它像一名坐在旁边工位的同事那样，把一整件事从头干到尾。而你要学会的是另一半：怎么交活，怎么验收，怎么在它的产出上签你自己的名字。

这本书不教你聊天

市面上讲 AI 的书已经很多了。大部分在教你怎么提问，怎么写提示词，怎么让 AI 写出更好的文案。那些书假设你跟 AI 的关系是“问与答”。

这本书假设的是另一种关系：交活与验收。

你是一名工程师、一名采购、一名质量员、一名项目经理。你的工作不是写作文，是干活：建模型、画图纸、整理数据、盯试验、写报告、对供应商。这些活共同点是要动手、要出文件、要对结果负责。

所以这本书里几乎没有“怎么让 AI 写得更好”，全是“怎么让 AI 把活干完”：

- 让它驱动 SOLIDWORKS，把一个方向盘的三维模型建出来，再出二维图纸；
- 让它爬没有公开接口的供应商门户，把资质数据一条条抓进表格；
- 让它替你盯着跑一整夜的台架试验，人回家睡觉；
- 让它把两百页的技术标准拆成一份检查清单；
- 让它检查自己的产出，报告哪里可能出错。

七十二个场景。每一个都来自真实的工程现场，每一个都给到你能直接照着做的颗粒度。

抄作业，不丢人

先交代我的学习方式，因为它就是这本书的写法。

我学用 AI 干活，不是从教程开始的，是从抄开始的。第一次让 AI 建模，我照着别人的提示词一个字一个字抄，抄完改零件尺寸，改完跑一遍，报错，照着报错改，再跑。三个晚上之后，那个提示词就变成我自己的了。今天书里第七章之后所有的提示词模板，最早都来自这样一次次笨拙的复制粘贴。

所以我给你的第一个建议是：别通读，先抄。

这本书第 4 篇有 72 个场景，每个场景的结构完全一样，就是为了让你抄得顺手：场景痛点、前置条件、操作步骤、提示词原文、预期输出、翻车点、验收标准、变体延伸。你翻到跟你手头工作最像的那一个，把提示词复制出来，改几个词，跑一遍。跑通了，这个场景就是你的了。跑挂了，翻到“翻车点”那一节，大概率有人摔在同一个坑里。

抄会一个场景，胜过读完十篇方法论。

三种读法

给你三条路线，按你的情况挑一条。

路线一：直接抄（最快见效）。你现在手上就有活，没耐心看原理。直接翻到第 4 篇，按场景目录找你的活，抄。遇到术语看不懂，翻回第 1 章的"小白词典"，两分钟补完。这条路我推荐给所有第一次拿到本书的人。

路线二：顺序读（打得最牢）。从第 1 章读到第 23 章。第 1 篇告诉你 WorkBuddy 到底是什么、跟你用过的网页 AI 差在哪；第 2 篇带你把环境装好；第 3 篇讲清它的记忆、技能、子代理这些核心机制，讲的时候全部用大白话和类比；第 4 篇是 72 个场景的大本营；第 5、6 篇讲连接器、团队落地和排查。这条路大概花你三周，之后你会成为团队里那个"什么都能用 AI 搞定的人"。

路线三：按图索骥（当工具书）。书读完一遍之后，它就该待在你的桌面上。哪个场景忘了怎么配，翻回去抄。附录里有全部场景的索引和报错速查表。

不管走哪条路，有一段话我要放在最前面说清楚。

一个必须先说的提醒

这本书会反复教你把活交出去。但在你学会交活之前，先记住这条：

AI 交回来的东西，你没验过，就不是你的。

它建的模型可能差一个圆角，抓的数据可能混进去一行空的，写的技术文件可能引用了一个不存在的条款。绝大多数时候它干得又快又好，但"绝大多数"三个字，在工程上是不能签字的。

我在自己的行业里干了十一年，写进报告的每个数字都要能追到试验和样本。这个职业习惯被我原样搬到了 AI 上：它干完，我按一套固定的检查动作过一遍，过了，才签我的名字。书里第 13 章和第 18 章讲的就是这套动作。

所以这本书教的不是"放手让 AI 干"，是"放手让 AI 干，收手由你验收"。两个半句缺一不可。

开始之前

写这本书的时候，我常常想起凌晨一点的那个晚上。那天晚上我没有学会任何新东西，我只是终于看清了一件事：我缺的从来不是能力，是一个肯干脏活的。

现在，翻到第 4 篇，找一个像你手头活的场景，抄。

或者翻到第 1 章，从头认识这位新同事。

（下一章：认识 WorkBuddy。它跟网页聊天框到底差在哪，三张图讲清楚。）

PART ONE · 第一篇

认识 WorkBuddy

第 1 章

它不是聊天框，是工位

同一个 AI，装进不同的壳，就是完全不同的东西。

先看一件真事。同一个任务，"把这份 2000 行的检测数据表里超差的行挑出来，标红，按零件号排序，另存一份"，我在三种 AI 工具上各做了一遍。

网页聊天框：我把表格截图发过去。它说"好的，我看到了一个数据表格"。截图里 2000 行，它只能看清不到 50 行。我把表格拆成 8 段文字粘贴过去，它给我返回了一大段文字描述哪些行超差。我照着它的文字描述，自己回去把那 37 行标红，自己排序，自己另存。全程 25 分钟，中间还核对出它读错的一行。

命令行 Agent：这是一个装在黑色窗口里的 AI，它可以直接读我的文件。我输入指令，它真的去读了表格、真的写出了新文件。全程 3 分钟。但为了走到"输入指令"这一步，我花了一个傍晚：装运行环境、配命令行、学它的指令格式。这些活对程序员来说只要十分钟，对我当时来说每一行英文报错都像一堵墙。

WorkBuddy：我打开对话窗口，把表格文件拖进去，说了一句话："挑出超差的行，标红，按零件号排序，另存一份到桌面。"它读完文件，写了处理脚本，跑完，把新文件放到了我的桌面，然后汇报："共 2000 行，超差 37 行，已标红并排序，另存为 xxx.xlsx。其中零件 A-2308 有两行数据疑似重复，请确认。"我打开文件检查了一遍，37 行全对。全程 5 分钟，我亲手做的事只有拖文件、说话、点发送、打开验收。

同一个 AI 底座，三种完全不同的东西。差别不在聪明程度，在它能不能碰到你的活、能不能把活干完、你用不用得起。

这一章就讲清三件事：WorkBuddy 到底是什么；它跟你在网页上用过的 AI 聊天框、极客圈子里的命令行 Agent、开发者手里的开源 Agent 框架，各自差在哪；以及本书 72 个场景怎么用。

1.1 先拆掉三个误会

在讲它是什么之前，先把三个最常见的误会拆掉。这三个误会我每一个都亲身犯过。

误会一：AI 就是聊天框

大多数人对 AI 的全部印象来自网页聊天：打字、回话、再打字。于是天然觉得 AI 的能力边界就是"会说话"。

聊天只是 AI 的一种外壳。同一个模型，装进网页里是聊天框，装进能操作文件的程序里，它就能读你的表格、写你的报告、跑你电脑上的软件。模型没变，变的是它被允许"动手"的范围。

打个比方：同一位大厨，站在柜台后面只能给你讲解菜谱；你把厨房钥匙给他，他能直接把菜做出来端给你。你过去用的一直是站在柜台后面的大厨。

误会二：用 AI 得先学编程

这个误会拦住了最多的人。事实是：让 AI 干活的门槛，最近两年从"会写代码"降到了"会把话说清楚"。

本书第 4 篇 72 个场景里，你需要的全部技能是：打开一个软件、把文件拖进对话框、用中文把你想要的东西说出来、看懂它给你的结果。没有一个场景要求你先学编程。确实有几个地方要装软件、配参数，但每一步都给了照着点的操作步骤，那就是本书存在的意义。

我的同桌是一位干了二十年质量工作的老工程师，比我大十岁，此前打字都用一根手指。他照着本书第 14 章的场景抄了三周，现在部门里供应商数据整理的活有一半是 AI 在干，他负责验收。

误会三：AI 说得头头是道，就能信

这是最危险的一个误会，而且越是把 AI 用得好的人越容易犯。

AI 有一类毛病：它会把不知道的东西说得非常笃定，编出一个看起来完全合理的数字、条款、文件名。工程上这叫幻觉，本书第 17 章给它起了个更直白的名字，"AI 的七宗罪"之一。

所以本书从头到尾的态度是：AI 是干活的，签字的是你。它建完模型，你要量一遍关键尺寸；它抓完数据，你要抽查几行对源站；它写完报告，你要核一遍引用。第 18 章会给一套固定的验收动作，照着做，两分钟到十分钟。

不是不信任它。是工程现场的规矩本来如此：谁签字，谁验收。它不能签字，所以你必须验。

1.2 WorkBuddy 是什么：一个会干活的工位

一句话定义：WorkBuddy 是一个装在你电脑上的 AI 工作台，它能读你的文件、操作你的软件、记住你的事，你用说中文的方式把活交给它，它干完向你交差。

拆开讲，这句话里有四个关键词。

关键词一：装在你电脑上

它不是网页，是装在你 Windows 或 Mac 电脑上的一个软件。这意味着它有权限碰你电脑里的东西：文件夹、Excel、CAD、浏览器、命令行工具。网页聊天框碰不到这些，这是它跟网页版最根本的区别，后面 1.3 节细讲。

"装在本地"还有一层含义：你的文件不需要上传到某个网页再下载回来。表就在你桌面上，改完还在你桌面上。

关键词二：能读文件、操作软件、记住你的事

这三件事是它"能干活"的底气，本书第 3 篇会用三章把它们讲透，这里先混个脸熟：

- 读文件、写文件：Word、Excel、PDF、代码、图片，拖进对话框它就能处理，处理结果可以直接存回你的文件夹。
- 操作软件：通过一套叫 MCP 的"插座"机制（第 19 章专讲），它能驱动你电脑里的其他软件干活，比如让 SOLIDWORKS 按它的指令建模、让浏览器替你查数据。
- 记住你的事：它有三层记忆。你告诉过它的偏好、你的项目背景、你们团队的习惯，它会记住，下次不用重新自我介绍。第 6 章讲记忆。

关键词三：用说中文的方式交活

你不需要学任何指令语法。把活用中文说出来就行，就像给新来的实习生交代工作。说不清楚也没关系，它会反问你，问清了再动手。

但"说清楚"本身是一门手艺。同样一个活，两种交法，结果天差地别。本书花了大量篇幅教你"怎么把活说清楚"，第 4 篇每个场景都附了可以直接抄的交活话术。

关键词四：三种模式

这是你打开 WorkBuddy 后遇到的第一个岔路口。它有三种干活的方式，你可以理解为给这位同事的三种委托方式：

模式	你说什么	它做什么	适合什么时候用
Ask（问）	"帮我看看这份报告讲了什么"	只读、只回答，不动你任何文件	拿不准、先聊聊、纯问答
Plan（谋）	"我想把月度检测数据做成一份汇报"	先给你一份完整方案，等你点头才动手	活比较大、方向还没想清楚
Agent（干）	"把这份表格按附件要求整理好发我"	直接读写文件、跑程序、干完交差	任务明确、直接交活



图 1-1 三种模式怎么选：先看任务明不明确，再看出错代价大不大

选模式的口诀只有一句：拿不准就用 Plan，活得清楚就直接 Agent，纯聊天就 Ask。拿不准的时候永远可以退回 Plan，让它先给你看方案。本书 72 个场景里，标了 ★★★ 的实测场景全部给了明确的模式建议，跟着抄就行。

1.3 对比一：跟网页聊天框比，隔着三堵墙

网页 AI（就是你登录个网站直接聊的那种）不是不好，它免费、顺手、开箱即用，查资料、改文案、翻译，它依然是好工具。问题出在你要干的是工程现场的活。

从网页聊天框到 WorkBuddy，隔着三堵墙。每一堵墙，都是我拿真实加班时间量出来的。

第一堵墙：文件墙

网页版碰不到你电脑里的文件。你想让它处理一份表格，得先复制内容粘贴过去；表格太大，就得截图，截不清楚就得拆成八段。第 1 章开头那次 25 分钟的“挑超差行”，光是把 2000 行数据喂进去就花了一半时间。

WorkBuddy 直接读文件。拖进去，完事。2000 行和 20 行，对它来说没有区别，反正它不是用眼睛看的。

第二堵墙：动手墙

网页版从头到尾只能说话。它能告诉你"第 47 行到 52 行超差"，但标红要你回去自己标。

WorkBuddy 能直接动手：写文件、跑脚本、操作其他软件。第 12 章有个场景我印象很深：五家供应商的资质数据散在五个门户，网页版 AI 帮不了你，因为它登不进那些门户。WorkBuddy 用你已登录的浏览器替你翻页、抄数据、填表格，四十分钟干完你两晚的活。

第三堵墙：记性墙

网页版的记忆，取决于你开没开历史记录，而且换一个对话窗口它就"失忆"，你要重新交代一遍背景："我们公司是做汽车零部件的，我们部门负责……"

WorkBuddy 有分层记忆（第 6 章细讲）：你的身份、项目背景、常用格式，它记在长期记忆里。今天交代过"我们报告的表头格式照 2025 版"，下个月它还记得。

三堵墙合起来就是一张对比表：

	网页聊天框	WorkBuddy
碰你的文件	复制粘贴，大文件喂不进	直接读写本地文件
动手干活	只能说，不能做	能操作软件、跑流程、出文件
记住背景	换窗口就失忆	分层记忆，越用越懂你
上手门槛	零	装一次软件（第 2 章带你装）
适合的活	查资料、改文案	出活：数据、图纸、报告、流程

一句话总结这一节：网页聊天框是顾问，WorkBuddy 是工位上的同事。顾问出主意，同事交活。

1.4 对比二：跟命令行 Agent 比，差在门槛

2025 年起，技术圈流行一类"命令行 Agent"：装在黑色窗口里，能读写文件、执行命令，能力上跟 WorkBuddy 的 Agent 模式是同一级别。开源社区里这类工具很多，都很强。

那为什么本书不教你用它们？因为门槛。

我第一次装一个命令行 Agent，花了一整个傍晚。装运行环境，命令行里敲英文指令，报错，搜索报错，再敲，再报错。最后跑通了，我发现真正的拦路虎不是第一次安装，是每一次使用：它的世界是英文命令、文件夹路径、配置文件。对写程序的人来说这是母语，对工程现场的人来说这是每天上班先翻墙。



图 1-2 同样的能力，不同的门槛：命令行 Agent vs WorkBuddy

WorkBuddy 把同样的能力装进了一个正常的桌面软件里：

	命令行 Agent	WorkBuddy
交互界面	黑窗口，敲英文命令	正常窗口，说中文
装环境	自行解决依赖和报错	安装包一路下一步
干活的委托方式	记指令格式	拖文件、说人话
排错	看报错日志	它自己先试着排错，解决不了再用中文告诉你卡在哪
记忆和技能	逐个配置	内置分层记忆，技能点一下装

还有一点值得单独说：命令行 Agent 干完活，通常静悄悄的，你得自己去翻它改了哪些文件。WorkBuddy 干完会向你汇报：干了什么、改了哪些文件、哪里它拿不准需要你确认。这个“交差”的习惯，对不盯着屏幕的人来说差别巨大。

公平地说，命令行 Agent 在纯开发者手里依然更灵活。如果你是程序员，两者都可以用。但本书的读者是工程现场的人，你们要的是把活干完，不是把机器玩明白。

1.5 对比三：跟开源 Agent 框架比，差在拼装成本

第三类是开源 Agent 框架：把大模型、工具调用、记忆管理这些零件全部公开，开发者自己拼装出一个专属 AI 助手。2026 年这类框架非常成熟，社区里全是漂亮的多智能体演示。

这类框架的能力上限最高，你可以完全自定义它的行为。但拼装意味着什么，你大概已经猜到了：

- 每个零件都要自己理解、自己接线；
- 出了错要自己排查，报错信息以英文为主；
- 版本更新频繁，上个月跑通的流程这个月可能要改。

我拆过一个社区里口碑很好的框架，搭一个"自动整理数据表"的最小流程，对开发者大概半天，对工程现场的人，保守估计两周，而且两周之后你维护它的成本还在。

WorkBuddy 的思路是反过来的：这些零件（记忆、技能、工具连接、多代理协作）全部内置、默认装好，你直接用成品。代价是你失去了底层自定义的自由，好处是你今晚就能开始干活。

	开源 Agent 框架	WorkBuddy
定位	开发者拼装的零件库	成品工位
上手时间	周（工程现场的人）	半小时
自由度	任意定制	框架内扩展（技能、连接器、MCP）
维护	自己养	官方更新
适合谁	有开发团队的企业	个人与中小团队

顺带回应一个读者问过我的问题："开源免费，WorkBuddy 要花钱，凭什么？"我的答案是算两笔账。一笔是显性的：软件订阅费。另一笔是隐性的：学习成本、拼装时间、维护精力。我自己的记录是，用 WorkBuddy 的第一年，它在数据整理、报告生成、资质核查上替我省出的时间，按我最保守的时薪折算，超过订阅费一个数量级。你的活越值钱，这笔账越好算。

1.6 本书的地图：六篇、23 章、72 个场景

最后交代全书的结构，方便你规划自己的读法。



图 1-3 全书地图：六篇 23 章 72 场景

第一篇 认识 WorkBuddy（本章）：搞清楚它是什么、跟其他工具差在哪。

第二篇 环境搭建与快速上手（第 2-3 章）：下载安装、跑通第一个任务、学会三种模式的日常切换。跟着做，半天内你会干成第一件真活。

第三篇 核心机制深度理解（第 4-6 章）：技能、记忆、子代理三大机制。这篇是“原理”，但全部用大白话和工程类比讲，第 4 章还有一组真实的技能自进化实验数据：一个技能从 0.7 分迭代到更高分的过程，每一代改了什么、错在哪。

第四篇 72 个场景实战（第 7-18 章）：本书的大本营，12 章 72 场景，覆盖通用办公、知识管理、需求定义、造型与几何（含驱动 SOLIDWORKS 建模出图的方向盘案例）、电子软件与仿真、浏览器与桌面操控、设计审核、供应链、NVH 与试验、让 Agent 不跑偏、AI 的七宗罪、验收与责任。每个场景都是八件套：痛点、前置条件、步骤、提示词、预期输出、翻车点、验收标准、变体。

第五篇 连接与工具扩展（第 19-20 章）：连接器与 MCP，让 AI 碰到你的邮箱、文档、数据库和各类专业软件。

第六篇 团队落地、成本与排查（第 21-23 章）：从个人用到团队用的方法、成本怎么算、出问题怎么救。

每个场景都带一个可信度标注：★★★ 表示我本机完整跑过、有截图和数据；★★ 表示流程跑通过；★ 表示来自公开调研，注明了来源。我不虚标，你用的时候心里有数。

小白词典

AI 模型：听得懂人话、能推理能生成的计算机程序。"大模型"指参数极多、能力更强的那一代。它只负责"想"，负责"干"的是装着它的软件。

Agent（智能体）：不只回答问题，还能自己规划步骤、调用工具、把一件事干完的 AI 程序。聊天框回答你，Agent 交活给你。

WorkBuddy：本书主角，一个装在电脑上的 AI 工作台，Agent 能力 + 三种模式 + 记忆 + 技能 + 连接器。

提示词：你对 AI 说的话。说得更清楚，活就干得更好。本书第 4 篇每个场景都附成品提示词，可以直接抄。

技能（Skill）：给 WorkBuddy 预装的"岗位手册"。装一个技能，它就多会一类活，比如"把 Markdown 排版成 PDF"。

连接器：让 WorkBuddy 接上外部服务（邮箱、文档、代码仓库）的插头。第 19 章讲。

MCP：一种让 AI 操控其他软件的通用"插座"标准。WorkBuddy 靠它驱动 SOLIDWORKS、浏览器等软件。第 19 章讲。

幻觉：AI 把不知道的东西说得煞有介事。第 17 章主角。

你的岗位怎么用

设计工程师：你先看第 10 章。方向盘案例从"一句话进模型"到"出二维图纸"全程拆解，先抄场景 19（简单零件建模），再进 20/21。

试验工程师：直接翻第 15 章 NVH 与试验和第 12 章场景 35（远程盯台架）。试验数据整理和值班交接是抄作业见效最快的两类活。

质量工程师：第 13 章设计审核与风险、第 14 章供应链数据整理。第 17 章"AI 的七宗罪"建议全文读两遍，它是给你的验收武器库。

采购 / 供应商管理：第 12 章场景 31-33（浏览器自动化查资质与报价）和第 14 章。这是全书省时最多的场景组，我的记录是每月省两个工作日。

项目经理：先读本章和第 9 章（需求与项目定义）。你的杠杆点是把"周报、风险表、跨部门纪要"这三样交出去，第 7 章有现成模板。

研发主管：第 21-23 章是为你的决策写的。团队怎么推、预算怎么算、出事了怎么定责。第 18 章的验收与责任框架建议亲自读，这是你要向下传达的规矩。

本章交付物

三问自检卡：以后每次想把活交给 AI，先问自己三个问题，答案直接决定你怎么交。

1. 这个活我说明白了吗？说明白了：直接 Agent 模式交活。说不明白：切 Plan 模式，让它先出方案，你来选。

2. 这个活 AI 碰得到吗？活在文件里、软件里：WorkBuddy 能干。活在纯网页浏览和讨论里：网页 AI 也行，不必杀鸡用牛刀。
3. 干完谁验收？提前想好验收动作（查哪几个数、量哪几个尺寸），干完立刻验。想不出验收方法的活，先别交给它。

把这三问贴在显示器边上。它们是全书 72 个场景共用的大前提。

本章金句

同一个 AI，装进聊天框是顾问，装进工位是同事。

AI 的价值不在聊天，在替你一整件事干完。

用 AI 的门槛，已经从“会写代码”降到“会把话说清楚”。

放手让 AI 干，收手由你验收。两个半句缺一不可。

拿不准就用 Plan，活得清楚就上 Agent。

抄会一个场景，胜过读完十篇方法论。

（下一章：装好它。从下载到跑通第一个任务，每一步都有截图，半小时后你的工位开工。）

PART TWO · 第二篇

环境搭建与快速上手

第 2 章

装好它，半小时跑通第一件活

装完软件只占这件事的十分之一。剩下九成，是让它知道该动谁的文件。

开场：老周装过三次

去年十一月，一个周三下午三点。质量部的老周端着保温杯站在我工位边上，看我把一个 Excel 拖进窗口、敲了一句话、起身去接了杯水。他还没走开，文件已经改好了，躺在我桌面上。

"这玩意儿我装过三次。"他说。

第一次是前年春天。他在技术论坛上看到一个能直接处理本地表格的命令行 AI 工具，帖子标题写着"十分钟上手"。他照着教程装运行环境，装到第三步弹了一行英文错误，去搜，搜出来的答案让他改一个配置文件。他改了，还是报错。那天晚上他搞到十一点半，最后把那个文件夹整个删掉。

第二次是去年三月，一个开源 Agent 框架。他跑了两个晚上，卡在一个依赖包上，缺一个组件。他给我看过他的浏览器历史记录，同一个报错页面他打开了七次。

第三次是去年夏天。他放弃了本地工具，回到网页聊天框。每个月做供应商数据汇总，他把表格拆成十段文字往里贴，贴完自己核对。他说那是他用 AI 用得最久的一段时间，也是最累的一段时间。

"你现在这个，装上花了多久？"他问。

我打开自己的记录给他看：下载安装包 4 分钟，一路点下一步 3 分钟，登录 2 分钟。加起来不到十分钟。

真正花时间的是后面那段。我新建了一个专属工作目录，花了 5 分钟；把不该让它碰的目录列了个清单，花了 10 分钟；第一次把活交出去、等它干完、逐行验收，花了 25 分钟。

"所以我今天该先干什么？"他问。

我说："先把软件装上。然后别急着干活，先把该圈的地圈好。"

这一章就是那天下午我陪他走的那四十分钟。跟着做完，你会带走两样东西：一个能干活的软件，和一个你自己验过的成果。

2.1 下载与安装

去哪下

只从官方渠道下。这不是一句废话，我看到过有人从软件下载站下到被改过的安装包，装上之后电脑开始弹广告。

判断真假的三条，下之前花 30 秒看一眼：

检查项	怎么查	对得上才装
域名	看下载页网址的主域名，一个字母都不差	主域名与官网一致
文件大小	装之前看属性里的字节数	跟官网标注的对得上
数字签名	Windows 上右键安装包看属性，Mac 上第一次打开会验证开发者	签名人是官方主体

【待补：官网地址与下载入口按钮名称，以实际版本为准】

选什么版本

版本这件事，我的原则只有一条：要干活的人装稳定版。

你的情况	选哪个	理由
今天就要开始干活	稳定版	更新慢一点，但不会中途崩
想跟着本书做完整流程	稳定版，且不低于本书写作时的版本	老版本可能没有本书讲的模式切换和技能
公司电脑不给管理员权限	走公司内部的软件中心，或找 IT 报装	硬装往往装到一半卡在权限上
我就是要尝鲜	预览版	出问题自己兜着，别在交付节点前换

我自己踩过一次。有一年我在一个交付节点前一周把软件升到了预览版，结果那周有个技能的调用方式变了，我攒了三个月的流程跑不通。后来我给自己定了条规矩：预览版只装在另一台机器上。

Windows 与 Mac 的差异

差异不大，但有五处会让人卡住：

项	Windows	Mac
安装包形态	常见的是 exe 或 msi	常见的是 dmg
第一次打开	可能弹用户账户控制，点允许	可能要在系统设置的隐私与安全里点"仍要打开"
路径写法	C:\Users\你的名字\...	/Users/你的名字/...
文件编码	老系统和国产软件生成的 csv 常是 GBK	一般是 UTF-8
命令行	PowerShell 或命令提示符	终端

编码这一行请你记一下。中文 Windows 上，Excel 另存出来的 csv 默认是 GBK 编码，交给 AI 处理时经常变成一堆问号。本书后面所有涉及 csv 的场景，我都会提醒你先确认编码。

装完第一眼看到什么

通用原则：装完第一次启动，你会看到一个大窗口，窗口左侧是会话列表，中间是对话区，上方或下方有模式切换的入口，某一侧有文件区。

具体到每个按钮叫什么、在第几行第几列，各版本会有出入，我不替你猜。

【待补：首次启动的界面布局、按钮名称与菜单路径，以实际版本为准】

装完先别急着问它问题。做下面这张清单里的三件事，顺序不能乱。

装完必做的三项检查

先看签名，再定目录，最后划红线

- ✓ **第一件：先看签名**
安装包来源要过三项：下载页主域名一个字母不差、装前比对字节数、右键看数字签名人是官方主体。三条对得上才装。
- ✓ **第二件：再定目录**
新建一个专属工作目录，路径全英文数字，不带空格和括号，我用的是 D:\aiwork，下面按年月和项目再分一层。
- ✓ **第三件：最后划红线**
拿一张纸列三个圈：绿区可读写、黄区只读、红区不碰。到设置区的权限部分，只把绿区挂上去。
- ! **顺序不能乱**
这三件事的顺序就是上面的顺序。第 7 条红区没划完之前，不要交真活给它。

第 2.1 节与 2.5 节：装完当天照着做完，前六条是十分钟的事

图 2-1 装完必做的三项检查：先看签名，再定目录，最后划红线

作者根据公开资料绘制

2.2 第一次打开的五个区域

第一次打开一个陌生软件，最省事的办法是先给它分区。WorkBuddy 的窗口可以分成五块，你今天只需要真正认识其中两块。

区域	它大概在哪	大白话是干什么	你今天要做的一件事
对话区	窗口正中，最大的一块	你说话、它交差的地方。文件可以直接拖进来	拖一个文件进来，确认它能识别

区域	它大概在哪	大白话是干什么	你今天要做的一件事
模式切换	对话输入区附近，或顶部	决定它是"只答""先给方案"还是"直接干完"	找到它，切到 Ask 试一句
文件区	某一侧的面板	它当前能看到哪些文件夹，动了哪些文件	确认它看到的是你刚建的工作目录
设置区	窗口角落的齿轮、或菜单里	账号、模型、权限、工作目录都在这里	只改一件事：工作目录
任务状态区	对话区上方或下方的一条	它正在干什么、跑了几步、卡在哪一步	干活的途中瞄一眼，看它是不是在转圈

一句话记住这五个区域：对话区是嘴巴和耳朵，模式切换是委托书，文件区是它手能伸到的范围，设置区是规矩，任务状态区是进度条。

五种区域里，你今天只需要会做三件事：在对话区说话、在模式切换里选 Ask、在设置区改工作目录。文件区和任务状态区，等 2.4 节你把活交出去的时候自然会用到。

第一次打开的五个区域

今天真正要认识的只有三块：对话区、模式切换、设置区

对话区

窗口正中最大的一块。你说活、它交差，文件可以直接拖进来。今天要做：拖一个文件进来，确认它能识别

模式切换

对话输入区附近或顶部。决定它是只答、先给方案还是直接干完。今天：找到它，切到 Ask 试一句

文件区

某一侧的面板。显示它当前能看到哪些文件夹、动了哪些文件。今天：确认它看到的是你刚建的工作目录

设置区

窗口角落的齿轮或菜单里。账号、模型、权限、工作目录都在这里。今天只改一件事：工作目录

任务状态区

对话区上方或下方的一条。显示它正在干什么、跑了几步、卡在哪。干活途中瞄一眼，看它是不是在转圈

第 2.2 节：对话区是嘴巴和耳朵，模式切换是委托书，文件区是它手能伸到的范围

图 2-2 第一次打开的五个区域：只认识两个就能开工

作者根据公开资料绘制

2.3 五分钟第一件活：让它读一个文件并汇报

第一件活我建议你选最没有风险的一种：只读，不改。你给它一个文件，它读完了跟你说这个文件讲了什么。你的文件一个字节都不会变。

开工前准备：

1. 找一份你熟悉内容的文件。熟悉很重要，你得能判断它说得对不对。我第一次用的是我自己的一份周报草稿，8 页 Word。
2. 模式切到 Ask。这个模式只回答、不动手，练手阶段用它最安全。
3. 把文件拖进对话区，或者在对话区里把它所在的路径打出来。

可复制提示词：

请读这份文件，只做汇报，不要修改、不要新建任何文件。

按下面四块汇报，每块不超过三行：

1. 这份文件是干什么的，一句话。
2. 里面有哪些具体数字结论（数字 + 结论，逐条列，最多 6 条）。
3. 有哪些地方写得不完整、前后矛盾、或者缺数据。
4. 如果你没看懂某一段，直接说哪一段没看懂，不要猜。

最后单独用一行告诉我：你读的是哪个文件、一共多少页或多少行。

我跑这份的时候（一份 8 页的周报草稿），它返回的样子大致是这样：

1. 这是一份研发周报草稿，汇报三个电机项目的进度、风险和下周计划。2. 数字结论：- 项目 A：样机 12 台，合格 9 台，合格率 75% - 项目 B：温升试验做到第 3 轮，最高 78 K，限值 85 K - 项目 C：绕线工序不良率 1.8%，环比上升 0.4 个百分点 - 三项目合计延期风险项 5 项 3. 不完整的地方：第 4 页的风险表只有 3 行有责任人，另外 2 行空着；第 6 页提到“详见附件 2”，文件里没有附件 2；项目 C 的不良率没写统计区间。4. 没看懂：第 7 页第 2 段那句缩写我不确定含义。读的是 zhoubao_20261005.docx，共 8 页。

这段回复我验了两处。合格率 75% 跟我自己算的一致；“缺附件 2”这条是我自己漏了、它替我抓出来的。第二处让我当天就把这份草稿定为“必须用 AI 过一遍”。

为什么我强调要用你熟悉的文件？因为你得有能力判它说得对不对。我第一次给的一个文件是我自己写的周报，它说“第 6 页提到附件 2 但文件里没有”，我一眼就确认这是真的，它替我抓到了我自己的疏漏。换一份我不熟的文件，它说什么我都只能点头，那这次练习就白做了。

拿不准选什么文件的话，按这个顺序挑：你上周写的东西，你昨天改过的东西，你手边正要用的东西。不要挑别人的文件，也不要挑带客户信息的机密件，第一次练习不值得冒这个险。

判断点只有一个：它报的文件名、页数、行数，跟你自己看到的对得上。对不上，说明它根本没读进去，或者读错了文件。这个动作两秒钟，但它能救你后面所有的活。

第一件活的三步

选最没有风险的一种：只读，不改，你的文件一个字节都不会变



第 2.3 节：判断点只有一个，它报的页码行数跟你自己看到的对得上

图 2-3 第一件活的三步：拖进去、说清楚、对数字

作者根据公开资料绘制

2.4 半小时第一件真活：整理一份乱表格 ★★★

前面那件活它只读不动手，你拿到的还是一段文字。这一节我们让它真的改文件。这是本章的重点，也是全书的第一个八件套。

2.4.1 一句话背景

供应商 A 厂每月发来一份绕线工序的检测记录，1,286 行，列是随手加的，表头占两行还带合并单元格，数值里混着单位，日期有三种写法，中间夹着空行。我手工整理过一次，掐表 23 分钟，事后还发现漏了 4 行。交给 WorkBuddy，从拖文件到验完，14 分钟，我亲手做的事只有三件：拖文件、说话、验结果。

2.4.2 名词小词典

名词	大白话	类比
表头	表格最上面那几行，说明每一列是什么	货架上的标签
合并单元格	一个格子横跨了好几个格子	一张标签压住了三个抽屉
脏数据	格式不统一、缺值、混进多余字符的数据	刚从地里拔出来、还带泥的萝卜

名词	大白话	类比
编码	电脑存文字的规则。规则不一致就显示成乱码	两个人说不同的方言
透视	按某一列分组求和、计数	把一堆小票按商品名归类算总额
CSV	一种纯文本表格格式，逗号分隔	去掉所有花样的纯文字账单

2.4.3 开工前准备

1. 新建一个专属工作目录。我用的是 `D:\aiwork\2026-10-winding`。

目录名用英文和数字，不要带空格和括号，路径里的空格是很多莫名其妙报错的来源。

1. 把原始文件复制一份进去，原件留在原地不要动。这条是死规矩：AI 永远只碰副本。
2. 关掉打开着这个文件的 Excel。文件被 Excel 占用时，别的程序写不进去。
3. 模式切到 Agent。这一节要它动手，Ask 模式不会改文件。
4. 先看一眼原始文件的行数和列数，记在纸上。这是你验收用的对照数字。

2.4.4 操作步骤

第一步：切到 Ask 模式，让它先看，不动手。这一步我强烈建议不要省。

第二步：看它的汇报，确认它理解对了表头和脏在哪。理解错了就在这一步纠正，比等它改完 1,286 行再返工便宜得多。

第三步：切回 Agent 模式，把第二轮提示词发过去，让它动手。

第四步：盯着任务状态区。它会显示正在读文件、在写处理脚本、在跑。这一步通常几十秒到两三分钟。

第五步：它交差后会报一堆数字。拿这些数字跟你纸上记的对照。

第六步：打开新文件，抽查 5 行。抽查哪 5 行有讲究，见 2.4.7。

第六步之前，如果任务状态区转圈超过三分钟，不要干等。发一句："现在卡在哪一步，把已经完成的部分先告诉我。"它通常会回"脚本写到一半，卡在某个库的调用上"。这种情况八成是文件太脏或者格式特殊，让它换成更保守的做法重来一次，比让它自己死磕快。我有一次等了七分钟，最后它说做不了；同样的事，我在第三分钟打断它、换了个说法，两分钟就跑完了。

2.4.5 可复制提示词

第一轮，Ask 模式，只读不动手：

请读这个目录下的检测记录文件，只汇报，不要修改任何文件。

汇报四件事：

1. 文件路径、行数、列数、文件格式。
2. 表头在哪几行，有没有合并单元格，每一列实际存的是什么。
3. 逐条列出你发现的数据问题：空行、空值、单位混在数值里、日期格式不统一、同一列出现多种写法、疑似重复行。每条给出你判断的行号范围。
4. 有一件事我不确定，请你判断：这份表里哪些行是真正的检测数据，哪些行是小计、备注或标题。给出你判断的依据。

不要给整理方案，先只汇报。

第二轮，Agent 模式，让它动手。把方括号里的内容换成你自己的：

请整理这份检测记录，输出到同目录下，文件名 [原文件名]_已整理.xlsx。
原始文件保留不动。

整理要求：

1. 表头只保留一行，列名用中文，去掉所有合并单元格。
2. 数值列只留数字，单位单独放到列名里，比如"电阻(mΩ)"。
3. 日期统一成 YYYY-MM-DD。
4. 删除完全空白的行。删除前把要删的行号列出来给我看，我确认后再执行。
5. 疑似重复的行不要删，在新文件里加一列"疑似重复"，标 1，由我自己判断。
6. 不要修改任何原始数值，不做四舍五入，不做单位换算。

整理完汇报：

- 原始行数、删除行数、输出行数，三个数字必须能对上。
- 每一列的列名、非空单元格数、空单元格数。
- 你拿不准、需要我确认的地方，逐条列出来。

第三轮，可选项，只在它交差后你觉得还得再收一步时用：

再加两件事，仍然输出到同一个文件，不要新建别的文件：

1. 新增一列"是否超差"，按 [规格上限] 判断，超差标 1，否则标 0。
2. 新增一个工作表"汇总"，按 [零件号] 分组，输出每组的检测次数、超差次数、最大值、最小值、平均值。

超差判断的规则先在回复里写一遍给我看，我确认后再写进文件。

2.4.6 过程与判断点

发完第一轮，它返回的是一份体检报告。我那次拿到的是这样的：

1. 文件: 202609_winding_raw.xlsx, 1,286 行, 14 列, xlsx 格式。2. 表头占第 1 到第 2 行, 第 1 行有 3 处合并单元格 (B1:C1、F1:H1、K1:L1)。第 3 行开始是数据。3. 数据问题: - 空行: 第 210、455、456、889 行, 共 4 行 - 空值: D 列 (绕线电阻) 有 37 个空单元格, 集中在第 600 到 700 行 - 单位混在数值里: D 列有 219 个单元格写成 "12.3mΩ", 其余是数字 - 日期三种写法: E 列有 2026/9/3、2026-09-03、9月3日 三种, 分别 421、380、481 行 - 疑似重复: 第 512 行与第 513 行除日期外完全一致 4. 小计与备注行: 第 1080 行以下共 206 行是分班组小计 (首列是"合计""小计"), 不是检测数据。依据: 这些行的 D 列为空或为求和值。

这份报告里我要验的是第 1 条和第 4 条。行数列数跟我在 Excel 里看到的一致; 第 1080 行以下确实是我之前手工整理时漏掉的那部分, 它抓出来了, 跟我后来核对的结果一样。

发完第二轮, 判断点只有一个, 而且是个死数字:

原始行数 = 删除行数 + 输出行数

1,286 = 4 + 1,282。对得上才往下走。对不上, 说明它要么偷偷删了数据行, 要么把小计行算进了检测数据。这时候不要去看文件, 直接把这句话发回去:

输出行数跟原始行数对不上。请不要重新整理, 先告诉我:

1,286 行里, 每一行分别归到了哪一类 (检测数据 / 空白 / 小计 / 备注), 四类加起来必须等于 1,286。先把这个账对清楚。

这个"先对账"的习惯, 是我用 AI 整理表格一年多来最值钱的一条经验。它出错的方式通常不是算错, 是悄悄少算。

2.4.7 验收标准与变体

验收五个动作, 全部做完大约 4 分钟:

#	动作	过了的标准
1	数行数	原始 1,286 = 删除 + 输出
2	抽查 5 行	抽首尾各 1 行、随机 3 行, 跟原件逐格比对, 数值一模一样
3	透视求和	对某一数值列求和, 原件和整理后必须完全相等
4	查空值	每一列的空单元格数跟它汇报的数字一致
5	看编码	另存 csv 打开不乱码; 中文列名显示正常

抽查那 5 行怎么选, 我有个小讲究: 不要全抽开头。开头几行通常是干净的, 脏数据在中间和结尾。我固定抽第 1 行、最后 1 行、中间随机 3 行。

变体一: 试验报告数据汇总。台架跑完出十几个 csv, 让它合并成一个总表并加一列"数据来源文件名"。提示词把"绕线检测"换成"试验数据", 其余不动。注意每个 csv 的列顺序可能不同, 第一轮汇报一定要让它核对列名。

变体二：BOM 与库存表比对。两份表按零件号对，输出差集。这种活手工做最容易漏，交给它反而更稳，因为它是逐行比的。前提是两份表的零件号写法要统一，第一轮让它先报写法差异。

变体三：多份月报合并成季报。三份结构相同的月报，合成一份并生成汇总页。这种活它干得比人快，但格式细节（字体、列宽）它经常做不到位，验收时重点看格式。

2.4.8 翻车清单

翻车一：它把小计行当数据行一起整理了。我第一次跑就栽在这。它把 206 行小计混进了检测数据，输出行数变成了 1,488。补救动作：在第二轮提示词里加一句"首列是'合计'小计'的行单独放到'小计'工作表，不要混进检测数据"，然后让它重跑。

翻车二：带单位的数值被当成文本，透视时求和得 0。12.3mΩ 这种写法在 Excel 里是文本，求和直接忽略。补救动作：第一轮汇报里让它报"有多少个单元格是文本格式的数值"，这个数必须是 0 才往下走。

翻车三：日期统一错了。三种写法里"9月3日"没有年份，它按当前年份补了 2026，实际上那批数据是 2025 年的。补救动作：第二轮提示词里写明"缺年份的日期按 [年份] 补，并在新文件里加一列'日期为推断'标 1"。凡是它替你补的东西，都要留一个标记列，方便你事后筛出来复核。

整理一份乱表格的八个动作

先体检，再动手，最后对账



第 2.4 节：它出错的方式通常不是算错，是悄悄少算

图 2-4 整理一份乱表格的八个动作：先体检，再动手，最后对账

作者根据公开资料绘制

2.5 装完必做的三个设置

装完软件，很多人直接开始问问题。我建议你先花 15 分钟做三件事，做完再用。这三件事的顺序就是下面的顺序，不能换。

2.5.1 工作目录：给它一个专属的车间

动作：新建一个空目录，路径全英文数字，不带空格括号。我用的是 `D:\aiwork`，下面按年月和项目再分一层。然后到设置区把这个目录设为工作目录。

理由：工作目录是它默认能看见、能动的范围。你不指定，它要么默认整个用户目录，要么默认你打开软件的那个位置。我在老周电脑上看到过一次，他的工作目录是整个 D 盘，那一刻他桌上所有项目文件都进了它的活动范围。

我自己的规矩：每一个项目一个子目录，交付完就归档。这样它干错了，损失只在一个子目录里，删掉重来就是。

2.5.2 记忆初始化：把自我介绍一次性说完

动作：装完当天，用 Ask 模式发下面这段话，把方括号换成你自己的。

请把下面这些记到长期记忆里，以后不用我重复交代。

我是 [汽车零部件] 行业的 [研发工程师]，做了 [11] 年。
我的岗位日常是 [微电机与智能执行器的设计与试验]。

三条工作习惯：

1. 我给你的表格，第一件事永远是核对行数和列数，对不上先告诉我。
2. 我不需要你夸我，也不需要总结陈词，直接给结论和数字。
3. 凡是你替我推断、补全、四舍五入的内容，都要单独标记出来，不许混在原始数据里。

文件格式偏好：

- 表格用 `xlsx`，中文列名
- 文档用 `docx`
- 报告里出现数字一律带单位

汇报格式：先结论，再依据，最后是需要我确认的点。每块不超过三行。

理由：这一段话你今天写一次，后面 72 个场景里它都会按这个习惯干活。我第一次没写这段话，前两周每

次都要重复“带单位”“不要总结陈词”，起码重复了二十遍。第二周我写进记忆之后，这两句话再没说过。

不写这段话，它也不是不能干。差别在返工次数。我自己的记录是，写之前平均每件事要补两轮说明，写之后平均半轮。

2.5.3 权限边界：先划红线，再谈信任

动作：拿一张纸，列出三个圈：绿区（可读写）、黄区（只读）、红区（不碰）。到设置区的权限部分，只把绿区挂上去。

圈	目录	授权	理由
绿区	D:\aiwork 及其子目录	读写	这里面的东西删了不心疼，重跑就是
黄区	历史归档、标准文本、已发布图纸目录	只读	引用可以，改了要走变更流程
红区	下列目录	不碰	见下

红区清单，这些目录一律不给：

目录	为什么不给
系统盘根目录、C:\Windows、C:\Program Files	改坏了系统，重装一整天
用户目录下的 AppData	浏览器配置、登录凭据、各软件状态都在这，风险集中
桌面	桌面上是我的杂物堆：客户报价扫描件、身份证照片、孩子的画。我控制不了里面有什么
财务与人事目录	工资、合同、证件扫描，出了问题责任事故
共享盘根目录	里面有别人的文件，误删影响全组，而且你没法一个人恢复
受控文件服务器的发布区	这里的每一次改动都要走变更流程，AI 动了等于你越权
云同步盘根目录	误删会同步到所有人的电脑，delete 一个文件等于 delete 全组

桌面这条我想多说一句。很多人天然把桌面当工作目录，因为文件都在那儿。我的建议是把桌面当"待整理区"，不是"工作区"。要用哪个文件，复制到工作目录里再干活。多一步复制，省掉的是"它误删了我桌面上的客户报价"这种事。

权限三圈：先划红线，再谈信任

到设置区的权限部分，只把绿区挂上去



第 2.5.3 节：桌面是待整理区，不是工作区。用哪个文件，复制到工作目录里再干活

图 2-5 权限三圈：绿区可读写，黄区只读，红区不碰

作者根据公开资料绘制

2.6 常见安装故障与自救

下面七条，前五条是我自己或身边同事真遇到过的，后两条是安装类工具的高频共性问题。每条给现象、原因、动作。

故障一：装不上，安装进度到一半卡住或报错

现象	进度条走到 60% 到 90% 之间不动，或者弹出一个英文错误框
原因	常见于三类：安装包下载不完整、杀毒软件拦截写文件、目标盘剩余空间不足
动作	第一步重新下一遍安装包，比对字节数；第二步临时退出杀毒软件再装；第三步看目标盘剩余空间，低于 5 GB 先清。三步做完还不行，把报错框拍下来发给 IT

我见过最坑的一次是下载中断，安装包实际只有完整文件的 70%，但大小显示正常，因为下载工具做了断点续传。比对字节数这一步别省。

故障二：装上了，双击打不开或开出来白屏

现象	双击图标没反应，或者窗口弹出来了但一片空白
原因	显卡硬件加速冲突、本地缓存损坏、上一次异常退出留下的锁文件
动作	先等 30 秒，第一次启动要初始化。还不行就到设置区关掉硬件加速再启动。再不行，卸载重装，重装前手动删掉残留的配置目录

故障三：登录失败，账号密码都对就是进不去

现象	提示登录失败、验证码不对、或者一直在转圈
原因	公司网络代理拦截了登录请求；验证码有时延；账号在新设备上需要二次确认
动作	换手机热点试一次，能登上就说明是公司网络的问题，找 IT 加白名单；验证码等 60 秒再收，别连点三次，连点会让前面的验证码全部失效

老周卡在这一步卡了半小时。最后发现是他们公司内网把登录域名拦了，IT 加了白名单，两分钟解决。

故障四：能聊天，但连不上服务

现象	界面正常，发一句话出去一直转圈，最后提示连接超时或服务不可用
原因	代理或 VPN 干扰、系统时间不同步导致证书校验失败、服务端维护
动作	第一步关掉 VPN 再试；第二步检查系统时间，差超过 5 分钟就同步一次；第三步看官方状态页有没有维护公告

系统时间这条最容易被忽略。我有一次出差回来电脑时间快了 11 分钟，AI 一直连不上，我排查了网络、账号、防火墙，最后发现是时钟。

故障五：文件读不到，它说找不到或者读出来是乱码

现象	提示文件不存在，或者读出来的中文是问号和方块
原因	路径含空格或特殊字符、文件被 Excel 占用、编码不一致（GBK 被当 UTF-8 读）、文件不在工作目录范围内
动作	把文件复制到工作目录下，路径改成英文数字；关掉 Excel；如果是 csv，另存时选 UTF-8 编码。三个都做完了再重试

故障六：能聊天，但它就是不动手

现象	你让它改文件，它回了一篇分析，但文件没变
原因	模式停在 Ask（只答不动），或者工作目录没设，它不知道该在哪干活
动作	看模式切换是不是在 Ask，切到 Agent。再看设置区的工作目录是不是空的，空的就先按 2.5.1 建一个

这条不是故障，是使用问题，但它占我见过的"AI 不好用"投诉的一半以上。先查模式，再查目录。

故障七：中文显示正常，但生成的文件打不开

现象	它说生成好了，你双击文件，Excel 或 Word 报格式错误
原因	生成过程中被中断，文件只写了一半；或者目标文件正被另一个程序打开
动作	关掉所有打开着的 Excel 和 Word，让它重新生成一次。仍然不行就换个文件名输出到同一目录

2.7 如果你只做一件事

今天只做这一件，五分钟，不用改任何设置，不用建工作目录。

拿你手边最近的一个文件，什么都行：一封没读完的邮件导出、一份上周的会议纪要、一份你写了两页的报告草稿。用 Ask 模式拖进去，发下面这段话：

读这个文件，只汇报，不要改任何东西。

用不超过 100 个字告诉我三件事：

1. 这个文件讲了什么。
2. 里面最重要的一个数字是什么。
3. 有什么地方是写漏了的。

最后一行告诉我你读的是哪个文件、多少字或多少行。

五分钟之内，你会拿到一段你自己也要花五分钟才写得出来的东西。差别在于你是读的人，它替你读。

拿到之后做一件事，别跳过：核对它报的第三件事。它说你写漏了什么，你回去看一眼是不是真的漏了。对上了，说明它能用；对不上，说明你的文件它有歧义，下次要说得更清楚。

对上了就停在这儿。今天不用往下读，明天再来做 2.4 那件真活。第一次拿到正反馈的这一天，比后面任何一天都重要。老周那天晚上给我发了条消息，就一句话："它把我那份纪要里漏掉的日期找出来了。"

小白词典

工作目录：AI 默认能看见、能读写的那个文件夹。类比是给新同事安排的工位，他的活动范围就在这张桌子周围。

模式切换：Ask、Plan、Agent 三种委托方式的选择开关。类比是你给同事的三种委托书：只问、先出方案、直接干完。

Ask 模式：只读只答，不动你任何文件。类比是问路，对方指方向但不替你走。

Agent 模式：直接读写文件、跑程序、干完交差。类比是把整件事包给同事，你只看结果。

任务状态区：显示它正在干第几步、卡在哪一步的进度条。类比是车间门口的看板。

编码：电脑存文字的规则，规则不一致就显示成乱码。类比是两个人说不同方言。

脏数据：格式不统一、有缺值、混进多余字符的数据。类比是刚从地里拔出来还带泥的萝卜。

csv：一种纯文本表格格式，用逗号分隔列。类比是去掉所有花样的纯文字账单。

透视：按某一列分组，然后求和、计数、取平均。类比是把一堆小票按商品名归类算总额。

签名（数字签名）：安装包上标明"谁做的这份文件"的电子印章。类比是药盒上的封条。

你的岗位怎么用

设计工程师：今天就把工作目录建在"在研项目 / 2026 / 图纸与计算书"这一层，别建在整个项目根目录。你的第一批活建议从"读一份设计计算书、列出输入参数与假设条件"开始，只读不改。等你能判断它读得对不对，再让它动手改 BOM 和技术条件。

试验工程师：2.4 那件活就是给你写的。台架数据最脏，空行、中断、单位混写全都有。先拿一份上个月你已经手工整理过的记录跑一遍，用你自己的结果当标准答案，一次就能知道它靠不靠谱。

质量工程师：你的红线意识最强，2.5.3 那张三圈表请照抄并补充。另外建议你在记忆初始化里加一条："涉及判定的结论，必须给出依据行号，我要逐条复核。"这条会让你后面所有验收都省一半时间。

采购 / 供应商管理：乱表格是你最常见的东西，各家供应商发来的格式没有两份是一样的。把 2.4.5 的第二轮提示词存成你自己的模板，每次只改方括号。我认识的一位采购同事靠这一条，把五家供应商的月度表从两小时压到了二十分钟。

项目经理：2.7 那个五分钟动作最适合你。你的信息源是纪要、周报、风险表，全是一读就要结论的东西。先让它读，等你信得过它读的结论，再让它写。跳过读直接让它写，出错概率高得多。

研发主管：你自己不必先动手，但 2.5.3 的权限三圈建议你亲自定一版给团队。个人装机各自为战，最可能出的问题不是效率，是有人把共享盘挂进了工作目录。

本章交付物

装机自检清单：一张 A4 纸能打印完的九条，装完当天逐条打勾。我把它贴在显示器边框上贴了半年。

#	检查项	做完的标准
1	安装包来源	官方渠道，比对过字节数
2	版本	稳定版，不是预览版
3	装完能启动	双击能开，不白屏
4	五个区域找到三个	对话区、模式切换、设置区

#	检查项	做完的标准
5	工作目录已设	一个英文数字的专属目录
6	记忆已初始化	2.5.2 那段话发过一次
7	权限三圈已划	红区目录全部没挂上去
8	第一件活跑通	2.3 那份汇报对得上页码行数
9	第一件真活验收	2.4 的三个数字对得上

怎么用：装完当天照着打勾，第 7 条没打勾之前不要交真活给它。前六条是十分钟的事，第七条是十分钟，后两条才是真正花时间的部分。

作者使用经验：第九条我从来不跳。有一次赶时间，跳过了行数核对，直接把整理完的表发给了客户，第二天发现少了一个班组共 143 行。从那以后，那三个数字我每次都写在纸上，对完才发。

本章金句

装完软件只占这件事的十分之一。剩下九成，是让它知道该动谁的文件。

AI 永远只碰副本。原件留在原地，这是死规矩。

先让它体检，再让它动手。理解错了在汇报阶段纠正，比改完 1286 行再返工便宜得多。

它出错的方式通常不是算错，是悄悄少算。所以原始行数必须等于删除行数加输出行数。

凡是它替你推断、补全、四舍五入的内容，都要有一个标记列。

桌面是待整理区，不是工作区。把文件复制到工作目录里再干活，多一步复制省掉一次事故。

第一次拿到正反馈的这一天，比后面任何一天都重要。

（下一章：三种模式的日常切换。装好了，接下来要学会的是什么时候让它想、什么时候让它干。）

第 3 章

什么时候问，什么时候谋，什么时候干

模式选错，AI 干得越快，你返工越狠。

先看我踩过的一個坑。

2025 年 11 月，一个周四下午四点，我接到一个活：把 47 份供应商 8D 报告（每份 8 到 15 页的 PDF）提炼成一张问题清单，周五上午十点要给部门经理过。

那会儿我用 WorkBuddy 才两周，正处在“什么活都直接丢给 Agent”的阶段。我把 47 个 PDF 拖进对话框，说了一句话：

把这 47 份 8D 报告读一遍，提炼成一张问题清单，输出到桌面。

它跑了 11 分钟，交给我一份 47 行的表格。每行一份报告：供应商名、零件号、问题描述、整改措施。格式整齐，写得干净。

然后我发现我要的不是这个。经理要的是“这三年供应商的问题集中在哪几类”，按失效模式横向归类：绕线绝缘、轴承异响、漆层脱落、尺寸超差，每一类下面挂多少个案子、涉及哪几家供应商。它交的是纵向的，我要的是横向的。

我说“不对，我要按失效模式归类”，它又跑了一遍，给了第二版。第二版的问题是我没指定分类口径，它自己定了 9 类，其中“外观问题”和“表面缺陷”是同一件事拆成了两类，“其他”里塞了 14 个案子。

第三遍我才反应过来：这个活我一开始就没想清楚自己要什么，直接让 Agent 动手，等于让它在替我想。

第二天早上我重做了一遍。这次先切 Plan 模式，把同样的话说一遍，它先给我一份方案：分几步、分类口径怎么定、“其他”类超过百分之多少要拆、每一类要给几个数字。我在方案上改了三处，点头，它才动手。从说话到验收完，38 分钟。

第一遍三个版本加返工，3 小时 40 分钟。差了接近六倍。

这个差别跟模型聪不聪明没关系。同一个模型，同一个我，同一批文件。差别只有一件事：我在什么时候让它开始动手。

第 1 章讲过三种模式是什么、选择口诀怎么念。这一章讲日常：怎么切、切了以后上下文还在不在、干到一半怎么中断、出现什么信号说明该换了。最后一节 3.6，我拿一个真实任务把 Ask、Plan、Agent 从头到尾走一遍，每一步的提示词原文都给你。

3.1 三种模式的边界再确认

先把边界摆清楚。第 1 章给的是概念表，这里给操作表：你按下发送之后，它到底会碰什么东西。

	Ask (问)	Plan (谋)	Agent (干)
读你的文件	会读，只读	会读，只读	会读，也会写
改你的文件	绝不	绝不	会
跑程序	不会	不会	会
联网查东西	一般不	一般不	会
交出来的东西	一段回答	一份方案	改好的文件 + 汇报
一次交活耗时	几秒到一分钟	一到三分钟出方案	几分钟到几十分钟
出错代价	答错了，你没损失	方案错了，你改文字	它真改了你的文件
翻车后的恢复成本	零	零	看有没有备份

三种模式的操作边界

按下发送之后，它到底会碰什么东西

看哪项	Ask (问)	Plan (谋)	Agent (干)
读你的文件	会读，只读	会读，只读	会读，也会写
改你的文件	绝不	绝不	会
跑程序	不会	不会	会
联网查东西	一般不	一般不	会
交出来的东西	一段回答	一份方案	改好的文件加汇报
一次交活耗时	几秒到一分钟	一到三分钟出方案	几分钟到几十分钟
出错代价	答错了你没损失	方案错了你改文字	它真改了你的文件
翻车后的恢复成本	零	零	看有没有备份

第 3.1 节：Agent 是唯一会真动你文件的模式

图 3-1 三种模式的操作边界：谁会动你的文件

图 3-1 三种模式的操作边界（作者根据公开资料绘制）

怎么切

【待补：界面路径，以实际版本为准】

不管界面长什么样，三种切法通用：

1. 对话输入框附近的模式选择器，点一下换。这是最直观的。
2. 直接在话里说。比如"先别动手，给我方案""你现在改成只读，别改文件"。它会照做。
3. 开新对话时先定调。第一句就写"这个任务先只问不动手"，后面它就不会自作主张。

我自己的习惯是第三种。每个新任务开口第一句先把模式说死，比中途改要稳。

切换后上下文还在不在

这是最多人问错的问题。分两种情况：

同一个对话窗口里切：上下文在。前面说过的话、它读过的文件、它已经摸出来的结论，全都还在。我有一次 Agent 干到一半，切 Ask 问它"你刚才为什么把第 4.7 节归到试验类"，它答得上来，还把依据的关键词复述了一遍。切回 Agent，它接着往下干，没重头再来。

新开一个对话窗口：短时记忆清空。它不记得你上一个窗口聊了什么。但你的长期记忆还在，比如你的岗位、你们的表格格式习惯、你说过"报告表头照 2025 版"。第 6 章专门讲这层记忆。

由此得出一条规矩：一个任务尽量在一个对话窗口里干完。不要开五个窗口各问一句，那样每个窗口里的它都是个刚入职的新人。

中断了怎么办

Agent 干活的时候，你随时可以叫停。叫停之后有三条后续：

你想要什么	你就这么说
把改动撤回	"停下来，把刚才改的文件恢复原样。"
想知道它干到哪了	"停，汇报一下当前进度和已经改了哪些文件。"
改要求重来	"停，方向不对。改成……，从头再来。"

我最常用的是第二条。看到它开始做一个我没让它做的事，先停、先问进度，比让它一路跑完再收拾要省事。

还有一条笨规矩，但我建议每个人都养：让 Agent 改原始文件之前，先复制一份。复制一份只要三秒，能救你的场合不止一次。

模式不是"设置一次管一天"

很多人以为模式是个开关，早上设成 Plan，今天就一直 Plan。实际用法是动态的：一个大任务里面，你会来回切好几次。

3.5 节专门讲什么时候该切，3.6 节那个完整演示里，我在一个任务里切了四次。

3.2 Ask 模式的正确用法

Ask 是最被低估的模式。新手觉得它"啥也不干"，老手用它省的时间最多。

它真正强的场合有三个。

场合一：查资料、搞懂一个概念

你要搞懂一件事，而不是要一个文件。这类活 Ask 就够了，开 Agent 是浪费。

对话示例一（搞懂概念）：

我在看一份客户的技术需求文档，里面写"电机在 -40 到 85 摄氏度范围内需满足 Class C 功能等级"。我不懂 Class C 是什么。用汽车零部件行业的语境解释一下这件事：1. Class C 在常见的功能等级划分里是什么意思？跟 Class A、B 的区别在哪？2. 如果达不到 Class C，工程上通常的后果是什么？3. 要验证它，一般要做什么试验？别给我建议，先让我搞懂。

它给我的回答里有一句我记到现在：Class A 是坏了要出人命，Class B 是坏了车还能开回家，Class C 是坏了车得拖走。这种一句话讲清三层区别的能力，是它最值钱的地方。

场合二：不动任何文件的安全试探

这是我最推荐的用法，也是工程现场最该养成的习惯。

在你让 Agent 动手之前，先用 Ask 问一句"如果我让你做这件事，你会动哪些文件"。它不会真的动，但会把它的打算说给你听。你听完就知道它理解得对不对。

对话示例二（动手前的试探）：

桌面上有一个文件夹"2026Q1_试验数据"，里面有 214 个 CSV 文件和一个 Excel 汇总表。先不要动任何文件，只回答我：1. 如果我要你把这些 CSV 合并进汇总表，你会按什么键做匹配？文件名还是文件里的某一列？2. 你打算改哪个文件？会新建文件吗？3. 你认为这个活最容易出错的地方在哪？4. 你觉得我还需要告诉你什么，你才能做对？

第四问是我后来加的，价值最高。它会反问你"文件名里的时间戳格式是统一的吗""汇总表里已有的行要不要保留"。这些问题你答完，Agent 阶段的返工基本就没了。

Ask 模式的好用法与坏用法

它是最被低估的模式：新手觉得它啥也不干，老手用它省的时间最多

好用法	坏用法
提问方式 说清背景，列具体问题，限定回答范围 提问数量 一次问 3 到 5 个编号问题 指明文件 给路径、文件名、页数 限定动作 明说只读，不要改任何文件 拿到答案后 追问它没把握的地方 搞懂之后 切 Plan 让它出方案	提问方式 帮我看看这个，附一个文件 提问数量 一句话问一个开放大问题 指明文件 只说那个报告 限定动作 不说，指望它自觉 拿到答案后 直接信，转头就干 搞懂之后 直接切 Agent 让它上手

第 3.2 节：动手前先问一句你会动哪些文件，这一步是最省的一次试探

图 3-2 Ask 模式好用法与坏用法对比

图 3-2 Ask 模式的好用法与坏用法（作者根据公开资料绘制）

场合三：干完活之后的复盘和追问

Agent 交完活，别急着关窗口。切回 Ask 问两句：

- "你刚才为什么选这个分类口径？"
- "这 61 条任务里，哪几条你最没把握？"

第二句特别有用。它自己说没把握的地方，就是你验收时该重点看的地方。

好用法 vs 坏用法

	好用法	坏用法
提问方式	说清背景 + 列具体问题 + 限定回答范围	"帮我看看这个"，附一个文件
提问数量	一次问 3 到 5 个编号问题	一句话问一个开放大问题
是否指明文件	给出路径、文件名、页数	"那个报告"
是否限定动作	明说"只读，不要改任何文件"	不说，指望它自觉
拿到答案后	追问它没把握的地方	直接信，转头就干

	好用法	坏用法
搞懂之后	切 Plan 让它出方案	直接切 Agent 让它上手

最后一行的差别就是这一章开头的那个下午。

3.3 Plan 模式：怎么读懂它给的方案

Plan 模式下它交出来的是一份文字方案。你得会读，不然"看了方案点点头"就变成了走过场。

先看一份真实的方案长什么样。这是 3.6 节那个任务里，它给我的第一版方案（我删掉了两处重复描述，其余原文）：

【拆分方案 v1】

目标产物：SOR_任务清单_某欧系平台_2026.xlsx，含 1 张总表 + 7 张职能分表

第一步 建立条款索引

输入：SOR 全文

动作：把全部 must/shall 条款编号 T001 起顺排，保留章节号与英文原文

产出：内存中的条款表（此步不落盘）

第二步 职能分配

分配口径：按条款关键词自动归类，一条款只归一个主责职能

重叠条款：允许第二职能，标"协办"

无法归类的：放进"待定"表，不猜

产出：条款表新增三列（主责职能、协办职能、归类依据关键词）

第三步 条款转任务

规则：一条 must/shall = 一条内部任务

合并规则：同一职能、同一章节、语义相邻的条款可合并，合并后保留全部条款号

产出：任务清单，字段如下

任务编号

来源条款号（如 4.2.3）

条款原文（英文）

中文解读（一句话）

主责职能

协办职能

任务类型：设计 / 校核 / 试验 / 采购 / 文档

交付物

截止时间（用相对时间，如"SOR 回复前 7 天"）

风险等级：高 / 中 / 低

状态：待启动

第四步 风险标注

依据：与本部门已量产平台的能力对比

判为"高"的三种情况：超出已量产范围、需要新增试验设备、需要供应商重新报价

产出：高风险任务列表，附在总表最前

第五步 输出文件

用 openpyxl 写 xlsx，中文表头，冻结首行，按主责职能分表

输出到桌面

本方案最容易出错的三处：

1. 条款归类靠关键词，可能误判。规避：保留"归类依据关键词"列，方便你人工改
2. 风险等级需要你们现有平台的能力数据，我没有。规避：默认标"待评估"，不编造
3. 截止时间我不知道你们的排期。规避：用相对时间表达，不写具体日期

需要你确认的三件事：

1. 全部条款都转任务，还是只转 must？
2. 风险等级要我评估吗？要的话请给我现有平台的能力边界数据

3. 截止时间用相对时间表达可以吗？

预计耗时：约 15 分钟

看方案只看三处

一份几百字的方案，看这三处两分钟就过完了

第一处 起点

第一步是不是你以为的第一步。起点错了，后面全白搭

第二处 假设点

中间有没有它替你做决定的地方，每一处都是在替你定规则

第三处 终点

最后交出来的东西是不是你要的形态，这一步最容易被忽略

第 3.3 节：改方案一次说完，不要一次改一条

图 3-3 看方案只看三处：起点、假设点、终点

图 3-3 看方案只看三处（作者根据公开资料绘制）

看方案只看三处

一份方案可能有几百字，你不用逐字读。看这三个地方就够。

第一处：第一步是不是你以为的第一步。起点错了，后面全白搭。上面这份方案的起点是“建立条款索引”，跟我想的一致。如果它第一步写的是“按职能拆成 7 个表”，那说明它把顺序搞反了，我要按职能发给别人，但编号得先统一。

第二处：中间有没有它替你做决定的地方。这类地方在方案里通常长这样：“分配口径：按条款关键词自动归类”“判为高的三种情况”“合并规则：语义相邻的条款可合并”。每一处都是它在替你定规则。你要问自己：这条规则我认吗？

上面这份里，我最不认的是“风险等级”那一处。它没有我们现有平台的能力数据，判出来的高、中、低是猜的。

第三处：最后一步交出来的东西是不是你要的形态。这一步最容易被忽略，也最容易返工。它说交“1 张总表 + 7 张职能分表”，我真正要的是 8 张表，多一张歧义条款表给客户电话会用。这个我在改方案时补上了。

看这三处，一份几百字的方案两分钟就过完了。

三条改方案的话术

改方案不用重写，说三句话就够。这三条是我用得最多的，直接抄。

话术一，改局部，其余不动：

第三步的任务编号规则改一下：改成"职能缩写-序号"，比如 NVH-001、ELE-002。因为我要按职能把表拆开分别发给七个人，编号带职能更好认。其余不变。

关键是"其余不变"四个字。你不说，它可能把别的也改了。

话术二，推翻它的一个默认：

第四步的风险等级不要你评估，全部填"待评估"。理由：你没有我们现有平台的能力数据，评估出来也是猜。另外把"待评估"单独做一列，方便我筛选后人工过一遍。

给它理由，它下次就不会再犯同一个错。这条尤其适用于它编数据的场合。

话术三，在末尾加一步：

在第五步之后再加一步：额外输出一张"歧义条款表"。字段：条款号、条款原文、歧义点、建议向客户提问的措辞（中英对照）。这张表我要直接用在下周的客户电话会上。

加一步比改中间步骤安全，不会打乱它已有的逻辑。

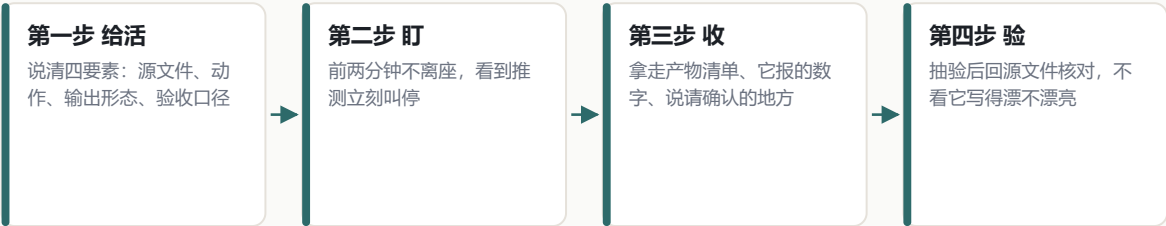
改方案还有个原则：一次说完，不要一次改一条。你把三条意见一口气给它，它一次改完；你一条一条给，它改三遍，每遍都要重读上下文。

3.4 Agent 模式：交活的完整动作链

Agent 是唯一会真动你文件的模式。它有一条完整的动作链：给活、盯、收、验。少一步都可能出事。

Agent 交活四步动作链

给活、盯、收、验，少一步都可能出事



第 3.4 节：四要素里最常被漏掉的是验收口径

图 3-4 Agent 交活四步动作链

图 3-4 Agent 交活四步动作链（作者根据公开资料绘制）

第一步：给活

一句话说清四要素：源文件、动作、输出形态、验收口径。缺一个，它就得猜。

源文件：桌面 / 2026Q1_试验数据 / 214 个 CSV，以及汇总表 summary.xlsx

动作：把 214 个 CSV 的数据合并进 summary.xlsx，按"样件编号 + 日期"匹配，已存在的行保留不动

输出形态：直接改 summary.xlsx，不要新建文件，改之前把原文件复制一份带 .bak

验收口径：合并后总行数、新增行数、匹配失败的行数，三个数字报给我

四要素里最常被漏掉的是验收口径。你不要它报数字，它就只说一句"已完成"，你还得自己打开文件数。

第二步：盯

交完活别离开。盯三个时间点的动静：

时间点	看什么	不对劲的表现
前 30 秒	它有没有先去找文件	直接开写，说明它没读源文件
第一次汇报	它理解的口径跟你一不一致	它复述的匹配键跟你说的不一样
中途汇报	有没有出现"我推测""应该是"	出现就是它在猜，立刻叫停

我自己定了一条规矩：Agent 干活期间，我在前两分钟内不离开座位。大部分翻车都发生在开头，开头看住了，后面基本稳。

第三步：收

它干完会汇报。收的时候拿走三样东西：

- 1. 产物清单：它新建或改了哪些文件，路径是什么。
- 2. 它说的数字：总行数、处理了多少、跳过多少。
- 3. 它自己承认没把握的地方，一般在汇报最后一段，常常以"请确认"开头。

第三样最值钱。它主动说"第 47 行的数据格式和其他行不一致，我按 xx 处理，请确认"，这一句能替你省掉半小时的抽查。

第四步：验

这步是你的，谁也替不了。抽验比例我的习惯是：

- 条目少于 50 条，全查；
- 50 到 200 条，抽 10 条，其中至少 3 条是它说没把握的；
- 超过 200 条，抽 5%，另外全查它标"待定"的部分。

抽查的动作是回源文件核对，不是看它写得漂不漂亮。第 18 章会给一套完整的验收动作清单。

四步上的常见错误

步骤	常见错误	后果	补救
给活	没说验收口径	它只说"已完成"	补一句"报三个数字给我"
给活	让它直接改原始文件	改坏了没退路	要求先备份 .bak
盯	交完活就去开会	它跑偏 20 分钟你不知道	前两分钟留在座位
收	只看它给的文件，不看汇报	漏掉它说的"请确认"	汇报最后一段必读
验	只看格式对不对	数据错了你签字	回源文件抽验

3.5 模式切换的四个信号

模式不是设一次管一天的开关。出现下面四个信号，说明该换了。

模式切换的四个信号

模式不是设一次管一天的开关，出现这四个信号就要换

信号一 它开始瞎猜

回答里出现通常、一般来说、可能是、我推测，或者开始编文件路径、标准号、数字。切 Ask，让它交代信息来源，交代不出来的一律删掉。

信号二 它问得越来越多

反问超过三轮，而且问题越来越细，全是格式细节。切 Ask，让它先把结构讲给你听，往往发现是基准还没定。

信号三 你已经改了三版

三个版本都不满意，每次改的是同一类问题。立刻停手，切 Plan 从头出方案，不要在第三版上继续打补丁。

信号四 做了一半方向不对

已经干到第三步才发现起点就错了。叫停，切 Ask 汇报进度和已改文件，再判断能不能在现有基础上改。

第 3.5 节：每个新任务开口第一句先把模式说死，比中途改要稳

图 3-5 模式切换的四个信号

图 3-5 模式切换的四个信号（作者根据公开资料绘制）

信号一：它开始瞎猜

表现：它的回答里出现“通常”“一般来说”“可能是”“我推测”，或者它开始编一个文件路径、一个标准号、一个你没给过的数字。

真实场景：2026 年 1 月，我让它整理一份材料的认证清单。它在 Plan 阶段写“该材料一般需要通过 xxx 认证”，那个认证编号我查了，不存在。我立刻切 Ask，问它“这个认证编号你是从哪份文件里看到的？”它承认是自己推的。

该怎么办：从 Plan 或 Agent 切到 Ask，让它交代信息来源。交代不出来的一律删掉。这是第 17 章“AI 的七宗罪”里的头一条。

信号二：它问的问题越来越多

表现：它反问你的次数超过三轮，而且问题越来越细，比如“这个文件里的日期格式是年月日还是月日年”“表头算不算数据行”。

真实场景：同一个月，我让它把三份不同年份的试验大纲合并成一份。它问到第四个小时时我停了。它问的全是格式细节，说明它没搞懂这三份大纲的结构差异在哪。

该怎么办：切 Ask，让它先把结构讲给你听。你听完往往发现，问题不在格式，在于你自己还没决定“以哪一份为基准”。这个决定只能你做。

信号三：你已经改了三版

表现：同一个活，它交给你三个版本，你都不满意，而且每次改的都是同一类问题。

真实场景：本章开头那个 47 份 8D 报告的下午，就是标准案例。三版都不对，问题不在它，在我没说清分类口径。

该怎么办：立刻停手，切 Plan，让它从头出一份方案。注意是从头出方案，不是在第三版上继续改。在错的版本上打补丁，越打越乱。

信号四：任务做了一半发现方向不对

表现：它已经干到第三步、第四步了，你突然发现第一步的产物就不是你要的。

真实场景：2026 年 3 月，我让它把一份客户审核的整改项做成跟踪表。它干到第四步开始填责任人，我才发现它按"审核发现项"建的表，我要的是按"内部整改动作"建表。一个发现项可能对应三个整改动作。它已经填了 40 行。

该怎么办：叫停，切 Ask，让它汇报当前进度和已改的文件。然后判断：能不能在现有基础上改？能给新指令继续；不能就让它回滚，重来。

那次我判断能改。我让它加一列"整改动作序号"，一个发现项拆成多行。它改了 6 分钟，比推倒重来快。

四个信号的对照表

信号	说明什么	切到哪
它开始瞎猜	信息不够，它在编	切 Ask，追来源
它问得越来越多	结构没摸清或你没定基准	切 Ask，让它先讲结构
你已经改了三版	你没说清需求	切 Plan，从头出方案
做了一半方向不对	起点就错了	先停，切 Ask 问进度，再决定改还是重来

3.6 一个真实任务走完三种模式 ★★★

这一节是本章的重点。我拿一个工程现场最常见也最烦的活，把三种模式从头到尾走一遍。每一步的提示词原文都在，你可以照抄。

任务：客户发来一份 68 页的技术需求文档（SOR），要求 10 个工作日内回复技术可行性。我要把它拆成一份内部任务清单，分给 7 个职能去认领。

为什么这个活难：文档是英文的，68 页里散落着 100 多条带 must、shall 的强制条款，有些条款之间有依赖，有些条款写得含糊。人工拆，我 2024 年干过一次同类活，两个人两天。

第一步：Ask 摸底（6 分钟）

我先不拆，先用 Ask 把这份文档摸清楚。摸底要拿到三个东西：文档结构、强制条款全集、歧义条款。

提示词原文：

这是一份客户技术需求文档（SOR），文件在桌面 / SOR_某欧系平台_2026.pdf，共 68 页。
先不要做任何修改，也不要生成任何文件，只回答我下面四个问题：

1. 这份文档一共有哪几个章节？列出章节号和标题。
2. 全文一共出现多少次 must、多少次 shall、多少次 should？分别统计数字。
3. 把所有含 must 或 shall 的条款挑出来，按章节号顺序列成一张表，包含：
条款号、条款英文原文（不要翻译、不要改写）、所属类别。
类别限这九种：性能、环境、寿命、电气、EMC、尺寸接口、文档交付、试验验证、其他。
4. 挑出你认为存在歧义的条款（定义不清、缺少边界条件、前后不一致），
说明歧义具体在哪，以及它会影响哪些后续判断。

要求：

- 第 3 问的表格不许省略任何条款，数量以你实际统计为准。
- 如果你对某条款的类别没把握，类别栏填“待定”，不要猜。
- 回答完先别给建议，我会接着问。

它给我的结果：

- 章节：11 个一级章节，从 1.0 范围到 11.0 交付物。
- 计数：must 出现 96 次，shall 出现 74 次，should 出现 38 次。扣除重复和说明性文字后，实际的强制条款是 127 条。
- 条款表：127 行，类别分布我记了下来，试验验证 31 条、性能 24 条、环境 19 条。
- 歧义条款：8 条。其中第 3 条我印象很深，原文大意是噪声应在背景噪声不高于 20 dB 的条件下测量，而我们台架的本底噪声是 28 dB。

Ask 阶段的第二问，只针对那条要命的歧义：

第 3 条歧义“噪声测量条件”这条，我们台架的本底噪声是 28 dB，达不到 20 dB。

帮我做两件事：

1. 在这份 SOR 全文里找一找，有没有别的地方对噪声测试条件做过补充说明？把原文抄给我，注明章节号。
2. 如果没有，把同样依赖“测试条件”这一前提的其他条款列出来，说明它们各自的测试条件要求。

它找了一遍，回答：全文没有其他补充说明；另有 6 条条款依赖测试条件，其中 2 条要求的也是 20 dB。

这一步的价值在于：我知道了这不是一条孤立的坑，是一个会影响 3 条条款的坑，得在客户电话会上专门问。

我承认，做这一步的时候我心里是急的。68 页的东西摆在那儿，十个工作日的期限挂在那儿，脑子里的声音一直在说“别问了，赶紧动手”。我强按着自己问完，是因为 2024 年那次人工拆分留下的教训：那次漏掉的 3 条条款，全都是因为没人先把条款点清楚。

这也是 Ask 摸底最容易被跳过的原因。它不产出任何文件，你看着屏幕上刷刷滚过去的回答，会觉得自己在浪费时间。实际上这 6 分钟买的，是后面每一步的地基。

还有一条规矩我在这一步定了：**must** 和 **shall** 转任务，**should** 单列参考，不转。这不是它替我定的，是我定的。SOR 里 **should** 通常有 38 条，如果把 **should** 也转成必办项，任务清单会虚胖，7 个职能负责人拿到手会先骂人。这个判断只能人来做。

Ask 阶段结束，我手上有了三样东西：文档结构、127 条强制条款的清单、8 条歧义条款。全程 6 分钟，没动过任何文件。

第二步：Plan 出方案（12 分钟，含我改两轮）

切 Plan 模式。3.3 节那份方案就是这一轮出来的。提示词原文：

基于刚才 Ask 阶段的摸底结果（127 条 **must/shall** 条款、11 个章节、8 条歧义条款），我要把这份 68 页的 SOR 拆成一份内部任务清单，分给 7 个职能去认领：电机设计、NVH、热管理、电子控制、试验验证、采购、质量。

请给我一份完整的拆分方案，必须包含下面五项：

1. 分几步做，每一步做什么、产出什么。
2. 按什么口径把 127 条条款分配到 7 个职能，一条款涉及多个职能时怎么处理，归不了类的怎么办。
3. 每一条内部任务要包含哪些字段，逐项列出。
4. 最终输出文件的格式（我要一份能直接发给 7 个职能负责人的 Excel）。
5. 你认为这个方案最容易出错的三个地方，以及你的规避办法。

要求：

- 只给方案，不要执行，不要生成任何文件。
- 方案里凡是你需要我来定的事，单列一节“需要我确认的事”。
- 预计耗时也告诉我。

它给的方案就是 3.3 节那份。我按“看三处”的方法过了两分钟，改了三条。

改方案的话术原文（三条一次性给）：

方案我看了，改三条，其余不变：

1. 第三步的任务编号规则改一下：改成“职能缩写-序号”，比如 NVH-001、ELE-002。
因为我要按职能把表拆开分别发给七个人，编号带职能更好认。
2. 第四步的风险等级不要你评估，全部填“待评估”。
理由：你没有我们现有平台的能力数据，评估出来也是猜。
另外把“待评估”单独做一列，方便我筛选后人工过一遍。
3. 在第五步之后再加一步：额外输出一张“歧义条款表”。
字段：条款号、条款原文、歧义点、建议向客户提问的措辞（中英对照）。
这张表我要直接用在下周的客户电话会上。

另外回答你“需要我确认的三件事”：

1. 127 条全部转任务，should 不转，但需要单独列一张 should 清单供参考。
2. 风险等级按我上面第 2 条处理，我不给你能力数据。
3. 截止时间用相对时间表达可以，但请统一按“SOR 回复前 N 天”的格式。

改完把新版方案给我，我还是先看，看完再让你动手。

它给了 v2 方案。v2 相对 v1 只动了三处，其余整段保留：

【拆分方案 v2】相对 v1 的改动

改动 1（第三步）

任务编号规则改为：职能缩写-序号

示例：MOT-001（电机设计）、NVH-001、THM-001（热管理）、ELE-001（电子控制）、
TST-001（试验验证）、PUR-001（采购）、QUA-001（质量）

改动 2（第四步）

删除“风险等级判定”的全部逻辑。

风险等级列统一填“待评估”，不再区分高中低。

新增“待评估”筛选列（是 / 否），供人工过筛。

改动 3（新增第六步）

第六步 输出歧义条款表

数据来源：Ask 阶段挑出的 8 条歧义条款

字段：条款号、条款原文（英文）、歧义点、建议向客户提问的措辞（中英对照）

用途：客户电话会

第七步 输出 should 参考清单

字段：条款号、条款原文（英文）

不转任务、不编号、不分配职能

其余（第一步、第二步、第三步合并规则、第五步输出格式）与 v1 一致，未改动。

预计耗时：约 18 分钟（比 v1 多 3 分钟，因为多了两张表）

v2 我只看了两个地方：一是它有没有把我没让它改的地方也改了，看下来没有，这点很重要；二是新增的第六步、第七步插在哪儿，看下来是排在第五步之后，没有打乱前面的顺序。看完我就点头了。

一个提醒：改方案会让它重新算耗时。v1 说 15 分钟，v2 说 18 分钟，后面 Agent 实际跑了 22 分钟。这三个数字一个比一个大，是正常的，别拿第一个数字去催它。

Plan 阶段 12 分钟，其中它出方案 4 分钟，我读加改 6 分钟，它改方案 2 分钟。

第三步：Agent 执行 (22 分钟)

切 Agent 模式。这一步的关键是把 Plan 定下来的东西复述一遍，别让它自己发挥。

提示词原文：

按你给的拆分方案 v2 执行。源文件：桌面 / SOR_某欧系平台_2026.pdf。

输出文件：桌面 / SOR_任务清单_某欧系平台_2026.xlsx。

执行要求：

1. 任务编号用"职能缩写-序号"，如 NVH-001、ELE-002。
2. 风险等级全部填"待评估"，不要评估，不要编造。
3. 除总表和 7 张职能分表外，额外输出一张"歧义条款表"，
字段：条款号、条款原文、歧义点、建议向客户提问的措辞（中英对照）。
4. 额外输出一张"should 参考清单"，不转任务，只列条款号和原文。
5. 归类不确定的条款放进"待定"表，标明不确定的原因，不要猜。
6. 改文件前把原位置同名文件备份为 .bak（如存在）。

过程要求：

- 每完成一步用一句话汇报，不要长篇。
- 遇到你判断不了的问题，停下来问我，不要自己决定。

完成后给我四个数字：

- 处理的条款总数
- 生成的任务总数
- 待定条款数
- 合并掉的条款数

盯的过程：它跑了 22 分钟，汇报了 5 次。前 30 秒它先去读了 PDF，这是对的。第 3 次汇报有一句我特意停下来看了：

第 4.7 节的 3 条条款引用了试验标准号，但文档里没有给出正文，我放进待定表，原因：无法判断试验条件。

这个处理是对的，我没干预。

收的产物：一个 xlsx，10 张表（总表、7 张职能分表、歧义条款表、待定表）。另外它在汇报最后写了两句"请确认"：一是第 4.7 节那 3 条，二是总表里有 4 条条款它认为同时属于性能和试验验证，它按主责归了性能、协办标了试验验证。

它还给了四个数字：条款总数 127、任务总数 61、待定条款 6、合并条款 66。

第一个数字跟我 Ask 阶段统计的 127 对上了，这是我最先看的一个数。

第四步：验收（15 分钟）

检查项	动作	结果
条款总数对不对	它报的 127 和 Ask 阶段的 127 对比	一致
条款号有没有抄错	随机抽 8 条任务，回 PDF 翻到对应章节核对条款号	7 条对，1 条错（它写 4.7.2，原文是 4.7.3）
待定表有没有漏判	逐条看 6 条待定，人工判断能不能归	6 条里 2 条能归到 NVH，我手动移过去
歧义表能不能用	看 8 条歧义的"建议提问措辞"是否中英对照、是否可直接发	7 条可用，1 条措辞太生硬，我自己改了
职能分表能不能直接发	打开 7 张分表，看表头、看有没有空行	都能发，一张表里有一行协办栏为空，我补了"无"

15 分钟，查出 1 个条款号错误、2 条待定误判、1 处措辞问题。

全程时间：Ask 6 分钟 + Plan 12 分钟 + Agent 22 分钟 + 验收 15 分钟 = 55 分钟。

对照我 2024 年那次人工拆分：两个人两天，而且那次的条款清单最后还漏了 3 条，是客户在电话会上提醒的。这次 127 条一条没漏。

对照：如果一上来就开 Agent

同一个任务，我第二天又拿另一份同类 SOR 试了一遍"直接开干"的做法，把过程记了下来，好让你看清差别在哪。

提示词只有一句："把这份 SOR 拆成内部任务清单，输出到桌面。"

它跑了 9 分钟，交给我一份 52 行的清单。问题有四个：

- 它把 should 也转成了任务，清单里混进了 21 条不必办的项；
- 它没有条款号，我无法回原文核对；
- 它按自己的理解定了 6 个职能，跟我们实际的 7 个对不上；
- 风险等级它照填了高、中、低，我问它依据是什么，它说"根据行业常见情况判断"。

我在这份 52 行的清单上改了 40 分钟，还是不能用，最后推倒重来。前前后后 49 分钟，产物作废。

差别不在它干得快不快，直接开干那次它只用了 9 分钟。差别在于：没有 Ask 阶段的摸底，它不知道有 127 条条款这个数；没有 Plan 阶段的方案，它的分类口径、字段、职能划分全是自己定的。你没定的事，它就替你定，而定错了你只能在成品上改。

这一节的三条经验

一，Ask 摸底的数字，是后面验收的基准。Ask 阶段统计出 127 条，Agent 阶段它报 127，两个数字对上，我心里就有底了。如果你跳过了 Ask，你连"该有多少条"都不知道，验收就只能靠感觉。

二，Plan 阶段改方案，比 Agent 阶段改文件便宜十倍。我在方案上改三条花了 6 分钟。如果这三条在 Agent 干完之后才发现，它得重跑，我得重新验收。

三，凡是它说“请确认”的地方，都要看。它汇报里的两处“请确认”，正好对应我验收时查出的两个问题。它其实知道自己在哪儿没把握，你只要肯看最后一段。

3.7 如果你只做一件事

一个动作就够了：每个新任务的第一句话，都以“先给我方案，我点头了你再动手”结尾。

就这一句。它把 Ask 和 Plan 之间的门槛降到了零，也把 Agent 的启动权拿回到你手上。

我那个 3 小时 40 分钟的周四下午，和 38 分钟的周五早上，差的不是能力，是这一句话。

小白词典

模式：WorkBuddy 干活的三种委托方式。Ask 是问，只读不动文件；Plan 是谋，先出方案等你点头；Agent 是干，直接读写文件把活做完。

上下文：一段对话里聊过的全部内容，包括你说的话、它读过的文件、它已经得出的结论。同一个对话窗口里切换模式，上下文保留；新开窗口会清空。

SOR：Statement of Requirements，客户发给供应商的技术需求文档。写清了产品要满足什么、怎么验证、什么时候交。汽车零部件行业里，一份 SOR 几十页到几百页都有。

must / shall / should：SOR 里的强制程度标记。must 和 shall 是必须满足，should 是建议满足。拆任务时只把 must 和 shall 转成必办项，should 单独留作参考。

条款：SOR 里带编号的一条要求，比如 4.2.3。验收时回原文核对，靠的就是这个编号。

歧义条款：写得含糊、缺少边界条件、或者前后不一致的条款。它不会自己消失，必须在回复客户前问清楚。

待定表：归类不确定的条目单独放的一张表。它的作用是防止 AI 硬猜，把判断权留给你。

8D 报告：供应商质量问题整改报告的通用格式，八个步骤，从问题描述到永久对策到预防再发。每份 8 到 15 页是常见长度。

失效模式：零件或系统失效的具体形式，比如绕线绝缘击穿、轴承异响、漆层脱落。按失效模式归类问题，能看出毛病集中在哪儿。

你的岗位怎么用

设计工程师：拿到客户 SOR 或内部设计任务书，先用 Ask 让它列出全部强制条款和歧义条款，再切 Plan 出拆分方案。你验收的重点是条款号有没有抄错，抽 8 条回原文核对，两分钟的事。

质量工程师：整理供应商 8D 报告或审核整改项这一类活，最容易犯的错就是本章开头那个：没定分类口径就让 Agent 干。先切 Plan 把分类口径定死，再动手。分类口径写在方案里，比写在脑子里可靠。

采购 / 供应商管理：比价、资质核查这类活涉及联网和外部数据，交给 Agent 之前先用 Ask 问一句“你会动哪些文件、会不会联网”。确认无误再放行，避免它把你的询价底稿改了。

试验工程师：试验大纲合并、试验数据汇总这类活，先让 Ask 讲清三份文件的结构差异，你自己决定以哪一份为基准，再进 Plan。基准这件事 AI 替你定不了。

项目经理：你的活大多是"把一堆输入变成一张能发出去的表"。这种活的标准路线就是本章 3.6 那条：Ask 摸底拿数字，Plan 定字段和口径，Agent 出文件，最后你按 Ask 阶段的数字验收。

研发主管：把"先出方案再动手"写成团队规矩，写进你们的任务模板里。新人最容易犯的错就是直接开 Agent，你要拦的是这一下。

本章交付物

模式决策卡：一张贴在显示器边上就能用的判断流程。

第一步，问自己一句：这个活我说明白了吗？

- 说不明白，或者还没想清楚自己要什么 → 切 Plan，让它先出方案。
- 我只是想搞懂一件事，不要任何文件 → 用 Ask。
- 说明白了，而且活儿在文件里 → 继续第二步。

第二步，问自己一句：它动到文件，我能接受吗？

- 不能接受，或者还没备份 → 停在 Plan 或 Ask，先备份。
- 能接受 → 进 Agent。

第三步，进 Agent 之后，四步动作链照做：

1. 给活说清四要素：源文件、动作、输出形态、验收口径。
2. 盯：前两分钟不离开座位，看到"我推测""应该是"立刻叫停。
3. 收：拿走产物清单、它报的数字、它说"请确认"的地方。
4. 验：抽验（少于 50 条全查，50 到 200 条抽 10 条，超过 200 条抽 5%），回源文件核对。

第四步，出现这四个信号立刻换模式：

信号	动作
它开始瞎猜	切 Ask，追信息来源
它问得越来越多	切 Ask，让它先讲结构
你已经改了三版	切 Plan，从头出方案
做了一半方向不对	先叫停，切 Ask 问进度，再决定改还是重来

怎么做：把上面这四步抄在一张便签上，贴在显示器边框。用两周，它会变成本能。

本章金句

模式选错，AI 干得越快，你返工越狠。

你没想清楚自己要什么，就别让它动手，那是让它在替你想。

看方案只看三处：起点对不对、它替你定的规则你认不认、终点是不是你要的形态。

在方案上改一条花两分钟，在成品上改一条花二十分钟。

它主动说"请确认"的地方，就是你验收时该重点看的地方。

同一个活，先谋后干是 55 分钟，边想边干是 3 小时 40 分钟。

（下一章：技能。装上一个技能，它就多会一类活，下一章讲怎么挑、怎么装、怎么自己攒一个。）

PART THREE • 第三篇

核心机制深度理解

第 4 章

把你用过的好招数存成技能

同一件事你教它第二遍，就是你自己的问题。

2026 年 3 月的一个周四下午，我在赶月度试验数据包。每个月 5 号之前，我要把上个月所有台架试验的数据汇总成一份 Excel，发给客户的项目接口人。

这份表的格式是三年前客户定的：第一页汇总，第二页原始曲线，表头必须是中英双语，超差的行整行标浅红底，页脚要带试验日期和操作人姓名。

那天我把导出的原始数据拖进 WorkBuddy，开始打字：

我：把这些 MST 台架数据按月度模板整理一下。第一页汇总，第二页原始曲线，表头中英双语，超差的行整行标浅红底，页脚写试验日期和操作人姓名。另外客户要求耐久测试的数据放在振动测试前面。

它干完了。我打开一看，序列又错了：振动在前，耐久在后。

我改口：

我：顺序不对，耐久在前。

它改好了。我又发现日期格式写成了 2026/3/15，客户要的是 2026-03-15。再改一轮。然后发现超差判定用的是我上个月临时定的值，不是标准里的值。再改一轮。前前后后八轮，四十分钟。

让我难受的不是这四十分钟。是我回头翻了一下自己的对话记录：同样这套格式要求，我从 2025 年 11 月开始，每个月都要跟它讲一遍。

到 3 月为止，讲了第五遍。中间有一次它甚至问我“这次要不要按上次的方式来”，我回了一句“对，按上次的”，就以为它记住了。它没有。换了个对话窗口，它就又变成那个什么都不懂的新同事。

那天晚上我做了一件事：把这套格式要求写成了一个文件，存进技能目录。从那以后，我每个月只说一句话：“跑月度数据包。”到现在第七个月，一个字都没再改过。

这一章讲的就是这件事：怎么把你说过第五遍的话，变成一份只写一次的岗位手册。

4.1 技能是什么：给 AI 的岗位手册

技能是一份存在你电脑上的说明文件，它告诉 WorkBuddy 遇到某一类活该怎么干、按什么顺序干、交出什么东西算合格。装上它，WorkBuddy 就多会一类活。

我给你三个类比，你挑一个顺手的记住就行。

类比一：岗位手册。工厂里每台设备旁边都挂着一本作业指导书。谁来操作都照着上面写的一步一步做，不会因为今天是新手还是老师傅就换一套做法。技能就是给 AI 挂的作业指导书：先做什么、后做什么、哪些情况不许往下走、做完交什么。

类比二：菜谱。你会做红烧肉，朋友不会。你把火候、顺序、调料克数写成一张卡片，他照做也能端出一锅差不多的肉。技能就是这张卡片，区别是写给 AI 看的。

类比三：检查单。飞行员起飞前有一份检查单，一条一条勾，不看心情。技能里那些"必须先……""如果……就停""最后核对……"，就是这份检查单。这也是我最看重的部分，因为它直接决定交出来的东西你敢不敢签字。

三个类比合起来，技能的本质就清楚了：它不是让 AI 变聪明，是把你已经摸清楚的干法固定下来，让它每次都照着走。

没有技能和有技能，差别在哪？我把同一份试验数据丢给 WorkBuddy 两次，一次不带技能一次带，结果如下：

比较项	没有技能	有技能
我要说的话	每次重讲一遍格式、顺序、判定值	"跑月度数据包"
往返轮数	5 到 8 轮纠正	通常 1 轮
出错类型	顺序错、日期格式错、判定值用错	数据本身有问题（这个本来就要人看）
换窗口之后	全部重讲	照旧执行
交给同事	得把要求再复述一遍给同事	把文件发给他放进目录就行
我花的时间	单次约 40 分钟	单次约 6 分钟

最后一行最实在。同一件活，从 40 分钟降到 6 分钟，一年十二次，就是七个钟头。写那个技能文件，我花了 25 分钟。

有一点要提前说清楚，免得你期望错位：技能不提升 AI 的智商，它提升的是 AI 的稳定性。一件它压根不会干的活，你写一百页技能它也干不了。

一件它本来能干、但每次干得都不太一样的活，技能能把它摁住。所以判断要不要写技能的标准很简单：这件事它做过并且做对过，只是每次要你重新交代，就值得写成技能。

同一件活，没有技能和有技能

技能不提升它的智商，提升的是它的稳定性

没有技能	有技能
我要说的话 每次重讲一遍格式、顺序、判定值	我要说的话 跑月度数据包
往返轮数 5 到 8 轮纠正	往返轮数 通常 1 轮
出错类型 顺序错、日期格式错、判定值用错	出错类型 数据本身有问题，这个本来就要人看
换窗口之后 全部重讲	换窗口之后 照旧执行
交给同事 得把要求再复述一遍给同事	交给同事 把文件发给他放进目录就行
我花的时间 单次约 40 分钟	我花的时间 单次约 6 分钟

第 4.1 节：从 40 分钟降到 6 分钟，一年十二次就是七个钟头

图 4-1 同一件活，没有技能和有技能的区别

图 4-1 同一件活，说过的话和花的时间差在哪。作者根据公开资料绘制。

4.2 技能的三种来源

技能有三个来路：自己写、从市场装、团队共享。三条路不互斥，大多数人的技能夹里最后都是三样都有。

自己写：专治你自己的烦

自己写是最值钱的一条路，因为市面上没有任何技能知道你们部门的表长什么样、客户的页脚要写什么。

适用场景：

- 你发现自己跟 AI 说过两遍以上的同一套要求；
- 这件事有固定的格式、顺序、判定值；
- 做错了要返工，返工成本你能感知到（时间、面子、客户投诉）。

存放位置在个人技能目录下，一个技能一个文件夹，核心文件叫 `SKILL.md`：


```
~/workbuddy/skills/
└─ 你的技能名/
    └─ SKILL.md          必填, 技能的正文
    └─ scripts/          可选, 要跑的脚本
    └─ references/       可选, 长篇参考资料
    └─ assets/           可选, 模板文件、示例数据
```

Windows 上 `~` 就是你的用户目录, 比如 `C:/Users/你的用户名/`。照这个路径一层层点进去, 没有的文件夹自己建。

写的方式有两种。一种是照着 4.3 节的结构自己敲; 另一种是干完一件活之后对 WorkBuddy 说一句:

我: 把我们刚才整理这份数据表的过程, 写成一个技能, 存到我的技能目录, 名字叫 test-data-digest。

它会自己总结这个流程并生成文件。写完必须你自己读一遍, 特别是它总结的判定值和边界条件, 这部分最容易写错。

从市场装: 别人已经趟过的路

社区市场里有别人做好的技能, 点一下就装。这条路的价值不在于省你多少写字时间, 在于有人替你踩过坑了。

适用场景:

- 这件事跟公司、客户的特殊要求无关, 是个通用活;
- 你没干过, 不知道该分几步;
- 你已经感觉到这里头有坑, 但还不知道坑在哪。

我自己装过的这类技能里, 有几个用得很频繁: `markdown-skill` 把 PDF、Word、PPT、图片批量转成 Markdown, 我们每周收到的客户规范 PDF 全靠它; `md-to-pdf` 把写好的 Markdown 转成排版干净的 PDF。

还有一个 `summarize`, 一句话总结网页和长文档。这三个的共同点是: 通用、边界清晰、坑被前人踩平了。

操作路径: 在对话里直接说"帮我找一个能做 XXX 的技能", 或者从推荐市场界面里挑。

【待补: 技能市场的具体入口路径, 以实际版本为准】

装之前做两件事。一是看清它能干什么、不能干什么, 通常写在描述里; 二是不要立刻拿真数据跑, 先拿一份不重要的样本跑一遍。

我装过一个合并 PDF 的技能, 第一次用就把我一份 80 页的原件覆盖了。后来我养成的习惯是: 新技能第一次跑, 一律先复制一份副本出来试。

团队共享：放进项目里一起用

第三种是把技能放进项目目录，跟代码或文档一起管理。同组的人拉下来就有。

适用场景：

- 这个做法你们组已经达成过共识（比如"我们的报告必须这么写"）；
- 新同事需要照着它学会你们的标准；
- 这个做法会跟着项目变，需要版本管理。

项目级技能的常见位置是项目根目录下的 `.workbuddy/skills/`。

【待补：项目级技能的具体目录名与加载优先级，以实际版本为准】

这条路最容易踩的坑是"写了没人用"。我见过一个组写了六个技能，三个月后没人用。原因很典型：写的人按自己脑子里的标准写，没跟同组对过，另外五个人各有各的干法。

我的建议是：别从零开始写团队技能，从抄开始。找一件组里某个人已经干得很好、其他人都在模仿的活，把他的干法写成技能，署他的名，让大家试一周，提意见改一版。先有一份被实际用起来的，再谈第二份。

来源	适用	花的时间	主要风险
自己写	带你们公司格式的事	15 到 40 分钟	写得抽象，等于没写
从市场装	通用活，别人踩过坑	几分钟	不安全或质量参差
团队共享	组里已有共识的做法	几小时加推广	写完没人用

技能的三种来源

先看这件事跟你们公司客户的特殊要求有没有关系



第 4.2 节: 三条路不互斥, 大多数人的技能夹里最后三样都有

图 4-2 三种来源怎么选: 先看这件事跟你们的特殊要求有没有关系

图 4-2 三种来源的岔路口。作者根据公开资料绘制。

4.3 手写一个技能: SKILL.md 的每个字该怎么放

这一节是本章的重点。我先给你一个能直接抄的完整例子，再逐段讲每段写什么、写多长、为什么。

先看完整例子

下面这个技能解决的就是本章开头那件事：把原始试验数据整理成月度汇总表。你可以整段复制，把里面的项目名、路径、判定值换成你自己的。

```

---
name: test-data-digest
description: 把台架试验原始数据整理成对外发布的月度汇总表。当用户提到月度数据包、试验数据汇总、发客户的数据、按模板整理试验数据时触发。不适用于原始数据清洗，不适用于异常值判定。
version: 1.0.0
allowed-tools: Read, Write, Bash
---

# 试验数据月度汇总

## 这份活是给谁干的

每月 5 号前，把上月所有 MST 台架试验数据整理成一份 Excel 发客户。
对接人在客户侧的 Project Quality 部门。这份表会进客户的月度评审材料，
出错要返工，返工由我们组承担。

## 开工前你必须拿到

- 原始数据导出的 CSV，通常名为 mst_export_YYYYMM.csv
- 本月有效试验清单（在 references/test-plan-2026.md 里）
- 操作人姓名（不确定就问我，不许自己编）

缺任何一样，先问我，不许往下做。

## 步骤

1. 读 references/test-plan-2026.md，取出本月该做的试验清单和判定值。
2. 读原始数据 CSV，按清单筛掉不在本月计划里的行。
3. 计算超差：超过判定值上限或低于下限的行，整行标浅红底（RGB 255, 242, 242）。
4. 生成两页：Sheet1 汇总，Sheet2 原始曲线数据。
5. 按 references/report-format.md 套表头和页脚。
6. 自查一遍（见下）。
7. 输出到 output/YYYY-MM_月度数据包.xlsx，汇报给我。

## 顺序不许变

测试类别在 Sheet1 里的排列顺序是硬规定，客户三年前定的：

耐久测试 → 振动测试 → 温升测试 → 噪声测试

不许按数据量大小排，不许按字母排，不许按你上次的习惯排。

## 格式硬规定

- 表头中英双语，例：试验名称 / Test Name
- 日期一律 YYYY-MM-DD，例：2026-03-15（不是 2026/3/15）
- 转速单位 rpm，扭矩单位 N·m，温度单位 °C
- 页脚：试验日期 | 操作人 | 页码

```

不许做的事

- 不许用默认判定值。判定值必须来自 test-plan-2026.md
- 不许删掉任何一行原始数据，包括看着像废数据的行
- 不许把多个试验的数据合并成一行
- 操作人姓名不许猜，不知道就停下来问

干完之后自查

逐条确认再汇报：

- [] Sheet1 里四类测试的顺序是耐久、振动、温升、噪声
- [] 所有日期格式是 YYYY-MM-DD，一个斜杠都没有
- [] 超差行数与我给的判定值算出来的数一致，写出这个数字
- [] 页脚四项齐全
- [] 文件存在 output/ 目录下，文件名带年月

向我汇报什么

一句话告诉我：本月共 N 条有效试验，超差 M 条，文件在哪个路径。

如果有哪条数据你拿不准（比如缺页、时间戳异常），单独列出来，别自己处理。

逐段拆解：每段写什么、写多长

一个 SKILL.md 只有两块：开头元信息（两个 --- 之间）和正文（第二段 Markdown）。我先说元信息，这部分最容易写废。

name：技能的身份证

```
name: test-data-digest
```

用小写英文加短横线，不要用中文，不要用空格。这个名字同时是它的文件夹名，所以 test-data-digest 对应的目录就得叫 test-data-digest。两边不一致会加载失败，这是我踩过的第一个坑。

description：决定它什么时候出场

这是整个技能里最要紧的一行，也是最容易写废的一行。它有两个作用：让 WorkBuddy 判断"这个活该不该叫它"，以及让你自己三个月后还记得这玩意是干啥的。

我踩过的坑是这么写的：

```
description: 处理试验数据相关任务
```

结果它在我只想打开文件看一眼的时候跳出来，在我真正要整理数据的时候又不出来。因为它太宽了。

有效的写法要包含三样东西：干什么 + 什么话会触发 + 什么情况别用它。

description: 把台架试验原始数据整理成对外发布的月度汇总表。当用户提到月度数据包、试验数据汇总、发客户的数据、按模板整理试验数据时触发。不适用于原始数据清洗, 不适用于异常值判定。

长度参考: 50 到 150 字。短于 30 字通常意味着你说得太笼统; 长过 200 字说明这个技能想干的事不止一件, 应该拆开 (见 4.5 节)。

"不适用于"这一段千万别省。这一段它不抢活, 比它会抢活重要得多。我有个技能早期没有写"不适用于", 结果部门里所有人只要提到"数据"两个字它都跳出来, 被吐槽了半个月。

version 和 allowed-tools

```
version: 1.0.0
allowed-tools: Read, Write, Bash
```

version 自己管就行, 改一次正文加一位小数。 **allowed-tools** 是给这个技能的工具权限: 读文件、写文件、跑命令行。这里的原则是够用就行。

一个只做排版的技能不要给它跑命令行的权限。这不是小题大做, 我见过有人的技能因为没限制权限, 在一次跑飞的过程中把他桌面上的临时文件全删了。

正文第一段: 这份活是给谁干的

两三句话交代清楚: 什么时间、给谁、做成什么样、错了谁背。

这一段看着像废话, 实际上是我后来越写越看重的一段。AI 在做取舍的时候会回到这里。你写"这份表会进客户的月度评审材料", 它就会更保守地处理拿不准的数据; 你写"这是我们自己内部看的", 它就会大胆一些。

这一段决定了它在信息不够时选择多做还是少做。

开工前你必须拿到

列出所有前置物和所有输入文件, 然后加一句"缺任何一样, 先问我, 不许往下做"。

这一句是止损线。没有它, AI 缺输入时会自己去想办法, 最典型的办法是猜。猜一个姓名, 猜一个判定值, 然后用一本正经的语气汇报给你。

步骤

写具体的动作, 一条一行, 动词开头。条数控制在 7 条以内, 超过 7 条说明这个技能该拆成两个。每条写一个动作, 不要一个格子里塞三件事。

对比一下这两种写法:

差的写法	好的写法
整理数据并生成报表	1. 按计划清单筛行 2. 计算超差行的判定值 3. 超差行整行标浅红底 4. 生成两个 Sheet
注意格式	表头中英双语; 日期 YYYY-MM-DD; 页脚带操作人

差的写法	好的写法
尽量检查一遍	核对超差行数与我给的判定值算出的数一致

判断标准只有一条：一个从没干过这件事的人，能不能照着你写的把活干完。每一条里出现"注意""尽量""适当""常见的"这类词，就是不合格。

顺序不许变 / 格式硬规定

这两段我建议单独拎出来，不要混在步骤里。因为它们是"规定"而不是"动作"，混在动作里会被当成可选项执行。

写成"不许"的句式比写成"应该"有效得多。对比一下：

- "建议按耐久、振动、温升、噪声的顺序排列"
- "测试类别顺序为耐久、振动、温升、噪声，不许按数据量大小排，不许按字母排"

第二句把我被 AI 坑过的具体情形都写进去了。你被坑过什么，就把什么写进去。这一段的内容只能来自你真实的翻车记录，写不出来说明你还没被坑够，那就先别写这个技能，等下一次翻车记下来再说。

不许做的事

写三到五条，挑那些出了事最贵的禁止项。我列的四条是：不许用默认判定值、不许删原始数据、不许合并行、不许猜姓名。

每条背后都是一次返工。

干完之后自查

这是我最舍不得删的一段，也是大多数人忘了写的一段。它把一个只会交差的技能，变成一个交差之前先自己过一遍的技能。

写成可勾选的清单，每条都要能被 Yes 或 No 回答。"仔细检查格式"不是自查项，"所有日期格式为 YYYY-MM-DD，无斜杠"才是。

开头那个例子里有一句"超差行数与我给的判定值算出来的数一致，写出这个数字"。这是我最喜欢的——一类自查项：要求它输出具体数字。数字是可以当场验的，形容词不行。

向我汇报什么

规定汇报格式。我喜欢要求它写清楚三个数：总数、异常数、文件路径。再加一句"拿不准的单独列出来，别自己处理"。

这一句话改变了我和 AI 的关系。以前它报喜不报忧，所有不确定的地方它自己填一个最像的值补上。现在它会明确地把"+2 行数据时间戳异常，请确认"列出来，我扫一眼就知道该看哪。

配套目录：scripts、references、assets

正文写不下或者不该写进正文的，放这三个目录：


```

~/workbuddy/skills/test-data-digest/
├─ SKILL.md
├─ references/
│   └─ test-plan-2026.md      本月试验计划与判定值
│   └─ report-format.md      表头页脚的详细格式
├─ scripts/
│   └─ build_sheet.py         生成 Excel 的固定脚本
└─ assets/
    └─ template.xlsx          空模板

```

`references/` 放长资料，正文里写"读 `references/xxx.md`"就行，AI 会按需加载。`scripts/` 放确定性的重复动作，写死一个脚本比每次让它自由发挥稳得多。`assets/` 放模板和示例。

配套目录的使用原则：正文里只放"规则"，不放"数据"。判定值会变、格式会改，改的时候你只想改 `references/` 里的一个表，不想在正文里到处找。我最初把三年的判定值全写在 `SKILL.md` 里，第二年更新的时候改漏了一处，那个月的报告全部用了旧值，还是客户先发现的。

长度：写多长合适

我一开始以为越长越好，最夸张的时候写过一个 800 行的技能。结果它自己都读不完，干出来的活还不如不加载技能。

后来我摸出来的区间：正文 100 到 400 行，多数技能在 150 行左右。超过 400 行就该拆，或者把长资料挪到 `references/`。

判断要不要拆的标准就一条：这个技能能不能用一句话说清它干什么。说不清，就是塞了两件以上的事。

SKILL.md 的两大部分

开头元信息决定它出不出场，正文决定它干得稳不稳

元信息 name

小写英文加短横线，不用中文不用空格，它同时是文件夹名

元信息 description

最要紧的一行：干什么、什么话会触发、什么情况别用它

version 与 allowed-tools

改一次正文加一位小数；工具权限够用就行

正文：这份活是给谁干的

交代时间、对象、后果，决定它在信息不够时怎么做

步骤与硬性规定

步骤不超 7 条；顺序不许变、格式硬规定单独拎出来

不许做、自查、汇报

挑最贵的禁止项；自查要能答是否；汇报给具体数字

第 4.3 节：正文 100 到 400 行，多数技能在 150 行左右

图 4-3 SKILL.md 的结构：元信息决定出场，正文决定质量

图 4-3 一个 SKILL.md 的两大部分和六个正文段落。作者根据公开资料绘制。

4.4 技能会自己变强吗：一组六代迭代的真实数据

写完第一个技能，你大概率会问这个问题：它能不能自己越用越好？

为了回答这个问题，我在本机搭了一套自我进化实验，跑了两个多月。这一节把数据摊开给你看，包括难看的部分。

数据全部来自我本机的实验记录，取的是 2026 年 5 月 12 日的一轮完整跑，每一项都是真实跑出来的，不是我估的。

实验是怎么跑的

整套流程分三步，每一步都对应一个脚本：

1. 攒评测集。从我过去两个月的记忆文件里挖出每一次任务的记录：当时说了什么、用了哪个技能、成功还是失败、报错信息是什么。产出是一个 80 条的评测数据集，其中 28 条标记为成功。
2. 生成变体。程序读取评测集，统计每个技能最常出现的错误类型，针对每种错误类型生成一个改进版本，最多生成 5 个变体。
3. 打分。先用一套固定规则打分，再让本机跑的一个开源小模型按三个维度打分，最后做统计检验。

参与这轮实验的有 13 个技能，每个技能 6 个版本（1 个原始版 + 5 个变体），一共 78 份文件。下面我挑出其中最具有代表性的一个，从头到尾摊开讲。

主角：一个浏览器自动化技能的六代

这个技能干的是驱动浏览器替我查数据、翻页、填表。它是我装机最早的技能之一，也是我当时用得最狠的一个。

版本	改进目标	当时的错误模式	得分
baseline	原版，无改进	original	0.70
v1	原版，模型分析失败	original	0.70
v2	添加权限检查和错误处理	权限错误	0.65
v3	规范路径处理，使用绝对路径	路径错误	0.75
v4	统一使用 UTF-8 编码，添加编码回退	编码错误	0.75
v5	实现指数退避重试和速率限制处理	API 错误	0.75

一个技能的六代得分

不是一路向上，是先跌后涨，再进平台期



第 4.4 节：进步空间大约 0.05 分，涨分来自不再犯同一个错

图 4-4 六代迭代的得分曲线：不是一路向上，是先跌后涨再进平台期

图 4-4 一个技能的六个版本，得分从 0.70 走到 0.75。作者根据公开资料绘制。这条曲线讲了五件事，每一件都比"AI 会自己进化"这句宣传语有用得多。

第一件事：第一次改不动，是常态。v1 那一行的改进目标写的是"原版，模型分析失败"。意思很直白：这一轮程序想分析它的错误，调用模型失败了，于是原样输出，评分 0.70，和 baseline 一模一样。

我第一次看到这个结果挺失望，觉得白跑了一轮。后来跑多了才明白，这就是这类自动化流程的日常。

真正让我踏实的反而在这儿：它有版本号留档，我一眼就知道这一轮没动，不会被一个假涨的分骗过去。我自己在手改技能的时候没有这个习惯，改完觉得"好像好点了"，然后说不清改了什么。

第二件事：改了可能更差。v2 加了权限检查和错误处理，逻辑上完全正确，挑不出任何毛病。得分从 0.70 掉到 0.65。

为什么？我回去对着评分规则一条条查，找到了原因。当时的规则打分里有"正文长度"这一项，理想区间是 500 到 2000 字符，落在区间里给满分 0.3。v1 的正文已经偏长了，v2 又追加了一整段权限排查，跑出了区间，这一项被扣了分。

这件事让我一下子清醒了：自动化流程也会"为了改而改"。它识别出一个错误类型，就往里加一段对应的处理，加完之后总体反而变差。如果没有评分留档，我会本能地认为"加了处理 > 没加处理"，然后带着一个更差的技能用半年。

第三件事：真正有用的改动只有一类。v3 是唯一一次实打实的涨分：从 0.65 到 0.75。它改的是路径处理：统一用绝对路径，分隔符一律用正斜杠，路径里有空格要加引号。

看到这个结果我拍了下桌子。因为这个技能是给 Windows 用的，而我真实的上班经验里，浏览器自动化有一半以上的报错来自路径：驱动器写成了小写、反斜杠没转义、桌面路径里有中文。

v3 那三条规则，正好是这个技能在我这台机器上最容易摔跤的三个方面。它之所以涨分，是因为它改到了真实的摔跤点。

对照 v4 和 v5 就更能说明问题。v4 改编码，v5 改接口重试，都是好东西，都是工程上的正确做法。但它们在这个技能上一分没涨，因为这个技能从来没在编码和接口限流上出过问题。

你给一个从来不犯某类错的技能，加上二十条防那类错的规矩，它不会因为规矩多了就变强。

第四件事：边际收益很快耗尽。v3、v4、v5 三代全是 0.75，纹丝不动。这一行数据是我整个实验里最有价值的部分之一。

它说明这样一个事实：靠"识别错误类型、套一段通用处理"这种模板化的改法，进步空间大约是 0.05 分，一两轮就吃完了。再往后想要涨，得靠别的东西。

什么东西？我的判断是真实的使用记录，也就是 v3 那种"知道它在这台机器上具体摔在哪"的信息。模板化改动可以给任何一个技能加；知道它摔在哪，只有用过它的人能给。

第五件事：最后一次不等于最好一次。这是我在另外 12 个技能里学到的，主角这个技能没体现出来。我把它单拎出来说，因为它最容易害人。

技能	baseline	最高分版本	v5 得分
gsd-checkpoint	0.80	v3 (0.95)	0.85
git-workflow	0.80	v1 (0.85)	0.80
last30days-cn	0.70	v3 / v4 / v5 (0.85)	0.85
huashu-design	0.60	v3 (0.80)	0.70
agent-reach	0.60	v1 / v3 (0.75)	0.60

看最后一行。`agent-reach` 这个技能，跑到第五代，得分回到了 0.60，跟原始版一模一样。但它在第一代和第三代的时候是 0.75。

如果我只保留最新版本，我就把这个技能最好的两个版本扔了。这件事对所有用技能的人都适用：别用"最新版"这个词管技能，要用版本号。技能夹里可以留两三个历史版本，文件名上带版本号，用不了几个字节。

打分的三个教训

实验里最难看的部分，恰恰是最值得说的部分。就这些数，我不美化。

教训一：当时的规则打分能被"写得好看"骗过去。规则里给分的项目是：正文长度落在 500 到 2000 字符、包含代码示例、包含编号步骤、命中了错误类型的关键词。

这意味着什么？我把一段废话写到 800 字，随手加个代码块，再编三个编号步骤，就能拿到接近满分的规则分。

这个打分标准衡量的是"文档写得像不像技能"，不是"技能干活行不行"。所以我从不只看分数，每次都要自己读一遍改了什么。

教训二：模型打分自己也不稳。同一个变体，我连着跑了三次，模型给出的综合分分别是 0.8167、0.7833、0.7833。三次两次不一样。差距不大，但足够让你在比较两个变体的时候得出相反的结论。

原始版的三维得分更能说明问题：合规性 0.8，输出质量 0.6，简洁性 0.9，综合 0.77。而同一份文件的规则打分是 0.70。

同一份东西，两套打分差了 0.07。有意思的是三维本身：原始版最占便宜的是"简洁"，因为没被加过任何东西；垫底的是"输出质量"。后面几代往上堆内容，简洁分就得往下掉。

这是一个跷跷板，不是一路向上的阶梯。

教训三：统计检验也会翻烧饼。同样是那个 v1 变体，同样是检验它相对原始版有没有显著改进，三次跑出来的结果是：

轮次	差值	p 值	结论
第 1 轮	+0.3167	1.0	不显著，谨慎应用
第 2 轮	+0.2833	0.5135	不显著，谨慎应用
第 3 轮	+0.2833	0.05	显著，建议应用

一模一样的输入，三次跑出三个结论，其中一次说行，两次说不行。原因不复杂：当时的检验没固定随机种子，每次重采样都是新的一组随机数。

我看到这张表的时候，第一反应是把那段代码修了。修完之后留在心里的东西更值钱：任何一个会给你结论的流程，都要问一句"再跑一次还是这个结论吗"。

不光是自动化工具的结论，也包括 AI 给你的结论。多跑一次，成本极低，收益极高。

所以，技能到底会不会自己变强

会，但方式和你想的不一样。

它不会自己悟出新招数。它变强的那 0.05 分，来自"不再犯同一个错"，而不是"学会了新本事"。v3 涨的那 0.05 分，改的全是我过去两个月真真切切踩过的坑。

真正让它变强的燃料是人。v3 之所以能改对位置，是因为评测集里躺着 80 条我用它干活的记录。这些记录是我干活的副产品，不是程序生成的。

没有这 80 条真实记录，整条流水线跑出来的一定是 v4 和 v5：正确、无用、零分变化。

所以这套东西对你的用处不是"让它自己进化"，而是这一条：把你用 AI 干活的过程留下来，让下一次改技能的时候有凭据。

哪怕你什么都不自动化，只是每个周五花十分钟，把这周 AI 干砸的地方记进一个 `troubles.md`，一个月之后你就有二十条，三个月之后你就有肥料。我这 80 条评测数据，就是这么来的。

写技能这件事，起点不是写，是把踩过的坑记下来。

4.5 技能用不好的四个原因

我手上因为不好用而废弃的技能有十几个。复盘下来逃不出四个原因，每个我都给症状和修法。

原因一：写得太抽象

症状：加载了技能，AI 干出来的活跟没加载时一模一样。或者它干完之后，你对照正文发现每一条它都"遵守"了，结果还是错的。

根因：正文里全是形容词和原则，没有一条可以被验证的硬规定。

抽象写法	具体写法
注意格式规范	表头中英双语，例：试验名称 / Test Name
数据要准确	超差行数与 references/test-plan-2026.md 算出的数一致，汇报时写出这个数
适当标注异常	数值超出 [下限, 上限] 的行，整行底纹设为 RGB(255, 242, 242)
保持专业	不出现感叹号，不出现"非常好""显然"这类主观词

修法：通读全文，把每一句里有"注意、适当、尽量、合理、相关、常见"的词都标出来，逐个替换成数字、文件名、颜色值、时间点。替换不掉的，说明你自己也没想清楚，那就删掉那一句。删掉一句含糊话，技能反而变好。

原因二：描述和实际不符

症状：该它出场的时候不出来，不该它出来的时候抢着出来。或者它一出场就报错，因为正文里提的那个脚本其实早就被你删了。

这条最难查，因为 description 和实际正文是两个地方，改了一处忘了另一处。

我有一个吃过三次亏的例子：某个技能的 description 里写着"生成 CSV 合并脚本"，但 scripts 目录里那个脚本去年就被我换掉了，旧名字早没了。

它按照 description 去找旧脚本，找不到，然后开始自己现编一个。这个坑我踩了三次才意识到，问题不在 AI，在我自己没同步。

修法：每次改完正文，回头问自己一句："我正文里提到过的文件，现在真的存在吗？" 然后让 AI 自己验一遍：

我：读一下 ~/.workbuddy/skills/test-data-digest/SKILL.md，把正文里出现的每一个文件路径都列出来，逐个确认它们是否存在，不存在的说出来。

这是我现在的固定动作，改完技能就跑一次，十秒钟。

原因三：触发词没写对

症状：你说的话明明就是这件事，它就是不加载技能。或者反过来，你说啥它都加载。

根因：两种情况。一是触发词写得过窄，只写了一个说法；二是写得过宽，一个技能包了一大片。

有效的写法是把同义词全列上。同一个活，我说"月度数据包"，同事说"客户月报"，领导说"发客户的那套"。

description: 把台架试验原始数据整理成对外发布的月度汇总表。当用户提到月度数据包、月度汇总、客户月报、发客户的数据、试验数据汇总、按模板整理试验数据时触发。不适用于原始数据清洗，不适用于异常值判定。

第二个原因是没写"不适用于"。这条我在 4.3 节讲过，这里再说一遍是因为它太常被省掉。一段"不适用于"占三十个字，半年下来能省几十次误触发。

修法：每次它该出场没出场，就把你当时那句话原样加进 description。每半年回头删一次已经用不上的触发词。

原因四：一个技能塞了太多件事

症状：技能正文超过 400 行，步骤超过 7 条。它每次执行都只完成一部分，而且每次落下的部分还不一样。你说"顺手把 XX 也做了"，它就开始混乱。

根因：你把"相关的事"当成了"同一件事"。整理数据、判定超差、写结论、发邮件，这四件事逻辑上连着，但输入和输出完全不同。

堆在一起，AI 就得自己决定先干哪个，而它每次的决定都不稳定。

修法：按"输出物"拆，不按"主题"拆。

塞在一起的写法	拆开之后
试验数据处理技能（整理、判定、报告、发送塞一个技能）	test-data-digest（整理，输出 Excel） test-data-check（判定超差，输出判定表） test-report-gen（写报告，输出 Word） mail-sender（发送，输出发送记录）

拆的标准：一个技能只交一种东西。整理数据交 Excel，判定超差交判定表，写报告交 Word。输出物不同，就拆开。拆完之后每个技能按 4.3 节重写一遍，反而更快。

技能用不好的四个原因

我手上因为不好用而废弃的技能有十几个，复盘下来逃不出这四个

- ✖ **原因一 写得太抽象**
正文全是形容词和原则，没有一条能被验证。修法：把每一句含糊话换成数字、文件名、颜色值、时间点，替换不掉就删掉。
- ✖ **原因二 描述和实际不符**
该出场不出来，不该出来抢着出来。修法：每次改完正文，让它逐个确认正文里提到的每一个文件路径是否真的存在。
- ✖ **原因三 触发词没写对**
触发词过窄只写一个说法，或者过宽包了一大片。修法：把同义词全列上，并且一定写上不适用于。
- ✖ **原因四 一个技能塞太多事**
正文超过 400 行、步骤超过 7 条，每次只完成一部分。修法：按输出物拆，不按主题拆。

第 4.5 节：删掉一句含糊话，技能反而变好

图 4-5 四个用不好的原因和各自的修法

图 4-5 技能不好用，八成逃不出这四个原因。作者根据公开资料绘制。

4.6 三十多个已验证的技能能干什么

我本机技能目录里现在有一百七十来个技能，其中我自己常用、跑通过的三十多个。下面按五类列出，每类挑两三个。省时都是我自己估的，一律加"约"字。

这张表不是让你照着装三十个，是让你看清技能能覆盖到什么地步，然后挑一两个最像的先抄。

类别	技能名	它干什么	单次省下
数据整理	markitdown-skill	PDF、Word、PPT、图片批量转 Markdown，带 OCR	约 30 分钟
数据整理	summarize	一句话总结网页、PDF、音视频	约 15 分钟
数据整理	deep-research	结构化调研：先出大纲，再并行查，最后出报告	约 3 小时
报告生成	md-to-pdf	Markdown 转排版干净的中文 PDF	约 20 分钟
报告生成	ppt-master	诊断并重构现有 PPT，带失败回退	约 1 小时
报告生成	competitive-analysis	七步竞品分析，从目标定义到报告输出	约 4 小时

类别	技能名	它干什么	单次省下
图纸与校核	solidpilot-cad-build	驱动 SOLIDWORKS 建模，并做闭式解验证	约 2 小时
图纸与校核	gear-expert	齿轮参数、渐开线、强度与精度的算式校核	约 40 分钟
图纸与校核	motor-expert	微电机电磁与控制的专业校核	约 40 分钟
试验与复盘	nvh-expert	NVH 与异响问题的诊断路线建议	约 1 小时
试验与复盘	workbuddy-daily-review-runbook	每日复盘自动化，带缺文件时的回退	约 20 分钟
流程自动化	git-workflow	Git 分支、提交、合入的规范流程	约 15 分钟
流程自动化	task-pattern-tracker	识别重复任务，30 天内 3 次就提示固化为技能	见下面说明
流程自动化	planning-with-files	用 Markdown 文件做跨会话的进度跟踪	约 30 分钟

最后那个 `task-pattern-tracker` 要单拎出来说。它本身不省时间，它的工作是盯着你：同一类任务 30 天里出现 3 次，它就提醒你“是不是该写成技能了”。

发现该写技能的时机，比写技能本身更难。本章开头那个周四下午，如果我早装了它，我在第二个月月底就会被提醒，而不是拖到第五遍才发现。

完整清单，包括安装方式和已知坑位，见附录 C：技能清单与安装源。

4.7 如果你只做一件事

今天下班前，把今天做过的某一件活，写成你的第一个技能。

无需挑选。挑一件今天刚做完、你还记得清过程的活，热气没散，写起来最快。

判断标准只有一条：这件事你在过去一个月里做过两次以上。花的时间：新手 15 分钟，熟练之后 5 分钟。

具体做法，直接把下面这段话复制给 WorkBuddy：

我：回顾我们这次做的「XXX」这件事，把它写成一个技能，存到 `~/workbuddy/skills/YYY/SKILL.md`。要求：1. `description` 里写清楚干什么、什么话会触发、什么情况不适用 2. 正文包含：这份活是给谁干的、开工前必须拿到什么、步骤（不超过 7 条）、硬性规定、不许做的事、干完自查清单、向我汇报什么 3. 所有判定值、格式要求写具体，不许出现“注意”“适当”“尽量” 4. 正文里提到的每一个文件路径，你先确认存在 写完读给我听。

然后你自己读一遍，重点是判定值和"不许做的事"。它总结的动作通常没问题，它总结的数字经常是它自己编的。

明天或下周你就用一次。要是发现哪一处格式做错了，别在对话里纠正它，打开 `SKILL.md` 把那一处改掉。改文件，不在对话里反复纠正。这是本章唯一要求你做的动作。

小白词典

技能 (Skill)：一份说明文件，告诉 WorkBuddy 某一类活该按什么顺序干、交出什么算合格。类比新员工的岗位手册。

SKILL.md：技能的核心文件。开头是两个 `---` 之间的元信息，下面是给 AI 看的正文。

元信息 (frontmatter)：文件最开头用两个 `---` 包起来的一小段配置，写技能名、描述、版本、权限。类比贴在箱子外的货运标签。

触发词：写在 `description` 里的那些话，AI 听到你说这些话就会加载这个技能。类比手机上的快捷指令。

评测集：一批用来给技能打分的测试题，通常来自历史任务记录。类比驾照考试的题库。

模型打分 (LLM-as-Judge)：让另一个模型当裁判打分。类比同行评审，好处是能看懂语义，坏处是它自己也不稳。

Bootstrap 检验：重复抽样几千次，看两个版本的差距是真还是碰巧。类比同一批零件反复抽测。

变体 (variant)：技能的某个改动版本，比如 v3。类比工程样件，一代一代往下改。

你的岗位怎么用

设计工程师：先写一个"出图前检查"技能，把你每次必查的十几项写进去。这是见效最快的一类，因为图纸出错很少错在计算，多半错在漏项。

再写一个"数模命名与归档"，按你们部门的图号规则来，这块每个厂差很多，只能自己写。

试验工程师：本章那个月度数据包技能可以直接套。你要做的第一件事是把判定值搬到 `references/` 里，因为试验判定值每年都在变，写死在正文里第二年一定有人出错。

质量工程师：写一个"来料检测报告整理"。重点放在"不许做的事"那一段：不许删行、不许合并、不许改判定值。你的活对可追溯性的要求比别人高，那一段是你最该下力气写的地方。

采购 / 供应商管理：写一个"供应商资质核查"，把核查项、有效期算法、脱敏规则写死。这条路收益最大，因为你每个月都要重来一遍，而且漏一项的代价最贵。

项目经理：写一个"周报生成"，把格式、必含项、风险分级写进去。你的回报不在省时间，在每周那十五分钟不用再重启大脑。

研发主管：你自己先写两个，用满一个月，然后把你用的那份发给组里。别从零开始制订团队标准，先有活的东西再有规矩。本章 4.2 节那个"六个技能三个月没人用"的例子，就是从零开始写标准最常见的下场。

本章交付物

《我的第一个技能文件》：一份能直接放进 `~/.workbuddy/skills/` 的 SKILL.md，写的是你一件做过两次以上的活。

怎么做：

1. 今天下班前，从今天的活里挑一件你做过两次以上的，写下名字（2 分钟）。
2. 把 4.7 节那段话复制给 WorkBuddy，让它生成第一版（3 分钟）。
3. 你自己读一遍，重点核对判定值和文件路径（5 分钟）。这一步不许省。
4. 明天或下次做这件活的时候用它。做错一处，就去改文件，不在对话里纠正（5 分钟）。
5. 文件名加版本号，比如 `test-data-digest-v1.md`，改之前先留个副本。别用"最新版"这种名字，见 4.4 节最后那个 0.60 的教训。

我的经验：多数人写第二个技能，是因为第一个写完才发现，自己那些"不成文的做法"其实讲得清楚。我写完月度数据包技能之后，第二周又写了三个。

另外一个提醒：别一次性写十个。写完立刻用，用过一轮再改。技能是养出来的，不是写出来的。

本章金句

同一件事你教它第二遍，就是你自己的问题。

技能不提升 AI 的智商，它提升的是 AI 的稳定性。

提示词是一次性的交代，技能是写一次的规矩。

同一个工具，有人十分钟完工，有人往返八轮，差别不在工具，在你有没有把规矩写完。

技能不好用，通常不是它出错，是它在正确地做无用功。

没有真实使用记录的自我进化，加的每一段都正确，加完之后一分不涨。

（下一章：记忆。技能解决的是"重复交代"的问题，记忆要解决的，是"它会自己记住什么、忘掉什么，以及怎么让它记住该记住的"。）

第 5 章

让它记住你和你的项目

换一个对话窗口它就不认识你了。不是它记性差，是你从来没让它记过。

2026 年 3 月的一个周二上午，我坐在工位上，第 9 次向一个 AI 介绍同一个项目。

那天我要它帮我做一件事：把 PJ-2318 项目的 47 份 DV 试验报告汇总成一张超差清单。活本身不难，难的是开头。我把对话窗口打开，开始交代背景：

我们做汽车微电机和智能执行器的，这次是一个充电口盖执行器，12 V 永磁直流微电机加两级行星齿轮箱，客户是某欧系平台，现在在 DV 阶段，今年 3 月要进 PV。报告表头照 2025 版模板，尺寸公差按 ISO 2768-m，图纸命名是 PJ2318-XX-XXX，超差放行要项目经理书面同意。客户真名不要写进外发文件。

打完这段话，我数了一下，218 个字。这 218 个字，我在过去两个月里打过 8 遍。

更烦的是第 6 遍之后的发现：我每次打的内容还不一样。有一次我忘了说“客户真名不要写进外发文件”，它给我生成的汇总表里，客户代号那一栏直接写了真名。我把那张表改了一遍才敢发出去。还有一次我说了图纸命名规则，但写成了 PJ-2318-XX-XXX，多了一个横杠，它后面生成的文件名全错。

那天上午我算了一笔账。两个月里，这类“重新自我介绍”平均每次 3 分钟，8 次是 24 分钟。加上因为漏说、说错而返工的两回，一共大概 50 分钟。50 分钟不是什么大钱，但它每一分钟都花在同一种烦躁上：我在跟一个明明见过我八次的同事，第九次握手时重新报一遍姓名和工号。

让我彻底改掉这个习惯的，是第 5 章要讲的这三样东西：它有三层记忆、我知道该往里面写什么、我知道喂完怎么验收。

5.1 三层记忆是什么

先把“记忆”这个词拆开。WorkBuddy 说的记住你的事，不是一件事，是三层。这三层各存各的东西，各管各的范围，搞混了就会出两种错：该全局生效的写进了项目里，换个项目就丢；该跟着项目走的写进了全局，换个项目就被旧规矩带偏。

	云端画像	用户级记忆	工作区记忆
存什么	你的身份、岗位、行业、语言与表达偏好	跨项目通用的工作习惯、常用格式、常用工具	某个具体项目的背景、术语、文件规则、审批流程
谁来写	它根据你的对话自动归纳	你直接说，或让它总结后确认	你写进记忆文件，或让它追加

	云端画像	用户级记忆	工作区记忆
什么时候生效	任何对话、任何项目	你这个账号下的所有对话	只在这个工作区、这个项目里
换项目还在吗	在	在	不在
忘了会怎样	每次都要重新介绍你是谁	每次都要重申格式和习惯	每次都要重讲一遍项目背景
类比	你这个人	你所在的部门	你手上的项目

三层记忆的分工

这三层各存各的东西，各管各的范围

云端画像：我是谁

存身份、岗位、行业、语言与表达偏好，跟着账号走，换电脑换项目都还在

用户级记忆：我怎么干活

跨项目通用的工作习惯、常用格式、常用工具，比如表格交 xlsx、客户写代号

工作区记忆：这个活是怎么回事

某个项目的背景、术语、文件规则、审批流程，换个工作区就读不到

一句话怎么记

搞混了会出两种错：该全局的写进项目里换个项目就丢，该跟项目走的写进全局会被旧规矩带偏

第 5.1 节：忘了会怎样，是检验这一层有没有写对的最快办法

图 5-1 三层记忆：个人习惯、部门规章、项目档案

图 5-1 三层记忆的分工与生效范围。作者根据公开资料绘制。

三层用一句话区分：云端画像是"我是谁"，用户级记忆是"我怎么干活"，工作区记忆是"这个活是怎么回事"。

用我们公司的结构打个比方，这对工程师最好懂：

- 云端画像像你的个人档案。上面写着你是研发工程师、做了十一年微电机和执行器、写文件用中文。这份档案跟着你走，你调到哪个项目组，它还是那份档案。
- 用户级记忆像部门规章。你所在部门规定，技术报告的表头照 2025 版、数据一律交 xlsx 不交截图、术语第一次出现要括号标英文缩写。这些规矩跟你做哪个项目无关，只要你在干活，它们就成立。

- 工作区记忆像项目档案柜。PJ-2318 的客户代号、图纸命名规则、超差放行的审批人，全在这个柜子里。你做 PJ-2402 的时候，这个柜子不打开。

我踩过的那个坑，正好是分层的反面教材。我最早把"客户真名不要写进外发文件"写进了工作区记忆，只在 PJ-2318 那个目录里生效。三个月后我在另一个项目里让它出一份外发的技术说明，它把那家客户的真名写上了。原因很简单：那条规矩根本不在它当时读到的记忆里。

后来我改了。这条规矩属于"我怎么干活"，于是我把它提到了用户级记忆，写成一条通用规则："任何可能外发的文件，客户一律用代号，代号清单见当前项目的记忆。"这样不管我在哪个项目，它都会先去查代号，查不到就来问我。

【待补：记忆的查看与编辑入口，以你使用的版本界面为准。通用原则是：用户级记忆有独立的配置入口或记忆文件，工作区记忆通常落在工作区根目录下的记忆文件里。】

5.2 该往记忆里写什么

记忆不是垃圾桶。我见过有人把一整份 80 页的技术标准全文塞进记忆，说"这样它以后就都知道了"。结果是他每次问别的事，回答里都混着那条标准的只言片语，像一个人背书背串了。

判断该不该写，只看一个标准：这条信息，三个月后你还会用吗？会用的写，只此一次的别写。

	该写	不该写
身份与偏好	我是汽车零部件研发工程师，做微电机与智能执行器，11 年一线经验	我这周在忙一个临时的比价任务
表达与格式	技术文件用中文，术语首次出现括号标英文缩写；表格任务输出xlsx	这一份报告要写 2000 字
项目背景	PJ-2318 是某欧系平台的充电口盖执行器，DV 阶段，3 月进 PV	今天这批样品到了 12 件
文件与命名规则	图纸命名 PJ2318-XX-XXX，2D 出 PDF，三维源文件 CATIA	上一版图纸存在 D 盘的某个文件夹里
审批与责任	图纸三级审，超差放行要项目经理书面同意	这次张工口头同意了放行
术语与代号	客户一律写代号；"A 厂"指齿轮箱供应商，"B 厂"指电机供应商	一整份标准的全文、一整本图纸的明细表

不该写的那三类的共同点是：它们有保质期。今天这批样品的数量、这次口头同意的放行、这个临时比价任务，三天后就是噪音。噪音进了记忆不会自己消失，它会一直占着位置，把你真正要用的那条规矩挤到后面。

还有一类不该写，是大段原文。80 页的标准不要塞进去，把它的位置写进去就够了："公司的尺寸公差标准全文在 [标准/IS02768应用说明.pdf](#)，需要时先读这个文件。"让它去读文件，比让它背文件靠谱得多。文件是死的，背下来的东西会串。

该写进记忆与不该写

判断只看一个标准：六个月后你还用不用

该写进记忆

- 身份与偏好
零部件研发工程师，做微电机与执行器
- 表达与格式
术语首次出现括号标英文缩写
- 项目背景
PJ-2318 充电口盖执行器，DV 阶段
- 文件与命名规则
图纸命名 PJ2318-XX-XXX，2D 出 PDF
- 审批与责任
图纸三级审，超差放行要书面同意
- 术语与代号
客户写代号，A 厂齿轮箱、B 厂电机

不该写

- 临时的活
我这周在忙一个临时的比价任务
- 一次性的要求
这一份报告要写 2000 字
- 当天才有效的数据
今天这批样品到了 12 件
- 随手的位置
上一版图纸存在 D 盘某个文件夹
- 口头约定
这次张工口头同意了放行
- 大段原文
一整份标准的全文、整本明细表

第 5.2 节：把位置写进去，让它去读文件，比让它背文件靠谱

图 5-2 该写与不该写：一条判断线，六个月后还用不用

图 5-2 用“六个月后还用不用”这一条线判断该不该写进记忆。作者根据公开资料绘制。

5.3 怎么喂记忆：三种写法

知道该写什么之后，是怎么写进去的问题。我常用的有三种，从快到慢排。

写法一：直接说“记住……”

最快，适合一句话能说清的规矩。要点是两个：说完要它复述，复述对了才算写完。

记住三件事，以后默认照做，不用我再重复。

1. 我的身份：汽车零部件研发工程师，做汽车微电机与智能执行器，11 年一线经验。
2. 我写技术文件用中文，专业术语第一次出现时括号标注英文缩写，之后不再重复标注。
3. 凡是涉及表格的任务，交付物一律是 .xlsx 文件，不要把结果用文字描述给我让我自己抄。

现在把上面三条复述一遍，逐条对应。哪一条你没把握，直接说。

它的回复应当长这样：

已记住，复述如下：1. 你的身份是汽车零部件研发工程师，方向为汽车微电机与智能执行器，有 11 年一线经验。2. 技术文件用中文撰写，术语首次出现时括号标注英文缩写，后续不再重复。3. 涉及表格的任务，交付 .xlsx 文件，不以文字描述代替。三条均已写入用户级记忆。

复述这一句"现在把上面三条复述一遍"不能省。我吃过亏：有一次我说完"记住我们报告表头照 2025 版"，它回了一句"好的，已记住"。三个月后我发现它写进记忆的是"报告表头照 2022 版"。就差一个数字，我那份报告被主管退回来重做。从此以后，凡是进记忆的内容，我一律要它回显原文，我逐字对。

写法二：让它自己总结，你确认后再写

适合刚干完一件比较复杂的活。这时候你和它都已经聊了很多轮，背景散在各处，你自己未必理得清哪些该留。让它先列，你来筛。

我们刚才一起做完了 PJ-2318 的 DV 数据汇总。请你做两件事。

第一件事：从这段对话里，挑出那些"下次还会用到"的信息，最多 10 条，每条一行。
挑的标准是：这条信息六个月后我还用得上。

第二件：给每条标一个归属，只标两个值：

- [人]：属于我这个人的习惯或身份，跟项目无关
- [项目]：属于 PJ-2318 这个项目，换个项目就不成立

只列清单，先不要写入任何记忆文件。我确认之后你再写。

它会给出一份清单，通常长这样：

1. [人] 技术文件用中文，术语首次出现括号标英文缩写 2. [人] 表格类任务的交付物是 .xlsx 3. [人] 客户名写代号，不写真名 4. [项目] PJ-2318：某欧系平台充电口盖执行器，12 V 永磁直流微电机 + 两级行星齿轮箱 + 霍尔位置反馈 5. [项目] 当前 DV 阶段，2026 年 3 月进 PV 6. [项目] 图纸命名 PJ2318-XX-XXX，2D 出 PDF 7. [项目] 尺寸公差默认 ISO 2768-m 8. [项目] 超差放行要项目经理书面同意 9. [人] 数据要留单位，不写裸数字 10. [项目] 齿轮箱供应商称 A 厂，电机供应商称 B 厂

这份清单的价值在于你一眼能看出它归纳错了什么。我就靠这一招抓到过一条：它把"这批样品的样件编号范围"列进了 [项目]，我划掉了，因为那是一次性的。

确认的回复也要给死：

第 1、2、3、4、5、6、7、8、10 条写入记忆，第 9 条改成"所有数据必须带单位，包括角度和扭矩"。
第 4 条里补一句"用于充电口盖的开闭驱动"。
删掉你原清单里我划掉的那条样品编号。

写之前先把最终版本完整回显一遍，我说"确认"之后你再执行写入。

最后那句"我说确认之后你再执行写入"是这整套动作的关键。不给它这句，它会一边回显一边已经写好了，你划掉的内容已经在里面了。

写法三：直接写进记忆文件

最慢，也最可控。适合项目背景这种成块的内容，尤其是你想让整个团队共用同一份背景的时候。

做法是你自己按格式写好一段，让它追加进工作区的记忆文件。

打开当前工作区的记忆文件，在"项目背景"这一节下面追加下面这段内容。
不要覆盖已有内容，不要修改其他任何章节，只追加。

```
## 项目 PJ-2318
- 零件：充电口盖执行器，用于驱动充电口盖开闭
- 结构：12 V 永磁直流微电机 + 两级行星齿轮箱 + 霍尔位置反馈
- 客户：某欧系平台（对外文件一律写这个代号，不写真名）
- 阶段：DV 阶段，2026 年 3 月进 PV
- 供应商代号：A 厂 = 齿轮箱，B 厂 = 电机
- 文件规则：图纸命名 PJ2318-XX-XXX；2D 出 PDF，三维源文件 CATIA
- 报告规则：表头照 2025 版模板；尺寸公差默认 ISO 2768-m
- 审批规则：图纸三级审；超差放行必须有项目经理书面同意
```

追加完成后，把这一节的完整原文回显给我，我要逐条核。

为什么要"回显"和"不要覆盖"这两句，我在写法一里说过一次，这里再说一次，因为它在这个场景下更致命。记忆文件是整个项目的档案柜，一句覆盖就没了。我第一次用这个写法时没写"不要覆盖"，它把文件重写了一遍，我原来那条"客户写代号"的规矩没了，而我是一个月后才发现的。

三种写法怎么选，一句话：一句话的规矩用写法一，干完一件复杂活之后用写法二，成块的项目背景用写法三。

5.4 记忆失效的三种情况与排查

喂进去了不代表能用。我遇到过的失效就三种，每一种的症状和治法都不一样。

记忆失效的三种情况

先看症状，再定动作，三种失效的排查顺序不一样



第 5.4 节：喂进去不代表能用，喂完要验收

图 5-3 记忆失灵排查树：先看症状，再定动作

图 5-3 三种记忆失效的排查顺序。作者根据公开资料绘制。

情况一：记了不用

症状：你明明说过“表格任务输出 `xlsx`”，它这次又用文字描述了一堆数据，还得你自己抄。

原因通常有两个。一是你当初写进的层级不对，写进了工作区记忆，这次对话换了个工作区，它读不到。二是那条规矩写得太模糊，“输出表格”和“输出 `.xlsx` 文件”是两回事，它按前者理解，用 Markdown 表格回给你，也算“输出了表格”。

排查动作：先问它一句，别急着改。

你现在的记忆里，关于“表格类任务的交付格式”，写的是哪一条？把原文发给我。
原文里没有的那一条，就是它没生效的那一条。

修复话术：

这条规矩要改成跨项目通用的，写到用户级记忆，不要只写在当前工作区。
原文改为：凡是涉及表格的任务，交付物一律是 `.xlsx` 文件，生成到当前目录下，不要用 Markdown 表格或文字描述代替。改完回显原文。

情况二：记错了

症状：它记住了，但记的内容和你说的不一样。比如你说"ISO 2768-m"，它记成了"ISO 2768-f"；你说"2025 版表头"，它记成"2022 版"。

原因：它在归纳时做了"合理化"。它见过更多的 m 还是 f 我不知道，但结果是它把你说的往它认为更常见的那个方向改了半个字。这种错最阴，因为它不会报错，它会在你最信任它的那份报告里出现。

排查动作：定期抽查。我的习惯是每个月的第一周，让它把自己记忆里的核心条目列一遍，我花三分钟对一遍数字和代号。

把当前生效的所有记忆条目按"人 / 项目"两类列出来，每条一行，原文照抄，不要改写。
涉及标准号、版本号、代号、人名、日期的条目，单独放到一个分组里。

修复话术：

第 7 条记错了。原文应该是"尺寸公差默认 ISO 2768-m (中等)"，不是 2768-f。
请修正，并说明这条错误可能影响我们之前哪些输出，列出来。

最后那一句"可能影响我们之前哪些输出"是我后来加上的。有一次它记错了齿轮箱的减速比默认值，我让它倒查，它翻出三份报告里的传动计算都用了那个错值。那三份报告我全部重算了一遍。

情况三：记太多反而乱

症状：它开始答非所问，回答里混进别的项目的术语，或者一件事给你两套互相矛盾的做法。

原因：记忆里堆了互相打架的条目。最常见的是同一个规矩写了两遍，一个是三个月前的旧版，一个是新版，两条都在，它每次随机认一条。我在 2025 年底就吃过这个亏：报告表头我前后说过 2022 版和 2025 版，两条都在记忆里，它在这个项目用新版，在那个项目用旧版，我一度以为是软件抽风。

排查动作：让它自查冲突。

检查当前所有记忆条目，找出互相矛盾或重复描述的条目，成对列出来。
每条说明：矛盾点在哪，你认为该留哪一条，理由是什么。先不要删，我来判断。

修复话术：

保留 2025 版，删除所有提到 2022 版表头的条目。
以后遇到同一条规矩有新旧两版，直接删旧版，不要保留两版让我选。
删完把最终条目回显一遍。

三种失效放在一起看：

	记了不用	记错了	记太多反而乱
你会看到	它完全不按规矩来	它按错的规矩来	它一会儿这样一会儿那样
根因	层级不对，或写得太模糊	归纳时把内容改了半个字	新旧两版共存

	记了不用	记错了	记太多反而乱
第一步排查	让它报出这条规矩的原文	让它列出所有含数字和代号的条目	让它自查矛盾条目
修复动作	提到用户级，把话说死	给正确原文，并倒查历史输出	删旧版，定一条"不留两版"的规矩
预防	写完当场回显	每月抽查一次	每次改规矩先删旧的

5.5 一次把新项目背景交代清楚 ★★★

这一节是本章的全部目的。前面四节都是零件，这一节组装。

我拿 PJ-2318 这个真实项目走一遍完整流程，客户和供应商全部脱敏。你照着这个结构，把你的项目填进去，四十分钟能做完，之后这个项目的所有对话都不用再介绍背景。



图 5-4 一次交代清楚的四步：写卡、喂入、确认、验收

图 5-4 项目背景从写到验的四步。作者根据公开资料绘制。

第一步：写一页背景卡

不要边聊边说。先拿十分钟，把背景写在一页纸（或一段文字）上。我用的模板在本章末尾的"本章交付物"里，这里先给你看我填好的样子：

项目背景卡: PJ-2318

【项目】充电口盖执行器，用于驱动某欧系平台车型的充电口盖开闭

【代号】对外文件一律写“某欧系平台”，不写客户真名

【阶段】DV 阶段，2026 年 3 月进 PV

【结构】12 V 永磁直流微电机 + 两级行星齿轮箱 + 霍尔位置反馈

【团队】我（研发经理）+ 2 名设计 + 1 名试验；客户接口人走项目经理

【供应商】A 厂供齿轮箱，B 厂供电机

【文件规则】

- 图纸命名 PJ2318-XX-XXX
- 2D 交付 PDF，三维源文件 .CATPart
- 报告表头照公司 2025 版模板
- 尺寸公差默认 ISO 2768-m，关键尺寸单独标注

【审批规则】

- 图纸三级审：设计自审 → 主管审 → 项目审
- 超差放行必须有项目经理书面同意
- 对外文件发出前，客户名必须复核一遍

【常用术语】

- DV：设计验证；PV：生产验证
- 执行器 = 微电机 + 减速机构 + 反馈元件的总成

【参考资料的位置】

- 公差标准：标准/ISO2768应用说明.pdf
- 2025 版表头模板：模板/技术报告表头_2025.docx

这一页我写完用了大概十分钟。它的价值在于：写的过程本身就是一次梳理。我写到【审批规则】的时候才发现，我从来没跟团队说清楚“超差放行要书面同意”到底指邮件还是签字件。当天中午我问了项目经理，答案是邮件，抄送质量。这条后来也写进了卡片。

第二步：一次性喂进去

卡片写好了，用一段提示词喂进去，一次讲完，不要分五次说。

我要开始一个新项目，下面是完整背景。请你先把这些内容写进当前工作区的记忆文件，放在“项目 PJ-2318”这一节下，不要覆盖已有内容。

有一个要求先说清楚：写完之后，你不要开始做任何任务，只做两件事，一是回显这一节的完整原文，二是回答我下面列的三个确认问题。我点头之后我们再开工。

背景卡内容如下：

(这里粘贴第一步那张完整的背景卡)

确认问题：

1. 上面这张卡里，哪几条属于“跟项目无关、以后所有项目都适用”的规矩？列出来，我会让你把这几条另外写进用户级记忆。
2. 上面这张卡里，有哪些地方你不确定、或者觉得我写得有歧义？逐条提出来问我。
3. 如果现在让你写一份给客户的技术说明，你会把客户名写成什么？把那一句话写出来给我看。

这段提示词里有三个设计，都是踩坑踩出来的。

“先不要开始做任何任务”。不给这句，它读完背景会顺着往下猜你要干什么，然后自作主张开始生成文件。我第一次喂背景的时候就是这样，它二十秒后给我写了三页项目计划，而我要的是它先记住。

确认问题 1 是分层的。让它自己把通用规矩挑出来，比我自己判断准。我原以为只有“客户写代号”是通用的，它挑出了四条，还包括“数据要带单位”，而那条确实是我所有项目都在用的。

确认问题 3 是验收。“你会把客户名写成什么”是一个具体的、有唯一正确答案的问题。它答对了，说明那条规矩真的进去了；它答“某欧系平台客户”或者把真名写出来，说明没进去或者进岔了。不要问“你记住了吗”，它永远回答记住了。

第三步：看它的确认回复，逐条对

它给的回复应当长这样。这是我那次的实际回复，删掉了两句废话：

已写入工作区记忆文件，"项目 PJ-2318" 一节。回显原文如下：## 项目 PJ-2318 - 项目：充电口盖执行器，驱动某欧系平台车型充电口盖关闭 - 代号规则：对外文件一律写"某欧系平台"，禁止出现客户真名 - 阶段：DV 阶段，2026 年 3 月进 PV - 结构：12 V 永磁直流微电机 + 两级行星齿轮箱 + 霍尔位置反馈 - 团队：研发经理 1 名、设计 2 名、试验 1 名；客户接口经项目经理 - 供应商代号：A 厂 = 齿轮箱，B 厂 = 电机 - 文件规则：图纸命名 PJ2318-XX-XXX；2D 交付 PDF，三维源文件 .CATPart - 报告：表头照 2025 版模板；尺寸公差默认 ISO 2768-m，关键尺寸单独标注 - 审批：图纸三级审（设计自审 / 主管审 / 项目审）；超差放行需项目经理书面同意；对外文件发出前复核客户名 - 术语：DV = 设计验证，PV = 生产验证；执行器 = 微电机 + 减速机构 + 反馈元件总成 - 参考文件位置：标准/ISO2768应用说明.pdf、模板/技术报告表头_2025.docx 回答你的三个问题：1. 属于通用规矩的四条：客户名写代号；报告表头照最新版模板（版本号会变，建议写成"照当前最新版"）；数据必须带单位；对外文件发出前复核客户名。这四条建议你另外写入用户级记忆。2. 有两点想跟你确认：- "书面同意"具体指邮件还是签字件？两者在流程上不一样。- "关键尺寸单独标注"，关键尺寸由谁定？是你在图纸上标，还是有一份关键特性清单？3. 我会写成："本项目为某欧系平台开发的充电口盖执行器。"

这份回复我要做的动作只有三个：

动作一，逐字对原文。我拿它回显的九条，跟我卡片上的九条一条一条对。那天对下来只有一处措辞不一样：我写的是"客户接口人走项目经理"，它写成"客户接口经项目经理"，意思一样，我放过了。真正的硬伤它自己没有，是我在第二步的确认问题里逼出来的。

动作二，回答它反问的两点。它问的两点都是真问题。"书面同意"我当天中午问清楚了，是邮件抄送质量，我让它改；"关键尺寸由谁定"我答了"有一份关键特性清单，清单在 项目/关键特性清单.xlsx，需要时先读"。这两问的价值超过我那张卡片本身，因为它问的正是我自己都没想清楚的地方。

动作三，把那四条通用规矩提到用户级。回复里它对第一条的判断是对的，而且它建议把"2025 版模板"改成"照当前最新版"，这个建议我采纳了。版本号会变，写死的数字迟早是坑。

第 1 条里你列的四条通用规矩，写入用户级记忆，原文按下面这版：

- 任何对外或可能外发的文件，客户名一律用代号，代号查当前项目记忆，查不到就问我
- 技术报告表头照公司当前最新版模板，不要写死年份
- 所有数据必须带单位，角度和扭矩也不例外
- 对外文件发出前，复核一遍客户名是否误写真名

另外，PJ-2318 那一节做两处修改：

- "超差放行需项目经理书面同意"改为"超差放行需项目经理邮件同意，邮件抄送质量"
- 术语部分补一句：关键尺寸以 `项目/关键特性清单.xlsx` 为准，需要时先读该文件

改完，把用户级记忆那四条和 PJ-2318 那两处，一起回显给我。

第四步：下次开新对话，验收它真的记住了

喂完不验收，等于没喂。验收动作只有一句，开一个新对话窗口，问它三个必须用记忆才能答对的问题。不要问"你还记得 PJ-2318 吗"，这种是非题它永远答是。

新的对话，三个问题，每个答案不超过两行，只回答，不要开始任何任务：

1. PJ-2318 是什么零件，用在哪，现在什么阶段？
2. 我要出一份 2D 图纸，文件名该怎么命名，交付什么格式？
3. 有一处尺寸超差了，放行需要什么？

答完之后，告诉我你的每道题的答案分别来自哪一层记忆（用户级 / 工作区 / 你现猜的）。

最后那一句“分别来自哪一层”是我加上去的，很管用。有一次它第三题答对了，但标注是“你现猜的”，我一看就知道那条根本没进去，只是它根据常识蒙对了。蒙对一次不算记住。

三个答案我核对的标准：

- 第 1 题必须答出“充电口盖执行器”“某欧系平台”“DV 阶段，3 月进 PV”。少一个就回去补。
- 第 2 题必须答出“PJ2318-XX-XXX”和“PDF”。它如果答成“按公司命名规则”，说明没记住，回去把话说死。
- 第 3 题必须答出“项目经理邮件同意，抄送质量”。只答“项目经理同意”不算过，漏了抄送质量。

三个全对，这个项目的背景才算交代完了。从写卡到验完，我那次用了 42 分钟。

这套动作后来替我省了什么

2026 年 6 月，PJ-2318 进 PV 阶段，客户那边临时要一份英文版的技术说明和一份超差清单。两份东西都要得急，当天下午就要。

我开了个新对话，说了一句：

用 PJ-2318 的项目背景，写一份英文版技术说明，客户名按代号规则写，格式照报告表头。另外把 DV 阶段的超差项列成一张 xlsx，标出哪些已经放行、放行的依据是什么。

它从工作区记忆里读了项目背景，从用户级记忆里读了代号规则和格式规矩，从参考文件位置里读了表头模板，然后开干。中间问过我一次：“超差清单里第 14 项在记忆里没有放行记录，这份清单里要怎么标？”

四十分钟，两份东西出来了。技术说明 6 页，超差清单 31 行，其中 28 行有放行记录，剩下 3 行标了“待补放行依据”。

我核对了那 3 行，确实没有书面放行。这三行最后走的是另一条流程。

这次如果按 3 月之前的做法，我大概要花的时间是：重新介绍背景 3 分钟，来回确认格式 10 分钟，客户名返工 1 次，一共 50 分钟往上，而且那份技术说明大概率还会带着客户真名出去。42 分钟的投入，在这一次就回本了。它后面还在继续回本。

这一节的翻车清单

- 一次说不完就分次说，结果前后矛盾。一条规矩分三次说，三次说法不一样，记忆里就有三条。要么一次给全，要么每次改之前先删旧的。

- 喂完立刻开工。它还没写稳你就开始派活，前面几轮的输出会用旧规矩。喂完先回显、先对、再开工。
- 验收只问"记住了吗"。是非题没有信息量。要问需要调用记忆才能答的具体问题，而且要它标注答案来自哪一层。

5.6 隐私与安全：什么绝对不要写进记忆

这一节请当作硬性规定读。

记忆有个特性，很多人没意识到：写进记忆的东西，会在之后的每一次对话里被调出来用，而且它可能随你的账号同步。你在 PJ-2318 项目里写的东西，有可能在你给另一个客户做方案时被想起来。这跟把文件存在本地文件夹不是一回事。

一句话记住这一节：写进记忆等于交出去。

记忆内容的四级分级

写之前先分级，L0 和 L1 一律不写

-  **L0 绝对不写**
密码、密钥、令牌、验证码、个人隐私。一次都不写，看到就删。
-  **L1 不写**
客户真实名单、真实报价与成本、未公开产品数据、未发布的项目节点。要用就当场贴进对话，用完不管。
-  **L2 脱敏后写**
换成代号或量级再写：某欧系平台、A 厂、预算在中位数区间、2026 年 Q2。
-  **L3 直接写**
身份、岗位、行业、格式规范、命名规则、审批流程、术语偏好。放心写，这些正是它最该记的。

第 5.6 节：写进记忆等于交出去，它可能随你的账号同步

图 5-5 隐私分级：写之前先分级，L0 和 L1 一律不写

图 5-5 记忆内容的四级分级清单。作者根据公开资料绘制。

级别	内容	例子	处理
L0 绝对不写	密码、密钥、令牌、验证码、个人隐私	各类账号口令、服务器密钥、身份证号、他人手机号	一次都不写，看到就删

级别	内容	例子	处理
L1 不写	客户真实名单、真实报价与成本、未公开产品数据、未发布的项目节点	给某客户的实际单价、未发布的车型年款规划、尚未公开的性能指标	不写。要用就当场贴进对话，用完不管
L2 脱敏后写	客户与供应商身份、价格量级、项目时间	"某欧系平台""A 厂""B 厂""预算在中位数区间""2026 年 Q2"	换成代号或量级再写
L3 直接写	身份、岗位、行业、格式规范、命名规则、审批流程、术语偏好	"研发工程师，做微电机与智能执行器""图纸出 PDF""超差放行要邮件"	放心写，这些是它最该记的

L0 那一行不需要解释。密码进了记忆，就是把你家钥匙交出去。

L1 是最容易犯的。我在写这一节之前，自己记忆里就躺着一条"给 A 厂的齿轮箱目标价区间"。它当时是好记，但它的风险是：我在跟 B 厂谈的时候，如果问到同类零件，它有可能把 A 厂那个数想起来，混进给 B 厂的材料。这种错一旦发生，不是返工的问题。我现在处理价格信息的方式是：当场贴进对话，用完就完，绝不进记忆。

L2 的脱敏有个具体做法，我一直在用：

- 客户名一律换代号，"某欧系平台""某日系平台""A 厂""B 厂"，代号和实际名的对照表自己留着，别给 AI。
- 价格换成量级，"单件成本在两位数区间""报价比上一版低一个量级"，够它理解语境，不够它出事。
- 时间换成季度，"2026 年 Q2 进 PV"，比精确日期安全，而且对它的理解和精确日期一样。

L3 是记忆该装的东西。你会发现，这一整章讲的技术规范、格式规矩、审批流程，全在 L3。真正让 AI 好用的信息，恰好是最不敏感的那一类。这是个好消息：你不需要拿机密去换效率。

最后给一条检查动作。每个季度做一次，五分钟：

把当前所有记忆条目列一遍，逐条检查有没有属于下面任一类的内容：

密码、密钥、令牌、个人身份信息、客户真名、具体价格、未公开的产品数据。

有的话列出来，标出条目编号，等我确认后删除。不要自行删除，先报告。

5.7 如果你只做一件事

如果这一章你只带走一样东西，就带这个：

今天下班前，给你手上最主要的那个项目，写一页背景卡，喂进去，然后开个新对话验收三个问题。

四十分钟。做完之后，这个项目上你跟 AI 说的每一句话，都不用再从"我们是做什么的"开始。

至于三层记忆的区分、该写什么不该写什么、失效怎么排查，那些是这个动作做完之后，你自然会回来查的东西。先有卡片，再谈优化。

小白词典

记忆：AI 工具把你交代过的背景、偏好、规矩存下来，下次对话直接调用的能力。网页聊天框换一个窗口就忘，WorkBuddy 分三层存。

云端画像：跟着你的账号走的那层记忆，存你的身份、岗位、行业、语言偏好。换电脑换项目都还在。

用户级记忆：你这个账号下的通用工作习惯，比如"表格交付xlsx""客户写代号"。跨项目生效。

工作区记忆：跟着某个具体文件夹或项目走的那层记忆，存项目背景、命名规则、审批流程。换个工作区就读不到。

记忆文件：存工作区记忆的那个文件，通常放在工作区根目录下。你可以让它读、让它追加，也可以自己改。

Prompt（提示词）：你对 AI 说的那段完整的话。本书所有提示词都放在代码块里，可以整段复制。

脱敏：把真实的名字和数值换成代号或量级。"某欧系平台"是客户名脱敏，"两位数区间"是价格脱敏。

回显：让它把它刚写进去的内容原文再打一遍给你看。这是验收记忆的唯一可靠动作。

幻觉：AI 把不知道的东西说得煞有介事。记忆里记错的标准号，属于幻觉的一种。第 17 章专讲。

你的岗位怎么用

设计工程师：把你的图纸命名规则、默认公差标准、常用材料代号写进用户级记忆。再给每个在跑的项目建一份工作区背景卡。这两件事做完，你让它出图纸和 BOM 时，开头那三分钟的背景交代全省了。

试验工程师：把试验报告的模板版本、测点命名规则、数据单位规矩写进去。NVH 和台架试验的通道命名各家长得都不一样，写死了它才不会给你混。

质量工程师：把"超差放行的审批链"和"客户代号规则"写进用户级，这两条最容易被漏说，也最容易出事。写完之后按 5.5 第四步的问题 3 验一次。

采购 / 供应商管理：供应商一律代号化（A 厂 B 厂），价格一律量级化，绝不进记忆。可以进记忆的是你的比价表格式、资质核查的必查项清单、证书有效期的提醒规则。

项目经理：你手上项目最多，工作区记忆对你价值最大。给每个项目一张背景卡，卡里固定包含阶段、节点、接口人、审批链四项。这四项是它替你写周报和风险表时要反复用到的。

研发主管：把团队的通用规矩（文件格式、命名、审批层级）写成一份用户级记忆，让全组共用同一份。新同事入职那天照着这份喂一遍，比让他看两周文档快。

本章交付物

一页项目背景卡模板。下面这段可以直接复制，把括号里的内容换成你的项目。

项目背景卡：【项目代号】

- 【项目】这是什么零件/产品，用在哪
- 【代号】对外文件里客户怎么写（脱敏代号，不写真名）
- 【阶段】当前阶段，下一个节点在什么时候
- 【结构/组成】主要组成部分，一行写完
- 【团队】几个人，各自负责什么，对外的接口人是谁
- 【供应商】有哪些供应商，各自供什么，用什么代号
- 【文件规则】
 - 命名规则（给一个完整的例子）
 - 交付格式（2D / 3D / 报告各是什么格式）
 - 报告模板版本
 - 默认公差或默认标准
- 【审批规则】
 - 图纸几级审，谁审
 - 超差放行要谁的什么形式的同意
 - 对外文件发出前必须复核哪几项
- 【常用术语】列出 3-6 个，每个一句话
- 【参考资料的位置】标准、模板、清单各自的文件路径

怎么做：

1. 花十分钟填完这十项。填不出来的一项不要空着，写"待定"，然后当天去问清楚。我填 PJ-2318 那次，待定的是"书面同意的形式"，问完之后补上的是"邮件，抄送质量"。
2. 用 5.5 第二步那段提示词，整段喂进去。一次讲完，不要分次。
3. 拿它回显的原文，跟你填的卡逐条对。数字、代号、日期一个一个对。
4. 回答它反问你的问题。它问的地方通常是你自己没想清楚的地方。
5. 开一个新对话，用 5.5 第四步那三个问题验收。三个全对才算完。

我的经验：这张卡填完的最大收益不是省时间，是逼我把规矩想清楚。我做了十一年研发，第一次发现自己从来没跟团队讲清"超差放行的书面形式是什么"。这件事是 AI 问出来的。卡片填完三个月后我回头看，它替我省下的时间大概每小时一次对话三分钟，真正值钱的是它把那些含糊的规矩变成了白纸黑字，全组都能照着用。

本章金句

换一个对话窗口它就不认识你了。不是它记性差，是你从来没让它记过。

云端画像是"我是谁"，用户级记忆是"我怎么干活"，工作区记忆是"这个活是怎么回事"。

判断该不该写进记忆，只看一条：六个月后你还用不用。

不要问它"你记住了吗"。要问一个必须调用记忆才能答对的具体问题。

写进记忆等于交出去。密码不写，价格不写，客户真名不写。

蒙对一次不算记住。让它标注答案来自哪一层，现猜的那一层等于没有。

（下一章：子代理。当一件事大到一个 AI 干不利索，就把它拆成几个各有专长的分身，你当那个派活的人。）

第 6 章

一个干不完，就拆开分给几个

一个 AI 干到发糊时，先别换更强的，先把活拆开分给几个。

我第一次把一份约 200 页的某行业标准整份交给 AI，是想让它直接做成设计检查清单。我说得很痛快："把所有要求提出来，分类，补上风险，再做一份汇报。"它开头干得像模像样，列出一批检查项。往后翻，我发现同一条温度要求出现了两个版本，一处写"必须验证"，另一处写"建议关注"；到了标准后半段，它开始把连续几页内容压成一句"其余要求类似"。

我追问：

我：后半部分的一项验证条件为什么没进检查表？ AI：可能在归纳时被合并了，我可以补充。我：那前面还有多少条被合并了？ AI：无法仅根据当前结果准确判断。

这句话让我停下来了。它已经忘了自己前面怎么取舍，我也没法从成品倒查遗漏。那次我没有继续催它"认真一点"，而是把活改成几份：两名临时助手分段摘要求，一名助手专查漏项和重复，一名助手做风险表，最后由主代理汇总。文件还是那份文件，任务还是那件任务，结果却第一次变得能查、能改、能验。

这一章讲的就是这套方法：什么时候该拆，按什么拆，怎么把每一份活交清楚，以及几个子代理交回来的东西怎样合成一件完整交付物。

6.1 子代理是什么：给主代理配几名临时助手

子代理可以先理解成主代理临时叫来的助手。主代理接住你的总任务，负责分工、收件和汇总；每个子代理只拿到其中一块，按自己的完成标准交回结果。

它很像项目组长带几名专员。组长不会让每个人都从第一页看到最后一页，再各写一份总报告。他会让甲核条款，乙查风险，丙做汇报，最后自己收口。也像总装线上的工位：每个工位动作有限、输入明确、检查方法固定，整条线才不会越干越乱。

子代理通常是临时的。任务结束，它的工作就结束。它不需要知道你全部项目经历，也不需要参加每一次讨论。它只需要拿到完成当前小任务所需的材料、规则和交付格式。

比较项	一个代理干到底	拆给多个子代理
注意范围	同时记目标、材料、格式、风险，越往后负担越重	每个只盯一块，范围较小
处理方式	从头做到尾，中途出错会带到成品	分块处理，单块可以重做

比较项	一个代理干到底	拆给多个子代理
查错难度	只能从总结果倒查，难找漏在哪	能追到具体子任务和来源
速度	大多按顺序推进	独立小块可以同时推进
统一性	一个人写，口吻通常统一	需要预先规定字段和口径
适合任务	小、短、一步能验	大、可分、有多种交付物

主代理：分工、收件、汇总

每个子代理只负责一块有边界、能单独验收的工作



第 6.1 节：主代理始终做三件事，发任务、验中间件、组装成品

图 6-1 主代理分工、子代理收件、主代理验收的基本结构

图 6-1 主代理分工、子代理收件、主代理验收的基本结构，作者根据公开资料绘制。

多叫几个子代理，并不会自动得到更好的结果。项目组多招几个人，也可能只是多开几场会。真正有效的地方在于：每个子代理只负责一块有边界、能单独验收的工作。

这里还有一个容易误会的点。拆分不等于所有工作都同时做。有些子任务能并行，比如把四份互不依赖的文件分别摘要；有些必须接力，比如先提取要求，再根据要求做风险表。子代理解决的是分工问题，并行只是其中一种安排。

6.2 什么活值得拆：先看三件事

我判断一个任务要不要拆，只看三个问题：各块能不能分开做，边界能不能写清楚，单块能不能独立验收。三个答案大多为“能”，就值得拆。只占一个，先让一个代理做更省事。

判断	特征	工程场景	建议
值得拆	文件之间相对独立	逐份审阅 8 份供应商技术资料，统一输出缺项表	按文件拆，统一字段后并行
值得拆	步骤之间交付清楚	先从规范提要求，再做检查表，最后做汇报	按步骤接力，每步验完再交下步
值得拆	同一材料需要多种视角	同一份设计说明分别做需求、质量和试验审查	按角色拆，最后处理分歧
值得拆	单块可以独立复核	每个章节都能用页码和条款号回查	给每块规定来源字段
不值得拆	前后强依赖	修改一个控制参数后，后续计算全部要重算	交给一个代理连续处理
不值得拆	必须保持同一口径	给客户写一页正式结论，措辞要由一人负责	一人主写，另一人只校核
不值得拆	总量很小	一页会议纪要提 5 个行动项	直接做，拆分反而增加交接
不值得拆	无法单块验收	"大家分别看看有没有问题"，没有检查项	先定义检查方法，再决定拆法

可并行，指两块活互相不等结果。甲在审 A 文件时，乙可以同时审 B 文件。边界清晰，指一句话能说清谁做哪一段、哪些内容不归他。单块独立可验，指不等总报告出来，你已经能判断这块合不合格。

举个我常见的反例。一个项目经理把"分析项目为什么延期"拆给三名子代理，每个人都看全部会议纪要，也都写原因、责任和建议。结果三份报告内容重叠，结论还互相冲突。这不叫分工，只是把同一道题复印了三份。

改法很简单：甲只建立计划变更时间线，乙只统计输入物延期记录，丙只核对风险登记和措施状态。三人的证据能分别验，主代理再根据证据写结论。这样才值得拆。

6.3 怎么拆：文件、步骤、角色三种刀法

拆活没有万能模板。我最常用三种刀法：按文件拆，按步骤拆，按角色拆。先找材料的自然边界；找不到，再看流程；流程也揉在一起，就让不同角色从各自角度审同一份材料。

三种拆法：文件、步骤、角色

先找材料的自然边界，找不到再看流程，都不行就换角色审同一份材料

按文件拆：一人一摞

材料彼此独立时最快也最稳。每条记录保留源文件与页码原文，天然不抢活，最容易并行。

按步骤拆：一环接一环

后一步必须吃前一步的结果。关键是交接验收，主代理先验再放行下一站。

按角色拆：多双眼睛看

同一份材料换几双眼睛。最有用的是写明不在范围，角色边界越具体重复越少。

第 6.3 节：实际的大任务常混用三种，先按文件吃完材料，再按步骤生成交付物

图 6-2 三种拆法：按文件并排、按步骤接力、按角色会审

图 6-2 三种拆法：按文件并排、按步骤接力、按角色会审，作者根据公开资料绘制。

6.3.1 按文件拆：一人一摞，最快也最稳

有一次我要整理一个项目文件夹，里面是若干份需求说明、试验记录和问题清单。文件彼此独立，目标都是提取“来源、要求、当前状态、待确认项”。这种活最适合按文件拆。

我会拆成下面几份子任务：

1. 子任务 A：只读需求类文件，提取带来源的要求清单。
2. 子任务 B：只读试验类文件，提取条件、结果和异常。
3. 子任务 C：只读问题类文件，提取责任人字段、计划日期和当前状态。
4. 主代理：统一三个表的字段名，检查重复项，生成项目总览。

按文件拆的优点是天然不抢活。A 不碰试验记录，B 不解释问题责任。它也最容易并行，因为每个人都不等别人。

风险也很明确：不同文件可能说的是同一件事。解决办法是给每条记录保留“源文件 + 页码或位置 + 原文摘录”。合并时看到两条相似内容，主代理能回源判断是重复，还是同一要求在不同阶段的变化。

6.3.2 按步骤拆：上一站交件，下一站开工

我做失效分析材料时，常用按步骤拆。原始输入很杂，后一步必须吃前一步的结果，硬并行只会让每个人各猜一套前提。

子任务清单可以这样排：

1. 子任务 A：整理现象、发生条件和已知事实，不下原因结论。
2. 子任务 B：读取 A 的事实表，列出待验证的原因假设和所需证据。
3. 子任务 C：读取事实表与验证结果，形成原因树和排除记录。
4. 子任务 D：读取已确认结论，生成汇报材料和后续行动清单。

这里的关键动作是"交接验收"。A 的事实表如果把推测混进事实，B 会沿着错误方向设计验证；B 漏了关键证据，C 的原因树再漂亮也站不住。

所以每一步结束，我都让主代理先验，再放行下一步。按步骤拆通常不会让总时间缩短一半，但它能把错误挡在前一站，不让错误一路滚到最终报告。

6.3.3 按角色拆：同一份材料，换几双眼睛看

设计评审适合按角色拆。同一份方案，需求人员看功能有没有漏，质量人员看失效和控制，试验人员看条件能不能执行，项目人员看日期和依赖。材料相同，问题清单却不同。

我会这样分：

1. 需求审查员：只查需求是否完整、可验证、来源明确。
2. 质量审查员：只查风险项、预防措施和检出方法是否对应。
3. 试验审查员：只查样件、设备、边界条件和判定值是否齐全。
4. 项目审查员：只查输入、输出、责任和计划之间是否有冲突。
5. 主代理：合并问题，标记冲突，不替专业人员擅自裁决。

按角色拆最容易出现"互相打架"。例如质量审查员希望增加验证项目，项目审查员认为当前节点来不及。主代理不应自行选边，它要把分歧写成一条待决策项：方案、证据、影响、需要谁决定。

实际的大任务常会混用三种拆法。先按文件把材料吃完，再按步骤生成交付物，关键节点再按角色复核。后面的完整演示就是这种组合。

6.4 委派的话术：每份活都要带完成标准

我早期派子代理，只写一句"请审一下这份文件"。它确实审了，但我不知道它审了什么，也不知道怎样算审完。后来我固定使用七个部分：任务、输入、边界、输出格式、完成标准、禁止动作、回报方式。

完成标准是最容易漏的一项。"输出一个表"只是产出形式，"每条记录都有来源位置，无法确认的内容进入待确认列，检查完指定范围后报告覆盖情况"才叫完成标准。

示例一：按文件拆的委派提示词

你是文件审阅子代理 A。

任务：审阅文件夹“[需求资料路径]”中的全部 PDF 文件，提取可验证的技术要求。

输入范围：只处理该文件夹中的 PDF；不要读取同级其他文件夹。

工作边界：

1. 只提取原文明明确提出的要求，不补写文件中没有的要求。
2. 不做风险评级，不给设计方案，不改源文件。
3. 遇到扫描不清、表格断裂或上下文不足时，标记为“待人工确认”。

输出：生成“子任务 A_需求提取.xlsx”，字段固定为：记录 ID、源文件、页码、章节、原文摘录、要求对象、要求动作、验证线索、待确认项。

完成标准：

1. 指定文件夹内每个 PDF 都有处理状态。
2. 每条要求都能通过源文件和页码回查。
3. 空白字段写“未提供”，不能自行猜测。
4. 文件末尾附处理统计：已处理文件、未处理文件及原因、待人工确认项数量。

完成后只汇报：输出文件路径、处理过的文件清单、未处理项和需要主代理决定的问题。

第一句给它岗位，限制它用什么眼光看。输入范围负责关门，避免它顺手翻到旧版本。工作边界写清“做什么”和“不做什么”，能直接减少重复劳动。

输出字段决定以后能不能合并。完成标准中的“每个文件有处理状态”防止它只挑容易读的文件；“能回查”防止它给出无来源结论；最后的汇报方式让主代理能快速验收，不必再问一轮。

示例二：按步骤拆的委派提示词

你是流程第 2 步的风险分析子代理。

前置输入：

- “[上一步要求清单路径]”
- “[项目边界说明路径]”

任务：根据已经确认的要求清单，生成风险初筛表。

工作边界：

1. 只能引用前置输入中的要求，不新增要求。
2. 每个风险必须关联至少一个要求记录 ID。
3. 证据不足时写“待评审”，不要替项目团队下最终结论。

输出：生成“第2步_风险初筛.xlsx”，字段固定为：风险 ID、要求记录 ID、偏离情形、可能影响、现有控制、缺口、建议检查动作、状态。

完成标准：

1. 要求清单中的每条有效记录都有风险处理状态。
2. 同一风险关联多条要求时保留全部记录 ID。
3. “现有控制”与“建议检查动作”分列，不得混写。
4. 输出一张待评审清单，列出所有证据不足项。

禁止修改前置输入文件。完成后汇报输出路径、覆盖情况、待评审项和发现的输入问题。

这个提示词最重要的一句是“只能引用前置输入中的要求”。它把第二步锁在第一步的已确认结果上，防止风险子代理另外发明一套需求。

“每条有效记录都有风险处理状态”则解决漏项。没有风险也要写“未识别到，待抽查”，不能让空白同时表示“没风险”和“忘了看”。

示例三：按角色拆的委派提示词

你是试验可执行性审查子代理。

审查材料：“[试验方案路径]”

参考材料：“[需求清单路径]”

任务：只从试验执行角度审查方案，找出会导致无法开试、无法复现或无法判定的问题。

请逐项检查：样件状态、设备能力、安装条件、环境条件、工况步骤、采样设置、判定规则、异常记录方式。

不在你的范围：

1. 不评价项目排期是否合理。
2. 不改需求目标。
3. 不替质量或设计角色给出最终接受结论。

输出：“试验可执行性问题表.xlsx”，字段为：问题 ID、对应章节、问题描述、缺失信息、执行影响、建议补充动作、严重程度建议、需确认角色。

完成标准：

1. 八类检查项逐类给出“通过、存在问题、材料未提供”三种状态之一。
2. 每个问题都指出材料位置。
3. 严重程度只给建议，并写明判断依据。
4. 单独列出会直接阻止试验开始的问题。

完成后汇报输出路径、阻断项、普通问题和需要其他角色回答的事项。

按角色拆时，最有用的句子往往是“不在你的范围”。角色边界写得越具体，最后重复问题越少。八类检查项则相当于一把尺子，能看出它究竟有没有把该看的都看完。

一张合格委派卡的七个部分

任务、输入、边界、输出格式、完成标准、禁止动作、回报方式

- **任务：这块活具体干什么**
第一句给它岗位，限制它用什么眼光看材料
- **输入：处理哪些材料**
指明文件夹与文件名，锁版本，不让他翻到旧版
- **边界：做什么和不做什么**
最有用的句子往往是不在你的范围，这一栏能直接减少重复劳动
- **输出格式：字段先固定**
字段顺序、允许值、ID 规则、空白写法全部统一
- **完成标准：我看哪几样**
每个文件有处理状态，每条记录能回查，空白写未提供
- **禁止动作：不许碰什么**
不改源文件，不补没有的要求，空白不自行猜测
- **回报方式：怎么交差**
汇报输出路径、已处理清单、未处理项和需要主代理决定的问题

第 6.4 节：输出一个表只是产出形式，不算完成标准

图 6-3 一张合格委派卡的七个组成部分

图 6-3 一张合格委派卡的七个组成部分，作者根据公开资料绘制。

6.5 完整演示：约 200 页标准怎样变成三件交付物

下面把章首那件事完整走一遍。输入是一份脱敏后的某行业标准工作副本，篇幅约 200 页。目标有三件：一份可执行检查清单、一份风险表、一份内部汇报材料。

我不会在书里复述标准条款，也不会给它虚构条目。下面展示的是任务结构、提示词原文、空字段示意和验收动作。你换成自己有权使用的文件，就能按同样步骤操作。

第 0 步：先把总任务写成验收单

总任务如果只有“读标准并做报告”，主代理没有分工依据。我先写最终验收条件：

1. 检查清单中的每一项，都带源文件、页码或章节、原文摘录、检查动作和结果栏。
2. 风险表中的每一项，都关联检查项 ID，不出现找不到来源的孤立风险。
3. 汇报材料中的数字和结论，都能回到检查清单或风险表。
4. 标准中读不清、说不准、有冲突的内容，集中进入待确认清单。
5. 三件交付物使用同一套记录 ID，后续修改可以追踪。

接着我让主代理只做计划，不急着读正文：

总任务：把“[某行业标准 PDF 路径]”处理成检查清单、风险表和内部汇报材料。

先不要生成最终交付物。请先完成任务规划：

1. 识别文件的页数、目录结构、正文起止位置和可读取状态。
2. 按页码或章节划分两个不重叠的提取范围。
3. 规划子代理的执行顺序、输入、输出文件和完成标准。
4. 统一记录 ID、字段名、状态值和文件命名。
5. 列出需要我确认的范围问题。

硬性要求：所有结论必须保留来源位置；无法读取或无法判断的内容单列；任何子代理不得改写源 PDF。

只输出分工计划和待确认问题，等我确认后再执行。

这一步要的结果是一张施工图，暂时不求最终答案。我重点看四件事：两个页码范围有没有重叠或空档，目录和正文页码是否一致，输出字段是否相同，后续子代理究竟读取哪个版本。

如果标准前面有封面、目录、修订记录，文件显示页码和印刷页码常常不同。我会要求统一使用“PDF 页序 + 文内章节”双定位。只写“第 35 页”，换一次版本就可能找错；双定位更稳。

第 1 步：两个子代理分段提取要求

这一步按文件内部范围拆。两名子代理可以同时工作，但输出表头必须一模一样。A 负责前半范围，B 负责后半范围。范围边界由第 0 步读出的目录确定，不在提示词里硬猜一个章节号。

给子代理 A 的原文如下：

你是标准要求提取子代理 A。

源文件：“[某行业标准 PDF 路径]”

处理范围：“[填写已确认的前半页序和章节范围]”

任务：逐页提取该范围内可转成检查动作的要求、限制、条件和交付规定。

边界：

1. 只处理指定范围，不进入子代理 B 的范围。
2. 保留原意，不把“建议”改成“必须”，不补充常识性要求。
3. 附录中的表格和注释如果属于本范围，同样检查。
4. 看不清或需要跨范围上下文的内容，放入待确认表。

输出文件：“01_A_前半要求.xlsx”。

主表字段：记录 ID、PDF 页序、文内章节、原文摘录、约束强度、适用对象、触发条件、要求动作、验收证据、备注。

待确认表字段：临时 ID、PDF 页序、文内章节、疑问、需要的上下文。

记录 ID 格式：A-四位顺序号。约束强度只允许“强制、建议、说明、无法判断”。

完成标准：指定范围逐页有处理状态；每条记录有双定位；表格、脚注、附录均有处理说明；不确定内容全部进入待确认表。

完成后汇报：输出路径、已处理范围、未读页、待确认项、需要主代理转交给 B 的跨范围问题。

给子代理 B 的原文只改变身份、范围、文件名和 ID 前缀：

你是标准要求提取子代理 B。

源文件：“[某行业标准 PDF 路径]”

处理范围：“[填写已确认的后半页序和章节范围]”

任务：逐页提取该范围内可转成检查动作的要求、限制、条件和交付规定。

边界：

1. 只处理指定范围，不返回子代理 A 的范围。
2. 保留原意，不把“建议”改成“必须”，不补充常识性要求。
3. 附录中的表格和注释如果属于本范围，同样检查。
4. 看不清或需要跨范围上下文的内容，放入待确认表。

输出文件：“01_B_后半要求.xlsx”。

主表字段：记录 ID、PDF 页序、文内章节、原文摘录、约束强度、适用对象、触发条件、要求动作、验收证据、备注。

待确认表字段：临时 ID、PDF 页序、文内章节、疑问、需要的上下文。

记录 ID 格式：B-四位顺序号。约束强度只允许“强制、建议、说明、无法判断”。

完成标准：指定范围逐页有处理状态；每条记录有双定位；表格、脚注、附录均有处理说明；不确定内容全部进入待确认表。

完成后汇报：输出路径、已处理范围、未读页、待确认项、需要主代理转交给 A 的跨范围问题。

两名子代理各自产出两个工作表：要求主表和待确认表。下面只展示结构，不代替任何真实标准内容：

记录 ID	PDF 页序	文内章节	原文摘录	约束强度	要求动作	验收证据
A-0001	[页序]	[章节]	[源文摘录]	[四选一]	[动作]	[记录或文件]
B-0001	[页序]	[章节]	[源文摘录]	[四选一]	[动作]	[记录或文件]

我验这一站时不会从头重读 200 页，那样就失去意义。我会先查边界页：A 的最后两页和 B 的最前两页；再查目录列出的每一章有没有处理状态；最后从表格、脚注、附录各抽一处回源。

只要出现以下任一情况，我就退回对应子任务：有记录无来源，约束强度超出四个允许值，整页没有处理状态，待确认内容被悄悄改成确定结论。

第 2 步：校核子代理查漏、去重、统一口径

A 和 B 交回来的表不能直接拼。标准前后常有引用、重复说明和术语变化。第三名子代理只负责校核，不负责写最终检查动作。

你是标准提取结果校核子代理。

输入：

- “01_A_前半要求.xlsx”
- “01_B_后半要求.xlsx”
- “[某行业标准 PDF 路径]”

任务：合并两份要求表，检查范围空档、重复记录、来源错误、约束强度误判和跨章节引用。

工作边界：

1. 不删除原记录，不覆盖 A、B 文件。
2. 疑似重复只能标记，不能直接删掉。
3. 需要专业判断的内容进入“待人工裁决”。
4. 只根据源 PDF 校核，不引用外部资料补写要求。

输出：“02_要求总表_已校核.xlsx”。

工作表一“要求总表”：保留原字段，新增统一 ID、校核状态、关联记录 ID、校核说明。

工作表二“覆盖检查”：按页序和章节记录已处理、无要求、无法读取、待确认。

工作表三“待人工裁决”：列出冲突、重复、跨章节依赖和原因。

统一 ID 格式：REQ-四位顺序号。原 A/B 记录 ID 必须保留。

校核状态只允许：通过、需修改、疑似重复、待人工裁决。

完成标准：

1. A、B 每条原记录都有去向。
2. 计划范围内没有未说明的页序空档。
3. 疑似重复项成组列出并保留双方来源。
4. 每个“需修改”都写明修改依据。

完成后汇报输出路径、覆盖空档、疑似重复组、来源问题和待人工裁决项。

这个子代理要交的是一份带审计痕迹的要求总表，短摘要不合格。A-0001 和 B-0001 仍保留，同时获得新的统一 ID。以后风险表、检查清单和汇报材料只引用统一 ID，来源追踪仍能回到原始分段。

我会亲自处理“待人工裁决”。例如前文给出总原则，后文对特殊条件另有说明，子代理可能标成冲突。这个决定涉及适用范围，不能让主代理为追求表面整齐直接删掉一条。

第 3 步：检查清单与风险表并行生成

要求总表验收通过后，两个新子代理可以同时开工。一个负责把要求变成检查动作，一个负责识别偏离情形。它们读取同一份已校核总表，所以不会各自回原标准再解释一遍。

检查清单子代理的提示词：

你是检查清单编辑子代理。

输入：“02_要求总表_已校核.xlsx”

只处理校核状态为“通过”或已经人工确认的记录。

任务：把每条适用要求改写成可执行、可记录结果的检查项。

输出：“03_标准检查清单.xlsx”。

字段：检查项 ID、要求统一 ID、阶段、检查对象、检查动作、所需输入、合格证据、检查结果、问题记录、责任角色、备注。

规则：

1. 一个检查项只能表达一个可判断动作；一条要求含多个动作时拆成多行。
2. “检查动作”必须能由人执行，避免“确保合规”“关注风险”这类空话。
3. 合格证据写清要看什么记录、数据或签字结果。
4. 不替使用者填写检查结果和责任人名。

完成标准：每条输入要求都有关联检查项或“不转检查项”的明确理由；所有检查项能回到要求统一 ID；空白信息写“待项目填写”。

完成后汇报输出路径、检查项覆盖情况、被拆成多项的要求、未转检查项及原因。

风险表子代理的提示词：

你是标准符合性风险子代理。

输入：“02_要求总表_已校核.xlsx”

只处理校核状态为“通过”或已经人工确认的记录。

任务：识别未满足要求时的偏离情形、可能影响和建议检查动作，形成评审用风险表。

输出：“03_标准风险表.xlsx”。

字段：风险 ID、要求统一 ID、偏离情形、可能影响、现有证据、证据缺口、建议检查动作、优先级建议、判断依据、评审状态。

规则：

1. 每个风险必须引用要求统一 ID。
2. 不虚构现有控制；输入未提供时写“待项目确认”。
3. 优先级只给建议，必须附判断依据。
4. 相同偏离情形涉及多条要求时，保留全部关联 ID。

完成标准：所有适用要求都有风险处理状态；所有高优先级建议都有依据；不存在无来源风险；待确认内容单独汇总。

完成后汇报输出路径、覆盖情况、高优先级建议、证据缺口和待评审项。

两份产出的关系要一眼看懂。检查清单回答“我要查什么、拿什么证明”，风险表回答“如果没做到，会出现什么偏离、先查哪里”。它们共享要求统一 ID，但不能把两张表硬挤成一张，日常检查和风险评审的使用场合不同。

主代理合并前做三项交叉检查：风险表里的要求 ID 是否都存在，检查清单是否覆盖所有适用要求，一条要求拆成多个检查动作后是否仍保留同一个来源关系。

第 4 步：汇报子代理只讲已确认结果

最后一名子代理做汇报材料。它不再阅读 200 页原文件，只读取已经验收的要求总表、检查清单、风险表和待确认清单。这样能减少它从原文另起解释的机会。

你是内部评审汇报子代理。

输入：

- “02_要求总表_已校核.xlsx”
- “03_标准检查清单.xlsx”
- “03_标准风险表.xlsx”
- “[人工裁决结果路径]”

任务：生成一份用于内部评审的 Markdown 汇报稿。

结构固定为：

1. 任务范围与版本
2. 材料覆盖情况
3. 检查清单使用方法
4. 主要风险与证据缺口
5. 待确认问题
6. 下一步动作

规则：

1. 所有数量从输入文件统计，不估算。
2. 每个风险结论标注风险 ID 和要求统一 ID。
3. 不展示未确认内容为确定结论。
4. 不复制大段标准原文，只保留定位和必要摘录。
5. 下一步动作必须写明动作、输入、输出和建议角色。

输出：“04_标准导入内部评审稿.md”。

完成标准：汇报中的数量能在输入表中复算；每条主要风险可追到风险表；待确认问题无遗漏；文件中不出现输入材料之外的新结论。

完成后汇报输出路径、引用的统计项、主要风险 ID、待确认项和无法生成的部分。

它交回来的汇报稿只有六部分，没有追求花哨。第一页说范围，第二页说覆盖，第三页教团队怎么用检查表，后面集中讲风险、待确认项和下一步。汇报越接近决策现场，越要少讲过程，多给能回查的结论。

第 5 步：主代理合并，但不替我签字

这时主代理手里有五类产物：A/B 原始提取表、已校核要求总表、检查清单、风险表、汇报稿。合并不是复制粘贴，它要完成四个检查门：

1. 来源门：每条检查项和风险都能沿 ID 回到源文件位置。
2. 覆盖门：计划范围内每页、每章都有处理状态，空档有原因。
3. 口径门：状态值、约束强度、优先级写法只有预先规定的选项。
4. 交付门：文件能打开，表头完整，汇报中的统计能由表格复算。

从标准到三件交付物

约 200 页的输入，拆成流水线之后能查、能改、能验



第 6.5 节：发现一条错误只退回产生它的那个子任务

图 6-4 从约 200 页标准到三件交付物的多代理流水线

图 6-4 从约 200 页标准到三件交付物的多代理流水线，作者根据公开资料绘制。

我自己的验收顺序也固定。先看待确认清单，因为这里最容易藏着真正的项目风险；再抽高优先级风险回源；然后随机抽普通检查项；最后复算汇报数字。发现一条错误时，我退回产生错误的那个子任务，不让所有子代理全部重跑。

比如风险表里出现一个要求总表没有的 ID，我只退回风险子代理，让它检查引用；如果要求总表少了一段页序，我退回校核子代理；如果原始提取就漏了整张表格，才退回负责该范围的 A 或 B。能局部返工，是拆分任务最实在的好处。

这个演示里，主代理始终做三件事：发任务、验中间件、组装成品。它没有把专业裁决偷偷替我做掉。凡是涉及适用范围、风险接受、责任归属的内容，都进待确认表，由我或对应角色决定。

6.6 分工不出乱子的三条规矩

规矩一：边界不能重叠

反例是：A "提取全部要求", B "检查重要要求"。两个范围必然重叠, 而且"重要"没有定义。最后同一条要求会出现两种写法。

正例是：A 负责已确认的前半章节, B 负责后半章节; 或者 A 只提要求, B 只在 A 的结果上查风险。边界要能画成一条线, 不能靠子代理自行体会。

我会在每份委派中写一栏"不在范围"。这几个字看似消极, 实际比"请认真完成"有用得多。它告诉助手在哪停手, 也告诉主代理出现漏项时该找谁。

规矩二：产出格式先统一

反例是：A 交 Word, B 交 Excel; A 用"高、中、低", B 用"红、黄、绿"; A 记录页码, B 只记章节。到合并时, 主代理大半时间花在改表头。

正例是：开工前给所有同类子任务一份空表, 字段顺序、允许值、ID 规则、空白写法全部固定。谁缺字段, 谁返工, 主代理不替它补猜测内容。

格式统一也包括语言口径。标准里的"应""宜""可"如果需要分类, 就先规定分类表。各子代理不能凭自己的理解分别发明状态值。

规矩三：合并前逐个验

反例是：四份结果一回来, 主代理立刻拼总表。等总表出现重复和断链, 你已经不知道错误来自谁。

正例是：每份子任务先过自己的完成标准, 再进入合并区。验收只看这份活该负责的范围, 不用提前评价总报告漂不漂亮。

我常用的顺序是"看回报、查文件、抽来源、验边界"。子代理说处理完 6 份文件, 我先数处理状态; 说没有漏页, 我抽边界页; 说每条能回源, 我点开几条来源。通过后才把文件交给下一个子代理。

6.7 四种常见翻车：症状、原因、补救

多代理最怕一种错觉：窗口里同时跑着几份活, 看起来很忙, 就以为效率一定高。下面四种问题, 我都遇到过。它们不靠"让 AI 更认真"解决, 只靠改分工。

翻车类型	你看到的症状	根本原因	立刻补救	下次预防
重复劳动	两张表里出现大量相同条目, 措辞略有不同	任务边界重叠, 材料范围没锁定	暂停合并, 按来源分组, 指定一份为主记录	每份任务写处理范围和不在范围
互相打架	一个说必须补试验, 另一个说现有证据足够	角色判断依据不同, 主代理又要求直接下结论	保留两方证据, 生成待决策项, 请责任角色裁决	角色只提建议和依据, 不代替最终决定

翻车类型	你看到的症状	根本原因	立刻补救	下次预防
结果对不上	汇报数字、风险表、检查表各有一套数量	每个子代理读了不同版本，统计口径也不同	冻结唯一输入版本，重新从总表计算	规定唯一数据源、版本和状态值
丢上下文	子代理重复询问已确定事项，或沿用旧约定	委派时只给任务，没有给必要约定	补发任务背景卡，明确哪些约定有效	每个任务附最小上下文和决策记录

重复劳动通常最容易发现，因为表会突然变长。别急着删相似行。先看来源位置，相似内容可能是标准在两个适用条件下分别提出的要求。只有来源、对象、条件都相同，才进入疑似重复组。

互相打架也不全是坏事。需求角色和试验角色得出不同建议，可能暴露了真实矛盾。错误做法是让主代理投票。正确动作是把双方依据并排放好，补上影响和待决定人。

结果对不上往往源于版本。A 看的是修订前文件，B 看的是修订后文件，最后再聪明的汇总也救不回来。开工时给输入文件加版本、日期或文件校验信息，所有子代理只认这一个入口。

丢上下文的补救也不是把全部聊天记录塞给每个人。材料越多，子代理越难知道哪句有效。只给最小背景卡：任务目标、已确认决定、术语表、输入版本、禁止改动项。旧讨论不等于现行决定。

6.8 如果你只做一件事

下次碰到大任务，先别数要叫几个子代理。先写一句完成标准：这块活交回来时，我看哪几个东西，就能判断它完成了？

如果你写不出这句话，说明任务还没拆到能验收。继续缩小范围：从“审标准”缩成“提取第 3 章要求”，再缩成“每条要求必须带章节、原文和检查动作”。当完成标准清楚，委派话术、产出格式和验收动作都会跟着清楚。

我现在拆任务的顺序固定为：先写最终交付物，再写每个中间件，接着标出谁生产、谁使用，最后才决定能不能并行。这个顺序能避免为了多代理而多代理。

小白词典

主代理：接住总任务、安排分工、验收中间结果并组装成品的 AI 助手。像项目组长，自己也干活，但更重要的是管住接口。

子代理：主代理为一块具体工作临时安排的 AI 助手。像临时专员，拿到限定材料和完成标准，交件后结束任务。

并行：几块互不等待的活同时进行。像几个人分别清点不同仓位，能一起开工。

串行：后一步必须等前一步结果。像先测尺寸，再根据结果判断是否合格，顺序不能换。

任务边界：一份子任务负责到哪里、在哪里停。像工位黄线，线内归你，线外不要碰。

完成标准：判断一份工作能不能收货的具体条件。像检验规范，不能只写“做得认真”。

唯一数据源：所有人共同认可的那一份现行输入。像受控图纸，旧版本不能混进现场。

可追踪：从结论能找到证据，从证据能回到原文位置。像零件批次号，出了问题能追到来源。

你的岗位怎么用

设计工程师：把多份需求资料按文件交给不同子代理提取，再让校核子代理查重复与冲突。最终检查表必须保留来源、适用条件和验证证据。

试验工程师：按步骤拆成方案完整性检查、数据整理、异常筛查和报告生成。每一步先验输入输出，避免异常数据直接进入结论页。

质量工程师：按角色让要求、风险、控制和证据分别受审。发现结论冲突时，不让主代理投票，改成列证据和待裁决事项。

采购与供应商管理：按供应商或资料包拆分资质检查，所有子代理使用同一张缺项表。合并前先验每包文件的处理状态和有效期字段。

项目经理：按材料类型拆会议纪要、计划变更、风险登记和行动项，再由主代理生成时间线。每条结论要带日期、来源和责任角色。

研发主管：把同一方案分给技术、质量、试验和项目角色会审。要求每个角色只报问题、证据和影响，把最终取舍留在评审会上。

本章交付物

多代理分工卡：一张可以直接复制的"拆活 + 派活 + 合并"模板。每次任务超过一份材料、一个步骤或一个专业视角时，先填这张卡。

【总任务】

要完成的事情：

最终交付物：

最终验收动作：

唯一输入版本：

禁止改动项：

【拆活】

拆法：按文件 / 按步骤 / 按角色 / 混合

子任务 1：

- 负责人：
- 输入：
- 处理范围：
- 不在范围：
- 输出文件：
- 固定字段：
- 完成标准：
- 失败时回报：

子任务 2：

- 负责人：
- 输入：
- 处理范围：
- 不在范围：
- 输出文件：
- 固定字段：
- 完成标准：
- 失败时回报：

【派活前检查】

- ☐ 每块边界能用一句话说清
- ☐ 同类产出使用相同字段和允许值
- ☐ 每个子任务都有来源记录要求
- ☐ 每个子任务都有可执行的完成标准
- ☐ 已标明哪些能并行、哪些必须等待
- ☐ 所有子代理读取同一输入版本

【逐件验收】

- ☐ 输出文件存在且能打开
- ☐ 指定范围全部有处理状态
- ☐ 抽查记录能够回到来源
- ☐ 待确认项没有被写成确定结论
- ☐ 没有越过任务边界
- ☐ 通过后才交给下一步或进入合并

【合并检查】

- ☐ ID 唯一且引用有效
- ☐ 重复项已分组，未直接误删

- [] 状态值和术语写法统一
- [] 汇总数字可以从底表复算
- [] 冲突项保留双方证据和待决定人
- [] 最终交付物逐项通过验收

怎么做：先填最上面的最终交付物和验收动作，再填子任务。填到某一项时，如果"完成标准"只能写成"认真检查"，就继续拆，直到能写出来源、字段、范围和检查动作。

派活后，把每个子代理的回报贴回这张卡。没通过逐件验收的文件，不进入合并区。最终合并时，只使用卡片上登记的正式输出，不从聊天记录里临时捡结论。

作者经验：我先看错误能不能退回到一个小任务，不看一次叫来多少助手。能只重做一段、一个表或一个角色的审查，这次拆分就有价值。凡是出错后只能全部推倒重来，通常说明边界或中间件还没设计好。

本章金句

一个 AI 干到发糊时，先把活拆开，不要继续往同一个窗口里塞要求。

子代理的数量不重要，每份活能不能单独验收才重要。

派活时最值钱的一句话，是"做到什么样才算完成"。

能并行的是材料，不能省略的是验收。

主代理负责组装，专业判断仍由签字的人负责。

好的分工出了错只重做一块，差的分工出了错全部重来。

（下一章：进入第四篇，72 个场景实战。从下一页开始，按岗位挑一个真活，照着步骤交出去。）

PART FOUR · 第四篇

72 个场景实战

第 7 章

通用办公自动化：先把每周那十小时抢回来

办公杂活吃掉的不只是每周十小时，是你被打断十一次之后再怎么也拼不回来的整块时间。

去年 11 月，我连续两周觉得“明明没干什么正事，却天天加班”。我起疑了，就在电脑上开了一张表，把自己一周里花在办公杂活上的时间，一笔一笔记下来。

记录方法很笨：每次我从工程活里切出去做杂活，就在表里加一行，记开始和结束时间。两周之后我把数据摊开，自己吓了一跳。

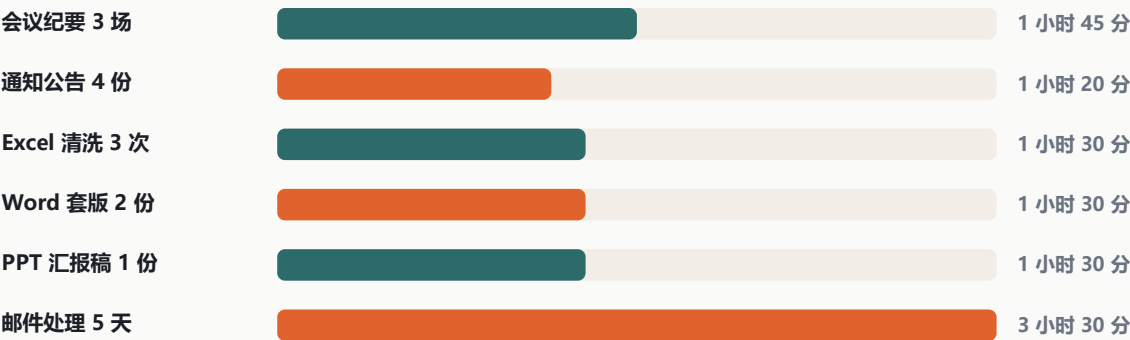
杂活	一周几次	单次人工	一周小计
会议纪要	3 场	35 分钟	1 小时 45 分
通知与公告	4 份	20 分钟	1 小时 20 分
Excel 数据清洗	3 次	30 分钟	1 小时 30 分
Word 报告套版	2 份	45 分钟	1 小时 30 分
PPT 汇报稿	1 份	90 分钟	1 小时 30 分
邮件收发与回复	5 天	42 分钟	3 小时 30 分
合计			11 小时 05 分

11 小时 05 分。按一周五个工作日算，平均每天两小时 13 分钟，全花在了不产生任何工程判断的事上。

更难受的在后面。我把这 11 小时拆开看，真正需要我动脑子、需要我拿工程经验去判断的部分，加起来不到 90 分钟。剩下那九个半小时，我在做的事是：听录音、打字、调格式、复制粘贴、改标点。

一周办公杂活时间分布

按一周五个工作日算，平均每天两小时 13 分钟



合计 11 小时 05 分，其中真要动脑子做判断的部分不到 90 分钟

图 7-1 一周 11 小时办公杂活的分布：邮件最多，纪要第二

图 7-1 一周办公杂活时间分布（作者根据公开资料绘制）

这就是本章的来历。这一章写六个场景，全是我自己工位上跑过的通用办公活。它们跟汽车研发没有半点关系，任何坐办公室的人都遇得到。把这六件交出去，一周能抢回来的时间，我实测在 8 小时到 9 小时之间。

本章六个场景速览

场景	场景名	人工耗时	AI 耗时	可信度
01	会议纪要：从录音转写到待办清单	35 分钟/场	6 分钟/场	★★★ 实测
02	通知与公告：一份模板批量生成 N 个版本	20 分钟/份	3 分钟出 4 份	★★★ 实测
03	Excel 数据清洗：乱表格变干净表	30 分钟/次	4 分钟/次	★★★ 实测
04	Word 报告格式化：照公司模板一键套版	45 分钟/份	8 分钟/份	★★★ 实测
05	PPT：从一份大纲生成可讲的幻灯片	90 分钟/份	15 分钟/份	★★★ 实测
06	邮件：批量处理与回复草稿	42 分钟/天	10 分钟/天	★★★ 实测

六个场景全部标 ★★★，意思是这六个流程我都在自己电脑上完整跑过、留了产物文件、记了耗时。没有一个是"理论上应该能行"。

关于 ★★★ 我多说一句：这不代表你照抄就一定成功。我跑通的环境是 Windows 11 + WorkBuddy 桌面版 + Office 2021。你的软件版本、文件大小、录音质量跟我不一样，耗时会有出入。我给的时间是我本机的时间，你拿它当参考，别当承诺。

还有一个贯穿全章的规矩，先在这里讲清楚：这六个场景里，AI 交出来的东西，没有一件可以不看就发出去。每一节的"过程与判断点"和"验收标准"，才是这一章真正值钱的部分。提示词你可以抄，验收动作你必须自己做。谁签字谁验收，这条在办公桌上跟在车间里一样成立。

场景 01 会议纪要：从录音转写到待办清单 ★★★

1. 一句话背景

一场 60 分钟的项目评审会，开完你要花 35 分钟整理纪要。真正的难点不在听写，在于你得来回拖进度条确认三件事：谁答应了什么、什么时候交、哪些事其实没谈拢。

2. 名词小词典

转写：把录音里说的话变成文字。以前靠人听一句打一句，现在靠程序自动识别，一分钟音频大概几秒钟出字。

说话人分离：程序自动判断"这句话是张三说的，下一句是李四说的"，在文字稿上标出人名。人少、说话不抢话时准，人多抢话时容易混。

时间戳：文字稿上标的时间点，比如 00:12:34。它的用处是让你能一键跳回录音的那一秒去核对。

待办清单（行动项）：会议纪要里最有用那部分。一件事 + 一个责任人 + 一个完成时间，三样齐全才算一条有效待办。

未指定：会上没说清楚谁负责、或者没说什么时候交。这个词必须老老实实写进纪要，不能靠猜补上。

3. 开工前准备

- WorkBuddy 已安装并登录（第 2 章）。
- 会议录音文件一份，格式 m4a、mp3、wav 都行，放在一个你知道路径的文件夹里。
- 会议的基本信息，写在一张便签上：会议主题、日期时间、参会人姓名与角色。
- 一个用于存放产物的空文件夹，建议命名 会议纪要_2026。
- 若会议涉及专业缩写（零件号、标准号、项目代号），把它们列成一张表备用，AI 认不出的词大多在这里。

4. 操作步骤

1. 打开 WorkBuddy，新建对话，把模式切到 Plan。第一次处理某类会议时，先让它出方案给你看。
2. 把录音文件拖进对话框。文件大的话（超过 100 MB）先切成几段，每 30 分钟一段。
3. 把下面第 5 节的提示词整段复制进输入框，把方括号里的内容替换成你自己的会议信息。

4. 点发送。它会先给你一份处理方案（怎么分段、怎么分离说话人、输出几部分）。看完确认，再切到 Agent 模式让它动手。
5. 它干完会向你汇报三件事。先看汇报，再打开产物文件。
6. 打开生成的 Markdown 文件，按第 6 节的三个判断点核对。
7. 核对通过，让它转成 Word 存一份，文件名带日期。
8. 发出去之前，把待办清单单独复制出来，贴进你的项目跟踪表。

关于录音从哪来，多说两句。我用的是手机自带的录音机，开会时把手机放桌子中间，会后导出 m4a 传到电脑。线上会议的话，多数会议平台自带录制和导出，导出的文件直接用。

有一点要注意：手机放桌子中间录，离得远的人声音会小，转写准确率明显下降。人多的会（超过 6 人）我建议用带多个麦克风的录音设备，或者会后自己复述一遍关键结论，录一段 30 秒的语音附在纪要里当参考。



图 7-2 会议纪要流程：录音进，四段式纪要加待办表出

图 7-2 会议纪要处理流程（作者根据公开资料绘制）

5. 可复制提示词

我给你一份会议录音文件（见附件），请把它整理成一份会议纪要。

【会议基本信息】

- 会议主题：[XXX 项目第 3 次设计评审]
- 时间：[2026-09-18 14:00-15:10]
- 参会人：[张工（结构）、李工（试验）、王工（质量）、陈经理（项目）、我（Eric，研发）]
- 我的称呼：[我 / Eric]

【专业词汇表，转写时按这个表订正】

[零件号 XA-2308、标准号 Q/XXX 2025、项目代号 P7]

【输出四部分】

第一部分：会议结论。不超过 5 条，每条一句话，只写会上真正拍板的事。

第二部分：待办清单。用 Markdown 表格输出，四列：序号、待办事项、责任人、完成时间。

责任人和完成时间必须是会上有人明确说出口的，不许你推测。

会上没说责任人的，责任人列写“未指定”；没说完成时间的，完成时间列写“未指定”。

第三部分：争议与未决事项。列出会上没谈拢、需要下次再议的问题，每条附上双方各自的观点，不要调和、不要替他们做判断。

第四部分：风险提示。把会上有人提到但没人接手的隐患单列出来。

【硬性约束】

- 全程用中文，专业术语保留英文原文。
- 所有数字、零件号、日期、标准号必须与录音一致，不许四舍五入、不许换算单位、不许改写。
- 听不清的地方，在对应位置标 [听不清 00:12:34]，绝对不许猜。
- 不许写“会议氛围热烈”“大家一致认为”“与会人员充分交换了意见”这类空话。
- 输出为 Markdown，文件名：会议纪要_[项目名]_[第几次]_[年月日].md，存到桌面。

【干完以后向我汇报三件事】

1. 录音总时长多少，你识别出几个说话人。
2. 待办清单共几条，其中责任人或完成时间写“未指定”的有几条。
3. 你标了几个 [听不清]，分别在几分几秒。

6. 过程与判断点

它会先给你一份方案（Plan 模式），大概长这样：分几段处理、打算输出几部分、有没有需要你补充的信息。你看一眼，重点确认它有没有理解“未指定不许猜”这条约束。

切到 Agent 之后，它会依次做：转写、订正专业词、分段归纳、生成四部分、写文件。60 分钟录音的处理时间我这边是 4 到 6 分钟。

三个数对得上才算过：

- 待办条数：把它生成的待办条数，跟录音里"我来""我跟进""下周""我这边安排"这类表态出现的次数对比。差 2 条以内正常，差 5 条以上说明它漏了大段，回去重跑。
- 未指定条数：一场一小时会，出现 1 到 3 条"未指定"是健康的，说明它没瞎猜。如果一条都没有，非常可疑，大概率是它替你把责任人和日期编出来了。
- 听不清标记数：录音质量一般的话，3 到 8 个 [听不清] 是正常的。一个都没有，要么录音极好，要么它硬猜了。

产物长什么样：一个 Markdown 文件，开头是会议基本信息表，然后是四部分。待办清单是一个四列表格，每条一行，责任人那一列要么写人名，要么写"未指定"。

举一个我自己的例子。上个月一场供应商评审会，它给我 9 条待办，其中 3 条责任人写"未指定"。我回去听录音，那 3 条确实是会上含糊带过、谁都没接的活。我把这 3 条单独拎出来在群里点了一次名，两天内就有人认领。纪要里出现"未指定"，其实是给你一张催办清单。

7. 翻车清单

翻车点一：人名被听成同音字。我遇到过"张工"被写成"章工"，"李工"被写成"吕工"。补救动作：把参会人名单和专业词汇表一起给它（提示词里的【专业词汇表】那段就是干这个的），并在验收时专门核一遍所有人名，一个一个对。

翻车点二：数字错。转写对数字的容错率比你想的低，"3.5"听成"35"、"15 号"听成"50 号"，我全遇到过。补救动作：把纪要里出现的每一个数字单独摘出来，回录音里点一下时间戳听一遍。一场会通常只有 8 到 15 个关键数字，两分钟能核完。

翻车点三：把讨论当结论。会上大家讨论了三个方案，最后只定了其中一个是备选，它很可能写成"会议决定采用三方案"。补救动作：看它输出的第一部分"会议结论"，每一条都问自己"这句话在录音里有没有人说'就按这个来'定了'这类拍板的话'。没听到，就降级到第三部分的"未决事项"。

8. 验收标准与变体

验收动作（照着做，5 分钟）：

1. 打开产物，把待办清单里所有责任人名字抄出来，跟参会人名单比对，不许出现名单外的人。
2. 把所有日期抄出来，回录音按时间戳核对，至少抽查 3 个。
3. 通读"会议结论"部分，任何一条你回忆不起来会上拍过板的，删掉。
4. 只读待办清单的"责任人"那一列，逐个确认上面的活真的有人在会上答应过。
5. 统计 [听不清] 的数量，按第 6 节的标准判断合不合理。

变体一：客户电话纪要。把提示词里的"会议"换成"电话"，去掉"参会人"改成"双方出席人"，其它不动。电话录音通常更吵，专业词汇表那段一定要填。

变体二：每日站会速记。15 分钟站会不需要四部分，把提示词的输出要求改成只要"待办清单"和"卡住的问题"两部分，输出纯文本不要 Markdown，30 秒出结果。

变体三：面试记录。把"待办清单"换成"候选人回答摘要"，加一条约束"只记录候选人说的话，不给评价、不打分"。

老兵说：我最早用这套流程时，图省事跳过了"未指定不许猜"这条约束，结果它给我编出了6条责任人全是我的待办。发出去以后我扛了三周。现在我每次都在提示词里把这条加粗，而且看到"未指定"反而放心。

场景 02 通知与公告：一份模板批量生成 N 个版本 ★★★

1. 一句话背景

同一件事要发给不同对象：全公司一份、部门一份、供应商一份、班组一份。人工做法是复制模板改四遍，每遍 20 分钟，烦在改的时候总有遗漏，落款、时间、收件范围四份各错一处。

2. 名词小词典

模板：一份通知里固定不变的部分，比如公司抬头、落款、联系方式。

变量：每次发布要替换的内容，比如时间、地点、对象、截止日期。写通知的正确姿势是把变量单独列出来，不要在正文里到处改。

收件对象：这份通知发给谁。同一件事对不同对象，能说的信息量不一样，这是要出多个版本的根本原因。

口径：同一件事在不同版本里的说法。给供应商的版本不能出现内部代号，给班组的版本不能出现预算数字。

脱敏：把不该出现在某个版本里的信息（内部代号、人名、金额）替换或删除掉。

3. 开工前准备

- 一份你以前发过的、已经审批过的通知原件（Word 或纯文本），作为模板来源。
- 一张变量表：把这次要写的通知里所有会变的内容列成表（时间、地点、参会范围、截止日期、联系人）。
- 一个"口径清单"：列出哪些信息能进哪个版本、哪些不能。这一条最容易被省掉，省掉必翻车。
- WorkBuddy 已登录，新建一个对话。

4. 操作步骤

1. 把通知原件拖进对话框，模式切 Plan。
2. 让它先做一件事：把原件拆成"模板部分"和"变量部分"，输出给你看。这一步你检查它对模板的理解对不对。
3. 你确认拆分没问题之后，把变量表和口径清单贴进去。

- 4. 用第 5 节的提示词，切 Agent 模式让它一次性出四个版本。
- 5. 它干完会汇报每个版本改了哪些地方。先看这份差异汇报，比通读四份全文快得多。
- 6. 打开四份文件，只看变量位置（时间、地点、落款、收件范围），逐个核对。
- 7. 确认无误后，让它把四个版本分别存成独立文件，文件名带收件对象标识。

变量的粒度要掌握好。我第一次做的时候把"时间"写成一个变量，结果有的版本写"10 月 24 日"，有的写"10 月 24 日 08:00"。后来我把日期和钟点拆成两个变量，四个版本就一致了。

判断标准很简单：如果一个变量在不同版本里可能被写成不同长度或不同格式，就再拆细一层。



图 7-3 一份模板出四个版本：变量表加口径清单是两条输入

图 7-3 模板批量生成多版本的输入与输出（作者根据公开资料绘制）

5. 可复制提示词

附件是我以前发过的一份通知原件，请以它为模板，生成 4 个不同收件对象的版本。

【第一步，先只做拆分，不要生成正文】

把原件拆成两部分报告给我：

- A. 模板部分：无论发给谁都不变的内容（抬头、落款、联系方式、格式结构）。
- B. 变量部分：每次发布要替换的内容，列成一张表，每行一个变量。

拆分结果先给我看，我确认后你再生成正文。

【本次的变量表】

- 事件：[年度设备检修]
- 时间：[2026-10-24 08:00 至 2026-10-26 18:00]
- 地点：[二号厂房及辅助区域]
- 影响范围：[二号厂房内所有工位 10 月 24 日至 26 日停止供电]
- 配合要求：[10 月 23 日 17:00 前清空工位周边易燃物]
- 联系人：[设备科 王工，内线 8xxx]

【四个版本与各自口径】

版本 1：全公司公告。可提影响范围，不提内部代号和预算。

版本 2：部门内部通知。可提内部代号，可提配合要求的详细分工。

版本 3：发供应商（A 厂、B 厂）。只提时间和影响范围，不提任何内部信息，不出现公司内部部门名称，不出现人名，联系人改为“采购部对接人”。

版本 4：班组张贴版。300 字以内，口语化，只讲时间、地点、要做什么、找谁。

【硬性约束】

- 四个版本的事实部分（时间、地点、范围）必须完全一致，一个字不许差。
- 每个版本开头用一行标明收件对象，格式：【发：xxx】。
- 落款和联系方式按版本走：内部版本用内部落款，对外的版本统一用“采购部”。
- 不许出现模板里没有的新事实。你要补什么信息，先问我，不要自己补。
- 四个版本分别存为独立文件：通知_v1_全公司.md、通知_v2_部门.md、通知_v3_供应商.md、通知_v4_班组.md，全部存到桌面“通知”文件夹。

【干完以后向我汇报】

1. 四个版本的事实部分（时间、地点、范围）是否一字不差，请你自己逐字比对一遍并告诉我结果。
2. 每个版本你删掉了哪些原文里的信息，逐条列出。
3. 有没有哪个地方你拿不准、需要我确认。

6. 过程与判断点

Plan 阶段它会先给你一份拆分表。你要看的是：它有没有把“落款”和“联系方式”正确地归到模板部分，有没有把“时间”归到变量部分。它要是把某个本该变的写成了固定内容，直接在拆分阶段纠正，不要等到四份都生成完再改。

Agent 阶段四个版本一起出，耗时 2 到 4 分钟。

判断点：

- 事实一致性：它的汇报第 1 条会自己比对一遍。但你不许只看它这句话，自己把四个文件里的时间各复制一次，贴到同一个记事本里比。四个时间必须完全一样。
- 删减清单：汇报第 2 条会列出每个版本删了什么。重点看供应商版本，它删掉的应该包括所有内部部门名和人名。
- 字数：班组版应该在 300 字上下。超过 500 字说明它没删够，让它重写。

产物长什么样：四个 .md 文件，每个开头一行【发：xxx】，正文格式跟原件一致。

我上个月发过一次停机检修通知，四个版本从模板到定稿一共 6 分钟，其中我自己核对花了 4 分钟。同样的活放在以前，复制四份、逐份改、再逐份读一遍，我要花 80 分钟。省下来的 74 分钟，我干完了那周压着的台架数据整理。

这类活的特点是：不难，但极其吃时间，而且错一处就要重发一次。

7. 翻车清单

翻车点一：事实漂移。四个版本里的时间或地点被它改成了不同说法（比如一个写"10 月 24 日"，一个写"下周五"）。补救动作：在提示词里加死一条"时间一律写成年月日格式，不许写'下周五'本周'这类相对时间"。

翻车点二：对外版本泄露内部信息。这是最危险的一条。补救动作：不要相信它，自己用查找功能在供应商版本里搜一遍内部部门名和人名。我在提示词里要求"联系人改为采购部对接人"，验收时还要手动搜一次"王工""设备科"。

翻车点三：它替你补了模板里没有的事实。比如它自己写了一句"检修期间请关闭电脑电源"，原件里根本没有。补救动作：汇报第 3 条就是为这个设的，看它列出的"拿不准"项；另外通读时专门找那些"看起来很合理但你没说过"的句子。

8. 验收标准与变体

验收动作（照着做，4 分钟）：

1. 四个文件里的时间、地点各复制一次，贴一起逐字比对。
2. 在对外版本（v3）里搜索内部部门名、人名、内部代号，命中数为 0 才过。
3. 数一遍班组版字数，300 到 500 之间。
4. 通读 v1（全公司版），找有没有原件中没有的新事实。
5. 把四个版本各读一遍开头两行，确认版本标识没有贴错（我遇到过 v3 的正文装进了 v4 的文件）。

变体一：节假日值班通知。变量表换成值班日期和值班人名单，四个版本换成"全公司 / 值班人员本人 / 安保 / 外部客户（自动回复）"。

变体二：工程变更通知。变量表换成零件号、切换日期、在制品处理方式。这是研发最常用的变体，四个版本是"内部研发 / 生产 / 质量 / 供应商"。供应商版本记得脱敏零件内部代号。

变体三：多语言版本。把"四个版本"的要求改成"中文版和英文版各一份"，加一条约束"英文版不得比中文版多任何信息"。

老兵说：我第一次做供应商版本，它把"设备科 王工，内线 8xxx"原封不动留在了落款里。那封信要是发出去，我要解释半个月。从那以后我的规矩是：对外文件发出前，用查找功能把内部人名和部门名各搜一遍，这是硬性动作，不看它的汇报。

场景 03 Excel 数据清洗：乱表格变干净表 ★★★

1. 一句话背景

别人给你的 Excel，合并单元格、空行、数字存成文本、日期三种格式、同一零件两种写法。手工整理一次 30 分钟，烦在你不知道表里到底有多少坑，改完心里没底。

2. 名词小词典

合并单元格：几个格子被合成一个大格子。看着好看，但排序、筛选、统计全会出错，是数据清洗的头号敌人。

文本型数字：长得像数字，其实被存成了文字，Excel 会在格子左上角标个绿色小三角。它们不能求和、不能比较大小。

脏数据：原始表里那些不合规、不一致、重复、残缺的数据。清洗就是把脏数据变成干净数据。

唯一值：某一列里不重复的内容。数一数"零件号"这一列有多少个唯一值，是判断重复和命名混乱最快的办法。

分列：把一格里的复合内容拆成几列。比如"XA-2308/电机/2件"拆成零件号、名称、数量三列。

幂等：同一个操作跑几次结果都一样。清洗脚本必须幂等，否则你跑第二遍数据就毁了。

3. 开工前准备

- 原始 Excel 文件一份，先复制一个副本，命名为 原始数据_备份.xlsx。清洗永远在副本上做。
- 一张"清洗规则表"：你想让表变成什么样，逐条写清楚（列顺序、列名、日期格式、数字小数位、去重依据）。
- 明确一件事：哪些行是"必须保留"的（比如有实测数据的行），哪些行可以删（比如完全空的行、小计行）。
- WorkBuddy 已登录，新建对话。

【待补：界面路径，以实际版本为准】Excel 的"文本导入向导""分列"等菜单项在不同版本位置不同，本章统一采用"让 AI 直接读写文件"的方式，绕开菜单操作。

4. 操作步骤

1. 先备份。把原始文件复制一份，后面所有操作只在副本上做。
2. 把副本拖进 WorkBuddy 对话框，模式选 Plan。

- 3. 先让它做体检，不要直接清洗。这一步它会输出这份表有哪些问题、各有多少处。
- 4. 你看着体检报告，把清洗规则表补全（体检会暴露出你没想到的问题）。
- 5. 把完整的清洗规则贴进去，切 Agent 模式，让它写脚本跑清洗。
- 6. 它干完会输出一份"清洗前后对比"。重点看行数变化。
- 7. 打开生成的新文件，按第 6 节的行数守恒公式核对。
- 8. 抽查 5 行，回原始文件比对内容。

文件特别大的时候（超过 5 万行），不要一次全丢给它。我一般按 5000 行一段拆开，先拿第一段跑通流程，确认规则没问题，再让它用同一套规则跑剩下的段。这样做的好处是规则写错了只损失第一段的重跑时间。

跑完之后把各段合并，再按行数守恒把总数核一遍。

Excel 数据清洗流程

三步主干：体检、清洗、核对行数守恒

- 1 先复制备份再动手**
清洗永远只在副本上做，这条没有例外
- 2 只做体检不改数据**
让它逐项列出问题有几处
- 3 用体检报告补规则**
体检会暴露你没想到的问题
- 4 按规则清洗**
取消合并、去重、统一日期格式
- 5 核对行数守恒**
 $\text{原始行数} = \text{清洗后行数} + \text{被删行数}$

被删的行必须能在一张叫被删行的工作表里全部找回来

图 7-4 Excel 清洗三步：体检、清洗、核对行数守恒

图 7-4 Excel 数据清洗的三步流程（作者根据公开资料绘制）

5. 可复制提示词

附件是一份待清洗的 Excel 文件（已备份，你只处理这份副本）。

【第一步：只做体检，不要改数据】

先给我一份体检报告，逐项列出这份表的问题和处数：

1. 合并单元格有几处，在哪些区域。
 2. 空行有几行，哪些是整行空、哪些是只有部分列空。
 3. 文本型数字出现在哪几列，大概多少格。
 4. 日期格式有几种写法，各举一个例子。
 5. 重复行有几行（按 [零件号 + 日期] 组合判断重复）。
 6. 同一列的同一内容有没有多种写法（比如零件号有的带空格有的不带、大小写不一致、全角半角混用）。
 7. 有没有小计行、合计行、备注行混在数据区里。
- 体检报告出完就停，等我确认清洗规则。

【第二步：按下面的规则清洗】

1. 取消所有合并单元格，合并前的值填充到合并区域内的每一个格子。
2. 删除整行完全为空的行。只有部分列空的行，保留，空着就空着，不许填 0。
3. 把 [实测值 / 电压 / 电流] 这几列转成数字，转不了的保留原样并在新增的 "异常标记" 列写 "无法转数字"。
4. 日期列统一改成 YYYY-MM-DD 格式。识别不出来的保留原样，异常标记列写 "日期无法解析"。
5. 零件号列：去掉首尾空格，英文统一大写，全角括号改成半角。
6. 按 [零件号 + 日期] 去重，保留第一次出现的那行。被删掉的行不要消失，全部复制到一个叫 "被删行" 的新工作表里。
7. 删除数据区里的小计行、合计行（判断依据：[零件号] 列为空且 [备注] 列含 "合计" 或 "小计"）。
8. 列顺序按这个来：[零件号、日期、批次、实测值、电压、电流、备注、异常标记]。

【硬性约束】

- 不许修改任何单元格里的原始数值，你只能改格式、改位置、改数据类型。
- 不许删除任何 "实测值" 列有内容的行，即使它其它列是空的。
- 所有被你改动或被你删除的行，都要能在 "被删行" 工作表里找到。
- 新文件另存为：清洗后_[原文件名]_[今天日期].xlsx，存到桌面。原文件不许动。
- 我可能要连着跑几次，你的处理必须可以重复执行，跑第二遍结果跟第一遍一样。

【干完以后向我汇报】

1. 原始行数、清洗后行数、被删行数。三个数必须满足：原始行数 = 清洗后行数 + 被删行数。
2. 你改了哪些列的格式，逐列列出。
3. 异常标记列里一共有几行有内容，分别在哪些行。
4. 有没有你拿不准、需要我拍板的地方。

6. 过程与判断点

体检报告是这一步最值钱的东西。我第一次跑一份 2147 行的检测数据表，它给我列了 7 类问题，其中 "同一零件号有 11 种写法" 这一条我自己完全没意识到。体检花 1 分钟，省下的返工远不止。

清洗阶段耗时 2 到 5 分钟，取决于行数。2000 行左右我这边 3 分钟。

行数守恒公式（必须成立）：

原始行数 = 清洗后行数 + 被删行数

这三个数它会在汇报第 1 条给你。如果加不起来，说明有行被它悄悄丢掉了，立刻停手，让它重新跑并逐条说明每行去向。

其它判断点：

- 异常标记列：这一列有内容的行数，应该跟你在体检报告里看到的"问题格数"量级相当。如果异常标记列一行都没有，而体检报告说有 30 个文本型数字，说明它把转不了的数硬转了或删除了。
- 唯一值数量：清洗前后，零件号的唯一值数量应该减少或持平，不应该增加。增加了说明它把同一个零件拆成了两种写法。

产物长什么样：一个新的 xlsx，第一张表是清洗后的数据，第二张表叫"被删行"，原文件保持不动。

有一回我拿到一份供应商给的物料表，2100 行。体检报告说零件号那一列有 11 种写法，我一开始不信，让它把这 11 种列出来，一看就服了：光 XA2308 这一个号就有四种写法，带横线、不带横线、末尾带空格、括号用全角。这四种在 Excel 眼里是四个不同的东西，做透视表会把同一个零件拆成四行。

7. 翻车清单

翻车点一：它改了你的原始数值。最常见的是把 3.50 改成 3.5（这个还好），把 0.05 转成了科学计数法，或者把带单位的 12.3V 直接变成 12.3。补救动作：在提示词里加死"不许修改任何原始数值"，并在验收时用原始文件的副本做一次逐格对比（让它写个对比脚本，输出差异清单）。

翻车点二：合并单元格取消后数据丢了。合并区域取消后，值只留在左上角那一格，其余格子变空。补救动作：提示词里我写了"填充到合并区域内的每一个格子"，验收时专门抽查原来有合并单元格的区域，看每一格是不是都有值。

翻车点三：去重依据选错。按"零件号"去重会把同一零件的多次测量数据删得只剩一条。补救动作：去重依据一定要写成组合键（零件号 + 日期），并且所有被删的行进"被删行"表，随时能捞回来。

8. 验收标准与变体

验收动作（照着做，6 分钟）：

1. 核对行数守恒：原始行数 = 清洗后行数 + 被删行数。对不上就整份作废重跑。
2. 打开"被删行"表，随便挑 3 行，确认它们确实该删（空行、重复行、小计行）。
3. 在清洗后的表里，随便挑 5 行有数据的，回原始文件找到对应行，逐格比对数值，一个字都不许差。
4. 检查日期列：全表拉一遍，格式是不是统一成了 YYYY-MM-DD。
5. 数一下零件号唯一值的个数，跟清洗前比，不许变多。
6. 检查数字列有没有被转成科学计数法，尤其是小于 0.001 的数值。

变体一：多表合并。把 12 个月的表合并成一张年表。加一条规则"新增一列'来源文件'，写上每行来自哪个文件"，合并后靠这一列溯源。

变体二：超差筛选。在清洗规则里加一条"新增一列'判定'，实测值超出上下限的写'超差'并整行标黄，上下限见我给你的规格表"。这是质量工程师最常用的一支。

变体三：跨表对账。两张表的同一零件号对不上，让它输出一张差异对照表，三列：零件号、表 A 的值、表 B 的值，只列有差异的行。

老兵说：我吃过一次大亏，没备份就让它清洗，跑完发现它删了 38 行有实测数据的行，因为那些行的日期格式不合规。原始数据已经没了，最后是同事从邮件附件里翻出旧版本才救回来。现在我清洗任何表，第一件事永远是复制副本，这条没有例外。

场景 04 Word 报告格式化：照公司模板一键套版 ★★★

1. 一句话背景

你写完了内容，但公司模板有固定要求：标题几号字、正文行距多少、图表编号规则、页眉页脚、目录自动生成。手工套一遍 45 分钟，烦在你改了内容之后格式又乱了，得再套一遍。

2. 名词小词典

样式：Word 里的一组格式设定，比如"标题 1""正文"。用样式而不是手动调字号，是格式不崩的唯一办法。

模板文件：`.dotx` 或 `.dotm` 后缀的 Word 文件，里面预置了样式、页眉页脚、封面。公司发的模板通常是这个。

域：Word 里能自动更新的东西，比如自动页码、自动目录、交叉引用。手动敲的页码不是域，改了内容不会跟着变。

大纲级别：每一段属于几级标题。目录和导航窗格都靠它，标题用了样式才有大纲级别。

分节符：把文档分成几节，每节可以有各自的页码格式和页眉。封面不显示页码就靠它。

3. 开工前准备

- 公司模板文件一份（`.dotx` / `.dotm` / 或一份格式正确的旧报告 `.docx`）。
- 你的报告内容：可以是 Word、Markdown、或纯文本。内容不用先排版，越素越好。
- 一份"格式要求清单"：把模板的硬性要求逐条列出来（标题字号、行距、页边距、图表题注格式、参考文献格式）。没有现成清单的话，让 AI 从模板里读出来。
- 图片素材单独放一个文件夹，图片文件名带上图号。
- WorkBuddy 已登录。

【待补：界面路径，以实际版本为准】不同 Word 版本中"样式""引用""题注"的菜单位置有差异，本节采用"AI 直接读写 docx 文件"的方式，避免依赖具体菜单。

4. 操作步骤

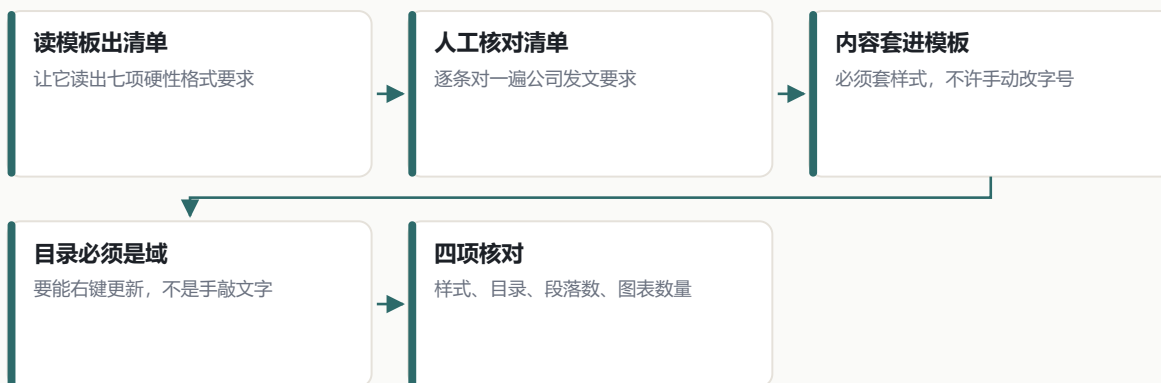
1. 把模板文件和你的内容文件一起拖进 WorkBuddy 对话框，模式选 Plan。
2. 先让它读模板，输出一份"这个模板的格式要求清单"给你。你核对这份清单跟你们公司要求一不一致。
3. 清单确认后，把你的内容文件交给它，切 Agent 模式，让它按清单套版。
4. 它干完会汇报：套用了哪些样式、生成了几级目录、处理了几张图、有没有它识别不出来的段落。
5. 打开生成的文件，按第 6 节的四项核对。
6. 重点查目录：右键更新域（或让它帮你更新），看页码对不对。
7. 用 Word 的导航窗格看大纲结构，确认标题层级没乱。

如果你的公司没有正式的模板文件，只有一份"格式正确的旧报告"，这套流程照样能跑。让 AI 从那份旧报告里读样式规则，效果差不多。

唯一差别是：它读到的是"这份文件里实际用了什么"，而不是"模板里规定了什么"。旧报告本身格式就有问题的情况下，它会吧错的也学过去。所以拿到格式清单之后，你一定要自己对一遍公司发文要求，这一步不能省。

Word 模板套版两阶段

先读模板出格式清单，再按清单套内容



32 页报告：手动套版 52 分钟，这套流程生成 7 分钟加核对 9 分钟

图 7-5 套版流程：先读模板出清单，再按清单套内容

图 7-5 Word 模板套版的两阶段流程（作者根据公开资料绘制）

5. 可复制提示词

附件有两个文件：一个是公司 Word 模板，一个是我的报告内容（未排版）。

【第一步：只读模板，输出格式清单】

请把这份模板的格式要求逐条读出来，输出成一份清单，包含：

1. 各级标题（标题 1 到标题 4）的字体、字号、加粗、段前段后间距、编号规则。
2. 正文的字体、字号、行距、首行缩进、段间距。
3. 页边距、纸张方向、装订线。
4. 页眉页脚内容，页码位置与格式，封面是否单独一节。
5. 图题注和表题注的格式（位置、编号方式、字号）。
6. 参考文献的编号与格式。
7. 目录要生成到第几级。

清单出完就停，等我确认。

【第二步：按清单把我的内容套进模板】

1. 内容里的一级标题套"标题 1"，二级套"标题 2"，以此类推。
不要用手动改字号的方式做标题，必须套样式。
2. 正文全部套"正文"样式，不许有任何一段是"手动格式"。
3. 图片按文件名里的图号顺序插入，每张图下方加题注，格式按清单第 5 条。
4. 表格上方加表题注，编号独立于图编号。
5. 自动生成目录，生成到清单第 7 条要求的级别，目录必须是可更新的域，不是手敲的文字。
6. 封面单独一节，不显示页码；正文从第一页重新开始编页码。
7. 页眉页脚按清单第 4 条设置。

【硬性约束】

- 只改格式，不改我写的任何文字。有一句话不通顺也不许你改，你可以在最后单独列一张"建议修改"清单给我，由我决定改不改。
- 遇到你判断不出该套哪个样式的段落，套"正文"，并在汇报里列出来问我。
- 不许删我的任何段落，包括空段落。
- 另存为：套版后_[报告名]_[日期].docx，存到桌面。模板文件不许动。

【干完以后向我汇报】

1. 你识别出的标题层级有几级，各有多少个。
2. 你判断不出该套什么样式的段落有几个，分别在第几页。
3. 图和表各处理了几个。
4. 目录是域还是纯文字（回答"域"或"纯文字"）。
5. 你想建议我修改文字的地方，单独列一张表。

6. 过程与判断点

Plan 阶段的价值在于：它读出来的格式清单，往往比你脑子里记的公司要求更全。我第一次让它读我们公司的模板，它列出来一条"三级标题不进目录但我一直手动加进去了"，这条我错了两年。

套版阶段耗时 5 到 10 分钟，取决于篇幅。30 页报告我这边 8 分钟。

四项核对：

- 样式而非手动格式：打开 Word，把光标放到任意一个标题上，看样式框显示的是不是"标题 2"这类。显示"正文 + 宋体 三号 加粗"这种就说明它偷懒了，要求它重做。
- 目录是域：汇报第 4 条它会直接回答。必须是"域"。检验方法：在正文里插入一个新标题，再更新目录，看新标题有没有出现。
- 段落数守恒：套版前后的段落数应该一样。让它汇报总段落数，你用 Word 的字数统计对比一下（或者让它统计原文件的段落数）。
- 图片数量：汇报第 3 条的图数、表数，跟你素材文件夹里的文件数、原文档里的表数对。差一个都要查。

产物长什么样：一份 docx，封面不带页码，第二页起有目录，正文标题层级清晰，图和表都有自动编号的题注，页眉页脚符合模板。

我做过一次对比。同一份 32 页的试验报告，我手动套版用了 52 分钟，其中光调图表题注编号就花了 15 分钟。用这套流程跑，它生成 7 分钟，我核对 9 分钟，合计 16 分钟。

差别最大的就是图表编号：我手动插到第 14 张图时编号已经乱了，它一次就编对。

7. 翻车清单

翻车点一：标题被做成了手动格式。表面看着一样，但目录生成不出来、导航窗格是空的、改一个标题字号其它不变。补救动作：验收时一定打开导航窗格（Word 左侧的标题列表）看一眼。导航窗格空着，立刻退回重做。

翻车点二：它顺手改了你的文字。这是最容易发生也最危险的一条，它会"帮你润色"。补救动作：提示词里我已经写了"只改格式不改文字"，验收时再用 Word 的"比较文档"功能，把套版前后的文件比一次，所有差异应该只有格式。

翻车点三：图片位置跑掉或分辨率被压。图片被挪到了文末，或者被压缩得看不清。补救动作：提示词里加一句"图片按原始分辨率插入，位置保持在原文档中对应的段落之后"，验收时逐张点开看清晰度。

8. 验收标准与变体

验收动作（照着做，8 分钟）：

1. 打开导航窗格，确认标题层级树完整、没有缺级、没有出现"正文"混进标题树。
2. 随便点 3 个标题，看样式框显示的是不是对应的标题样式。
3. 在正文末尾插入一个测试标题，更新目录，看它有没有进目录；测完删掉。
4. 用"比较文档"比对套版前后，确认差异只有格式没有文字。
5. 翻到封面和第二页，确认封面无页码、正文从 1 开始。
6. 逐张检查图片清晰度与位置。
7. 全文搜一遍常见的互联网套话（那些把动词当名词用的词），命中数为 0。这类词最容易从训练语料里冒出来，且删掉后不影响任何信息。

变体一：投标技术文件套版。这类模板通常要求极严（章节编号必须连续、不能有孤行）。在提示词里加"章节编号用多级列表，编号与标题样式绑定，不许手敲编号"。

变体二：把 Markdown 手稿转成正式报告。你平时用 Markdown 写，交付要 Word。这一步把第 5 节提示词的内容换成你的 .md 文件，其它完全一样。

变体三：旧报告批量换新版模板。十份旧报告要统一换成新模板。让它先读新模板出清单，再循环处理十份文件，每份存独立文件，最后给一张"十份各自的处理结果"表。

老兵说：我们部门新来的同事套版，用的是最笨的办法：把内容一段一段复制到模板里，然后手动调字号。一份 40 页报告他弄了一整天。我让他照这个场景跑了一遍，11 分钟出稿，他核对了 8 分钟。他问我为什么早两年不告诉他，我说因为两年前我自己也是这么手动弄的。

场景 05 PPT：从一份大纲生成可讲的幻灯片 ★★★

1. 一句话背景

你有一份写好的大纲或报告，要做成汇报用的 PPT。人工 90 分钟，其中 60 分钟花在"把文字搬到幻灯片上、调版式、找图"这三件不需要动脑的事上。

2. 名词小词典

大纲：汇报的骨架，一层一层的标题。PPT 做得好不好，八成取决于大纲清不清楚。

母版：PPT 里控制所有页面统一外观的底稿。改母版一次，所有页面跟着变。手动逐页调就是在绕过母版。

版式：母版里预定义的页面布局，比如"标题页""标题加正文""两栏对比""全图页"。

讲者备注：幻灯片下方的备注区，写你上台要说什么。它不显示在投影上，但决定你讲得好不好。

一页一意：一张幻灯片只讲一件事。这是 PPT 唯一真正重要的原则，也是 AI 最容易违反的一条。

3. 开工前准备

- 你的大纲或报告原文一份（Word、Markdown、纯文本都行）。
- 汇报时长和场合（比如"部门周会 15 分钟""客户评审 40 分钟"），这决定页数。
- 公司 PPT 模板一份（.pptx），或者至少确定主色调和字体。
- 要用到的图片、图表截图，单独放一个文件夹。
- WorkBuddy 已登录。

【待补：界面路径，以实际版本为准】PowerPoint 各版本的"母版视图""版式"入口位置不同，本节采用"AI 直接生成 pptx 文件"的方式。

4. 操作步骤

1. 把大纲文件拖进对话框，模式选 Plan。
2. 先让它做"大纲转页面规划"：一页讲什么、用什么版式、要不要图。输出成一张表给你。
3. 这一步你一定要改。它排的页数和顺序通常有问题，你按自己的讲法调。改完再往下走。
4. 确认页面规划后，把公司模板（如有）拖进去，切 Agent 模式，让它生成 pptx。
5. 它干完会汇报：共几页、用了哪几种版式、备注写了多少字、有没有哪页文字超量。
6. 打开 PPT，按第 6 节的三个数核对。
7. 通读一遍讲者备注，那是你上台要说的话，不通顺就让它改。

配图这件事我单独说一句。它生成的 PPT 里，凡是标了"需要配图"的页，默认是空着的，等着你填。别指望它自己画示意图，它画出来的工程示意图比例和标注基本不对，放进评审材料要出事。

我的做法是：规划表阶段就把"用我给的图"标清楚，截图按页码命名（比如 p05_台架布置.png），让它按文件名插进去。这 5 分钟的命名工作，能省掉后面逐页拖图的 20 分钟。

大纲转 PPT 两步走

先出页面规划表，人工改过再生成文件

- 1 大纲转页面规划表**
页码、讲什么、版式、配图、讲几分钟
- 2 这一步你一定要改**
十几页里我通常要改四五页
- 3 加入公司模板**
用它的母版和版式，不自己造配色
- 4 核对三个数**
页数与时长、单页字数、备注字数
- 5 通读讲者备注**
那是你上台直接念的口语

15 页稿子生成约 10 分钟；花 6 分钟改规划表，省掉 30 分钟重做

图 7-6 从大纲到幻灯片：先出页面规划表，人工改过再生成

图 7-6 大纲转 PPT 的两阶段流程（作者根据公开资料绘制）

5. 可复制提示词

附件是我的汇报大纲，请帮我做成一份可以直接讲的 PPT。

【汇报信息】

- 场合：[部门技术评审]
- 时长：[20 分钟]
- 听众：[研发内部，熟悉项目背景，不需要铺垫]
- 目标：讲完之后听众要能回答两个问题：[方案选了哪个] [下一步谁做什么]

【第一步：只出页面规划表，不要生成 PPT】

把大纲转成一张页面规划表，五列：页码、这一页讲什么（一句话）、建议版式（标题页 / 要点页 / 两栏对比 / 全图页 / 数据页 / 结尾页）、需要配图吗（要 / 不要 / 用我给的图）、这一页讲几分钟。
页数按 [20 分钟] 控制在 [12 到 15 页]。规划表出完就停，我要改。

【第二步：我确认规划表之后，按下面规则生成 PPT】

1. 严格按我改过的规划表生成，页数不许增减。
2. 一页只讲一件事。任何一页上超过 3 个要点，拆成两页。
3. 每页要点控制在 6 到 30 个字一条，不许把整段文字搬上去。
4. 每页必须写讲者备注，80 到 150 字，写成我上台直接念的口语，包含这一页的结论和过渡句。
5. 数字要突出：凡是涉及对比、趋势、达成率的数字，单独占一行放大。
6. 使用我给的公司模板的母版和版式，不要自己造配色。
7. 结尾页必须是"下一步"：三到五条待办，带责任人和时间。

【硬性约束】

- 不许出现模板里没有的动画和切换效果。
- 不许出现任何套话、空话、互联网流行词。写完之后自查一遍，凡是删掉也不影响信息的句子，全部删掉。
- 大纲里没有的数据，不许自己编。需要补数据就在备注里标【需补数据】。
- 每页正文字数超过 120 字的，自动拆分并告诉我拆成了哪几页。
- 另存为：汇报_[主题]_[日期].pptx，存到桌面。

【干完以后向我汇报】

1. 总共几页，各用了哪几种版式，各几页。
2. 超过 120 字的页有几页，你拆了没有。
3. 备注总共多少字，平均每页多少字。
4. 标了几个【需补数据】，分别在哪些页。
5. 按 20 分钟算，平均每页讲几分钟，这个节奏合不合理。

6. 过程与判断点

Plan 阶段的页面规划表是这套流程的核心，不要跳过。它会把你的大纲切成十几页，你会立刻发现"原来我这个大纲第三和第四部分其实是同一件事"。我自己在规划表上动刀的比例大概 30%，也就是十几页里要改四五页。

生成阶段耗时 8 到 15 分钟，取决于页数。15 页我这边 10 分钟。

三个数对得上才算过：

- 页数与时长：15 页讲 20 分钟，平均每页 1 分 20 秒。低于 1 分钟说明内容太挤，超过 2 分钟说明某页信息量不够。它汇报第 5 条会算给你。
- 单页字数：打开 PPT，翻一遍，任何一页上密密麻麻超过 120 字，就是不合格。它汇报第 2 条会告诉你有没有拆。
- 备注字数：总备注字数除以页数，应该在 80 到 150 之间。低于 80 说明你上台会卡壳，高于 150 说明你在念稿。20 分钟的稿子，总备注在 1500 到 2500 字之间比较合适。

产物长什么样：一份 pptx，第一页标题页，中间要点页和对比页交替，结尾页是待办表。每一页下方备注区都有一段可以直接念的话。

有一回我要在客户评审上汇报一个方案选型，给 20 分钟。它给我排了 14 页，我看规划表时发现前 4 页都在讲背景，而这些客户全都清楚。我把前 4 页砍成 1 页，多出来的时间挪给方案对比那三页。评审结束时客户问的两个问题，正好落在我加了时间的那几页里。

7. 翻车清单

翻车点一：一页塞太多字。这是 AI 做 PPT 的头号毛病，它会把你大纲里的一整段搬到一页上。补救动作：提示词里我已经写了 120 字上限，验收时翻一遍，看到字多的页就让它拆，拆完重新检查页码总数。

翻车点二：页数失控。你说 15 页，它给你 28 页，因为它觉得每个三级标题都该单独一页。补救动作：提示词里写死页数区间，并在规划表阶段就数一遍行数。

翻车点三：它编了数据。你说"这里放个趋势图"，它给你编了一条曲线和一组数字。补救动作：提示词里要求缺数据标【需补数据】，验收时搜索这三个字，逐个补真实数据。这个动作不能省，编出来的数字进了评审材料是要出事的。

8. 验收标准与变体

验收动作（照着做，10 分钟）：

1. 数页数，对照规划表，一个不许差。
2. 翻一遍所有页，任何一页正文字数超过 120 字就退回重拆。
3. 搜【需补数据】，逐个补真数据。
4. 通读一遍备注，读出声，卡住的地方让它重写。
5. 自己计时试讲一遍，超过时长 20% 就砍页。

变体一：从长报告生成汇报版。输入换成 40 页的报告（不是大纲）。让它在第一步多做一件事："先读报告，输出一份 800 字摘要，再把摘要转成页面规划表"。

变体二：两份 PPT 合并。两个人各写了一半，要合成一份。让它先出"两份的页面对照表"，标出重复内容，你决定保留哪些，再生成合版。

变体三：一页纸汇报。把页数改成 1，版式改成"要点密集页"，用于微信群里发图片汇报。这个变体我每周用。

老兵说：我做 PPT 的规矩是“先改规划表，再生成文件”。有一回我偷懒让它直接生成，拿到 22 页的稿子才发现结构不对，全废。后来我算了一下，花 6 分钟改规划表，能省掉 30 分钟的重做。这笔账在任何时长超过 15 分钟的汇报上都成立。

场景 06 邮件：批量处理与回复草稿 ★★★

1. 一句话背景

每天 40 封邮件，人工读完分类、挑出要回的、写回复，42 分钟。烦在回复内容高度重复，同一类问题你每周要写七八遍几乎一样的答案。

2. 名词小词典

分类标签：给邮件打上类别，比如“要回复”“只要看”“垃圾”“待他人处理”。先分类再处理，比逐封处理快得多。

回复草稿：AI 写好的回复，但由你确认后才发送。本书不推荐让 AI 直接发邮件，中间必须有人。

模板回复：对某类高频问题的标准答案。模板不是偷懒，是保证口径一致。

脱敏：把邮件里的客户真名、金额、内部代号替换掉再给 AI 处理，尤其是要用在线服务的时候。

抄送与密送：抄送人所有人可见，密送人其他人看不到。批量发外部邮件时用密送，否则你把几十个客户的邮箱互相暴露了。

3. 开工前准备

- 把要处理的邮件导出成一个文件（Outlook 可以导出为 .csv 或 .pst，WPS 邮件和网页邮箱一般支持导出为 .csv）。导出字段至少包含：收件时间、发件人、主题、正文。
- 一份“分类规则表”：你希望分成哪几类，每类的判断标准。
- 一份“标准答复库”（可以为空，第一次跑让它帮你从历史邮件里总结）。
- WorkBuddy 已登录。

【待补：界面路径，以实际版本为准】各邮件客户端的“导出”功能位置与格式差异较大，请以你使用的版本为准。若导出受限，可改用“复制邮件列表到文本文件”的方式，只要包含发件人、主题、正文即可。

安全提醒：涉及客户名、金额、内部代号的邮件，处理前先做脱敏替换。这是动作，不是建议。

4. 操作步骤

1. 导出邮件到 .csv，复制一份做脱敏：把发件人邮箱替换成“发件人 A/B/C”，客户名替换成“某欧系平台”“A 厂”“B 厂”。

2. 把脱敏后的文件拖进对话框，模式选 Plan。
3. 先让它做分类，输出一张分类表：每封邮件编号、归到哪类、一句话理由、建议动作。
4. 你看分类表，重点检查"要回复"这一类有没有漏。
5. 确认后切 Agent 模式，让它按分类批量生成回复草稿。
6. 它干完会输出一份草稿清单：编号、收件人代号、草稿正文、是否需要你补信息。
7. 逐条看草稿，需要补信息的自己补，然后复制进邮件客户端发送。不要让它自动发送。

关于处理的频率，我试过三种：每封实时处理、半天批量一次、一天批量一次。实时处理最累，因为你要一直盯着它；一天一次最省事，但急件会压到第二天。

我现在用的是"上午一次、下班前一次"，一天两批。每批 20 到 40 封，处理时间 5 到 8 分钟，正好一杯咖啡的时间，不会打断手上的工程活。

邮件批量处理四步

脱敏、分类、草稿、人工发送，最后一步永远是人

- 1 导出邮件并脱敏**
把发件人和客户名替换成代号
- 2 Plan 模式做分类**
输出编号、类别、理由、建议动作
- 3 批量写回复草稿**
两百字内，不许承诺时间数量和价格
- 4 人工核对后发送**
它只写草稿，发送由我点

以前每天 42 分钟，现在 10 分钟；一天两批，每批 20 到 40 封

图 7-7 邮件批量处理：脱敏、分类、草稿、人工发送四步

图 7-7 邮件批量处理流程（作者根据公开资料绘制）

5. 可复制提示词

附件是今天待处理的邮件（已脱敏，发件人用代号表示）。

【我的分类规则】

- 类别 1「必须我回复」：发件人是项目相关方，且邮件里有明确问题等我回答。
- 类别 2「只要看」：通知、抄送、周报、系统自动推送，不需要回复。
- 类别 3「转他人」：该由别的同事处理的，你告诉我应该转给谁（按我下面给的职责表）。
- 类别 4「可模板回复」：常见问题，用标准答案就能回。
- 类别 5「疑似无关」：广告、群发、误投。

【同事职责表（用于判断转给谁）】

[张工：结构件与图纸；李工：试验与台架；王工：质量与检具；陈经理：项目与排期]

【第一步：只做分类，不要写回复】

输出一张分类表，四列：邮件编号、类别、一句话理由、建议动作。

分类完就停，等我确认。

【第二步：我确认后，按类别处理】

- 类别 1：逐封写回复草稿。200 字以内，直接回答对方的问题，不知道的地方标【需你补充】，不许编。
- 类别 3：写一封转办说明，写清“这件事应该由谁处理、为什么”，我拿去转发给对应同事。
- 类别 4：用标准答案回复。如果你还没有标准答案库，先从这批邮件里总结出 3 到 5 类高频问题和对应答案，给我看，我确认后再套用。
- 类别 2 和类别 5：不写任何东西，只在汇总表里列出数量和编号。

【硬性约束】

- 所有回复用中文，称呼用“您好”，结尾用“顺祝 工作顺利”。
- 不许承诺任何时间、数量、价格。涉及这三类，一律标【需你补充】。
- 不许出现任何套话、空话、互联网流行词。语气像人说话，不像公文。
- 每封草稿开头写一行：【致：发件人代号】【原邮件主题：xxx】，方便我核对。
- 输出成一个 Markdown 文件：邮件草稿_[日期].md，存到桌面。
- 绝对不要发送任何邮件，你只写草稿。

【干完以后向我汇报】

1. 五个类别各多少封，加起来是否等于总数。
2. 类别 1 里你标了几个【需你补充】，分别是哪些编号。
3. 你总结出的高频问题有哪几类（类别 4 用）。
4. 有没有邮件你觉得分类拿不准，需要我看原文。

6. 过程与判断点

分类阶段耗时 1 到 3 分钟，40 封邮件我这边 2 分钟。分类表的价值在于它把“要不要回”这个判断先做完了，你不用自己读 40 封。

草稿阶段 3 到 6 分钟。

三个数对得上才算过：

- 分类总数守恒：五个类别的封数加起来，必须等于邮件总数。它汇报第 1 条会算。对不上说明有邮件被跳过了。
- 需你补充的数量：40 封里通常有 3 到 8 个【需你补充】，这是正常的，说明它没瞎编。如果一个都没有，反而要警惕。
- 类别 1 的封数：跟你自己的感觉对比。你印象里今天该回 8 封，它说只有 2 封，那就回去看分类表，多半是它把几封归到了"只要看"。

产物长什么样：一个 Markdown 文件，按类别分节。类别 1 的每封草稿开头有【致：发件人 A】【原邮件主题：xxx】，然后是 200 字以内的正文。

我统计过自己的收件：一周 200 封上下，其中真正需要我回复的不到 30 封，剩下的全是通知、抄送、系统推送。也就是说，我过去每天花在邮箱上的 42 分钟里，有 30 分钟是在判断"这封要不要回"。

分类表解决的正是这 30 分钟。

7. 翻车清单

翻车点一：它替你承诺了交期。这是邮件的头号事故，写了一句"我们下周三前给您答复"，你没说过。补救动作：提示词里写死"不许承诺时间、数量、价格，一律标【需你补充】"，验收时搜索所有日期和数字。

翻车点二：脱敏没做干净。客户真名、金额留在文件里被处理了。补救动作：处理前用查找功能搜一遍客户名和内部代号，命中数为 0 再往下走。

翻车点三：自动发送。有些工具配置后可以自动发邮件。补救动作：不要开这个权限。本书的立场很明确，AI 只写草稿，人点发送。原因不是它写得不好，是邮件一旦发出去就收不回。

8. 验收标准与变体

验收动作（照着做，7 分钟）：

1. 核对分类总数守恒。
2. 在草稿文件里搜索所有日期和数字，逐个确认是你说过的话。
3. 搜索【需你补充】，逐个补上。
4. 检查每封草稿的称呼和落款格式。
5. 逐封通读一遍，确认语气像你（这一点只能靠人，AI 判断不了）。

变体一：周报邮件汇总。把一周的邮件按项目归类，输出一张"本周各项目往来情况"表，作为周报素材。

变体二：投诉邮件初筛。加一个类别"疑似投诉"，判断标准写"正文含'不满'延误'为什么还没'等词，或同一发件人 3 天内第二封催问"。这类邮件优先人工处理。

变体三：供应商询价批量回复。类别 4 的标准答案库换成"询价回复模板"，变量是零件号、数量、报价周期。这个变体采购同事用得最多。

老兵说：我从不让 AI 直接发邮件，一次都没有。不是它写得不好，是我见过一次事故：同事开了自动发送，AI 给客户回了一句含糊的“应该没问题”，客户拿这句话当承诺，后面扯了两个月。邮件的规矩我立得很死：它写草稿，我看一遍，我点发送。多花的 5 分钟，买的是不背锅。

小白词典

转写：把录音里说的话变成文字的程序活儿。相当于一个打字极快但会听错的速记员。

说话人分离：程序自动判断哪句话是谁说的。人少时准，抢话时容易混，所以纪要里的人名必须人工核。

待办清单（行动项）：会议纪要里最有用的部分。一件事加一个责任人加一个完成时间，三样齐全才算一条有效待办。

未指定：会上没说清谁负责或什么时候交。老老实实写“未指定”，比瞎编一个责任人安全得多。

合并单元格：几个格子合成一个大格子。看着整齐，排序筛选统计全会出错，是数据清洗的头号敌人。

文本型数字：长得像数字其实被存成了文字，不能求和。Excel 会在格子左上角标绿三角。

脏数据：原始表里不合规、不一致、重复、残缺的数据。

幂等：同一个操作跑几次结果都一样。清洗脚本必须幂等，否则跑第二遍数据就毁了。

样式：Word 里的一组格式设定，比如“标题 1”“正文”。用样式而不是手动调字号，是格式不崩的唯一办法。

域：Word 里能自动更新的东西，比如自动页码、自动目录。手敲的页码不是域，改了内容不会跟着变。

母版：PPT 里控制所有页面统一外观的底稿。改一次母版，所有页跟着变。

一页一意：一张幻灯片只讲一件事。这是 PPT 唯一真正重要的原则。

脱敏：把不该外传的信息（客户真名、金额、内部代号）替换掉再处理。这是动作，不是建议。

密送：收件人互相看不到彼此邮箱。批量发外部邮件必须用密送。

你的岗位怎么用

行政与文秘：场景 02 和场景 04 是你的主战场。把公司的通知模板交给它，一次出四个版本；每月的会议纪要交给场景 01。我建议先在场景 02 上练手，它产出最快、最容易看出效果。

项目经理：场景 01 和场景 05。每周的项目例会纪要直接进你的跟踪表，周汇报 PPT 从大纲生成。你的杠杆在场景 01 的“待办清单”那一栏，那张表可以直接当任务分派依据。

质量工程师：场景 03 和场景 04。检测数据清洗是你每天都要做的事，先跑体检报告再清洗；月度质量报告套公司模板。场景 03 的“被删行”表建议留档，它能说明你为什么删了那些行。

采购与供应商管理：场景 02 的对外部版本和场景 06 的变体三。发供应商的通知一定走脱敏检查；询价邮件批量回复用模板库。记一条规矩：对外文件发出前，用查找功能搜一遍内部部门名和人名。

试验工程师：场景 03 的超差筛选变体，和场景 05 的试验汇报。台架数据清洗完直接标超差行，汇报 PPT 从试验大纲生成。

研发工程师（跟我一样）：六个场景全都用得上，但优先级是 03、04、01。数据、报告、纪要，这三件占掉研发人员办公杂活的七成以上。

本章交付物

每周杂活交接清单：一张 A4 纸，贴在你显示器边上，六个场景各自的"什么时候交给它、交给它之前要备什么、干完验哪三样"。

怎么做：

1. 从本章六个场景里，各抄一条"验收标准"里你最容易记住的动作，写在一张纸上。比如场景 03 抄"原始行数 = 清洗后行数 + 被删行数"。
2. 在纸的最上方写一行：它写草稿，我点发送；它出稿子，我验完再交。
3. 六个场景的提示词，各存成一个 .txt 文件，放在同一个文件夹里，命名成"01_会议纪要.txt"这样。用的时候直接复制粘贴，不要每次重新写。
4. 用一周时间，每用一个场景就在纸上打一个勾，记录实际耗时。一周之后你会有一份自己的数据，而不是我的数据。

作者经验：这张纸我自己贴了三个多月，上面已经写满了批注。最大的收获不是省时间，是我终于知道哪些活可以交出去、哪些不能。比如我发现"给客户的技术解释邮件"我从来不交给它写，因为那类话只有我知道该怎么说。这份判断，是贴了三个月才摸出来的，你也会有自己的版本。

本章金句

办公杂活吃掉的不只是每周十小时，是你被打断十一次之后再拼不回来的整块时间。

一场一小时会，冒出 1 到 3 条"未指定"是健康的，一条都没有反而可疑。

清洗任何表之前第一件事永远是复制副本，这条没有例外。

标题做成了手动格式，目录就生成不出来，导航窗格是空的，一眼就能看出来。

AI 做 PPT 的头号毛病是把整段文字搬到一页上，你给它设 120 字上限，它才会拆。

邮件规矩立死：它写草稿，我看一遍，我点发送。多花的 5 分钟，买的是不背锅。

谁签字谁验收，这条在办公桌上跟在车间里一样成立。

（下一章：个人效率与知识管理。把每周这十小时抢回来之后，下一件事是让那些散落在聊天记录、邮件和桌面上的经验，不再丢。）

第 8 章

个人效率与知识管理：让资料自己长腿

你找不到那份资料，跟记性没关系。你只有一堆文件夹，从来没有过一个库。

开场：38 分钟，找一份我自己写的东西

2026 年 3 月 11 日，星期二上午 9 点 40 分，我在找一份三个月前的试验结论。

问话的人很客气，就一句："上一版方案里，绕组最高温度做到多少度？"我记得清清楚楚，那个数我写在过一个 Excel 里，还配了一张曲线图。

我先翻桌面。214 个文件，按修改时间排，翻到第 3 屏就放弃了。再去 D 盘的"项目资料"，里面 1900 多个文件，子目录 11 层，我一层层点下去，看见三个叫"备份"的文件夹。然后我去翻企业微信，搜索"温度"，跳出来 47 条聊天记录，两条像。再翻邮件附件，搜"温升"，找到 5 封，附件都下下来看了，不是。

最后我在一个叫"新建文件夹 (3)"的目录里找到了它。文件名是"数据备份-最终版-真的最终.xlsx"。第 38 分钟。

这不是第一次。我后来数了一遍自己的家底：工作电脑加上同步盘、企业微信收藏、邮件附件和一个 U 盘，一共 3182 个文件躺在 6 个地方。命名全靠当天心情，目录全靠当时手快。每个月我大概要为这种事花掉 3 到 4 个小时，而且每次找完，心情都很差。

真正刺痛我的不是这 38 分钟。是我意识到：这 3182 个文件里，有大量我亲手做过、亲手验证过的结论，但在我这儿的价值等于零，因为我取不出来。一个取不出来的结论，跟没做过没有区别。

那个周末我花了两天的时间，把这套家底重做了一遍。这一章是那两天的作业，六个场景，从"建一个能问的库"到"每天自动给我送行业动态"。全部是我自己电脑上跑出来的，标注照旧，跑过就是跑过，没留档的我标★★。

3182 个文件躺在 6 个地方

找一个三个月前的试验结论，花了 38 分钟

它们都在这些地方

- 工作电脑：桌面 214 个文件
- 同步盘：项目资料 1900 多个
- 企业微信收藏：搜温度 47 条
- 邮件附件：搜温升找到 5 封
- 还有一个多年没清的 U 盘
- 命名靠当天心情，目录靠当时手快

一次查找付出的代价

- 先翻桌面
第 3 屏就放弃了
- 再翻同步盘
子目录 11 层，见三个备份
- 再翻聊天记录
47 条里两条像
- 再翻邮件附件
5 封全都不是
- 第 38 分钟找到
在一个叫新建文件夹的目录里
- 每月为这事花三到四小时
而且每次找完心情都很差

它们不是没有价值，是取不出来；一个取不出来的结论跟没做过没有区别

图 8-1 3182 个文件散在 6 个地方：找一份资料要穿 5 道门

图 8-1 资料散落示意（作者根据公开资料绘制）

本章六个场景速览

场景号	场景名	人工耗时	AI 耗时	可信用度
07	个人知识库：散落笔记变成一个能问的库	建库 2 天，查找 38 分钟/次	建库 90 分钟，查找 20 秒	★★★ 实测
08	长文拆解：200 页技术标准拆成检查清单	6 小时	45 分钟	★★★ 实测
09	待办与日程：混乱记录变成可执行清单	30 分钟/天	8 分钟	★★★ 实测
10	周报月报：从工作痕迹自动生成	90 分钟/周	15 分钟	★★★ 实测
11	文件命名与归档：几百个乱文件一次性归位	2 天（3182 个文件）	3 小时	★★ 跑通
12	信息订阅与日报：每天自动汇总行业动态	2 小时/天	25 分钟	★★★ 实测

六个场景有先后依赖：先建库（07），库里才有东西可拆（08）；待办和周报（09、10）吃的是同一批工作痕迹；归档（11）做完，库的质量会跳一个档次；日报（12）可以独立跑，也可以反过来喂给库。

场景 07 个人知识库：把散落的笔记和文件变成一个能问的库 ★★★

一句话背景

找一个三个月前的试验结论要 38 分钟，因为资料在 6 个地方、命名靠心情。建好库之后，同一个问题 20 秒出答案，还带文件名和页码。

名词小词典

个人知识库：不是文件夹，是“一堆文件 + 一张目录 + 一个能回答问题的入口”。文件夹只能靠你记路，库能靠问。

索引：给每个文件做一张身份证卡片，写上“这是什么、什么时候、关于哪个项目、结论是什么”。以后找东西先查卡片，不用翻文件。

检索：先从一堆卡片里找出相关的那几张，再回去翻原文。跟先翻完整本书再答题是两码事，后者容易编。

Markdown：一种纯文本写法，用 # 表示标题、- 表示列表。好处是任何软件都能打开，AI 读起来也最省事。

出处引用：每条回答后面标上“来自哪个文件、第几页、哪一段”。没有出处的答案，一律不算数。

开工前准备

1. 在 D 盘新建一个空文件夹，比如 D:\我的库。这一步是硬要求：只往库里复制，不动原件。原件在你确认库能用之前，一个字都别改。
2. 装好 WorkBuddy 并跑通第一个任务（第 2 章）。没跑通过的，先回去把第 2 章做一遍。
3. 把待入库的资料按来源分成三个子文件夹：01_本地文件、02_导出记录、03_邮件附件。分来源是为了后面出问题能定位。
4. 打开一个记事本，写下“我最常问的 10 个问题”。这 10 个问题就是你后面验收库的标准答案。
5. 留出 90 分钟不被打断的时间。建库不是拖进去就完事，中间要验收。
6. 准备一个备份盘，把原件整体拷一份。这一条在我这儿是铁律。

操作步骤

步骤 1：先只搬一个来源。我第一次建库时一口气把 3182 个文件全拖了进去，结果它处理到第 400 个就慢下来，我也不知道进度到哪了。第二次我改成先搬“本地文件”这一个来源，跑通，再搬第二个。

步骤 2：让它出清单，不要让它直接干活。先把家底盘清楚：一共多少文件、什么类型、哪些它读不了。

步骤 3：给每个文件写身份证卡片。这一步是建库的全部核心。卡片的四栏是固定的：日期、项目、一句话结论、关键词。

步骤 4：把卡片合并成一张总目录。一个 Markdown 文件，按项目或按时间分组。以后它先查这张目录。

步骤 5：问问题，验证。用你写下的那 10 个问题挨个问。问完必须看见文件名。

步骤 6：把这套动作存成技能。第 4 章讲过怎么把用顺手的流程存起来。我存了一个叫“往库里加文件”的技能，以后新资料进来，一句话就归位。



图 8-2 建库四步：盘家底、写卡片、并目录、问答验收

图 8-2 建库流程（作者根据公开资料绘制）

可复制提示词

步骤 2 用的盘点提示词：

我要把 D:\我的库 这个文件夹做成一个可以提问的个人知识库。
现在先不要建库，先做第一步：盘点。

请做三件事：

1. 递归扫描这个文件夹，统计文件总数，按扩展名分类列出数量和占比。
2. 列出你"读不了"或者"读起来会丢信息"的文件，说明原因。
比如扫描版 PDF、加密文件、超过 50 页的长文档、图片里的表格。
3. 列出文件名可疑的项：包含"最终""备份""新建""副本""旧"的，
以及内容相同但文件名不同的疑似重复文件（按文件大小和修改时间判断）。

输出一张表格，不要给建议，不要总结，先给我看事实。

步骤 3 用的卡片提示词：

现在给每个文件写一张"身份证卡片"，存到 D:\我的库_卡片 目录下，
一个文件对应一个 .md 文件，文件名与原文件同名。

每张卡片固定四栏，用下面这个格式，缺哪一栏就写"未知"，
不许猜，不许编，不许留空：

日期：YYYY-MM-DD（取文件修改时间，如从内容能看出更早的日期，写内容里的）

项目：【项目名称，看不出来就写"未标注"】

一句话结论：这个文件最后得出了什么结论。没有结论就写"无结论，仅为过程记录"。

关键词：3 到 6 个，用逗号分隔

下面是这个文件的摘要，不超过 150 字，只写事实，不写评价：

【摘要内容】

最后附一行：原文位置（文件名 + 页码或章节号）。

规则：

- 只读你真正打开过的文件，没打开过的不许写卡片。
- 卡片总数必须等于你实际处理的文件数，最后汇报"已处理 X 个，跳过 Y 个，跳过原因如下：....."。

步骤 5 用的提问提示词：

接下来我会向你提问，问题都关于 D:\我的库 里的资料。

回答规则，必须遵守：

1. 先检索 _卡片 目录下的卡片和 _总目录.md，再决定怎么答。
2. 每条回答后面必须给出出处：文件名 + 页码或章节号。
没有出处的结论不许输出。
3. 如果库里确实没有这个信息，直接回答"库里没有"，
并且列出你搜索过的关键词，方便我判断是我没存还是你没找到。
不许用通用知识补全，不许推测。
4. 如果找到多个互相矛盾的记录，全部列出来，标注各自的日期，
告诉我哪个最新，不要替我选。

我的第一个问题是：【三个月前那次温升试验，绕组最高温度是多少度？】

过程与判断点

跑完步骤 2，它给我的盘点结果里，几个数字我是对过的：3182 个文件里，Excel 占 41%，PDF 占 19%，图片占 14%，剩下是 Word、压缩包和一些我认不出的老格式。它标出 231 个"读不了"，主要是扫描版 PDF 和两张截图里的表格。这个数字我抽查了 20 个，19 个确实读不了，1 个其实能读，是它保守了。

步骤 3 的卡片，判断点是卡片数必须等于处理数。我第一次跑，它汇报"已处理 291 个"，但 _卡片 目录里只有 287 个文件。差 4 个，我让它补齐，它才承认那 4 个打开失败被静默跳过了。这就是我为什么在提示词里强制写"汇报已处理 X 个，跳过 Y 个"。

步骤 5 的验收最直观。我问了那 10 个问题，它答对了 8 个，每个都给了文件名。答错的两个，一个是因为那次试验的结论写在企业微信聊天里，我压根没导进库；另一个是它把 2024 年的一次老试验当成了三个月前的。第二个错很典型：日期判断错。所以我在提问提示词里加了"多个矛盾记录全部列出"。

整库建完耗时 90 分钟，其中我动手的部分约 15 分钟（复制文件、下发指令、验收），剩下是它自己在跑。

翻车清单

1. 它给没读过的文件也编了卡片。表现是卡片写得言之凿凿，但原文里根本没有那句话。补救动作：在提示词里写死"只读你真正打开过的文件"，并要求它汇报"处理数 / 跳过数"，两边必须对得上。发现对不上，让它重跑跳过的那批。

2. 同名文件被当成同一个。我有三个叫"最终版"的文件，它合并成了一张卡片，结论用的是最新的那份。补救动作：建库前先让它按"修改时间 + 文件大小"生成一份疑似重复清单，人工确认哪些是真重复、哪些是同名不同物。真重复的合并，同名不同物的强制改文件名。

3. 问到库里没有的，它照样编。这是最危险的一条。我问过一个库里根本没有的供应商交期，它回了我一个精确到天数的日期，还附了文件名。我按图索骥打开那个文件，里面根本没有日期。补救动作：提问提示词里强制"库里没有就说没有，并列出发过的关键词"，同时验收时随机抽 3 条答案，逐条打开原文核对。

验收标准与变体

验收标准：从你写下的 10 个问题里随机抽 5 个提问，至少 4 个能在 30 秒内给出"答案 + 文件名 + 位置"。再随机抽 3 条答案打开原文核对，内容必须一致。两条都满足，库才算能用。

变体一：客户与供应商资料库。把历年报价单、资质文件、往来邮件导进去。最省时间的问题是"这家供应商上一次报价是什么时候、多少钱、有效期到哪天"。

变体二：个人错题本。把我每次被打回的东西存进去：评审意见、被领导改过的报告、客户投诉。每周问一句"我最近三个月最常犯的错误是什么"，答案比我自己总结的准。

变体三：家庭资料库。保修卡、合同、体检报告、孩子的疫苗本，全扫进去。问"空调保修到哪天"比翻抽屉快。

变体四：项目交接库。换岗或离职前，把这个项目的全部资料建成一个库交给下一个人，附一张"常见问题 20 问"。这套我同事用过一次，交接会开了 40 分钟就结束了。

老兵说：库的价值不在建，在喂。我建完库的头两周，每天都会挑 3 个新文件丢进去，两周后它才真正好用。建完就放着不管的库，三个月后就跟你原来的文件夹没区别。

场景 08 长文拆解：一份 200 页技术标准拆成检查清单 ★★★

一句话背景

一份 200 页的技术标准，人工读完并拆成能照着检查的条目要 6 小时，还容易漏。交给 AI 之后 45 分钟出清单，我只需要抽查。

名词小词典

标准：规定产品必须做到什么、怎么验证的文件。它跟说明书的区别是：标准里的条款是要被逐条核对的。

条款号：标准里每一条的编号，比如 5.3.2。写检查清单必须带条款号，否则出问题没法追溯。

检查清单：把标准里"要做到 A"改写成"怎么确认做到了 A"的一张表。三栏：检查动作、判定标准、留什么证据。

强制与推荐：英文标准里 shall 表示必须做，should 表示建议做。这两个在检查时权重完全不同，混了会出大事。

引用文件：标准里写着"按 XXX 执行"的其他文件。正文没写细节，细节在引用文件里。

开工前准备

1. 标准的 PDF 原件，确认页码连续、没有缺页。我用过一次扫描件，缺了 12 页，拆出来的清单漏了一整章。
2. 确认标准版本和生效日期。同一份标准的新旧版条款号可能完全不同。

- 3. 一个 Excel 模板，列头定死：条款号、条款原文摘要、检查动作、判定标准、证据形式、强制/推荐、我方的现状。
- 4. 把 PDF 按章节切成 4 到 6 段。一次喂 200 页，它会偷懒跳过后半部分。
- 5. 记事本记下你最关心的 3 个章节，验收时重点查这几章。

操作步骤

- 步骤 1：先要骨架，不要条目。让它把目录和章节结构列出来，你先看它对不对。骨架错了，后面全错。
- 步骤 2：逐章抽条款。一段一段来，每段抽完立刻存一个文件，别攒到最后。
- 步骤 3：把每条条款改写成检查动作。这一步是场景的精髓。"绕组温升不得超过 120 K"要变成"在额定电压、85 摄氏度环境、连续运行 30 分钟后，用热电偶测绕组温度，读数与环境温度之差小于 120 K 判定通过，记录存测试报告第 X 页"。
- 步骤 4：标强制与推荐。让它把每条标成 shall 或 should，并给出原文依据。
- 步骤 5：去重与编号。它会把同一件事在两章里各抽一次，合并掉。
- 步骤 6：人工抽查 20 条。随机抽，对着原文逐条核。这 20 条的准确率就是整份清单的准确率。
- 一个我后来才补上的动作：拆完之后，把清单拿给真正要去执行的人看一遍。我在第 5 章拆完就自己签了字，结果试验室回来问："第 5.7 条说在环境温度 85 摄氏度下测，可我们的箱子最高只到 80 度，这条我们做不了。"清单里写着"做不了"和"没写"是两码事，前者是标准与我方能力有冲突，是要立刻上报的事。从那以后，我拆完必过一道执行方的手。

从条款到检查清单

标准里的话要改写成一个人能照着做的动作

条款原文	检查动作	判定标准	证据形式
绕组温升不得超过 120 K	额定电压、85 摄氏度环境、连续运行 30 分钟后，用热电偶测绕组温度	读数与环境温度之差小于 120 K	测试报告
看不出量的条款	写目视检查的部位与光照条件	目视无异常这类可判据描述	照片
环境要求 85 摄氏度	先确认试验箱的温度上限够不够	设备到不了就写做不了并上报	设备台账
附录里的条款	规范性附录同样要拆，不许跳过	按附录原文给出的量化指标判	第三方证书

判定标准不许写符合要求；拆完要请真正执行的人过一道手

图 8-3 从条款到检查动作：原文、改写、判定标准、证据四栏

图 8-3 条款转换示意（作者根据公开资料绘制）

可复制提示词

步骤 1 骨架提示词：

这是一份技术标准，共 200 页。先不要拆条款，先给我骨架。

请输出：

1. 标准编号、版本、生效日期（如果文件里有的话；没有就写“未标注”）。
2. 完整的章节结构，保留原文的条款编号，缩进表示层级。
3. 附录列表：编号 + 标题 + 是规范性附录还是资料性附录。
4. 引用文件列表：这份标准引用了哪些其他文件，标注条款号。

只输出事实，不要评价，不要建议。

如果某一页读不出来，明确告诉我“第 x 页读不出”，不许跳过不说。

步骤 3 改写提示词：

现在把第 5 章的条款改成检查清单，一条一行，输出成表格。

表格列头固定为：

条款号	原文摘要	检查动作	判定标准	证据形式	强制/推荐
-----	------	------	------	------	-------

转换规则，必须遵守：

1. 条款号照抄原文编号，不许自己重新编号。
2. “原文摘要”不超过 40 字，只摘关键限定条件（电压、温度、时间、次数），不许改写成你自己的话。
3. “检查动作”必须写成一个人可以照着执行的动作，开头用动词，包含具体条件：在什么工况下、用什么工具、测多久。
写不出具体条件的，在检查动作里写“条件不足，需查引用文件【编号】”。
4. “判定标准”必须是能判断通过与不通过的，有数字的写数字，没有数字的写“目视无异常”之类的可判据描述。不许写“符合要求”。
5. “证据形式”写清楚留什么：测试报告、照片、记录表、第三方证书。
6. “强制/推荐”按原文用词判断：shall / must / 必须 = 强制；
should / may / 建议 / 推荐 = 推荐。判断不了就写“待定”并附上原文用词。
7. 附录里的条款同样要拆，不许跳过。

如果某一条你读不懂，在“检查动作”里写“原文含义不清：+ 原文照抄”，不许用你的推测填满。

步骤 6 抽查提示词：

下面这份清单是我让你从标准第 5 章拆出来的。
现在做一次自查，逐条对照原文：

1. 数一数：原文第 5 章一共有多少条编号条款，清单里有多少条。
两个数字报出来，对不上就列出哪些条款漏了。
2. 检查有没有“清单里有、原文里没有”的条目，有的话删掉并说明。
3. 检查强制/推荐标注，凡是标了“强制”的，给出原文里的对应用词。
4. 列出你判断为“含义不清”或“需查引用文件”的条目，我要单独处理。

输出四段，每段一个数字或一张小表，不要写总结。

过程与判断点

我拆的那份是 200 页，正文 7 章加 3 个附录。骨架阶段它输出的章节结构，我拿原目录对了一遍，7 章全对，附录漏了附录 C。这就是为什么要先要骨架：附录 C 里有 14 条强制条款，如果直接拆条目，这 14 条会整个消失。

条款抽取阶段，一个关键判断点是条数。原文第 5 章一共 63 条编号条款，它第一次报上来 58 条。差的 5 条，4 条是嵌套在一段里的子条款（5.4.1 下面还有 a、b、c），1 条在表格里。我让它补，它补出了 4 条，表格里那条始终没抽出来，最后我自己手工加的。

改写阶段，判断点是判定标准能不能判。它第一版给我写了 17 条“符合要求”，这种条目等于没写，检查的人还是不知道怎么做。我在提示词里加了“不许写符合要求”，第二版这 17 条里 13 条补出了数字，剩下 4 条它标了“原文无量化指标”，这个标注本身也是有用的信息：说明标准这条写得不严，我方可以跟客户谈。

强制与推荐的标注，我抽查了 20 条，19 条对，1 条把 should 标成了强制。这条错在：它按“这条看起来很重要”来判断，不是按原文用词。所以我在抽查提示词里强制它“凡是标了强制的，给出原文对应用词”。

全程 45 分钟，其中我动手约 12 分钟。人工做同一件事，我上次是 6 小时，还漏了附录 C。

翻车清单

1. 跳过了附录。附录在 PDF 后半部分，它处理到后面容易敷衍。补救动作：先要骨架、强制它列出附录清单，再逐段喂附录，并要求它汇报“本段共抽出多少条”。

2. 把推荐当成强制。后果是我方按强制要求去准备，白白增加成本。补救动作：验收时把全部“强制”条目过一遍，每条都要它给出原文用词。

3. 条款号自己重新编了一遍。表现是清单里的编号跟原文对不上，后面追溯全线崩。补救动作：提示词里写死“照抄原文编号”，验收时随机抽 10 条，拿编号回原文翻，能翻到就算过。

验收标准与变体

验收标准：随机抽 20 条，逐条对照原文。条款号能翻到原文、检查动作能被第三方照着执行、判定标准能判通过或不通过，三个条件全中的条目 ≥ 18 条，清单才能用。再查一遍总条数，跟原文的编号条款数差不超过 5%。

变体一：合同拆成风险清单。把 80 页的供货合同拆成“我方义务、对方义务、违约条款、付款节点”四栏，每栏标注条款号和截止日。

变体二：专利拆成避让清单。把一份专利的权利要求拆成“技术特征”逐项列出来，跟自家方案逐项比对，判断有没有落入保护范围。

变体三：作业指导书拆成培训考题。把工位上的作业指导书拆成带答案的判断题，新员工考完就能上岗。

变体四：客户规范拆成报价依据。拆出来的强制条款里，凡是需要第三方检测或特殊工艺的，直接对应成本项，报价时不容易漏。

老兵说：拆完的清单，我一定打印一份带到评审会上，让写标准的人逐条确认。清单是拿来对账的，不是拿来存档的。我靠这份清单跟客户争下过两条条款，因为原文里那两条写的是 should。

场景 09 待办与日程：从混乱记录到可执行清单 ★★★

一句话背景

我的待办散在 4 个地方：桌面便签、企业微信给自己发的消息、邮件星标、还有一个靠脑子。每天早上重建一遍清单要 30 分钟，还照漏不误。

名词小词典

待办：一件还没做的事。它跟“想法”的区别是：待办必须有动作、有终点。

可执行：一条待办写到什么程度才算可执行？标准很简单：一个不了解背景的人看完，知道第一步要干什么。

原子任务：不能再拆的最小动作。“整理供应商数据”不是原子任务，“把 5 家供应商的报价导进 Excel”才是。

依赖关系：A 做完了才能做 B。写清单时要把这条标出来，否则排了也做不了。

截止日与承诺日：截止日是文件规定的，承诺日是你答应别人的。两个不一样时，按早的那个排，但要在清单里分开写。

等待中：需要别人先给你东西才能继续的任务。这类任务不能躺在待办里假装在做，要单独归一类。

开工前准备

1. 把 4 个来源的待办收集成一段纯文本。桌面便签直接复制，企业微信给自己发的消息长按多选转发到文件传输助手再复制，邮件星标逐个复制标题和第一句，脑子里的用语音输入说一遍。
2. 打开今天和明天的日历，把已有会议抄下来。
3. 15 分钟，以及一条纪律：这个过程中不处理任何一件事，只整理。
4. 准备一个固定的存放位置，我用的是一个叫 `todo.md` 的文件，放在同步盘里，手机也能看。
5. 准备一条自己的禁忌词表。我的是四个词：跟进、推进、优化、配合。清单里出现这四个词中的任何一个，就说明这条我还没想清楚，必须重写到能动手为止。

操作步骤

步骤 1：汇总，只要事实。把四段文本拼在一起丢给它，让它去重、不改内容。

步骤 2：归一化成原子任务。每条以动词开头，一条一个动作。

步骤 3：判断优先级。我用的判据只有一句："今天不做，谁会来找我？"有人找的排前面。

步骤 4：估时间，要区间。让它给每条估一个区间而不是一个数，比如 25 到 40 分钟。

步骤 5：排进日历。按会议的空档填，一次只排到当天容量的 70%。

步骤 6：挑出今天必须完成的三条。三条以上等于没有重点。

待办整理六步

收集、归一、判优先级、排进日历空档



每天这套流程 8 分钟，之前重建一遍清单要 30 分钟，还照漏不误

图 8-4 待办四步：收集、归一、判优先级、排进日历空档

图 8-4 待办整理流程（作者根据公开资料绘制）

可复制提示词

步骤 1 到 2 的收集与归一化提示词：

下面是我从 4 个地方收集来的待办原文，中间用 ---- 分隔。
 请做归一化，输出一张表格。

表格列头固定为：

| 编号 | 原子任务（动词开头） | 来源 | 依赖关系 | 估时（区间） | 类别 |

规则，必须遵守：

1. 一条原文如果包含多个动作，拆成多条原子任务。
 "整理供应商数据并回复采购"要拆成两条。
2. 每条以动词开头，比如"导出""发送""核对""打电话给"。
 不许出现"跟进""推进""优化"这类没有终点的词，
 遇到这类词，你必须改写成具体动作，改不出来就在该行后面标【需要我澄清】。
3. "来源"照抄：便签 / 微信 / 邮件 / 口述。
4. "依赖关系"写"无"或"需先完成编号 X"。
5. "估时"写区间，比如 25-40 分钟。不许写单个数字。
6. "类别"只能是四个值之一：今天做 / 本周做 / 等待中 / 也许以后。
7. 一条原文都不许丢。最后必须输出一行：
 "输入 X 条，输出 Y 条，其中拆分产生 Z 条，合并产生 W 条。"
 如果输出条数少于输入条数，必须单独列出被合并或被删除的原文，
 并说明原因，不许静默丢弃。
8. 不许新增任何我原文里没有的任务。

下面是原文：

【粘贴第一段】

【粘贴第二段】

步骤 5 到 6 的排期提示词：

这是我今天的日历，已被占用的时段如下：

9:30-10:30 项目评审会

14:00-15:00 客户电话会

16:30-17:00 部门周会

可用时段：8:30-9:30、10:30-12:00、13:00-14:00、15:00-16:30、17:00-18:30

下面是归一化后的待办表：

【粘贴上一步的表格】

请排一个今天的计划，规则：

1. 只排"今天做"类别的条目，其余不许排。
2. 总排入时长不超过可用时段的 70%，剩下的留作缓冲。
算完报一个数："可用 480 分钟，排入 X 分钟，占比 Y%。"
3. 有依赖关系的按依赖顺序排。
4. 需要整块时间的（估时上限超过 40 分钟的）排进连续时段，
不许跨会议切开。
5. 输出格式：
时段 | 编号 | 任务 | 估时
6. 最后单独输出三条"今天必须完成"，从排入的任务里挑，
挑选依据是"不做会产生外部后果"。每条后面写一句后果是什么。
7. 如果"今天做"排不进 70% 容量，把排不进的单独列出，
标明建议挪到明天，不许硬塞。

过程与判断点

我第一次跑这套，输入 23 条，它输出 19 条。少了 4 条。我按验收规则查"被合并或被删除的原文"，它列出来了：3 条它判断为重复（确实重复，我的便签和微信里各写了一遍），1 条它判断为"不是待办，是想法"。那 1 条其实是我答应要给同事的一份资料，它判错了。

从那以后我把"不许静默丢弃"和"输出条数对账"写进了提示词。这是这套流程里最重要的一条纪律：进多少出多少，对不上就问。AI 丢东西，从来不会告诉你它丢了。

估时这块，它第一次给的是单个数字，我照着排，结果两条估 30 分钟的实际各干了 70 分钟，下午全乱。改成区间之后我按上限排，准确率明显提高。它估时偏乐观是通病，我的做法是全部按上限排，实际做快了就提前收工。

排期那步，判断点是那个占比数字。我要求不超过 70%，它算出来经常是 85% 到 90%。它会先排满再说。所以我把这条写成了硬约束，并且让它把占比算出来报给我看，数字不对我就打回去重排。

举一个被它改写的真实例子。我原写的是"推进 B 项目电机选型"，它给标了【需要我澄清】。我回头想了三分钟才说清楚：我要做的第一步是"把三家供应商的选型表导进同一个 Excel，对比额定转矩和效率曲线"。改写完这条，我 20 分钟就做完了。原来那条"推进"在便签上躺了两周，我每次看见都绕着走。

从那以后我自己定了条规矩：清单里出现"跟进""推进""优化""配合"这四个词中的任何一个，就是我还想清楚。想不清楚的活，排进日历也没用。

三条"今天必须完成"加"后果是什么"这一栏，是我用得最久的设计。写上后果之后，我自己就不太敢拖了。有一条我记得很清楚：后果写的是"不给的话，张三下周的试验排不上台架"。那天我本来想拖到明天，看见这句，晚上七点还是发过去了。

每天这套流程 8 分钟，包括我复制粘贴的时间。之前是 30 分钟，而且经常漏掉微信里那条。

翻车清单

1. 它悄悄合并或删除了任务。后果是事情无声无息消失。补救动作：提示词里写死"输入 X 条，输出 Y 条，对不上必须列出被删原文"，每天扫一眼那一行数字。

2. 估时太乐观。后果是下午全盘崩。补救动作：强制给区间，排期时一律按上限，并在日历里留 30% 缓冲。

3. 把别人的事排进了我的日历。后果是我坐在工位上等一个永远不来的东西。补救动作：类别里必须有"等待中"，凡是"需要某人先给我 X"的一律归入等待中，单独列一张表，每周清一次。

验收标准与变体

验收标准：数一遍条数，输入数 = 输出数 + 被合并数，减出来的差额必须有明文说明。再随机抽 5 条，每条 30 秒内说得出口"做完没有"的判据，说不出就说明这条写得不具体，退回去重写。

变体一：会议纪要转待办。会议纪要贴进去，让它只抽"有人被指派了动作"的句子，每条带上责任人和截止日，没指派的不抽。

变体二：项目周会转任务分派。输入全组的讨论记录，输出按人分组的任务表，每人开头三条，直接发群里。

变体三：出差行程转日程。把机票、酒店、会议邀请、拜访安排贴进去，输出按天的行程表，标注每段之间的通勤时间和缓冲。

变体四：邮件收件箱清零。把 50 封未读邮件的标题和首句贴进去，输出"需要我回复的 / 只需知会的 / 可直接归档的"三类，需要回复的自动转成待办。

老兵说：我每天只挑三条必须完成，挑了两年。挑三条的好处不是做完三条，是逼我承认别的事今天做不完。以前我列 15 条，晚上看着 11 条没做完睡觉，第二天更不想开工。

场景 10 周报月报：从工作痕迹自动生成，不用回忆 ★★★

一句话背景

每周五下午，我要花 40 到 90 分钟回忆这一周干了什么，写出来的还常常漏掉最累的那件事。改成从工作痕迹自动生成之后，15 分钟交稿。

名词小词典

工作痕迹：你这周在电脑上留下的客观记录。文件修改时间、代码提交记录、日历条目、已发送邮件、聊天记录。它的好处是不会撒谎，也不会忘。

周报与日志的区别：日志是流水账，按时间记；周报是按结果归纳，要有结论。从痕迹生成的是日志，需要再加工成周报。

量化：把"推进了某项工作"改成"完成了 3 项，剩余 1 项，剩余项卡在 X"。周报里没有数字的段落基本都是废话。

已发送：只算你发出去的邮件，不算收到的。收到的是别人的工作，发出去的才是你的。

下周计划：必须有责任人和判据。写"继续跟进"的计划等于没写。

开工前准备

1. 确定你这周有哪几类痕迹。我的是五类：本地文件修改记录、Git 提交记录、日历条目、已发送邮件、企业微信导出的聊天记录。
2. 准备一个导出方法。文件修改记录用文件管理器的"修改日期"排序就能看；Git 提交记录在项目目录里用命令导出成文本；已发送邮件按本周筛选后导出标题清单。
3. 找一份你写得最满意的历史周报当格式模板。格式必须给，不然它每次写的风格都不一样。
4. 一个敏感词表：不能写进周报的项目代号、客户名、金额。提前写下来，让它在生成后过滤一遍。
5. 20 分钟，最好安排在周五下午 4 点以前。太晚了它生成的数字你没时间核。

操作步骤

步骤 1：采集痕迹。五类痕迹各导出一份文本，丢在同一个文件夹里。

步骤 2：让它归纳，不让他写。先出一张"这周做了哪些事"的清单，按项目分组。

步骤 3：你挑。从清单里挑出值得写进周报的。这一步不能省，AI 不知道哪些事在你老板眼里重要。

步骤 4：按模板生成。把挑出来的条目和模板一起给它。

步骤 5：数字校对。逐个数字回溯到痕迹来源。

步骤 6：删形容词。这一步我每次都做，它总会夹带"高效""顺利""稳步"。

周报生成六步

从工作痕迹自动生成，不用回忆

- 1 采集五类痕迹**
文件修改、提交、日历、已发送邮件、聊天
- 2 先归纳，不让它写**
输出一张本周做了哪些事的表
- 3 这一步人工挑**
它不知道哪些事在你老板眼里重要
- 4 按模板生成**
挑出来的条目加一份历史模板
- 5 逐个数字校对**
每个数字都要能回溯到痕迹
- 6 删形容词**
它总会夹带那些漂亮的副词

现在 15 分钟交稿，之前是 40 到 90 分钟，而且常漏掉最累的那件事

图 8-5 周报六步：采集痕迹、归纳、人工挑选、套模板、核数字、删形容词

图 8-5 周报生成流程（作者根据公开资料绘制）

可复制提示词

步骤 2 归纳提示词：

下面是本周（3 月 9 日周一 到 3 月 13 日周五）我的工作痕迹，分五个来源。
请先不要写周报，先做归纳。

输出一张表格：

| 项目 | 做了什么（动词开头） | 痕迹来源 | 日期 | 量化的结果 |

规则，必须遵守：

1. 只写痕迹里能印证的事。痕迹里没有的，一律不写。
2. "痕迹来源"必须具体到哪一类的哪一条，
比如"文件：温升报告 v3.xlsx, 3 月 11 日修改"
或"邮件：发给张三，主题《XX 方案确认》，3 月 12 日"。
3. "量化的结果"能写数字就写数字（页数、条数、次数、人数、天数），
写不出就写"无量化结果"，不许编数字。
4. 按项目分组，同一项目的事排在一起，组内按日期排。
5. 最后输出一行统计：
"共归纳 x 件事，其中 y 件有量化结果，z 件无量化结果。"
6. 不要写评价，不要写"高效""顺利""稳步"这类词。
7. 如果某个痕迹你看不出对应什么事，单列在最后，
标题写"以下痕迹我无法归类"，让我自己看。

下面是痕迹：

【来源一：文件修改记录】

【粘贴】

【来源二：Git 提交记录】

【粘贴】

【来源三：日历条目】

【粘贴】

【来源四：已发送邮件】

【粘贴】

【来源五：聊天记录导出】

【粘贴】

步骤 4 到 6 生成与校对提示词：

现在按下面的模板生成本周周报。

我只选了以下条目写进周报，其余的不许出现：

【粘贴你挑出来的条目编号】

模板如下：

===

一、本周完成

(按项目分小节，每节 3 到 5 条，每条一句话，格式：动作 + 结果 + 数字)

二、当前卡点

(每条格式：卡在什么环节 + 需要谁给什么 + 预计什么时候解决)

三、下周计划

(每条格式：动作 + 负责人 + 完成判据，判据要能判断做没做完)

===

生成规则：

1. 一、本周完成 里每一条必须带至少一个数字。
没有数字的条目不许出现在这个小节。
2. 不许出现"高效""顺利""稳步""积极""大力推进"这类形容词和副词。
写完了自己检查一遍，有就删掉。
3. 二、当前卡点 里的"需要谁给什么"必须写具体的人或角色，
写"需要相关部门配合"的，退回重写。
4. 三、下周计划 里的每条必须有完成判据，
写"继续跟进""持续推动"的一律重写成可判断的动作。
5. 全文写完后，做一次自查，输出一张对照表：
| 周报里的数字 | 来自哪条痕迹 | 是否一致 |
每一个数字一行。数字与痕迹对不上的，标"存疑"并说明。
6. 最后过滤一遍敏感词，遇到下面这些词一律替换成【脱敏】：
【粘贴你的敏感词表】

过程与判断点

我连续用了 14 周，攒下的经验是三条。

第一条：它归纳出来的事，比我回忆的多。第一周它归纳出 23 件，我自己回忆只能想起 11 件。漏掉的 12 件里，有 4 件其实是我那周最费劲的活，比如给一个老项目补了三天历史数据。这种事因为"不好看"，我回忆的时候自动跳过，但痕迹里抹不掉。

第二条：它造数字。第三周，周报里写"完成评审 5 次"，痕迹里只有 3 次。它把日历上 5 个评审相关的条目全算成了"已完成"，其中 2 个是取消了的和没参加的。从那以后我加了自查对照表这一条，每个数字都要回溯。这是这套流程的命门：周报里的数字错了，比周报写得差严重得多。

第三条：它会把计划写成完成。第六周，"下周计划"里写过一条"完成 B 样测试报告"，下一周的"本周完成"里它就把这条抄了过来，实际上那报告我根本没写完，因为测试台架坏了。它的逻辑是：上周说要做，这周就当做了。凡是它从历史周报里继承来的完成项，必须逐条核。

第四条，也是最让我意外的一条：它不知道哪些事重要。有一次它把"回复客户邮件"排在了"整理三个月试验数据"后面，理由是后者耗时更长。实际上那封邮件当天不回，客户就要升级投诉。我后来在排期提示词里加了"挑选依据是会产生外部后果"这一条，才算摆平。这件事教我一个道理：AI 能排序，但优先级的标准必须由你给，它猜不出来。

现在这套流程 15 分钟，其中 8 分钟是挑条目和核数字。之前是 40 到 90 分钟，而且靠回忆写的版本，我漏掉过一次关键节点，被领导当场问住。

顺带说一个我坚持了两年的做法：周报写完不发，放一晚上，周六早上再读一遍。隔一夜之后，那些"高效""顺利"的空话和不对的数字，你自己一眼就能看出来。让 AI 生成，让人睡一觉再签字。

翻车清单

1. 数字造得比真相漂亮。补救动作：强制输出"数字 痕迹来源 是否一致"的对照表，逐个回溯。周报里的数字，一个都不放过。

2. 把上周的计划当成这周的完成。补救动作：凡是出现在历史周报"下周计划"里的条目，一律标红，逐个打电话或翻文件确认是否真完成。

3. 敏感信息漏出去了。周报是要发给一群人的，客户代号、项目预算、未发布的节点都可能不合适。补救动作：提前写敏感词表，生成后强制过滤，并且自己再通读一遍。

验收标准与变体

验收标准：周报里每一个数字，都能在 30 秒内指出来自哪条痕迹。逐条过一遍"本周完成"，凡是只在历史周报里出现过、这周痕迹里找不到的，删掉。通读一遍全文，没有形容词和副词。三条全过才能发。

变体一：月度汇报。把四份周报的痕迹汇总，按月归纳，增加一节"本月与目标的差距"，每项目标后面写实际值和目标值。

变体二：项目阶段汇报。输入项目计划表和实际痕迹，输出"计划 vs 实际"对照表，超期的标红并写原因。

变体三：述职材料。输入半年的痕迹，按"做了什么、带来什么变化、学到什么"三栏归纳，每个结论后面挂一条痕迹。

变体四：报销说明。出差痕迹（日历、机票、酒店、拜访记录）自动生成行程说明，附上每一笔的时间地点事由。

老兵说：我从第三周起就不再靠回忆写周报了。靠回忆写出来的周报，会系统性地漏掉两类事：一类是不好看的事，一类是没做完的事。而这两类恰恰是你老板最该知道的。

场景 11 文件命名与归档：几百个乱文件一次性归位 ★★

一句话背景

3182 个文件，命名靠心情，目录十一层。人工理一遍保守估计两天，还理不干净。用 AI 出改名对照表再执行，3 小时。

说明：这套流程我在两个文件夹上完整跑通过，最大的一次是 3182 个文件。但那次我没有完整留档截图，所以按全书规矩标 ★★，不是 ★★★。

名词小词典

命名规则：文件名的固定格式。我用的是"日期-项目-内容-版本"，比如 2025-12-03_A项目_温升试验报告_v3.xlsx。有规则才能靠名字找文件。

对照表：一张 Excel，两列：原路径、新路径。所有改名操作先看对照表，不直接动手。

干跑：先只输出"我打算怎么改"，不真的改。改文件前必须干跑一次。

文件指纹：按文件大小加内容算出来的一串字符，用来判断两个文件内容是否真的一样。文件名相同不代表内容相同。

回滚：改错了能退回原样。做法是先备份，且对照表保留原路径列。

开工前准备

1. 完整备份。把整个文件夹复制到另一个盘。这一条没有商量余地。我第一次跑没备份，它把两个内容不同但同名的文件改成了同一个名字，第二个直接覆盖了第一个，我丢了一份数据。
2. 一份命名规则，自己先写出来，别让它替你定。我的是四段式：日期、项目、内容、版本。
3. 一个临时工作目录，用来放对照表和日志。
4. 一个文件管理器，能按修改时间排序。改名之后要靠它验收。
5. 心理准备：不要一次处理超过 5000 个文件，太多就分批。

操作步骤

步骤 1：备份，然后确认备份能打开。随机打开 5 个备份文件。

步骤 2：生成文件清单。让它扫描全目录，输出一张含"路径、大小、修改时间、扩展名"的表。

步骤 3：让它提命名方案，但只做参考。它提的方案我从来不全盘接受，通常要改项目名和版本号规则。

步骤 4：人工定稿规则。把规则写死成一段文字，后面所有操作都引用这一段。

步骤 5：先出对照表，不改名。这一步是干跑，输出 Excel。

步骤 6：人工抽查 50 行。重点看：日期对不对、项目名有没有认错、版本有没有乱编。

步骤 7：执行改名。按对照表逐行改，输出到日志。

步骤 8：抽查。随机搜 30 个文件名，看能不能按名找到。

文件命名与归档八步

几百个乱文件一次性归位



图 8-6 归档八步：备份、列清单、提方案、定规则、出对照表、抽查、执行、复查

图 8-6 文件归档流程（作者根据公开资料绘制）

【待补：界面路径，以实际版本为准。批量重命名在不同系统上的入口位置不一样，本章不写具体菜单路径，改用 WorkBuddy 直接读写文件的方式执行。】

可复制提示词

步骤 2 到 3 的清单与方案提示词：

我要整理 D:\项目资料 这个文件夹，先不要改任何东西。

第一步：扫描整个文件夹，输出一张清单表到 D:\整理\清单.xlsx。

列头固定为：

| 序号 | 完整路径 | 文件名 | 扩展名 | 大小(KB) | 修改时间 | 目录层级 |

第二步：基于这份清单，提一套命名规则建议，输出成文字，包含：

1. 你建议的文件名格式（用占位符表示，比如 YYYY-MM-DD_项目_内容_版本）。
2. 你从现有文件名和路径里识别出了哪几个项目名，各有多少文件归入。
识别不出来的文件有多少个，列出来。
3. 你打算怎么处理这些特殊情况：
 - 内容相同但文件名不同的文件
 - 文件名相同但内容不同的文件
 - 文件名里带"最终""备份""副本""旧""新建"的
 - 没有扩展名的文件
 - 系统文件或隐藏文件
4. 你认为这套规则的漏洞在哪，列出至少 3 条。

第三步：输出一张"风险清单"，列出你不建议自动处理的路径，说明原因。这些路径我会手工处理。

全程不许改动任何文件，只输出表格和文字。

步骤 5 到 6 的对照表提示词：

现在按下面这条命名规则生成改名对照表，只生成对照表，不要执行任何改名操作。

命名规则（照抄，不许改动）：

- 格式：YYYY-MM-DD_项目名_内容描述_版本.扩展名
- 日期取文件修改时间，格式必须是四位年两位月两位日
- 项目名只能从这里选，不许自己发明：
A项目 / B项目 / C项目 / 其他
判断不了归属的，项目名写"其他"
- 内容描述不超过 15 个字，只写这份文件是什么，
看不出内容的文件，写"待确认"，不许猜
- 版本：能从文件名或内容看出的写 v1、v2 这样的格式，
看不出的统一写 v1
- 扩展名保持原样，一律不改

输出到 D:\整理\对照表.xlsx，列头固定为：

| 序号 | 原完整路径 | 原文件名 | 新文件名 | 新完整路径 | 是否移动目录 | 备注 |

规则：

1. 目录结构也一并整理：按"项目名\年份\"两级重新组织，
"其他"类放进"其他\年份\"。
2. 遇到下面三种情况，在"备注"列写清楚，并且不要改这个文件的名字：
 - 目标路径已存在同名文件
 - 内容描述判为"待确认"
 - 文件正在被占用或无法读取
3. 全部行处理完后，输出统计：
"共 X 行，建议改名 Y 行，跳过 Z 行，潜在冲突 N 行。"
4. 如果发现两个文件内容完全相同（按大小和开头内容判断），
在新文件名后加 _dup 后缀，并在备注写"疑似重复，未自动合并"。
不许自动删除任何文件。
5. 新文件名总长度（含路径）不许超过 200 个字符。
6. 新文件名里不许出现这些字符：\ / : * ? " < > |

步骤 7 的执行与日志提示词：

对照表已经人工抽查过，现在执行改名。

执行规则，必须遵守：

1. 严格按 D:\整理\对照表.xlsx 逐行执行，
"备注"列不是空的那些行，一律跳过不处理。
2. 用移动的方式改名，不要用复制，避免产生两份。
3. 每处理完一行，往 D:\整理\执行日志.txt 追加一行：
序号 | 原路径 | 新路径 | 成功/失败 | 失败原因
4. 遇到失败不要中止，记录原因后继续下一行。
5. 全部执行完，输出统计：
"共 X 行，成功 Y 行，失败 Z 行，跳过 N 行。"
并把失败的行完整列出来。
6. 执行完不要删对照表，我要留着做回滚依据。
7. 如果发现目标位置已有文件，不许覆盖，
在新文件名后加 _1 后缀，并在日志里标注。

过程与判断点

3182 个文件那次，几个数字我记到现在。

它识别出的项目名有 4 个，其中 3 个对，1 个是它自己发明的：把一批"NVH"开头的文件归成了一个叫"NVH专项"的项目，实际上那批文件分属两个不同项目。判断不了归属的有 611 个，占 19%，这个比例让我意外地高。我的补救是：判断不了的全归"其他"，宁可不归，也不归错。

对照表生成完，行数对上了：3182 行。建议改名 2471 行，跳过 711 行。跳过的里面，"待确认"占大头，402 个。冲突 87 行。

我抽查了 50 行，问题集中在两处。一是日期取错：它有 6 行取的是文件创建时间而不是修改时间，我后来在规则里写死了"取修改时间"。二是版本号乱编：有一批文件它标成了 v1 到 v7，实际上那批文件根本不是版本关系，是七个不同零件的数据，只是名字里都带"报告"。版本这一栏我后来改成"看不出的一律写 v1"。

执行阶段 3 小时，其中它跑改名约 40 分钟，其余时间是我在抽查和修规则。成功 2455 行，失败 16 行，失败原因是 12 个文件被占用（我自己的 Excel 开着），4 个权限不足。

改完之后我做了验收：随机想 30 个文件，按新名字搜，30 个全部一次命中。改之前同样的测试，我只想起 18 个文件在哪，其中 11 个找对了。

翻车清单

1. 没备份就动手，重名文件被覆盖。我栽过一次，丢了一份数据。补救动作：备份放第一步骤，且备份完随机打开 5 个文件确认能打开。执行提示词里写死"不许覆盖，加后缀"。

2. 文件名太长或有非法字符。深层目录加上长名字，总长度很容易破 200 字符，Windows 上会失败。补救动作：规则里限定总长度，且禁用 \ / : * ? " < > | 这九个字符，让它生成后自己校验一遍。

3. 它替你发明了项目名。后果是文件被归到了一个不存在的项目下，比不归还糟。补救动作：规则里把项目名列成枚举，写死"只能从这里选，判断不了写其他"。

验收标准与变体

验收标准：先查三个数字，清单行数 = 对照表行数 = 执行日志行数。再随机想 30 个文件按新名搜索，命中 ≥ 28 个。最后按新目录结构随机翻 10 个文件，确认内容跟预期一致。三条全过才算完成，对照表和备份保留至少一个月。

变体一：照片归档。按拍摄日期和地点重命名，输出"年月日_地点_序号.jpg"，同一天的照片自动归入月份文件夹。

变体二：试验数据归档。台架导出的数据文件通常叫一堆编号，按"日期_样件号_工况_通道数"重命名，并生成一张数据索引表。

变体三：邮件附件归档。把某个项目往来邮件里的附件全部导出，按"日期_发件人_主题关键词"命名，生成一张附件清单附在项目目录下。

变体四：扫描件归档。一批没有文字层的扫描 PDF，让它逐页读出标题行再命名，读不出的统一进"待人工"文件夹。

老兵说：这套活我只在周末做，且做完当周不动。原因很实在：改完名的头几天，我已经形成的肌肉记忆还会去找旧名字，如果立刻开始新项目，新旧两个目录会同时长，一周后比整理之前更乱。

场景 12 信息订阅与日报：每天自动汇总行业动态 ★★★

一句话背景

我在一个行业社区里做日报，每天要扫二十来个信息源。人工做一天两小时，AI 做 25 分钟，而且它比我更能忍住不看标题党。

名词小词典

订阅源：一个会持续更新内容的网址。你不用每天去打开，它更新了会告诉你。常见的有 RSS、公众号、行业网站的更新栏目。

RSS：一种老但好用的订阅格式。网站把更新内容按固定格式吐出来，订阅工具直接读，不用打开网页、不用看广告。

去重：同一条消息被多个源转发，只留一条，标注"经 N 个源转载"。

摘要：两句话。第一句说发生了什么，第二句说跟我们有什么关系。超过两句没人看。

定时执行：让这套流程每天固定时间自己跑一遍，跑完把结果放在一个固定位置。第 6 章讲过多代理，这里用的是更简单的定时任务思路。

开工前准备

1. 一份订阅源清单，10 到 20 个。少于 10 个不够看，多于 20 个你不会看。我用的是 16 个：4 个行业协会、5 个企业技术博客、3 个标准更新页、2 个政策发布页、2 个海外技术媒体。

2. 一个固定归档目录，比如 `D:\日报\2026\03\`。每天一个文件，文件名带日期。
3. 一个固定时间。我设在每天早上 7 点 30 分，跑完我上班刚好能看。
4. 一份"我关心什么"的清单，5 到 8 条关键词。这份清单决定它筛掉什么。
5. 一份"不关心什么"的清单。同样重要，比如招聘广告、会议通知、纯观点文章。
6. 一个推送出口。我发在社区群里，你也可以只存本地。

操作步骤

步骤 1：列源。把 16 个源写进一个文本，每个标注类型和可信度。

步骤 2：抓取。让它按源逐个取回最近 24 小时的内容。取不回来的要明说。

步骤 3：去重。同一事件多源转载的合并，保留最先发的那个源。

步骤 4：按主题归类。我固定分四类：技术进展、标准与法规、企业动态、市场数据。

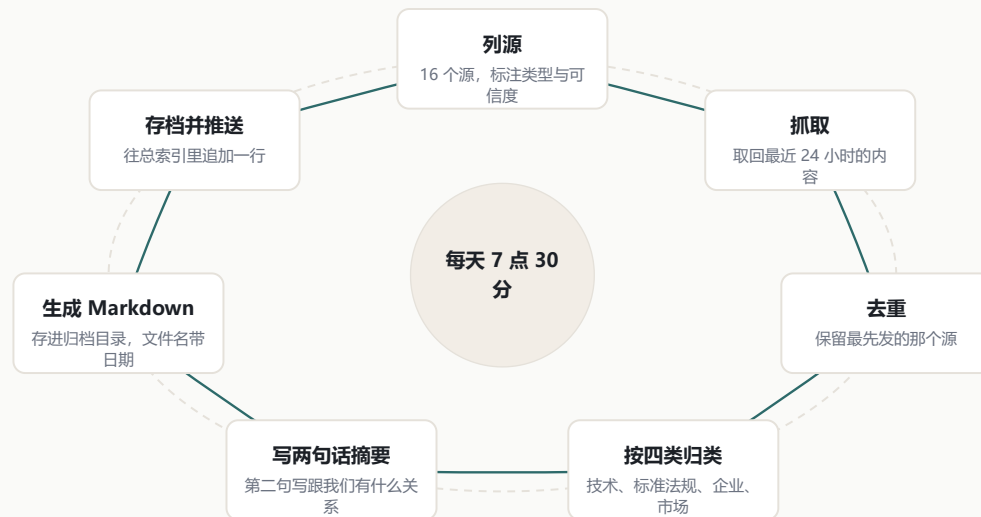
步骤 5：逐条写摘要。两句话，第二句必须写"跟我们有什么关系"。

步骤 6：生成 Markdown。存进归档目录，文件名带日期。

步骤 7：存档并推送。同时往总索引文件里追加一行，方便以后检索。

行业日报七步

定好一次，之后每天自己跑



全程 25 分钟，我通读一遍加抽查 5 条链接约 5 分钟，人工做同一件事平均 2 小时

图 8-7 日报七步：列源、抓取、去重、归类、写摘要、生成、推送

图 8-7 日报生成流程（作者根据公开资料绘制）

可复制提示词

步骤 2 到 4 的抓取与归类提示词：

现在生成 2026 年 3 月 12 日的行业日报。

订阅源清单在 D:\日报\订阅源.txt，每行一个，格式是：

源名称 | 网址 | 类型 | 可信度(高/中/低)

请执行：

1. 逐个访问这些源，取回最近 24 小时（3 月 11 日 0 点到 3 月 12 日 0 点）的内容。
2. 对每个源，输出一行状态：
源名称 | 访问成功/失败 | 取回条数 | 失败原因
访问失败的源不许静默跳过，必须列出来。
3. 把取回的条目做去重：标题高度相似或事件相同的合并为一条，
保留发布最早的源，并在后面标注"另经 x 个源转载"。
4. 按下面四类归类，不属于任何一类的归入"其他"：
 - 技术进展
 - 标准与法规
 - 企业动态
 - 市场数据
5. 用下面的关键词清单做一次筛选，标出相关度：
我关心的关键词：【粘贴你的 5 到 8 条】
我不关心的内容类型：【粘贴你的排除清单：招聘、会议通知、纯观点等】
相关度分三档：高（直接相关）/ 中（间接相关）/ 低（边缘）。
6. 输出统计：
"共访问 x 个源，成功 y 个，取回 z 条，去重后 n 条，
其中高相关 A 条、中相关 B 条、低相关 C 条。"
7. 暂时不要写摘要，先把这张归类好的表给我看。

步骤 5 到 7 的摘要与成稿提示词：

基于上面归类好的条目，生成今天的日报正文。

只保留“高相关”和“中相关”的条目，低相关的放进文末“其他动态”里，
每条一行只写标题和来源。

正文格式：

行业日报 2026-03-12

今日要点

(3 条，每条一句话，从高相关里挑最重要的)

技术进展

【条目标题】

第一句：发生了什么（客观事实，含数字或时间点）。

第二句：跟我们有什么关系（具体到对我们哪类工作有影响）。

来源：【源名称】 | 链接：【URL】 | 发布：【日期】 | 相关度：【高/中】

(其余三类同样格式)

今日数据

(把正文里出现的数字单独列一张表：指标 | 数值 | 来源 | 日期)

其他动态

(低相关条目，标题 + 来源，一行一条)

今日缺口

(列出访问失败的源，以及你认为本期可能漏掉的方向)

写作规则，必须遵守：

1. 每条摘要两句话，不许超过两句话。
2. 第一句只写事实，不许出现“据悉”“有消息称”“业界认为”这类转述腔。
原文里写了“计划”“拟”“预计”的，摘要里必须保留这些限定词，
不许写成已经发生的事。
3. 第二句必须具体到“对我们哪类工作有影响”，
写“值得关注”“有参考意义”这种空话的，退回重写。
4. 每条必须有链接。没有链接的条目不许进正文。
5. 不许出现空话和套话。写“引领”“全新升级”“值得期待”的，退回重写。
6. 数字单独进“今日数据”表，正文里出现的数字不许与表里的不一致。
7. 全文写完自查一遍，输出：
“正文 X 条，其中两句话的 Y 条，带链接的 Z 条，数字 N 个。”
三个数字必须对得上。

最后：

- 正文存到 D:\日报\2026\03\2026-03-12.md

- 往 D:\日报\总索引.md 追加一行：

日期 | 条目数 | 今日要点第一条 | 文件路径

过程与判断点

这套流程我跑了 200 多期，几个数字稳得很。

一是源的成功率。16 个源里，平均每天有 1 到 2 个访问失败。失败原因八成是网站改版或临时抽风。我的规矩是：连续三天失败的源，我人工去看一眼，是源死了就换掉。这个"今日缺口"小节是我特意加的，因为日报最怕的不是内容少，是你不知道漏了什么。如果它默默跳过一个失败的源，你以为今天行业里没新闻，实际上是你没看见。

二是去重比例。平均取回 60 到 80 条，去重后剩 25 到 35 条。也就是说，一大半是同一件事的互相转载。人工做的时候，我常常把转载当成两件事，还以为今天信息量很大。

三是相关度分布。高相关平均 4 到 6 条，中相关 8 到 12 条，剩下都是低相关。我只看前两类，一天真正要看的也就 12 到 18 条。

判断成不成的硬指标是数字对不对得上。我在提示词里写了三个自查数字：正文条数、两句话的条数、带链接的条数。这三个必须一致。加这条是因为它经常把重要条目写成四句话，一写长，读者就不看了。

"今日要点"只放三条，是我试过五条、十条之后定下来的。放十条的那几期，后台数据显示点开率反而更低，因为读者要自己挑重点，等于我把活又推回给了他。三条的好处是：读者不用挑，你替他挑了，你就得对自己的判断负责。

摘要失真这一条我抓得最狠。有一次它把某家企业"计划明年投产"写成了"投产"，我差点在群里发错。所以规则里写死了：原文有"计划""拟""预计"的，摘要必须保留限定词。这条规矩救过我三次。

全程 25 分钟，其中它跑 20 分钟，我花 5 分钟通读一遍、点开 5 条链接核对。人工做同一件事，我记录过，平均 2 小时。

翻车清单

1. 源抓不全，它还装作抓全了。后果是你以为今天没新闻。补救动作：强制输出每个源的成功失败状态，并要求文末有"今日缺口"小节，失败的源必须写明。

2. 标题党混进正文。有些源专发耸动标题，内容空洞。补救动作：在提示词里给源的"可信度"分档，低可信度源的条目必须在来源后面标注，我人工决定要不要放。

3. 摘要把"计划"写成"已发生"。这是最严重的一条，会让你在群里发错信息。补救动作：规则里写死保留限定词，并且每天随机点开 5 条链接，对照原文看摘要有没有夸大。

验收标准与变体

验收标准：每天通读一遍，随机点开 5 条链接对照原文，摘要不能夸大、不能漏限定词。三个自查数字（条数、两句话条数、带链接条数）必须一致。连续三天"今日缺口"里出现同一个源，就人工去处理这个源。

变体一：竞品动态监控。订阅源换成竞品的官网、招聘页、专利公开页。招聘页特别有用，招什么岗位往往暴露他们在做什么。

变体二：标准更新监控。只盯标准发布机构的更新页，一旦有你关注的标准号更新，立刻提醒。

变体四：客户舆情。订阅客户所在行业的媒体，出现客户公司名的条目单独标红，每周汇总。

变体五：内部日报。 订阅源换成内部的文档库、项目看板、测试系统，每天汇总“哪些项目动了、哪些指标变了”。

老兵说：日报做了 200 多期，我最大的收获不是省时间，是我终于敢说“这个方向最近三个月没动静”。以前凭印象判断行业动向，现在有 200 多个文件可以翻，一搜就有。判断力是从可检索的记录里长出来的。记性给不了你这个。

小白词典

个人知识库：一堆文件加一张目录加一个能提问的入口。文件夹只能靠你记路，库能靠问。类比：文件夹是抽屉，库是带索引卡的档案柜。

索引：给每个文件做一张身份证卡片。类比：图书馆的卡片柜，先查卡片再找书。

检索：先从卡片里找出相关的几张，再回去翻原文。类比：开卷考试，先翻书再答题，不是闭卷硬背。

出处引用：每条答案后面标上来自哪个文件第几页。没有出处的答案不算数。类比：论文的参考文献。

Markdown：一种纯文本写法，用 # 表示标题、- 表示列表。类比：不带格式的记事本，但谁都能打开。

条款号：标准里每一条的编号，比如 5.3.2。类比：法律条文的第几条第几款。

强制与推荐：英文标准里 shall 是必须做，should 是建议做。类比：交通法规里的“禁止”和“建议”。

原子任务：不能再拆的最小动作。类比：菜谱里的一步，不是“做一道菜”。

依赖关系：A 做完了才能做 B。类比：先和面才能擀皮。

工作痕迹：你在电脑上留下的客观记录，文件修改时间、邮件、日历。类比：手机的步数记录，不靠你回忆走了多少。

RSS：网站按固定格式吐出更新内容的订阅方式。类比：报社给你送报上门，不用你每天去报摊。

去重：同一条消息被多个源转载，只留一条。类比：五个人跟你说了同一件事，你只记一次。

对照表：一张 Excel，两列，原路径和新路径。类比：搬家清单，先写好再搬，不边搬边想。

干跑：先只输出“我打算怎么改”，不真动手。类比：先在纸上排一遍版，再往墙上钉。

定时执行：让流程每天固定时间自己跑一遍。类比：定了闹钟的咖啡机，到点自己煮。

你的岗位怎么用

设计工程师：先做场景 08。你手边一定有几份客户规范或行业标准，拆成检查清单之后，评审的时候不用翻原文。再做场景 07，把历次评审意见建成库，新方案出来先问一遍库，能挡掉一半重复问题。

试验工程师：先做场景 11。台架数据文件命名混乱是通病，归档做完之后，场景 10 的周报可以直接吃文件修改时间。场景 12 换成订阅试验方法和设备厂商的更新页，也有用。

质量工程师：场景 08 就是给你写的。检查清单是你的主武器，拆完之后把它变成审核表。场景 07 建一个不合格案例库，新问题来了先问库里有没有类似的。

采购 / 供应商管理：场景 12 改成供应商动态监控，订阅供应商官网、招标公告、行业媒体，出现供应商名字就标红。场景 07 建供应商资料库，问"这家上次报价多少"比翻邮件快十倍。

项目经理：场景 09 和场景 10 是你的杠杆。待办从会议纪要自动抽，周报从痕迹自动生成，每周省下来的时间拿去盯风险。场景 07 建一个项目历史库，接手新项目先问"以前类似项目踩过什么坑"。

行政 / 人事 / 完全不碰技术的岗位：从场景 11 开始。几百个乱文件一次性归位，见效最快、最没有技术门槛。再做场景 09，把散在微信和便签里的事理成一张能执行的清单。

本章交付物

一张个人资料家底表。做完它，你第一次知道自己到底有多少文件、散在几个地方、哪些能读哪些不能读。

怎么做：

1. 在 D 盘新建 `D:\整理`，把你最乱的那个文件夹复制进去（只复制，不动原件）。
2. 用场景 07 步骤 2 的盘点提示词，让它扫一遍。
3. 把三个数字抄下来：文件总数、读不了的文件数、名字可疑的文件数。
4. 看一眼"名字可疑"那一栏。如果里面带"最终""备份""副本"的文件超过总数的一成，说明你的命名已经失控，先做场景 11。
5. 如果"读不了"的文件超过两成（多数是扫描件），先想办法补文字层，否则建库的效果会打折。
6. 把这张表打印出来贴在显示器边。三个月后再跑一次，对比两组数字。

我自己的数字是：3182 个文件，231 个读不了，名字可疑 407 个。三个月后再跑，库里 3400 个文件，读不了 88 个，名字可疑 9 个。

本章金句

一个取不出来的结论，跟没做过没有区别。

文件夹只能靠你记路，知识库能靠问。

AI 丢东西，从来不会告诉你它丢了。进多少出多少，对不上就问。

周报里的数字错了，比周报写得差严重得多。

靠回忆写出来的周报，会系统性地漏掉两类事：不好看的事，和没做完的事。

改文件名的第一步骤是备份，第二步骤是确认备份能打开。

日报最怕的不是内容少，是你不知道漏了什么。

（下一章：需求与项目定义。资料理顺之后，下一个问题是怎么把一句模糊的客户需求变成能开工、能验收的东西。）

第 9 章

需求与项目定义：把客户那段话翻译成能开工的任务

一份 40 页的客户需求文件，真正决定项目成败的条款通常不超过 12 条，剩下的 28 页是给法务和将来吵架用的。

同一份文件，三个人读出三种理解

2025 年秋天，我们接到某欧系平台的一个微电机减速机构询价包。客户发来的技术需求文件 47 页，附件里还有一份 12 行的工况参数表。

周三下午两点，三个人坐在会议室里各自读这份文件，读了 90 分钟。散会的时候，我们三个人脑子里的项目是三个不同的项目。

项目经理老张关心的是交付：样件 30 套，2026 年 6 月交第一批，量产节点在 2027 年 3 月。他记了满满一页日期。

试验工程师小陈关心的是试验：噪音小于 40 dBA，测距 500 mm，耐久性按客户企标做。他记了七条试验项。

我关心的是结构和空间：中心距 34 到 35 mm，外径小于 60 mm，输出扭矩 10 Nm。我在纸上画了个草图。

散会前，老张问了一句："这活能接吗？"

我说能，小陈说噪音没把握，老张说时间够。

第二天上午，我把那 12 行工况参数单独拎出来，放在一起看：输入 7200 rpm，输出 72 rpm，减速比 100 比 1，输出扭矩 10 Nm，外径小于 60 mm，材料是钢蜗杆配 POM 蜗轮。

100 比 1 的单级蜗轮蜗杆，蜗轮要做到 100 齿，按模数 0.7 算外径 71.4 mm，超过 60 mm 的限制将近 11 mm。这一条和那一条，隔着 8 页纸，谁也没把它们摆在一起。

还有一条：10 Nm 输出扭矩下，POM 输出齿轮的弯曲应力算出来 97 MPa，而 POM 在 10 万次循环下的许用值只有 40 到 50 MPa。差了接近一倍。

我把这两条发到群里，90 分钟的会白开了。

后来我用 AI 重做了一遍这件事：把 47 页文件丢进去，让它把散落在各章节的参数全部抽出来，摆进一张表，再做交叉检查。8 分钟，它给我列出了 12 条冲突和待确认项，上面那两条都在里面，它还多找了四条我们三个人都漏掉的。

从那天起，我不再自己从头读需求文件。我让 AI 先把条款摊平，人只做判断。

摊平这件事 AI 做得比人好，因为它不会因为"这页看着像法务条款"就跳过去。判断这件事 AI 做不了，因为它不知道 71.4 mm 那个数字意味着模具要重开。

本章六个场景速览

场景	场景名	人工耗时	AI 耗时	可信度
13	客户需求拆解：一份 SOR 变成内部任务清单	3 到 4 小时	8 到 15 分钟	★★ 跑通
14	技术标准提取：从标准里摘出跟你产品有关的条款	半天	20 到 30 分钟	★ 调研
15	项目计划生成：里程碑、依赖关系、关键节点	1 个工作日	25 分钟	★★ 跑通
16	风险清单辅助：把经验变成结构化的失效模式表	2 天（含评审）	40 分钟	★★ 跑通
17	技术方案对比：两套方案摆在一起比，比什么先定好	1.5 个工作日	30 分钟	★★★ 实测
18	行动项跟踪：会议决定变成有人有日期的任务	40 分钟每次	3 分钟	★★ 跑通

可信度的含义在第一章讲过：★★★ 是我本机完整跑过、留了数据和截图；★★ 是流程跑通过；★ 来自公开资料调研，没在我自己的项目上完整验证过。场景 14 我标 ★，因为标准条款这件事，我做过的版本是拿公开标准做的练习，没有在正式交付项目上用它出过最终结论。

下面六个场景按项目推进顺序排，你可以整章顺着抄，也可以只挑一个场景先试。

场景 13 客户需求拆解：一份 SOR 变成内部任务清单 ★★

一句话背景

客户发来的 SOR 通常 40 到 60 页，条款散落在十几个章节里，同一个参数在不同章节口径还不一致。人工拆成一份能分派下去的内部任务清单，我自己的记录是 3 到 4 小时，而且是那种做完了不敢保证没漏项的 3 到 4 小时。

名词小词典

SOR (Statement of Requirements, 需求说明书)：客户写给供应商的"我要什么"文件。它不是技术方案，是将来验收的依据，也是吵架时的凭据。

工况参数：产品干活时的外部条件，比如扭矩、转速、温度范围、电压范围、循环次数。它们决定了设计的边界。

可验证性：这条需求能不能用测量、试验、计算或查证来判定合不合格。写不出判定方法的条目，等于没写。

接口边界：你的零件跟别人零件交界的地方，比如安装法兰尺寸、接插件型号、通信协议、线束长度。

CSR (Customer Specific Requirements, 客户特殊要求)：客户在通用标准之外额外加的条款。它最容易被漏，因为它常常写在附件的最后一页。

开工前准备

1. SOR 原件，PDF 或 Word 都行。不要用扫描件，扫描件它会把 0 读成 8，把小数点读丢。
2. 把附件也一起备齐。客户经常把关键参数表放在附件而不是正文里。
3. 一份内部岗位清单：设计、试验、质量、采购、项目、工艺。让它只能从这六个里挑，它就不会造出"市场部"这种岗位。
4. 你要有一份同类项目的旧任务清单做参照，用来比对条目数量。
5. 模式选 Plan。先让它出方案，你看过结构再让它动手写文件。

操作步骤

第一步：把 SOR 拖进对话框，先不给任务，只让它做一件事：报文件结构。

第二步：看它报出来的章节数和页数对不对。页数对不上，说明它没读全，先解决这个再往下走。

第三步：把下面的提示词整段发过去。

第四步：它返回三张表。先看第二张"冲突与待确认表"，这张表价值最高。

第五步：抽查条目表里的 5 条原文摘录，回到 PDF 里搜索，确认逐字对得上。

第六步：确认没问题，让它另存 Excel，导入你们的项目管理系统分派。

一份 SOR 变成三张表

先把条款摊平，人只做判断



47 页的文件抽出 143 条，合理区间是 100 到 200 条，少于 80 说明它跳读了

图 9-1 一份 SOR 变成三张表：条目表、冲突表、漏项表

图 9-1 作者根据公开资料绘制

可复制提示词

你现在是我项目组的助理工程师。我们拿到了一份客户发来的需求说明书（SOR），需要把它拆成内部能分派的任务清单。

文件在对话里，请先完整读一遍，不要只读目录，不要只读前 10 页。

读完后依次做下面三件事。

第一步：给我一份“需求条目表”，每一行包含六列：

| 条目编号 | 原文摘录 | 所在章节 | 需求类型 | 可验证性 | 建议承接岗位 |

填写规则：

- 条目编号：从 R-001 开始递增
- 原文摘录：照抄原文，一个字都不要改写，不要翻译，不要概括
- 所在章节：写章节号，如果你判断不出章节号就写页码
- 需求类型：只能填这八类之一：性能 / 尺寸 / 材料 / 试验 / 认证 / 接口 / 交付物 / 商务
- 可验证性：只能填这四个之一：可测 / 可算 / 可查 / 不可验证。判定方法是：能用台架或量具测出来的填可测，能用公式或仿真算出来的填可算，能用文件或证书核对的填可查，想不出判定方法的填不可验证
- 建议承接岗位：只能从这六个里选：设计 / 试验 / 质量 / 采购 / 项目 / 工艺

第二步：单独给我一张“冲突与待确认表”，包含四列：

| 编号 | 冲突描述 | 涉及条目编号 | 建议动作 |

只要发现下面任意一种情况，就写进这张表：

- 同一个参数在两个章节出现，数值不一致或单位不一致
- 一条需求和另一条需求在物理上互相冲突，比如空间尺寸和传动比冲突、温度范围和材料耐温冲突
- 给了指标但没给试验方法或试验条件
- 写了“按相关标准执行”但没指明标准号和版本
- 写了“详见附件”但附件里找不到对应内容

第三步：给我一张“漏项检查表”，列出你认为客户应该写但没写的条目，按影响程度从高到低排序，最多 10 条，每条写一句为什么重要。

输出要求：

- 三张表都用 Markdown 表格输出，不要用段落描述
- 不要总结，不要评价这份 SOR 写得好不好，不要夸奖客户
- 任何一条原文摘录，如果你在文件里找不到依据，就不要写进表里
- 三张表全部输出完后，另存为桌面上的 sor_tasklist.xlsx，三个工作表分别命名为 需求条目、冲突待确认、漏项检查

过程与判断点

它会先花一两分钟读文件。47 页的 PDF，我这次等了大约 90 秒。

然后它会逐张吐表。第一张条目表最长，我这次拿到 143 条。这个数字要跟你的经验对一下：一份 40 到 60 页的 SOR，条目数通常在 100 到 200 之间。少于 80 条说明它跳读了，多于 300 条说明它把一句话拆成了好几条。

判断点一：条目数量在合理区间。差太多就得回头查它是不是只读了部分章节。

判断点二：抽查 5 条原文摘录能逐字搜到。我做这一步从来没省过。抽的 5 条要分散在文件的前、中、后三段，不要都抽前面的。这次我抽的 5 条里，有一条它把“-40 °C”抄成了“40 °C”，负号丢了。这一条如果流进设计输入，低温试验全做错。

判断点三：冲突表不能是空的。一份真实 SOR，交叉检查之后至少能找出 3 到 8 条冲突。它如果返回“未发现冲突”，那大概率是它没做交叉检查，直接把提示词的第三步跳过了。这时你要单独追一句：“把第 3 章的尺寸条款和第 7 章的性能条款逐条交叉比对，再列一次冲突。”

判断点四：漏项表里至少有一条是你自己想到的。漏项是最考验它的一张表。它如果列出来的 10 条全是泛泛而谈的“建议明确包装要求”这种，说明它没理解产品。补救办法是把产品描述再喂一次，补一句“我们做的是微电机减速机构，塑料蜗轮，请按这个产品类型重新列漏项”。

产物长什么样：桌面上一个 Excel，三个工作表。条目表可以直接按“建议承接岗位”做筛选，一次筛出所有设计相关的条目，复制进设计输入清单。

翻车清单

翻车一：它把“建议”当成“要求”写进条目表。SOR 里常有“建议采用某某材料”这类措辞，那是参考不是强制。它会一律当成要求抄进去，导致你的任务清单凭空多出一堆硬约束。补救动作：在提示词里加一句“原文含建议、推荐、可选、可选配等措辞的条目，在条目编号后面加 * 号标记，并在需求类型里额外标注‘非强制’”。

翻车二：长条目被它截断。有些条款一整页长，它只抄了前半句，后半句的限制条件丢了。补救动作：输出完成后，单独问一句“条目表里有哪几条原文摘录是截断的？把这几条的完整原文重新抄一遍。”它会自己认出来一部分。剩下的靠你抽查。

翻车三：它给的“可验证性”判断过于乐观。它倾向于把什么都说成可测。有一条需求写“运行平顺无异常”，它判定为可测，实际上没人定义过什么叫平顺。补救动作：把“不可验证”这一列单独筛出来看一遍。如果整张表一条“不可验证”都没有，那它判得太松，你要自己过一遍，把写不出判定方法的条目手动改过来。这些条目是要发回客户问清楚的。

验收标准与变体

验收标准（三件事全做完才算过）：

1. 从条目表随机抽 8 条，回 PDF 逐字核对。8 条里错 1 条以上，整表重做。
2. 把“不可验证”和“冲突待确认”两类条目合并成一张客户澄清清单，当天发给客户。能当天发，说明你真的看懂了。
3. 条目表导入项目管理系统后，每一条都有 owner 和预计完成日期。有条目分不下去，说明这条需求本身不清楚，回到第 2 步。

变体 A：没有正式 SOR 的小项目。很多小项目客户只发了一封邮件，或者给了一份会议纪要。把提示词里的"需求说明书（SOR）"改成"客户邮件往来记录"，把所有往来邮件合并成一个 PDF 丢进去，其余不变。邮件里前后矛盾的地方特别多，冲突表会很好看。

变体 B：反向检查，你给供应商写 SOR 时用。把你写好的 SOR 发给它，提示词改成："假设你是接到这份 SOR 的供应商，列出你无法报价、无法设计、无法验证的条目，并说明缺什么信息。"这是我在写供应商技术协议前的固定动作，能提前堵掉一半来回。

变体 C：跨版本比对。客户改版 SOR 是常态。把旧版和新版都丢进去，让它只输出差异表：新增、删除、数值变更三类。人工比对两版 50 页文件，我从来没在两小时内做完过。

老兵说：我给条目表加"可验证性"这一列，是因为吃过一次亏。有个项目客户写了"噪音不得有异音"，我们做了三轮整改都没通过，最后客户派人来听，说我们要听的是那种"咔哒"声。那一句话里没有频率、没有测距、没有工况，我们猜了三个月。现在只要看到"不可验证"的条目，我当天就发邮件问客户，绝不自己猜。

场景 14 技术标准提取：从标准里摘出跟你产品有关的条款 ★

一句话背景

一份强度计算标准正文加附录上百页，你真正关心的可能只是其中关于塑料齿轮的那几段。人工翻半天，而且最容易漏的是规范性附录里的表格，那部分往往才是硬要求。

名词小词典

标准号：标准的身份证，比如 ISO 6336 是齿轮承载能力计算，ISO 1328 是圆柱齿轮精度体系，ISO 26262 是道路车辆功能安全。带年份的版本号很重要，同一份标准 2006 版和 2019 版的公式可能不一样。

Scope（适用范围）：标准第一章，说明它管什么、不管什么。先看这一章，能省掉后面 90% 的无效阅读。

规范性附录与资料性附录：规范性附录是必须执行的，资料性附录只是参考，常见的标注是 normative 和 informative。这两类混在一起，是标准误读的主要来源。

精度等级：ISO 1328 里用 Grade 加数字表示，数字越小越精密。Grade 6 比 Grade 8 难做，价格也贵。

偏差符号：标准里用来表示具体公差项目的代号，比如 ff_α 表示齿廓形状偏差， F_p 表示齿距累积总偏差。看不懂符号，条款就读不明白。

开工前准备

1. 标准 PDF。必须是正版来源，不要用来来源不明的下载件，页码和条款号可能对不上。

2. 一段产品描述，100 到 300 字。要写清产品类型、材料、精度要求、工作温度范围、预计产量。写得越具体，它筛得越准。
3. 确认标准的现行版本年份。不知道就去标准发行机构的官网查，或者问你们公司的标准化工程师。
4. 模式选 Ask。这一步不写文件，只对话。核对完条款号再让它出表。

操作步骤

第一步：把标准 PDF 拖进去，先问一句："这份标准的编号、版本年份、发布日期分别是什么？写在封面上。"它答对了才继续。

第二步：让它读 Scope 章，判断你的产品在不在范围内。不在范围内直接停，别浪费时间。

第三步：把下面的提示词整段发过去。

第四步：拿到适用条款表之后，逐条回原文核。这一步不能省，一个字都不能省。

第五步：核完让它另存 Excel，作为设计输入的附件归档。

标准提取的四道闸门

版本不对就停，范围不在就停

1 先问版本号

编号、版本年份、发布日期答不出就停

2 读适用范围那一章

三句话讲清管什么、不管什么、产品在不在范围

3 条款号逐条回原文核

搜到不到的当条删除，宁可少不能假

4 分清两类附录

规范性附录进表，资料性的不许混进来

ISO 6336 那次练习：适用条款 23 条，其中 3 条在原文里搜不到，1 条是它编的

图 9-2 标准提取的四道闸门：版本、范围、条款号、附录性质

图 9-2 作者根据公开资料绘制

可复制提示词

你是我们组的标准专员。我给你一份标准文件和一段产品描述，请帮我把跟这个产品有关的条款摘出来。

产品描述：

(把你的产品描述粘贴在这里)

第一步：先读标准的第一章 Scope，用三句话回答：

1. 这份标准管什么
2. 这份标准明确不管什么
3. 我们的产品落在不落在范围内

如果不在范围内，直接停止后面所有步骤，只告诉我原因和它不管什么。

第二步：给我一张"适用条款表"：

| 条款号 | 条款标题 | 原文摘录 | 适用性 | 我们对对应要做什么 |

填写规则：

- 条款号：写到最细一级，例如 5.3.2、Table 4、Annex B.2
- 原文摘录：照抄原文，不要翻译，不要改写，不要概括
- 适用性：只能填 适用 / 不适用 / 需确认。"需确认"必须同时写出要确认什么
- 我们对对应要做什么：一句话，写具体动作，例如"按此表选 Grade 6 并写入图纸技术要求"

第三步：给我一张"看起来相关但判定为不适用"的表：

| 条款号 | 条款标题 | 判定不适用的理由 |

第四步：单独检查标准里有没有规范性附录 (normative annex)，有就单独列出来，写明附录编号和它约束的内容。同时标明标准里哪些附录是资料性的 (informative)，不要混进适用条款表。

第五步：列出这份标准里引用的其他标准号，以及你判断哪些我们也需要看。

硬性要求：

- 每一条必须有精确条款号，不许出现"参见第 5 章"这种模糊引用
- 原文摘录照抄，不许改写
- 如果你对某个条款号不确定，就写"条款号待核"，不许猜
- 不许把资料性附录的内容写进适用条款表
- 不许把其他标准的内容当成这份标准的内容
- 全部输出完后另存为桌面的 standard_extract.xlsx，工作表命名为 适用条款、不适用条款、规范性附录、引用标准

过程与判断点

我拿 ISO 6336 做过一次练习，配的是前面那个钢蜗杆配 POM 蜗轮的产品描述。它返回的适用条款表 23 条，不适用条款表 9 条，规范性附录 2 个。

判断点一：条款号必须能在原文里搜到。这是这个场景唯一硬性的判断点。我逐条搜了那 23 条，20 条能搜到，3 条搜不到。搜不到的 3 条里，有 2 条是它把章节号记错了一位，还有 1 条是它自己编的。1 条编造，就足以让整份表的可信度归零，所以这一步必须全查，不能抽查。

判断点二：适用条款的数量。一份上百页的标准，针对一个具体产品，适用条款通常在 10 到 40 条之间。它如果给你 80 条，说明它没筛选，只是把目录抄了一遍。这时要追一句："只保留那些会改变我们设计决策或图纸标注的条款，纯定义和术语条款删掉。"

判断点三：它有没有分清 normative 和 informative。我在提示词里专门设了第四步，就是因为它默认会把两类混在一起。你拿到结果后自己也要看一眼附录标题下面有没有 normative 字样。

判断点四：不适用条款表不能太短。一份标准里看起来相关但实际不适用的条款，通常有 5 到 15 条。它如果只给你 1 条，说明它偷懒了。这张表的价值在于，将来有人质疑"这条你怎么没做"，你能立刻拿出判定理由。

判断点五：符号表有没有一起出来。标准是符号驱动的，同一份 ISO 1328 里 ff_a、Fp、Fr 这些代号混在一起，把一个代号看错，公差就抄错一档。我会在最后追加一句："把这份标准里出现的所有偏差符号单独列一张表，写符号、中文名称、定义、出处条款号。"这张符号表单独存一份，后面看图纸和检验报告的时候随时能查，比每次回标准里翻快得多。

有一次我把这份符号表发给了供应商，对方工艺工程师回了一句"这个我们内部培训讲过，但没人整理成表"。一张表能跨公司用，是因为标准本身是公共的，只是没人愿意花半天去抄。

翻车清单

翻车一：它编条款号。这是标准类任务最高频的错误。它有的时候会把"5.3"写成"5.3.1"，有的时候直接编一个不存在的 Table 编号。补救动作：全表核对，不要抽查。我自己的做法是让它把条款号单独列一列，我复制进 PDF 阅读器批量搜索，搜不到的当条删除。宁可少几条，不能留假的。

翻车二：版本错。有些标准文件 PDF 里带着多个版本的影子，或者你手上这份是作废的旧版。它会照着旧版给你结论。补救动作：第一步就让它报版本号，你跟官网核对。版本号不对，全部推翻重来。我见过一次，拿着 2006 版的公式算出来的安全系数，跟 2019 版差了 15%，够让一个合格的设计变成不合格。

翻车三：它把标准里的示例当成要求。标准正文里经常有算例，用的是示例参数。它会把示例里的数值当成规定值抄进适用条款表。补救动作：看到条款里出现具体数字，回原文看一眼上下文是不是"Example"或"示例"。是示例就删掉。

验收标准与变体

验收标准：

1. 适用条款表的每一条，条款号都能在原文搜到，且原文摘录逐字一致。这是硬标准，一条不满足就整表作废。
2. 适用条款表里每一条"我们对对应要做什么"都写成了具体动作，能直接抄进设计任务书或图纸技术要求。写不出动作的条款，说明它没理解，删掉或重写。

3. 让组里第二个人用同一份提示词独立跑一遍，两份表的交集应该超过 70%。交集低于一半，说明这个任务本身定义不清，先把产品描述写具体。

变体 A：新旧版本差异比对。标准换版是常事。把旧版和新版 PDF 都丢进去，提示词改成："逐条比对两份标准，只输出差异：新增条款、删除条款、公式变更、数值变更四类，每类给条款号和变更内容。"人工比对两版上百页标准，我做一次花了整整两天。

变体 B：客户 CSR 与内部企标对照。把客户特殊要求和你们内部企标都丢进去，让它输出对照表：客户更严的、我们更严的、互相冲突的三类。这张表可以直接拿去跟客户谈，是谈判前的准备材料。

变体 C：把标准条款转成检验项目。拿到适用条款表之后追加一句："把适用条款表里涉及公差和性能指标的条款，转成一份进货检验项目清单，包含检验项目、量具、抽样方案三列。"这个功能我用在给供应商写检验规范上。

老兵说：这个场景我标了 ★，是我六个场景里唯一没在正式交付项目上用它出最终结论的。原因很实在：条款号编错这件事，出错率我实测下来在 10% 左右，工程上不能接受，必须全查。全查花的时间比人工翻标准少不了太多，所以我至今只在学习和准备阶段用它。你要用，就一定要全查，别抽查。

场景 15 项目计划生成：里程碑、依赖关系、关键节点 ★★

一句话背景

项目经理问我要节点，我拍脑袋报日期，三次里两次不准。工期我算得没错，漏掉的是长周期件：模具、认证、样件物料。这些东西的等待时间不在我的脑子里，在采购的脑子里。

名词小词典

里程碑：项目里必须交付的节点，比如设计冻结、首件下线、DV 完成。里程碑不等同于任务，它是任务的成果。

WBS (Work Breakdown Structure, 工作分解结构)：把项目一层层拆成能分派的小任务。拆到什么程度算够：一个任务能在两周内干完，并且只有一个负责人。

依赖关系：A 干完 B 才能开始，这叫 B 依赖 A。写计划最容易错的就是这里，把本来能并行的事写成了串行。

关键路径：决定总工期的最长那条依赖链。关键路径上耽误一天，整个项目就晚一天。

DV 与 PV：DV 是设计验证，验证设计是否满足要求；PV 是生产验证，验证量产工装和工艺能不能稳定做出合格件。

SOP (Start of Production, 量产启动)：项目终点，开始批量生产供货。

开工前准备

1. 场景 13 出来的任务清单。计划是从任务来的，没有任务清单直接让它排计划，它编出来的东西没法用。
2. 客户给的节点：样件交付日期、DV 窗口、SOP 日期。硬约束必须先给。
3. 团队人数和每个人能投入的比例。不给这个，它排出来的计划是理想资源下的计划。
4. 一份长周期件清单：开模、认证、长交期物料、台架排期。这份清单是你自己补的，AI 想不出来你们公司模具要等多久。
5. 模式选 Plan。排计划一定要先看到它的思路，再让它写文件。

操作步骤

第一步：把任务清单和上面四份信息一起丢进对话框，让它复述一遍硬约束。它复述对了才继续。

第二步：发下面的提示词。

第三步：拿到依赖关系表，重点看"并行能不能并行"这一列。它默认倾向于串行，你要手动纠正。

第四步：让它做倒排校验，从 SOP 日期往回推，看推出来的开始日期是不是早于今天。早于今天，说明计划不可行。

第五步：导出甘特图数据源，用 Excel 或你们的项目工具画出来。

排计划先排依赖

哪些必须等，哪些其实可以一起干

它默认排出来的样子

- 按顺序往下排
31 个任务只标出 4 处可以并行
- 长周期件当普通任务
模具开发被当成两周的设计活
- 工作日和自然日混用
一份表里两种口径

我要它改成的样子

- 逐对判断能不能并行
我自己手工改成 11 处
- 材料认证与结构设计并行
工装设计可在冻结前启动七成
- 工期一律用工作日
关键路径的工期手工加一遍

任务清单 96 条归成 31 个末级任务，关键路径 14 个任务，总工期 168 个工作日

图 9-3 排计划先排依赖：哪些必须等，哪些可以并行

图 9-3 作者根据公开资料绘制

可复制提示词

你是我们项目组的计划工程师。我要给一个新项目排计划，下面是全部输入。

输入一，任务清单（来自需求拆解）：

（粘贴场景 13 输出的需求条目表，或者粘贴你们自己的任务清单）

输入二，客户硬性节点：

- 样件交付：（日期）
- DV 窗口：（起止日期）
- PV 窗口：（起止日期）
- SOP：（日期）

输入三，团队资源：

- 设计工程师 x 人，投入比例 x%
- 试验工程师 x 人
- 其他：（补充）

输入四，长周期事项（这些事项的等待时间必须单独列出来，不能压缩）：

- 模具开发：（周数）
- 认证周期：（周数）
- 长交期物料：（周数）
- 台架排期待：（周数）

请按下面顺序输出。

第一步：把任务清单归并成 WBS，两级就够。每个末级任务满足两个条件：能在两周内完成，且只有一个负责人。输出表：

| WBS 编号 | 任务名称 | 负责人角色 | 工期（工作日） | 产出物 |

第二步：给一张“依赖关系表”：

| 前置任务 | 后置任务 | 依赖类型 | 能不能并行 | 判断理由 |

依赖类型只能填 完成到开始 / 开始到开始 / 完成到完成。

“能不能并行”必须给判断理由，不许只填能或不能。如果两个任务之间没有物理上的先后顺序，就填能并行，不要因为习惯做成串行。

第三步：标出关键路径，把关键路径上的任务按顺序列出来，并算出总工期（工作日）。

第四步：做倒排校验。从 SOP 日期往回推，用关键路径的工期反推项目开始日期。如果反推出来的开始日期早于今天，明确告诉我“此计划不可行”，并给出三条压缩工期的具体建议，每条建议写清代价。

第五步：列出你认为风险最高的五个节点，每个节点写清为什么危险。

输出要求：

- 全部用 Markdown 表格

- 工期一律用工作日，不要用自然日，不要写"约两周"这种模糊表达
- 不许安慰我，不许说"计划留有充足余量"这类话
- 最后把 WBS 表和依赖关系表另存为桌面的 project_plan.xlsx，两个工作表命名为 WBS 和 依赖关系

过程与判断点

我拿一个微电机执行器项目跑过这套流程。任务清单 96 条，归并成两级 WBS 后 31 个末级任务，关键路径 14 个任务，总工期 168 个工作日。

判断点一：末级任务数量。任务清单 96 条归成 31 个末级任务，这个比例（大约 3 比 1）是合理的。它如果归并得太狠，把 96 条压成 8 个大任务，那你拿到的还是一份目录，不是计划。追一句："每个末级任务的工期必须小于等于 10 个工作日，超过的继续拆。"

判断点二：依赖关系表里的"能并行"这一列。这是我最看重的一列。第一次跑，31 个任务它只标了 4 处可以并行。我自己看了一遍，手工改成 11 处。比如"材料认证"和"结构设计"完全可以并行，"工装设计"可以在"设计冻结"之前启动 70%。这些它不会主动想到，因为它默认按任务清单的顺序往下走。

判断点三：倒排校验的结果。它返回"此计划不可行，需提前 23 个工作日启动"，并给了三条压缩建议：模具并行开发（代价是模具返修风险上升）、DV 分两批做（代价是试验费用增加约 30%）、长交期货料提前下单（代价是产生呆滞库存风险）。这三条建议我采纳了两条。这个输出是有用的，因为它把代价写出来了。

判断点四：长周期事项有没有被单独列出来。我在输入四里明确给了模具和认证的周数，它写进去了。如果你不给，它会按普通任务处理，把模具开发当成两周的设计任务，那这份计划就是废纸。这一步完全依赖你自己的输入质量。

产物长什么样：桌面一个 Excel，两张表。WBS 表可以直接导入 Project 或各类项目工具，依赖关系表用来连箭头。

翻车清单

翻车一：它把能并行的做成串行。这是排计划最大的坑。它拿到任务清单，默认是按顺序往下排。补救动作：在提示词里专门设了"能不能并行"这一列并要求写理由，就是为了逼它逐对判断。拿到结果后你自己还要过一遍，我自己的经验是它能标出的并行处，大概只有实际可行的一半。

翻车二：它漏了长周期件。模具、认证、台架排期，这些时间不在任务清单里，它在你的脑子里或者采购的脑子里。补救动作：必须在输入四里手写成硬数据给它。这一项我建议在你们部门建一个共享的长周期件清单，每次排计划直接贴进去，别靠记忆。

翻车三：它把工作日和自然日混用。工期填 10，它有时候按 10 天算，有时候按两周算，一份表里两种口径混着来。补救动作：提示词里写了"一律用工作日"，拿到结果后还是要核一遍总数。做法是把关键路径上的工期手工加一遍，看跟它报的总工期对不对得上。

验收标准与变体

验收标准：

1. 关键路径上的任务工期手工加一遍，跟它报的总工期一致。对不上就是口径混了。

2. 依赖关系表里, "能不能并行"填"能"的条目, 你逐个确认过物理上确实能并行。这一步必须人工过, AI 标不全。
3. 倒排校验跑出来, 开始日期晚于今天。如果它说不可行, 你必须在三条压缩建议里选至少一条, 改完再跑一次。
4. 把这份计划给实际干活的人看, 让他们报每个任务的工期。他们报的数跟 AI 报的数差 30% 以上的任务, 用人的数。

变体 A: 倒排计划, 从 SOP 往回推。客户只给 SOP 日期, 中间全靠你排。把提示词第四步单独拿出来用: "只给我一张倒排表, 从 SOP 日期往回推每个里程碑的最晚完成日期。"这张表拿来跟客户谈节点很有用, 因为它能直观显示"你这个 SOP 日期要求我们在两周内完成模具"。

变体 B: 周报自动生成。每周把项目进展(完成了哪些任务、卡在哪)丢给它, 让它对照 WBS 表更新状态, 输出周报: 本周完成、下周计划、风险项三块。我做两个月, 每周省 40 分钟。

变体 C: 多项目资源冲突检查。有三个项目并行时, 把三份 WBS 表一起丢进去, 让它找出同一角色在同一时间段被分配了两次的地方。这种冲突人工很难看出来。

老兵说: 这套流程我跑通过, 但有一件事它至今做不好: 它不知道哪个供应商的模具厂最近在赶工, 也不知道试验台架下个月被大项目占了。这些"场外信息"永远要你自己补。所以我把 AI 排的计划当第一稿, 实际节点一定是跟采购和试验室打完电话之后定的。

场景 16 风险清单辅助: 把经验变成结构化的失效模式表 ★★

一句话背景

DFMEA 这张表, 老工程师脑子里的东西比表上写的多十倍。问题是怎么把它们掏出来。人工做法是一个下午的评审会, 五个人吵三小时, 出来的表还常常漏掉最要命的那条。

名词小词典

DFMEA (Design Failure Mode and Effects Analysis, 设计失效模式与影响分析): 一张把"它可能怎么坏、坏了多严重、为什么坏、怎么提前发现"列清楚的表。它在设计阶段做, 不是等产品坏了再做。

失效模式: 零件坏的具体样子。写法有讲究, 必须是"零件加动词加偏差", 比如"蜗轮齿面磨损", 不能写"强度不足"。

失效影响: 这个失效发生之后, 客户或者下一道工序会看到什么。

S/O/D: 严重度 Severity、发生度 Occurrence、探测度 Detection, 通常各按 1 到 10 打分。S 看后果有多严重, O 看发生的可能性, D 看在出厂前能不能发现。

RPN: 风险优先数, S 乘 O 乘 D。数值越高越该先处理。

AP (Action Priority, 措施优先级): 汽车行业现在更常用的做法, 按 S、O、D 的组合查表给出高、中、低三档优先级, 比单纯看 RPN 乘积合理。

开工前准备

- 1. 产品结构清单：有哪些零件、什么材料、怎么装配。清单越细，它想出的失效模式越多。
- 2. 工况参数：扭矩、转速、温度范围、循环次数、环境条件。
- 3. 你自己的经验条目。至少手写 5 条，作为参照系。它如果连你手写的 5 条都想不全，说明它没进入状态。
- 4. 同类产品的售后数据或试验失败记录，有的话一并给。
- 5. 模式选 Plan。先让它出结构，你确认维度再批量生成。

操作步骤

- 第一步：把结构清单和工况参数丢进去，让它先复述产品，确认它理解对了。
- 第二步：先不要让它出完整表，先让它列失效模式的候选清单，你自己过一遍。
- 第三步：把你手写的 5 条经验混进去，看它能不能识别出这些是你补的。能识别说明它在认真读。
- 第四步：发下面的提示词，让它生成完整表。
- 第五步：重点看"探测度"这一列。它给的探测度通常过于乐观。
- 第六步：导出 Excel，作为 DFMEA 评审会的输入，不要当作评审会的结论。

DFMEA 表的六个支柱

模式、影响、原因，再加 S、O、D 三个打分

失效模式

零件部位加动词加偏差，例如蜗轮齿面磨损

失效影响

客户或下道工序能观察到的具体现象

失效原因

挖到能采取措施的层级才算写完

严重度 S

一到十分，七到八分是功能丧失

发生度 O

写明依据是经验判断还是实测数据

探测度 D

出厂前没法发现的要打九分或十分

零件清单 11 项，它列出失效模式 34 条，这张表是评审会的起点不是结论

图 9-4 一张 DFMEA 表的六个支柱：模式、影响、原因、S、O、D

图 9-4 作者根据公开资料绘制

可复制提示词

你是我们组的可靠性工程师，要和我们设计工程师一起做一份 DFMEA。

产品信息：

- 产品：（一句话说清是什么）
- 零件清单：（逐个列出，含材料）
- 工况参数：扭矩（数值）、转速（数值）、温度范围（数值）、循环次数（数值）、环境（描述）
- 类似产品的已知问题：（列你手上的经验，至少 5 条）

第一步：先按零件逐个列出候选失效模式，输出一张表：

| 零件 | 失效模式 | 失效影响 | 失效原因 | 现有控制手段 |

填写规则：

- 失效模式必须写成"零件部位 + 动词 + 偏差"，例如"蜗轮齿面磨损"，不许写"强度不足""性能下降""可靠性差"这类抽象表述
- 失效影响要写客户或下道工序能观察到的具体现象，不许写"影响用户体验"
- 失效原因写到能采取措施的层级。原因是"材料选择不当"就要继续追问为什么不当，往下再挖一层
- 现有控制手段写具体：哪个试验、哪份检查表、哪道工序。没有就写"无"，不许留空

第二步：给上面每一行打 S、O、D 三个分，各 1 到 10，并给出 AP 档位（高 / 中 / 低）。

打分规则必须显式说明依据：

- S：1 到 3 为不影响功能，4 到 6 为功能降级，7 到 8 为功能丧失，9 到 10 为涉及安全或法规
- O：依据是什么？如果是凭经验，写明"经验判断"；如果有试验数据或售后数据支撑，写明数据来源
- D：如果在量产前没有任何手段能发现，D 必须打 9 或 10，不许打中间分。判定"能发现"必须有具体的检验或试验方法

第三步：把 AP 为"高"的行单独列出来，每行给两条建议措施，一条设计改进，一条验证手段。措施必须可执行，不许写"加强设计评审"这种。

第四步：检查我给你的"类似产品的已知问题"那几条，有没有全部体现在失效模式表里。有遗漏的，明确告诉我哪几条没进去，并补上。

输出要求：

- 全部用 Markdown 表格
- 不许写"本产品整体可靠性良好"这类总结
- 打分依据这一列不许留空，写不出依据就写"无依据，需评审确定"
- 最后整表另存为桌面的 dfMEA_draft.xlsx

过程与判断点

我用这套流程跑过前面那个减速齿轮箱。零件清单 11 项，它列出失效模式 34 条。

判断点一：失效模式的写法。它第一次给我的表里，有 6 条写的是"强度不足""磨损严重""噪音超标"。这些都不是失效模式，是结论。我在提示词里加了"零件部位 + 动词 + 偏差"的规则之后，重跑一次，34 条里有 31 条合格，剩下的 3 条我自己改了。这个规则是这整份提示词里最值钱的一句。

判断点二：它有没有把你的经验吃进去。我手写了 5 条经验，其中一条是“蜗杆齿面粗糙度 Ra 必须低于 $0.2\text{ }\mu\text{m}$ ，否则 POM 蜗轮三个月后磨成粉末”。它把这条写进了失效模式表，还往下挖了一层原因：蜗杆磨削工艺参数未固化、砂轮修整周期过长。这个挖掘是有价值的。

另一条我写的是“POM 齿轮在 $45\text{ }^{\circ}\text{C}$ 下热膨胀大约 0.012 mm 每齿高，修形的时候必须考虑，齿顶修缘量要取到 0.015 到 0.020 mm ，比钢齿轮多修一倍”。它没有写进去。我按提示词第四步追问，它认了，补上并标了 AP 高。

判断点三：探测度有没有被高估。它给的 D 分普遍偏低，也就是过于乐观。塑料齿轮的磨损在量产前确实很难发现，它打 5 分，我改成 8 分。我在提示词里写了“没有任何手段能发现就打 9 或 10，不许打中间分”，就是为了堵这个。

判断点四：失效原因的深度。“材料选择不当”这种原因没有价值，因为它不能指导行动。好的原因是“POM 未加玻纤增强，10 万次循环许用弯曲应力只有 40 到 50 MPa ，而实际工作应力 97 MPa ”。后者能让工程师立刻知道该做什么。

产物长什么样：一份 Excel，34 行，六到八列。评审会上投屏，逐行过。我那次评审会开了 2 小时 20 分，比往常的半天短，因为起点是一张填好的表，不是一张空白表。

翻车清单

翻车一：它写的失效模式是空话。“强度不足”“可靠性差”“性能下降”，这些词看着像术语，实际没有信息量。补救动作：提示词里那句“零件部位加动词加偏差”必须保留。拿到表之后，把抽象词汇批量筛一遍：搜索“不足”“不良”“下降”“异常”，搜到的一律退回重写。

翻车二：打分没有依据。它会给一排数字，看着很专业，但问它为什么打这个分，它给不出来源。补救动作：提示词里强制要求写打分依据。拿到表之后，把“打分依据”这一列筛一遍，写“经验判断”超过一半的，说明这张表的分数只能当排序参考，不能当决策依据。

翻车三：它把设计和过程混在一起。DFMEA 管设计，PFMEA 管制造过程。它经常写“注塑保压时间不足”这种过程原因进 DFMEA。补救动作：加一句“失效原因只写设计能控制的部分，涉及工艺参数、操作手法、设备状态的归入 PFMEA，单独列在表外”。

验收标准与变体

验收标准：

1. 全部失效模式符合“零件部位 + 动词 + 偏差”写法，抽象词汇零出现。
2. 你手写的经验条目 100% 出现在表里。少一条就追问，直到补齐。这一条是硬标准。
3. AP 为高的每一行都有可执行的措施，且指定了责任人和完成日期。
4. 评审会上，参会工程师至少新增 5 条 AI 没想到的失效模式。达不到 5 条，说明 AI 想得比你们全，这时候要反过来怀疑你们是不是在走过场。

变体 A：设计评审检查表。把 DFMEA 表丢给它，追加一句：“把 AP 为高和中的失效模式，转成一份设计评审检查表，每条写清评审时看什么、怎么判合格。”评审会上照着念一遍。

变体 B：供应商风险问卷。把涉及外购件的失效模式挑出来，让它转成供应商问卷：每条失效模式对应一个提问，要求供应商回答他们的控制手段。发给供应商之前，这份问卷比“请把你们的质量控制文件发来”有效得多。

变体 C：试验大纲的补充依据。让它把“探测度大于等于 7”的行单独列出，这些就是试验大纲必须覆盖的项目。我试过一次，它列出 12 条，其中 3 条我们原本的试验大纲里没有。

老兵说：这张表我从不当结论用，只当评审会的起点。AI 列出来的那 34 条有价值，评审会上老工程师看着它们想起来的那 7 条更值钱。AI 的作用是把空白页变成填空页，人填的那部分才是命。

场景 17 技术方案对比：两套方案摆在一起比，比什么先定好 ★★★

一句话背景

两套方案摆一起比，最难的一步发生在比之前：先定好比什么。没有评价维度的对比，最后一定变成谁嗓门大谁赢。

名词小词典

单级传动与两级传动：一级减速就把总减速比做完叫单级，分两级做叫两级。总减速比是两级各自减速比的乘积。

减速比：输入转速除以输出转速。前面那个项目是 7200 除以 72，等于 100 比 1。

包络尺寸：零件占的最大外形尺寸。这个项目卡的是外径小于 60 mm。

变位系数：齿轮加工时刀具相对毛坯的偏移量，用 x 表示。正变位能把齿根加厚，提高弯曲强度，也能避开根切。

根切：齿数太少时，加工会把齿根挖掉一块，削弱强度。直齿轮不修形时理论最少齿数是 17。

传递误差 TE：实际输出角度跟理论输出角度的差值，是齿轮噪音的主要来源之一，单位 μm 。

开工前准备

1. 两套方案的完整参数，或者至少是方案描述加关键参数。
2. 硬性约束清单：尺寸上限、成本上限、已有的工装和产线限制。
3. 一份你自己先写的评价维度草稿，3 到 8 条。别指望它替你想维度，它想的维度是通用的，你的才是这个项目的。
4. 每个维度的权重。不会定就先都定 1，跑完一轮再调。
5. 模式选 Plan，这个场景必须先看到维度表再让它打分。

操作步骤

第一步：先把方案描述丢进去，只让它做一件事："先不要比，先列出你认为应该比的维度，并说明每个维度为什么重要。"

第二步：看它列的维度，跟你自己的草稿合并，删掉通用的，留下这个项目特有的。这一步必须你来

做。

第三步：把最终维度表发回给它，让它确认每个维度的判定方法和数据来源。判定方法写不出来的维度，删掉。

第四步：发下面的提示词让它填表打分。

第五步：看它的结论，重点看它有没有主动提出"两套都不行"。

第六步：导出对比表，作为方案评审会的材料。

方案对比的两步走

先定比什么，再填数字

- 1 先用硬性约束淘汰
违反任一条就直接淘汰，不进打分
- 2 让它先列评价维度
删掉通用的，留这个项目特有的
- 3 确认判定方法与来源
写不出判定方法的维度删掉
- 4 打分不许改维度权重
一到十分，依据栏必须写具体数值
- 5 结论只能三选一
明显更优、需补信息、两套都不行
- 6 列出两套都没想到的
最多五条，这是给我兜底用的

方案 A 的单级蜗轮外径算出来 71.4 mm，超出限制 11.4 mm，第一步就被淘汰

图 9-5 方案对比的两步走：先定维度，再填数字

图 9-5 作者根据公开资料绘制

可复制提示词

你是我们组的方案评审工程师。我手上有两套（或三套）技术方案，需要摆在一起比。

方案 A：（完整描述，含关键参数）

方案 B：（完整描述，含关键参数）

硬性约束（违反任一条的方案直接淘汰，不参与打分）：

- （约束一，例如：外径小于 60 mm）
- （约束二，例如：空载噪音小于 40 dBA，测距 500 mm）
- （约束三）

第一步：先用硬性约束过滤。任何方案违反任一条硬性约束，直接写明淘汰，并指出违反的是哪一条、超出多少。被淘汰的方案不进入打分。

第二步：用下面这张维度表打分，不许增删维度，不许改权重：

| 维度 | 权重 | 判定方法 | 数据来源 |
（把你最终确认的维度表粘贴在这里）

输出打分表：

| 维度 | 权重 | 方案 A 得分 | 方案 A 依据 | 方案 B 得分 | 方案 B 依据 | 加权后 A | 加权后 B |

规则：

- 得分用 1 到 10
- "依据"这一列必须写具体数值或具体事实，例如"外径 71.4 mm，超出限制 11.4 mm"。不许写"较好""略优""表现不错"
- 如果某个维度你算不出数值，就写"无法判定"，不许凭印象给分

第三步：给结论。结论必须是下面三种之一，不许和稀泥：

1. 方案 X 明显更优，理由：.....
2. 两套方案在关键维度上互有胜负，需要补充什么信息才能定：.....
3. 两套方案都不满足某条硬性约束，建议：.....

第四步：单独列出"这两套方案都没考虑到的东西"，最多 5 条。这一列是给我兜底用的。

输出要求：

- 全部用 Markdown 表格
- 不许说"两套方案各有优势，具体选择取决于实际需求"这类话
- 任何数值都必须注明来源：算出来的写公式和代入值，查出来的写查自哪里，估出来的写"估算"
- 最后另存为桌面的 scheme_compare.xlsx

过程与判断点

这个场景是我实测留档最完整的一个。

我给它的两个方案是：方案 A 单级蜗轮蜗杆，减速比 100 比 1；方案 B 两级传动，第一级蜗轮蜗杆，第二级平行轴齿轮。

它做的第一件事就掏出硬性约束把方案 A 淘汰了，压根没先打分：蜗轮要做到 100 齿，按模数 0.7 算，外径 71.4 mm，超出 60 mm 限制 11.4 mm。

这个判断我当时自己也算过，但我是在看了 8 页文件之后才发现的。它是拿到参数的第 40 秒说的。

判断点一：它有没有先做淘汰而不是先打分。很多次对比是浪费时间，因为其中一套方案根本不可行。硬性约束过滤这一步必须在打分之前。它这次做对了，但有一次它没做，直接给违反约束的方案打了 7 分。

判断点二：每个格子里有没有数字。它给的方案 B 打分表里，“弯曲强度”这一项的依据写的是：“POM 输出齿轮在 10 Nm 下弯曲应力 97 MPa，POM 在 10 万次循环下的许用值 40 到 50 MPa，超出约一倍。建议齿宽从 20 mm 加到 30 mm，材料改用 POM 加玻纤增强级。”

这句话里每一个数字都能追到来源。这就是合格的依据栏。

判断点三：它敢不敢说两套都不行。提示词第三步专门设了“两套都不满足”这个选项，就是为了逼它别和稀泥。这次它给的结论是第二种：方案 A 淘汰，方案 B 在噪音维度上处于临界，需要客户确认 10 Nm 是不是峰值力矩。

这条追问后来证明是对的。客户的 10 Nm 是堵转峰值，正常工作扭矩只有 4 Nm 左右。这个信息在原来的工况参数表里没有，是它逼出来的。

判断点四：第四步“都没考虑到的东西”。这次它列了 5 条，其中一条是“两级传动的效率损失没有计入，蜗轮蜗杆效率按 0.45 到 0.6 估算，平行轴按 0.97，总效率约 0.45 到 0.58，需要核算电机功率是否够”。这条提醒让我们及时发现了电机选型偏小。

产物长什么样：一个 Excel，一张打分表，一张结论。评审会上我直接投屏打分表，讨论集中在三个有分歧的维度上，会议 50 分钟结束。

评价维度先定，打分才有意义

权重不会定就先都定 1，跑完一轮再调

维度	权重	判定方法	数据来源
包络尺寸	1	算最大外径再比上限	两套方案的参数表
弯曲安全裕度	1	工作应力与许用应力之比	材料手册加手算
噪音水平	1	规定测距下的实测读数	同类产品台架数据
能否沿用现有模具	1	查现有工装清单比对	工装台账
客户认不认可供应商	1	发邮件向客户书面确认	客户往来邮件
齿根会不会根切	1	算实际齿数比最少齿数	齿轮几何计算书
传递误差 TE 多少	1	取输出角度差值的实测值	齿轮检测报告

依据栏出现较好、略优这类词，这一格就等于没填

图 9-6 评价维度定在前面，打分才有意义

图 9-6 作者根据公开资料绘制

翻车清单

翻车一：它列的维度太通用。‘它会给你"成本、性能、可靠性、可制造性"这种四海皆准的维度。这种维度比不出结果。补救动作：维度必须由你来定。做法是让它先列，你看完只保留跟这次决策真正相关的，然后补上你这个项目特有的，比如"能否沿用现有模具""客户是否认可该供应商"。

翻车二：它给的分没有依据，或者依据写"较好""略优"。补救动作：提示词里那句"不许写较好略优表现不错"必须留着。拿到表之后，把依据栏扫一遍，出现形容词的一律打回重写。没有数字的格子就是没有信息。

翻车三：它算错数，而且错得很自信。这次它把两级传动的总效率算成了 0.65，我复核了一遍，按 0.55 乘 0.97 应该是 0.53 左右。它承认了并改了。补救动作：凡是涉及乘除的结果，自己按一遍计算器。这一步在这类场景里不能跳。我自己的规矩是：所有带单位的数字自己核，纯逻辑判断可以信它。

验收标准与变体

验收标准：

1. 打分表里每一格依据栏都有数字和来源，形容词零出现。
2. 所有带单位的数值，你自己按一遍计算器，全部对得上。
3. 结论是三种之一，不是和稀泥的第四种。

4. 把这份对比表给没参与讨论的第三人看，他能看懂为什么选这个方案。看不懂，说明依据栏写得不够具体。

变体 A：供应商方案对比。把三家供应商的技术方案书都丢进去，用同一套维度表打分。这个变体我用在供应商定点上，效果比方案对比还好，因为它不受"这是我自己想的方案"这种心理影响。

变体 B：材料选型对比。方案换成材料：POM 与 POM 加玻纤，或者钢与粉末冶金。维度换成拉伸强度、许用弯曲应力、吸水率、收缩率、单价、加工方式。同样的提示词结构，只换维度表和方案描述。

变体 C：把对比表反过来用，做方案的自我体检。只给一套方案，让它扮演评审专家找出这套方案的薄弱维度，按严重程度排序。我在提交方案前固定跑一次，提前把会被问住的问题准备好答案。

老兵说：AI 不讨好你，算出来不够就是不够。方案 A 那条"外径 71.4 mm，超出 11.4 mm"，如果我拿给一个想省事的人看，他会说"差不多吧，想办法挤一挤"。AI 不会这么说，它只给数字。我后来养成一个习惯：方案评审前先让 AI 过一遍，它挑出来的毛病，比评审会上同事挑出来的多。

场景 18 行动项跟踪：会议决定变成有人有日期的任务 ★★

一句话背景

周会开 60 分钟，说了 17 条口头决定。下周开会，其中 5 条没人认领，3 条没人记得说了什么。这个活每周都在发生，每次都耗掉 30 到 40 分钟整理纪要。

名词小词典

行动项：会议里定下来的、需要有人在某个日期前做完的具体动作。讨论不是行动项，决定才是。

DRI (Directly Responsible Individual, 直接责任人)：一件事只有一个最终负责的人，可以有协助的人，但负责人只能有一个。

截止日期：具体到年月日的日期。"尽快""下周""月底"都不是日期。

状态：未开始、进行中、已完成、已阻塞。加一个"已阻塞"能救很多项目，因为它逼着人说出来卡在哪。

阻塞项：一件事干不下去，卡在等别的人或别的信息。

开工前准备

1. 会议录音转写文本，或者手写纪要。录音转写效果更好，因为口头讨论转写下来，AI 能分辨哪些是决定哪些是闲聊。
2. 参会人名单，含部门和姓名。不给名单，它会编人名。
3. 上次会议的行动项清单。用来做滚动跟踪。
4. 你们的行动项编号规则，比如 A-001 或者按会议编号。

5. 模式选 Agent。这个活目标明确，直接让它干完出文件。

操作步骤

- 第一步：把转写文本拖进去，先让它报一句："这份转写一共多少字，涉及多少个说话人。"确认它读全了。
- 第二步：发下面的提示词。
- 第三步：重点检查"截止日期"这一列，凡是出现"尽快""尽快安排""下周"的一律打回。
- 第四步：把上次会议的行动项清单也丢进去，让它做滚动：上次没完成的，本次重新列出并标注滞后天数。
- 第五步：确认后导出，发到群里，并让每个人回复确认。



图 9-7 会议转写到行动项：决定、责任人、日期三关

图 9-7 作者根据公开资料绘制

可复制提示词

你是我们项目组的会议记录员。下面是一份会议转写文本，请把它整理成行动项清单。

参会人：（列出姓名和部门）

第一步：给我一张"行动项表"：

| 编号 | 行动项描述 | 责任人 | 协助人 | 截止日期 | 完成判据 | 状态 |

填写规则：

- 编号：从 A-001 开始
- 行动项描述：必须以动词开头，例如"完成蜗杆齿面粗糙度检验方案的编制"。不许出现"讨论""关注""跟进""推动"这类无完成标志的动词
- 责任人：只能从参会人名单里选，且每条只能有一个。如果会议里没有明确指定责任人，责任人栏写"待定"，并在备注里写明会议里谁提的这件事
- 截止日期：必须是具体的年月日。如果会议里只说了"尽快""下周""月底"这类模糊表达，就按会议日期推算一个具体日期填进去，并在该行后面加"（推算）"标记
- 完成判据：一句话，写清做到什么程度算完成，要能判定。例如"输出一份含检验项目、量具、抽样方案的 Excel 并发给质量组"
- 状态：一律填"未开始"

第二步：给我一张"未形成行动项的讨论"表，把会议里讨论过但没有结论的话题列出来：

| 话题 | 讨论了什么 | 卡在哪 | 建议下次会议怎么处理 |

第三步：给我一张"待决策事项"表，把需要领导或客户拍板的事单独列出来：

| 事项 | 需要谁决策 | 需要什么信息才能决策 | 不决策的后果 |

输出要求：

- 三张表都用 Markdown 表格
- 不要写会议总结，不要写"本次会议高效务实"
- 不许编造参会人姓名，名单里没有的人一律写"待定"
- 如果某段话你判断不出是不是决定，就放进第二张表，不要硬凑成行动项
- 最后把三张表另存为桌面的 meeting_actions_（会议日期）.xlsx，三个工作表命名为 行动项、待讨论、待决策

过程与判断点

我拿一次真实的周会转写跑过，转写文本约 8200 字，5 个说话人，会议 62 分钟。

判断点一：行动项的数量。它给出 14 条。我自己听了一遍录音，能确认的决定是 15 条。差 1 条，是那种会上说了半句、被别的话题打断的。这个准确率我满意，比我自己边听边记的准确率高，我开会时经常忙着说话忘了记。

判断点二：有没有把讨论当成决定。这是这个场景最关键的一点。转写里有大段讨论蜗杆粗糙度标准的对话，最后没人拍板。它把这段放进了第二张"未形成行动项的讨论"表，标注"卡在：Ra 数值取 0.2 还是 0.4 没有定，需要查客户企标"。这个判断是对的。

它有一次做错了，把一段讨论直接写成了行动项"优化蜗杆磨削工艺"。我后来在提示词里加了那句"不许出现讨论、关注、跟进、推动这类动词"，这种错误就少多了。

判断点三：日期栏里有多少个"（推算）"标记。这次 14 条里有 5 条带推算标记。这 5 条就是你需要当场在会上确认的。看到推算标记多，说明你们开会习惯不好，说事不带日期。

判断点四：责任人栏有没有"待定"。这次 3 条待定。我在会上直接把这 3 条念了一遍，各用了 20 秒认领完。这件事如果拖到会后发邮件问，通常要来回两天。

产物长什么样：桌面一个 Excel，三个工作表。行动项表发群里，待决策表单独发给领导。

翻车清单

翻车一：它编人名。转写里如果说话人标识不清，它会自己补一个名字，而且补得像真的。补救动作：必须给参会人名单，并在提示词里写明"名单里没有的人一律写待定"。拿到表之后，责任人栏逐行扫一遍，确认每个人都在名单里。

翻车二：它接受"尽快"当日期。会议里说"尽快"，它会老老实实写"尽快"。补救动作：提示词里强制它推算具体日期并加"（推算）"标记。这样你一眼就能看出哪些日期不是会上定的。

翻车三：它漏掉会议最后五分钟。会议最后五分钟经常是补的杂事，转写质量也最差。它有时候读到后面就松了。补救动作：单独追一句："会议最后 10 分钟的内容单独再过一遍，列出这段时间里新增的行动项。"我试过，有一次最后五分钟里有 2 条重要事项被漏了。

验收标准与变体

验收标准：

1. 行动项表在会后 10 分钟内发到群里。做不到 10 分钟，这套流程的价值就打对折。
2. 每条行动项有唯一责任人，且责任人在群里回复了确认。没回复的，会后单独找。
3. 带"（推算）"标记的日期，开会当场念一遍确认。
4. 下次会议开始时，先过一遍上次行动项的完成情况。这一步用 AI 做滚动，30 秒出结果。

变体 A：邮件收件箱转行动项。把一周的项目相关邮件导出成一个文件，让它按同样格式提取行动项，标注每条来自哪封邮件。我用这个做每周一的开局，比翻收件箱快。

变体 B：评审意见跟踪。设计评审会产生的意见，按同样格式整理，多加一列"关闭条件"和"关闭日期"。客户审核意见跟踪也用这个，尤其是客户给的审核清单，逐条跟踪关闭状态。

变体 C：个人待办整理。把你自己一天里记在便签、微信、邮件里的零散待办，一次性丢给它，让它按紧急度和依赖关系排出顺序，输出今天要做的三件事。这个变体我每天都用。

老兵说：我用这套流程之后，省下整理纪要的 40 分钟只是小头，真正的变化是会上开始有人说日期了。因为大家都知道说完会被记下来，还带推算标记，丢人。工具改变行为，这话说起来像管理鸡汤，但它是真的。

小白词典

SOR: 客户写给供应商的需求文件。它是验收依据，不是技术方案。

工况参数: 产品干活时的外部条件，扭矩、转速、温度、循环次数这些。

可验证性: 一条需求能不能判定合格。写不出判定方法的需求，等于没写。

CSR: 客户在通用标准之外额外加的特殊要求，常藏在附件里。

WBS: 工作分解结构。把项目拆成能在两周内干完、只有一个负责人的小任务。

关键路径: 决定总工期最长的那条依赖链。这条链上晚一天，项目晚一天。

DV 与 PV: 设计验证与生产验证。DV 验设计对不对，PV 验量产做不做得出来。

DFMEA: 设计失效模式与影响分析，一张把"怎么坏、多严重、为什么、怎么提前发现"列清楚的表。

RPN 与 AP: 风险优先数与措施优先级，两种给风险排序的方法，汽车行业现在更常用 AP。

变位系数: 齿轮加工时刀具的偏移量。正变位能加厚齿根、避开根切。

传递误差 TE: 实际输出角度跟理论角度的差值，齿轮噪音的主要来源之一。

DRI: 一件事的直接负责人，只能有一个。

你的岗位怎么用

设计工程师: 场景 17 最该先抄。你每次做方案对比前，先花 5 分钟把评价维度定死，再让 AI 填数字。场景 16 的 DFMEA 表拿来当评审会起点，别当结论。

试验工程师: 场景 18 和场景 13。会议行动项你每周都要整理，3 分钟出表这件事最省时间。需求拆解出来的试验类条目，直接就是你的试验大纲输入。

质量工程师: 场景 14 和场景 16。标准条款提取要全查不要抽查，这是给你的规矩。DFMEA 表里的探测度打分，AI 会偏乐观，你把关最合适。

项目经理: 场景 13 和场景 15 是你的主战场。需求条目表直接进你的项目管理系统，WBS 和依赖关系表直接进你的甘特图。记住一件事：长周期件的数据必须你自己给 AI。

采购 / 供应商管理: 场景 17 的变体 A（供应商方案对比）和场景 16 的变体 B（供应商风险问卷）。发给供应商之前用 AI 把问卷做细，收回来的答案质量会高一截。

研发主管: 场景 18 的待决策表单独看。它把"需要谁决策、缺什么信息、不决策的后果"三列摆出来，是你的决策清单。

本章交付物

一份需求翻译提示词组，六段，可直接复制。

做法: 把本章六个场景的可复制提示词各复制一遍，存进一个文本文件，命名"第 9 章提示词组"。

每段提示词里的括号部分，是你要填的变量。六个场景的变量分别是：产品描述、标准文件、任务清单与资源、零件清单与经验、方案描述与维度表、参会人名单。

第一周只做一件事：拿场景 18 试一次。你手上一定有一份会议纪要或者录音转写，3 分钟出结果，当天就能看到效果。

第二周试场景 13。找一份你现在手上的客户文件，跑一遍，看看它找出来的冲突条数是不是比你多。

六个场景不必全用。我在自己项目上常用的是 13、17、18 这三个，14 因为核对成本高用得少，15 和 16 在项目关键节点才用。

本章金句

一份 40 页的需求文件，真正决定成败的条款不超过 12 条，剩下的是给将来吵架用的。

摊平条款这件事 AI 做得比人好，因为它不会因为"这页看着像法务条款"就跳过去。

写不出判定方法的需求，等于没写。当天发邮件问客户，不要自己猜。

方案对比最难的一步发生在比之前：先定好比什么。没有评价维度的对比，最后是谁嗓门大谁赢。

依据栏里出现"较好""略优"，这一格就等于没填。

所有带单位的数字自己按一遍计算器，纯逻辑判断可以信它。

AI 不讨好你，算出来不够就是不够。

（下一章：造型与几何。需求定完了，接下来让 AI 真正碰到你的三维模型，从一句话到一张能下生产的图纸。）

第 10 章

造型与几何：让 AI 真的去动你的建模软件

AI 画出来的不是一张图，是它在你的建模软件里按下的每一次鼠标；它强过你的地方，是按下第三百次的时候手还不抖。

今年国庆假期，我在家里的电脑前坐了一整天。屏幕上那个方向盘，是一点一点长出来的。

十一点十四分，第一条草图出现在 SOLIDWORKS 的前视基准面上，一个椭圆，32 毫米乘 38 毫米。下午两点零二分，这条椭圆开始沿着一条 D 形中心线跑，跑到一半停了。晚上八点整，它跑完了最后一圈，屏幕上出现一台造型完整的运动型方向盘：外径 370 毫米，轮圈两侧对手指留了 R16 的倒角，中心是三辐骨架和一圈九颗的换挡指示灯。

真正让我放下手里那杯凉透的咖啡的，不是它的样子，是它交上来的清单。

二十三个 `.SLDPRT` 零件文件，一个 `.SLDASM` 总装文件，六张 `.SLDDRW` 工程图，一份 STEP 交换文件，一段两百多秒的建模过程视频。总装里四十四处零件实例，干涉检查零对。它给我发的最后一条汇报是："二十三种零件全部单实体通过，十四种零件的体积与闭式解一致，最大相对误差 $4.8e-15$ 。"

我把这段话读了两遍。我做微电机和智能执行器研发十一年，带过团队，审过上百份图纸。一个人类工程师从零把这个东西按规矩画完、配上参数表、校核、出图，我保守估计三个完整工作日。它用了不到两个小时，中间没休息，也没问我要不要先喝口水。

那天夜里我把整个交付包翻了一遍。二十三个零件每一个都有自己的特征树，名字是我看得懂的那种，不是"拉伸1""切除2"这种没法维护的天书。六张工程图是 GB 模板的 A3，五个剖视，二十三处关键尺寸，每一处都落在真实模型边上而不是靠手画。我把原来那个零件目录临时改了名，只从解压出来的副本里打开总装，四十四组件全部解析成功，干涉依然为零，体积差在 $1e-16$ 这个量级。

那一刻我意识到一件很具体的事：AI 建模跟图像生成是两回事。它不是在画布上给你糊一个看着像方向盘的东西出来，它是真的在 SOLIDWORKS 里画线、拉伸、切除、倒圆角，一笔一笔把这套几何动作做完。正因为它是这么干的，它画出来的东西才能二次编辑、能出图、能进车间，也才能被你验收。

这一章把这套打法拆成六个场景，让你也能对着自己的软件说一句人话，然后看着它把活干完。

有一点先交代清楚，免得你读完有误会。下面场景 20 到场景 23 这个方向盘案例，当年驱动建模的那一头不是 WorkBuddy，是我另一套 Agent 环境跑的同一条工具链（MCP 桥加 SOLIDWORKS 的 COM 接口）。WorkBuddy 走 MCP 走的是同一条路，能力级别一样，我在相关场景末尾都留了 WorkBuddy 侧的配置说明。书里保留这个案例，是因为它的过程、它的失败、它的验收数字都是我亲手跑出来的真东西，比任何演示都值钱。

下面是本章六个场景的速览，你先挑跟你有关的那个看。

场景	场景名	人工耗时	AI 耗时	可信度
19	从一句话到第一个零件	约 20 分钟	约 3 分钟	★★★ 实测
20	方向盘三维数模：完整案例	约 3 人天	1 小时 49 分	★★★ 实测
21	从数模到二维工程图	约 1.5 人天	约 35 分钟	★★★ 实测
22	数模体检：单位、选边、体积累计	约半天	约 10 分钟	★★★ 实测
23	装配体与干涉检查	约 1 人天	约 25 分钟	★★★ 实测
24	工业造型与渲染 Rhino / Blender 侧	视效果数小时	视情况	★ 调研

场景 19 从一句话到第一个零件：简单零件建模入门 ★★★

一句话背景

画图这件事，绝大多数工程师的日常其实不是设计，是把一块板上的孔按规矩排一遍。一块安装板上四个孔、中心一个过孔、四个角倒圆角，扎实的手要二十分钟：画线、标尺寸、拉伸、打孔、加约束、倒角、保存。烦不在难，在这活每来一个新尺寸就要重做一遍。这个场景教你把这套活交出去，三分多钟出活，改尺寸再一句话。

名词小词典

草图 (Sketch)：在三维软件里先画的那个平面轮廓，比如矩形、圆。轮廓画完才知道往哪个方向长出厚度。类比：先在纸上剪出一个形状，再把它立起来。

拉伸 (Extrude)：把画好的平面轮廓沿垂直方向拉出厚度，得到一个立体。这是三维建模里用得最多的动作。

切除 (Cut)：在已有的立体上挖掉一块。打孔就是切除。

圆角 (Fillet)：把尖锐的棱磨成圆弧，代号写作 R。倒圆角既是为了不割手，也是为了零件好成型。

特征树：软件左侧那棵树，按顺序列着你做过的每一个动作。三维模型不是一张图，是一串动作的回放。

开工前准备

逐条核对，少一条都会卡住：

1. 一台 Windows 电脑，SOLIDWORKS 已安装并能正常打开（我这套是 SOLIDWORKS 2026，正版授权）。
2. Python 环境，版本 3.10 或以上，装好 pywin32：`pip install pywin32`。
3. 一个 MCP 桥，比如社区里的 SolidWorks-MCP（我这次用的版本是 0.2）或者 SolidPilot。它的作用只有一个：把 AI 说的话翻成 SOLIDWORKS 能听的底层调用。

4. WorkBuddy 里把这座桥登记成 MCP 服务器（方法见本场景结尾）。
5. 一个干净的工作目录 `D:\AI-CAD\demo01`，路径不要有空格和中文。
6. 先把 SOLIDWORKS 手动打开，停在空白界面。绝大多数连不上的报错，根源都是软件没开。

操作步骤

跟着点就能完成，全程不用敲一行代码：

1. 打开 SOLIDWORKS，确认软件左下角处于就绪状态。
2. 打开 WorkBuddy，新建一个对话，模式选 Agent。
3. 确认 MCP 服务器已经挂载，工具列表里能看到建草图、拉伸、切除、倒圆角、保存这类工具。看不到就是配置没生效，别往下走。
4. 把下面那段提示词整段复制粘贴进输入框，回车。
5. 盯着它的汇报看。它会一个动作一个动作往下报，每个动作都带真实数字。
6. 它喊停的时候，去工作目录里看文件，`.SLDPRT` 和 `.step` 应该都在。
7. 打开软件左边的特征树，看五个特征的名字是不是照你的规矩命名的。
8. 自己动手量一次：打开工具菜单里的质量属性，读出体积，跟下面的数字对一遍。

从一句话到第一个零件

八个动作的链条，每一步都要回报真实数字



提示词里把单位换算规矩写死，回读值一律标注单位

图 10-1 从一句话到第一个零件：动作链与每一步的回报内容

图 10-1 作者根据公开资料绘制，画面中不含任何第三方零件图纸。

可复制提示词

整段复制，一个字都别改，路径换成你自己的：

你现在可以通过 solidworks 这台 MCP 服务器，直接操作我电脑上已经打开的 SOLIDWORKS 2026。请按下面的要求建一个零件。每一个动作做完都要向我汇报一次真实返回值，不许用“已完成”“一切正常”这种空话带过。

【零件】一块 120 × 80 × 12 毫米的矩形安装板，四个直角各倒 R10 圆角，四个 Ø9 安装通孔，中心一个 Ø40 过孔。

【单位规矩】我说的全部是毫米。SOLIDWORKS 接口入参的内部单位与毫米不同，你负责换算；回读给我看的每一个结果一律换算回毫米，并在数字后面标注单位。

【草图】在“前视基准面”上画 120 × 80 的矩形，矩形中心与原点重合。

【主体】沿正 Z 方向拉伸 12 毫米。

【孔】四个 Ø9 的通孔，孔中心距最近的两条边各 12 毫米。再在正中心打一个 Ø40 的通孔。

【圆角】四个直角各倒 R10 圆角。

【命名】特征必须按规矩命名：草图1_底板、拉伸1_底板、切除1_安装孔、切除2_中心孔、圆角1_倒角。

【校核】拉伸完成后立刻回读体积与包围盒；全部做完后回读最终体积，与解析值比对，报出相对误差。

【保存】存为 D:\AI-CAD\demo01\mount_plate.SLDPRT，再导出 D:\AI-CAD\demo01\mount_plate.step。

【纪律】任何一步失败，先把 SOLIDWORKS 返回的原始报错原样贴给我，然后停下来等我指示。不许自己猜原因，不许带着报错继续往下做。

过程与判断点

它应该返回一段像这样带数字的汇报。这段里的每一个数字你都能手算复核，这就是可验收的意思：

- > 草图1_底板：120.00 × 80.00，完全定义，无欠约束。
- > 拉伸1_底板：深度 12.00 mm，体积 115200.00 mm³，包围盒 120.00 × 80.00 × 12.00。
- > 切除1_安装孔：4 处 Ø9 通孔，单处去除 763.41 mm³，合计 3053.63 mm³。
- > 切除2_中心孔：1 处 Ø40 通孔，去除 15079.64 mm³。
- > 圆角1_倒角：4 处 R10，合计去除 1030.09 mm³。
- > 最终体积 96036.86 mm³，解析值 96036.72 mm³，相对误差 1.5e-6，通过（门槛 1e-5）。
- > 已保存 mount_plate.SLDPRT，已导出 mount_plate.step。

这段汇报里我盯三个地方。

第一，最终体积对不对。手算方式是：底板 120 乘 80 乘 12 得 115200；四个孔每孔去除体积是 π 乘 4.5 的平方乘 12，得 763.41，四孔合计 3053.63；中心孔是 π 乘 20 的平方乘 12，得 15079.64；四个 R10 圆角去除的是 4 乘（10 的平方减 π 乘 10 的平方除以 4）再乘 12，得 1030.09。四项减完约等于 96036.7 立方毫米。你自己拿计算器按一遍，三十秒。

第二，特征名字。如果它建的树里名字全是默认的"切除1""切除2"，说明你的命名规矩它没吃进去。这不是外观问题，三个月后你要改那个孔的位置，你会在这棵树里找半天。

第三，它有没有沉默。干这种活最危险的信号不是报错，是它突然安静三十秒然后说一句已完成。报错说明它在动作并且愿意告诉你，沉默说明它可能跳步了。

一块安装板的体积收支

三个动作各减去多少，最后剩下多少



手算方式：底板减四孔减中心孔减圆角，约三十秒算完

图 10-2 一块安装板的体积收支：三个动作各减去多少，最终剩多少

图 10-2 作者根据公开资料绘制。

翻车清单

翻车一：模型大了整整一千倍。症状是拉伸出来的板有一个操场那么大，或者反过来小到看不见。原因是单位，软件接口的内部单位跟你说话用的单位不是一回事。补救：在提示词里就把换算规矩写死，并要求阅读值必须标注单位。

翻车二：连不上软件，报一堆英文。症状是第一条指令就挂了，报错里出现 COM、dispatch 这类词。原因是 SOLIDWORKS 没打开，或者它是以某个权限级别开着而你的桥不是。补救：两个程序用同一个权限级别重开，先把软件手动打开再说话。

翻车三：路径看着对，文件却没落地。症状是它汇报已保存，目录里空空如也。原因通常是路径里有空格或中文，或者那一层目录根本不存在。补救：路径一律英文、无空格、带盘符，动手前先让它列一遍目录确认存在。

验收标准与变体

验收四条，两分钟做完：

1. 打开特征树，五个特征齐全，命名合规。
2. 质量属性里读出的体积在 96036 附近，误差不超过千分之一。
3. 用测量工具量一次侧边距，四个 Ø9 孔的中心都应距最近的两条边 12 毫米。
4. 把生成的 .step 拖回 SOLIDWORKS 重新打开，模型能读回且体积不变。

变体一：一套数八个规格。把你手上一族零件的所有尺寸列成一张表，让它一口气生成八个 .SLDPRT。过去这活是一下午，现在是一条提示词加一遍抽检。注意要求它每种规格单独出一份体积校核，别只给你最后一行的数字。

变体二：给三维打印出活。把最后的保存指令换成另存 STL，导出单位选毫米，三角面公差 0.01。打印件的验收动作变成围长和壁厚，不再是体积。

变体三：拿你手上真实的一个垫圈、一个支架开刀。这是最重要的一条。别拿书上的例子练手，拿明天就要交的那个零件练，你会立刻知道它哪里好用哪里不行。

老兵说：第一个零件千万别画方向盘，要画一块板。板的价值在于你能在一分钟内用手算验完它的体积，而验得完的东西，你才敢把下一个活交出去。

场景 20 方向盘三维数模：一次跑通的完整案例 ★★★

一句话背景

一台外径 370 毫米的运动型方向盘，二十三种零件、四十四处零件实例、干涉零对、六张工程图、一个能独立打开可二次编辑的交付包。人工按公司规矩画完约三个工作日。这套打法的实测成绩是建模执行段一小时四十九分。这是全章最厚的一节，因为它是一份可以照抄的完整作业。

名词小词典

数模：三维数学模型，也就是零件在软件里的立体样子。它不是一张图，是一堆可以参与计算的实体。

扫掠（Sweep）：让一个截面沿着一条指定的路径走一遍，走过的空间就是你要的形状。握持圈就是这么来的：一个 32 乘 38 的椭圆截面，沿一条 D 形中心线跑一圈。

圆角（Fillet）：把两个面之间的尖锐交线磨成圆弧。方向盘握持圈两侧是 R16，这个尺寸直接决定手感。

外径 / 内径：对一个环形零件来说，外径是外轮廓的最大直径，内径是中心那一圈空的直径。方向盘的 370 和 302 这两个数决定了它的整体尺寸。

STEP：三维模型的通用交换格式，工业界的普通话。你用 SOLIDWORKS 画的，别人拿 CATIA 也能打开。

MCP：一套让 AI 操控其他软件的通用插座标准。本章里它指的是把 SOLIDWORKS 的能力暴露给 AI 调用的那座桥。

COM：Windows 上软件之间互相调用的一套老接口。SOLIDWORKS 之所以能听 AI 的话，是因为它对外开了这道门。pywin32 就是 Python 一侧用来敲门的库。

开工前准备

这台一眼看不到底的活，准备条款一共六条：

1. 硬件：办公级工作站就够起步。SOLIDWORKS 吃的是 CPU 和内存，不吃显卡算力，AI 推理不在本机。
2. 软件：SOLIDWORKS 正版授权。工业软件没有捷径，正版才有稳定的 COM 接口。
3. Python 侧：Python 3.10 以上，装好 pywin32。
4. 桥：SolidWorks-MCP 0.2（社区开源，用 pywin32 调 SOLIDWORKS 的 COM），或 SolidPilot 这类同类开源桥。
5. AI 侧：一个能连续调用工具、看到返回值能自己纠错的 Agent。WorkBuddy 的 Agent 模式就属于这一类。
6. 纪律准备：把你想要的每一个数字先写在纸上。这一条看着最虚，实际最管用：外径 370、内径 302、轮圈高 36、圆角 R16、中心孔 $\varnothing 40$ ，一个都不能含糊。

操作步骤

这套活我总结成九步，顺序不能乱，乱一步就等着返工：

步	做什么	为什么必须在这一步
1	把需求写成带数字的清单	需求说不清，AI 必跑偏
2	冻结评分表，并记下文件哈希	规则后改等于没标准
3	写唯一参数源 design.py，生成 params.json	建模、装配、出图读同一份数
4	接口冒烟测试	先把工具的脾气摸出来，再干活
5	一个零件一个零件建，每步自校	错的地方当场抓，别攒到最后
6	总装并查干涉	装不上就等于白建
7	STEP 导出再导回	能交换才叫真数模
8	出二维工程图	车间只认图纸，不认屏幕

步	做什么	为什么必须在这一步
9	交付校验、封包、解压复开	拿证据说话，再交付

这九步里，最容易被新手跳掉的是第二步和第四步，而它们恰好是把活一次做成的分水岭。第三步的意义在于：建模的时候用了某个数，出图的时候就不能是另一个数。我在电机项目上见过有人三维改了壁厚、图纸没跟着改，车间照图加工，一批件报废。一个唯一参数源解决的就是这类事。

下面这组数据是我亲手跑出来的，它是整个案例的骨架：

项目	实测值	说明
握持圈外径 / 内径	370 / 302 毫米	设计给定值
轮圈高	36 毫米	扫描方向上的高度
轮圈圆角	R16	两侧外缘倒角
轮圈扫描结果	单实体，5 个面	扫描体面数就这么多
轮圈体积	1,060,058 立方毫米	软件回读值
与解析值之差	4e-16	达到浮点极限
零件种类 / 实例数	23 种 / 44 处	总装插 44 个组件实例
干涉对数	0 对	保存重开复验仍为 0
精确外形	370 × 351 × 91.8 毫米	含拨片的总高
STEP 往返	44/44 读回，体积差 1.45e-5	未过 1e-5 门槛，如实披露
建模执行段	1 小时 49 分	不含我中途的两次干预

AI 到 SOLIDWORKS 的三级链路

AI 写脚本，桥做翻译，软件真动笔

AI 侧

写出一段 Python 脚本，说清楚要画什么、用哪些数

MCP 桥

开源桥比如 SolidWorks-MCP 0.2，把 Agent 的话翻成底层调用

COM 接口

SOLIDWORKS 对外开的那道门，真正在软件里画线、拉伸、切除

WorkBuddy 走 MCP 走的是同一条路，能力级别一样

图 10-3 AI 到 SOLIDWORKS 的三级链路：AI 写脚本，桥做翻译，软件真动笔

图 10-3 作者根据公开资料绘制，图中仅为原理示意，不含任何第三方图纸内容。

可复制提示词

这是那一趟任务的总开工指令，整段可以照抄，尺寸换成你自己的：

【任务】用 SOLIDWORKS 建一台运动型 D 形平底三辐方向盘的三维数模，产出装配体与 STEP。

【单位】全部以毫米为单位说话。SOLIDWORKS 接口入参的内部单位与毫米不同，你负责换算；
 回读给我看的结果一律换算回毫米，并在数字后标注单位。

【造型】运动型 D 形平底三辐方向盘。

握持圈：32 × 38 椭圆截面，沿一条 D 形中心线扫掠；

中心线由 R169 大圆弧 + 两段 R45 相切过渡 + 一段平底直线组成，各段之间必须相切；

轮圈外径 370，内径 302，轮圈高 36，两侧外缘倒 R16 圆角。

三辐骨架：骨架深 14，轮毂中心带 Ø40 中心孔。

细节：换挡拨片 2 片，三色换挡指示灯 9 颗（圆周阵列），滚轮 2 处带滚花，中心人字徽标 1 处。

接口：装饰件、卡槽与辐条之间一律留 0.5 毫米单边间隙，不许用“大概”“差不多”。

【流程纪律】不许把整套方案在内部推理里推演完再动手。顺序必须是：

冻评分表 → 写参数源 → 接口冒烟测试 → 一个零件一个零件建 → 每个零件保存前自校

→ 全部建完再总装 → 干涉检查 → STEP 往返 → 交付校验。

每完成一个零件立刻汇报，附上：零件名、体积、包围盒、自校结论。

【参数源】所有尺寸、位置、颜色、装配变换集中写进 design.py，

由它生成 params.json 作为唯一参数源。建模、装配、出图三个阶段只读这一份文件，

不许各写一套数。改尺寸只准改 design.py。

【自校规矩】每个零件保存前必须确认三件事：单实体、体积为正、包围盒与解析值一致。

能算闭式解的零件一律用闭式解做硬校核，相对误差超过 1e-10 立即停下并向我报告，

不许改模型去凑数字。

【证据纪律】不许用“已完成”“没有问题”这类话汇报，每步汇报必须带接口回读的数字。

失败时把 SOLIDWORKS 的原始报错原样贴给我，然后等我指示。

第二条提示词，用来让它先做冒烟测试。这一步是整个九十分钟里性价比最高的：

在正式建模之前，先用六段最简脚本把 SOLIDWORKS 接口的脾气摸清楚，

每段跑完把返回值原样贴给我：

1. 偏置拉伸：确认正偏置的实际方向是朝 +Z 还是 -Z，负偏置会不会自动翻转。
2. 偏置切除：确认它从哪个面开始切，切到实体外会不会返回空。
3. D 形中心线：建一条单段路径做扫掠，回报实体数、面数、体积，与解析值比对。
4. 自带包围盒接口：对一个含样条的简单体取包围盒，与六个方向上的极值点算法对比，告诉我哪一个更紧。
5. 圆角：对一个已知边倒 R16，回报返回值的数据类型是什么。
6. 特征检索：用 FeatureByName 取一个已知特征，确认它作用在哪个对象层级上才有效。

六段全部跑完后，给我一张表：每条结论一句话，后面跟支持它的那个原始返回值。

没有跑出来的条目写“未验证”，不许写成“通过”。

第三条是参数源源文件的写法示例，这是我那套流程里的真实写法：

```
# design.py: 唯一参数源。改尺寸只改这里。
import json
from math import pi

P = {}

# 握持圈
P["rim_outer_dia"] = 370.0 # 外径
P["rim_inner_dia"] = 302.0 # 内径
P["rim_thickness"] = 36.0 # 轮圈高
P["rim_fillet"] = 16.0 # 两侧外缘倒角
P["sec_major"] = 38.0 # 截面椭圆长轴
P["sec_minor"] = 32.0 # 截面椭圆短轴
P["c_line_arc_r"] = 169.0 # D 形中心线大圆弧
P["c_line_trans_r"] = 45.0 # 两段过渡圆弧
P["c_line_flat_len"] = 118.0 # 平底直线段

# 骨架与轮毂
P["spoke_depth"] = 14.0
P["hub_bore_dia"] = 40.0

# 接口
P["iface_gap"] = 0.5 # 单边间隙, 毫米

# 派生量
P["sec_area"] = pi * (P["sec_major"] / 2) * (P["sec_minor"] / 2)

with open("params.json", "w", encoding="utf-8") as f:
    json.dump(P, f, indent=2, ensure_ascii=False)

print("params.json written, keys =", len(P))
```

过程与判断点

这一段我把那一趟真实跑下来的关键节点说清楚，包括它出错的地方。

握手之后它做的第一件事。它没有闷头就画，先写了一份评分表文件，还把文件的哈希值记下来。意思是：规则先定死，后面不管结果好不好看，都按这张表打分，不许临时改。这一步看着多余，实际是防止做完之后为了让分数好看而改标准。我在帶團隊做設計評審的时期吃过同样的亏。

它空转的那一段。有一轮我让它全力完成，它果真闷头在内部推理里把几何、接口、流程推了一遍又一遍，直到推理 token 烧到 3.2 万的上限，然后什么都没输出，一个工具也没调。我白等了将近七十分钟。后来我们改成先动手、边做边验证，把设计推演直接写进 `design.py`，不再让它在脑子里空转。这条教训比它做对的事更值得记：不要叫 AI 想清楚再动手，要叫它先动手再想。

建模过程跑了三遍。第一次做到后毂就中止了，因为它把 FeatureByName 这个接口用错了对象。第二次做到气囊盖圆角又卡住，因为圆角返回的是一个动态派发对象，跟它预期的类型对不上。两次失败的全部录像和日志都留着没删，清空记录从头重建。第三遍跑完：二十三种零件、四十四处实例，每个特征都命名、记录、重建。

每个零件保存前的三道断言。单实体、体积为正、包围盒精确。能算闭式解的还得拿解析值做硬校核。轮圈是最典型的一个：32 乘 38 的椭圆截面沿 D 形中心线扫描一周，软件回报的体积是 1,060,058 立方毫米，与解析值相差 $4e-16$ 。 $4e-16$ 是什么概念？双精度浮点的机器精度就在这个量级，也就是说这个数字已经精确到了计算机能表示的极限，再准就没有意义了。

这里有个细节值得记住：解析值不能靠套整圆公式去算。D 形中心线是一段大圆弧接两段相切过渡再接一段平底直线，三段曲率各不相同，脚本必须按段把总弧长积出来，再乘截面面积。这就是为什么“看起来差不多”的手算在这一步经常对不上，而机器算出来的数字能对到 $4e-16$ 。

它自己报道了唯一一项没通过。STEP 是工业界交换三维数据的通用格式，我定的验收门槛是体积相对差不超过 $1e-5$ 。它导出的 STEP，四十四个零件全部读回，但体积差 $1.45e-5$ ，卡在门槛外。它的做法让我很服气：没有改模型去凑数字，而是花时间做根因分析，最后定位到这个偏差来自 SOLIDWORKS 对椭圆曲面的体积累计分辨率，不是扫描几何本身的错误。换一种构造方式只会改变原生一侧的积分值，STEP 文件本身不动，那样通过只是数字游戏。于是它保留单特征扫描的轮圈，在交付里明明白白写未通过，并附上根因分析文件。这份敢把失败写进交付的态度，我觉得比它做对一百个零件都值钱。

同一道题，我用两个不同厂商的旗舰模型各跑过一次，两边都是当时的旗舰档。工具链是同一座 MCP 桥、同一套 SOLIDWORKS，变量只在模型和 Agent 一侧。结果摆在这儿：

对比项	本机 Agent 路线	云端 Agent 路线
三维几何有效性	零干涉，解析校核 $4.8e-15$	零干涉，解析校核 $4e-16$
STEP 往返	$1.45e-5$ ，未过门槛，如实披露	$2.21e-6$ ，通过
建模执行耗时	1 小时 49 分	47 分钟后中止
零件种类 / 实例	23 种 / 44 处	19 种 / 43 处
二维工程图	6 张全交付	未制作
可交付性	打包完成，解压复开通过	未打包，交付校验未过
中断原因	无	云端额度耗尽，线程被掐断

云端那一跑不是输在能力上。它的几何精度、它的 STEP 指标、它的速度三项都漂亮，输在最后关头撞上了额度墙，压缩包没打成。这也是我把这一整章写在本机路线上的原因：在工厂里有工程师在等结果的地方，额度这种不可控因素最好不要排在关键路径上。本机跑，它可以从下午一直干到晚上，中途重启软件、重跑校验都不受限。

方向盘九步作业流

每一步做什么、为什么必须在这个位置



第二步冻表与第四步冒烟最容易被跳掉，它们是能一次做成的分水岭

图 10-4 方向盘九步作业流：每一步的输入、输出与卡点

图 10-4 作者根据公开资料绘制。

方向盘交付包的五类产物

零件加总装加图纸加交换文件，能独立打开可二次编辑



交付包里最值钱的不是那二十三个零件，是那份敢写未通过的根因分析

图 10-10 方向盘交付包的构成：23 个零件加 1 个总装加 6 张图纸加 STEP

图 10-10 作者根据公开资料绘制，仅表示文件构成关系，不含任何第三方图纸内容。

翻车清单

翻车一：内部推理空转，七十分钟零输出。症状是它长时间没有调用任何工具，最后返回一句总结性的空话。根因是你在提示词里让它先把方案想清楚。补救：把指令改成先跑一个最小例子、看返回值再往下写，并且要求它每完成一个零件必须汇报。

翻车二：建模中途中止，报错指向某个特征名。症状是做到某个零件突然停下，报错信息里有 FeatureByName 或者“对象不支持”。根因是接口被调用到了错误的对象层级上。补救：在冒烟测试阶段就把每条要用的接口单独试一遍，把有效写法固定下来写进桥的封装里，后面统一调封装。

翻车三：STEP 差一点点过不去，就想改模型凑数。症状是它提议换一种构造方式重做，让 STEP 数字好看起来。根因是把验收当成考试而不是体检。补救：在提示词里明确写死不许为通过校核而更换几何构造，过不去就写原因和根因分析。这一句话值千金。

验收标准与变体

验收七条，按顺序做一遍约十分钟：

1. 打开二十三个零件文件，特征树可读，命名有规矩。
2. 抽查三个零件的质量属性，体积为正、单位为立方毫米、数量级合理。
3. 打开总装跑一次干涉检查，结果必须是零对，且报告中不含忽略项。

4. 导出 STEP 再导回，逐个零件比对体积，记录相对误差。
5. 保存关闭再重开一次，确认所有引用都能解析，没有找不到文件的提示。
6. 把原始零件目录临时改名，只从解压副本打开总装：四十四四个组件全部解析成功，干涉为零，体积差在 $1e-16$ 量级。
7. 自己动手改一处尺寸（比如把轮圈高从 36 改成 34），看整条链路能不能顺着 design.py 自动重算出新体积。

变体一：电机端盖。一个典型的三段式零件：拉伸主体、圆周阵列六个螺栓孔、中心轴承座加一个止口。它的验收动作是六孔位置度用坐标表核对，以及止口与轴承的配合尺寸。

变体二：换一整类完全不同的产品。微型执行器外壳、传感器支架、线束固定卡扣，这类产品 L 形和 U 形的小件最适合第一批上手，因为它们试错成本低，验起来也快。

变体三：一套参数源出三个规格。只改轮径（350、370、390），其余全部派生。验收动作是每个规格各出一份体积校核报告，并且特别留意轮毂比例有没有跟着失真。

老兵说：这套流程里最贵的不是时间，是我坐在那儿当把关人的那一百分钟。它四十四四个零件全对，是因为每一个零件都被要求当场自证；它最后一项 STEP 没过，也是因为这一条纪律。别省掉人一直在环里这件事，省了它，你就只是在等一份不知道对不对的图。

WorkBuddy 侧怎么接同一条链路

这一段是给 WorkBuddy 读者的。上面那一趟，AI 一侧换成一个能跑 MCP 的 Agent 就是同一件事。WorkBuddy 走这条路的配置只有三步：

1. 装上那座桥。在本机把 SolidWorks-MCP 或 SolidPilot 这类同类开源桥跑起来（许可证以项目主页标注为准），确认它能在命令行里被 Python 拉起。
1. 在 WorkBuddy 里登记。MCP 服务器通常以一段 JSON 配置登记，形态大致如下，具体配置文件的位置与字段名以你安装版本为准：

```
{
  "mcpServers": {
    "solidworks": {
      "command": "python",
      "args": ["-m", "solidworks_mcp", "--host", "127.0.0.1", "--port", "8765"],
      "disabled": false
    }
  }
}
```

1. 验一次握手。新建一个 Agent 模式的对话，让它跑一句列出这台服务器上所有可用工具。工具列表里出现建草图、拉伸、切除、倒圆角、保存、导出 STEP 这几项，就成了。看不到工具列表先别提建模需求，回头查桥有没有起来、SOLIDWORKS 有没有先开。

之后的活，就是本场景那段提示词原样粘贴进 WorkBuddy 的对话框。它说的每一句中文，都要经过同一座桥翻译成同一批 COM 调用。

【待补：WorkBuddy 中 MCP 配置的界面路径，以正式发布版为准。此处给出的只是通用配置形态。】

场景 21 从数模到二维工程图：六张图的产出过程 ★★★

一句话背景

车间里的老师傅和检验员看的不是屏幕里的立体模型，是印在纸上、标着公差和剖视的二维图纸。这一步过去要占掉整个设计任务的四成工时。这一趟下来我最大的感受是：会建漂亮三维的人不少，能交出合规二维图纸的才稀缺。

名词小词典

工程图：把三维数模投影成二维图纸，标上尺寸、公差、剖视、明细表，专门为加工和检验服务。

三视图 / 剖视：三视图是从三个正方向看过去的样子；剖视是把零件假想切开，看里面。

标题栏：图纸右下角那张表，写着图名、图号、比例、材料、设计者。缺了它图纸就是废纸。

明细表：也就是 BOM，物料清单。这方向盘由哪些零件、各几个组成，一表列清，它通常由装配体的原生表自动生成。

序号：图纸上给每个零件编的号，与明细表的行号一一对应，工人照号找件。

GB 模板 / A3：国家标准的图纸模板和纸张幅面。工程图不是随便画的，是按标准框出来的。

开工前准备

1. 场景 20 的交付成果必须完整躺在本地：二十三个零件、一个总装。
2. SOLIDWORKS 里准备好符合你公司标准的图纸模板（我这次用的是 GB 模板，A3 幅面）。模板没有的话，先让工程师做一份出来，这一步省不掉。
3. MCP 桥处于可用状态，列工具时应当能看到新建图纸、插入视图、生成剖视、插入明细表、自动序号这些工具。
4. 定好图纸清单：哪几张、每张画什么、各几个剖视。清单不定，AI 会替你定，然后你就等着重做。

操作步骤

1. 先写一版图纸清单：第一张总装图含明细表与序号，后五张挑五个关键零件，每张至少一个剖视，代号从 A 排到 E。

- 2. 让 AI 挑一个最难缠的零件试画。我这次挑的是后毂，因为它面多、内部有孔，最容易暴露处理不了的地方。让它画三视图加一个剖视。
- 3. 把它在这一轮踩出来的问题逐条记下来。我这一轮记了七条，见下面的判断点。
- 4. 第一轮产物单独归档，不许删除。归档的意义是两个星期后有人问为什么改成这样，你能答得上来。
- 5. 修完那批问题之后才算正式开工，第二轮一次性出六张。
- 6. 逐处回读尺寸数值。每一处尺寸都要它动手去读模型里那个几何对象的真值，然后比对。
- 7. 生成明细表与自动序号，必须走原生表，不许手绘表格。
- 8. 全部通过后另存一套 PDF 备用，发给评审和外协的时候用得上。



图 10-5 二维出图流水线：试画一轮、修七类问题、第二轮全出

图 10-5 作者根据公开资料绘制。

可复制提示词

第一轮：

【任务】基于已建好的方向盘三维数模，出一套可直接发给车间和检验的二维工程图。

【形式】GB 模板，A3 幅面，共 6 张。

第 1 张：总装图，含 SOLIDWORKS 原生明细表与自动序号。

第 2 至 6 张：五个关键零件的零件图，每张至少 1 个剖视，剖视代号从 A 排到 E。

【尺寸】全图合计标注 23 处关键尺寸。每一处尺寸必须落在真实模型边上，即与该尺寸的几何对象关联。不许用草图点手绘尺寸，不许出现悬空尺寸。标注完成后逐处回读数值并与几何真值比对，不一致的立即重标，并把不一致的那一条记入交付说明。

【标题栏】填写图名、图号、比例、材料、设计单位。比例按实际取值，不许沿用模板默认值。标题栏里不许出现任何占位符。

【明细表】总装图走 SOLIDWORKS 原生明细表，行数等于零件种类数 23，自动序号 23 个，序号与明细表行号一一对应，序号不许压到轮廓线上。

【纪律】第一步先从全部零件里挑一个零件（后毂）试画三视图加一个剖视，把暴露出来的问题全部列清楚修掉，再正式出六张。第一轮试画产物单独归档，不许删除。每一个动作做完都要汇报真实返回值，不许用“已完成”概括。失败时把软件的原始报错原样贴给我，停下来等我指示。

第二轮，把上面那段的第一段替换掉即可：

【第二轮】试画阶段暴露的七类问题已修复，现在正式出六张。

按图纸清单逐张生成，每完成一张立刻汇报：图纸名、视图数、剖视代号、

尺寸数与回读比对结果、标题栏字段、明细表行数与序号数。

六张全部通过后，逐个文件另存 PDF 到同目录下的 pdf 子目录。

过程与判断点

第一轮试画后毂，它踩出七类问题。我把它们原样列在这里，因为这就是你要替它预支的经验：

#	问题	表现
1	剖切线画错空间	剖视切的位置跟你想看的那个截面不是一回事
2	自加注释不显示	注释写进去了，图上看不见
3	标题栏比例不对	沿用了模板默认值，跟视图实际比例不一致
4	自动序号标不出来	序号工具在总装图上没有任何反应
5	选边失败	标注尺寸时选不到那条边
6	尺寸落到图框外	尺寸数字跑到图纸边界以外，打印出来是截断的
7	标题栏显示占位符	字段没被替换，打印出来是 NAME、SCALE 这种字样

七条修完，第二轮六张一次通过，战绩是这样：

- 五个剖视，代号从 A 排到 E，剖切位置都对上了想看的内部结构。
- 二十三处关键尺寸，全部落在真实模型边上，数值与几何真值逐处核对一致。这一条最花时间，也最值钱。
- 明细表二十三行，自动序号二十三个，序号与明细表行号一一对应。
- 标题栏字段齐，比例与视图实际相符。

如果你只打算看一处，我建议 SSH 的原则换成这条：随便挑一个尺寸，用软件的测量工具去量模型上对应的那条边，看看数值是不是跟标注的一模一样。这是唯一一条不能靠肉眼判断的验收。

六张 GB A3 图纸的结构

一张总装图带明细表，五张零件图各带一个剖视

第 1 张 总装图

原生明细表 23 行，23 个序号，——对应

第 2 张 零件图

剖视代号 A，共五张零件图，代号排到 E

第 3 张 零件图

剖视代号 B，尺寸落在真实模型边上

第 4 张 零件图

剖视代号 C，尺寸带单位与公差要求

第 5 张 零件图

剖视代号 D，标题栏比例按实际取值

第 6 张 零件图

剖视代号 E，标题栏不许出现占位符

验收要盯的一处：序号压线或乱序，多半是实例序与种类序混了

图 10-6 六张 GB A3 图纸的结构：一张总装图带明细表，五张零件图各带剖视

图 10-6 作者根据公开资料绘制，示意图中的图框与标题栏仅为结构示意图。

翻车清单

翻车一：尺寸是画上去的，不是量出来的。症状是图上看不出毛病，改了模型之后尺寸纹丝不动。根因是它为了省事用了手动绘制的尺寸而非与几何关联的尺寸。补救：验收时必须挑三处尺寸做改模型看尺寸跟着变的测试，跟不上就是废标。

翻车二：标题栏里藏着占位符。症状不明显，往往到打印后才被发现。补救：出图后立刻让它把每张图的标题栏字段读一遍打出来，你逐行核。这两分钟能救一次尴尬。

翻车三：序号压线或者乱序。症状是序号指到别的零件上，或者跟明细表行号对不上。根因是总装里的四十四处实例是实例序，而明细表里要的是种类序，两套顺序天然不同。补救：出序号之前先让它按零件种类排一次序，再重新生成序号，并逐条核对序号与行号。

验收标准与变体

验收六条：

1. 六张图的标题栏字段齐全，无占位符，比例与视图实际一致。
2. 五个剖视代号连续，剖切方向与视线方向一致。
3. 抽查五处尺寸：标注值与模型测量值完全相同，不是接近。
4. 明细表行数等于零件种类数，序号数等于行数，序号无压线、无重复。
5. 打印一张 A3 出来，拿尺子量图上一条已知 100 毫米的边，按图纸比例换算后应当等于 100。
6. 把 PDF 发给一个不看三维的人，问他能不能照图说出这个零件怎么加工。说不出来，图就是没做对。

变体一：出爆炸图加明细表。总装图配一张爆炸视图，说明书和维保手册上都用得到。

变体二：给外协厂出包。一次打包 PDF 加 STEP，PDF 给人看，STEP 给对方的编程软件读。这时候验收重点转向 STEP 的单位声明和你公司该标的版本号。

变体三：反向校验。把六张 PDF 丢回给 AI，让它照着图纸重建模型，再把重建出来的体积与原模型比对。这是对图纸完整性最狠的一种检验，也很接近真实场景里的来图还原。

老兵说：三维是给设计看的，二维是给车间看的。模型人人都想画得漂亮，图纸上一个尺寸错，铣床就多走一刀，那刀花的是真金白银。

场景 22 数模体检：毫米单位制的六个陷阱与三个自检动作 ★★★

一句话背景

AI 建完模，你签字之前必须做体检。体检只有三件事：单位有没有搞错、边有没有选错、体积能不能对上。三件事做完大约十分钟，能挡掉工程现场九成的事故。这是本书里最实用的一节，建议直接贴在显示器边上。

名词小词典

单位制：软件内部用什么长度单位存你的数。你说毫米，它内部可能是米，中间隔着一千倍。

几何指纹：给一条边一个独一无二的身份记号，让 AI 能准确选中那条边，而不是靠编号。

解析值 / 闭式解：用数学公式直接算出来的理论值，不依赖软件。它是软件回读值的裁判。

帕普斯定理：一个截面沿路径扫掠出来的体积，等于截面面积乘以截面形心走过的路径长度。这是校核所有回转体和扫掠体的万能钥匙。

质量属性：软件对一个实体算出来的一堆数，包括体积、表面积、质量、重心。它是体检报告里最重要的一页。

开工前准备

1. 一份待检的数模，`.SLDPRT` 或 `.SLDASM` 均可，STEP 也行。
2. SOLIDWORKS 打开这份数模，MCP 桥处于可用状态。
3. 一支笔一张纸，或者一个打开的计算器。你要手算至少一个数。
4. 一份你自己写的、而不是 AI 写的验收门槛。我这套用的是体积相对误差 $1e-5$ 、干涉零对。门槛不许事后改。

操作步骤

1. 先量单位，不要先量尺寸。打开文件后第一件事是让 AI 报出文档的当前单位设置，以及它是从哪儿读到的。
2. 量一条已知长度的基准边。比如方向盘的外径应当是 370 毫米。如果它报出 0.37，那你遇到了米；如果报出 370000，那是另一种单位。
3. 让它按几何指纹选边重做一次测量。不要接受第三条边这种表述，参见下面的指纹法。
4. 让它回读每个实体的质量属性，特别注意体积的单位后缀。
5. 手算一遍解析值，再让它推一遍解析值，两份解析值能对上说明公式没错，解析值与软件值能对上说明模型没错。少一份都不可靠。
6. 把体检结果写成文件存档，别只留在聊天记录里。

毫米单位制的六个陷阱

每一个陷阱都配了对应的补救动作

- !

陷阱一：接口单位与说话单位不一致
补救：提示词里写死换算规矩，回读值必须标注单位
- !

陷阱二：外来 STEP 带的单位不知道
补救：先看 STEP 文件头单位声明，导入后立刻量已知边
- !

陷阱三：质量属性单位随文档设置走
补救：验收数字以接口回读值为准，不以界面截图为准
- !

陷阱四：中间过程坐标被四舍五入
补救：坐标一律保留三位小数，关键间隙单独做断言
- !

陷阱五：跨软件单位制不一致
补救：导出前定死单位，目标侧再量一次基准长度
- !

陷阱六：图纸比例与单位混为一谈
补救：打印一张 A3，拿尺子量一条已知边反算

屏幕上的显示值不算数，接口回读值才算数

图 10-7 毫米单位制的六个陷阱与各自的补救动作

图 10-7 作者根据公开资料绘制。

毫米单位制的六个陷阱

这六条是我自己踩过的，每一条都配了补救动作：

#	陷阱	症状	补救
1	接口单位与说话单位不一致	模型大了整整一千倍或者小到看不见	提示词里写死换算规矩，回读值必标单位
2	打开外来 STEP 不知道它带的单位	导入后模型跟周围零件对不上	先看 STEP 文件头的单位声明，导入后立刻量一条已知边
3	质量属性的单位随文档设置走	同一个零件换台电脑报出的体积不一样	所有验收数字以接口回读值为准，不以界面截图为准
4	中间过程的坐标被四舍五入	0.5 毫米的设计间隙在系统里变成 0	坐标一律保留三位小数，关键间隙单独做断言
5	跨软件单位制不一致	SOLIDWORKS 的毫米件到了别的软件里成了千米	导出前定死单位，目标侧量基准长度双重确认
6	图纸比例与单位混为一谈	图上写 1 比 2，工人按 1 比 1 做	打印一张 A3，拿尺子量一条已知边反算

第三条我要多说一句。很多人验收的时候对着屏幕截图，说你看，这里写着 1,060,058，然后就签了。屏幕上的数随时可能被文档单位设置改掉。接口回读值才是硬证据，因为它直接来自几何内核，跟界面显示设置无关。

几何指纹选边法

这一步解决一个非常具体、非常高频的翻车：让 AI 给某条边倒圆角，它倒错了地方。

最常见的失败写法是给第三条边倒 R16。问题是特征树一改，边的编号就漂了。上一步加一个孔，原来的第三条边变成第七条。你以为倒的是外缘，它倒的是内圈。

几何指纹的思路是：不要给边编号，给边发身份证。

一条边的几何指纹 = 边的类型（直线 / 圆弧 / 样条）
 + 长度（保留三位小数）
 + 两个端点的坐标（保留三位小数）
 + 相邻两个面的夹角（保留两位小数）
 + 该边所属特征的名字

这套指纹里有五个字段，任意两条不同的边几乎不可能同时撞上五个字段。选边的时候让 AI 先按条件筛出候选，再用指纹做唯一性断言：

【选边规矩】不许用“第 N 条边”这种说法定位任何边。
 每次要倒圆角、切除、标注之前，先按下面的顺序做：

1. 取全部候选边，每条边输出它的几何指纹。
2. 按我给的目标指纹做匹配，输出匹配到的边数。
3. 匹配到的边数必须等于 1。等于 0 或大于 1，立即停下报告，不许就近选一条。
4. 匹配成功后，把选中边的指纹原样回贴给我做二次确认，再执行动作。

这条规矩值得单独存成技能。任何一次选边操作，匹配结果必须是唯一的。匹配到 0 条说明你的目标写错了，匹配到 2 条以上说明你的指纹不够细，两种情况都必须停下来。

可复制提示词

体检这件事要整段交给它，这一段可以直接复制，数字换成你自己的：

【任务】对我当前打开的这份数模做一次完整体检，产出可存档的体检报告。先别改任何东西。

【第一项：单位】先读出文档当前的长度单位设置，告诉我你是从哪个接口读到的，以及这个单位到毫米的换算系数是多少。没查清楚之前不许报任何尺寸。

【第二项：基准尺寸】量一条已知长度的外形边（这份数模的外径设计值是 370 毫米），把实测值报给我，保留两位小数，标注单位，并算出与设计值的相对误差。

【第三项：实体清单】逐个实体输出：名称、是否单实体、体积（立方毫米）、最小 $X / Y / Z$ 与最大 $X / Y / Z$ 。任何体积小于等于零的实体立刻停下报告。

【第四项：几何指纹选边】按下面这条规矩操作，不许用“第 N 条边”定位任何对象：
指纹 = 边的类型 + 长度（三位小数）+ 两端点坐标（三位小数）+ 相邻两面夹角（两位小数）+ 所属特征名。
每次选边必须输出匹配到的边数，边数不为 1 立即停下报告，不许就近选一条。

【第五项：体积校核】对能算闭式解的实体，先用数学公式推一遍解析值，把公式写清楚给我看，再与软件回读值比对，报出相对误差。门槛是 $1e-5$ ，超过就写未通过，并且不许改动模型去凑数字。

【第六项：STEP 往返】导出 STEP 再导回，逐个零件比对体积，记录相对误差，与上面的单位核查结论合并写进同一份报告。

【输出】把六项结论写成 D:\AI-CAD\check\check_report.md，
每一项占一段，每段结尾写“通过”或“未通过”，未通过的必须附原因分析。
口头汇报不许用“没问题”“一切正常”这类话。

过程与判断点

一段合格的体检汇报长这样，六行对得上才算过：

- > [单位] 文档长度单位码 = 0，到毫米换算系数 = 1000.0，来源 GetUserUnit(0)。
- > [基准尺寸] 外径实测 370.00 mm，设计值 370.00 mm，相对误差 0.0e+00，通过。
- > [实体清单] 共 23 个参与者实体，全部单实体，体积均为正。
- > [选边] 目标长度 118.000 mm，匹配到 1 条边，指纹：直线 / 118.000 / (0.0, 151.0, 0.0) / (118.0, 151.0, 0.0)，通过。
- > [体积校核] 握持圈软件值 1060058.00 mm³，解析值 1060058.00 mm³，相对误差 4e-16，通过。
- > [STEP 往返] 44/44 读回，最大体积相对差 1.45e-5，未通过（门槛 1e-5），已附根因分析。

这份报告里我最看重的不是那些通过的行，恰恰是最后那行未通过。写出来，说明体检是真的在做；写不出来，说明这份报告是走过场。

下面是这段体检脚本的骨架，用 pywin32 通过 COM 去读几何数据：

```

# cad_check.py: 数模体检, 单位、指纹选边、体积比对
import win32com.client as win32

TOL = 1e-5          # 体积相对误差门槛
TARGET_LEN = 118.0  # 目标边长度, 毫米

sw = win32.Dispatch("SldWorks.Application")
model = sw.ActiveDoc

# 1. 单位体检: 先问文档自己用的是啥单位
UNIT_TO_MM = {0: 1000.0, 1: 10.0, 2: 1.0, 3: 25.4} # 依次代表毫米、厘米、米、英寸
u = model.GetUserUnit(0)                          # 0 表示长度单位
scale = UNIT_TO_MM.get(u.GetUnits(), 1.0)
print(f"[单位] 文档长度单位码 = {u.GetUnits()}, 到毫米换算系数 = {scale}")

# 2. 几何指纹选边: 给每条边发身份证
def edge_fingerprint(edge):
    start, end = edge.GetStartVertex(), edge.GetEndVertex()
    ps, pe = start.GetPoint(), end.GetPoint()
    return {
        "type": edge.GetType(),
        "len": round(edge.GetLength() * scale, 3),
        "p1": tuple(round(c * scale, 3) for c in ps),
        "p2": tuple(round(c * scale, 3) for c in pe),
    }

hits = []
body = model.IBody2
for face in body.GetFaces():
    for edge in face.GetEdges():
        fp = edge_fingerprint(edge)
        if abs(fp["len"] - TARGET_LEN) < 0.01:
            hits.append(fp)
print(f"[选边] 匹配到 {len(hits)} 条边, 必须为 1")
for h in hits:
    print("    ", h)

# 3. 质量属性体检
props = model.Extension.CreateMassProperty()
vol = props.Volume * (scale ** 3)
print(f"[体积] {vol:.2f} mm^3")

# 4. 体积比对: 与解析值对照
ANALYTIC = 1060058.0
err = abs(vol - ANALYTIC) / ANALYTIC
print(f"[比对] 软件值 = {vol:.2f}, 解析值 = {ANALYTIC:.2f}, "
      f"相对误差 = {err:.2e}, {'通过' if err < TOL else '未通过'}")

```

这段脚本里的 API 名字，Dispatch、GetUserUnit、IBody2、CreateMassProperty 是我在自己这套环境里跑过的写法，函数名以你安装的 SOLIDWORKS 版本实际为准。看不懂这几行也没关系，真正的用法是把整段丢给 AI，跟它说照这个骨架给我写成能在我这个版本上跑的版本。

翻车清单

翻车一：拿软件自带的包围盒接口当精确外形。症状是算出来的长宽高跟实测值差出好几毫米，模型里有样条曲面时尤其明显。根因是那个自带接口对样条只取松散近似。补救：改用六个方向上的极值点分别取值来算，我这套环境的精确外形 370 乘 351 乘 91.8 就是这么量出来的。

翻车二：体积单位看错，还以为校核没过。症状是软件报的体积跟你的解析值差出一千倍甚至一百万倍，你满世界找几何错误，其实是单位错。补救：先做量纲检查，用已知密度的铝块估一次质量，看看吨位对不对，再做精度检查。

翻车三：解析公式是 AI 写的，写错了还没人发现。症状是两边数字漂亮地对上了，但它们是同一个错误的两种表达。补救：你自己手算一次最简单的那个数，比如一块板的体积，能对上再信它的复杂公式。

验收标准与变体

验收：一张六行体检查验单，每通过一行打一个勾：

检查项	通过条件
单位	文档长度单位已确认，回读值带单位后缀
基准尺寸	一条已知长度的边，实测值与设计值相对误差小于 1e-6
体积	每个实体的体积为正，与解析值相对误差小于 1e-5
干涉	零对，报告中不含忽略项
选边	每次选边的匹配结果唯一，恰好 1 条
STEP 往返	导出再导回，逐个零件比对体积并记录相对误差

变体一：来料体检。供应商发来一份 STEP，先用这套流程跑一遍，看单位、看有没有破面、看体积数量级。这一步赶在它进入公司流程之前做，能省掉后面一堆扯皮。

变体二：改模前后对比。改一处尺寸之前先存一份体检报告，改完再存一份，两份并排放置。差异出现在哪一行一目了然，评审会上不用争。

变体三：把体检存成一条技能。把上面那份六行查验单加这段脚本，打包成一个 WorkBuddy 技能。以后每份数模出笼跑一次技能，十分钟，报告自动落到同一个目录里。

老兵说：你可以不懂 pywin32，但你不能不懂单位。三个体检动作、十分钟，隔开的是这是图纸和这是一堆错数。

场景 23 装配体与干涉检查：多零件怎么一起交给 AI ★★★

一句话背景

单个零件建完只是开始。真正的麻烦在总装：四十四处零件实例要按规矩插进去，件与件之间不能打架，所有位置关系必须在保存重开之后还记得住。人工做这活大约一整天，一半时间花在为什么这个零件跑位了。这一趟的实测成绩是五阶段插入、干涉零对、重开后四十四处变换逐一回读误差不超过 $1e-9$ 。

名词小词典

装配体 (Assembly)：把若干零件按相对位置拼在一起的文件，后缀是 `.SLDASM`。

配合 (Mate)：告诉软件两个零件之间的位置关系，比如两个面贴在一起、两个孔同轴。

干涉 (Interference)：两个零件在空间里穿模了，撞在同一个位置上，物理上装不到一起。

间隙 (Clearance)：两个零件之间特意留的距离。0.5 毫米单边间隙在工业造型里是常态。

BOM：物料清单，列出这件产品由哪些零件组成、各几个。财务、采购、生产都照它吃饭。

轻量化：为了打开大装配不卡，把远处的零件只载入外形不载入细节的一种模式。验收时必须退出轻量化再检查。

开工前准备

1. 场景 20 的二十三个零件全部备齐，目录结构没有动过。
2. 有你公司统一的坐标原点规矩。这一条很多小团队没有，那就趁这次定一条。
3. 把可能互相打架的接口列成一张接口包络表：谁挨着谁、留多少间隙。这活看似多余，实测能挡掉一半返工。
4. 确认所有零件都不在轻量化状态。

操作步骤

1. 先出接口包络表，让 AI 把所有零件之间的邻近关系与预留间隙列一遍。我这套留的是 0.5 毫米单边间隙。
2. 统一坐标系。所有零件的原点规矩必须一致，不然插入之后会各自飞天。
3. 分五阶段插入实例，不要一次全插。第一阶段插轮毂和骨架，确认位置，第二阶段再插握持圈，以此类推。
4. 每插完一个阶段跑一次干涉检查。攒到最后一起查，等你发现问题的时候已经找不到是哪个零件造成的。
5. 录一段 360 度环绕，肉眼过一遍。这一步听着土，实际上挡住过一次某个装饰件装到背面去的错误，而那处错误干涉检查报的是零对，因为它确实没撞到任何东西，只是装反了。
6. 保存关闭再重开，回读所有变换。四十四处变换逐一比对，误差不超过 $1e-9$ 。
7. 做解压复开测试。把原始零件目录临时改名，让它找不到原文件，只从解压副本里打开总装。

总装五步与干涉检查

多零件怎么一起交给 AI：先列邻近关系，再分批插入

1

出接口包络表

邻近关系与间隙列一遍，单边间隙 0.5 毫米

2

统一坐标系

所有零件原点规矩一致，插入之后不会各自飞天

3

分五阶段插入

每插完一轮立刻跑一次干涉与间隙检查

4

保存再重开回读

44 处变换逐一比对，误差不超过 $1e-9$

5

改名做解压复开

原目录改名后打开，44 个组件全部解析

处于轻量化状态的零件必须退出轻量化再检查

图 10-8 总装五步：接口包络、分阶段插入、干涉检查、重开回读、解压复开

图 10-8 作者根据公开资料绘制。

可复制提示词

【任务】把已完成的 23 种零件组装为总成，产出总装文件与 BOM。

【接口】先给我一张接口包络表，列出所有零件之间的邻近关系与预留间隙，单边间隙一律 0.5 毫米。表里每个条目写清楚：甲零件、乙零件、接触类型、预留间隙、校验方法。表我先过一遍，我点头了你再动手。

【坐标】所有零件按同一套原点规矩定位。插入前把每个零件的设计原点先报一次给我。

【插入】不许一次性全部插入。分五个阶段：

第一轮：轮毂与三辐骨架。

第二轮：握持圈。

第三轮：换挡拨片与滚轮。

第四轮：指示灯阵列与人字徽标。

第五轮：装饰圆角件与卡槽。

每插入一轮，立即跑一次干涉检查与间隙检查，把结果贴给我，等我确认再进下一轮。

【检查规矩】处于轻量化状态的零件必须退出轻量化再检查。

干涉检查的结果里不许出现已忽略项。任何非零干涉立即报告对象编号与穿模体积，不许自行修改零件去消除干涉。

【收尾】全部插入完成后：

1. 保存关闭，重新打开，逐一回读 44 处实例的变换，与原值比对，报出最大误差。
2. 生成原生明细表，行数等于零件种类数 23。
3. 把整个目录打包，然后在另一个空目录解压，从解压副本打开总装，确认 44 个组件全部解析、干涉为零。

【纪律】每轮汇报带真实数字。失败时把原始报错原样贴给我，停下来等我指示，不许改动设计去消除报错。

最后一句是全场景的灵魂：不许改动设计去消除报错。这句话同样适用于你手下的工程师。

过程与判断点

这一趟的真实产出：

- 四十四处零件实例，五阶段插入完成。
- 全总成干涉检查零对，并且在保存关闭再重开之后复验，仍然零对。
- 四十四处变换逐一回读，与原值比对，误差不超过 $1e-9$ 。
- 解压复开测试：四十四处组件全部解析成功，干涉为零，体积差在 $1e-16$ 量级。
- 原生明细表二十三行，不是四十四行。这两者的区别是新手最容易错的地方：四颗同样的螺钉在明细表里是一行，数量列写 4。

为什么最后那步把原目录改名这么重要？因为这是唯一能证明这个包能独立打开、能发给同事、能进厂归档的测试。你要的是能发给别人用的东西，不是一台只有你自己电脑上才打得开的孤本。

翻车清单

翻车一：打包发给同事，对方打不开。症状是对方那台电脑上一堆文件未找到。根因是装配体存的是绝对路径，而你俩的盘符不同。补救：用相对路径加打包工具（SOLIDWORKS 里的 Pack and Go 就是干这个的），并且必须在另一台电脑上真的打开一次。

翻车二：同名零件互相覆盖。症状是明明插了两个不同的装饰件，最后只剩一个。根因是两种规格的零件存了同一个文件名。补救：文件名里带上规格，总装前让 AI 列一遍文件名清单，你自己扫一眼有没有重复。

翻车三：把正常间隙误报成故障。症状是干涉检查报告一片红。根因是检查阈值没设，软件把 0.5 毫米的设计间隙也算成了问题。补救：先设阈值，比如小于 0.1 毫米才算干涉，并把含间隙检查和干涉检查分成两份报告来看。

验收标准与变体

验收五条：

1. 干涉零对，报告中无忽略项，且这是在退出轻量化状态后得到的。
2. 保存重开后，四十四处变换与原值比对的最大误差不超过 $1e-9$ 。
3. 解压副本能独立打开，引用全部解析。
4. 明细表行数等于零件种类数，数量列合计等于实例总数四十四。
5. 录一段 360 度环绕自己看完，确认没有位置没错、朝向错了的零件。

变体一：给一台执行器做总装。电机、齿轮组、输出轴、壳体、端盖，装配关系清晰。这类产品的验收重点是同轴度，抽三个配合面测同轴即可。

变体二：产线工装的位置检查。工装上有几十个定位销、夹紧缸、传感器，用这套方法检查它们之间有没有撞。这一类和产品的差别在于干涉往往出现在运动过程中，所以要多给一组运动位置。

变体三：导出爆炸图与 BOM 表给采购。让 AI 一并输出一张 Excel 格式的 BOM，字段按你公司的模板。这是整章里跟钱最贴近的一步，一张错表会让采购订错东西。

老兵说：装配体的最后一次体检得换一台电脑做。你打不开别人的包，别人也就打不开你的；所以那个解压复开测试，请务必在一台干净的机器上完成。

场景 24 工业造型与渲染：Rhino / Blender 一侧的能力与边界 ★

一句话背景

前面五个场景围着严谨的机械设计转。但你迟早会遇到另一类活：让产品看着好看、让评审的人看懂、让汇报的材料里有张像样的图。这一侧的常客是 Rhino 和 Blender。

本节标 ★，内容来自公开资料与社区案例（相关开源项目主页与文档、软件官方文档、公开技术分享），我在这套环境上没有完整跑通过那些流程，涉及的具体菜单、参数、版本号一律以你安装的为准。

名词小词典

Rhino：以 NURBS 曲面著称的造型软件。曲面质量要求高的地方常见它的身影。它的 Grasshopper 插件可以玩参数化。

Blender：开源免费的三维创作软件，建模、雕刻、渲染、动画一条龙。

NURBS：一种用数学曲面精确描述形状的方式。高等级的外观曲面靠的就是它。

渲染：给模型打灯、上材质、出效果图的过程。这一步 AI 参与的门槛极低，因为好不好看没有标准答案。

Blender MCP：社区里的开源项目（许可证以项目主页标注为准），通过 MCP 把 Blender 的能力暴露给 AI 调用。

开工前准备

1. Blender 安装好，或 Rhino 安装好（商业软件，单位推之前先把授权这件事说清楚）。
2. Blender 这一侧要装对应的 MCP 插件，装完之后它的端口应当能被本机访问到。
3. Rhino 这一侧，参数化玩法通常要用 Grasshopper 或 Rhino.Inside，先在软件里手动连一遍电路，确认能跑。
4. WorkBuddy 里登记这台 MCP 服务器，然后用一句列出可用工具验握手。
5. 先看清许可证。这一步不酷，但它是能不能落地的分界线。

操作步骤

这一类工具的界面版本差异很大，我给的是通用原则，不是菜单路径：

1. 在目标软件里手动做完一次最小流程，比如 Blender 里新建一个立方体并导出 GLB。手动都做不成，AI 也做不成。
2. 登记 MCP 服务器，验握手，确认工具列表里有创建对象、设置材质、导出这几类。
3. 交给 AI 一个比你真正的需求小十倍的任务，看它的返回值长什么样。
4. 逐步加码，每加一级就加一条校核：看对象数、看顶点数、看导出文件大小。
5. 渲染这一步单独做，不要混在建模指令里。
6. 出图之后必须有人眼看一遍。渲染恰恰是你最容易被忽悠的一类产物，因为它好看。

Rhino 与 Blender 的能力边界

这一节内容来自公开资料与社区案例，非本人实测

它稳定能做到的	它的三条边界
批量建简单对象 重复五十次手会抽筋的那类活 批量改材质 一个产品五种配色、三个视角 批量导出交换格式 GLB、STEP 这类交换文件 脚本做重复操作 流程固定之后交给脚本跑 参数化驱动变体 Grasshopper 里连一组电池	曲面质量它自己看不见 曲率连续性与斑马线分析，靠人眼加专业工具 渲染会无限迭代 给它一个明确轮数上限，比如最多改三轮 曲面转网格必然有损失 公差必须你自己定，松了变成多边形

造型这活可以有 AI 的份，曲面质量的最后一票在人手

图 10-9 Rhino 与 Blender 两侧的能力与边界对比

图 10-9 作者根据公开资料绘制，内容依据公开软件文档与社区案例整理，非本人实测。

可复制提示词

Blender 这一侧的最小起步任务，挂载好 Blender MCP 后粘贴：

你现在可以通过 blender 这台 MCP 服务器操作我电脑上打开的 Blender。

【任务】在一个新场景里建一个圆角的方块：
尺寸 100 × 100 × 10 毫米，四条竖棱各倒 R5 圆角，材质给一个中灰的塑料感。
打一盏主灯和一盏补光，渲染一张 1920×1080 的图，存为 D:\AI-CAD\render\block.png。
再导出一份 D:\AI-CAD\render\block.glb。

【单位】Blender 内部场景单位与毫米不同，你负责换算，
所有汇报数字一律换算成毫米并标注单位。

【汇报】每一步做完都要回报：场景里的对象数、每个对象的顶点数、包围盒尺寸。

【纪律】渲染分辨率、采样数这类参数一律先问我，不许自行取一个默认值。
任何失败把原始报错原样贴给我，停下来等我指示。

Rhino 这一侧的最小起步任务，走 Grasshopper 参数化路线：

【任务】在 Grasshopper 里搭一组最简单的参数化电池：

输入三个滑块（长 L、宽 W、圆角半径 R），输出一个带圆角的矩形板并 bake 到 Rhino 里。

L 默认 100，W 默认 80，R 默认 5，单位毫米。

【要求】改任一个滑块的值，已经 bake 出去的对象要能跟着更新。

做不到没关系，做不到就把做不到和原因告诉我，不许假装做到了。

【汇报】把电池连接关系的清单贴给我：每个电池的名字、输入端口、输出端口。

【纪律】单位按毫米；曲面公差设为 0.001 毫米并把它作为参数暴露出来；

不许使用软件默认值却不告诉我那个默认值是多少。

过程与判断点

公开资料里这一类集成能稳定做到的几件事：批量建简单对象、批量改材质、批量导出各类交换格式、用 Python 脚本做重复操作、用参数化驱动成系列变体。也就是说，凡是重复五十次手会抽筋的活，它都能接。

边界也很清楚，我按自己的判断列三条：

第一，曲面质量它自己看不见。高等级外观曲面的评价手段是曲率连续性和斑马线分析，这些都是人眼加专业工具的事。它可以给你生成一个曲面，但没法替你回答这条高光线顺不顺。

第二，渲染会无限迭代。好看没有终点，你让 AI 改一次它就改一次，一次渲染几十秒到几分钟。这块的费用和时间是全章最不可控的。我的经验是给它一个明确的轮数上限，比如最多改三轮。

第三，曲面到网格的转换必然有损失。NURBS 转到 GLB、STL 这类网格，是拿三角面去逼近曲面，精度靠公差参数控制。公差设松了，曲面变多边形；设紧了，文件大到打不开。这个参数必须你自己定，不许让它随便取一个。

翻车清单

翻车一：内部单位是米，你以为是毫米。症状是建出来的东西大一千倍或者小一千倍。补救：动手前第一句话就锁定单位，并要求所有汇报值标注单位。

翻车二：修改器没有应用，导出的几何跟屏幕上看到的不一样。症状是导出到别的软件里，圆角没了、细分没了。补救：导出前把所有修改器应用一遍，导出后在那个软件里重新量一次尺寸。

翻车三：渲染追求完美，时间和费用就这么烧进去了。症状是一上午过去了第七版图还没出来。补救：给它一个硬性轮数上限，并且先渲小尺寸确认构图，再渲最终分辨率。

验收标准与变体

验收四条：

1. 按你的产品定一处已知尺寸，对着模型量一次基准边，误差在你定的公差内。外观件通常 0.1 毫米足够。
2. 导出的 GLB 或 STEP 拖回另一个软件重开，体量与本机一致。

3. 渲染图出来之后，让一个不看模型的人说出这东西的材质手感。说不出来，材质就没做对。

4. 同一份提示词跑两遍，两次结果的几何应完全一致。跑不一致说明它掺了带随机数的默认值。

变体一：外观评审用的渲染图。产品外观评审会上那几张图，是最适合被它批量产出的东西：一个产品的五种配色、三个视角。

变体二：装配爆炸动画。这类动画人工做要半天到一天，脚本化之后几十分钟。评审、培训、维保手册三处都用得上。

变体三：把 STEP 转成 GLB 放到网页上。数字样机展示、官网的三维预览，都是这个套路。这时候验收重点是文件大小和加载时间，不是几何精度。

老兵说：造型这活可以有 AI 的份，曲面质量的最后一票在人手里。它能给你一百张图，但哪一张能通过评审，判断的是你。

小白词典

数模：三维数学模型，也就是零件在软件里的立体样子。它不是一张图，是能算体积、能查干涉的几何实体。类比：照着照片没法开车床，照着数模可以。

工程图：把三维数模投影成二维图纸，标上尺寸、公差、剖视、明细表，专门给加工和检验用。类比：数模是给设计看的说明书，工程图是给车间下的订单。

STEP：三维数模的通用交换格式，工业界的普通话。你用 SOLIDWORKS 画的，别人拿 CATIA 也能打开。类比：不同品牌的插头插同一个插座。

MCP：一种让 AI 操控其他软件的通用插座标准。本章里它指的就是把 AI 的话翻译成 CAD 操作的那座桥。

COM：Windows 上软件之间互相调用的一套老接口。SOLIDWORKS 对外开了这道门，AI 才能隔着桥伸手进来。类比：软件之间的对讲机。

干涉：两个零件在空间里撞到一起了，装不到一块去。类比：抽屉塞多了推不进去。

BOM：物料清单，这东西由哪些零件、各多少个组成，一表列清。类比：做菜前的配料表，采购照它买。

扫掠：让一个截面沿着一条指定的路径走一遍，走过的空间就是你要的形状。类比：拿一块模子沿轨道刮出来的一条橡皮泥。

圆角：把尖锐的棱磨成圆弧，代号是 R，比如 R16 就是半径 16 毫米的圆角。类比：桌角的尖要在磕到手之前先磨圆。

装配体：若干零件按相对位置拼起来形成的文件，后缀 `.SLDASM`。类比：把散件按说明书拼好之后的整机。

参数化：把尺寸写成变量，改一个数字，整条链路自动跟着改。类比：表格里改一个单元格，所有公式结果一起变。

草图 / 拉伸 / 切除：草图是平面上画的轮廓，拉伸是把它拉出厚度变成实体，切除是从实体上挖掉一块。这三兄弟是机械建模的基本动作。

剖视：把零件假想切开，展示内部的样子，代号按 A、B、C 依次排列。类比：切开的半个橙子。

NURBS：一种用数学公式精确描述曲面的方式，外观等级越高的曲面越靠它。类比：矢量图放多大都不糊，位图一放大就出马赛克。

渲染：给模型打灯、上材质、算出照片级效果图的过程。类比：给毛坯房做软装再拍一组照片。

你的岗位怎么用

设计工程师：先抄场景 19，用一个你明天就要用的真实零件开局，别拿书上的例子练。摸熟之后进场景 20，把它的九步改造成你自己的作业清单。你最大的收益在改尺寸这件事上：过去改十处要重画半天，现在是改一个参数源重跑一遍。

工艺与制造工程师：你的着力点在场景 21 和场景 22。车间拿到的是图纸，所以图纸环节任何一处错都是直接成本。把场景 22 那张六行体检查验表打印出来贴在显示器边上，供应商的 STEP 先过一遍再进门。

质量工程师：场景 22 和场景 23 是给你的。AI 产出的证据链（体积校核、干涉报告、STEP 往返、重开回读）本质上就是一份可追溯的检验记录。你要做的把关动作，是把这些数字跟你自己的量具测量值对一次。

项目经理：盯场景 20 那个冻结评分表的动作。它不是 CAD 技巧，是项目管理技巧：标准先定死并锁哈希，后面就不扯皮了。把这一条用在 AI 之外的所有外包任务上同样有效。

采购与供应商管理：场景 23 的 BOM 与解压复开测试跟你直接相关。对外协厂提交交付要求时，明确写要求提供 STEP 加 PDF 加原生文件，且能在另一台电脑上打开。这句话能替公司省掉大量返工邮件。

试验工程师：场景 22 的体检查验单可以原样搬到试验工装上。夹具与试件之间的间隙上限、干涉零对，这些是台架安全的前提，跑试验之前先验一遍。

本章交付物

一份能带走的 AI 建模九步作业单，外加一张六行体检查验卡。

做法：把场景 20 那张九步表格抄进你自己的笔记软件，把每一列的后两项换成你公司的真实规矩，包括模板名、验收门槛、责任人。再把场景 22 的六行体检查验表贴在显示器边上。

第一件验收的活：挑一个你手上最简单、最不重要的零件，走完九步，产出一颗零件加一份体检报告。跑通的那天，你就有资格把下一个真正的活交给它。

本章金句

AI 画出来的不是一张图，是它在你的建模软件里按下的每一次鼠标。

屏幕上的显示值不算数，接口回读值才算数。

不许叫它想清楚再动手，要叫它先动手、边做边想。

错一处尺寸，铣床就会多走一刀，为这一刀买单的是签字的人。

它没有灵感，但它能在两小时内连接三百次鼠标并且手不抖。

会建漂亮三维的人不少，能交出合规二维图纸的才稀缺。

（下一章：电子、软件与仿真。方向盘还没捂热，我们把同一套九步打法搬到电路板和仿真求解器上，看看那儿的味道跟这儿差在哪。）

第 11 章

电子、软件与仿真：AI 进得了 PCB、控制器和求解器吗

在电路板和控制器代码上，AI 最危险的地方不是它做错，是它做对了九十九处、错在第一百处，而错的那一处长得很像对的那一处。

去年三月的一个周四下午，办公室只剩我一个人。屏幕上开着一份电机控制器的底层驱动代码，一千八百多行 C，用的是我们项目上跑了三年的那套风格。

我想让它帮我看一遍。

手指停在回车键上有十几秒。那一刻我心里转的不是技术问题，是一个很具体的画面：这批控制器装在车上，车在高速上跑，某个中断服务程序里的一行判断被改坏，电机在错误的时间点换了一次相。做汽车的人对软件错误是零容忍的，这个零容忍不是口号，是召回单上的数字，是服务站里排队的车。

我最后还是敲了下去，但我改了说法。我没有问“这段代码有没有问题”，我问的是：

下面这段是我司电机控制器的 PWM 占空比映射函数。请你只做一件事：逐行列出你“不确定”的地方。不要修改任何代码，不要给优化建议，不要评价代码风格。对每一处不确定，写清楚三件事：行号、你不确定的是什么、你需要看到什么资料才能确定。如果你对某一段完全没有把握，就写“我没有把握”，不许用“可能存在风险”这类模糊说法搪塞。

它回了一份清单，十一条。我一条一条看下来，有六条是我自己也同意的疑点，三条是我知道没问题但它看不出来，两条是它把语言标准和公司规范搞混了。

真正让我放下心的不是那六条。是它愿意写“我没有把握”这四个字。一个肯说自己没把握的东西，你才知道它说“这里有问题”的时候，那句话是有分量的。

后来我把这套问法固定下来，用在电路板、用在仿真前处理、用在结果解读上。这一章的六个场景，就是从那次开始的。

先交代一句实话：这一章里，我没有第 10 章那种“我亲手跑完一整台方向盘”的案例。电子和仿真这一侧，要么软件授权贵得我没法在家里装，要么工具链还处在社区活跃、版本乱飞的阶段。所以这一章三个场景标 ★（调研，来自公开资料与社区案例），三个场景标 ★★（跑通，我在自己的环境上跑过流程，但没有完整留档）。没有一处标 ★★★。我宁可让你看清楚哪些是我跑过的、哪些是我读来的，也不愿意让你照着一段我没碰过的流程去签字。

下面是本章六个场景的速览，你先挑跟你有关的那个看。

场景	场景名	人工耗时	AI 耗时	可信度
25	PCB 设计辅助：让 AI 驱动 EDA 软件干活	约 1 人天	未实测（公开案例为数十分钟量级）	★ 调研
26	PCB 检查与 BOM 核对：网表与物料清单的一致性	约 4 小时	约 25 分钟	★★ 跑通
27	控制器软件开发：模型与代码的生成辅助	约 2 人天	未实测	★ 调研
28	代码审查与规范检查：AI 当第一道关	约 3 小时	约 20 分钟	★★ 跑通
29	CAE 仿真前处理：让 AI 准备求解器输入	约 1 人天	未实测	★ 调研
30	仿真结果解读与报告：从结果文件到结论	约 4 小时	约 30 分钟	★★ 跑通

场景 25 PCB 设计辅助：让 AI 驱动 EDA 软件干活 ★

一句话背景

一块四层控制板，从原理图导网表、建板框、摆关键器件、定规则到出第一版布局，熟手一整天。烦的地方不在难，在于这活有三分之二的时间在重复：改一个封装要重摆、改一个网络要重连、规则表改一遍要重刷一遍。这一节讲怎么让 AI 替你去按那些鼠标。

名词小词典

PCB：印刷电路板，那块绿色的板子。铜箔刻成线路，元器件焊在上面。类比：城市里的道路网加沿街的房子。

EDA：电子设计自动化软件的总称，画原理图、画板子、查规则都在这类软件里做。本章里点名的两款是商业软件 Altium Designer 和开源软件 KiCad。

网表 (Netlist)：一份纯文本文件，写着谁跟谁连在一起。原理图导出的网表和 PCB 导出的网表本该完全一致，不一致就是出事了。类比：一张通讯录，左边是人名，右边是电话号码。

封装 (Footprint)：一个元器件在板子上占的那块焊盘图形。同一个电阻可能有十种封装，选错就焊不上。

走线 (Routing)：把网表里的连接关系，变成板上实实在在的铜箔路径。

DRC：设计规则检查。线宽够不够、间距够不够、孔打得下打不下，软件按你定的规则查一遍，报违规。

MCP：一套让 AI 调用其他软件能力的通用接口标准。AI 说话，桥来翻译，软件动手。第 10 章那条 SOLIDWORKS 的链路走的也是它。

开工前准备

1. EDA 软件装好并授权到位。Altium Designer 是商业授权，KiCad 免费开源，两条路的准备完全不同，别混着看。
2. 确认软件版本。这一侧的版本差异极大，脚本接口可能隔一个大版本就换名字。这一条比装软件本身还重要。
3. 一台桥。社区里的开源项目 eda-agent（出处：开源代码托管平台上的公开项目，许可证以项目主页标注为准）走的是"文件桥"路线：AI 写一段脚本文件，软件执行，再把输出写回另一个文件，AI 读回来。公开资料里它面向 Altium 暴露的工具在四百个量级。KiCad 这一侧走官方 Python 接口（文档里通常写作 kipy），是原生接口，不需要绕文件。
4. 一个干净的工作目录，路径全英文、无空格。这一条在第 10 章已经坑过我一次。
5. 先看清许可证和你公司的合规要求。把公司项目文件发给任何外部服务之前，这一步不可跳过。

操作步骤

这一节标 ★，下面给的是通用原则和通用提示词形态，不承诺某个版本的菜单路径：

1. 先在软件里手动做完一次最小流程：新建工程、放一个电阻、连一根线、保存。手动都做不成，AI 也做不成。
2. 把桥起起来，在 AI 侧登记成 MCP 服务器，用一句"列出这台服务器上所有可用工具"验握手。工具列表里有创建元件、放置、连线、导出网表这几类，才往下走。
3. 交给它一个比你真正需求小十倍的任务。我要的是一个 50 乘 50 毫米的板框加三个器件，不是整块控制板。
4. 看返回值。它每做完一步必须报出真实数字：器件位号、坐标、网络名。
5. 逐步加码。每加一级加一条校核：器件数、网络数、DRC 违规数。
6. 让它导出网表，你自己打开那个文本文件看一眼。文件是最不会撒谎的产物。
7. 全程不许让它直接改你唯一的工程文件。副本先行，这一条没有例外。

AI 到 EDA 软件的两条路

一条把话写成脚本文件过桥，一条走原生 Python 接口

Altium：脚本文件桥	KiCad：官方 Python 接口
桥的样子 AI 写一段脚本文件，软件执行，AI 再读回输出文件 软件 Altium Designer，商业授权 工具规模 公开资料里面向 Altium 暴露的工具在四百个量级 要防的坑 脚本桥静默失败，要求回报输出文件时间戳	桥的样子 KiCad 官方 Python 接口 pcbnew / kipy，原生不用绕文件 软件 KiCad，免费开源 推荐先做的一步 只读体检：板框尺寸、器件清单、网络数、DRC 违规数 两边共同的纪律 副本先行，任何失败把原始报错原样贴回来

这一节为调研级内容，菜单路径与版本号以你安装的为准

图 11-1 从 AI 到 EDA 软件的两条路：脚本文件桥与原生 Python 接口

图 11-1 作者根据公开资料绘制，仅表示原理关系，不含任何第三方设计文件内容。

可复制提示词

Altium 一侧，走脚本文件桥的路线。整段复制，尺寸和路径换成你自己的：

你现在可以通过 eda 这台 MCP 服务器，驱动我电脑上已经打开的 Altium Designer。
它走的是脚本文件桥：你写脚本文件，软件执行，你再读回输出文件。

【任务】在一个新建的 PCB 工程里做一件最小的事，先别做大：

1. 建一个 50 × 50 毫米的板框，原点在左下角。
2. 放置三个器件：R1 (0805 封装)、C1 (0805 封装)、U1 (SOT-23 封装)。
坐标分别是 (10, 10)、(20, 10)、(30, 25)，单位毫米。
3. 把 R1 的第 2 脚与 C1 的第 1 脚连成网络 NET_MID。

【单位】我说的全部是毫米。接口内部单位与毫米不同，你负责换算；
回读给我看的每一个数字一律换算回毫米并标注单位。

【汇报】每完成一步立刻回报：器件位号、实际坐标、实际封装名、
网络名与网络连接数。不许用“已完成”概括。

【纪律】

1. 任何一步失败，把软件返回的原始报错原样贴给我，然后停下来等我指示。
不许自己猜原因，不许带着报错继续往下做。
2. 封装名如果对不上库里的实际名字，停下报告，不许就近选一个相似的。
3. 全程只操作 D:\AI-EDA\demo01 这个副本目录，不许碰任何原始工程文件。
4. 做完后导出网表到 D:\AI-EDA\demo01\netlist.txt，
并把文件的前 20 行原样贴给我看。

KiCad 一侧，走官方 Python 接口 (kipy) 的路线：

你现在可以通过 kicad 这台 MCP 服务器，用 KiCad 的 Python 接口 pcbnew / kipy
操作我电脑上打开的 KiCad。

【任务】打开 D:\AI-EDA\kicad_demo\demo.kicad_pcb，做一次只读体检，先不要改任何东西：

1. 报出板框的外形尺寸（宽 × 高，毫米）。
2. 列出板上所有器件：位号、封装名、坐标（毫米，保留三位小数）。
3. 统计网络总数，列出网络名清单。
4. 跑一次 DRC，报出违规数与每一条违规的类型、坐标、涉及的网络或器件。

【单位】KiCad 内部单位与毫米不同，你负责换算，回报一律换算回毫米并标注单位。

【汇报】不许用“没有问题”“一切正常”这类话。

每一项都报数字；没有违规就写“违规数 0，DRC 已执行，检查时间 X”，
不许只写“通过”。

【纪律】只读，不许保存。任何失败把原始报错原样贴给我，停下来等我指示。

第二条提示词我要多说一句。只读体检是我在这一侧最推荐的第一件事，因为它不产生任何破坏，而它给你的那个器件清单和网络清单，正是后面所有工作的底账。

过程与判断点

一份合格的回报长这样，四行里有三行能手算复核：

```
> [板框] 50.000 × 50.000 mm, 原点 (0.000, 0.000)。
> [器件] R1 / R_0805 / (10.000, 10.000), C1 / C_0805 / (20.000, 10.000),
>      U1 / SOT-23 / (30.000, 25.000), 共 3 个, 封装名与库内一致。
> [网络] 新增 NET_MID, 连接 2 个焊盘 (R1-2、C1-1), 网络总数 1。
> [DRC] 违规 0 条, 已执行, 检查时间 0.4 秒。
```

我盯三个地方。

第一，封装名。它写的封装名必须能在你的库里原样搜到。它写 R_0805，你的库里叫 R0805，那这一步就是废的。封装名是最容易"就近选一个相似的"的地方。

第二，坐标。三个器件的坐标要跟你说的一模一样，不是接近。坐标错通常意味着单位换算链路上有问题，而这个问题会一路带到后面所有步骤。

第三，网表文件。让它在最后把导出的网表前二十行原样贴给你。这不是形式主义。网表是纯文本，你能看懂，你能数出里面有几个网络、几个器件。屏幕上的图形会骗人，文本文件不会。

翻车清单

翻车一：封装名幻觉。症状是它自信地报出一个封装名，你的库里根本没有。根因是它按常见命名习惯猜了一个。补救：在提示词里写死"封装名必须与库内实际名字完全一致，对不上就停下报告"，并且开工前先让它把库里的封装名清单列一遍给你。

翻车二：脚本桥静默失败。症状是它报"已执行"，但板子上什么都没变。根因是走文件桥这条路时，脚本文件写了、软件没执行、或者输出文件是上一轮的旧内容。补救：要求每次执行后回报输出文件的时间戳和字节数，时间戳不刷新就说明软件根本没跑。

翻车三：它直接改了你的原始工程。症状是你打开原来的工程，发现多了三个器件。根因是工作目录没隔离。补救：副本先行，并且在提示词里把"只操作副本目录"写成一条硬性纪律，结尾再让它列一遍它实际打开的文件路径。

验收标准与变体

验收四条，五分钟做完：

1. 打开导出的网表文本，器件数、网络数与你要求的一致。
2. 三个器件的坐标逐一对得上，误差为零，不是接近。
3. 每个器件的封装名都能在库里原样搜到，不是"看起来像"。
4. 关掉软件重开一次，确认改动还在。改完就没保存，等于没改。

变体一：批量改封装。一块板上三十个电阻要从 0805 换成 0603。让它列出待改清单、你确认后再批量执行，改完导出网表对一遍器件数。

变体二：出一份器件坐标文件给贴片厂。让 AI 从板子上读出全部器件的位号、坐标、旋转角度，写成 CSV。这份文件直接决定贴片机怎么走，验收动作是随机抽五个器件用软件量一遍坐标。

变体三：规则表体检。把你的线宽间距规则表交给它，让它对着板子查一遍有没有违反规则却没被 DRC 抓到的地方。这一条属于加分项，别当主菜。

老兵说：在 PCB 这一侧，AI 最值钱的用法不是帮你画板，是帮你数数。器件数、网络数、封装名，这三个数你数清楚了，板子就已经对了一半。

场景 26 PCB 检查与 BOM 核对：网表与物料清单的一致性 ★★

一句话背景

一块板子打样回来才发现一个电容没焊上去，根因可能是三个月前的一次工程变更改了原理图、忘了改 BOM。这类漂移人工查一遍要三四个小时，因为你要在两张几百行的表之间来回找。这一节给你一条二十多分钟的查法。

名词小词典

BOM：物料清单。这块板子由哪些物料、各几个、用什么编码，一表列清。采购照它买，仓库照它发，贴片厂照它上料。

ECO：工程变更指令。改一个电阻值、换一个供应商、加一个电容，都算。它是工程现场最频繁的动作，也是最频繁的事故源。

ECO drift：变更漂移。原理图改了，PCB 没跟着改；或者 PCB 改了，BOM 没跟着改。三份文件的版本就这么悄悄错开了。

位号 (RefDes)：板上每个器件的身份证，R1、C3、U7 这种。它是三份文件之间唯一的对照键。

多 Part 器件：一个物理芯片里装着四个一样的运放，位号写作 U2A、U2B、U2C、U2D。BOM 里它是一行、数量 1，网表里它是四个位号。这个错位是新手查不出来的头号坑。

交叉引用 (Cross-reference)：拿两份清单按同一个键对一遍，列出只在左边有的、只在右边有的、两边都有但值不一样的。

开工前准备

1. 三份文件：原理图导出的网表、PCB 导出的网表、最新版 BOM。三份都要能导出成文本或表格，PDF 不行，PDF 没法比。
2. 一个干净的工作目录，三份文件放进去，文件名带上日期版本号。
3. 一份你公司的物料编码规则说明。没有的话，这一步你会卡在"这两个编码是不是同一个东西"上。
4. 决定好对照键。绝大多数情况用位号，特殊情况下用"位号加值"的组合。

操作步骤

这套流程我在自己的环境上跑通过，六步：

- 1. 先把三份文件各自的行数和器件数统计出来。数字对不上就是信号，先把数字记下来再往下查。
- 2. 让 AI 拿位号做键，把原理图网表和 PCB 网表对一遍，输出三张差异表：只在原理图、只在 PCB、两边都有但网络名不同。
- 3. 把 PCB 网表和 BOM 对一遍，同样三张表：板上有但 BOM 没有、BOM 有但板上没有、位号相同但物料编码不同。
- 4. 单独处理多 Part 器件。让 AI 把带后缀字母的位号归并回主位号，再重新对一次。
- 5. 让它把每一条差异写成一行结论：差异类型、位号、原理图侧的值、PCB 侧的值、BOM 侧的值、建议动作。
- 6. 你自己抽五条去原始文件里核对。抽五条是底线，不是抽查十条的十分之一。

三份文件的交叉引用

拿位号做键，原理图网表、PCB 网表与 BOM 两两比对

比对	差异表	含义	建议动作
原理图对 PCB	A 只在原理图出现	板子上缺了这个件	改 PCB
原理图对 PCB	B 只在 PCB 出现	原理图那边漏改	改原理图
原理图对 PCB	C 网络名不一致	连错线，DRC 查不出来	需人工确认
PCB 对 BOM	D 板上有 BOM 无	BOM 漏了这一行	改 BOM
PCB 对 BOM	E BOM 有板上无	板上没有这个件	需人工确认
PCB 对 BOM	F 物料编码不同	可能是替代料或旧版本	需人工确认

先报六个底账数字：两份网表各自的器件数与网络数，BOM 的行数与数量合计

图 11-2 三份文件的交叉引用：原理图网表、PCB 网表与 BOM 的两两比对

图 11-2 作者根据公开资料绘制。

可复制提示词

整段复制，文件名和目录换成你自己的：

【任务】对我放在 D:\AI-EDA\xcheck 目录里的三份文件做一致性检查，只出报告，不许修改任何原始文件。

【文件】

- sch_netlist.txt: 从原理图导出的网表
- pcb_netlist.txt: 从 PCB 导出的网表
- bom.xlsx: 最新版物料清单

【第一步：底账】先统计三份文件各自的规模并报给我：

原理图网表的器件数、网络数；PCB 网表的器件数、网络数；BOM 的行数与数量列合计。六个数字先列出来，不许直接跳到比对。

【第二步：原理图对 PCB】以位号为键做交叉引用，输出三张表：

- A. 只在原理图里出现的位号（PCB 上缺件）
- B. 只在 PCB 里出现的位号（原理图漏改）
- C. 两边都有，但所属网络名不一致的位号（连错线）

每张表如果为空，写“空表，已执行，比对行数 N”，不许只写“无差异”。

【第三步：PCB 对 BOM】以位号为键做交叉引用，输出三张表：

- D. 板上有、BOM 里没有的位号
- E. BOM 里有、板上没有的位号
- F. 位号相同但物料编码不同的位号

注意处理多 Part 器件：U2A / U2B 这类带后缀的位号要归并回 U2 再比对，归并规则先写清楚给我看，我确认了再执行。

【第四步：差异结论】把上面六张表合并成一张结论清单，每行五列：

差异类型（A-F）、位号、原理图侧的值、PCB 侧的值、BOM 侧的值、建议动作。建议动作只能填这四种之一：改原理图、改 PCB、改 BOM、需人工确认。

【输出】结论清单写成 D:\AI-EDA\xcheck\xcheck_report.md，开头必须写本次比对的六个底账数字。

【纪律】

1. 不许用“基本一致”“差异不大”这类话。差异就是差异，一条一条列。
2. 任何一处你读不懂、解析不了的字段，原样贴给我并写“此处我无法解析”，不许跳过，也不许按常见格式猜。
3. 比对过程中发现三份文件里有任何一份看起来不是最新版，立刻停下告诉我，不许继续往下比。

过程与判断点

一份合格的报告开头长这样：

- > [底账] 原理图网表：器件 312，网络 418。
- > PCB 网表：器件 312，网络 417。
- > BOM：218 行，数量列合计 312。
- > [A] 只在原理图：空表，已执行，比对行数 312。
- > [B] 只在 PCB：空表，已执行，比对行数 312。
- > [C] 网络名不一致：3 条。
- > R47：原理图侧 NET_FB1，PCB 侧 NET_FB2
- > R48：原理图侧 NET_FB2，PCB 侧 NET_FB1
- > C91：原理图侧 GND，PCB 侧 AGND
- > [D] 板上有 BOM 无：2 条 (R103、R104)。
- > [E] BOM 有板上无：0 条。
- > [F] 物料编码不同：1 条 (U7：BOM 侧 XXX-1234，板上器件值侧 XXX-1234-B)。

这份报告里我看三处。

第一，六个底账数字必须先出来。原理图 312 个器件、PCB 312 个器件、BOM 数量列合计 312，这三个数对上了，说明总量没跑。网络数 418 对 417，差一个，那一个就是 C 表里要查的东西。数字先摆出来，后面的差异才有参照系。

第二，空表也要写"已执行，比对行数 N"。这是我从第 10 章带过来的规矩。一张空表，可能是真的没有差异，也可能是它压根没比。写了比对行数，你才知道它真的拿 312 行去对过 312 行。

第三，C 表里的 R47 和 R48 是一类典型信号。两个电阻的网络名互换了，说明某次改动把反馈网络的两条支路接反了。这种错误 DRC 查不出来，因为线都连着、间距都够，它只是连错了地方。只有交叉引用能抓到它。

第四，F 表里 U7 那条是另一种信号。编码差一个后缀 B，可能是替代料，可能是 BOM 版本旧。这种条目不属于"错误"，属于"需人工确认"，你必须自己去看那个后缀是什么意思。

第五，它有没有发现我拿错了文件。我在提示词里写了"发现疑似旧版立刻停下"，这一条是花钱买来的。有一回我手里那份 BOM 是上一版的，它闷头查出来三十多条差异，我越看越不对劲，回头翻文件名，日期是两个月前的。它要是当时不吭声，我那一下午就白查了。

翻车清单

翻车一：多 Part 器件把数量算重了。症状是 BOM 里明明 218 行，它报出 300 多行差异。根因是 U2A、U2B、U2C、U2D 被当成四个独立器件。补救：在提示词里单独写一条归并规则，并要求它先报归并规则给你确认，再执行比对。

翻车二：位号大小写和前后空格。症状是同一个器件被报成"两边都有但不一致"。根因是一侧写 r12，另一侧写 R12，或者某侧带了个空格。补救：比对前先做一次规范化，去掉首尾空格、统一大写，并把规范化前后各报一次器件数，数字变了就说明有问题。

翻车三：拿到的是旧版文件还在查。症状是查出来一堆差异，最后发现 BOM 是上个月的。根因是文件名没带版本号。补救：文件名一律带日期版本号，并且在提示词里要求它发现疑似旧版就立刻停下。查三小时查出个旧文件，是我见过最亏的一种翻车。

验收标准与变体

验收四条：

1. 报告开头的六个底账数字，与你自己在软件里看到的数字一致。
2. 六张差异表全部存在，空表也写了"已执行，比对行数 N"。
3. 随机抽五条差异，回原始文件里人工核一遍，五条全对得上。
4. 多 Part 器件的归并规则你已经看过并确认过，不是它自己定的。

变体一：改模前后对比。一次 ECO 之前存一份报告，改完再存一份，两份并排看。差异出现在哪一行一目了然，评审会上不用争。

变体二：供应商来料的 BOM 核对。外协厂发回来的 BOM 跟你的原版对一遍，重点看 F 类（编码不同），那往往是替代料混进来的地方。这一步赶在贴片之前做。

变体三：把这套检查存成技能。把这段提示词加六个底账数字的输出格式，打包成一个 WorkBuddy 技能。以后每次出版本跑一次，二十分钟，报告自动落到同一个目录。

老兵说：PCB 上最贵的错误不是连错线，是三份文件的版本号悄悄错开了。线错了板子不工作，你当天就知道；版本错开了，板子照样工作，你三个月后才知道。

场景 27 控制器软件开发：模型与代码的生成辅助 ★

一句话背景

一个电机控制器的电流环，从搭 Simulink 模型到生成能编译的 C 代码，熟手两天。中间最耗时的不是写算法，是把接口、标定量、数据类型、代码生成配置一遍遍调对。这一节讲 MATLAB 与 Simulink 这一侧官方给出的 AI 能力能走到哪一步，以及它走不到的地方在哪。

名词小词典

Simulink：MATLAB 里的图形化建模工具。控制算法不写成代码，画成框图：一个增益块、一个积分块、一根连线。

代码生成：把画好的框图自动变成 C 代码。做汽车控制器几乎都走这条路，因为手写一千行 C 和生成一千行 C 的验证代价完全不是一回事。

AUTOSAR：汽车开放系统架构，汽车电子软件的行业标准接口规范。分 Classic 和 Adaptive 两支，前者跑在资源紧的单片机上，后者跑在算力强的域控制器上。

ARXML：AUTOSAR 的配置文件格式，本质是 XML。接口、端口、数据类型都写在里面，它是软件和软件之间对账的凭据。

MIL / SIL：模型在环与软件在环。MIL 是拿模型跑仿真，SIL 是把生成出来的 C 代码拿回来在电脑上跑。两者结果应当一致，不一致说明代码生成这一跳出了问题。

ASIL：汽车安全完整性等级，从 A 到 D，D 最高。它是 ISO 26262 里给功能定的风险等级，等级越高，验证要求越严。

Polyspace：一款做代码静态分析的工具，用形式化方法证明代码里没有某些类别的运行时错误。

开工前准备

1. MATLAB 与 Simulink 安装到位，版本确认。这一侧的 AI 能力是随版本加进来的，版本不对，功能就没有。
2. 相关工具箱的许可证到位。代码生成要对应的 Coder 工具箱，AUTOSAR 要对应的支持包。
3. 一个 MCP 服务器。公开资料里 MathWorks 官方在代码托管平台上放了一个名为 MATLAB MCP Core Server 的开源项目（许可证以项目主页标注为准），还有一个面向 AI 工作流的 Agentic Toolkit 项目。出处与许可证以项目主页标注为准。
4. 一个 Simulink 侧的能力入口。公开资料里提到 Simulink 产品线上有官方的对话式助手能力，Polyspace 也有对应的助手。具体名称、随附版本与可用范围，以 MathWorks 官方文档和你买到的授权为准。
5. 一个待改的模型副本。老规矩，副本先行。
6. 一份基线仿真结果。没有基线，你没法判断它改完之后是变好了还是变坏了。

操作步骤

这一节标 ★，内容来自公开资料，下面的顺序是通用工程顺序：

1. 先在软件里手动跑一遍最小链路：打开模型、仿真、看曲线、生成代码、编译。手动通不过，后面一切免谈。
2. 把 MCP 服务器起起来，在 AI 侧登记，用一句"列出这台服务器上所有可用工具"验握手。
3. 先让它只读。让它报出模型的顶层结构：有几个子系统、每个子系统的输入输出端口、采样时间、数据类型。它报得对，你才敢让它改。
4. 让它做一处最小改动：改一个增益、加一个限幅块。改完立刻仿真，与基线对比曲线的最大差值。
5. 再让它做结构性改动：加一个子系统、接上端口。这一步要盯住数据类型和采样时间有没有被它悄悄改掉。
6. 生成代码，做 SIL 对比。模型输出和代码输出的差值必须在一个你事先定死的门槛内。
7. 全程保存每一次的模型版本和仿真结果文件，别只留在聊天记录里。

控制器开发的四道关口

改模型、跑 MIL、生成代码、跑 SIL，顺序不许颠倒

- **第一关：改模型**
先只读体检报出结构与端口，再做一处最小改动
- **第二关：跑 MIL**
改动前后各跑一次仿真，同激励同求解器设置
- ! **第三关：生成代码**
数据类型显式声明不许继承，采样受影响就停下
- **第四关：跑 SIL**
模型输出与代码输出的差值，在事先定死的门槛内

模型画错了仿真会告诉你，代码生成错了仿真不会告诉你

图 11-3 控制器开发的四道关口：改模型、跑 MIL、生成代码、跑 SIL

图 11-3 作者根据公开资料绘制，内容依据公开软件文档与社区案例整理，非本人实测。

可复制提示词

第一段，只读体检。这一段在哪个版本上都能用，我建议所有人都从这一步开始：

【任务】用 MATLAB MCP 服务器对我指定的 Simulink 模型做只读体检，先不要改任何东西。

【模型】D:\AI-MBD\motor_ctrl\current_loop.slx

【第一步：结构】报出模型的顶层结构：

子系统清单（每个子系统的名字、层级路径）、
顶层输入输出端口的名字与数据类型、
每个子系统的采样时间（以及有没有多速率）、
求解器类型与固定步长。

【第二步：信号流】从顶层输入到顶层输出，把主信号路径上的模块按顺序列出来，
每个模块写：模块类型、模块名、关键参数值。
参数值不确定的写“未读出”，不许填常见默认值。

【第三步：标定量】列出模型里所有来自工作区或数据字典的变量：
变量名、数值、数据类型、存储类。任何一个读不出来的变量单独列一行。

【第四步：风险】列出你认为需要人工确认的地方，每条写清楚：
在哪里、你不确定什么、需要什么资料才能确定。

【纪律】

1. 只读，不许保存，不许改任何参数。
2. 不许用“结构清晰”“设计合理”这类评价。只报事实和数字。
3. 每完成一步立刻回报。失败时把软件的原始报错原样贴给我，停下来等我指示。

第二段，一处最小改动加验证。这一段的价值在于它把验证写进了指令，而不是留给事后：

【任务】在上面那个模型里做一处最小改动，然后自己验证。

【改动】在电流环的 PI 输出之后，插入一个饱和限幅块，上限 0.95，下限 -0.95，数据类型与上游信号保持一致。

【验证一：改前基线】改动之前先跑一次仿真，保存输出曲线为 baseline.mat。

【验证二：改后再跑】改动之后跑一次仿真，保存输出曲线为 modified.mat。两次仿真必须用同一组输入激励、同一套求解器设置、同一个停止时间。

【验证三：对比】把两条曲线逐点相减，报出最大绝对差、出现最大差的时刻、以及差值超过 $1e-6$ 的采样点数占比。

【验证四：结构确认】改动之后重新报一次模块清单，确认只多了一个限幅块，其余模块的顺序、参数、名字都没有变。

【验证五：代码生成】生成 C 代码到 D:\AI-MBD\motor_ctrl\code_v2，报出生成的文件数与总行数，并说明编译是否通过（用你可用的编译方式）。

【纪律】

1. 不许为了缩小差值而修改限幅值或改求解器设置。过不去就写未通过并分析原因。
2. 数据类型必须显式声明，不许沿用继承。报出限幅块输出的实际数据类型。
3. 采样时间如果被你的改动影响了，立刻停下报告，不许继续。
4. 只在副本上操作，原模型不许动。

过程与判断点

合格的回报里，我盯四个数字。

第一，模块清单的差集。改前 N 个模块，改后 N 加 1 个。如果改后报出来的模块数是 N 加 3，那它顺手加了别的东西，你必须问清楚那两个是什么。

第二，两次仿真的最大差值。加了限幅块之后，被限住的那一段曲线必然不同，未被限住的部分应当完全一样。所以差值报告里那个“超过 $1e-6$ 的采样点占比”是有意义的：占比应当等于被限幅的时间占比，而不是百分之百。

第三，限幅块输出的数据类型。它如果报出继承，你必须让它改成显式声明。继承类型在不同配置下可能解析成不同的实际类型，而这个差异往往到实车标定才暴露。

第四，生成代码的文件数与行数。上一次是 12 个文件 2400 行，这一次是 12 个文件 2380 行，你要知道少的那 20 行是什么。

翻车清单

翻车一：版本不对，功能根本没有。症状是它说没有这个工具、接口不存在。根因是你看的资料是新版，你装的是旧版。补救：动手前先让 AI 报出你本机 MATLAB 的版本号和已安装工具箱清单，对照官方文档确认功能在这个版本上有没有。

翻车二：数据类型被悄悄改了。 症状是仿真结果看着正常，生成的代码编译也过，实测值却偏了。根因是某个模块的数据类型从单精度变成了双精度，或者反过来。补救：在提示词里要求所有数据类型显式声明，并在改动前后各导出一次数据类型清单做差集。

翻车三：把生成的代码当成不用验证的代码。 症状是模型仿真过了，代码生成完了，就直接进集成。根因是把“代码是工具生成的”理解成了“代码是对的”。补救：SIL 这一关不许跳过。模型到代码这一跳是真实存在的一跳，跳过它等于放弃唯一一次能便宜地抓到它的机会。

验收标准与变体

验收五条：

1. 改动前后的模块清单差集恰好等于你要求的那一个模块。
2. 两次仿真使用同一组激励与求解器设置，差值报告里有具体数字。
3. 所有新增模块的数据类型显式声明，不出现“继承”。
4. 采样时间未被改动影响，多速率模型的速率关系保持不变。
5. SIL 对比通过，模型输出与代码输出的差值在你事先定死的门槛内。

变体一：AUTOSAR 接口对账。 让 AI 读你的 ARXML 和模型端口，两边对一遍：端口名、数据类型、接口方向。这一类的差异往往出现在改名之后。

变体二：标定量清单整理。 让 AI 把模型里所有标定相关的量导成一张表：变量名、默认值、取值范围、单位、所属子系统。这张表给标定工程师，能省掉大量来回问的时间。

变体三：生成代码的结构走查。 让 AI 读生成出来的 C 代码，报出函数清单、每个函数的行数、全局变量清单。你不一定看得懂每一个函数，但函数名的可读性、有没有生成出你没预料到的函数，这两件事你能判断。

老兵说：模型画错了仿真会告诉你，代码生成错了仿真不会告诉你。所以模型这一侧的 AI 你可以放手让它干，代码那一侧你必须留一道 SIL 在中间。

场景 28 代码审查与规范检查：AI 当第一道关 ★★

一句话背景

一千八百行代码交上去之前自己先过一遍，看规范、看边界、看异常处理，熟手三小时。AI 做这道关只要二十分钟，但它交出来的东西是“线索”，不是“结论”。这一节讲怎么让它的线索真的有用，以及它的结论为什么不能替你签字。

名词小词典

MISRA C：汽车和嵌入式行业广泛采用的 C 语言编码规范，一条一条编号，写明什么写法不允许、什么写法必须带说明。

静态分析：不运行程序，只读代码文本和语法树来找问题的技术。编译器报警、规范检查、形式化验证都属于这一大类。

运行时错误：程序跑起来才会暴露的错误，比如数组越界、除零、空指针解引用。静态分析里有一类工具专门证明这类错误不存在。

MC/DC 覆盖：修正条件判定覆盖，一种比“每行都跑到”更严格的测试覆盖率要求。安全等级高的代码通常要求做到它。

追溯性：一条需求、一段代码、一个测试用例三者之间能互相指认。评审的时候问的不是代码对不对，是这段代码对应哪条需求。

误报：工具报了问题，实际上不是问题。误报率决定了一个工具你还会不会用第二次。

开工前准备

1. 待审的代码目录，副本。副本先行在这件事上同样成立。
2. 你公司实际执行的规范文件。没有规范文件就先别做这件事，没有标准的审查等于闲聊。
3. 一份静态分析工具的报告（如果有的话）。AI 的发现和工具的发现对照着看，是这一节里最有价值的一步。
4. 代码的编译环境。它报的问题你要能立刻编译验证。
5. 一支笔。你要在纸上记它的误报条数。

操作步骤

这套流程我跑通过，六步：

1. 先定规范清单。把你公司实际执行的规范条目列出来，不要让它自己挑标准。
2. 分批喂代码。一次三百到五百行，不要一次丢一千八百行。批太大它会开始概括。
3. 要求它按四列输出：文件名、行号、违反的规范条目、原文引用。没有行号的条目一律退回。
4. 你自己逐条复核。这一步不能省，复核的时候顺手记误报。
5. 把它报的问题和你静态分析工具的报告放在一起对一遍，分三类：两边都报、只有工具报、只有它报。
6. 把“只有它报”的那类单独拎出来，逐条人工判断。这一类才是它真正的增量价值所在。

代码审查的三类结果

把 AI 的发现与静态分析工具的发现放在一起对



AI 的输出是线索，评审记录上签字的仍然是人

图 11-4 代码审查的三类结果：两边都报、只有工具报、只有 AI 报

图 11-4 作者根据公开资料绘制。

可复制提示词

整段复制，规范名和路径换成你自己的：

【任务】对我指定的 C 代码做一轮规范审查。你只是第一道关，不是最终判断。

【代码】D:\AI-CODE\motor_drv\src\pwm_map.c, 共 1823 行。

【规范】按我下面列出的条目检查，不许自己发明规范，也不许引用我没给的条目：

- MISRA C:2012 规则 10.1 (操作数类型不当)
- MISRA C:2012 规则 11.5 (指针转换)
- MISRA C:2012 规则 15.5 (函数单一出口点)
- 我司规范：所有除法运算前必须显式检查分母非零
- 我司规范：所有数组访问必须显式检查下标上界
- 我司规范：中断服务程序内禁止调用任何可能阻塞的函数

【输出格式】每一条发现必须写成一行，四列：

文件名 | 行号 | 违反的规范条目 (写我上面给的原文编号) | 该行代码原文引用
不许输出没有行号的条目，不许输出“建议关注某某函数”这类泛指条目。

【置信度】每条发现后面加一个置信度，只能填这三个值之一：

- 确定 (我有把握，且依据是你给的规范条目)
- 可能 (我看到了疑点，但需要看更多上下文)
- 我不是在做规范检查 (这条属于风格建议，不属于规范违反)

第三类单独归到报告末尾，不许混在前面。

【纪律】

1. 不许修改任何代码。只报告。
2. 不许引用我没给的规范条目。如果你认为某处违反了别的规范，
写进“待人工确认”一节，并注明你推断的规范名，不许写成结论。
3. 每 300 行暂停一次并汇报当前累计发现数，等我确认再继续下一批。
4. 全部完成后统计：总发现数、按规范条目分组的计数、
以及你标记为“可能”的条数。

过程与判断点

一份合格的报告长这样：

```
> [批次 1 / 6] 行 1-300, 累计发现 4 条。
> pwm_map.c | 142 | MISRA C:2012 规则 10.1 | duty = (uint16)(period * ratio); | 确定
> pwm_map.c | 203 | 我司规范：除法前检查分母 | freq = base / div; | 确定
> pwm_map.c | 271 | MISRA C:2012 规则 15.5 | 函数 PwmMap_Update 内有 3 个 return | 可能
> pwm_map.c | 288 | MISRA C:2012 规则 11.5 | ctx = (PwmCtx *)raw_ptr; | 确定
> ...
> [统计] 总发现 23 条。规则 10.1 共 6 条；规则 11.5 共 2 条；
> 规则 15.5 共 3 条；我司规范除法共 7 条；数组上界共 5 条。
> 标记为“可能”的 4 条。
> [待人工确认] 3 条，涉及我推断的规范 (已注明规范名)，不作为结论。
```

我看四处。

第一，行号能不能直接跳。你要能在编辑器里按下那个行号，跳过去，看到的就是它引用的那行原文。跳过去对不上，说明它在编。这是最快的一种验伪动作。

第二，"确定"和"可能"的比例。我跑的那轮是 23 条里 4 条标"可能"。如果一份报告里全部标"确定"，它要么非常强，要么在硬撑。

第三，第三类有没有被单独隔开。风格建议混进规范违反里，是这类工作最常见的污染。你按规范审的目的是过审，不是为了把代码写得好看。

第四，分组计数。按规范条目分组之后，你会发现某一组的数字特别集中。我那轮是除法检查 7 条最多，说明这个文件的作者在这一类上习惯性省略，后面可以针对性地跟他谈一次。

翻车清单

翻车一：它编了一个规范条目号。症状是报告里出现你没给过的编号，写得非常正式。根因是它在凭记忆填编号。补救：提示词里写死"只许引用我给的条目"，并要求每条发现后面附你给的原文。编号对不上你给的原文，就是编的。

翻车二：把风格问题当安全问题报。症状是报告里一堆命名、缩进、注释的建议，淹没了真正的问题。根因是你没给它规范清单，它自己按通用最佳实践发挥。补救：先定清单再开审，并且要求风格类单独归类。

翻车三：漏报。这类翻车最难受，因为它没有症状。你不知道它漏了什么。补救：拿一段你已经人工查出问题的代码先让它审一遍，看它能不能找出来。这一步叫"用已知答案给它做摸底考"，五分钟，值。

验收标准与变体

验收四条：

1. 每一条发现都有行号，且行号能直接跳到它引用的那行原文。
2. 所有规范条目编号都在你给的清单里，没有自创条目。
3. 抽三条"确定"的发现，你自己按规范原文核一遍，三条都成立。
4. 摸底考通过：一段你已知有问题的代码，它能找出来。

变体一：变更集审查。不审整个文件，只审这次改动的那几十行。这一类审查的价值最高，因为上下文新、记忆热、评审人也愿意看。

变体二：跨文件接口审查。让它读两个模块的头文件和调用点，查接口约定：参数顺序、单位、取值范围、错误码含义。这类问题单看一个文件永远发现不了。

变体三：给新人做预习。新人提交之前先跑一遍这套审查，自己先改一轮。

一条必须单独说的规矩：工具认证不等于模型认证

这一小节是本章最要紧的一段。如果你这一章只看一段，看这段。

在汽车功能安全的语境里有一个词叫工具认证。意思是：你用来开发或验证软件的那个工具，如果它出错会导致你漏掉一个安全问题，那么这个工具本身需要被证明是可信的。行业里对应 ISO 26262 的相关部分，常见做法是对工具做鉴定，出具证据包，说明它在什么范围内可信、置信度等级是什么。

一个静态分析工具可以走完这套流程。它有确定的版本、确定的规则集、确定的行为。同一份代码喂进去两次，它输出两次一样的结果。所以它拿得到那张纸。

一个 AI 模型走不了这套流程，至少目前走不了。原因有三条，每一条都很硬：

第一条，它没有确定的版本行为。同一段代码、同一句提示词，问它两次，它可能给出两份不完全一样的清单。不可重复的东西，没法做工具鉴定。

第二条，它没有确定的规则集。它的判断来自训练，不是一个你能打开看的规则库。你没法指着第 47 条规则说“这一条在我们的适用范围内”。

第三条，它会说错，而且说错的时候语气跟说对的时候一样。

所以下面这三句话，请你在做汽车软件的时候记住：

第一句：AI 辅助生成的代码，不因为“AI 说过没问题”就免除验证责任。该做的单元测试、该做的 SIL、该做的实车标定，一样都不能少。第二句：AI 辅助审查过的代码，不因为“AI 审过了”就免除人工评审。它的输出是线索，评审记录上签字的仍然是人。第三句：一个通过工具鉴定的静态分析工具，不会因为外面套了一个 AI 外壳就自动获得那份鉴定。反过来也一样：AI 很好用，不意味着它现在能上那张纸。

它们在流程里的正确位置是这样的：

环节	AI 可以做什么	谁签字
需求梳理	整理、查漏、列检查项	系统工程师
模型搭建	搭结构、改参数、生成变体	控制工程师，MIL 结果为准
代码生成	配置生成选项、整理生成物	工具链本身，SIL 结果为准
代码审查	出线索、分组计数、提示疑点	评审人，逐条复核
规范检查	辅助归类、辅助解释	静态分析工具出正式报告
测试	生成测试用例草稿、补覆盖率	测试工程师，覆盖率数据为准
安全论证	整理证据、写文档草稿	安全工程师，负全部责任

这张表最右边那一列是本小节的结论：每一行的签字人都是人，没有一行写着“AI”。

老兵说：工具认证证明的是这个工具不出错，模型认证要证明的是这个模型不出错。前者可以靠重复试验做，后者现在没人做得出来。在这件事上，别拿好用当可信。

场景 29 CAE 仿真前处理：让 AI 准备求解器输入 ★

一句话背景

一次结构强度仿真，从几何清理、画网格、加材料、加边界条件到写出求解器能读的输入文件，熟手一整天。这活里最烦的是重复：每改一次几何就要重来一遍流程，每换一个求解器就要重写一遍输入文件。这一节讲社区里已经做到什么程度。

名词小词典

CAE：计算机辅助工程。用计算机算结构强度、流场、电磁场、温度场的这一类工作的总称。

前处理 / 求解器 / 后处理：仿真的三段。前处理是准备输入，包括几何清理、画网格、加材料与边界条件；求解器是真正算数那个程序；后处理是看结果、出云图、提数字。

网格（Mesh）：把连续的几何切成几万到几百万个小块，求解器只在小块上算。网格质量直接决定结果准不准、算不算得出来。

边界条件：告诉求解器这个件是怎么被约束的、受了什么力、跟外界怎么换热。边界条件错了，网格再漂亮结果也是错的。

输入文件：求解器读的那个文本文件。Abaqus 读 `.inp`，很多开源求解器也读类似的文本格式。它是纯文本，能被读、能被比、能进版本管理。

ODB：Abaqus 的结果数据库文件格式，二进制。仿真算完之后的结果都在里面。

开工前准备

1. 求解器装好并授权到位。Abaqus、ANSYS 系列、Altair 系列都是商业授权；CalculiX、LAMMPS 这类是开源的，可以先用开源的练手。
2. 一个 MCP 服务器或 Agent 桥。公开资料里社区有一个名为 CAE-Agent-Hub 的项目（出处：开源代码托管平台上的公开项目，许可证以项目主页标注为准），公开介绍里覆盖的求解器包括 Abaqus、ANSYS Fluent、ANSYS Workbench、ANSYS HFSS、Altair 系列、CalculiX、LAMMPS 等。面向 Abaqus 还有一类项目（公开介绍里写到工具数在 100 个以上，并提到以 Solver Doctor 为代表的求解器报错模式库，以及以 ODB Lens 为代表的结果文件读取能力）。ANSYS 一侧官方有基于 PyAnsys 的 MCP 项目。名称、覆盖范围与许可证，一律以项目主页和官方文档为准。
3. 一个能跑通的最小算例。一根悬臂梁、一块板，越简单越好。你要有它的解析解。
4. 工作目录全英文无空格。
5. 一份你手算得出的解析解。没有解析解，你无从判断它算出来的是不是对的。

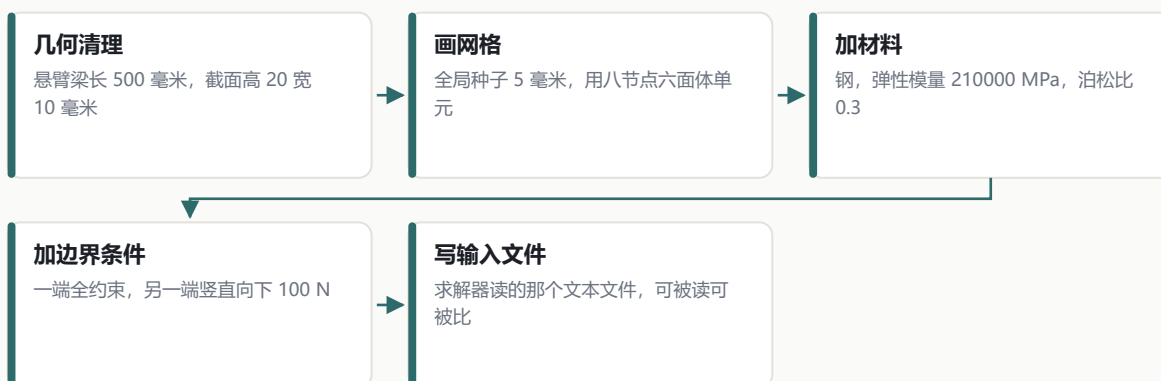
操作步骤

这一节标 ★，来自公开资料，下面给通用顺序：

1. 先在软件里手动跑完最小算例：建几何、画网格、加边界、求解、看结果，与解析解对一遍。手动通不过，就不要谈 AI。
2. 起桥、登记、验握手。工具列表里应当能看到建几何、画网格、设材料、加边界条件、写输入文件、提交求解这几类。
3. 让 AI 只读一份已有的输入文件，把它读懂并复述给你：有多少个单元、多少个节点、什么单元类型、几处边界条件、什么材料。它读得对，才敢让它写。
4. 让它写一份最小算例的输入文件。不要让它直接操作几何，先让它写文本。文本你能读。
5. 你人工核这份输入文件。核四个数：节点数、单元数、边界条件数、材料参数。
6. 提交求解，与解析解对比。误差门槛事先定死。
7. 逐步加码到真实几何，每加一级核一次那四个数。

仿真前处理的五段链路

先让它读一份输入文件，读懂了才敢让它写



人工核四个数：节点数、单元数、边界条件数、材料参数

图 11-5 仿真前处理的五段链路：几何、网格、材料、边界条件、输入文件

图 11-5 作者根据公开资料绘制，内容依据公开软件文档与社区案例整理，非本人实测。

可复制提示词

第一段，只读一份已有输入文件。这一步是我在这一侧最推荐的开局：

【任务】读我给的这份 Abaqus 输入文件，做只读体检，先不要改任何东西。

【文件】D:\AI-CAE\case01\beam.inp

【报出以下内容，每项都要有数字】

1. 节点总数、单元总数、单元类型清单（每种类型各多少个）。
2. 材料定义：材料名、弹性模量、泊松比、密度。单位一并报出。
3. 边界条件：共几处，每处的类型（位移约束 / 力 / 压力 / 温度）、作用的节点集或单元集名字、数值与方向。
4. 分析步：几个分析步、每个分析步的类型、时间长度、增量步设置。
5. 输出请求：要求输出哪些变量，输出到什么文件。

【单位】把文件里出现的单位全部列出来，并检查是否自治（长度、力、弹性模量、密度之间的量纲能不能对上）。

不自洽的地方单独列一节写清楚。

【纪律】

1. 不许用“文件看起来完整”这类话，只报数字。
2. 任何读不懂的关键字，原样贴出来并写“此关键字我无法确认含义”，不许按常见用法猜。
3. 只读，不许保存，不许提交求解。

第二段，让它写一份最小算例的输入文件，并把验证写进指令：

【任务】为我写一份 Abaqus 输入文件，算一根悬臂梁的末端挠度。
只写文件，不许提交求解。

【几何与载荷】

悬臂梁：长 500 mm，矩形截面 20 mm × 10 mm（高 × 宽）。

材料：钢，弹性模量 210000 MPa，泊松比 0.3。

一端全约束，另一端施加竖直向下的集中力 100 N。

【网格】全局种子尺寸 5 mm，单元类型用八节点六面体单元。

【验证：你必须自己算一遍解析解】

末端挠度解析公式： $\delta = F \times L^3 / (3 \times E \times I)$

其中 $I = b \times h^3 / 12$ ，b 为截面宽，h 为截面高。

把你的代入过程逐步写出来给我看，报出解析解的数值（毫米，保留四位小数）。

写完输入文件之后预测一下：有限元结果应当比解析解略大还是略小，为什么。

【输出】

1. 完整的输入文件原文写到 D:\AI-CAE\case01\cantilever.inp。
2. 在同一个目录写 readme.md，记录：节点数、单元数、材料参数、边界条件清单、解析解数值、你的预测。
3. 把输入文件里的节点数、单元数、边界条件数三个数字报给我。

【纪律】

1. 单位必须自洽，全部用毫米、牛顿、兆帕这一套，写进文件注释里。
2. 不许为了让我看不出问题而省略任何一行。文件要能直接被求解器读。
3. 解析解不许套错公式：这是矩形截面，不是圆截面。
公式里有疑问就写“此处我无法确认”，不许硬套。
4. 你不许提交求解，也不许预测一个具体的收敛值当作实测值。

过程与判断点

它交出来的东西，我看四处。

第一，解析解的代入过程。这一条我把公式直接写进提示词了，就是为了能对着看。500 长度、100 力、210000 模量、截面 20 乘 10，惯性矩 I 等于 10 乘 20 的三次方除以 12，等于 6666.67 四次方毫米。挠度等于 100 乘 500 的三次方除以 3 乘 210000 乘 6666.67，约 2.976 毫米。它报出来的数跟这个差一个数量级，就是公式套错了。

第二，单位自洽。毫米、牛顿、兆帕这一套是自洽的：兆帕等于牛顿每平方米，所以弹性模量 210000 直接配毫米和牛顿，不用换算。它如果在文件里混进了米或者帕，弹性模量那个数就会差六个数量级，而求解器不会报错，它只会给你一个错得很离谱的结果。

第三，它敢不敢做预测。我让它预测有限元结果比解析解大还是小。这是有标准答案的：有限元用位移插值，刚度偏刚，挠度结果通常略小于解析解。它答对了不一定说明它强，但它答不出、或者绕着走，说明它没有真正理解自己在写什么。

第四，三个计数值。节点数、单元数、边界条件数。5 毫米种子、500 乘 20 乘 10 的梁，单元数量级在 4000 上下，节点数在 5000 上下。它报出 4 万个单元，说明种子尺寸理解错了或者几何尺寸理解错了。

翻车清单

翻车一：单位不自洽，求解器不报错。症状是算完了，结果数值离谱但不报错。根因是长度用毫米、模量用帕，量纲差六个数量级。补救：在提示词里写死单位体系并要求它做量纲自洽检查，文件开头用注释声明单位。

翻车二：解析公式套错。症状是两边数字漂亮地对上了，其实是同一个错误的两种表达。补救：公式由你在提示词里给定，或者你自己手算一次最简单的那个数。这一条和第 10 章体检检查单里那条是同一件事。

翻车三：网格画了但边界条件漏了。症状是求解器报"刚度矩阵奇异"或者算出个天文数字。根因是件没有被完全约束，它会刚体飞走。补救：提交求解之前，让它把约束的节点集和自由度逐一列出来给你看，六个刚体自由度必须全被约束住。

验收标准与变体

验收五条：

1. 输入文件能被求解器直接读，不需要你手改。
2. 节点数、单元数与按种子尺寸估算的量级相符。
3. 单位体系自洽，并在文件注释里写明。
4. 求解结果与解析解的相对误差在你定的门槛内（悬臂梁这种简单算例，百分之几以内是合理的）。
5. 加密一次网格再算一遍，结果应当向解析解方向收敛。不收敛说明模型有问题，不是网格问题。

变体一：参数扫掠。一个尺寸扫五个值，让它生成五份输入文件并批量提交，最后给你一张表。这是前处理这一侧最典型的批量活。

变体二：换求解器。同一个算例让它分别写出 Abaqus 和 CalculiX 的输入文件，两边结果对比。这一步能暴露它对某个求解器格式的理解有多深。

变体三：给求解器报错当医生。求解失败时把报错原文丢给它，让它定位原因。公开资料里有项目专门整理了求解器的常见错误模式库。这一类的验收动作是：它给的修改你要能说出为什么，说不出就别改。

老兵说：前处理这一侧最划算的一招，是先让它读、再让它写。读得懂一份输入文件的 AI，写出来的输入文件才敢提交求解。

场景 30 仿真结果解读与报告：从结果文件到结论 ★★

一句话背景

一次仿真算完，你要从结果文件里提出最大值、找出它在哪、判断这个数能不能接受、写成一页能进评审的结论。这一步熟手三到四小时，因为你要在软件里点几十次才能把该提的数提全。AI 做这步很快，但有一个它绝对不能碰的位置。

名词小词典

结果文件：求解器算完之后写出来的文件。Abaqus 的 ODB、ANSYS 的 RST、很多开源求解器用的 VTK，都是这一类。它们是二进制，肉眼读不了，得靠接口读。

云图：把结果按颜色画在模型上，红的地方大、蓝的地方小。云图是给人看的，不是给机器判的。

应力集中：几何突变的地方应力会显著偏高，比如孔边、台阶根部。它是真实存在的物理现象，也可能是网格太糙造成的假象。

收敛：网格越密，结果越接近一个稳定值。结果不再随网格变化时，才算收敛。

安全系数：材料能承受的极限除以实际算出来的值。它小于 1 就是坏了，但多少算够，是你的行业规范说了算，不是软件说了算。

后处理：算完之后读结果、提数字、出图、写报告这一整段工作。

开工前准备

1. 结果文件在手，并知道它是哪个算例、哪一次求解的产物。文件名带版本号。
2. 读取接口准备好。能用 Python 读就用 Python 读，能用软件自带接口就用自带接口。不要让 AI 凭文件名猜内容。
3. 一份你已经知道的基线值。哪怕是上一次算出来的数，或者手算的粗略估计。没有基线，你没法判断它提的数字对不对。
4. 验收判据在手：什么值算过，什么值算不过。这个判据来自你的行业标准或者公司规范，不是 AI 定的。
5. 报告模板。字段名、单位、有效位数，先定死。

操作步骤

这套流程我跑通过，六步：

1. 先让它只读元数据：这个文件是什么分析类型、多少个增量步、多少个节点、哪些变量可用。先读元数据再读数据。
2. 让它列出可用变量清单，你从里面挑你要的那几个，不要让它替你挑。
3. 让它提取你指定的量：最大值、最小值、出现位置（节点编号与坐标）、所在单元。

- 4. 交叉验证一个数字。挑一个量，用另一种方式（比如软件界面手动点一次）核一遍，看是不是同一个数。
- 5. 让它做收敛性说明。这个值是网格加密后稳定下来的，还是随网格一直在变。
- 6. 让它写报告，但报告里的结论那一栏留空，由你填。

从结果文件到结论

元数据、变量清单、提取指定量、交叉验证

- 1 先读元数据
分析类型、增量步数、节点总数、单元总数
- 2 列可用变量清单
变量名原样列出，你自己挑，不让他替你挑
- 3 提取你指定的量
最大值、最小值、出现位置的节点编号与坐标
- 4 交叉验证一个数字
换一条读取路径再取一次，报出相对差异
- 5 做收敛性说明
是否随网格加密稳定，说不清就写无法判断
- 6 写报告，结论留空
判据在你手里，结论栏写待人工填写

它可以替你提数、替你找位置，报告最后那行结论得你自己写

图 11-6 从结果文件到结论的四段：元数据、变量、提取、交叉验证

图 11-6 作者根据公开资料绘制。

可复制提示词

整段复制，文件名、变量名换成你自己的：

【任务】读我给的仿真结果文件，提取我指定的量，写成一份报告草稿。
你可以解读，但不许下结论。

【文件】D:\AI-CAE\case07\bracket.odb

【第一步：元数据】先读元数据并报出：
分析类型、分析步清单与步数、节点总数、单元总数、
可用的结果变量清单（把变量名原样列出来，不许翻译、不许合并）。

【第二步：我指定的量】我只关心以下三个量，不许提取别的：
1. 变量 S（应力）的 Mises 最大值
2. 变量 U（位移）的合位移最大值
3. 变量 RF（支反力）的合计值
对每一个量报出：数值（带单位）、出现的节点编号、该节点的坐标、
所在的部件名。

【第三步：位置描述】对上面三个最大值，用几何语言描述它们出现在模型的什么部位
（比如"靠近安装孔边缘""位于加强筋根部"）。
描述不了的坐标就写"位置我无法描述"，不许猜。

【第四步：交叉验证】对 Mises 最大值这个量，用另一种读取路径再取一次
（比如先从单元取值再取节点值，或换一个读取接口），
把两次的结果并排列出来，报出相对差异。

【第五步：收敛性】说明本次结果对应的网格尺寸是多少，
以及这个网格尺寸下结果是否处于收敛区间。
如果你没有足够信息判断收敛性，写"收敛性我无法判断"，不许写"已经收敛"。

【第六步：报告】写到 D:\AI-CAE\case07\report_draft.md，结构如下：

- 算例信息（文件、分析步、网格尺寸）
- 提取结果表（三个量，各四列：量名、数值、位置、读取路径）
- 交叉验证结果
- 收敛性说明
- 结论：留空，写"【结论待人工填写】"

【纪律】

1. 不许写"强度满足要求""设计安全""可以放行"这类判断。
判据在我手里，不在你手里。
2. 任何读不出来的量，写"未读出"，不许用相近变量替代，不许插值估算。
3. 不许修改结果文件，只读。

过程与判断点

一份合格的草稿，我看三处。

第一，变量清单是原样列的。它列出来的变量名必须跟软件里的一模一样。它写"应力"，你得追问是Mises 应力还是主应力、是单元值还是节点值、有没有做外推。变量名这一步含糊，后面所有数字都白提。

第二，交叉验证的两次结果。单元值和节点值本来就不应该完全相等，这是正常的。你要看的是相对差异的数量级。差百分之零点几，正常；差百分之三十，说明某一次读取取错了对象。

第三，位置描述。让它用几何语言说出"最大值出现在安装孔边缘"，比让你自己在云图上找要快得多。但你要防它猜：它说不出的时候，必须写"位置我无法描述"。

第四，结论那一栏留空。这是这条提示词里最要紧的一句。我特意把它写成了模板里的固定字段。

还有一件事要单独讲：云图的色标。我跑那一趟的时候，它一口气给了我四张图，四张图看着完全不一样，一张近乎全蓝，一张大片红。数据是同一份，区别只在色标范围：全蓝那张把上限设成了材料屈服强度的三倍，红的那张把上限设成了最大值的 1.1 倍。同一份结果能讲出两个故事。这就是我把"云图出完必须人眼看一遍"写进验收的原因，并且我要求它在每张图下面把色标的上限值和下限值写出来。

翻车清单

翻车一：它替你挑了变量。症状是报告里出现你没要求的量，而且它写得像结论一样。根因是你没限定变量清单。补救：在提示词里明确"只提取这三个量，不许提取别的"，并且要求第一步先列完整清单给你挑。

翻车二：单元值和节点值混着报。症状是同一个量在报告的两处出现两个不同的数。根因是单元中心值和节点外推值是两套东西。补救：要求每一次提取都注明读取路径，报告里"读取路径"单独占一列。

翻车三：它给了结论。症状是报告末尾写着"安全系数充足，设计可行"。根因是它把判据当成了常识。补救：模板里把结论栏写成固定占位符，并在纪律里禁止判断类措辞。这一条我再说清楚一点：它不知道你的安全系数取多少，不知道你的载荷是极限工况还是疲劳工况，不知道这个件坏了会伤到人还是只是响一声。这三件事它都不知道，所以它没资格下结论。

验收标准与变体

验收四条：

1. 三个指定量的数值，至少有一个你用另一种方式（软件界面或另一段脚本）核过。
2. 每个量都带单位、带节点编号、带读取路径。
3. 交叉验证那一行有具体数字，不是"两次结果一致"。
4. 报告末尾的结论栏确实留空，没有出现任何判断性措辞。

变体一：多工况扫描。十个工况，每个工况提同一组量，输出一张十行表。这一类活人工做要点一百次鼠标，脚本化之后几分钟。

变体二：改模前后对比。改一处几何之前存一份提取结果，改完再存一份，两张表并排。最大值降了多少、位置有没有转移，一眼就能看出来。

变体三：给评审做图。让它批量导出指定视角、指定变量的云图。图出完之后仍然要人眼看一遍，因为云图的色标范围是可以被人为调的，同一份数据能画出完全不同的两张图。

老兵说：AI 可以替你提数出来，可以替你说出最大值在哪，甚至可以替你写报告。但报告最后那一行结论，得你自己写。签字的那天站在评审会上的是你，不是它。

小白词典

PCB：印刷电路板，那块绿色的板子，铜箔刻成线路，元器件焊在上面。类比：城市道路网加沿街的房子。

EDA：电子设计自动化软件的总称，画原理图、画板子、查规则都在里面做。常见的有商业软件 Altium Designer 和开源软件 KiCad。

网表 (Netlist)：一份文本文件，写着谁跟谁连在一起。原理图导出的和 PCB 导出的本该完全一致。类比：一张通讯录。

BOM：物料清单，这块板子由哪些物料、各几个、什么编码，一表列清。类比：做菜前的配料表。

ECO：工程变更指令。改一个电阻值、换一个供应商，都算一次 ECO。

封装 (Footprint)：一个元器件在板上占的那块焊盘图形。类比：鞋码，鞋对了脚才穿得进去。

求解器 (Solver)：真正算数那个程序。前处理给它输入，它算完给结果文件。类比：厨房里那个真正在炒菜的灶。

前处理：仿真三段里的第一段，几何清理、画网格、加材料、加边界条件、写输入文件。类比：备菜。

后处理：仿真三段里的最后一段，读结果、提数字、出云图、写报告。类比：摆盘和上菜。

网格 (Mesh)：把连续几何切成很多小块，求解器只在小块上算。类比：积分就是切得足够细之后求和。

收敛：网格越密结果越接近一个稳定值。结果不再随网格变，才算收敛。类比：像素越高越接近真实照片。

ISO 26262：汽车功能安全的国际标准。它管的是“这个电子系统坏了会不会伤到人，以及你怎么证明它不会”。

ASIL：汽车安全完整性等级，A 到 D，D 最高。等级越高，验证要求越严。类比：不同危险程度的作业要不同级别的防护。

AUTOSAR：汽车电子软件的行业标准接口规范，分 Classic 与 Adaptive 两支。类比：不同厂家零件之间的统一安装孔位。

MISRA C：嵌入式行业广泛采用的 C 语言编码规范，一条一条编号。类比：交通法规，每一条都对应一类事故。

MIL / SIL：模型在环与软件在环。MIL 拿模型跑仿真，SIL 拿生成的 C 代码跑仿真，两者结果应当一致。类比：草稿和定稿要对得上。

MCP：一套让 AI 调用其他软件能力的通用接口标准。AI 说话，桥翻译，软件动手。

工具认证：功能安全语境下，证明你用的开发工具本身可信的一套鉴定流程。它是给工具发的证，不是给人发的证。

你的岗位怎么用

硬件与电路板工程师：先抄场景 26。它不依赖任何授权，三份文件就能跑，二十分钟出一份差异报告。跑顺了再试场景 25，从只读体检开始，别一上来就让它动画板。

软件与算法工程师：场景 28 直接能用，但请连同那节"工具认证不等于模型认证"一起读。你的收益在"我提交之前先自查一轮"，不是"AI 替我过评审"。场景 27 属于调研级，动手前先确认你的版本和授权。

仿真与 CAE 工程师：场景 30 是你最容易上手的一节，因为它只读不改。先用你手上一个已经算完的算例跑一遍，看它提的数跟你的基线差多少。场景 29 属于调研级，先用开源求解器练手，别拿商业授权的第一个月去试。

质量与功能安全工程师：请直接翻到场景 28 后面那一小节。那张"谁签字"的表是你跟开发团队谈这件事时可以贴在会议室墙上的东西。AI 的证据链可以进档案，AI 的结论不能进档案。

项目经理：盯场景 26 那个"先报六个底账数字"的动作。它跟第 10 章那个冻结评分表是同一件事：标准先定死，后面不扯皮。这一条放在 AI 之外的任何外包任务上都成立。

采购与供应商管理：场景 26 的变体二跟你直接相关。外协厂的 BOM 进来之前先对一遍，重点看物料编码不同的那些行。

本章交付物

一份四列代码审查模板，外加一张三份文件交叉引用的底账卡。

做法一：把场景 28 那段提示词的四列输出格式抄进你的笔记，字段换成你公司实际执行的规范条目编号。以后每次提交代码之前跑一遍，二十分钟。

做法二：把场景 26 的六个底账数字写进你的检查清单第一位。原理图器件数、PCB 器件数、BOM 数量合计、三份网络数，六个数先摆出来，再谈差异。

第一件验收的活：挑一个你手上已经算完的仿真算例，走一遍场景 30，看 AI 提的最大值和你自己在软件里点的那个数差多少。差在百分之一以内，你就可以把它用在下一个算例上；差得离谱，你就知道这一侧的桥还得再调。

本章金句

在电路板和控制器代码上，AI 最危险的不是做错，是做对了九十九处、错在第一百处，而错的那一处长得很像对的那一处。

工具认证证明的是这个工具不出错，模型认证要证明的是这个模型不出错。前者能靠重复试验做，后者现在没人做得出来。

AI 辅助生成的代码，不因为"AI 说过没问题"就免除验证责任。

它的输出是线索，评审记录上签字的仍然是人。

它不知道你的安全系数取多少，不知道这是极限工况还是疲劳工况，不知道这个件坏了会伤到人还是只是响一声。这三件事它都不知道，所以它没资格下结论。

（下一章：让 AI 碰到你的电脑。电路板和求解器好歹还是文件，下一章它要碰的是你的文件系统、你的终端和你那些正在跑着的进程，那才是真正需要先把刹车装好的地方。）

第 12 章

让 AI 碰到你的电脑：浏览器、桌面、远程

卡住 AI 的从来不是它不够聪明，是你没让它碰到你的鼠标。工程师手上一半的脏活，都锁在那些只有网页、没有接口的系统里。

2025 年 12 月的那个晚上，我在做一件从入职第一年就在做的事：供应商资质核查。

一个新项目要定点两家供应商，我要核对五家候选供应商的资质。质量体系证书、实验室认可范围、关键设备清单、近三年交付记录、报价，五项内容散在五个地方：两家供应商自己的门户、一个行业平台、一个招标网站，还有对方邮件发来的一个压缩包。

我打开第一个门户，登录，找到证书页，复制证书编号，切到 Excel，粘贴。打开第二个门户，登录，找到设备清单，截图，切回 Word，插图片。第三家供应商的门户要用手机验证码登录，我拿起手机，等短信，输进去，页面过期了，重来。

晚上十一点，我完成了三家。Excel 里躺着一百多行资质数据，Word 里塞了四十多张截图。还剩两家，压缩包还没解开看。凌晨一点，我在解压包里翻设备清单的 PDF，翻到第四十页的时候，手停了一下。

那天晚上我干的全部动作，没有一个需要“我”这个人。

序章里我讲了那个夜晚的痛，这一章讲的是那个夜晚的解法。现在我干同样的活：把五个门户的登录方式、要摘的字段、表格样例写成一段话交给 AI，它去登录、去翻页、去摘数、去填表，中间在一个门户的验证码那里停下来问我，我点了一下，它接着干。四十分钟，五个门户全部跑完，桌面上多了一张汇总表，还有一页提示：两家供应商的三张证书在半年内到期。

两个晚上变成四十分钟，靠的不是模型变聪明了，是我终于让它碰到了我的电脑。

浏览器、桌面软件、远程桌面，这三样是 AI 从“动嘴”跨到“动手”的三道门。前几章你看到的是它怎么读写文件、怎么跑脚本；这一章你看到的是它怎么用你已经登录的浏览器替你点，怎么在真正的桌面软件里按鼠标，以及你不在电脑前时它怎么替你守着一台跑试验的机器。第四道门我放在最后单独说：什么东西不能让它碰。

下面是本章六个场景的速览，你先挑跟你有关的那个看。

场景	场景名	人工耗时	AI 耗时	可信度
31	浏览器自动化：让 AI 用你的登录态干活	约 2 小时	约 40 分钟	★★★ 实测
32	批量抓取没有接口的网页数据：四类站点	约 1 个工作日	约 50 分钟	★★★ 实测
33	卡住了怎么办：human-in-loop 接管	每次卡住 5 至 15 分钟	每次接管约 30 秒	★★★ 实测
34	Computer Use：让 AI 直接操作桌面软件	视任务而定	视任务而定	★★ 跑通

场景	场景名	人工耗时	AI 耗时	可信度
35	远程操控：人在家盯台架	一趟高铁 6 小时或整个周末	约 20 分钟	★★★ 实测
36	安全边界：什么能交给 AI 碰，什么不能	一次梳理约 2 小时	梳理本身可交给 AI	★★★ 实测

场景 31 浏览器自动化：让 AI 用你的登录态干活 ★★★

一句话背景

你每天要在十几个必须登录的网站之间搬数据：供应商准入门户、竞品配置库、电子招标平台、专利检索库、BOM 系统。这些系统没有一家给你开放接口，全是网页登录态。Agent 再聪明，碰不到你的登录态，就等于蒙着眼睛干活。这个场景教你把浏览器交给它，两小时的复制粘贴压到四十分钟。

名词小词典

登录态：你在这个网站上已经登录的那个状态。浏览器里存着一小段凭据，网站每次见到它就认你是你。AI 自己没有这段凭据，所以自己打不开你的门户。

浏览器扩展：装在浏览器里的小插件，能看见你打开的页面、能替你点击和填表。AI 就是靠它伸手的。

隔离窗口 / Agent 窗口：扩展专门新开的一个浏览器窗口，AI 只在这个窗口里干活。你正在用的那些窗口纹丝不动，你继续刷你的网页，它在旁边跑它的流程。

本机回环：一台电脑自己跟自己通信用的那个内部地址，不走外网。命令行工具把活派给浏览器扩展，走的就是这条路，数据不出本机。

会话 (session)：一次"AI 接管浏览器"的时段。开会话、关会话是两道必须做的动作，活干完不关，等于把家门敞着。

开工前准备

逐条核对，少一条都会卡住：

1. 一台 Windows 或 macOS 电脑，装好 Chrome 或同类的 Chromium 系浏览器。
2. 在浏览器应用商店里手动安装 BrowserSkill 的扩展。这一步 AI 替不了你，扩展必须你本人点安装。
3. 在本机装好 bsk 命令行工具，命令行里敲 `bsk version` 能出版本号。
4. 在你要跑的那些门户上，用自己的账号正常登录一遍，确认手动能打开目标页面。
5. 把要摘的字段写成一张字段字典：字段名、在哪一页、什么格式、空了怎么办。这张字典是后面所有活的地基。
6. 一个干净的输出目录，比如 `D:\AI-WEB\run01`，路径英文、无空格。

操作步骤

- 1. 命令行里开会话： `bsk session start` 。看到会话号说明通了。
- 2. 让 AI 列一遍当前浏览器里已登录的站点，确认它看得见你的登录态。
- 3. 把第一段提示词整段粘贴进对话框，回车。它是新开一个窗口跑的，你原来那个窗口不要动。
- 4. 它每完成一个站点汇报一次，你扫一眼数字对不对。
- 5. 它喊停的时候，去输出目录看文件。
- 6. 活干完，立刻执行 `bsk session stop` ，并手动关掉那个 Agent 窗口。
- 7. 需要它操作你正在看的标签页时，先 `bsk tab borrow` 借过来，干完 `bsk tab return` 还回去。

AI 到浏览器的三级链路

命令行工具、本机回环、隔离窗口

命令行工具

你在这头发指令：开会话、借标签页、关会话

本机回环通道

一台电脑自己跟自己通信用的内部地址，不走外网，数据不出本机

浏览器扩展与隔离窗口

扩展替它点击填表，活只在专门新开的窗口里跑，你原来的窗口纹丝不动

扩展这一步必须你本人手动点安装，AI 替不了你

图 12-1 AI 到浏览器的三级链路：命令行工具、本机回环、隔离窗口

图 12-1 作者根据公开资料绘制，仅表示原理链路，不含任何真实站点界面。

可复制提示词

整段复制，站点名、字段名、路径换成你自己的：

你现在可以通过 bsk 这台工具接管我浏览器里的隔离窗口。
这个窗口用的是我已经登录的登录态，你不需要也不许去猜任何账号密码。

【任务】在我给你的这份站点清单里，逐个站点把指定字段摘出来，
汇总写进一张 CSV。站点清单如下：

- 站点 A：某供应商门户，登录后进“资质证书”页，摘证书编号、发证机构、
发证日期、有效期至、证书状态五项。
- 站点 B：某招标平台，搜索关键词“微电机”，摘公告标题、发布日期、
投标截止日、招标人四项，只要最近 12 个月的。
- 站点 C：某专利检索库，按申请人检索，摘公开号、名称、申请日、法律状态四项。
- 站点 D：某 BOM 系统，登录后进物料查询页，按我给的 6 个物料号逐个查，
摘物料号、名称、单价、交货周期、最小起订量五项。

【字段纪律】每一条记录必须带三个附加列：来源站点、抓取时间、来源页面 URL。
摘不到某项就写“未找到”，不许填空、不许写“无”、不许自己推测一个值。
某一项在同一页出现多次时，全部摘出来，一行一条，不许只摘第一条。

【分页纪律】每一页摘完先报这一页的记录条数，再翻下一页。
翻页失败就停下报给我，不许跳过、不许假装翻完了。

【输出】写到 D:\AI-WEB\run01\result.csv，UTF-8 带 BOM，第一行为表头。

写完之后再输出一份 D:\AI-WEB\run01\summary.md，内容四项：

1. 每个站点实际抓到多少条，与站点清单里的预期条数对比；
2. 有多少条记录存在“未找到”字段，列出这些记录的主键；
3. 有没有同一主键被抓到两次以上的重复行，有就全部列出来；
4. 你不确定或没做到的地方，逐条写出来，不许含糊带过。

【红线】不许碰银行、企业单点登录页、密码管理器这几类页面。
不许用脚本去读取、导出、转发任何 cookie 或令牌。
遇到二次验证、验证码、确认弹窗，立刻停下来叫我接管，不许自行尝试绕过。

【纪律】每一步都要汇报真实返回值，不许用“已完成”“一切正常”这类话。
失败时把页面上的原始提示原样贴给我，然后等我指示。

过程与判断点

它应该返回一段像这样带数字的汇报。这段里的每一个数字你都能自己复核：

- > [会话] session 已建立，编号 a7f3，隔离窗口已打开。
- > [站点 A] 资质证书页共 4 条记录，已摘 4 条。其中 1 条“有效期至”为未找到，主键 ZS-2023-0871。
- > [站点 B] 搜索命中 23 条，筛选最近 12 个月后剩 11 条，已摘 11 条，分 2 页。
- > [站点 C] 检索到 38 条，已摘 38 条，其中 5 条法律状态为未找到。
- > [站点 D] 6 个物料号全部命中，已摘 6 条，单价 3 条为空值，已标未找到。
- > [汇总] 总计 59 条，未找到字段 9 处，重复主键 0 对。
- > [存疑] 站点 B 第 2 页的分页按钮点了两次才翻成功，第一次疑似页面未加载完，已重试。

这段汇报里我盯四个地方。

第一，条数对不对得上。站点 B 报搜索命中 23 条、筛选后 11 条，这两个数你自己手动搜一遍就能验。它报的数跟你手搜的数差一条，就是筛选条件它理解错了。

第二，有没有“未找到”。有“未找到”是好事，说明它在如实报告而不是编。一条都没有反而要警惕，那可能是它把空字段填成了上一行的值。

第三，重复行。翻页时最容易出重复，有些站点的第 2 页会重复第 1 页的最后几条。它报 0 对，你就抽查两个主键手动数一遍。

第四，它有没有自己报存疑项。上面那段里的“分页按钮点了两次”是它自己说出来的。会自己报问题的 Agent 值得多信一点，一句“全部顺利完成”反而最危险。

一次抓取的四项自检

它报回来的那一段汇报，你盯这四个地方



条数对不对得上

它报的数跟你手搜的数差一条，就是筛选条件被它理解错了



有没有如实写未找到

一条都没有反而要警惕，那可能是它把空字段填成了上一行的值



有没有重复行

翻页最容易出重复，它报零对你就抽查两个主键手动数一遍



有没有自己报存疑项

会自己报问题的 AI 值得多信一点，一句全部顺利完成反而最危险

每一项都要它给出数字，给不出数字的不算数

图 12-2 一次抓取的四项自检：条数、未找到、重复行、存疑项

图 12-2 作者根据公开资料绘制。

翻车清单

翻车一：扩展没装，或者装了没启用。症状是 `bsk session start` 直接报连不上，或者会话起来了但窗口里一片空白。补救：先在浏览器扩展管理页确认扩展处于启用状态，再确认命令行工具和浏览器是同一台机器上的同一套。扩展这一步必须人手动点，AI 替不了。

翻车二：抓回来的数据看着整齐，其实混进了上一页的行。症状是条数对、格式也对，但同一主键出现两次，或者某几行的日期跟相邻行一模一样。补救：在提示词里强制要求每个站点报分页条数，并在汇总里列出重复主键。这两句话是唯一能抓住这种错的办法，靠肉眼扫不出来。

翻车三：活干完忘了关会话。症状不体现在数据上，体现在安全上。你人走了，Agent 窗口还挂着你的登录态。补救：把 `bsk session stop` 写进收尾清单，跟保存文件、备份数据排在同一行。

验收标准与变体

验收五条，五分钟做完：

1. 打开 CSV，行数等于四个站点记录数之和，表头字段名与你的字段字典一字不差。
2. 随机挑三行，照"来源页面 URL"那一列手动打开页面核对三个字段，三个全对才算过。
3. "未找到"的格子确实是页面上没有，不是它没翻到位，抽查两处。
4. 全文搜一遍重复主键，结果为零。
5. 命令行敲 `bsk session list`，当前没有活跃会话。

变体一：比价。把站点 A 换成三家供应商各自的报价门户，字段换成单价、阶梯价、交货周期、付款条件。这类活人工一下午，它四十分钟。验收动作变成单价单位是否统一（元/件还是元/千件），这是比价表里最容易错的一栏。

变体二：盯更新。让它每天早上跑一遍招标平台的关键词检索，把昨天新增的公告摘出来写进一张增量表。第一次跑完后存一份基线，之后每次只输出增量。这一变体的价值不在省时间，在你不会漏掉截止日。

变体三：跨站点对账。同一批物料，在 BOM 系统里查一遍单价，在供应商门户里查一遍报价，让 AI 出一张差异表，标出差异超过 5% 的行。它同时需要两个登录态，人干起来最烦，也就最该自动化。

老兵说：这类工具是中性的。会用的人把重复劳动甩给它，自己腾出手做判断；不会用的人把登录态全盘交出去，等于把家门钥匙塞给陌生人。区别就在你能不能守住最后一节里的几条红线。

场景 32 批量抓取没有接口的网页数据：供应商门户、招标平台、专利库、BOM 系统 ★★★

一句话背景

上一节是"让它碰浏览器"，这一节是"让它碰完能交出一张能用的表"。四类站点的脾气完全不同：供应商门户有登录有分页，招标平台的列表会过期，专利库的字段藏着法律状态这种坑，BOM 系统的查询结果按物料号走。人工把这四类跑一遍是一个工作日，这一趟的实测是五十分钟，产出 59 条记录、9 处如实标注的"未找到"。

名词小词典

接口 (API)：软件之间约定好的一条正式数据通道。有接口的系统，AI 可以直接读数据；没有接口的，只能像人一样去点网页。本章这四类站点全是没有接口的那一类。

结构化：把页面上散着的文字整理成一行一行、一列一列的表格。抓取这件事的本质就是结构化。

主键：一行记录的唯一身份证。资质表里证书编号是主键，BOM 表里物料号是主键。没有主键，你没法查重也没法对账。

未找到：页面上确实没有这个字段时的标准写法。它跟空值、跟"无"、跟 0 都不同，它是一个明确的结论。

增量：相对于上一次抓取新出现或发生变化的部分。盯更新类的活只看增量，不看全量。

开工前准备

1. 场景 31 的环境已经跑通：扩展装好、命令行工具能用、会话能开。
2. 一张你自己写的字段字典，每个站点一份，写到字段级。字段级的意思是"有效期至"而不是"证书信息"。
3. 一个预期的记录条数。这个数你先手动跑一遍拿到，它是后面判断"它有没有漏"的唯一基准。
4. 四类站点各挑一个小样本先试：每类不超过 5 条记录。样本跑通再放开量。
5. 输出目录准备好，CSV 与汇总报告各一个文件名。

操作步骤

1. 先只跑一个站点、一个字段，确认它摘出来的东西跟你眼睛看到的一致。这一步叫单点验证，五分钟，省后面两小时。
2. 单点通过后把字段扩到完整字段字典，条数限制在 5 条以内再跑一遍。
3. 放开条数，但要它在每一页报条数。分页是这类活的主要出错点。
4. 四个站点跑一个存一个文件，最后合并，中途挂了不至于全丢。
5. 让它出汇总报告，重点看"未找到"和"重复主键"两栏。
6. 你自己手动核三条。只核三条，但必须核。

四类站点的抓取路径

登录、检索、分页、字段落地地各不同

供应商门户

需登录，有分页，一页二十条。最容易错在筛选条件，精确匹配碰上模糊状态词会漏记录

招标平台

公开可查，列表会过期。最容易错在把已截止的过滤掉，要同时出增量与过期两份

专利检索库

公开可查，字段层次深。坑在法律状态被归纳，一律摘页面原文，不许归并成两档

BOM 系统

需登录，按物料号逐个查。坑在计价单位，单价必须跟计价单位绑在一起摘

四类各挑一个不超过五条的小样本先试，样本跑通再放开量

图 12-3 四类站点的抓取路径：登录、检索、分页、字段落地地各不同

图 12-3 作者根据公开资料绘制，站点名称均为脱敏示意。

可复制提示词

第一类是供应商门户。这段提示词可以直接复制：

【站点类型】供应商门户，需登录，有分页，一页 20 条。

【任务】登录后进入“资质证书”页，把全部记录摘出来。

字段：证书编号（主键）、证书名称、发证机构、发证日期、有效期至、证书状态。

筛选条件：证书状态为“有效”，发证日期在最近 5 年内。

【分页】每页摘完立刻回报：本页序号范围、本页条数、本页第一个和最后一个主键。

翻到最后一页的判断依据是“下一页”按钮变为不可点，不许靠猜总页数。

【到期提醒】摘完后单独算一遍：有效期至距今不足 180 天的记录有几条，

把它们的证书编号、剩余天数、对应供应商名列出来，按剩余天数升序排。

【输出】D:\AI-WEB\run02\supplier_cert.csv，UTF-8 带 BOM。

第二类是招标平台，重点在增量：

【站点类型】电子招标平台，公开可查，列表会过期。

【任务】按关键词"微电机""执行器""电机总成"分别检索，
摘：公告标题（主键）、发布日期、投标截止日、招标人、公告链接。
只摘发布日期在最近 12 个月内的。

【增量】同目录下已有 baseline.csv。这次只输出 baseline 里没有的新主键，
写到 D:\AI-WEB\run02\tender_delta.csv，
并另写一份 D:\AI-WEB\run02\tender_expired.csv，
记录 baseline 里有但这次检索不到的主键（可能已过期下架），
这两份的行数之和要与本次检索命中条数对得上。

【纪律】截止日早于今天的记录照常摘出来，但要在最后一列标"已截止"，
不许自行过滤掉。我看的是全貌，不是你想让我看的那一半。

第三类是专利检索库，坑在法律状态：

【站点类型】专利检索库，公开可查，字段层次深。

【任务】按申请人名称检索，摘：公开号（主键）、专利名称、申请日、授权日、
法律状态、同族数量。

【法律状态纪律】法律状态这一栏必须摘页面上的原文，
包括"有权""无权""审中""未缴年费"这类原始字样，
不许你归纳成"有效/无效"两档。原文是什么就写什么。

【同名处理】同一申请人可能有多个近似名称（比如带括号的、带外文名的），
每个名称单独检索一次，最后按公开号去重，
并把去重前与去重后的条数都报给我。

【输出】D:\AI-WEB\run02\patent.csv。

第四类是 BOM 系统，重点在单位：

【站点类型】内部 BOM 系统，需登录，按物料号逐个查。

【物料号清单】

M-100234

M-100871

M-102455

M-103902

M-104118

M-104660

【任务】逐个物料号查询，摘：物料号（主键）、物料名称、单价、币种、计价单位、交货周期（天）、最小起订量、供应商名称。

【单位纪律】单价必须带币种，计价单位必须原样摘出来（个 / 套 / 千克 / 千件，页面写什么就写什么）。
不许把不同计价单位的单价放在一起比大小。

【零值纪律】单价为 0 的记录照常摘出来，最后一列标"单价为零待核"，不许跳过，也不许当成未找到。

【输出】D:\AI-WEB\run02\bom_price.csv，顺序与物料号清单一致。

过程与判断点

四类站点跑下来的关键差别，我按站点说。

供应商门户最容易错的是筛选条件。我第一趟跑的时候让它筛"证书状态为有效"，它摘出来 4 条，我手动看是 5 条。差的那一条的证书状态页面上写的是"有效期内"，不是"有效"。它做的是精确匹配，页面用的是模糊状态词。补救办法是在提示词里把所有可能的状态词列全，并要求它把页面上出现的全部状态值先去重报一遍给你看，你再定筛什么。

招标平台最容易错的是把过期的过滤掉。列表会过期是这类站点的天性，AI 的默认动作是"把能打开的都摘了"，这就导致那些已经截止但仍能打开的公告混进你的表，而那些刚下架的你永远看不到。上面那段提示词里"增量 + 过期两份文件行数之和要对得上"这一句，就是为了防止这个。

专利库的坑在法律状态被归纳。我见过它把"未缴年费"归纳成"无权"，这两个在法律上完全是两回事，前者补缴还能恢复。所以我在提示词里写死了"摘原文，不许归纳"。这一条对任何带法律属性的字段都适用，不只是专利。

BOM 系统的坑在计价单位。同一个物料，A 供应商按个报价 3.2 元，B 供应商按千件报价 3200 元。数字摆在一起，AI 会告诉你 B 贵了一千倍。所以单价必须跟计价单位绑在一起摘，比较之前先归一化。这条我在电机项目上吃过一次亏，采购拿着那张表去谈价，谈崩了才发现是单位问题。

四类站点的字段落地对照

主键、易错字段、验收动作

站点类型	主键	易错字段	验收动作
供应商门户	证书编号	证书状态	状态词先去重报一遍再筛
招标平台	公告标题	截止日与过期	增量与过期两份行数要对得上
专利检索库	公开号	法律状态	摘页面原文，不许归纳
BOM 系统	物料号	单价与计价单位	单价列不许有裸数字

未找到与零是两回事：前者要补数据，后者要去核价格

图 12-4 四类站点的字段落地对照：主键、易错字段、验收动作

图 12-4 作者根据公开资料绘制。

翻车清单

翻车一：筛选条件写窄了，漏记录还不自知。症状是它报的条数比你手动搜的少，但每条数据都对。数据全对会让你放松警惕，这是这类翻车最阴的地方。补救：让它先把页面上该字段的全部取值去重报一遍，你确认了它再去筛。

翻车二：计价单位或币种不统一，表看着整齐其实不能比。症状是单价列混着个、套、千件三种单位，或者元跟美元混在一列。补救：把单位列设为必填，并在提示词里明确"不许把不同计价单位的单价放在一起比大小"。

翻车三：把"未找到"和"零"混为一谈。症状是单价为 0 的记录被写成了"未找到"，或者反过来。这两者意味着完全不同的动作：前者要去核价格，后者要去补数据。补救：在提示词里为这两类各定一个标记词，并在汇总报告里分别统计。

验收标准与变体

验收六条：

1. 主键唯一，全表无重复行。
2. 随机挑三行，照 URL 手动打开页面核对三个字段，三个全对。
3. 每个站点的条数与手动检索的条数一致，差一条都要查清原因。

4. "未找到"的数量与你预期的空字段数量大致吻合，一个都没有或特别多都要复查。
5. 单价列全部带币种和计价单位，无裸数字。
6. 汇总报告里的存疑项非空。它一项存疑都提不出来，说明它没在认真做。

变体一：竞品配置库抓取。把站点换成公开的竞品参数页，字段换成尺寸、功率、防护等级、认证。这类活的价值在横向对比，验收动作是所有记录的字段齐全度，缺一项这条就没法进对比表。

变体二：把抓取结果直接喂给下游分析。抓完不落地成报告，而是让 AI 接着做差异分析、到期预警、价格趋势。抓取只是上游，值钱的是下游那一步。

老兵说：抓取这件事，八成的功夫不在抓，在字段字典。你把字段写到字段级，它就能交出能用的表；你写"证书信息"四个字，它就只能交给你一堆你自己还得重拆的东西。

场景 33 卡住了怎么办：human-in-loop 接管 ★★★

一句话背景

企业内网里到处是 AI 过不去的地方：二次验证要手机收码、验证码要人眼认、确认弹窗要点"我已知悉"。号称全自动的工具在这些地方要么瞠点一气，要么直接挂掉。human-in-loop 的意思很简单：它处理不了就弹窗叫你，你点一下，它接着干。我这一趟跑五个门户，被叫了三次，每次我花不到半分钟。

名词小词典

human-in-loop：人在环里。流程不是全自动，也不是全手动，是 AI 跑到自己搞不定的地方把控制权交回给人，人处理完再交还给它。

二次验证（2FA）：登录时除了密码还要第二道验证，通常是手机短信码或动态口令。AI 看不到你的手机，所以过不去。

验证码：页面上那些让你认图、点选、拖滑块的关卡，设计的初衷就是挡机器。

确认弹窗：点某个按钮之后跳出来的"确定要执行此操作吗"。它在流程里出现，说明这一步有后果。

接管超时：你被叫了但没及时响应，会话等太久自己断了。断了之后前面的活不一定丢，但要重新把状态理清楚。

开工前准备

1. 场景 31 的环境已跑通，会话能开。
2. 你的手机放在手边，能收短信。这一步很土，但它决定了你被叫的时候要等多久。
3. 提前想清楚：哪些卡点你愿意接管，哪些你不愿意。愿意接管的留在流程里，不愿意的（比如银行页）从任务清单里划掉，别写进提示词。

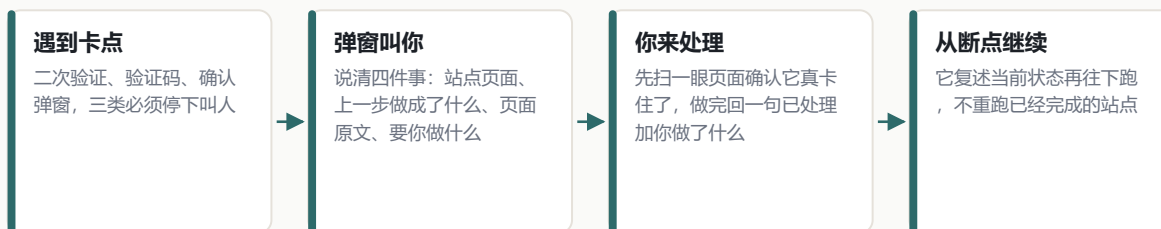
4. 一个重要提醒：跑之前把浏览器窗口放在你能一眼看到的位置。它在隔离窗口里弹窗，你没看见，就变成接管超时。

操作步骤

1. 在提示词里就把接管规矩写死：遇到哪三类卡点必须停下叫人，不许自行尝试绕过。
2. 开工前跟它约定：叫我的时候要说清楚四件事，卡在哪个站点、卡在哪一步、页面上显示的是什么、需要我做什么动作。
3. 跑起来，浏览器窗口保持可见。
4. 它弹窗叫你，你照它说的做。做之前自己扫一眼那个页面，确认它卡的地方是它说的那个地方。
5. 你处理完，回来告诉它"已处理，继续"。不要只说"好的"，要说清楚你做了什么。
6. 它接着往下跑，从断点继续，不从头重来。
7. 全部跑完后，让它把所有接管点列成一张表，这是你下次优化流程的输入。

三类卡点的接管流程

它停下来叫你，你点一下，它接着干



不许自行绕过，不许超时跳过，我五分钟不回你，你再叫我一次

图 12-5 三类卡点的接管流程：弹窗叫人、人处理、交还继续

图 12-5 作者根据公开资料绘制。

可复制提示词

把这段作为每一趟浏览器任务的开头固定贴上去：

【接管规矩】这趟任务里你会遇到三类你处理不了的情况。遇到任何一种，立刻停下当前动作，把控制权交还给我，不许自行尝试绕过，不许跳过，不许猜。

第一类：二次验证。页面要求输入短信验证码、动态口令、或扫码确认。

第二类：验证码。页面出现图形验证码、点选验证、滑块验证。

第三类：确认弹窗。点某个按钮之后跳出“确定”“我已知悉”“此操作不可撤销”这类提示。

【叫我的方式】弹窗叫我的时候，必须同时说清楚四件事，缺一件我来处理：

1. 卡在哪个站点、哪个页面，把页面标题原样贴给我；
2. 卡在哪一步，上一步你做成功了什么；
3. 页面上显示的提示文字，原样贴给我，不许你转述；
4. 需要我做的是什麼具体动作，比如“请在弹窗里点确认”或“请把收到的 6 位数字填进这个框”。

【等待纪律】叫我之后就停在那里等我，不许超时自动跳过，不许换一条路径重试，不许改用别的站点。我五分钟不回你，你再叫我一次。

【交还纪律】我告诉你“已处理”之后，你从断点继续，不要从头重来。继续之前先复述一遍：已完成哪些站点、当前在第几个、下一步要做什么。

处理完之后你回给它的那句话，也建议照这个格式写：

已处理：第三类确认弹窗，我点了“我已知悉”。
 已完成站点：A、B。当前在第 3 个站点 C 的第 2 页，已摘 18 条。
 请从断点继续，不要重跑 A 和 B，继续之前先复述一遍你的当前状态。

过程与判断点

一次合格的接管播报长这样，四件事齐全：

- > [需要你接管] 第二类：验证码
- > 1. 站点：某招标平台，页面标题“安全验证 - 某招标平台”
- > 2. 上一步已成功：关键词“执行器”检索完成，命中 23 条，已摘第 1 页 20 条
- > 3. 页面原文提示：“请按顺序点击图中所有的交通信号灯，完成后点击验证”
- > 4. 需要你做的：在隔离窗口里完成这次点选验证，完成后告诉我“已处理”
- > 我停在这里等你，不会超时跳过。

我盯三件事。

第一，它有没有把页面原文贴出来。贴原文说明它真的看到了那个页面；转述说明它可能只是在猜自己卡在哪。这一步看着啰嗦，实际上是你唯一能判断“它是不是真的卡住了”的依据。我有一次遇到它报“遇到验证码需要处理”，我去看了根本不是验证码，是页面还在加载。

第二，它有没有说清上一步做成了什么。这一条决定了它能不能从断点继续。它说不清上一步到哪，就说明它的状态是乱的，那你宁可让它重来这一段。

第三，它有没有老实等着。最危险的行为是我五分钟没回，它自己换了个路径绕过去了。绕过去的结果通常是拿到了一个空页面或者错误页面，而它以为自己成功了。

我这一趟五个门户，被叫了三次。一次是供应商门户的短信验证码，一次是上面这个招标平台的点选验证，一次是 BOM 系统里"导出超过 500 行需要确认"的弹窗。三次加起来我花了不到两分钟，而如果没有这个机制，它要么挂着等我回来发现，要么瞎点一气拿到一堆脏数据。

五门户三卡点的实测记录

一次真实跑批的整理，站点名已脱敏

卡点类型	出现在哪	需要你做什么	这次花了多久
二次验证	供应商门户，短信验证码	看手机收码，填进页面	每次约三十秒
验证码	招标平台，点选验证	在隔离窗口里完成点选	每次约三十秒
确认弹窗	BOM 系统，导出超过五百行	点确认，让导出继续	每次约三十秒

三个卡点加起来不到两分钟，没有这个机制它要么挂着等你，要么瞎点一气

图 12-6 五门户三卡点的实测记录：卡点类型、等待时长、处理方式

图 12-6 作者根据公开资料绘制，记录为一次真实跑批的整理，站点名已脱敏。

翻车清单

翻车一：窗口在后台，弹窗没看见，超时断线。症状是回来一看任务停了，或者更糟，它因为超时走了一条你没预期的路径。补救：开工前把隔离窗口放到可见位置，并且在提示词里写死"我五分钟不回你，你再叫我一次"，给它一次重复提醒而不是让它自己决定。

翻车二：你回了一句"好的"，它不知道你做了什么。症状是它继续跑，但从错误的断点开始，后面的数据全串位。补救：回复时固定用"已处理 + 卡点类型 + 我做的是什麼 + 当前进度"这个四段式，别偷懒打一个好字。

翻车三：它把正常的加载慢误报成卡住。症状是它频繁叫你，你去了发现页面好好的。根因是它没区分"页面还没加载完"和"页面在等我"。补救：要求它叫你之前先等页面加载完成一次，并把等待时的页面状态一起报出来。

验收标准与变体

验收四条：

1. 每一次接管，它都按四件事的 format 播报了，一件不缺。
2. 接管之后它从断点继续，没有重跑已完成的站点。
3. 全部跑完后它给了一张接管点清单，包含站点、卡点类型、你的处理方式三项。
4. 你手动核一遍：清单里的卡点数跟你实际被叫的次数一致。

变体一：把高频卡点前置。跑完几趟之后你会发现，某几个站点的验证码是每次都会遇到的。把它写成"开工前先让我手动过一次验证，再交给 AI"的两段式流程，比每次中途叫你要顺。

变体二：给团队做一张卡点地图。把你们常用的十几个系统各自在哪一步会卡、卡的是哪一类、需要谁来处理，整理成一张表。这张表比任何教程都值钱，因为它是你们公司独有的。

变体三：把不该接管的划出去。跑了几趟之后，把那些每次都卡、卡了你也不愿意处理的系统（通常是涉及资金和权限的）从自动化清单里删掉。省下来的不是时间，是风险。

老兵说：愿意叫你的工具，比号称全自动的工具靠谱。全自动这三个字在工程现场是个贬义词，因为它意味着出事的时候没人会在场。

场景 34 Computer Use: 让 AI 直接操作桌面软件 ★★

一句话背景

浏览器之外，你还有一大半时间花在桌面软件里：Blender、Adobe 系列、Autodesk 系列、Ableton 这类创作与设计工具，以及各种工程软件。2025 年 4 月，某头部厂商发布了一条让 AI 助手接入 8 大创意软件的能力，消息出来之后外包群里炸锅，一半人恐慌一半人兴奋。本节标 ★★，这套流程我跑通过，但没有完整留档；涉及的具体菜单、参数、版本号以你安装的为准。

名词小词典

Computer Use：让 AI 像人一样操作一台完整的电脑，看屏幕、动鼠标、敲键盘。它跟浏览器自动化的区别是，浏览器只管网页，Computer Use 管的是整合机器上的任何软件。

脚本驱动：AI 操作桌面软件的主流方式不是模拟鼠标点，而是写一段软件自己的脚本去调它。Blender 开放 Python 接口，Adobe 系列有 ExtendScript，Autodesk 系列也有自己的脚本接口。

MCP：一套让 AI 调用其他软件能力的通用插座标准。软件把自己的能力通过这个插座暴露出来，AI 就能调。

能力边界：它能干的事和它干不了的事之间的那条线。搞清楚边界比搞清楚功能重要，因为越界的部分会浪费你大量时间。

审美判断：这个方案好不好看、这个配色够不够高级。这一类判断 AI 做不了，也不该让它做。

开工前准备

1. 目标软件装好，正版授权。工业与创作软件没有捷径，盗版往往缺脚本接口。
2. 搞清楚这个软件支持什么脚本。Blender 是 Python，Adobe 系列是 ExtendScript，Autodesk 系列有自己的一套。你不用会写，但你得知道它们存在。
3. 在软件里手动跑一遍你要自动化的那个流程。手动都做不成，AI 也做不成。
4. 登记好 MCP 服务器或者脚本通道，用一句"列出可用工具"验一次握手。
5. 准备一个比你真实需求小十倍的任务当起步。别一上来就让它做个完整项目。

操作步骤

1. 手动做完一次最小流程。比如 Blender 里新建一个立方体并导出 GLB，或 Photoshop 里批量打开一张图并存成 PNG。
2. 验握手，确认工具列表里出现创建对象、设置材质、导出、批量处理这类能力。
3. 交给它一个最小任务，只看它的返回值长什么样。
4. 逐步加码，每加一级就加一条校核：看对象数、看文件大小、看导出的条数。
5. 单位这件事第一句就锁死。毫米、米、像素，说清楚并要求所有汇报值标注单位。
6. 渲染、导出这类有副作用的操作单独下指令，不要混在建模指令里。
7. 出图之后必须人眼看一遍。这一类产品最容易被忽悠，因为它好看。

桌面软件自动化的三级链路

AI 写脚本、软件执行、结果回读

AI 写脚本

主流做法不是模拟鼠标点，是写软件自己的脚本去调它

软件执行

先手动做通一遍最小流程，再验一次握手，然后逐步加码

结果回读

每一步都要回报对象数、顶点数、包围盒尺寸，出图之后必须人眼看一遍

单位这件事第一句就锁死：毫米、米、像素，所有汇报值一律标注单位

图 12-7 桌面软件自动化的三级链路：AI 写脚本、软件执行、结果回读

图 12-7 作者根据公开资料绘制，内容依据公开软件文档与社区案例整理。

可复制提示词

Blender 这一侧的最小起步任务：

你现在可以通过 blender 这台 MCP 服务器操作我电脑上打开的 Blender。

【任务】在一个新场景里建一个带圆角的方块：

尺寸 100 × 100 × 10 毫米，四条竖棱各倒 R5 圆角，材质给一个中灰的塑料感。

打一盏主灯和一盏补光，渲染一张 1920 × 1080 的图，

存为 D:\AI-DESK\render\block.png，再导出一份 D:\AI-DESK\render\block.glb。

【单位】Blender 内部场景单位与毫米不同，你负责换算，

所有汇报数字一律换算成毫米并标注单位。

【汇报】每一步做完都要汇报：场景里的对象数、每个对象的顶点数、包围盒尺寸。

【纪律】渲染分辨率、采样数这类参数一律先问我，不许自行取一个默认值。

任何失败把原始报错原样贴给我，停下来等我指示。

Adobe 这一侧的批量处理任务，这类活是它最稳的强项：

【任务】写一个 Photoshop 脚本，批量处理 D:\AI-DESK\in 目录下的所有图片：

打开每一张，把背景抠掉换成纯白色，保持原图尺寸与分辨率，

按原文件名保存为 PNG 到 D:\AI-DESK\out。

【汇报】脚本跑之前先告诉我这个目录下一共有多少个文件、分别是什么格式。

跑的时候每处理完 20 张报一次进度。

跑完之后报：成功多少张、失败多少张、失败的文件名与原始报错。

【纪律】不许覆盖原目录，一切都写到 out 目录。

不许为了跑通而降低分辨率或改色彩模式，这两项必须保持原样，

做不到就告诉我做不到，不许悄悄改。

Autodesk 与 Ableton 这一类，我没有完整跑通过，给的是通用原则而不是可照抄的提示词：先在软件里手动录一遍动作，看它自己生成的动作脚本长什么样，把那份脚本贴给 AI，让它照着改。先有脚本样本，再谈自动化，这是这类软件唯一可靠的上手路径。

过程与判断点

这一类集成能稳定做到的几件事：批量建简单对象、批量改材质、批量导出各类格式、用脚本做重复操作、用参数驱动成系列的变体。一句话概括，凡是重复五十次手会抽筋的活，它都能接。

Adobe 2024 年发布的一份设计生产力报告里提到，整合 AI 工具的设计师平均节省约 47% 的重复性工作时间。这个数我没有条件复核，引在这里只为说明一件事：省下来的是重复性工作，不是设计工作。

边界也很清楚，我按自己的判断列四条。

第一，审美判断它做不了。它能给你生成十个配色方案，但哪个方案能通过评审，判断的是你。把"这个配色够不够高级"翻译成代码这件事，目前没有人做得了，包括它。

第二，它看不见质量。曲面顺不顺、高光线断不断、声音混得好不好，这些要靠人眼和人耳加专业工具。它可以给你生成，但没法替你回答好不好。

第三，渲染和导出会无限迭代。好看没有终点，你让它改一次它就改一次。我的经验是给它一个硬性轮数上限，比如最多改三轮，并且先渲小尺寸确认构图，再渲最终分辨率。

第四，脚本越复杂调试成本涨得越快。二十行的脚本它一次能写对，两百行的可能要来回五六轮。所以起步任务一定要小。

桌面软件的能力边界

能稳定做到什么，做不到什么

能稳定做到

- **批量建简单对象**
批量改材质、批量导出各类格式
- **用脚本做重复操作**
凡是重复五十次手会抽筋的那类活，它都能接
- **参数驱动成系列变体**
一个产品五种配色三个视角，一次出完

做不到

- **审美判断**
哪个配色能通过评审，判断的人是你
- **质量好坏**
曲面顺不顺、高光线断不断，靠人眼加专业工具
- **迭代要有上限**
好看没有终点，给定轮数上限，先渲小尺寸确认构图
- **复杂脚本调试**
二十行一次能写对，两百行可能要来回五六轮

省下来的是重复性工作，不是设计工作

图 12-8 桌面软件的能力边界：能稳定做到什么、做不到什么

图 12-8 作者根据公开资料绘制，内容依据公开软件文档与社区案例整理。

翻车清单

翻车一：内部单位是米，你以为是毫米。症状是建出来的东西大一千倍或者小一千倍。补救：动手前第一句话就锁死单位，并要求所有汇报值标注单位。这一条我在第 10 章已经摔过一次，在桌面软件里摔得更狠，因为创作软件的默认单位更不统一。

翻车二：修改器或图层效果没应用，导出的东西跟屏幕上看到的不一样。症状是导出到别的软件里，圆角没了、滤镜没了、调整图层没了。补救：导出之前先让所有修改生效，导出之后在另一个软件里重新量一次尺寸、看一眼效果。

翻车三：追求完美，时间和算力就这么烧进去了。症状是一上午过去了，第七版图还没出来。补救：给定轮数上限，给定分辨率上限，并且把"先出小样确认构图"写成强制步骤。

验收标准与变体

验收五条：

1. 按你的产品定一处已知尺寸，对着成果量一次基准边，误差在你定的公差内。
2. 导出的文件拖回另一个软件重开，体量与本机一致，效果没有丢。
3. 批量任务的实际处理条数与你给的输入条数一致，失败项有明确清单。
4. 出图之后让一个不看过程的人说出这东西的材质手感或视觉重点，说不出来就是没做对。
5. 同一份提示词跑两遍，两次结果的几何应完全一致。跑不一致，说明它掺了带随机数的默认值。

变体一：批量切图与批量改尺寸。产品图要出五种尺寸、三个格式，人工一下午，脚本几分钟。这是我最推荐的第一步，因为它验收最容易：数文件个数、看尺寸对不对。

变体二：渲染队列半夜跑。设好参数让机器夜里自动渲完，早上来收图。价值不在省时间，在你睡觉的时候它在干活。

变体三：成系列的变体输出。一个产品五种配色、三个视角，用参数驱动一次出完。评审会上最常用，也最适合被它批量产出。

老兵说：这一类工具不是给纯小白准备的，它是给有一定基础的人准备的效率放大器。你不用会写代码，但你得知道图层是什么、蒙版是什么，不然你连怎么跟它提需求都不知道。

场景 35 远程操控：人在家盯台架 ★★★

一句话背景

台架耐久试验、环境舱循环、NVH 阶次分析，一跑就是几十上百小时，必须有人盯，但盯的过程又极其枯燥。旧办法是派人值班，或者我周末开车几百公里跑过去。新办法是让台架上位机一直开着，我人在家里或者出差的酒店，用手机连过去看数据、拉曲线、改参数。这一节讲 UU 远程这套东西怎么装、怎么注册，以及我跑通过三个真实画面。

名词小词典

上位机：试验台架旁边那台控制试验、采数据的 Windows 电脑。它是被控端，一直待在实验室。

主控端 / 被控端：被控端是你连过去的那台机器，主控端是你连过去用的设备，笔记本、手机、平板都行。

远程开机：机器关机之后还能被网络唤醒，基于 WOL 协议，实测三十秒内能醒，不用人过去按电源。

隐私防窥模式：你远程控制的时候，被控端那块屏幕是黑的，既不干扰旁人也不暴露内容。

长效验证码：默认的动态码一次一换，长期连某台设备太折腾。自定义一个 8 到 16 位的长效码可以一直用。

开工前准备

1. 被控端：试验台架的上位机，Windows 系统，装好 UU 远程被控端。
2. 主控端：你的笔记本、手机或平板，装好 UU 远程主控端。它支持 Windows、macOS、Android、iOS，跨平台这块不用挑。
3. 一个手机号用于注册，走验证码登录，不是那种要填一堆资料的重型注册。
4. 被控端设为开机自启动，并同时配好远程开机。只配自启没配远程开机，断电重启之后软件起不来；只配远程开机没配自启，机器起来了软件没跑。两个都要有。
5. 跨运营商的链路先实测一次。办公网是电信、供应商厂区是联通，第一次连要打洞，打洞失败会走中继，延迟会涨。

操作步骤

第一步，装被控端。去 UU 远程官网按设备类型下载，Windows 安装包大约 84 MB。装完之后第一件事是把它设为开机自启动。

第二步，装主控端并配好外设。装完可以直接用手机或平板操作。通勤路上用的话，把常用设备提前收藏，点一下就连，不用每次翻列表。

第三步，注册。走手机号加验证码，装完主控端第一次启动会提示登录。同一个账号跨端自动识别：你手机上装一次、电脑上装一次，登录同一个账号之后，设备列表会把两端都列出来，在家的手机上能直接看到"办公室那台电脑""试验基地 3 号上位机"，点一下就连。

第四步，配远程开机与长效验证码。要长期连的设备设一个 8 到 16 位的自定义验证码，配一个自定义生效时间，免得忘关。

第五步，跨运营商实测一次。连上之后看延迟：局域网 5 到 15 毫秒，跨运营商公网 15 到 30 毫秒，5G 环境下手机控 Windows 大约 18 到 25 毫秒。超过了这个范围就要查是不是走了中继。

远程值守链路

上位机被控端、跨网连接、人在家主控端



开机自启与远程开机两个都要配，少配一个断电就白跑一趟现场

图 12-9 远程值守链路：上位机被控端、跨网连接、人在家主控端

图 12-9 作者根据公开资料绘制，仅表示链路关系，不涉及任何真实台架编号。

三个真实画面

画面一：台架上位机跑耐久，人在家拉 NVH 曲线。

这是我最常用的一个。上位机 24 小时不关机，跑 12 小时、24 小时、48 小时的耐久循环。周一早上到办公室之前，我先在家里用手机连上那台上位机，看过去一晚上跑的数据有没有异常。

发现某个频段阶次在涨，我直接在这台电脑上打开 NVH 分析软件，把曲线拉出来，先自己看一遍，再决定要不要现场重跑。跑数据、看曲线、决定下一步，三件事在家就能做完。

文件传输不限速不限大小在这里特别好用。试验数据动辄几个 G，用 U 盘或网盘来回拷光等待就是半小时；实测 1.2 GB 的文件约四分钟传完，且支持断点续传。

画面二：远程值守 DV 试验，现场异常有人接。

DV 试验的另一个痛点是人在现场但人不在电脑前。跑环境舱耐久，工程师要在旁边看温度、看振动、看电流，中间去接个电话、去开个会，回来数据已经漏了一段。

我们的配置是：被控端是环境舱上位机，主控端是值班同事的笔记本，放在休息区。同事不用一直坐在上位机边上，隔三差五扫一眼；真出异常时，用剪贴板同步加文件传输，把关键段的曲线截下来丢进当天试验报告。再配远程开机，上位机定期重启后自己醒过来等连接，实测三十秒内唤醒。

画面三：跟供应商跨地域对问题。

供应商在别的城市，你的样件在他厂里跑测试，指标出问题了要看，一趟高铁六小时。做法是三步：双方都装好（供应商那边走免登录协助也行，装个精简版双击就跑，不用注册账号，把生成的设备 ID 加动态验证码发给你即可）；被控端开启隐私防窥模式，黑屏半秒内生效，你在他实验室电脑上看数据时那块屏是黑的，结束后自动锁屏；直接在远端跑分析软件。

传统流程是"我发邮件说指标 X 在 Y 工况超标，麻烦复测"，对方回"已复测，数据见附件"，解压一看还有 Z 工况有问题，再发一封，往返三四天。远程连过去，两小时能走完这个循环。

可复制提示词

远程连上之后，AI 的用武之地在这里。第一份提示词用来生成值守交接班清单，跑长试验之前先出一份：

【任务】给我这次 DV 试验写一份夜间值守交接班清单，一页纸，能直接打印贴在显示器边上。

【试验信息】

试验名：某微电机 1000 小时耐久 + 环境舱温度循环

被试样品：6 件，编号 S01 至 S06

监控项：母线电流、转速、壳体温度、环境舱温度、振动加速度

采样周期：1 秒

【清单内容】

1. 每个监控项的正常范围与报警阈值，写成表格，阈值这一列留空让我填；
2. 出现哪几种情况必须立刻叫醒值班人，逐条列出，每条写清判断依据；
3. 出现哪几种情况可以先记录、等白班处理，逐条列出；
4. 交接班的六项固定动作，每项一句话，不许写"检查设备状态"这种没法执行的空话；
5. 一份异常记录表的表头，字段包括：时间、样品编号、现象、当时的四个监控值、已采取动作、是否通知某人。

【纪律】阈值我不给你的不许你自己编，一律留空并标"待填"。

每条判断依据都要能用一个数字表达，不许写"异常升高"这种没法判断的话。

第二份提示词，是你在远端上位机上让 AI 帮你读数据的写法：

【任务】我现在远程连着试验台架的上位机，你来帮我读数据。

【数据位置】D:\BENCH\RUN_2025_1214\ 目录下，每 10 分钟落一个 CSV，文件名带时间戳。今天跑的是第 3 轮循环。

【你要做的】

1. 先列出这个目录下的文件数与时间跨度，确认没有断采（相邻文件时间间隔超过 15 分钟就算断）；
2. 读最近 24 小时的数据，对 6 个样品分别算：母线电流均值、最大值、波动范围，壳体温度均值与峰值；
3. 把母线电流的 24 小时曲线画出来，6 个样品一张图，存为 D:\BENCH\RUN_2025_1214\report\current_24h.png；
4. 找出任一监控项超出我给的阈值的时段，列出：样品编号、超出项、超出的起止时间、峰值、持续时长；
5. 横向比 6 个样品，标出跑得最不一样的那一个，说明你的判断依据。

【纪律】断采的时段不许插值补齐，要在报告里单独标出来。

所有数字都要能追到具体文件的具体行，我要抽查三条。

不许用“基本正常”“略有升高”这类话，要么给数字，要么说没数据。

过程与判断点

远程这条链路，我盯三个数字。

第一，延迟。局域网 5 到 15 毫秒，跨运营商公网 15 到 30 毫秒，5G 下手机控 Windows 大约 18 到 25 毫秒。看曲线、翻数据、点按钮这些操作，在这个延迟下跟本地手感差距不大。超过 50 毫秒你就会有明显的拖影感，这时候要查是不是走了中继。

第二，弱网抗性。官方与第三方测评的公开数据是 3% 丢包环境下卡顿率低于 1%，这个数我没有条件复核，但我在高铁和酒店 Wi-Fi 下都跑过，断连很少。换一个新环境还是要实测一次。

第三，文件传输实际速度。你自己的环境要跑一次才知道。这个数决定了你是“数据回传要单开一个环节”还是“顺手就传完了”。

还有一条不是数字的判断：被控端能不能自己醒过来。断电重启之后机器起不来，你人在家里干瞪眼。开机自启加远程开机必须一起配，并且要在真的断一次电之后验一次。我第一次只配了自启，断电后连不上，白跑一趟现场。

三个远程画面各自的验收点

延迟、唤醒、数据完整，各盯一项

画面一 耐久值守

人在家拉曲线，盯文件传输速度，实测一个 1.2 GB 的文件约四分钟传完

画面二 现场有人接

值班同事隔三差五扫一眼，盯远程开机，断电后三十秒内自己醒

画面三 跨地域对问题

被控端开隐私防窥，黑屏半秒内生效，盯延迟有没有走中继

局域网 5 到 15 毫秒，跨运营商公网 15 到 30 毫秒，5G 下手机控桌面大约 18 到 25 毫秒

图 12-10 三个远程画面各自的验收点：延迟、唤醒、数据完整

图 12-10 作者根据公开资料绘制。

翻车清单

翻车一：只配了开机自启，没配远程开机。症状是断电重启之后怎么都连不上。补救：两个都配，并且在一次真实断电之后验一次通电到能连上的总时长。

翻车二：主控端在开放工位，远端画面被旁人看到。症状不明显，但风险实打实。被控端开隐私防窥防的是被控端被旁人看到，你主控端的屏幕同样显示远端画面。补救：敏感的试验在独立区域做主控端，或者用手机加物理遮挡。

翻车三：默认动态验证码，长期连的设备每次都要重新问一遍。症状是高频场景下特别折腾，最后你索性不连了。补救：给长期设备设一个 8 到 16 位的自定义长效验证码，配上自定义生效时间，避免忘关。

验收标准与变体

验收五条：

1. 断一次电，从通电到能远程连上的总时长在两分钟内，且软件已自动启动。
2. 跨运营商连一次，延迟在可接受范围内（参考上面的三段区间），曲线上的小幅波动能看清。
3. 传一个 1 GB 以上的文件，计时，并中断一次网络验证断点续传能续上。
4. 开隐私防窥之后，从被控端那侧看屏幕确实是黑的，结束后自动锁屏。

5. 在远端连着走完一圈"看数据、改参数、重新采数", 全程不联系现场人员。

变体一：异地供应商联合调试。把画面三的做法固定下来，跟两三家核心供应商各建一条远程通道。这件事的收益不在省差旅费，在把三四天的往返压成两小时。

变体二：把上位机变成一台 24 小时在线的数据岗。上位机一直开着，配一个定时脚本，每天凌晨自动把前一天的数据打包、跑一遍异常扫描、把结果发到你的邮箱。你早上打开手机就看到昨夜的结论。

老兵说：这一套工具真正的价值，是把"人必须到现场"这条默认假设拆掉。工具是明面上的成本，假设是暗地里的成本，行业里真正贵的那部分，常常是后者。

场景 36 安全边界：什么能交给 AI 碰，什么不能 ★★★

一句话背景

前面五个场景都在教你怎么把权限交出去。这一节教你怎么把权限收回来。交出去容易，收回来难，因为权限一旦交出去，你往往不知道它到底碰了什么。这一节给你一张系统分级清单表，三类分级，每类举工程现场的真实系统例子。这是本章最该被撕下来贴墙的一页。

名词小词典

登录态：你在这个系统里已经登录的那个状态。AI 借你的登录态干活，意味着在这个系统眼里，它的每一次操作都是你干的。

SSO（单点登录）：一次登录之后，公司里所有系统都认你。它是权限的总闸门，也是最高危的位置。

令牌 / cookie：系统发给浏览器的一小段凭据。谁拿到它，谁就是你。

最小权限：只给它干这件事必需的权限，多一分都不给。这是所有权限分配的第一原则。

审计留痕：谁在什么时候做了什么，系统有记录可查。类比：楼道里的监控，出事了能倒回去看。不留痕的系统不配被自动化。

开工前准备

1. 一张你自己公司里所有系统的清单。不用一次列全，先列你每周都要登的二十个。
2. 每个系统的三个属性：里面有什么数据、碰坏了会怎样、有没有操作留痕。
3. 你们公司的信息安全规矩。没有明文规矩的，找信息部门问一次，别自己拍脑袋。
4. 一支红笔。分级这件事，最后是要划线的，不是要讨论的。

操作步骤

1. 把你手上的系统全部列出来，一行一个，先不分级。

- 2. 对每个系统问三个问题：里面有没有个人信息或财务数据？操作错了能不能一键恢复？系统本身有没有操作日志？
- 3. 三个问题里任何一个指向"有个人信息或财务数据""不能一键恢复""没有日志"，就往高危侧挪。
- 4. 按可碰、需审批、绝对不碰三类分完，写成一张表，打印出来。
- 5. 把这张表给信息部门看一遍，他们改过的版本才是最终版。
- 6. 把"绝对不碰"那一类写进每一份提示词开头，作为固定红线段。每季度重看一次。



图 12-11 系统分级三问：有没有敏感数据、能不能恢复、有没有日志

图 12-11 作者根据公开资料绘制。

系统分级清单表

下面这张表按工程现场的真实系统举例，三类分级，每类四到五个例子。你照着这个格式换成你自己公司的系统。

分级	系统举例	可以做什么	硬条件
可碰	公开招标平台	检索、摘字段、导出 CSV	只摘不提交，不点任何下单或投标按钮
可碰	专利检索库	按申请人检索、批量摘法律状态	只读，导出条数设上限

分级	系统举例	可以做什么	硬条件
可碰	供应商产品目录页	摘参数、比价、下载公开规格书	不下单、不询价、不留联系记录
可碰	内部 BOM 查询页	按物料号查单价与交期	只读账号，用只查询权限的子账号
可碰	试验台架上位机（只读数据目录）	读 CSV、画曲线、出异常报告	不给改试验参数的权限，改参数人自己来
需审批	内部 BOM 系统（写入侧）	改物料状态、补录交期	每次事前审批，事后逐条核，留操作人
需审批	PLM 系统的文档上传	上传 AI 生成的图纸与报告	上传必须带版本号，且由责任人账号执行
需审批	供应商门户的报价提交	填报价单	提交前人工逐项复核，提交动作由人点
需审批	试验台架上位机（参数写入侧）	改转速、改负载、改循环次数	双人确认，改完立刻回读确认值
需审批	内部知识库写权限	写入整理好的笔记	先入草稿区，人工过一遍再发布
绝对不碰	企业 SSO 登录页	无	AI 不介入，包括不读、不截图
绝对不碰	银行与企业网银	无	全线禁止，不登录、不截图、不读页面
绝对不碰	密码管理器	无	全线禁止，任何形式的凭据读取都不允许
绝对不碰	生产制造执行系统（下发工位）	无	下发给产线的指令只能由产线责任人发
绝对不碰	人事与薪酬系统	无	涉及个人身份信息，全线禁止

表里有几条我要单独解释。

试验台架的读和写是两回事。读数据目录是可碰，改试验参数是需审批。场景 35 里让你远程改参数，那是你本人在远程改，不是 AI 在改。让 AI 改试验参数这件事我到现在也不做，改错了会毁掉一件跑了几百小时的样件。

BOM 系统的读和写也是两回事。查询页可碰，写入侧需审批。同一套系统权限不同分级就不同，所以分级要按权限分，不按系统分。

生产制造执行系统为什么绝对不碰。它下发的东西直接进工位，工人照着做，这里的错误不是数据错误，是实物错误。

可复制提示词

第一份用来让 AI 帮你梳理自己公司的系统清单，注意它只做整理和提问，不做判断：

【任务】帮我把手上的系统做一次安全分级梳理。你只做整理和提问，最终的分级结论由我来定，你替我定下来的地方要单独标出来。

【输入】下面是我每周都要登录的系统清单，一行一个：
(此处粘贴你的系统清单，一行一个)

【你要做的】

1. 对每个系统，向我提三个问题，我来回答：
这个系统里有没有个人身份信息或财务数据？
在这个系统里操作错了，能不能一键恢复？
这个系统本身有没有操作日志，能不能查到谁在什么时候做了什么？
2. 收集完我的回答后，按三类输出一张表：可碰、需审批、绝对不碰。
每一行的列包括：系统名、分级、可以做什么、硬条件、我的判断依据。
3. 单独列出一栏"我信息不足、需要你去问信息安全部门的系统"，逐个写清楚缺什么信息。
4. 单独列出一栏"分级有争议的系统"，说明争议点在哪。

【纪律】三个问题的回答里，只要有一个指向"有个人信息或财务数据" "不能一键恢复" "没有日志"，就往高危侧放，不许往低危侧放。
分级这件事宁可严一档，不许松一档。
任何你不确定的，一律进"需要去问"那一栏，不许自己猜一个级别。

第二份是每一趟浏览器任务开工前固定贴的红线段，改都不用改：

【红线：开工前必读，违反任何一条立刻停止】

1. 不许访问、打开、截图这几类页面：银行与企业网银、企业单点登录页、密码管理器。
2. 不许读取、导出、转发任何 cookie、令牌、密码、密钥。
3. 不许在任何页面上执行提交、下单、付款、审批、删除这几类动作。
需要做的，把要提交的内容准备好交给我，由我来点。
4. 不许把本次抓到的数据写到我指定的输出目录以外的任何位置。
5. 不许把本次抓到的数据上传到任何外部服务。
6. 每一趟任务结束必须执行会话关闭动作，并把关闭结果贴给我。
7. 遇到任何你不确定的页面，先问我这个页面能不能碰，我说了算。

过程与判断点

分级做完之后，我盯三件事。

第一，"可碰"那一类里有没有藏着写入动作。很多系统的查询页上带着一个不起眼的"保存"或"提交"按钮。它以为是查询，顺手点了保存，一条错误数据就进了系统。所以每一行的"硬条件"那一列必须写死"只搞不提交"这类动作约束，不能只写系统名。

第二，"需审批"那一类有没有真的走审批。表做出来容易，执行难。我的做法是把这类动作从 AI 的提示词里彻底删掉：它只负责把要提交的内容准备好，最后那一下由我点。这一条执行起来最省事，也最有效。

第三，"绝对不碰"那一类有没有被写进每一份提示词。上面那段红线段存成固定片段，每次开工前贴上去。它不是装饰，它是你事后唯一能说"我交代过"的证据。

还有一条经验：账号要分。给 AI 用的账号跟你自己日常用的账号最好是两个。用只读权限的子账号跑自动化，就算它跑错了，损失也限制在那个子账号的权限范围内。这件事做起来只要半天。



图 12-12 三道防线：分级清单、固定红线、只读子账号

图 12-12 作者根据公开资料绘制。

翻车清单

翻车一：按系统分级，没按权限分级。症状是同一套 BOM 系统，查询侧天天在跑，某天它在写入侧留了一条错数据。补救：分级表的最小单位改成"系统 + 权限"，一个系统的读写两行分开写。

翻车二：分级表做完就再没看过。症状是半年后某系统升级了权限模型，你的表还停在原地。补救：定一个季度复查的提醒，复查动作就是把三个问题再问一遍。

翻车三：用你自己的主账号跑自动化。症状是它跑错了，影响范围覆盖你账号下的全部权限。补救：单独开一个只读子账号专供自动化使用，权限范围按最小权限原则给。这件事的成本是半天，收益是封住最坏情况。

验收标准与变体

验收五条：

- 1. 分级表覆盖了你自已每周登录的全部系统，一个不漏。
- 2. 每一行的"硬条件"列写的是动作约束（能做什么、不能做什么），不是系统描述。

3. 读写权限不同的系统，在表里占两行。
4. 这张表给信息部门看过，并按他们的意见改过。
5. 自动化的活全部用独立的只读子账号在跑，主账号不参与。

变体一：给团队做一张共用表。一个人做是个人习惯，一个组做是团队规矩。把这张表放进新人入职材料，比任何安全培训都管用，因为它具体到系统名。

变体二：把红线做成提示词片段。上面那段红线段存成 WorkBuddy 里的一个片段，每次开工前一条命令贴上去，不用每次手打。同一张三问还可以反过来用：评估供应商给你开的门户账号该给什么权限，你要提供的不是“能不能用”，是“给它什么权限最合适”。

老兵说：会用的人把重复劳动甩给 AI，自己腾出手做判断和决策；不会用的人把登录态全盘交出去，等于把家门钥匙塞给陌生人。会不会用，看的就是你能不能守住这张表上那几条线。

小白词典

登录态：你在某个网站或系统里已经登录的那个状态。浏览器里存着一小段凭据，网站每次见到它就认你是你。类比：你已经刷过门禁进到楼里，不用每层都再刷一次。

浏览器扩展：装在浏览器里的小插件，能看见页面、能替你点击填表。类比：给浏览器加的一只机械手。

隔离窗口：AI 专门新开的浏览器窗口，它只在这个窗口里干活，你正在用的窗口纹丝不动。类比：厨房里另开一个灶台，你原来那锅照样在煮。

本机回环：一台电脑自己跟自己通信用的内部地址，数据不出本机。类比：同一栋楼两间办公室之间的内线电话。

human-in-loop：人在环里。AI 跑到自己搞不定的地方把控制权交回给人，人处理完再交还。类比：自动驾驶遇到施工路段把方向盘还给司机。

二次验证（2FA）：登录时除了密码还要第二道验证，通常是短信码或动态口令。类比：进小区刷了门禁，进单元门还要再输一次密码。

Computer Use：让 AI 像人一样操作一整台电脑，看屏幕、动鼠标、敲键盘。类比：不只是给你打电话，是坐在你的椅子上替你干活。

上位机：试验台架旁边那台控制试验、采数据的电脑。类比：实验室里那个不回家的值班员。

主控端 / 被控端：被控端是你要连过去的那台机器，主控端是你连过去用的设备。类比：遥控器是主控端，电视是被控端。

WOL 远程开机：机器关机之后还能被网络唤醒。类比：不用走到机器前面按电源，喊一声它就醒。

隐私防窥：远程控制时被控端屏幕是黑的，旁人看不到。类比：你在这头看，那头的屏幕拉上了帘子。

SSO（单点登录）：一次登录之后公司里所有系统都认你。它是权限的总闸门。类比：一把万能钥匙，丢了全楼门禁都失守。

最小权限：只给干这件事必需的权限，多一分都不给。类比：给访客开临时门禁，只开他要去的那一层。

你的岗位怎么用

采购与供应商管理：场景 31 和场景 32 是给你的。三家供应商各查二十多项资质和报价、数据散在五个门户，人肉两小时，它四十分钟。你最该看的是场景 32 里“单价必须带币种和计价单位”那一条，我在这上面吃过亏。

试验工程师：场景 35 直接照抄。台架上位机装被控端、设开机自启、配远程开机，你人在家就能拉 NVH 曲线。验收动作是断一次电看它能不能自己醒过来，再传一个 1 GB 的文件实测速度。

质量工程师：场景 33 和场景 36 是给你的。每一次接管都是一次人工介入记录，整理成卡点地图，评审会上能说明哪些环节有人把关过。场景 36 那张分级表，你比谁都清楚哪些系统碰坏了不能恢复。

设计工程师：场景 34 跟你相关，但请先读完第 10 章。桌面软件这条路上，AI 能接的是重复五十次手会抽筋的那类活，审美和质量判断仍在你手里。

项目经理：盯场景 36 那张表。它不是安全文档，是责任划分文档：哪些能交出去、哪些要审批、哪些绝对不碰，写清楚了团队才敢放手干。

信息安全与 IT：场景 36 那三个问题可以直接搬进权限申请流程，另外请推动一件事：给自动化单独开只读子账号。

本章交付物

一张属于你自己的系统分级清单表，加一段固定的开工红线提示词。

做法分两步。第一步，用场景 36 里那份梳理提示词，把你每周登录的系统全部分成可碰、需审批、绝对不碰三类，每一行的硬条件列写动作约束，读写权限不同的系统占两行，做完给信息部门改一遍。第二步，把那段红线提示词存成一个固定片段，每次让 AI 碰浏览器之前贴上去。

第一次验收的活：挑一个“可碰”类的系统，用只读子账号跑一次场景 31 的完整流程，开会话、抓取、出汇总、关会话四步走完。跑通的那天，你就有资格把第二个系统加进表里。

本章金句

卡住 AI 的从来不是它不够聪明，是你没让它碰到你的鼠标。

愿意叫你的工具，比号称全自动的工具靠谱。全自动这三个字在工程现场是个贬义词，因为它意味着出事的时候没人会在场。

抓取这件事八成的功夫不在抓，在字段字典。你写到字段级，它交出来的是能用的表。

试验台架的读和写是两回事：读数据可以交给它，改参数得你本人来，因为改错了毁掉的是一件跑了几百小时的样件。

会用的人把重复劳动甩给 AI，不会用的人把登录态全盘交出去。区别就在那张分级表守不守得住。

（下一章：设计审核与风险。AI 已经能碰到你的电脑了，下一章我们解决更难的那件事：怎么判断它交上来的东西能不能签字。）

第 13 章

设计审核与风险：让 AI 当那个挑刺的人

评审会上最值钱的一句话从来不是"同意"，而是"等一下，这里我说不通"。AI 不能替你签字，但它能在你推开会议室门之前，先把这句"等一下"喊出来。

2023 年 11 月的一个周三，上午九点零五分，三号评审室。

那是我做微电机研发的第 9 年，第一次以设计负责人身份主持一场 DR4 评审。DR4 是我们内部量产前的最后一道设计闸门，过不了就往下走不了。

材料一共 38 页：一套总成图纸、一份轴向尺寸链计算书、一份 DFMEA、三份试验报告、一张开口项跟踪表，提前一天发给了到会的 7 个人。

会开到第 17 分钟，质量部的老张把材料翻到第 9 页，指着一行数问我："输出轴轴向间隙，图纸上写 0.05 到 0.35，这个上限你怎么保证？"

我答得挺快，说是五个环累加出来的极值法结果。

他又往下追了一句："那第三个环，端盖轴承室的深度公差 0.12，这个数哪来的？是供应商的长期过程能力，还是你自己拍的？"

那句话我到现在还记得。我沉默了大概五秒钟。答案是后者。我是照着上一代产品的图纸抄的，抄的时候没人问我为什么，我也没有问上一代为什么。

会上我没当场承认，只说了一句回头核实，然后把它记进开口项。评审结果是通过，挂 14 条开口项，其中 3 条两周内必须关闭。

真正让我后背发凉的不是挨了这一下。散会之后我把那份 DFMEA 从头翻到尾，发现失效模式表里写了 43 行，没有一行提到装配顺序错误导致的轴承预压异常。

而这个失效在前一年的样机阶段真的发生过一次，处理结论写在一封邮件里，那封邮件就躺在我自己的收件箱，谁也没把它搬进 FMEA。

那 14 条开口项，我花了两个礼拜才关完。一条项目的节拍，就这样被我自己的材料拖住了半个月。

三年之后，我换了一种做法：开会之前，先把材料整包丢给 AI，让它按我最苛刻的口径先挑一遍。它挑出来的东西不是都对，虚警大概四成，但它每次都能挑出那么三四条我自己绝对想不到的事。

这一章讲的就是这套打法。六个场景，全部围着同一个目标：让你带着问题清单走进会议室，而不是带着自信走进。

同样的材料，两条路

带着自信进去，还是带着清单进去

带着自信进去

材料发下去就等开会

三十八页发给七个人，读到后半段眼睛已经麻了

最容易被问住的那一处

第三个环的公差哪来的，当场答不上来

散会之后才发现

失效模式表四十三行，没有一行提到装配顺序错

带着清单进去

开会前先让 AI 挑一遍

整包材料丢进去，二十五分钟出一张待确认清单

每一条都带证据位置

文件名加页码加原文片段，标不出来的那一条直接划掉

先体验一次被问住

带着问题清单进门，而不是带着自信进门

AI 挑出的问题要人复核，AI 没挑出的问题不代表不存在

图 13-0 同样的材料，两条路：带着自信进去，带着清单进去

图 13-0 作者根据公开资料绘制。

本章六个场景速览

场景	场景名	人工耗时	AI 耗时	可信度
37	评审材料预审：开会前先让 AI 挑一遍刺	约 3 小时	约 25 分钟	★★★ 实测
38	图纸与数模检查清单：从经验清单到自动核对	约 2 小时	约 15 分钟	★★★ 实测
39	失效模式联想：DFMEA 辅助	约 2 人天	约 40 分钟	★★ 跑通
40	公差与尺寸链校核：让 AI 帮你算，你来判断	约 4 小时	约 20 分钟	★★★ 实测
41	变更影响分析：改一个尺寸会牵动哪些件	约 1.5 人天	约 30 分钟	★★ 跑通
42	经验教训库：把过去的问题变成可检索的资产	建库约 3 人天	单次检索约 5 分钟	★★ 跑通

六个场景有一条共同的红线，先摆在前面。

【本章铁律】AI 挑出的问题要人复核，AI 没挑出的问题不代表不存在。评审签字的人是你，担责的人是你。这一条在本章任何场景里都不打折。

这句话的意思是：AI 预审完交给你的那张单子，叫"待确认清单"，不叫"结论清单"。单子上每一条，你要么关掉它，要么写清楚为什么关掉。

风险不会因为跑了一遍 AI 就消失，它只是从你没想到的地方，挪到了你没复核的地方。

场景 37 评审材料预审：开会前先让 AI 挑一遍刺 ★★★

一句话背景

一场正式的设计评审，材料往往是几十页的一整包：图纸、计算书、试验报告、DFMEA、BOM、开口项表。人工通读一遍再交叉核对，保守两到三小时，而且读到后半段眼睛已经麻了，最容易漏的恰恰是最后那几页。这个场景把整包材料交给 AI 先跑一遍预审，25 分钟左右出一张带证据位置的待确认清单。

名词小词典

设计评审 (Design Review)：设计推进到某个节点时，把材料摊开给一群相关的人看，让他们挑问题、决定能不能往下走。常见节点代号是 DR1 到 DR5，各家叫法不同，逻辑一致。

交付物清单 (Deliverable List)：这道闸门规定你必须交出哪几份东西。缺一份，会就开不成。

开口项 (Open Issue)：评审上被提出来、当场没有关掉的问题。它必须有人、有日期、有关闭准则，否则等于没提。

关闭准则 (Exit Criteria)：这一轮算通过的条件。比如"零条致命开口项，一般开口项不超过 5 条"。准则要在看材料之前定死。

虚警 (False Positive)：AI 报了一条问题，你去核了，发现它是错的。虚警不可怕，按虚警去改设计才可怕。

开工前准备

1. 把这一轮要评审的全部材料收进同一个文件夹，路径不要有中文和空格。我用的是 `D:\review\dr4\`。
2. 材料格式尽量统一为 PDF、DOCX、XLSX、PPTX、Markdown 这几类。扫描件做成图片 PDF 的话，先做一次文字识别，否则读不到字。
3. 手边准备一份你公司的交付物清单。没有的话，直接用下面提示词里那份通用清单。
4. 一个评审包的总页数控制在 200 页以内。超了就分批喂，先喂图纸与计算书，再喂试验报告。
5. WorkBuddy 新建一个对话，模式选 Agent。别接着聊别的项目的旧对话往下用，上下文串味会让它把别的项目的结论搬过来。
6. 事前想清楚让它用谁的视角挑。同一个材料包，用质量工程师视角跑一遍和用工艺工程师视角跑一遍，挑出来的东西完全不一样。

操作步骤

- 1. 先让它把材料编一遍目，不许它发表任何意见。这一步用来确认它真的读进去了，而不是只看文件名在猜。
- 2. 给它一份交付物清单，逐项核对缺哪一份。缺件是一票否决项，先处理。
- 3. 让它按"挑刺者"的角色跑第一轮：只找矛盾和缺失，不许提改进建议。两件事混在一起会稀释注意力。
- 4. 第一轮出结果后，你自己先过滤虚警。这一步别省，也别指望它自己过滤。
- 5. 挑它能拿到真东西的那个视角跑第二轮专项深挖。做微电机项目时，我常让它只盯"低温工况"这一个轴。
- 6. 让它把每一条都标上证据位置：文件名加页码加原文片段。标不出来的那一条直接划掉。
- 7. 让它按"是否可能导致评审不通过"排序，输出成一份 Markdown 文件。
- 8. 带着这份清单去开准备会，或者自己先关一轮能关的。



【任务】对我放在 D:\review\dr4\ 目录下的这一整包设计评审材料做预审。
你现在是一名来参加评审的审核人，不是写材料的本人。你的立场是挑毛病。

【第一轮：只编目，不许评价】

把这个目录里的文件逐个列出来，做成一张表：文件名、页数或行数、文档类型、你能判断出来的主题。列完之后告诉我你一共读到了多少页。

这一轮不许提任何意见，不许说“建议”。我只想确认你真的把每一份都读进去了。

【第二轮：核交付物清单】

以下是本次评审规定的交付物清单，逐项核对并在表里标注“有 / 缺 / 存疑”：

1. 总成图纸与全部零件图，含图号清单
 2. 尺寸链计算书
 3. DFMEA 表
 4. 试验大纲与完整的试验报告
 5. BOM
 6. 特殊特性清单
 7. 开口项跟踪表，含责任人、日期、关闭准则
 8. 上一轮评审开口项的关闭证据
- 缺项直接写“缺”，不要用“建议补充”这种软话。

第二段，挑刺本体。这一段是整个场景的核心：

【第三轮：挑矛盾与缺失】

以挑刺者的身份把整包材料通读一遍，只找下面四类问题，每条都要给证据：

- A 类：硬矛盾。材料内部两处说法不一致。例如计算书里某个数与图纸标注不一致、BOM 里的零件数与图纸明细表不一致、试验报告的样品状态与大纲要求不一致。
- B 类：无来源的关键数。任何一个影响安全、法规或功能的数值，材料里没交代来源。比如某个公差、某个安全系数、某个试验样本的选取依据。判定写法必须是“材料未给出来源”，不许写成“该值不合理”。
- C 类：口径缺失。该写没写。例如 DFMEA 某行给了探测度分数却没有对应的探测手段、试验报告有结论却没有原始数据附件、开口项没有关闭准则或责任人。
- D 类：和历史断了的地方。本轮材料与上一轮材料、与已结案问题的结论之间不一致。

【输出格式】每条一行，格式固定，不许改：

编号 | 类别 | 一句话问题描述 | 涉及文件与页码 | 原文引用（40 字以内） | 把握（高/中/低）

【纪律】

1. 找不到证据位置的问题不许写。没页可引的猜测一律扔掉。
2. 不许提任何改进建议，这一轮你只负责找问题。
3. 不许用“值得关注”“建议考虑”这种含糊说法，每条必须是能验证的陈述句。
4. 全部列完后统计各类条数，并明确列出你这一轮没能覆盖的部分，例如扫描件读不到字、某份表格太长只看了前半段。没覆盖的必须如实交待。
5. 输出写进 D:\review\dr4\预审清单.md。

第三段，换视角做专项深挖：

【第四轮：单视角深挖】

换一个身份重读整包：你是装配车间的工艺工程师，明天就要按这套图纸做首件。

只回答三个问题：

1. 图面上有没有你装不出来的地方，或者需要额外做工装才能装的地方？
2. 有没有装配顺序会影响最终精度的位置，而图纸没给出装配顺序要求？
3. 有没有需要专用检具才能测的尺寸，而这份材料里没提到检具方案？

每条同样要带文件名与页码。答不上来的问题直接写“材料未提供”，不许编。

过程与判断点

我在一次真实的 DR4 材料包上跑出来的结果，是这样一张表：

类别	它报的条数	我复核后留下	其中真正致命
A 类硬矛盾	7	5	2
B 类无来源关键数	14	9	1
C 类口径缺失	16	8	0
D 类和历史断了	4	1	1
合计	41	23	4

41 条里留下 23 条，虚警率 44%。这个量级我跑过三次，都在四成到五成之间。先对这个数字有心理准备，别因为它错了一半就判定这活不值。

真正值钱的是那 4 条致命项。挑两条拆给你看。

第一条是 A 类。它指出计算书第 9 页里端盖轴承室深度写的是 12.0 正负 0.12，而总成图纸第 3 页同一处标的是正负 0.08。两个数不一样，图纸和算书打架。这一条我自己开会前重读了两遍都没发现，因为人眼扫数字的时候会自动往期望值上靠。

第二条是 D 类。它指出这一版的 DFMEA 删掉了上一版里的一行，内容是“输出端油脂低温稠化导致启动扭矩超差”，而删除动作没有出现在任何变更记录里。

我当时不信，回去翻了上一版文件，它是对的。上一轮我确实删了，理由是“已经在第一轮试验里通过”，但通过的是零下 30 度，而这一轮的技术要求写的是零下 40 度。

证据位置这一条我要多说一句。提示词里硬性要求每条必须给文件名加页码，效果非常直接：几条没谱的猜测因为它标不出页码，被它自己毙掉了，剩下 41 条全部可追溯。标不出处的意见，等于没听见。

带把握列这一列也有用。我复核的顺序是按把握从低到高走，先收拾“低”的那几条虚警，因为它们占的时间最少，砍掉之后单子立刻短一半。

预审意见的分类漏斗

报出来、能证实、真正致命

类别	它报的条数	我复核后留下	其中真正致命
A 类 硬矛盾	7	5	2
B 类 无来源关键数	14	9	1
C 类 口径缺失	16	8	0
D 类 和历史断了	4	1	1
合计	41	23	4

四十一里留下二十三，虚警率四成到五成，先对这个数字有心理准备

图 13-2 41 条预审意见的分类漏斗：报出来、能证实、真正致命

图 13-2 作者根据公开资料绘制。

翻车清单

翻车一：它开始提改进建议，把真问题淹了。症状是报告里一半是"建议进一步完善""建议增加说明"这种话。根因是提示词里没封死这一条。补救：明确写"不许提改进建议"，并要求每条必须是能验证的陈述句。

翻车二：材料是扫描件，它照着文件名编了一段看上去很像的报告。症状是它引用的原文你在 PDF 里搜不到。根因是图片 PDF 没有文字层，它只能靠上下文猜。补救：让它先报每个文件读到了多少个字符，字符数明显偏少的那几份先做文字识别再重跑。

翻车三：你让它在材料之外给你补了一个数。症状是报告里出现"按常见做法，该公差通常取 0.1"这种话，然后你把它抄进了正式材料。补救：凡是要写进正式文件的数，来源只能是标准表、实测数据、供应商过程能力数据三者之一，来源不在三者里的必须留白并写"待核"。

验收标准与变体

验收五条：

1. 清单里每一条都能在本包材料的某一页找到原话，逐条抽查五条做溯源，找不到就删掉整批里所有同类型的条目。
2. 缺件核对的结论与你手数的结果完全一致，尤其是"缺"的那几项。
3. 至少找出一条它标为"把握高"、经你核实却是错的意见。找不出来，说明你复核得不够狠。

4. 你自己复核一轮的耗时不超过 25 分钟。超过说明它在灌水，回头收紧提示词里的输出格式。
5. 单子里每一条拿到准备会上都能当着人说出来，不用回头再翻材料。

变体一：给供应商的材料做预审。供应商交上来的一整包认可资料，用同一套提示词跑一遍，重点看 A 类硬矛盾。这一类最值钱，因为对方的材料往往是好几个人拼出来的，内部打架的概率比你自己写的还高。

变体二：给三年前的老材料做预审。拿一个已量产项目的归档包跑一遍，你会意外地找出若干处当年没人追问过的地方。这一招拿来练新工程师的手感特别好。

变体三：让它从对话记录里直接出开口项表。在预览了一大轮问题之后，指定输出格式（编号、问题描述、责任人、日期、关闭准则五列），一句话就能导出一张能直接贴进跟踪系统的表。

老兵说：预审不是为了让它替你把材料批合格，是为了让你在进会议室之前先体验一次被人问住的感觉。它挑十四条，里面有九条能用，剩下五条是它替你练的反应速度。

场景 38 图纸与数模检查清单：从经验清单到自动核对 ★★★

一句话背景

老工程师脑子里都有一张检查清单：标题栏填没填、比例对不对、材料写没写、形位公差给没给基准、粗糙度有没有、焊接符号有没有技术条件。这张清单靠人过一遍，一张 A3 图大约七八分钟，一套二十几张图的包要两个小时，而且下午四点以后过图，出错率是上午的三倍。这个场景教你把这张老清单交给 AI 自动过一遍，同时说清楚一件更要紧的事：它能查什么，查不了什么。

名词小词典

GD&T（几何尺寸与公差）：一套用符号规定形状位置允许的偏差范围的语言。比如一个框格里写着垂直度符号加 0.05 加基准字母 A，意思是这个面相对于 A 面，垂直方向的偏差不能超过 0.05。

基准（Datum）：测量时的起始参照。没有基准就提垂直度，相当于没说参照哪儿垂直。

公差带：尺寸被允许变动的那段范围。标注 20 正负 0.1，公差带就是 0.2 宽。

粗糙度（Ra）：表面微观凹凸的平均高度。它直接影响摩擦、磨损和密封。

投影角：第一视角和第三视角，图纸把立体画成平面的两种约定。国内常用第一视角，混用会让工人把图看反。

明细表（BOM）：这张图对应的产品由哪些零件、各几个组成的表。

开工前准备

1. 待检的图纸，PDF 优先，DOCX 和原生工程图文件也可以。混着放也行，但要放在同一个目录里。
2. 一份你公司的图纸检查清单。没有的话，用下面提示词里那份通用清单先跑，再把你们的行补进去。

3. 三维数模如果要一起查，需要先在第 10 章那套 MCP 桥里把数模打开。本节主线只查图纸，数模那一段放到最后一条变体里讲。
4. 明确这张图的零件用什么工艺。压铸、注塑、机加各自的关注项完全不同，这条不告诉 AI，它就只能查通用项。
5. 一支笔和一台打印机。自动查完之后那一道人眼不能省，最后一遍一定在纸上走。

操作步骤

1. 让它先逐张把标题栏字段读出来，这一步不许它发表任何评价，只报原文。
2. 逐项过通用检查项，把缺漏记进它返回的表里。格式定为每张图一行、每个检查项一列，格子里写“有 / 缺 / 不适用 / 存疑”四选一。
3. 针对第一轮标为“缺”的项，让它回头再找一遍全文证据，看是不是写在别处以另一种形式出现了。这一步能砍掉一半误报。
4. 对关键尺寸做一轮高级检查：每个功能尺寸有没有对应的公差、每个形位公差框有没有基准、同一个基准在不同视图里指代是否一致。
5. 让它把格式问题和技术问题分两张表输出。格式表直接发给画图的人改，技术表留给你自己判断。
6. 打印一张最典型的图，对着实物或数模自己再走一遍，然后跟它的单子逐行对，把两边不一致的地方圈出来，这就是你下一版清单要补的行。
7. 最后一轮让 AI 把这一次的结果写成检查报告文件归档，下次同一个零件复查时拿来比对。

一张图纸过两遍

第一遍逐项过，第二遍回头找证据

- 1 第一遍：逐项过通用清单**
每张图一行，每项一列，格子里只写有、缺、不适用、存疑
- 2 第二遍：对缺项找证据**
回头全文再找一遍，结论改成确实缺失或在别处出现
- 3 分两张表输出**
格式表直接交给画图的人改，技术表留给你自己判断
- 4 打印出来自己再走一遍**
对着实物或数模走一遍，跟它的单子逐行对，补全下一版清单

它标缺的八成是读漏了值得复核，它标存疑的八成是真问题值得深挖

图 13-3 一张图纸过两遍

图 13-3 作者根据公开资料绘制。

可复制提示词

整段复制，工艺改成你自己零件的：

【任务】对 D:\draw\dr4\ 目录下的全部图纸做一次完整性检查。
这批零件的工艺是：铝合金压铸加数控精加工（换成你自己的工艺）。

【第一步：报数】先告诉我你读到了几份文件、每份多少页，然后逐张列出标题栏字段原文：
图名、图号、材料、比例、投影角、设计、审核、日期、版本。
这一轮不许评价，只报原文。缺的字段写“（空）”，不许写“未提供”这种模糊词。

【第二步：通用完整性检查】逐项过下面这张清单，输出一张表，
每张图一行，每项一列，格子里只允许写：有 / 缺 / 不适用 / 存疑。
检查项：

1. 标题栏字段是否齐全（图名、图号、材料、比例、投影角、设计、审核、日期、版本）
2. 每个视图是否标注了比例，比例是否与图纸标注一致
3. 是否有未标注公差的线性尺寸（通用公差条款覆盖的除外，要指明条款在哪）
4. 所有功能尺寸是否有对应的公差带
5. 每个形位公差框是否给出了基准；没有基准的要单独标出来
6. 基准符号是否在同一张图的不同视图中重复出现且指代同一要素
7. 是否有表面粗糙度标注，Ra 值是否合理到 $\pm$ 一位有效数字
8. 技术要求条目是否齐全：未注公差、未注圆角、去毛刺、表面处理、热处理
9. 明细表行数与图上序号数是否一致
10. 是否有仅在某一视图出现、却在整个图上没有任何关联标注的尺寸要素

【第三步：证据复核】对第二步标为“缺”的每一项，回头再全文找一遍，
确认它是真的缺，还是以别的写法出现在技术要求里。复核后把结论改为“确实缺失”或“在别处出现”。
在别处出现的，必须给出那句话的原文和你找到它的位置。

【第四步：分表输出】把结果分成两张表：

表 A 是格式与记录类问题，这些可以直接交给画图的人改。

表 B 是需要设计判断的问题，这些交给我。

两张表都写进 D:\draw\dr4\图审报告.md，每张表后面附你自己的不确定项清单。

【纪律】不许评价任何设计意图，不许说某个尺寸“看起来偏大”这类话。
你只回答“有没有写”和“写得是否自治”，不回答“应不应该这么设计”。

过程与判断点

我在一套 27 张图的减速箱体包上跑过，它交回来的报告开头长这样：

- > 共读到 6 份 PDF，合计 27 页。
- > [图号 6 张] 标题栏：图名齐、图号齐、材料齐、比例齐、投影角齐、
> 设计齐、审核（空）、日期齐、版本齐。审核一栏 6 张全空。
- > [检查项 1] 6 张均“有”。
- > [检查项 3] 图 3、图 5 各有一处线性尺寸未标公差，且通用公差条款指向的技术要求第 2 条
> 在这两张图上写的是见同系列图号 xxx，跨图引用，判定“存疑”。
- > [检查项 5] 图 4 有两个形位公差框未给基准，分别是 $\phi 18$ 孔的位置度与端面圆跳动。
- > [检查项 9] 图 1 明细表 21 行，图上序号 22 个，不一致。

这一段报告里我最看重三行。

第一行是审核栏全空。六张图，一张没签。这是再典型不过的格式问题，也恰恰是最容易翻的地方，因为签字这件事没法由画图的人自己补。AI 扫一遍两分钟；人扫六张图，有时候就是看不见。

第二行是通用公差条款指向了技术要求第 2 条，而那条写的是见另一个图号，属于跨图引用。这里它判的是"存疑"而不是"缺"，分寸拿捏得对。跨图引用本身不违规，但它是评审桌上最常被揪住的位置。

第三行是明细表 21 行对序号 22 个。数字对不上就必须查清楚，它常常对应一颗螺钉被漏掉了，或者某个零件被重复编号。这条我当场核了，是两个同样的卡簧被编了两个号。

我的判断经验是：它标"缺"的条目，八成是它读漏了，值得复核；它标"存疑"的条目，八成是真问题，值得深挖。存疑是它自己也拿不准但又不好意思跳过的地方，那正是人的注意力该落的地方。

AI 能查什么，查不了什么

一张三分表，谁下结论写在最后一列

类别	具体例子	谁下结论
能查：格式与完整性	标题栏字段、比例、图号与文件名是否一致、明细表行数	AI 直接判，画图的人照着改
有条件能查：自治性	公差有没有写、基准有没有给、粗糙度值在规格里有没有列	AI 挑出来，人确认它不是看错行
查不了：设计意图	这个公差为什么给零点零五、这个基准体系是不是工艺基准	只能是人，永远是人

你只回答有没有写和写得是否自治，不回答应不应该这么设计

图 13-4 AI 能查什么、查不了什么：一张三分表

图 13-4 作者根据公开资料绘制。

这篇必须写清的边界：AI 查不了什么

这是本节最重要的一张表，建议打印：

类别	具体例子	谁下结论
能查：格式与完整性	标题栏字段、比例、图号与文件名是否一致、明细表行数、技术条件条目数	AI 直接判，画图的人照着改

类别	具体例子	谁下结论
有条件能查：自治性	公差有没有写、基准有没有给、粗糙度值是否在规格里有列	AI 挑出来，人确认它不是看错行
查不了：设计意图	这个公差为什么给 0.05、这个倒角方向合理吗、这个基准体系是不是工艺基准	只能是人，永远是人

第三类再说透一点。AI 可能会说某处公差偏严，因为它的语料里同类特征通常给到另一个量级。但它不知道这个孔要装的是一支编码器，编码器对安装面的跳动有它自己的要求。这些信息写在设计师的脑子里和上几次会议的纪要里，不在这包材料里，它就无从判断。

所以提示词最后那句纪律，不许评价设计意图，不是客气话。一旦允许它评价，它给出的判断就会和资料强混在一起，你很难再把两者分开。

翻车清单

翻车一：它把跨图引用的内容判成缺失。症状是缺一行出现一大片，你跑去一看，都写在别处。补救：提示词里必须加第三步证据复核，让它自己回头再找一遍并给出原文位置。我这套流程加了这一步之后，缺的条目从 19 条降到 6 条。

翻车二：多张图放在一个 PDF 里，它把后几张图读串了。症状是报告里出现某张图上根本不存在的尺寸。补救：一个 PDF 放一张图，文件名带图号；实在改不了，就在提示词里要求它先报每页左上角读到的图号再开查。

翻车三：它开始替你做设计判断。症状是报告里出现“建议此处增加加强筋”这类话。补救：见一条删一条，并且立刻在提示词里补一句“你只回答有没有写和写得是否自治，不回答应该怎么设计”。

验收标准与变体

验收五条：

1. 任选三张图，把它的表和图逐项对一遍，错判率应当低于你自己徒手过图的错判率。跑三次就有数。
2. 所有标缺的项已经走过一轮证据复核，缺的数量是可信的。
3. 明细表行数、图上序号数、BOM 三者一致的结论，必须你自己手数一遍再信它。
4. 那份报告交到画图的人手上，他不需要你翻译就能照着改。
5. 你自己照着纸质图复核一遍的时间，应当明显短于从零开局。

变体一：拿它给供应商提交的外来图做入厂检查。外来图的投影角、标题栏字段、公差体系这套尺度常常和我们内部不一致，这张通用清单刚好能当入厂的第一道筛子。

变体二：按工艺换清单。把检查项里的第 8 行技术要求换成你那条产品工艺的专项：压铸看壁厚与拔模斜度，注塑看缩水与熔接线位置，机加看装夹基准与退刀槽。

变体三：给新人当练手工具。让新来的工程师先自己做一遍清单，再跟 AI 结果对，两边不一致的地方就是他要补的课。

老兵说：图纸检查这活，AI 干的是那种明明一眼能看出来、却总没人看的事。它替不了你判断公差给得对不对，但它能拦住你六张图都没签名。

场景 39 失效模式联想：DFMEA 辅助 ★★

一句话背景

一份 DFMEA 表，两个工程师从头写到能通过评审，大约三天。最难的部分不是填表，是想：这个零件到底能怎么坏。

AI 在这里的定位非常清楚。它做联想和补全，把你可能想漏的失效模式先铺出来；评分那三栏归人填，它不能替你打分。

本场景标 ★★：这套流程我跑通过，也用在真实项目上，但没留下完整的留档证据。

名词小词典

DFMEA（设计失效模式与后果分析）：一份表，逐条列出这个产品可能怎么坏、坏了会怎么样、为什么会坏、现在有什么措施挡着、危险有多大。类比：动手前的排雷图。

失效模式：零件坏的方式。比如齿轮断齿、密封圈老化、磁钢脱落。

失效后果：坏了之后用户感受到的是什么。比如执行器行程衰减、系统报故障码、低温下起动无力。

失效起因：为什么会走到这一步。比如润滑脂选型不当、热处理硬度不足、装配顺序约定缺失。

严重度 S：后果有多严重，1 到 10 分。涉及安全与法规的通常打到 9 或 10。

发生度 O：这个起因出现的可能性，1 到 10 分。

探测度 D：在量产之前，现有手段能把这个问题查出来的能力，1 到 10 分，数字越大越查不出来。

RPN：风险优先数，S、O、D 三个数相乘。

它只用来排先后顺序，不代表风险高低。

新版 AIAG-VDA 手册改用措施优先级替代 RPN 做行动排序。

AP（措施优先级）：高、中、低三档，用来定先做哪件事。

开工前准备

先把功能清单写出来。这个产品要干的事逐条列清楚，输出扭矩、行程、响应速度、寿命次数，一样一行。列不出来就先别往下走。

再要一份结构分解，把这个产品拆到零件那一层，有 BOM 就够了。

然后准备三样东西：公司正在用的评分标准表、历史故障单和 8D 报告的存放目录、以及 WorkBuddy 里新建好的一个 Agent 模式对话。

评分标准表这一样不能省。没有它，AI 打出来的分数用的就是它自己发明的标准。

操作步骤

第一步，把功能清单和结构分解喂进去，让它用自己的话复述一遍。你确认它没理解歪再往下走，这一步十几秒钟。

第二步，让它按结构逐个部件输出失效模式。每条写成某个部件干不了某项功能的形式。不许写成零件损坏这种追不下去的短语。

第三步，给每条失效模式补至少两个独立起因。并标出受影响的下游部件。起因要写到能追溯到某个具体物理机理的程度。写成设计不当这种话，等于什么也没说。第四步，拿这一稿跟你们脑子里的老经验对一遍。你把手上那些吃过亏的旧事填进去。

第五步才轮到打分。S、O、D 三栏由人填，AI 一栏都不许碰。这一步的纪律我在下面单独讲透。

第六步，把评分之后的定稿表再喂回给它，让它换一个身份重读，只找逻辑打架的地方，不许改任何一个分数。

第七步，按你们公司 DFMEA 模板的列名和列宽导出成 XLSX。别让它另起一套表头，否则你回去还得手工对一遍。

第八步，留档。把这一版连同喂进去的功能清单、BOM、以及你用的提示词原文一起存进项目目录，文件名带日期和版本号。半年后有人问这行当初为什么删，你得答得上来。

DFMEA 辅助的四步

它做联想和补全，评分那三栏归人填

复述确认

功能清单与结构分解喂进去，它复述一遍，你确认没理解歪

失效模式联想

句式固定：某零件在某工况下失去某项功能

后果与起因

后果分两层写，起因至少两个且写到物理机理那一层

评分与留档

S、O、D 三栏由人填，定稿连同提示词原文一起存

历史对照单独开一张表：历史上出过而联想里没有的，逐条列出来

图 13-5 DFMEA 四步流程示意图

图 13-5 作者根据公开资料绘制。

可复制提示词

第一段，联想与补全。整段复制，路径换成你自己的：

【任务】帮我补一份 DFMEA 的失效模式部分。你现在是失效分析工程师，不是设计工程师。你的活是联想和补全，不是评分。

【输入】

1. 功能清单: D:\fmea\功能清单.md
2. 结构分解与 BOM: D:\fmea\BOM.xlsx
3. 历史故障单与 8D 报告: D:\fmea\历史问题\ (该目录下全部文件)

【第零步：复述】先用你自己的话复述这个产品要实现的每一项功能，以及它拆到零件层的结构。复述完停下来等我确认，不要直接往下跑。

【第一步：失效模式联想】按结构逐层往下走，对每个零件回答一个问题：

这个零件在实现哪一项功能时，会以什么方式失去这项功能？

输出句式固定为“某零件 + 在某工况下 + 失去某项功能”。例如：

“输出轴深沟球轴承在零下 40 度冷启动工况下失去提供稳定回转支撑的功能”。

不许写“零件损坏”“性能失效”“出现异常”这类追不下去的词。

每条后面必须标出它对应哪一项功能，对不上功能的条目直接删掉。

【第二步：失效后果】对每一条失效模式分两层写：

第一层是这个零件往下还会发生什么，写成物理事实；

第二层是终端用户感受到什么，写成用户能听懂的话。

只影响内部、用户感知不到的，在这一行标“仅内部”。

【第三步：失效起因】每条失效模式至少给两个相互独立的起因。

起因要写到物理机理那一层，例如：

“润滑脂基础油在零下 40 度下析出导致稠化，启动摩擦扭矩上升”。

不许写“设计不当”“选型错误”“工艺问题”这类不可追溯的词。

每个起因后面标注它会牵动的下游零件。

【第四步：历史对照】把 D:\fmea\历史问题\ 下的故障单与 8D 报告全部读一遍。

凡是历史上真实发生过、而你在前三步里没有联想到的失效，单独开一张表：

历史问题来源文件名 | 失效描述原文 | 对应到本次哪一条 | 若无对应写“未覆盖”。

历史上出过的事没进表，这一轮就白跑了。

【第五步：现行控制】对每一个起因，只列你现在能拿出证据的措施：

已有的设计要求、已有的试验项、已有的检具。

拿不出证据的写“无”，不许写“待定”“后续补充”。

【纪律】

1. 不许输出任何 S、O、D 的数值，不许算 RPN，不许给 AP 档位。
评分由我来做，你一栏都不许碰。
2. 不许评价我的设计，不许出现“建议加强”“建议优化”。
3. 拿不准的条目照写，但在备注列标“联想，无证据”，去留由我判断。
4. 输出写进 D:\fmea\DFMEA_联想稿.xlsx，列名用我给你的模板，不许自己改表头。

第二段，评分之后的逻辑自查。这一段是我自己加的，原版 DFMEA 流程里没有：

【第六步：逻辑自查】下面这张表是我评分之后的定稿。

你换一个身份重读：你是来评审这份 DFMEA 的质量工程师。

只找逻辑打架的地方，不许评价任何一个分数给得对不对。

1. 严重度打了 9 或 10 的行，有没有对应的现行预防措施？没有的列出来。
2. 探测度打了 1 到 3 的行，有没有对应的试验或检具？在表内查不到出处的列出来。
3. 同一条失效起因在表里重复出现但写法不同的，列出来并给出两处的行号。
4. 上一版里有、这一版被删掉的行，逐条列出，

并说明你是否在变更记录里找到了删除依据。找不到就写“未见删除依据”。

每条必须给行号。答不上来的写“表内未提供”。任何分数都不许改。

过程与判断点

2025 年 4 月，我在一个带位置反馈的小型执行器项目上跑过这套流程。那个产品由直流电机、两级塑料斜齿轮、输出轴、端盖和一块带霍尔元件的线路板组成。

我先喂进去 9 条功能清单和一份 47 行的 BOM，让它跑了第一轮联想。它交回来 68 条失效模式。

我核完之后的状态是这样：

环节	它给的条数	我留下的	说明
失效模式联想	68	31	37 条是通用项，与这个产品的具体结构无关
失效后果分层	31	29	两条后果写得比后果还像起因，我改了
失效起因	74	52	22 条写到了“选型不当”这种层，退回重写
历史对照	11	9	9 条全部是我自己想不起来的
最终进表		31	其中 9 条来自历史，22 条来自联想

37 条通用项里，最典型的一类是这样写的：“端盖在长期振动工况下失去固定功能”。这句话没有错，但它对铝压铸端盖、塑料端盖、钢板端盖都成立，放到任何一个产品上都挑不出毛病。这种条目唯一的用处是占行数，我一条没留。

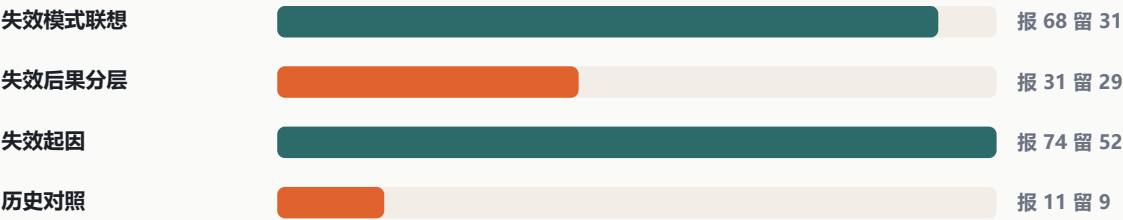
真正让我觉得这 40 分钟花得值的是历史对照那 9 条。

其中一条是“霍尔线路板焊点在温度循环后开裂，导致位置信号跳变”。这个失效在两年前的一个同类项目上出过，8D 报告躺在我自己的归档目录里。我在这次的联想稿里没有想到它，AI 也没有想到，是它在读历史目录时把那封 8D 捞出来的。

另一条更有意思：“输出端油脂在长期高温存放后结焦，导致第一次启动扭矩超上限”。这一条它标的是“联想，无证据”。我去问了做润滑的同事，他说这个基础油确实不耐 120 度长期存放。后来这一条打进了表，起因改成“油脂基础油高温氧化结焦”。

联想条数的四道漏斗

它给的条数，我留下的条数



历史对照那九条全部是我自己想不起来的，这四十分钟花得值

图 13-6 68 条联想里最后留下 31 条：通用项大半，历史项全部留用

图 13-6 作者根据公开资料绘制。

评分这件事，我把话说死：严重度、发生度、探测度、RPN、AP，全部由人定，AI 一栏都不许填。理由有三条，一条比一条硬。

第一，分标准是你们公司自己的。同样是"用户能感知但不影响安全"，有的公司打 6，有的公司打 7。AI 用的是它在语料里见过的某种平均口径，跟你那份评分标准表对不上。

第二，探测度取决于你现在手上真有什么手段。它不知道你们实验室有没有低温箱，也不知道那个检具到底是买了还是还在走采购。它填出来的探测度，是在替你幻想你拥有的能力。

第三，也是最要紧的一条：分数一旦落到纸上，就变成了一份被签过字的判断。判断可以委托给工具去准备，不能委托给工具去作出。出了事，评审纪要上签字的是你的名字。

我在提示词的纪律里写了四条，第一条就是不许输出分数。这一条不是流程洁癖，是责任边界。

翻车清单

翻车一：它偷偷把分打了。症状是返回的表里直接带上了 S、O、D、RPN 四列，数字还都挺像回事。根因是你没在第一段就封死，或者它在第二段复述时把你上一轮的旧表当模板继承了。补救：见一列删一列，然后重发第一段提示词，在纪律里加一句"输出表中不得出现 S、O、D、RPN、AP 任何一列，出现即整表作废重跑"。

翻车二：历史目录没喂进去，联想出来的全是通用项。症状是 68 条里你挑不出一条是这个产品独有的。根因是它只能基于结构做通用联想，没有你们淌过的那些坑。补救：把历史故障单、8D 报告、历次试验失效记录的目录整个挂上去，并在第四步强制它单开一张历史对照表。

翻车三：失效模式写成了"零件损坏"这种粒度。症状是同一行里出现"齿轮损坏""轴承失效""线路板异常"，往下追问起因时它只能给出"强度不足"。根因是第一步的句式约束没生效。补救：在第一步里把输出句式写成模板并要求逐条对标，不符合句式的直接判不合格退回。

验收标准与变体

验收五条：

1. 最终进表的每一条失效模式，都能回答"它对应哪一项功能"。答不上来的全删。
2. 表里至少有三分之一的内容来自你自己的历史问题目录。低于这个比例，说明历史没喂够。
3. 随机抽五条起因，每一条都能一路追问到某个物理机理。追问两层就断的，退回重写。
4. S、O、D 三栏里没有任何一个数是 AI 给的。这一条不满足，整份表作废。
5. 这张表拿到评审会上，被问到任何一行你都能说出它从哪来。说不出来的那一行当场划掉。

变体一：拿它给老产品的 DFMEA 做增补。找一个已经量产三四年、DFMEA 从写完就没动过的项目，把历史故障单喂进去跑一遍，你会得到一张"这些年实际出过、但表里没有"的清单。这张清单拿去跟售后索赔数据对，往往能对上。

变体二：给供应商的零件做失效联想。外购件进厂前让 AI 先按你的功能要求联想一遍，把输出发给供应商，让他在自己的 PFMEA 里逐条回应。这一步能把"你的要求"和"他的理解"之间的沟提前挖出来。

变体三：反向用它查现有 DFMEA 的空洞行。把已经定稿的表喂进去，只问一句：哪些行的起因一栏写的是不可追溯的套话。它会把"设计不当""工艺波动"这类行全挑出来，这些行在评审上最容易被打回。

老兵说：DFMEA 里最贵的那几行，从来不是想出来的，是吃过亏记下来的。AI 能替你想，替不了你吃过亏，所以你要先把亏喂给它，它才能替你想得像样。分数那一栏，谁签字谁填。

场景 40 公差与尺寸链校核：让 AI 帮你算，你来判断 ★★★

一句话背景

一条六环的轴向尺寸链，手工算一遍极值法、再算一遍统计法，然后回头检查有没有把某个数抄错、有没有把某个环的符号判反，我大概要花四个小时。这活最折磨人的地方不是算术，是怀疑自己。

这个场景让 AI 替你列环、判增减、做加减、出表，你只负责三件事：判封闭环、给每个环的公差来源、最后对一次总数。20 分钟出结果，剩下的时间你去跟工艺和供应商谈哪个环能收。

名词小词典

尺寸链：一串首尾相接的尺寸，最后一个尺寸是被前面几个尺寸挤出来的结果。前面任何一个动了，最后一个跟着动。

封闭环：这条链里那个你没法直接标出来、只能由其他尺寸凑出来的尺寸。它通常就是你要保证的那段间隙或过盈。

组成环：除封闭环之外的每一个环。

增环与减环：这个环变大时，封闭环跟着变大就是增环，跟着变小就是减环。判反一个符号，整条链就废了。

极值法：假设所有组成环同时取到最不利的极限值，算出封闭环的两个极端。结果保守，意味着零件的加工成本高。

统计法（RSS）：假设各环相互独立且呈正态分布，用平方和开根号来算。结果宽松，成本低，前提不成立就不能用。

敏感度：某个组成环的公差变动，对封闭环总变动贡献的比例。收紧敏感度最高的那一个环，最划算。

开工前准备

1. 一张能看清全部相关尺寸的图纸或数模截图，把你要保证的那段间隙圈出来。
2. 每个组成环的公称尺寸与上下偏差，手工抄进一个表格文件。别让 AI 自己从图上读数，它在图注密集的地方读串过。
3. 给每个环加一列工艺来源：机加、压铸、注塑、冲压、外购标准件。这一列很关键，它决定你能不能套统计法。
4. 一个写死的封闭环目标区间。没有目标，它算出来也不知道该判合格还是不合格。
5. 查一下你们公司有没有规定必须用哪种算法。有的写在设计标准里，先查再动手。
6. WorkBuddy 新建对话，Agent 模式，别接旧项目的上下文。

操作步骤

1. 先让它把图纸上相关的尺寸逐条列出来，并说出它认为的封闭环是哪一个，然后停下来等你确认。
2. 你确认封闭环之后再让它往下判增减环。顺序不能反，封闭环判错了后面全错。
3. 让它逐个环标增或减，并给一句理由。理由写不出来的标"待判"，不许猜。
4. 让它用极值法算一遍，中间过程完整写出来，不许只给最后两个数。
5. 你随机抽两个环手算一遍符号，确认它没判反。这一步三十秒，省你半天。
6. 让它用统计法再算一遍，并明确列出统计法成立的三个前提，你逐条对着工艺判断。
7. 让它输出敏感度排序，指出收紧哪一个环最有效、收多少才够。
8. 最后一步归你：拿着敏感度表去跟工艺和供应商谈。能收多少、成本多少，这张表说了不算，谈下来才算。

尺寸链校核的八步

判封闭环、判增减、算两遍、排敏感度



封闭环判错了后面全错，顺序不能反

图 13-7 尺寸链校核的八步：判封闭环、判增减、算两遍、排敏感度

图 13-7 作者根据公开资料绘制。

可复制提示词

整段复制，路径和工艺换成你自己的：

【任务】帮我校核一条轴向尺寸链。你是尺寸链计算员，不是设计工程师。你负责列环、判增减、做加减、出表。判断哪个环该收紧，由我来做。

【输入】

1. 各环尺寸与公差：D:\dim\尺寸链输入.xlsx
2. 相关图纸：D:\dim\总成图.pdf
3. 目标：输出轴轴向间隙要求在 0.05 到 0.35 之间（换成你自己的目标区间）

【第一步：列环并判封闭环】把图纸上与这段间隙相关的尺寸逐条列出来，做成一张表：环号 | 尺寸名称 | 公称值 | 上偏差 | 下偏差 | 工艺来源 | 出处（图号加视图）。列完之后，明确指出你认为的封闭环是哪一个，并说明理由。然后停下来，等我确认封闭环，不要继续往下算。

【第二步：判增减环】我确认封闭环之后，对每个组成环标“增”或“减”，并给一句理由，说明这个环变大时封闭环是变大还是变小。理由写不出来的标“待判”，不许猜，不许默认给“增”。

【第三步：极值法】按极值法算出封闭环的最大值与最小值。中间过程必须完整写出：每个环取的极限值、代入的算式、中间累加结果。不许只给最后两个数。算完与目标区间比对，明确写“满足”或“不满足”。不满足时给出超差量，并列所有可能的调整方向，不许指定某一个方向。

【第四步：统计法】用统计法（RSS）再算一遍，同样写出完整中间过程。算完之后，明确列出统计法成立的三个前提：各环相互独立、各环呈正态分布、各环的过程能力处于受控状态。逐条对照我在输入表里给的工艺来源，判断每个前提在你的数据下是否成立。任一条不成立，就明确写“统计法结果仅供参考，不能作为判定依据”。

【第五步：敏感度排序】按各组成环对封闭环总变动的贡献做敏感度排序，输出：环号 | 尺寸名称 | 公差带宽度 | 贡献占比 | 工艺来源。指出收紧哪一个环的收益最高。只给排序和占比，不许替我决定收哪一个。

【纪律】

1. 任何一环的公差数值都不许由你生成或修改。
输入表里没有的数，一律写“待补”，不许按常见做法填。
2. 不许评价我的公差给得合不合理，你只回答算出来是多少、是否满足目标。
3. 输出写进 D:\dim\尺寸链校核报告.md，三张表（列环、极值法、统计法）加一张敏感度表。

过程与判断点

2025 年 9 月，我在一台小型执行器的轴向间隙上跑过一遍。这条链一共六个组成环：壳体总长、端盖轴承室深度、输出轴轴肩到端面的距离、轴承宽度、波形垫圈自由高度、卡簧槽位置。

我按上面的提示词跑，它第一步列出来的封闭环就判对了，理由是“这个间隙在图纸上没有独立标注，图上的各零件的长度尺寸”。这一步看着简单，但我在带新人的时候见过好几次把封闭环判成轴承宽度的。

极值法算完，它给出的轴向间隙范围是 0.02 到 0.41，目标区间是 0.05 到 0.35，判定不满足，超差量两端各约 0.06。

统计法算完，范围是 0.09 到 0.31，判定满足。

同一条链，两种方法给出两个相反的结论。这种局面在尺寸链校核里天天发生，也是这个场景最需要人站出来的地方。

我做的判断分三步。

第一步，看统计法那三个前提成不成立。它自己列出来了，我逐条过：

前提	我的判断	依据
各环相互独立	部分成立	壳体总长与端盖轴承室深度出自同一副压铸模的同一批次，不独立
各环呈正态分布	不成立	波形垫圈是外购冲压件，供方给的是全检分选后的截尾分布
过程能力受控	成立	机加三环有近三个月的 SPC 数据，Cpk 均在 1.33 以上

三条里两条有问题，那统计法给出的 0.09 到 0.31 就不能当判定依据用。我在报告上写的是：按极值法判定不满足，需要调整。

第二步，看敏感度表。它算出来的贡献占比是这样：

环号	尺寸名称	公差带	贡献占比
1	端盖轴承室深度	0.12	34%
2	输出轴轴肩到端面	0.10	28%
3	壳体总长	0.08	18%
4	波形垫圈自由高度	0.06	12%
5	轴承宽度	0.05	5%
6	卡簧槽位置	0.03	3%

第一名是端盖轴承室深度，贡献 34%。这个环恰恰就是本章开场那个故事里老张问我的那个 0.12，当年我是照着上一代图纸抄的。三年之后 AI 把敏感度排出来，指向的还是同一个环。

第三步，拿着这张表去谈。端盖是压铸件，轴承室深度从 0.12 收到 0.08 需要加一道精镗，供应商报的单件成本上升我记不清了，量级在几毛钱。波形垫圈那一环换个供货状态，成本几乎不动，但只能贡献 12%。

最后我选的是端盖加精镗加输出轴轴肩改磨削，两个环一起收。谈完回过头再算一遍，极值法结果落在 0.06 到 0.33，进区间了。

同一条链，两种算法两个结论

极值法与统计法，选哪个由人判

极值法

- 算出来 0.02 到 0.41
目标区间 0.05 到 0.35，判定不满足
- 超差量两端各约 0.06
结果保守，零件的加工成本高
- 它的假设
所有组成环同时取到最不利的极限值

统计法

- 算出来 0.09 到 0.31
判定满足，结果宽松，成本低
- 三个前提两条有问题
同模批次不独立，垫圈是分选后的截尾分布
- 我的结论
不能当判定依据，报告上写按极值法判定不满足

极值法花的是钱，统计法赌的是假设

图 13-8 同一条链，两种算法两个结论：极值法不满足、统计法满足，选哪个由人判

图 13-8 作者根据公开资料绘制。

这里我要说清一条边界。AI 在这个场景里做的是算术和排版，做的是你不愿意花四小时去做的那部分。它做不了三件事：判封闭环、判断统计法的前提在你这条链上成不成立、决定收紧哪一个环。

第一件在提示词里我用“停下来等我确认”强制拦住了。

第二件它其实能做一半：它会老老实实把三个前提列出来，但它不知道你的端盖和壳体出自同一副模，也不知道你那个垫圈是分选出来的。这条信息在工艺那边，不在材料里。

第三件它最想替你做，也最不能让它做。收紧哪一个环，本质是花钱的决定。钱怎么花、跟谁谈、谈得下来谈不下来，全是人的事。

翻车清单

翻车一：增减环判反。症状是极值法算出来的范围明显偏离你的直觉，比如间隙算成了负值。根因是第二步的理由那列被它跳过了，直接默认全增。补救：要求逐环给理由，然后你抽查两个环手算一遍符号。这一步三十秒，是整套流程里性价比最高的动作。

翻车二：它直接套统计法，把前提那一节跳过去了。症状是报告里只有统计法结果，还写着“推荐使用统计法降低成本”。根因是第四步没写死。补救：把三个前提和“任一不成立就写仅供参考”这句话原样留在提示词里。

翻车三：某个环的公差是它自己填的。症状是输入表里你留空的那一格，到了报告里变成了一个具体的数。根因是你没有在纪律里封死。补救：纪律第一条原样保留，报告里凡是"待补"以外的来源，一律退回重查。

验收标准与变体

验收五条：

1. 封闭环是你亲口确认的，不是它自己定的。
2. 极值法的中间过程完整，你抽查任意两个环的符号都对得上。
3. 统计法那三个前提在你这条链上逐条判过，判不成立的已经标出。
4. 报告里每一个公差数值都能在输入表里找到原格，没有一个是它生成的。
5. 敏感度排序出来之后，你能说清为什么选了现在这两个环去收，而不是排第一那一个。

变体一：径向配合的过盈量校核。把轴向链换成径向，封闭环从间隙改成过盈量，其余提示词原样用。轴承外圈与壳体孔的配合、齿轮与轴的热装过盈，都是这一类。

变体二：装配累积误差校核。多个零件叠加后的位置偏差，比如线路板上三个安装孔与壳体三个螺柱之间的累积错位。这一类最容易在首件装配时暴露，提前算一遍能省一次工装返工。

变体三：拿它复核老项目的尺寸链计算书。找一份五年前归档的计算书，把输入表和图纸喂进去让它重算，跟当年的人工结果比对。我试过两次，其中一次发现当年某一环的符号取反了，那台产品已经量产了四年，靠垫圈选型一直在兜着。

老兵说：尺寸链算错了不可怕，可怕的是算完之后没人问那三个前提。极值法花的是钱，统计法赌的是假设，赌注押下去之前，先看一眼你手上的工艺到底配不配得上这个假设。

场景 41 变更影响分析：改一个尺寸会牵动哪些件 ★★

一句话背景

产线来了一个变更请求：输出轴的某个轴径从 8 毫米改到 9 毫米，理由是强度校核过不去。听起来是改一个数的事。真要查清楚它牵动多少东西，你得翻图纸、翻 BOM、翻工装清单、翻检具清单、翻试验大纲、翻 DFMEA、翻已经发出去的图纸版本，一个人查一天半，还常常漏掉"这个尺寸被三个项目共用"这种事。

这个场景让 AI 替你把牵动范围铺出来，30 分钟出一张待确认清单。本场景标 ★★：这套流程我跑通过，也用在真实项目上，没留下完整留档。

名词小词典

ECN（工程变更通知）：正式下发的设计变更单。它写清楚改什么、为什么改、什么时候切换、旧件怎么处理。

ECR（工程变更请求）：提出变更的那张单子。它先走评估和批准，批了才转成 ECN。

受影响件清单：这一次变更会牵动到的所有物件。包括直接改的那个零件，也包括所有跟它有关系的東西。

工装与检具：做这个零件用的夹具、刀具、模具，和检这个零件用的量具和检具。尺寸一改，这两类东西常常要重做。

借用件：同一个零件在好几个项目上被拿来用。改它一个，几个项目一起动。

特殊特性：图纸上打了标记的那几项，出问题会影响到安全、法规或功能。改到它头上，试验和客户认可都要重走。

开工前准备

1. 变更请求本身写清楚：改哪个件、哪个尺寸、从多少改到多少、为什么要改。含糊的变更请求会产出含糊的影响清单。
2. 这个零件的图纸与数模，以及它所在的上一级总成图纸。
3. 完整 BOM，最好带"哪些项目在用"这一列。没有这一列，借用件那一段就只能靠它猜文件名。
4. 工装清单、检具清单、试验大纲、DFMEA、特殊特性清单。缺哪一份就在最后的影响表里标"该项未评估"。
5. 一份你们公司的变更评估表模板，AI 的最后输出要长成这个样子。
6. WorkBuddy 新建对话，Agent 模式。

操作步骤

1. 第一步先让它复述这次变更：改哪个件、哪个尺寸、从多少到多少。复述错了一个数，后面全白跑。
2. 让它在 BOM 里把"直接受影响"的件列出来：就是这个零件本身，以及跟它直接装配的那些件。
3. 让它查跨项目借用。这一步要在整个 BOM 目录里搜同一个图号出现在哪些项目下。
4. 让它查工装与检具：清单里凡是带这个图号或这个尺寸特征的行，全部列出来。
5. 让它查文件层：图纸、数模、试验大纲、DFMEA、特殊特性清单里出现这个图号或这个尺寸的位置，逐条给出文件名和章节。
6. 让它查历史：过去三年有没有对同一个特征做过变更，如果有，把当时的评估结论捞出来。
7. 让它把影响分四层排序，并明确列出"本次未能覆盖"的项。
8. 最后两步归你：给每层标"需要动作"还是"仅知会"，然后拿去开评估会。

变更影响的四层

改一个尺寸，牵动的不止一个件

第一层 直接受影响件

这个件本身，加上与它有直接装配关系的配合件、相邻件、连接件

第二层 跨项目借用

在整个目录里按完整图号精确搜，搜不到就写明未发现跨项目借用

第三层 工装与检具

以这个尺寸定位的夹具、直接测这个尺寸的量具、靠它导向的刀具模具

第四层 文件与历史

图纸、数模、试验大纲、DFMEA、特殊特性清单，以及近三年变更记录

每层末尾加一节本次未能覆盖，如实列出没查的目录与读不了的文件

图 13-9 变更影响的四层：直接件、跨项目、工装检具、文件与历史

图 13-9 作者根据公开资料绘制。

可复制提示词

整段复制，图号和路径换成你自己的：

【任务】帮我做一次工程变更的影响分析。你是变更评估工程师。
你的活是把牵动范围铺全，不是判断这个变更该不该做。

【本次变更】

零件图号：XXXXXX（换成你的图号）

变更内容：输出轴轴径由 8 毫米改为 9 毫米（换成你的变更）

变更原因：强度校核不满足（换成你的原因）

【输入目录】

1. BOM 与项目清单：D:\ecn\BOM\
2. 图纸与数模：D:\ecn\图纸\
3. 工装与检具清单：D:\ecn\工装检具\
4. 试验大纲、DFMEA、特殊特性清单：D:\ecn\质量文件\
5. 近三年历史变更记录：D:\ecn\历史变更\

【第零步：复述】先用一句话复述：改哪个零件、哪个尺寸、从多少改到多少。
复述完等我确认，不要往下跑。

【第一步：直接受影响件】在 BOM 里找出这个图号所在的所有总成，
并列与它有直接装配关系的全部零件：配合件、相邻件、连接件。
输出：图号 | 零件名 | 关系类型（配合/相邻/连接） | 所在总成 | 出处文件名。

【第二步：跨项目借用】在 D:\ecn\BOM\ 全目录里搜索这个图号出现在哪些项目的 BOM 下。
逐个列出项目名与 BOM 文件路径。搜不到就明确写“未发现跨项目借用”，
不许写“可能还有其他项目”这种模糊话。

【第三步：工装与检具】在工装与检具清单里，找出所有与该图号相关的行，
特别关注：以这个尺寸为定位或测量基准的夹具、直接测量这个尺寸的量具与检具、
靠这个尺寸导向的刀具或模具。
输出：工装或检具编号 | 名称 | 与本次变更的关系 | 判定（需改制 / 需新制 / 不受影响 / 存疑）。
存疑的必须说明为什么存疑。

【第四步：文件层】在质量文件目录里逐份检索这个图号，列出出现位置：
文件名 | 章节或表内行号 | 原文引用（40 字以内） | 判定（需修订 / 需重评 / 不受影响）。
特别注意 DFMEA 里以这个尺寸或这个特征为失效起因的行，
以及特殊特性清单里是否包含这个特征。

【第五步：历史对照】在历史变更目录里检索过去三年对同一零件或同一特征的变更。
找到的列出：变更单号 | 时间 | 变更内容 | 当时的评估结论要点 | 与本次是否相关。
未找到就写“近三年未见对该特征的变更”。

【第六步：分层汇总与自曝】把以上结果按四层汇总：
第一层 直接受影响件；第二层 跨项目借用；第三层 工装与检具；第四层 文件与历史。
每层末尾加一节“本次未能覆盖”，如实列出你没查的目录、读不了的文件、
以及你不确定的判断。不许省略这一节。

【纪律】

1. 不许判断这个变更该不该做，不许给建议方案。

2. 找不到出处的条目不许写。没路径可引的一律扔掉。
3. 每个"不受影响"的判定必须给一句理由，不给理由的一律改为"存疑"。
4. 输出写进 D:\ecn\变更影响分析_图号XXXXXX.md。

过程与判断点

我在一个输出轴轴径变更上跑过这套流程。变更本身很干净：一根电机输出轴，轴径从 8 毫米加粗到 9 毫米，起因是扭转强度校核在极限工况下过不去。

它交回来的四层表，最后一层最有用。我把四层的条数和我的复核结果列一下：

层级	它报的条数	我核后保留	其中需要动作
第一层 直接受影响件	6	6	3
第二层 跨项目借用	2	2	2
第三层 工装与检具	9	7	4
第四层 文件与历史	13	8	3
合计	30	23	12

第二层那两条是我自己绝对查不出来的。它在 BOM 目录里搜到这个图号还出现在另外两个项目的 BOM 下：一个是某日系项目的同类执行器，另一个是一个已经量产三年的小批量产品。

我一开始以为它搜错了，去翻了那两个项目的 BOM，它是对的。这根轴是当年做通用化的时候统下来的，图纸是同一张，只是两个项目各自取了自己的物料号。图纸改一刀，两个项目一起动，其中一个已经量产三年，改图纸意味着客户认可要重走。

这一条的代价远大于轴径本身。最后这个变更我们没走加粗，改成换材料，强度从 45 钢换成 40Cr 调质，轴径一寸没动。

第三层里有四条需要动作，其中一条是磨削工序用的那个无心磨的导轮角度按 8 毫米调的，改 9 毫米要重新调。还有一条是装配线上的一支气动夹爪，抓取直径的行程上限就是 8.5。

这两条都在产线，不在我手边的文件里，是 AI 从工装清单里翻出来的。我自己的经验清单里，变更影响一般想到图纸、BOM、检具就停了，很少会想到夹爪。

第四层里有一条我核对了很久。它说 DFMEA 里有三行以"输出轴轴径"为失效起因，我翻过去看，实际只有一行。另外两行写的是"输出轴刚度不足"，它把刚度不足也归到了轴径头上。这一条我判成部分虚警，但保留了提醒，因为刚度确实会跟着轴径变。

四层影响的条数与保留

三十条里最贵的两条藏在跨项目借用层

第一层 直接受影响件

报六条，核后留六条，其中三条需要动作

第二层 跨项目借用

报两条，核后留两条，两条都要动：图纸同一张，两个项目一起改

第三层 工装与检具

报九条，核后留七条，四条需要动作，含磨床导轮与装配夹爪

第四层 文件与历史

报十三条，核后留八条，三条需要动作，其中一条判成部分虚警

每个不受影响的判定必须给一句理由，给不出来的一律改为存疑

图 13-10 30 条影响里，最贵的那两条藏在跨项目借用层

图 13-10 作者根据公开资料绘制。

这个场景的判断点有两个。

第一个是“存疑”这一栏。我在提示词里要求每个“不受影响”的判定必须给理由，给不出来的改成“存疑”。跑完这一轮，它给出的“不受影响”里有 5 条被它自己改成了“存疑”。这 5 条最后我去核了，2 条真受影响。如果当初让它自由判定，这 2 条就会被埋在一片“不受影响”里。

第二个是“本次未能覆盖”那一节。它自己交了三件事：历史变更目录里有两个 PDF 是扫描件读不到字；工装清单里有一个 Excel 带了工作表保护只读取了前 50 行；数模文件它打不开，尺寸特征那一栏是照着图纸填的。

这三句话比它报出来的 30 条影响更值钱。因为它明说了哪些地方没查，我就知道该补哪里。换成一个不交代的人，我会以为它全查了。

翻车清单

翻车一：只查了直接受影响件就停了。症状是清单上一共七八条，全是配合件和相邻件，看着很完整。根因是你的提示词只写了第一步。补救：四层必须全写进提示词，尤其是跨项目借用那一层，它是唯一一层你自己查不动的。

翻车二：跨项目借用搜成了同名不同件。症状是它报了十几个项目，你去核发现图号前缀不一样。根因是它按零件名模糊匹配而不是按图号精确匹配。补救：在第二步里写明“按完整图号精确匹配，不许按零件名模糊搜索”，并要求它给出每一处的 BOM 文件路径供你核对。

翻车三：它替你判断了这个变更该不该做。症状是报告末尾出现"建议采用加大圆角过渡的方案"这类话。根因是纪律第一条没生效。补救：见一句删一句，并且在纪律里补一句"输出中不得出现任何方案性建议"。

验收标准与变体

验收五条：

1. 变更的图号、尺寸、从多少到多少，它复述的和你写的一致。
2. 跨项目借用那一层，你按它给的文件路径逐个打开核过，没有一条是模糊匹配出来的。
3. 所有标"不受影响"的判定都带了理由，没带理由的已全部改为"存疑"。
4. "本次未能覆盖"那一节非空，且你按它交代的内容补查了至少一项。
5. 整张单子拿到变更评估会上，每一条都能说出该谁去动、什么时候动。

变体一：给供应商发起的变更做影响分析。供应商提了一个材料替代或者工艺变更，用同一套提示词跑一遍，输入目录换成你们内部与该供应商相关的全部文件。这一类最容易漏的是检具和已发往客户的认可文件。

变体二：给标准件的停产替代做影响分析。某个紧固件或轴承要停产，替换型号的尺寸有微小差异。把替换前后的参数喂进去，重点看第三层工装与第四层文件。

变体三：反向用它做变更前的自检。在你准备提 ECR 之前，先让 AI 跑一遍影响分析，把结果附在 ECR 后面。评估会上别人问的第一个问题通常是"你查过影响吗"，这时候你直接把单子推过去。

老兵说：变更影响分析里最贵的那一条，从来不在你改的那个件上，在那些你不知道自己也在用这个件的项目上。AI 替你做的是全目录搜索，它搜出来的东西你要逐条去核，尤其是它说"不受影响"的那些。

场景 42 经验教训库：把过去的问题变成可检索的资产 ★★

一句话背景

一个做了十几年的研发团队，手里一定攒着几百份东西：8D 报告、故障单、试验失效记录、评审纪要、售后索赔分析。它们躺在共享盘的文件夹里，按年份分目录，文件名像"2021-08-13 某项目低温问题 8D 最终版 v3.docx"。

真要用的时候你找不到。不是因为没有，是因为你记得有那么一回事，但记不清它在哪个目录、叫什么名字。

这个场景做两件事：把这几百分文件抽成一张能检索的表，然后在新项目开工前花五分钟检索一次。建库我跑了两半天，单次检索五分钟。本场景标 ★★：这套流程我跑通过，库现在还在用，没有完整留档。

名词小词典

经验教训 (Lessons Learned): 一件已经出过的事，连它的起因和当时的处理结论一起记下来，让下一次做同类设计的人能提前看见。

8D 报告: 处理一个质量问题的八步标准表格。第八步是团队表彰，第七步是预防再发。对建库来说，第三步临时措施和第五步根本起因最有用。

条目: 库里的一行。一个问题对应一行，不多不少。

结构化字段: 每一行的列。列定死了，检索才检得准。

源文件名与路径: 这一行是从哪份文件里抽出来的。没有这一列，整库不可信。

增量更新: 新出的问题按同一个模板往库里追加，不要另起一套。

开工前准备

1. 先把历史文件集中到一个目录。我用的是 `D:\lessons\原始文件\`，下面按年份分子目录，不再按项目分，因为按项目分的话同一个问题会出现在好几个项目下。
2. 统一格式。扫描件先做文字识别，老版本的 .doc 另存成 .docx。读不到的文件会在最后被 AI 自己列出来，你可以后面补。
3. 定死字段。我用的九列：编号、发生日期、产品类别、零件或特征、失效现象、根本起因、采取措施、验证结果、源文件名。字段一旦开跑就不要改，改一次全库要重抽。
4. 准备一份你们自己的产品类别字典。比如电机类、齿轮类、壳体类、线路板类、装配类。它按这个归，不让它自己发明类别。
5. WorkBuddy 新建一个专门的对话，这个对话后面长期留着做增量，不要跟别的活混。

操作步骤

1. 先让它报数：目录里一共多少份文件、能读到字的多少份、读不到字的多少份。读不到的先记住。
2. 让它抽第一份文件做样本，你逐列核对，把字段理解对不齐的地方当场改正。
3. 样本确认后，让它批量抽取，一份文件一行。一份文件里写了三个问题的，拆成三行。
4. 抽完让它自检：现象和起因写反的、措施和验证结果混在一起的，它自己标出来。
5. 你抽检 10%。抽检比例不能再低，低于这个数你会发现整库的信任度建立不起来。
6. 让它给每一行生成一列检索关键词，三到五个词，从现象和起因里取，不许用文件名的词。
7. 存成一张 XLSX，另外导出一份 Markdown 版本放在同一个目录里。Markdown 那一份是给检索用的，XLSX 是给人翻的。
8. 定一个增量规矩：每关闭一份 8D，三天之内追加进库。规矩要写进你们的质量流程，不然三个月就断了。

经验教训库的建库八步

报数、打样、批量、自检、抽检、标引、双份、增量



库的价值不取决于有多少条，取决于每一条能不能溯源到源文件

图 13-11 经验教训库的建库八步：报数、打样、批量、自检、抽检、标引、双份、增量

图 13-11 作者根据公开资料绘制。

可复制提示词

第一段，建库。整段复制：

【任务】把我放在 D:\lessons\原始文件\ 目录下的历史质量文件，抽成一张可检索的经验教训表。你是档案编目员，不是问题分析员。你负责如实摘录，不负责重新分析。

【字段定义】每一行九列，列名不许改：

编号 | 发生日期 | 产品类别 | 零件或特征 | 失效现象 | 根本起因 | 采取措施 | 验证结果 | 源文件名

【产品类别字典】只允许从下面选一个填，不许自己发明类别：

电机类、齿轮类、轴类、壳体类、线路板类、密封类、装配类、试验类、其他

【第一步：报数】先报：目录下共多少份文件、你能读到文字的多少份、读不到或字符数明显偏少的多少份。读不到的列出文件名。然后停下来，不要往下抽。

【第二步：打样】只抽 D:\lessons\原始文件\ 里的第一份文件，按九列输出一行给我看。我确认字段理解没问题之后再往下批量。

【第三步：批量抽取】逐份文件抽取，规则如下：

1. 一份文件只讲一个问题的，抽一行。一份文件讲了三个问题的，抽三行。
2. 失效现象写物理事实，不写结论。写"输出轴轴向窜动 0.4 毫米"，不写"输出轴设计不合理"。
3. 根本起因抄原文里的根本起因那一栏。原文没写的写"原文未给根本起因"，不许推测。
4. 采取措施与验证结果分两列。措施是做了什么，验证结果是做完之后测出来什么。原文只写了措施没写结果的，结果列写"原文未记录验证结果"。
5. 源文件名必须写全，带扩展名。

【第四步：自检】全表抽完之后，自己检查三类问题并单独列出：

- A 现象与起因写反的行；B 措施与验证结果混在一列的行；
C 同一行里出现"可能""大概""疑似"这类不确定词的行。
这三类列出来交给我处理，不许你自己改。

【第五步：标引】给每一行加三到五个检索关键词，从失效现象和根本起因里取。不许直接抄文件名里的词，不许用"问题""异常""故障"这种没有区分度的词。

【输出】写两张文件：

D:\lessons\经验教训库.xlsx，九列加关键词列。

D:\lessons\经验教训库.md，同样内容，每条之间用 --- 分隔。

第二段，检索。这一段是你以后每次开工前要跑的：

【任务】在 D:\lessons\经验教训库.md 里检索与我这次设计相关的历史问题。
你是检索员，只做匹配，不做分析。

【我要做的东西】

产品类别：齿轮类（换成你的）
关键特征：两级塑料斜齿轮，输出转速约 60 转/分，工作温度零下 40 度到 85 度
（换成你的工况）

【检索要求】

- 1. 按下面三个方向分别检索，每个方向单独出一张表：
方向一：相同产品类别下出过的问题。
方向二：相同工况（尤其是低温和高温）下出过的问题，跨类别也算。
方向三：失效现象里出现过“卡滞”“异响”“断齿”“磨损”这类词的问题。
- 2. 每张表按相关度排序，每条给出：编号 | 失效现象 | 根本起因 | 采取措施 | 源文件名。
- 3. 每个方向最多给 8 条。不足 8 条的写“该方向下仅命中 N 条”，不许凑数。
- 4. 相关度判断标准写清楚，写在一张表前面。

【纪律】

- 1. 只许引用库里真实存在的条目，不许补充库里没有的内容。
库里没有的，直接写“库内无相关记录”。
- 2. 不许对历史问题做新的原因分析，不许评价当年的措施做得对不对。
- 3. 每条必须带源文件名，没有源文件名的条目不许出现在结果里。

过程与判断点

我在 2025 年春天建过一次。原始目录里一共 236 份文件，其中 27 份是扫描件，AI 报读不到。
剩下 209 份，它抽出来 214 行。多出来的 5 行来自几份文件里讲了不少一个问题，其中一份 8D 里挂了三个失效。
我抽检了 22 行，约等于 10%。抽检结果是这样：

问题类型	命中行数	我的处理
字段填写准确	14	直接通过
措施与验证结果混在一列	4	按原文重填
根本起因是它推测的	2	改成“原文未给根本起因”
现象写成了结论	2	按原文的物理描述重写

22 行里 8 行要改，改的比例 36%。这个比例我记下来了，后来做增量的时候每次都按这个比例预留复核时间。

自检那一轮它自己报了 19 行 A 类、31 行 B 类、7 行 C 类。这个数比我抽检发现的多，因为它查的是全表。A 类那 19 行里我最后确认是它自己写反的占 11 行，剩下 8 行是原文就写反了。原文写反的那 8 行我没有改，只在备注里标了“原文现象与起因疑似颠倒”。

库建完之后，真正的用途在检索这一头。

2025 年 7 月，我接手一个新执行器的方案设计，开工前跑了一次检索。方向二里命中了一条 2021 年的记录，源文件名是"2021-08-13 某项目低温问题 8D 最终版 v3.docx"。

那条记录的现象写的是"低温存放 24 小时后首次启动，输出扭矩超上限约 40%"，起因是"输出端润滑油基础油在零下 40 度析出稠化"。

我在方案阶段就把油脂换成了低温型，成本上浮我记得不多，单件几分钱。如果等到 DV 试验才暴露，一次低温试验加一轮整改，光试验排期就是三周。



图 13-12 建库两天半，检索五分钟，省下的是三周试验排期

图 13-12 作者根据公开资料绘制。

这个场景里我要说清一条边界：库的价值不取决于它有多少条，取决于每一条能不能溯源。

我要求每一条必须带源文件名，理由很直接。检索出来一条记录，你去问当年处理这件事的人，他说的和库里写的常常不一样。这时候你能打开源文件看原文，才谈得下去。没有源文件名的条目，在会上被质疑一次就废了，而且会连累旁边那些有出处的条目一起不被信任。

另外一条：它不会替你想起你没问的问题。检索是按你给的关键词匹配的，你如果没提"低温"，它就检不出低温那条。我现在的规矩是三个方向里必须有一个方向是跨类别的工况检索，就是为了防止自己只盯着零件类别。

翻车清单

翻车一：它把措施写成了结论，或者把结论写成了措施。症状是采取措施那一列写着"问题已解决"，验证结果那一列写着"更换油脂"。根因是原文这两栏就写得含糊，它照抄的时候分不清。补救：打样那一步必须你亲自核，把两列的判定标准当场讲清楚，宁可多花十分钟。

翻车二：抽出来的条目没有源文件名。症状是表里有一批行的源文件名列是空的或者写着"见附件"。根因是批量抽取时它没有把文件名跟着走。补救：在纪律里写死"没有源文件名的条目不许输出"，抽完之后按源文件名的非空率统计一遍，低于 95% 就整批重跑。

翻车三：检索时它开始编。症状是你问了一个库里没有的方向，它还是给出了七八条，看着还挺像。根因是你没封死"只许引用库里真实存在的条目"。补救：纪律第一条原样保留，并且要求它在每个方向写清命中条数，你随手抽一条去库里搜编号，搜不到就整批作废。

验收标准与变体

验收五条：

1. 你抽检 10% 以上，改动比例记下来，作为下次增量的复核工时依据。
2. 全库源文件名的非空率在 95% 以上，抽查五条都能在目录里打开对应文件。
3. 检索结果里每一条都能按编号在库里搜到，编号对不上的整批作废。
4. 库里找不到的方向，它明确回"库内无相关记录"，而不是给你编几条。
5. 增量规矩已经写进你们的质量流程，有明确的负责人和时限。

变体一：给试验失效记录建索引。历次 DV/PV 试验里那些没通过的项，单独建一张小表，字段简化成五项：试验项、失效现象、当时的分析、设计改了什么、复测结果。这一张表在新项目排试验大纲时最有用。

变体二：给评审纪要建决策索引。把历次评审纪要里的开口项和最终结论抽出来，按零件归。下次评审同一个零件时，先把当年被问过的问题过一遍。

变体三：给新人当入职检索入口。新人上班第一周给他这个库和检索提示词，让他按自己要接手的零件检索一遍。他读到的不是教材，是这个团队真出过的事。

老兵说：一个团队最贵的资产不是那几台设备，是它出过的那些事被记下来了没有。记在文件夹里不算记下来，能被五分钟检索出来才算。

小白词典

设计评审 (Design Review)：设计走到某个节点，把材料摊开给一群人挑毛病、决定能不能往下走。类比：交卷前的最后一次自查，只不过查你的人有七个。

DR1 到 DR5：设计评审的几个节点代号。各家叫法不同，逻辑一致：越往后越接近量产，越往后越难改。

开口项 (Open Issue): 评审上被问住、当场没答上来的问题。它必须有人、有日期、有关闭准则, 缺一样等于没提。

DFMEA: 设计失效模式与后果分析。 一张表, 逐条写这个产品能怎么坏、坏了会怎么样、为什么会坏、现在有什么挡着。类比: 动手前的排雷图。

失效模式: 零件坏的方式。类比: 生病的名字。

失效后果: 坏了之后用户感受到什么。类比: 症状。

失效起因: 为什么会走到这一步。类比: 病因。

严重度 S: 后果有多严重, 1 到 10 分。涉及安全和法规的通常打到 9 或 10。

发生度 O: 这个起因出现的可能性, 1 到 10 分。

探测度 D: 量产之前, 现有手段能把这个问题查出来的能力, 1 到 10 分, 数越大越查不出来。

RPN: 风险优先数, S、O、D 三个数相乘。它只用来排先后顺序, 不代表风险高低。

AP (措施优先级): 高、中、低三档, 新版 AIAG-VDA 手册用它来定先做哪件事。

GD&T: 几何尺寸与公差, 一套用符号规定形状和位置允许偏差范围的语言。

基准 (Datum): 测量时的起始参照。没给基准就提垂直度, 等于没说参照哪儿垂直。

公差: 尺寸被允许变动的范围。类比: 及格线上下各留的一条缓冲带。

尺寸链: 一串首尾相接的尺寸, 最后一个是被前面几个挤出来的结果。

封闭环: 尺寸链里那个你没法直接标、只能由别的尺寸凑出来的尺寸。它通常就是你要保证的那段间隙。

极值法: 假设所有尺寸同时取到最不利的极限来算。结果保守, 成本贵。

统计法 (RSS): 假设各尺寸独立且正态分布, 用平方和开根号来算。结果宽松, 成本低, 假设不成立就不能用。

敏感度: 某一个尺寸的公差变动, 对最终结果贡献的比例。收紧占比最高的那一个最划算。

ECN / ECR: 工程变更通知与工程变更请求。 ECR 是提申请, ECN 是正式下发。

借用件: 同一个零件被好几个项目拿来用。改它一个, 几个项目一起动。

特殊特性: 图纸上打了标记的那几项, 出问题会影响到安全、法规或功能。

8D 报告: 处理质量问题的八步标准表格。第三步是临时措施, 第五步是根本起因, 第七步是预防再发。

经验教训库: 把过去出过的事连起因带措施记成一张能检索的表。类比: 团队自己写的错题本。

SPC: 统计过程控制, 用连续的测量数据盯住一道工序稳不稳。

Cpk: 过程能力指数, 衡量一道工序做出来的尺寸有多集中。1.33 是常见的门槛值。

PPAP: 生产件批准程序, 量产之前向客户提交的一整套认可资料。

你的岗位怎么用

设计工程师: 评审前 25 分钟跑一遍材料预审, 把 AI 挑出来的单子自己关一轮再进会议室。画完图之后用图纸检查清单过一遍, 重点看它标"存疑"的那几行。做尺寸链时让它算两遍, 你负责判封闭环和统计法那三个前提。

质量工程师：把 DFMEA 的评分权牢牢握在自己手里，只让 AI 做联想和补全。拿到它的联想稿之后，先核对历史问题目录里那些实际出过的事有没有进表。变更评估会上，要求提变更的人附一张影响分析单。

工艺与制造工程师：在变更影响分析里盯住第三层工装与检具，那是你会被波及的地方。尺寸链校核时，把每个环的真实工艺来源给全，AI 才能判断统计法的前提成不成立。

试验工程师：把历次试验里没通过的项抽成一张小表，按失效现象和工况标引。新项目排大纲之前先检索一遍，能省掉一轮重复的失效。

采购与供应商质量：给供应商发变更影响分析的输出，让他在自己的过程文件里逐条回应。供应商提交的一整包认可资料，用评审预审那套提示词先跑一遍，重点看内部打架的硬矛盾。

项目经理：把"开工前检索经验教训库"写进项目流程的节点清单，作为方案评审的准入条件之一。变更评估会上第一句问"影响分析做了吗"，没做的打回去。

本章交付物

一件能带走的东西：一张《评审前 30 分钟自检单》。

怎么做：

1. 新建一张表，五列：检查项、用的提示词、要喂进去的文件、产出、谁复核。
2. 把本章六个场景各填一行。比如"尺寸链校核：喂输入表和图纸，产出校核报告，由设计本人复核封闭环与统计法前提"。
3. 每一行后面加一列"本次未覆盖"，把 AI 自己交代没查的部分抄进去。
4. 打印出来压在键盘下面，评审前一天晚上照着走一遍。
5. 每次评审结束，把这一轮新出现的漏项补进行里。这张单子是会长大的。

我的经验：这张单子我做了三个版本。第一版只有四条，全是技术项。第二版加了"材料里有扫描件吗"和"上一版的开口项关了吗"这两条行政项，因为这两条在实际评审里被卡住的次数最多。第三版加了"本次未覆盖"这一列，加完之后我在评审会上被问住的几率明显降了，不是因为我懂得多了，是因为我提前知道自己哪里没查。

本章金句

AI 挑出的问题要人复核，AI 没挑出的问题不代表不存在。评审签字的人是你，担责的人是你。

预审不是为了让它替你批材料合格，是为了让你在进会议室之前先体验一次被人问住的感觉。

它标"缺"的条目，八成是它读漏了，值得复核；它标"存疑"的条目，八成是真问题，值得深挖。

DFMEA 里最贵的那几行，从来不是想出来的，是吃过亏记下来的。分数那一栏，谁签字谁填。

极值法花的是钱，统计法赌的是假设。赌注押下去之前，先看一眼你手上的工艺配不配得上这个假设。

一个团队最贵的资产不是那几台设备，是它出过的那些事被记下来了没有。记在文件夹里不算记下来，能被五分钟检索出来才算。

（下一章：供应链与产线。设计图纸过了评审，只是麻烦挪了个地方，接下来轮到供应商和产线接手。）

第 14 章

供应链与产线：从一堆报价和报表里看出真话

报价单上最便宜的那一家，常常不是成本最低的那一家。差的那些钱从来不写在单价那一列里。

2025 年 12 月的那个晚上，我在干一件从入职第一年就在干的事：五家候选供应商的资质核查。

二十多项资质，散在两家供应商自己的门户、一个行业平台、一个招标网站，还有对方邮件发来的一个压缩包里。凌晨一点，我在解压包里翻设备清单的 PDF，翻到第四十页的时候，手停了一下。

序章里我讲了那晚的痛。但后来复盘我发现，真正吃掉我两个晚上的不是翻页和复制，是翻完之后的那一段。

五家的证书格式完全不一样。A 厂给的是扫描件，字号糊到要看第二遍；B 厂给的是网页截图，只有半页；C 厂的证书状态栏写的是"有效期内"，不是我以为的"有效"。有效期有的精确到日，有的只写到月。更要命的是 D 厂和 E 厂属于同一个集团下的两个法人主体，各有一套证书，我得自己判断哪一套才对应我这个项目要定的那个料。

抓回来的东西只是原料。我要交给项目会的是一张能横向对比的表，外加一句我敢签字的结论。这两件事之间隔着口径、缺项、判定和排序，全是判断活，也全是最耗人的部分。

第 12 章场景 31 到 33 讲的是前半段：怎么让 AI 登录你的门户、怎么把字段摘出来、卡住了怎么接管。这一章讲后半段：数据回到你手上之后，怎么统一口径、怎么对比、怎么排序、怎么从一堆数字里得出一个结论。

六个场景，从供应商资质一路排到 8D 报告。先挑跟你手头活最像的那个抄。

场景	场景名	人工耗时	AI 耗时	可信度
43	供应商资质核查：五家资质一次对完	约 2 个晚上	约 40 分钟	★★★ 实测
44	报价对比与降本分析：把不同格式的报价拉成一张可比表	约 1.5 个工作日	约 50 分钟	★★★ 实测
45	来料检验数据整理：从检验记录到趋势	约 1 个工作日	约 45 分钟	★★★ 实测
46	产线异常记录分析：把零散记录变成问题排序	约 2 个工作日	约 1 小时	★★★ 实测
47	产能与节拍数据梳理：找出瓶颈在哪	约 1 个工作日	约 40 分钟	★★ 跑通
48	质量问题的 8D 报告辅助：从事实到报告	约 1.5 个工作日	约 1 小时	★★★ 实测

场景 43 供应商资质核查：五家资质一次对完 ★★★

一句话背景

新项目要定点两家供应商，五家候选，每家二十多项资质要核。人工干这件事要两个晚上，其中大部分时间花在“这几家的写法不一样”上，而不是花在判断上。这个场景教你把五家的资质摊成一张矩阵表，一次对完，四十分钟出结论。

名词小词典

SOR (Statement of Requirements)：项目发给供应商的那份要求文件，里面写清了要什么、要多少、要达到什么标准。它是资质核查的输入，你要核的每一项资质都应该能从 SOR 里追到一句要求。

IATF 16949：汽车行业的质量管理体系认证。通俗讲，这家工厂有没有一套被认可的规矩来保证做出来的东西稳定。

ISO 17025：实验室认可。有这张证说明它的实验室测出来的数据被承认，没有的话它给你出具的报告你可能要自己复测一遍。

认可范围：证书背面那几页写的具体能做什么。有 IATF 16949 的厂很多，但认可范围写着“冲压件”的，做不了你要的电机总成。

有效期至：证书失效的那一天。它跟你比价时的报价有效期是两回事，这张过期了整张证书作废。

开工前准备

1. 一份资质核查清单，逐项列出：证书名称、发证机构要求、认可范围要求、是否在有效期内、缺了怎么办。这张清单是你自己定的，AI 替不了。
2. 五家供应商的资质材料。已经在本地的（PDF、扫描件、CSV）放在一个目录里；还在门户上的，先用第 12 章场景 31 到 33 的方法抓下来。
3. 一个输出目录，路径英文、无空格，比如 `D:\AI-SUPPLY\run01`。
4. 一份供应商主体清单：每家供应商的法人全称。集团和子公司分开写，这一步错了后面全错。
5. 你心里先有个数：这二十多项里，哪几项是一票否决项。这个判断要你下。

操作步骤

1. 把资质核查清单和供应商主体清单先给 AI，让它复述一遍它要核什么，确认它理解对了。
2. 一个供应商一个供应商地喂材料，先跑一家，看它交出来的格式对不对。
3. 格式对了再放开到五家，要求它输出一张“供应商 × 资质项”的矩阵。
4. 让它把矩阵里每一个格子的判定分成三档：有且有效、有但存疑、没有或未找到。
5. 让它单独出一份存疑清单，每一条写明存疑原因和需要补什么。
6. 让它按你的意思算有效期：距今不足 180 天的单独列出来。
7. 最后让它出结论页：可定点、需补充材料、建议淘汰，各写判断依据。

8. 你自己逐条复核矩阵，尤其是"有且有效"那一类里认可范围那几行。

资质核查矩阵：供应商 × 资质项

每格只填四档之一，另跟一列证据写清来自哪个文件哪一页

资质项	A 厂	B 厂	C 厂	D 厂 (集团)	E 厂 (子公司)
IATF 16949 体系	有且有效	有但存疑	有且有效	有且有效	有但存疑
ISO 17025 实验室	有且有效	未找到	有且有效	有但存疑	有但存疑
认可范围是否覆盖	有且有效	有但存疑	有且有效	有但存疑	有但存疑
有效期至	有且有效	有且有效	有但存疑	有且有效	有但存疑
一票否决项	有且有效	有且有效	有且有效	有但存疑	有且有效
法人主体一致	有且有效	有但存疑	有且有效	有且有效	有但存疑

供应商名脱敏示意；同一集团的不同法人主体，证书不许互相借用

图 14-1 资质核查矩阵：供应商 × 资质项，每格三档判定

图 14-1 作者根据公开资料绘制，供应商名均为脱敏示意。

可复制提示词

整段复制，括号里的内容换成你自己的：

【任务】帮我把五家候选供应商的资质核成一张可对比的矩阵表，并给出定点建议。

【输入】

资质材料目录：D:\AI-SUPPLY\run01\certs\

核查清单文件：D:\AI-SUPPLY\run01\checklist.md

供应商主体清单：A 厂 / B 厂 / C 厂 / D 厂（集团法人）/ E 厂（集团子公司）

【第一步：先复述】

开工前先把你要核的资质项和判定标准复述一遍给我看，
我确认之后你再动手。不许跳过这一步直接开始。

【第二步：矩阵】

输出 D:\AI-SUPPLY\run01\matrix.csv，UTF-8 带 BOM。

行为资质项，列为五家供应商。每个格子只许填四个值之一：

有且有效 / 有但存疑 / 没有 / 未找到

"未找到"指你翻遍了材料也没看到这项；"没有"指材料里明确写了没有。

这两者不许混用。

每个格子后面跟一列"证据"，写清这个判定来自哪个文件的哪一页或哪一行。

【第三步：认可范围】

对每一张体系类证书，单独摘出认可范围原文，

并判断该范围是否覆盖我方产品类别（微电机与执行器总成）。

覆盖不到位的，判"有但存疑"，不许判"有且有效"。

同一集团的不同法人主体，证书不许互相借用，各自按各自的主体核。

【第四步：有效期】

所有带有效期的证书，算出距今剩余天数。

剩余不足 180 天的单独列一份 D:\AI-SUPPLY\run01\expiring.csv，

字段：供应商、证书名称、证书编号、有效期至、剩余天数，按剩余天数升序。

【第五步：存疑清单】

输出 D:\AI-SUPPLY\run01\doubt.md。

每一条写三件事：存疑的是哪一项、为什么存疑、需要供应商补什么材料。

不许写"建议进一步确认"这种没有内容的句子。

【第六步：结论】

输出 D:\AI-SUPPLY\run01\verdict.md。

把五家分成三档：可定点 / 需补充材料 / 建议淘汰。

每家写不超过三句判断依据，每句必须能追到矩阵里的具体格子。

一票否决项缺失的直接进"建议淘汰"，不许因为别的项目多就放过去。

【纪律】

证书状态一律摘原文，不许归纳成"有效/无效"两档。

日期一律精确到日，材料里只写到月的，按该月最后一天算并在证据列标注。

任何你没在材料里看到的内容，一律写"未找到"，不许推测、不许补全。

过程与判断点

它交回来之后，我盯四个地方。

第一，格子里的四档有没有被混用。我第一趟跑的时候，二十多项里它填了十九个"有且有效"，一个"未找到"。我心里咯噔一下，因为按经验绝不可能这么干净。回头一查，它把三个"材料里只给了证书封面、没给认可范围页"的项也填成了"有且有效"。所以后来我在提示词里加了那句"覆盖不到位的判有但存疑"，第二次跑出来是十四个"有且有效"，四个"有但存疑"，两个"未找到"，这才像真的。

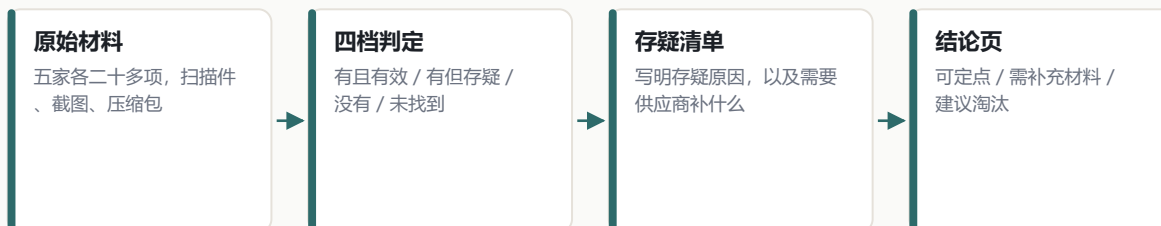
第二，认可范围那一列有没有原文。没有原文的判定一律退回。这一列是整张表里最容易出事的地方，因为有证和证能覆盖你的产品，是两件事。

第三，集团两个主体的证书有没有串。D厂和E厂那次，它第一版把D厂的实验室认可挂到了E厂名下。这种错人看一眼能看出来，但前提是你要专门看这一眼。

第四，结论页里"可定点"的有没有写清依据。它写"综合来看A厂最合适"这种话的时候，直接打回。我要的是"可定点，因为二十项中十九项有且有效，唯一存疑项是某证书的认可范围未覆盖，需补充附件"。

四档判定流转：从材料到结论页

抓回来的只是原料，中间隔着口径、缺项、判定和排序



矩阵里每一格都要能追到证据，认可范围必须摘原文

图 14-2 四档判定的流转：从原始材料到结论页的三次收敛

图 14-2 作者根据公开资料绘制。

翻车清单

翻车一：把"有证"当成"证能覆盖你"。症状是矩阵里一片绿，会议上一问认可范围，答不上来。补救：强制要求认可范围摘原文，并让 AI 自己判一句覆盖与否，覆盖不到的一律降档。

翻车二：集团内证书串用。症状是两家供应商的某项资质判定一模一样，连证书编号都相同。补救：在供应商主体清单里写明法人全称，并在提示词里写死"不同法人主体证书不许互相借用"。

翻车三：日期精度被抹平。症状是有效期只写到月的，它按当月 1 号算，凭空多出一个月。补救：在提示词里写死"只写到月的按该月最后一天算，并在证据列标注"，事后抽查三条。

验收标准与变体

验收五条：

1. 打开矩阵 CSV，格子里只有四种取值，没有"有效""OK""待定"这类自造词。
2. 随机挑三个格子，照"证据"列打开对应文件的对应页，三个全对。
3. 认可范围那一列有原文，且每一条后面跟着覆盖与否的判断。
4. 存疑清单里每一条都有"需要补什么材料"，没有一条是空的。
5. 有效期不足 180 天的清单，你自己用日历核一遍剩余天数，抽查两条。

变体一：年度复评。把矩阵存成基线，第二年只核变化的部分：到期换证的、新增要求的、供应商自身变更的。第一次建表最累，之后每年四十分钟。

变体二：新供应商准入。换成一个新供应商、同样的二十多项，跑出来的矩阵直接就是准入评审的输入材料。这一变体我推荐给采购同事，因为它产出的是一份可以直接发给对方的补件清单。

变体三：反向自查。把你们自己厂的证书按同样的二十多项核一遍。这件事做完你会发现，有几项我们自己也是"有但存疑"。

老兵说：资质核查的价值不在查出谁不合格，在查出谁"看起来合格"。看起来合格的那几家最危险，因为它们会一路走到定点，然后在量产爬坡的时候才暴露。

场景 44 报价对比与降本分析：把不同格式的报价拉成一张可比表 ★★★

一句话背景

五家供应商，五份报价，五套完全不同的格式。一份是 Excel，一份是 PDF 扫描件，一份是邮件正文里的一张表，还有两份是对方系统导出的 CSV。人工拉成一张可比表要一个半工作日，而且拉出来的东西常常不能比，因为口径没统一。这个场景教你先把口径统一，再比价。

名词小词典

RFQ (Request for Quotation)：询价。你发给供应商的一份"这个东西你要多少钱"的正式文件，里面会写明数量、交期、币种、报价有效期这些要求。

最小起订量 (MOQ)：供应商愿意接单的最小数量。低于这个数它不接，或者要加价。

模具费分摊：开一套模具的一次性投入，摊到每一件产品上。一次性收和摊进单价，看着差很多，算总账是一样的。

账期：货到之后多少天付款。月结 60 天和预付，之间的差别是一笔真金白银的资金占用。

阶梯价：买得越多单价越低的一张价格表。500 件一个价，5000 件一个价，5 万件又一个价。

开工前准备

1. 五份报价的原始文件，放在一个目录下，不要改名，保留原始文件名便于追溯。
2. 一份你自己写的口径清单：要统一哪些项、每项统一成什么。这清单就是下面那张比价口径表。
3. 一个年用量数字。这个数决定你取阶梯价的哪一档，没有它阶梯价没法比。
4. 汇率基准日。涉及外币报价时必须先定死，不然今天和明天算出来不一样。
5. 资金成本的年化利率，用来把账期折成钱。你们财务有这个数，问一次就够。
6. 输出目录 `D:\AI-SUPPLY\run02`。

操作步骤

1. 先让 AI 把五份报价的原始字段列出来，一份一行，不许做任何换算。这一步是让你看见差异有多大。
2. 它列完之后，你对照口径清单，逐项告诉它统一成什么。不要让它自己定。
3. 让它按你定的口径做归一化，输出一张主表。
4. 让它单独出一张口径差异表：哪些项它做了换算、换算的假设是什么。这张表比主表重要。
5. 让它算三种口径下的排名：原始报价排名、归一化单价排名、三年总成本排名。
6. 你看三个排名不一致的那些行，那些就是真正值得谈价的地方。
7. 最后让它列一份"要跟供应商确认的问题清单"，一条一个问题，可以直接发邮件。

比价口径表：十项最容易对不上

算术错了还能发现，后三行错了要三年后才疼

口径项	常见的两种写法	不统一会怎样	统一办法
币种	人民币 / 美元 / 欧元	直接比大小，差一个汇率	全部折成同一币种，写明汇率基准日
含税与否	含税价 / 未税价	差十几个点	全部折成不含税价，税额单独一列
计价单位	元每件 / 元每套 / 元每千件	差一千倍	全部折成元每件
最小起订量	500 件 / 5000 件	阶梯价根本不是一个台阶	按实际年用量取对应台阶
模具费	一次性收 / 摊进单价 / 免收	一次性收的看着便宜	按项目生命周期总量摊成单件
付款账期	月结 30 天 / 60 天 / 预付	差一笔资金占用成本	折成资金成本，按年化利率估
运费与包装	出厂价 / 到厂价 / 含托盘	差几个点	全部折成到厂价
报价有效期	30 天 / 90 天 / 未写	过期报价比了也没用	有效期单独一列，过期标红
年降条款	每年降 3% / 不降	三年后价差反转	按三年算一个总价
量产爬坡	首年一个价 / 不分年度	首年低价拿单，后面涨	分年度列价，逐年算

口径没统一就比出来的排名，比不比还危险

图 14-3 比价口径表：十项最容易对不上的东西

图 14-3 作者根据公开资料绘制。这是本章最该被撕下来贴墙的一页。

比价口径表

下面这张表是我这些年攒下来的，十项。你照着这个格式改成你自己行业的版本。

口径项	常见的两种写法	不统一会怎样	统一办法
币种	人民币 / 美元 / 欧元	直接比大小，差一个汇率	全部折成同一币种，写明汇率基准日
含税与否	含税价 / 未税价	差十几个点	全部折成不含税价，税额单独一列
计价单位	元每件 / 元每套 / 元每千件	差一千倍	全部折成元每件
最小起订量	500 件 / 5000 件	阶梯价根本不是一个台阶	按你的实际年用量取对应台阶
模具费	一次性收 / 摊进单价 / 免收	一次性收的看着便宜	按项目生命周期总量摊成单件
付款账期	月结 30 天 / 60 天 / 预付	差一笔资金占用成本	折成资金成本，按年化利率估
运费与包装	出厂价 / 到厂价 / 含托盘	差几个点	全部折成到厂价
报价有效期	30 天 / 90 天 / 未写	过期报价比了也没用	有效期单独一列，过期标红
年降条款	每年降 3% / 不降	三年后价差反转	按三年算一个总价
量产爬坡	首年一个价 / 不分年度	首年低价拿单，后面涨	分年度列价，逐年算

这张表里我最想让你注意的是后三行。前三行是算术问题，错了还能发现；后三行是生意问题，错了要三年后才疼。

可复制提示词

【任务】把五份不同格式的报价，拉成一张可以横向比的比价表。

【输入】

报价文件目录：D:\AI-SUPPLY\run02\quotes\

我方年用量：约 12 万件/年（示例数，按你的实际量改）

项目生命周期：3 年

汇率基准日：2026-01-05

资金成本年化利率：5%（问你们财务要这个数）

【第一步：原样列出，不许换算】

把五份报价里的价格相关字段原样列出来，一份报价一段，

字段包括：单价数值、币种、计价单位、含税与否、最小起订量、

模具费金额与收取方式、付款账期、运费与包装条款、

报价有效期、年降条款、量产爬坡期价格。

原样列的意思是：对方文件里写什么你就写什么，一个数字都不许动。

列完之后先停下来报给我，我确认了你再往下做。

【第二步：口径差异表】

输出 D:\AI-SUPPLY\run02\caliber_diff.md。

逐项列出十项口径：币种、含税、计价单位、最小起订量、模具费、

账期、运费、报价有效期、年降、量产爬坡。

每一项写：五家各自的原始写法、你打算统一成什么、换算假设是什么。

任何你不确定该统一成什么的，写“待你确认”，不许自己定。

【第三步：归一化主表】

输出 D:\AI-SUPPLY\run02\price_main.csv, UTF-8 带 BOM。

按第二步里我确认过的口径做换算，列为供应商，行为成本项：

到厂不含税单件价、模具费摊到单件、账期折成的资金成本、

运费、三年总成本。

每一个数字后面加一列“换算过程”，写清从哪个原始数怎么算过来的。

【第四步：三种排名】

输出 D:\AI-SUPPLY\run02\ranking.md，三张小表：

1. 按对方原始报价单价排名；

2. 按归一化后的到厂不含税单件价排名；

3. 按三年总成本排名。

三张表名次不一致的供应商单独列一段，说明差在哪一项。

【第五步：问题清单】

输出 D:\AI-SUPPLY\run02\questions.md。

列出需要向每家供应商确认的问题，一条一个问题，

写成可以直接粘进邮件的句子，不许写“请确认贵司的价格构成”这类空话。

每条后面标注：这个问题影响上面十项口径里的哪一项。

【纪律】

不许在第一步就做换算。第一步做换算，我就看不见差异在哪了。

任何假设都要写进口径差异表，不许藏在换算过程里。
报价有效期早于今天的，照常参加比价，但在主表最后一列标"报价已过期"。
不许因为某家报得特别低就替它找理由，也不要替高的那家找理由。

过程与判断点

我盯三件事。

第一，第一步有没有真的原样列。它第一版给我列的时候，已经自作主张把美元折成了人民币。数字是对的，但我看不见"这家用美元报"这个事实了，而这件事本身是要写进出口径差异表的。所以我把"原样列出不许换算"提到了第一步，并加了一句"列完停下报给我"。

第二，三种排名是否一致。我这趟跑出来，原始报价排名第一的那家，按归一化单件价掉到了第三，按三年总成本掉到了第四。差在两处：它的模具费是一次性收取（总额不低，但没进单价），以及它写明年降 0%。三年摊下来，它反而是最贵的。这个反转就是这一趟全部的价值所在。

第三，问题清单能不能直接发。它第一版写的是"请贵司确认模具费的收取方式及价格构成"。这句话发出去对方不会回。我要求改成"贵司报价中的模具费共计约 XX 万元，请问是一次性收取还是分摊进单价？若一次性收取，请问在何种条件下可转为分摊？"能直接粘进邮件的句子，才叫清单。

三种排名的反转

同一家供应商，换一种口径换一个名次

三种口径下的排名	差在哪两处
按原始报价单价 第 1 名 按归一化到厂不含税单件价 第 3 名 按三年总成本 第 4 名 看什么 名次不一致的那几行，就是值得谈价的地方	模具费 一次性收取，总额不低，但没有进单价 年降条款 这家写明年降 0% 结果 三年摊下来，它反而是最贵的 纪律 不许替报得低的那家找理由

数值为量级示意，非真实报价；项目生命周期按 3 年算

图 14-4 三种排名的反转：原始报价、归一化单价、三年总成本

图 14-4 作者根据公开资料绘制，数值为量级示意，非真实报价。

翻车清单

翻车一：账期被当成白送。症状是主表里根本没有账期这一行，于是月结 90 天和预付看起来一样。补救：把账期折成资金成本单列一行，公式写进换算过程列，按年化利率乘天数算，数字可以让财务帮你定。

翻车二：阶梯价取错台阶。症状是拿 A 厂 5000 件的单价去比 B 厂 500 件的单价，比出来的结论完全反。补救：先给它你的年用量，让它在主表里注明每家取的哪一档，档位不同的单独标出来。

翻车三：报价已经过期还在比。症状是比完发现最便宜那家的报价是三个月前的，重新问一遍价格全变了。补救：有效期单独一列，过期的标红，并且问题清单里自动生成一条"请更新报价"。

验收标准与变体

验收六条：

1. 口径差异表十项齐全，每一项都写了五家的原始写法，没有一项是空的。
2. 归一化主表里每一个数字后面都有换算过程，没有裸数字。
3. 用换算过程里的公式，手算三个数字复核一遍，三个全对。
4. 三种排名都出来了，且不一致的地方有说明。
5. 问题清单里每一条都是能直接粘进邮件的完整句子。
6. 报价过期的那一行标了红，没有悄悄混在正常报价里。

变体一：降本提案。把主表按成本项拆开，让 AI 对每一项提出两到三个降本方向，并标注每个方向大概影响几个点、需要谁配合。它提的方向不都能落地，但能帮你把降本会议从"大家一起想办法"变成"这六条逐条过"。

变体二：年降谈判准备。把三年总成本那张表单独拿出来，按年拆开做成一张年降谈判的底表。你带着这张表去谈，谈的是三年账，不是今年这一单。

变体三：跨项目横向比。同一个供应商在不同项目上的报价拉到一起，看它给你的价是不是统一的。这一变体做完，你会很清楚自己在对方那里的议价位置。

老兵说：比价这件事，算术只占三成，剩下七成是口径。口径没统一就比出来的排名，比不比还危险，因为它给了你一个错误的确定感。

场景 45 来料检验数据整理：从检验记录到趋势 ★★★

一句话背景

来料检验的记录散在三个地方：检验员手填的纸质单、班组自己维护的 Excel、还有质量系统里的导出表。字段不统一，判定结论有"合格"有"OK"有"放行"有"让步接收"四种写法。人工把三个月的记录整理成一份能看趋势的报告要一个工作日。这一趟实测四十五分钟，产出一张帕累托图和一份供应商横向对比。

名词小词典

来料检验 (IQC): 外购件进厂时做的那道检查。检查通过才能入库上线。

计数型 / 计量型: 计数型是"合格/不合格"这种只能数个数的; 计量型是"尺寸 12.34 毫米"这种有具体数值的。两者算出来的东西完全不同, 不能混。

不良模式: 不合格的具体表现是什么, 比如"尺寸超差""外观划伤""包装破损"。它跟"哪个零件"是两回事。

让步接收: 这批货其实不完全合格, 但因为生产急用, 经过审批先收下用。它是有代价的, 必须在报告里单独算。

CPK: 过程能力指数。通俗讲, 这个供应商做出来的东西, 尺寸稳定到什么程度。数值越大越稳, 一般要求 1.33 以上。

开工前准备

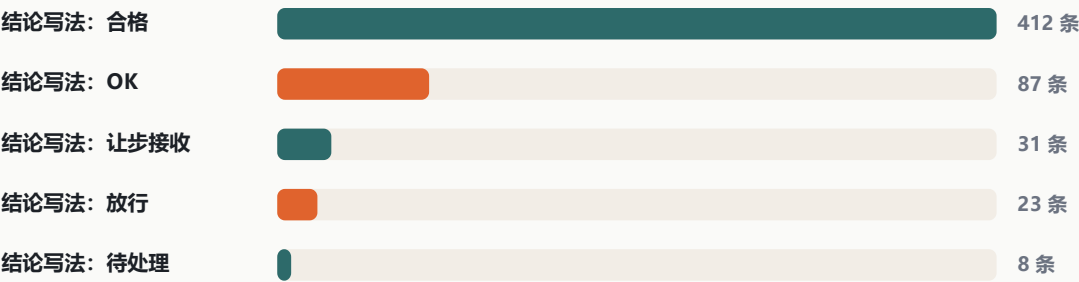
1. 三个数据源的原始文件, 放在一个目录下, 保留原文件名。
2. 一份物料清单: 哪些物料号属于这个品类, 每家的供应商是谁。
3. 一份不良模式清单。如果你自己有标准分类就给 AI, 没有就让它先聚类再让你确认。
4. 抽样方案说明: 每批抽多少、怎么判。不同抽样方案的批次不能直接比合格率。
5. CPK 要算的话, 样本量至少 25 件, 且是计量型数据。凑不够就不要算。
6. 输出目录 `D:\AI-SUPPLY\run03`。

操作步骤

1. 先让 AI 把三个数据源的字段名和各自的取值列出来, 不做合并。这一步让你看清脏在哪。
2. 让它把判定结论的写法去重列一遍, 你确认哪些算合格、哪些算不合格、哪些算让步接收。
3. 确认后让它做字段归一化, 输出一张合并表, 每一行一批次, 带数据来源列。
4. 让它按不良模式做帕累托, 出图。
5. 让它按周做趋势, 标出连续上升或连续下降的。
6. 让它按供应商横向比, 标出跑得最不一样的那一家。
7. 计量型的字段单独算 CPK, 样本量不够的如实报"样本量不足"。
8. 最后让它列三条最值得查的问题, 每条写清依据和下一步该看什么数据。

来料检验：从判定写到帕累托

三个来源的结论写法先去重归类，再做帕累托与趋势



帕累托累计占比 80% 线落在第 3 种不良模式；趋势按周看 13 周，连续 3 周升降单独标出

图 14-5 来料检验的帕累托与趋势：不良模式排序、周度走势、供应商对比

图 14-5 作者根据公开资料绘制。

可复制提示词

【任务】把三个来源的来料检验记录合并成一份能看趋势的分析报告。

【输入】

记录目录: D:\AI-SUPPLY\run03\iqc\

物料清单: D:\AI-SUPPLY\run03\items.csv (含物料号、名称、供应商)

时间范围: 最近 3 个月

【第一步: 先列差异, 不许合并】

把三个数据源各自的字段名、字段取值样例、行数分别列出来。

判定结论那一列, 把出现过的全部取值去重列一遍, 并告诉我每种出现了多少次。

列完停下报给我, 我确认归类之后再往下做。

【第二步: 归一化】

输出 D:\AI-SUPPLY\run03\iqc_merged.csv, UTF-8 带 BOM。

统一字段: 日期、班次、物料号、供应商、批次号、检验项、

检验类型(计数型/计量型)、实测值或判定、结论、数据来源。

结论只许取三个值: 合格 / 不合格 / 让步接收。

让步接收必须单独成一档, 不许并入合格, 也不许并入不合格。

归类不确定的行, 结论写"待确认"并保留原文, 不许猜。

【第三步: 帕累托】

按不良模式统计不合格批次数, 降序排, 算出累计占比。

画出帕累托图(柱加折线), 存为 D:\AI-SUPPLY\run03\pareto.png。

图里标出累计占比达到 80% 的那条分界线, 以及分界线左侧有几种模式。

【第四步: 趋势】

按周统计不合格批次率, 画出 13 周的折线, 存为

D:\AI-SUPPLY\run03\trend.png。

标出连续 3 周上升的区间和连续 3 周下降的区间, 各写一句可能的原因方向。

原因方向必须标注"这是待验证的猜测", 不许写成结论。

【第五步: 供应商横向】

按供应商统计: 检验批次总数、不合格批次数、不合格率、

让步接收次数、主要不良模式(取前二)。

标出跑得最不一样的那一家, 写清你的判断依据用了哪几个数字。

【第六步: CPK】

对计量型检验项, 按供应商分别算 CPK。

样本量少于 25 的, 一律写"样本量不足", 不许照算。

算出 CPK 的, 同时报出样本量、均值、标准差。

【第七步: 报告】

输出 D:\AI-SUPPLY\run03\report.md。

列出三条最值得查的问题, 每条写: 现象、涉及哪几家哪几个物料、

依据的数字、下一步该调什么数据来看。

【纪律】

不同抽样方案的批次，在横向对比时要单独标出来，不许混在一起算合格率。
让步接收的批次必须在每一张图表里都能单独看见。
所有“可能的原因”一律标为猜测，不许写成结论。

过程与判断点

第一，判定结论的四五种写法有没有被正确归类。我这趟的实际数据是：合格 412 条、OK 87 条、放行 23 条、让步接收 31 条、待处理 8 条。它第一版把“放行”归进了不合格，因为它在 Excel 里跟不合格行挨着。归类错了之后后面全错。所以我把“去重列一遍每种出现多少次”写成了第一步。

第二，让步接收有没有被单独留着。让步接收的批次是最有信息量的一批：它说明这批货有问题，但因为排产压力用了。把它们并进合格，趋势图就永远好看，也就永远没用。

第三，帕累托的 80% 线落在第几种模式上。我这趟落在第三种上，说明前三种不良模式占了八成，改善资源就该砸在这三项上。如果落在第八种上，说明不良模式太分散，问题在更上游，这时候要回头查供应商的制程能力而不是查检验。

第四，CPK 的样本量。它第一次算的时候，有个供应商只有 9 件数据，它也照样给了个 CPK 值出来。那个数毫无意义。所以在提示词里写死“少于 25 件一律写样本量不足”。

来料检验四类数字

每一类数字都有它自己的纪律

不合格批次率 结论只许三档：合格、不合格、让步接收，另加待确认	不良模式占比 帕累托 80% 线落在第 3 种，改善资源砸在前三项
让步接收批次 并进合格，趋势图就永远好看，也就永远没用	CPK 样本量少于 25 一律写样本量不足，一般要求 1.33 以上

不同抽样方案的批次要分组展示，不许混在一起算合格率

图 14-6 来料检验四类数字：批次率、模式占比、让步数、CPK

图 14-6 作者根据公开资料绘制。

翻车清单

翻车一：计数型和计量型混着算。症状是它对一个"合格/不合格"字段算出了均值和标准差。补救：在归一化阶段就要求每一列标出检验类型，报告里两类分开算，不许交叉。

翻车二：不同抽样方案混在一起比合格率。症状是某家供应商看起来不合格率特别高，其实只是它的抽检比例高、抽得更严。补救：抽样方案作为一列带进合并表，横向对比时分组展示。

翻车三：擅自合并不良模式。症状是"尺寸超差"和"尺寸偏小"被并成一类，或者"外观划伤"和"外观脏污"被并成一类，于是帕累托图看不出真问题。补救：让它把归类结果先给你看一遍，你确认了再出图，并要求每一类后面附两条原始记录原文做样例。

验收标准与变体

验收五条：

1. 合并表的行数等于三个数据源行数之和，没有凭空多出也没有少。
2. 结论列只有三种取值加"待确认"，没有其他自造词。
3. 随机挑三行，照"数据来源"打开原始记录核对，三个全对。
4. 每一张图里让步接收都能单独看见，没有被并进任何一类。
5. CPK 那一栏，样本量不足的确实写了"样本量不足"，没有硬算出一个数。

变体一：月度供应商评分。把这套分析每月跑一次，存基线，输出一张供应商月度评分表。评分权重你自己定，让 AI 只做计算不做权重设计。

变体二：新物料导入期跟踪。一个新物料导入的前三个月，每周跑一次，重点看趋势而不是看绝对值。导入期的趋势比稳态的绝对值有信息量得多。

变体三：把结果变成索赔依据。让步接收的次数和批次号整理成一张表，配上发生日期和物料号，就是跟供应商谈索赔时的原始材料。这件事以前要翻三个系统，现在一次跑完。

老兵说：来料检验数据最容易被做成一张好看的月度报表。好看的报表通常是让步接收被并进了合格，或者不良模式被并成了"其他"。凡是出现"其他"占比超过百分之十的帕累托图，都要重做。

场景 46 产线异常记录分析：把零散的异常记录变成问题排序 ★★★

一句话背景

产线上的异常记录有三个来源：工位旁的纸质记录本、班组群里的即时消息、还有设备系统的报警导出。写的人不同，写法完全不同，一条记录常常只有一句"换型后第二件不合格"。人工把这些整理成一份能排优先级的问题清单要两个工作日。这一趟实测一小时，产出一张加权排序表和五个待查问题。

名词小词典

停线时长：从产线停下来到重新开动之间的分钟数。它是异常严重程度最硬的一个指标，比"发生了几次"更

有用。

影响数量：这次异常造成了多少件产品受影响，包括已经做出来的和需要返工的。

人机料法环（5M1E）：异常归因的六个方向，人是操作者、机是设备、料是物料、法是工艺方法、环是环境、测是测量。它是分类用的架子，不是结论。

现象与原因：现象是你看到的（"第二件不合格"），原因是造成它的（"换型后的首件确认流程没执行"）。把现象当成原因是这类分析最常见的错误。

加权排序：给每个异常按几个维度打分再排序，而不是只按发生次数排。次数多的不一定是该先解决的。

开工前准备

1. 三个来源的原始记录，纸质记录拍照或录入后放在同一目录。
2. 一份班次与日期的对照表。跨班次、跨日期的记录不对照会串。
3. 三个维度的权重：停线时长、影响数量、发生次数各占多少。权重你来定，AI 不许自己定。
4. 一份工位清单，每个工位的标准节拍是多少。没有这个数，停线时长换算不成损失产量。
5. 输出目录 `D:\AI-SUPPLY\run04`。

操作步骤

1. 让 AI 把三个来源的记录各列十条样例，不做整理。先看清楚脏到什么程度。
2. 让它把每条记录拆成结构化字段：时间、班次、工位、现象原文、停线时长、影响数量、临时处置。
3. 缺字段的记录单独成表，不许填空，也不许删。
4. 让它按 5M1E 做第一轮归类，归类结果给你确认。
5. 确认后按你给的权重做加权排序，出一张 Top 20 的表。
6. 让它对 Top 5 各写一份"下一个该查什么"，指明要调什么数据、看什么记录。
7. 让它出一张周度热力图：星期 × 班次，看异常有没有时间规律。
8. 你自己核对 Top 5 里的每一条，确认它没把现象写成原因。

异常记录的三段收敛

三个来源：纸质记录本、班组群消息、设备报警导出



现象和原因分开写，原因一律标待验证假设，不许写成定论

图 14-7 异常记录的三段收敛：结构化、归类、加权排序

图 14-7 作者根据公开资料绘制。

可复制提示词

【任务】把三个来源的产线异常记录，整理成一份能排优先级的问题清单。

【输入】

记录目录：D:\AI-SUPPLY\run04\logs\

工位清单：D:\AI-SUPPLY\run04\stations.csv（含工位号、标准节拍秒数）

时间范围：最近 2 个月

权重（我定的，你不许改）：停线时长 50%、影响数量 30%、发生次数 20%

【第一步：样例】

把三个来源的记录各列十条原文样例，不做任何整理。

列完停下报给我。

【第二步：结构化】

输出 D:\AI-SUPPLY\run04\exc_main.csv，UTF-8 带 BOM。

字段：日期、班次、工位号、现象原文、停线时长分钟、影响数量、临时处置、记录来源。

停线时长和影响数量缺的，写“未记录”，不许填 0，不许估算。

一条记录里写了多个现象的，拆成多条，每条保留同一原始序号。

【第三步：缺字段清单】

输出 D:\AI-SUPPLY\run04\exc_missing.md。

列出缺停线时长或影响数量的记录条数与占比，

并按工位分组，指出哪几个工位的记录质量最差。

这一份是给我去改记录规范的，不是给你填空的。

【第四步：归类】

按 5M1E 六个方向做第一轮归类：人、机、料、法、环、测。

每一类后面附三条现象原文做样例。

归不进任何一类的，单独成一类叫“待定”，不许硬塞。

归类结果先给我看，我确认了你再往下做。

【第五步：加权排序】

按我给的权重算加权分，输出 Top 20 的表格，

列：排名、现象描述、工位、归类、停线总时长、影响总数量、发生次数、加权分。

同一根因的多条记录要合并成一行，合并依据写进备注列。

合并这件事你只能提出建议，最终哪些合并由我确认。

【第六步：下一步清单】

对 Top 5 每一条，写一份“下一个该查什么”：

要调什么数据、看哪段时间的什么记录、找谁问、

预期看到什么现象能证实或推翻当前假设。

每一份不超过五句，不许写“深入分析原因”这类空话。

【第七步：时间规律】

按“星期 × 班次”统计异常次数，输出一张热力图

D:\AI-SUPPLY\run04\heatmap.png,
并指出是否存在明显的集中时段，写清依据的数字。

【纪律】

现象和原因必须分开写。你能看到的是现象，
原因一律标"待验证假设"，不许写成定论。
权重是我的，不许调整，也不许因为某类数据不好看就换算法。
所有数字要能追到原始记录的原始序号，我要抽查五条。

过程与判断点

第一，缺字段的比例。我这趟跑出来，停线时长缺了四成多。这个数字本身就是一个结论：不是产线没停过，是没人记。所以我把"缺字段清单"单独做了一步，它是给管理层看的证据，用来推动改记录规范，价值不比 Top 20 那张表低。

第二，它有没有把现象写成原因。它第一版 Top 1 写的是"操作员未按作业指导书执行，导致换型后首件不合格"。这句话里前半句是它猜的，记录在纸上只有后半句。我在提示词里加了"现象和原因必须分开写"之后，第二版变成"现象：换型后第二件不合格，发生 14 次；待验证假设：换型后首件确认流程未执行，验证方法是调取该工位近两周的换型记录与首件确认签字表"。

第三，合并是否合理。它有次把"供料不及时"和"缺料停线"合并了，其实前者是上工位的问题，后者是本土位的。合并之后责任工位就找错了。所以合并让它只提建议，最终由我确认。

第四，热力图有没有真的规律。我这趟发现周一早班异常明显高，比周均值高出不少。顺着查下去是周末库存周转和周一开工准备时间不足。这个规律靠肉眼两千条记录是看不出来的。

翻车清单

翻车一：只按发生次数排序。症状是排在第一的是"标签贴歪"这种发生五十次但每次三十秒的问题，而"设备故障停线两小时"排到第十。补救：强制三维加权，权重由人定，且停线时长给最高权重。

翻车二：缺字段被填成 0。症状是停线时长那一列一片 0，于是所有异常看起来都不严重。补救：提示词里写死"缺的写未记录，不许填 0，不许估算"，并在验收时抽查原始记录。

翻车三：跨班次记录串位。症状是夜班的记录被算到了白班，热力图因此得出错误结论。补救：结构化阶段就要求日期和班次两个字段同时存在，缺班次的单独列出来让你补。

验收标准与变体

验收五条：

1. 结构化表的行数与原始记录条数对得上，拆分的多条有备注说明。
2. 缺字段的记录确实写着"未记录"，抽查五条原始记录核对。
3. Top 20 表里每一行的数字，能追到一个或多个原始序号，抽查五条全对。
4. Top 5 的"下一个该查什么"里，每条都点名了具体要调的数据和找谁。
5. 归类结果里"待定"那一类非空。它一类都归不进待定，说明它在硬塞。

变体一：新线导入期跟踪。新产线跑的前两个月每周跑一次，重点看归类分布的变化。导入期问题从"人"和"法"为主逐渐转向"机"，这是正常的，一直不变反而不正常。

变体二：改善效果验证。改善措施落地之后，用同一套提示词再跑一次，对比 Top 20 的排名和加权分变化。降下去的说明有效，没动的说明措施没落到实处。

变体三：把缺字段清单变成记录规范。第三步产出的那份清单，直接改成新的异常记录表单字段。填什么、必填哪几项，用数据说话比开会吵有用。

老兵说：产线上发生次数最多的问题，通常不是最该先解决的问题。排第一的应该是停线时间最长的那个，因为产线停着的每一分钟都在烧钱，而贴歪一张标签不烧。

场景 47 产能与节拍数据梳理：找出瓶颈在哪 ★★

一句话背景

产量上不去，所有人都说要加设备。但在加设备之前得先知道瓶颈在哪。各工位的实测工时散在班组的 Excel 里，换型记录在一张纸上，停机记录在系统里。把这三个凑成一张产能表，人工要一个工作日。这一节标 ★★，这套流程我跑通过，但产线数据在我手上留档不完整，你跑出来要自己复核一遍。

名词小词典

节拍 (Takt Time)：为了满足客户需求，每隔多久必须出一件产品。它由需求决定，不由设备决定。

周期时间 (Cycle Time)：某个工位实际做一件要花多久。节拍是你想达到的，周期时间是你实际做到的。

瓶颈工序：整条线上周期时间最长的那个工位。整条线的产能由它决定，改别的工位都没用。

OEE：设备综合效率，等于可用率乘性能效率乘合格品率。三个因子分开看才有意义，只看总数是看不出问题的。

换型时间：从做上一个型号切换到做下一个型号，产线要停多久。多品种混线生产时，这一项常常比瓶颈还可怕。

开工前准备

1. 一份工位清单：工位号、工位名、标准工时、人数。
2. 各工位的实测工时数据。人工掐表测的话，每个工位至少测 30 个周期，少于这个数不可信。
3. 换型记录：每次换型的起止时间、换的是什么型号。
4. 停机记录：每次停机的起止时间和原因。
5. 一段时期内的合格品数与投入数，用来算合格品率。
6. 客户需求的节拍数。没有这个数，你算不出该不该加设备。

7. 输出目录 D:\AI-SUPPLY\run05。

操作步骤

1. 让 AI 先把三个数据源的时间跨度列出来，确认它们是不是同一段时间。时间段对不上是最常见的数据事故。
2. 让它对每个工位算周期时间的中位数、均值、最大值、最小值、样本量。
3. 让它同时报均值和中位数。两个数差得远，说明数据里有极端值，这时候用中位数。
4. 让它画一张工位周期时间柱状图，把瓶颈标出来。
5. 让它算瓶颈工位的理论产能，和客户需求的节拍对比，得出缺口。
6. 让它分解 OEE 的三个因子，指出损失主要来自哪一块。
7. 让它算换型损失：总换型时间占可用时间的比例。
8. 最后让它给出一到三个改善方向，每个方向写清预计能把瓶颈降到多少、需要什么投入。

产能梳理五步

加设备之前，先花一天把瓶颈找出来

- 1 统一时间范围**
三个数据源取交集，报出各丢了多少条
- 2 周期时间统计**
均值与中位数都报，相差超 15% 取中位数
- 3 定位瓶颈工位**
算理论产能，与需求节拍对比得出每日缺口
- 4 分解 OEE 三因子**
拆可用率、性能效率、合格品率，找最小的
- 5 换型损失与改善**
换型占可用时间比例；改完瓶颈转到哪个工位

样本量少于 30 的工位标注样本量不足，不许跟其他工位混着排名

图 14-8 产能梳理五步：时间统一、周期统计、瓶颈定位、OEE 分解、改善测算

图 14-8 作者根据公开资料绘制。

可复制提示词

【任务】把三个数据源凑成一张产能分析表，找出瓶颈工序并给出改善方向。

【输入】

实测工时目录：D:\AI-SUPPLY\run05\cycle\

换型记录：D:\AI-SUPPLY\run05\changeover.csv

停机记录：D:\AI-SUPPLY\run05\downtime.csv

工位清单：D:\AI-SUPPLY\run05\stations.csv

产出数据：D:\AI-SUPPLY\run05\output.csv (含日期、投入数、合格品数)

客户需求节拍：每 45 秒出一件（示例值，按你的实际改）

【第一步：时间范围统一】

分别报出三个数据源的起止日期与记录条数。

三者时间范围不一致的，取交集，并告诉我各丢了多少条。

不许在不一致的时间段上做计算。

【第二步：周期统计】

对每个工位算：样本量、均值、中位数、最大值、最小值、标准差。

均值和中位数相差超过 15% 的工位单独标出来，

这些工位以中位数为准，并在备注里写明原因。

样本量少于 30 的工位，标注“样本量不足，结论仅供参考”。

【第三步：瓶颈定位】

画一张柱状图，横轴工位、纵轴周期时间中位数，

标出最长的那一根，存为 D:\AI-SUPPLY\run05\bottleneck.png。

算出瓶颈工位的理论小时产能与理论班产能（按 8 小时算），

与客户需求节拍对比，算出每日缺口件数。

【第四步：OEE 分解】

算可用率、性能效率、合格品率三个因子，再算 OEE。

三个因子分别列出，指出最小的那一个，

并对最小的那一个再往下拆一层原因（用停机记录或产出数据）。

【第五步：换型损失】

算总换型时间、换型次数、平均每次换型时长，

以及换型时间占计划生产时间的比例。

占比超过 10% 的，单独写一段：如果通过改善把平均换型时长降一半，

每天能多出多少可用时间、折算成多少件产能。

【第六步：改善方向】

给出一到三个改善方向，每个方向写四件事：

改哪个工位、预计把周期时间降到多少、需要什么投入、

这个改善之后新的瓶颈会转移到哪个工位。

第三个改善方向如果会引出新瓶颈，必须写出来。

【纪律】

不许把均值当中位数用，也不许反过来。

样本量不足的工位，结论必须带标注，不许跟其他工位混在一起排名。
改善测算里的“预计降到多少”必须写明是估算，并给出估算依据。
不许因为某个方向投入大就自行排除，投入大小由我判断。

过程与判断点

第一，均值和中位数差多少。我这趟有个工位的均值是 52 秒，中位数是 41 秒，差了两成多。原因是那天的记录里有三次设备卡料，每次两分多钟，把均值拉上去了。如果用均值，这个工位会被误判成瓶颈；用中位数，真正的瓶颈是另一个工位。所以我要求两个数都报。

第二，瓶颈会不会转移。它第一版给了三个改善方向，三个都指向同一个工位。我加了那句“第三个改善方向如果会引出新瓶颈必须写出来”之后，第二版才出现“改完之后瓶颈会转移到第三工位，届时该工位周期时间约 44 秒”。这句话很关键，它告诉你改善不是一次性的，是一条要一步步走的路。

第三，OEE 三个因子哪个最小。只看 OEE 那个总数没有用。可用率低要查停机，性能效率低要查节拍和设备速度，合格品率低要查质量。三个方向的改善团队完全不同。

第四，换型损失的占比。多品种混线的产线上，这一项常常被忽略。我这趟算出来换型占了近两成可用时间，比瓶颈工位的问题还大。如果只看工位周期时间，永远不会看到它。

翻车清单

翻车一：三个数据源时间对不上。症状是算出来的产能忽高忽低，找不出原因。补救：第一步就强制报出各自的时间跨度，取交集，并明确告诉你各丢了多少条。

翻车二：用均值找瓶颈。症状是找到的“瓶颈”其实只是那天卡了三次料的工位。补救：均值和中位数都报，相差超过 15% 的以中位数为准。

翻车三：改善完瓶颈转移没被发现。症状是花了钱改了 A 工位，产能一点没涨，因为瓶颈已经悄悄到了 B 工位。补救：每一个改善方向都必须回答“改完之后新的瓶颈在哪”。

验收标准与变体

验收五条：

1. 三个数据源的时间范围一致，取交集时丢了多少条有明确数字。
2. 每个工位的样本量都写出来了，少于 30 的有标注。
3. 瓶颈工位的柱状图里，最长的那一根你自己掐表复核一次，误差在你接受的范围内。
4. OEE 三个因子分开列出，最小的那个有往下拆的原因。
5. 每个改善方向都回答了“改完之后瓶颈在哪”这个问题。

变体一：新线产能测算。产线还没建的时候，用各工位的标准工时和预估换型时间算一条理论产能。这时候的数据是估算的，要在每一行标注“估算值”，别把它当实测数据用。

变体二：多品种混线排产。不同型号的周期时间不同，按型号分别算瓶颈。你会发现同一条线做 A 型号时瓶颈在第二工位，做 B 型号时在第五工位，排产策略因此完全不同。

变体三：把节拍数据接进日报。每天自动跑一遍，输出一张"今天的瓶颈是谁、跟昨天比变化了多少"的一页纸。这件事的价值不在分析，在让瓶颈每天被看见一次。

老兵说：加设备是最贵的解决办法，也是最容易被提出来的。在签字买设备之前，先花一天把瓶颈找出来，很多次你会发现瓶颈不是设备，是换型。

场景 48 质量问题的 8D 报告辅助：从事实到报告 ★★★

一句话背景

客户投诉来了，八小时内要出 8D 报告的 D1 到 D3。最耗人的不是分析，是把一堆零散的事实整理成一份结构完整、每句话都能追溯的报告。人工写一份完整的 8D 要一个半工作日。这一趟实测一小时出到 D3，D4 之后的部分它给候选，判断由你做。

名词小词典

8D：八步问题解决法。D1 组建团队、D2 描述问题、D3 临时遏制措施、D4 根本原因、D5 永久纠正措施、D6 验证有效性、D7 预防再发、D8 总结。汽车行业客户投诉的标准回复格式。

5W2H：描述问题的七个维度。什么物料、什么时候、在哪里、谁发现的、发生什么、多少件、严重程度。D2 缺一项就不完整。

发生原因与流出原因：发生原因是"为什么会做出不良品"，流出原因是"为什么没被拦住流到客户手上"。8D 里这两条要分别回答。

临时措施与永久措施：临时措施是"现在先把问题按住"，比如全检、隔离、换货；永久措施是"以后不会再发生"，比如改工艺、改模具。混淆这两者是 8D 最常见的硬伤。

5Why：对一个问题连问五次为什么，追到可以采取那一层。问到"员工疏忽"就停下来的，等于没问。

开工前准备

1. 客户投诉的原始邮件或系统记录，保留原文。
2. 涉及批次的追溯信息：生产日期、班次、工单号、物料批次号。
3. 已有的检验记录、返修记录、同类问题的历史记录。
4. 团队名单和职责。D1 要填。
5. 一份你们公司的 8D 模板。有模板就给 AI，让它按模板填，不要让它自己发明格式。
6. 输出目录 `D:\AI-SUPPLY\run06`。

操作步骤

- 1. 先把客户投诉原文和追溯数据给它，让它只做 D2：把事实整理成 5W2H 七项齐全的描述。
- 2. 让它把 D2 里每一个数字标出来源。标不出来源的数字，说明这个事实还没拿到。
- 3. D2 确认后再做 D3：临时遏制措施，要求每条措施能说清"谁、什么时候、做到什么程度"。
- 4. D4 让它给候选根因，按发生原因和流出原因两条线分别列，每条标注需要什么证据来证实或推翻。
- 5. D5 让它把措施写成可验收的句子：改什么、改成什么、怎么确认改到位了。
- 6. D6 让它设计验证方案：看什么数据、看多长时间、达到什么标准算通过。
- 7. 最后让它对整份报告做一次自检，按一张检查表逐项打勾。

8D 八步：机器接四步，人签四步

八小时内出到 D3，D4 之后它给候选，判断由人做

AI 能接的四步

- | D2 描述问题
按 5W2H 七项整理事实，每项后面跟数据来源
- | D3 临时遏制
每条写做什么、谁负责、何时完成、什么程度算完成
- | D4 根因候选
发生原因与流出原因两条线，标注支撑到什么程度
- | D5 与 D6 候选
措施写成可验收句子，验证标准要是个数字

必须由人签的四步

- | D1 组建团队
名单与职责由你给，它替不了
- | D4 定根因
从候选里选哪一条，判断由你做
- | D7 预防再发
落到作业指导书、控制计划、PFMEA 哪一份，你定
- | D8 结论与放行
签字的人是你，责任也归你

追到员工疏忽就停下来的根因直接删掉重写；客户描述的现象一律用原文

图 14-9 8D 八步里 AI 能接的四步与必须由人签的四步

图 14-9 作者根据公开资料绘制。

可复制提示词

第一份用来出 D2 和 D3，八小时内要交的那部分：

【任务】把这次客户投诉的事实整理成 8D 报告的 D2 和 D3。

【输入】

投诉原文: D:\AI-SUPPLY\run06\complaint.txt

追溯数据: D:\AI-SUPPLY\run06\trace.csv (含工单号、生产日期、班次、物料批次)

历史同类问题: D:\AI-SUPPLY\run06\history.csv

【D2: 问题描述】

按 5W2H 七项写, 一项一行:

1. 什么物料 (物料号与名称)
2. 什么时候发现的 (客户侧日期与我方生产日期)
3. 在哪里发现的 (客户工序或位置)
4. 谁发现的
5. 发生了什么现象 (用客户原文, 不许你改写)
6. 多少件 (投诉数量、涉及批次数量、我方库存数量)
7. 严重程度 (客户的要求与影响)

每一项后面加一列“数据来源”, 写清这个数字或事实来自哪个文件的哪一段。

七项里任何一项拿不到数据的, 写“待补充”, 不许推测、不许写“暂无”。

【D2 补充: 范围界定】

单独写一小段: 已确认受影响的批次有哪些、

需要排查但还未确认的批次有哪些、

已确认不受影响的有哪些, 各自写清判断依据。

【D3: 临时遏制措施】

输出 D:\AI-SUPPLY\run06\d3.md。

每一条措施写四件事: 做什么、谁负责、什么时候完成、

做到什么程度算完成 (要有一个可确认的状态)。

措施按三类分别列: 客户侧、在途与库存、在制品与产线。

不许写“加强检验”“提高警惕”这类无法确认的句子。

【纪律】

客户描述的现象一律用原文, 不许你归纳成技术语言。

你有技术判断的地方, 另起一段标注“我方技术判断”, 与事实分开。

任何数字标不出来源的一律写“待补充”。

第二份用来做 D4 之后的根因与措施, 注意它给的是候选:

【任务】这次 8D 的 D4 到 D7，你给候选方案，判断由我做。

【D4：根本原因候选】

分两条线列，一条线是发生原因，一条线是流出原因，两条不许混。

每条线给三到五个候选，每个候选写三件事：

这个候选的内容、支持它的证据、推翻它需要什么证据。

按 5Why 追，追到可以采取的措施的那一层为止。

追到“员工疏忽”“操作不当”“培训不足”就停下来的候选，直接删掉重写。

每个候选标注：现有证据能支撑到什么程度（已证实 / 有迹象 / 纯推测）。

【D5：永久措施候选】

对每个已证实的根因，给一到两个永久措施候选，

每条写成可验收的句子：改什么、改成什么、什么时候改完、

用什么来确认改到位了（要有一个可测量的东西）。

不许写“加强管理”“优化流程”这类无法验收的句子。

【D6：验证方案】

对每个永久措施，设计验证方案：

看什么数据、看多少个批次或多长时间、

达到什么标准算通过、达不到怎么办。

标准要是个数字，不许写“明显改善”。

【D7：预防再发】

列出这次的教训要落到哪些文件上：作业指导书、检验规范、

控制计划、PFMEA、培训材料，逐条写清改哪一处、改成什么。

不确定该落到哪个文件的写“待确认”，不许随便塞一个。

【自检】

按下面这张表逐项自检，每项打勾或打叉，打叉的写清缺什么：

D2 七项是否齐全且每项有数据来源；

D3 每条措施是否有负责人和完成状态；

D4 是否分开写了发生原因与流出原因；

D4 是否出现了员工疏忽类的无效根因；

D5 每条措施是否可验收；

D6 的通过标准是否是一个数字；

D7 是否点名了具体文件。

自检结果原样输出，不许只报“全部通过”。

过程与判断点

第一，D2 的七项里有没有“待补充”。有“待补充”是好现象，说明它在如实报告。我最怕的是七项全填满了，其中两项是它猜的。所以验收时我专门抽查那些填满了的数字，看来源列写的是不是真的能追到。

第二，客户原文有没有被改写。客户写的是“装车时发现异响”，它第一版写成了“客户端反馈产品 NVH 性能不达标”。这是两种完全不同的描述，前者是现象，后者是我方的技术判断，而且这个判断还没验证。后来我要求“现象用原文，技术判断另起一段标注”。

第三，D4 有没有追到员工疏忽就停。它第一版给的第二个候选根因是"操作者未按作业指导书操作"。这句话没法采取措施，因为你能做的只有"再培训一遍"，而培训解决不了流程问题。我在提示词里写死这类候选直接删掉重写，第二版它给的是"作业指导书里该工位的自检项未写明判定标准，操作者无法判断"，这才是一个能改的根因。

第四，发生原因和流出原因有没有分开。这两者分属不同的责任方，也对应完全不同的措施。混在一起写，客户一定会退回报告。

第五，自检表有没有真的打叉。它第一版自检全勾。我加了那句"不许只报全部通过"，第二版出现两个叉：D6 标准不是数字、D7 没点名文件。这两个叉就是我要改的地方。

翻车清单

翻车一：把猜测写成结论。症状是 D4 里出现"根本原因是某某"，但证据列里只有一条间接线索。补救：强制每个候选标注支撑程度，并要求写明"推翻它需要什么证据"。一条写不出推翻条件的候选，说明它不是可验证的假设。

翻车二：临时措施和永久措施混淆。症状是 D3 里写了"修改作业指导书"，而作业指导书改完要两周，客户那边明天就断线了。补救：D3 只写能在当天到三天内执行完的动作，长期动作一律放到 D5。

翻车三：措施写得无法验收。症状是 D5 里出现"加强过程控制""提升员工质量意识"这类句子。补救：要求每条措施写成"改什么、改成什么、用什么确认改到位"，确认的东西必须是一个能测的数或者一个能查的记录。

验收标准与变体

验收六条：

1. D2 七项齐全，每项都有数据来源列，抽查三项能追到原始文件。
2. 客户描述的现象是原文，没有被改写成技术语言。
3. D3 的每条措施都有负责人、完成时间和可确认的完成状态。
4. D4 分开写了发生原因与流出原因，且没有出现员工疏忽类的无效根因。
5. D6 的每条通过标准都是一个数字。
6. 自检表里至少有一个叉，或者你能确认那张表它真的逐项查过。

变体一：内部质量问题的 8D。客户投诉之外的内部重大质量问题，同样走这八步。区别在 D3 的范围界定：客户侧那一类换成"客户端库存与在途"，内部问题换成"厂内库存与在制品"。

变体二：8D 退回重做的自查。客户退回来的 8D，用第二份提示词里的自检表跑一遍，看看被打回的原因是不是就在这七项里。大多数退回都出在 D4 根因无效和 D6 标准不是数字这两项上。

变体三：把历年 8D 做成一张问题地图。把过去两年的 8D 报告全部喂进去，让 AI 按根因分类统计，看哪些根因反复出现。反复出现的那几个，说明你们的预防措施从来没真正落到文件上。

老兵说：8D 里最贵的一栏是 D4 根因。根因写错了，后面 D5 到 D7 全跟着错，而且错得特别整齐，看起来还挺像一份好报告。

小白词典

SOR (Statement of Requirements): 项目发给供应商的要求文件, 写清要什么、要多少、要达到什么标准。类比: 你给装修队的那张需求清单, 后面所有验收都照它来。

RFQ (Request for Quotation): 询价文件。你正式问供应商"这个东西多少钱"的那封信。类比: 装修前的报价邀请。

IATF 16949: 汽车行业的质量管理体系认证。类比: 一家餐厅的食品安全等级, 有等级不代表每道菜都好吃, 但没等级你不太敢进。

ISO 17025: 实验室认可。类比: 体检机构有没有资质, 没资质的体检报告医院不认。

认可范围: 证书上写的具体能做什么。类比: 驾照写的是 C1, 就不能开大货车。

PPAP (生产件批准程序): 量产前把样件、检测报告、工艺文件全套提交给客户批准的一套流程。类比: 正式上岗前的一次全面考核, 过了才让你上工位。

最小起订量 (MOQ): 供应商愿意接单的最小数量。类比: 面包店整批起订, 你买一个它不卖。

阶梯价: 买得越多单价越低的价格表。类比: 批发价和零售价的差别, 只不过它有很多档。

模具费分摊: 开模具的一次性投入摊到每件产品上。类比: 买一台打印机的钱, 除以你这辈子要打的张数。

账期: 货到之后多少天付款。类比: 信用卡的免息期, 越长你手上的钱越活。

来料检验 (IQC): 外购件进厂时的那道检查。类比: 收快递时先验货再签收。

让步接收: 货不完全合格但生产急用, 经审批先收下。类比: 衣服有个小瑕疵, 你急着穿就先留下了, 但心里记着这笔账。

CPK (过程能力指数): 做出来的东西尺寸稳定到什么程度, 数值越大越稳, 一般要求 1.33 以上。类比: 投篮命中率的稳定性, 进得多不算, 每场都进才算。

8D: 八步问题解决法, 汽车行业客户投诉的标准回复格式。类比: 一份事故调查报告的固定模板, 八栏都得填。

5W2H: 描述问题的七个维度。类比: 报案时要说的那几件事: 什么事、什么时候、在哪、谁、怎么了、多少、多严重。

发生原因 / 流出原因: 前者是为什么会做出不良品, 后者是为什么没被拦住。类比: 为什么会着火, 以及为什么喷淋没启动。

5Why: 连问五次为什么往下追根因。类比: 孩子追问"为什么", 追到第五层你才真正答得上来。

节拍 (Takt Time): 为满足客户需求每隔多久必须出一件。类比: 收银台的速度必须跟上排队进入的速度。

周期时间 (Cycle Time): 某个工位实际做一件花多久。类比: 这个收银员实际扫一件商品要几秒。

瓶颈工序: 整条线上最慢的那个工位, 整线产能由它决定。类比: 高速公路上最窄的那一段, 前面修多宽都没用。

OEE (设备综合效率): 可用率乘性能效率乘合格品率。类比: 一台机器真正干活的时间, 占它本该干活的时间的比例。

换型时间: 从上一个型号切到下一个型号产线要停多久。类比: 收银台换班交接时暂停服务的时间。

帕累托 (Pareto): 按发生频次降序排, 找出占八成的那少数几类。类比: 抓主要矛盾, 先把最大的那几个窟窿堵上。

你的岗位怎么用

采购与供应商管理: 场景 43 和场景 44 是给你的。资质核查矩阵一次对完五家, 比价口径表十项逐条统一。我最想让你带走的是那张口径表, 尤其是年降条款和量产爬价那两行, 它们决定了三年后谁最便宜。

供应商质量工程师 (SQE): 场景 43 的认可范围那一列、场景 45 的让步接收单独成档, 这两处是给你留的。认可范围不到位而判合格的供应商, 会在量产爬坡的时候找上你。

质量工程师: 场景 45 和场景 48 直接抄。来料数据整理那套提示词里的"让步接收不许并进合格", 和 8D 提示词里的"员工疏忽类根因直接删掉重写", 这两句是我踩过坑之后加进去的。

生产工艺工程师: 场景 46 和场景 47 是你的。异常排序的权重你自己定, 但停线时长给最高权重这一条别改。产能那套里"改善之后瓶颈转到哪"那一问, 我建议你写进每一次改善提案。

项目经理: 盯场景 43 的结论页和场景 44 的三年总成本排名。定点会上最容易被问的两个问题, "为什么选这家"和"为什么不选最便宜那家", 答案就在这两份文件里。

成本与财务: 场景 44 的账期折算和资金成本年化利率是你的输入。给一个明确的利率数字, 比价表才成立。另外模具费一次性收还是分摊, 也请你一起定口径。

本章交付物

一张你自己的比价口径表, 加上一套资质核查矩阵模板。

做法分三步。

第一步, 把场景 44 里那张十项的口径表复制到你的笔记里, 逐行改成你行业和你公司的版本。至少补三行: 你们特有的费用项、你们常用的账期档位、你们客户要求的报价有效期。这张表改完打印出来, 比价的时候摊在键盘旁边。

第二步, 用场景 43 那份提示词跑一遍你手上的供应商资质, 把跑出来的矩阵 CSV 存成模板。矩阵的行(资质项)你往后只做增减, 不再重建。

第三步, 把两张表放进你团队的新人入职材料。

第一次验收的活: 挑下一次真实的询价, 用这张口径表跑一遍完整流程, 看三种排名是否一致。三个排名一致说明这几家的报价方式本来就相近; 不一致的那一次, 才是你真正赚到钱的那一次。

本章金句

报价单上最便宜的那一家, 常常不是成本最低的那一家。差的那些钱从来不写在单价那一列里。

资质核查的价值不在查出谁不合格，在查出谁"看起来合格"。

比价这件事算术只占三成，剩下七成是口径。口径没统一就比出来的排名，比不比还危险。

凡是"其他"占比超过百分之十的帕累托图，都要重做。

产线上发生次数最多的问题，通常不是最该先解决的问题。排第一的应该是停线时间最长的那个。

8D 里最贵的一栏是根因。根因写错了，后面几栏全跟着错，而且错得特别整齐。

（下一章：NVH 与试验。供应链这头算的是钱，下一章那头算的是声音和振动，同样一堆数字，真话藏在不同的地方。）

第 15 章

NVH 与试验：从一堆曲线里捞出真话

测得出来的是能量，听得出来的是烦。这两件事从来不在同一条刻度上，而签字的人只认后者。

那是好几年前的一个冬天，我在半消声室里待了两周，手头压着 30 多个电动执行器的声学样本。

任务是给这批产品建一个声学数据库：每个样本录一段音，测一组客观指标，再找一组人听一遍打分，最后把两边摊在一张表上，看哪些能放行。

第 17 号样本我记得很清楚。它的 A 计权声压级约 42 dB(A)，比 45 dB(A) 的目标线低了将近 3 分贝，1/3 倍频程逐段比对也没有一段超线。我盯着那张频谱图看了十分钟，觉得稳了。

然后听评组的结果回来了。六个人里四个给了不合格分，评语里出现最多的一个词是"毛刺"。

同一天还有第 23 号样本，A 计权声压级约 46.5 dB(A)，超线 1.5 分贝左右，六个人全票给了高分。评语写的是"干净、闷、不烦"。

那天下午我拿着这两张表去找声学实验室的同事老张。他扫了一眼，说了句我到现在还记得的话："你测的那条线，跟你耳朵听的那条线，它俩本来就不是同一条线。"

后来我们把第 17 号样本的时频图调出来，在 3 kHz 到 5 kHz 之间有一段很窄的能量，幅度不高，但它是断续冒出来的。总声压级是能量平均值，算不出来；响度和尖锐度能算出一部分；真正判它不合格的是听的人。

从那以后我给自己立了一条规矩：客观数据交给工具处理，主观结论我自己签。这两件事不归同一个人管。

这一章的六个场景，就是从那 30 多个样本里长出来的。（故事中的数字做过脱敏，量级与当时记录一致。）

下面是本章六个场景的速览，你先挑跟自己有关的那个看。

场景	场景名	人工耗时	AI 耗时	可信度
49	试验数据整理	约 2 人天	约 25 分钟	★★★ 实测
50	阶次分析与噪声源识别	约 1.5 人天	约 30 分钟	★★★ 实测
51	试验报告自动生成	约 1 人天	约 20 分钟	★★★ 实测
52	声学数据库搭建与检索	约 3 人天	约 40 分钟	★★★ 实测
53	微电机与齿轮传动件声学定位	约 2 人天	约 35 分钟	★★★ 实测
54	把老师傅的经验蒸馏成 AI 技能	约 6 个月（我的第一版）	约 3 小时	★★★ 实测

本章最重要的一条工程态度 客观数据过线，不等于主观感受能接受。AI 能帮你处理数据：它能读文件、能算阶次、能出图、能填报告模板。它不能替你做主观评价的判断。总声压级达标而听评全票否决的样本，我手里不止一个；总声压级超标而听评全票通过的样本，我手里也不止一个。这两个方向都可能出错，出错的代价由签字的人承担。所以本章所有场景里，凡是涉及“这个声音能不能接受”的结论，一律留给人。

场景 49 试验数据整理：从原始记录到可用的曲线与表格 ★★★

一句话背景

试验做完，你手上是一堆原始记录：采集软件导出的 CSV、设备自己存的二进制、手写的操作记录表、几十段录音。文件名是设备自动生成的，通道名每台设备都不一样，采样率也不统一。把它们整理成能进报告的形式，我做过最惨的一次是两个人干了两天半。烦不在难，在每一步都要人眼去核对，而人眼核对到第 40 个文件的时候就开始放水了。

名词小词典

原始记录：采集设备直接吐出来的数据文件，没经过处理。特点是真实，缺点是难用。

通道：一路传感器就是一条通道。一个麦克风加一个三向加速度计，合起来四条通道。

采样率：每秒采多少个点，单位 Hz。采样率不同的两份数据不能直接放在一起比。

时域 / 频域：时域是幅度随时间怎么变的那条曲线，频域是能量分布在哪些频率上的那张谱。同一份数据的两种看法。

CSV：逗号分隔的纯文本表格。任何软件都能打开，代价是没有格式、没有单位、没有说明。

元数据：描述数据的数据。样本号、工况、测点、采样率、日期、操作人都算。缺了它，三个月后就是一堆废数字。

开工前准备

1. 一个原始数据目录，建议路径全英文、无空格，比如 `D:\NVH\2026-03-batch\raw`。
2. 一个空的输出目录，跟原目录分开。原目录在整个过程中一个字节都不许动。
3. Python 环境 3.10 以上，装好 pandas 和 matplotlib： `pip install pandas matplotlib`。
4. WorkBuddy 里开一个 Agent 模式的对话，并把工作目录指到这一批数据所在的文件夹。
5. 一份你自己写的文件命名规矩，写在纸上。命名规矩不定，AI 会替你定，然后你就等着重做。
6. 一支笔。你要手算至少一个数：总行数乘以采样间隔，看看跟记录的时长对不对得上。

操作步骤

1. 先让它盘点，不许先动手改。盘点报告你亲自过一遍再往下走。

2. 看它报上来的列名差异表。差异超过三处，说明这批数据来自不止一台设备，规矩要按设备分别定。
3. 让它把整理规矩写成一页说明，你不点头它不许执行。这一步是整个场景里最省钱的一步。
4. 规矩确认后，让它批量清洗并写到新目录。
5. 让它生成一份 `metadata.json` 和一份 `summary.csv`。这两份文件是后面所有场景的地基。
6. 每个样本出两张图：时域图、频谱图。图上必须标采样率和通道名，不然三个月后你分不清。
7. 挑三个样本，你自己打开原始文件和清洗后的文件，逐列对一遍。三个够，别三十个都自己看。

试验数据整理五步

烦不在难，在每一步都要人眼去核对



原目录一个字节都不许动；挑三个样本自己逐列对一遍

图 15-1 试验数据整理五步：盘点、立规矩、清洗、出图、汇总

图 15-1 作者根据公开资料绘制。

可复制提示词

整段复制，路径和参数换成你自己的：

你现在是一名试验数据处理助手。我要你把一批原始试验记录整理成能直接进报告的形式。

工作目录：D:\NVH\2026-03-batch\raw

输出目录：D:\NVH\2026-03-batch\clean

【第一步：先盘点，不许先动手改任何文件】

1. 列出目录下全部文件，按扩展名分组统计数量。
2. 对每个数据文件，把它的前 3 行和后 3 行原样贴给我，不许概括。
3. 逐个文件报出：文件名、大小、行数、列数、列名、首行时间戳、末行时间戳。
4. 把列名不一致的文件单独列一张表，标出差异在哪几列。
5. 统计缺失值、空行、重复行，逐文件报出数量，不许合并成一个总数。
6. 把盘点结论写成 D:\NVH\2026-03-batch\clean\00_inventory.md。

【第二步：统一规矩，我确认之后你才许执行】

把下面这套规矩整理成一页说明给我看，我不点头你不许改任何文件：

1. 文件命名：{产品族}_{样本号}_{工况}_{测点}_{日期}.csv
示例：M12_S017_12V_load_A_20260314.csv
2. 列命名：time, ch1_mic, ch2_acc_x, ch3_acc_y, ch4_acc_z, rpm
全部小写，不许有空格，不许有中文列名。
3. time 列统一为相对时间，单位秒，保留三位小数，第一个采样点为 0.000。
4. 采样率统一写进一个 metadata.json，不许写在数据文件里。
5. 缺失值一律保留为空。不许填 0，不许填上一行的值，不许插值，不许删除行。

【第三步：确认后执行】

1. 按上面的规矩重写全部文件到输出目录，原目录一个字节都不许动。
2. 生成 metadata.json，字段：文件名、样本号、工况、测点、采样率、通道数、行数、时长、缺失值数量、原始文件名。
3. 生成 summary.csv，一行一个样本，字段同上。
4. 每个样本出一张时域图和一张频谱图，存到输出目录下的 figures 子目录，图名与样本号一致，图上必须标注采样率与通道名。
5. 出完全部图之后，给我一张清单：样本号、图文件名、图是否在磁盘上真实存在。

【纪律】

1. 每一步都把真实返回值贴给我，不许用“已完成”“一切正常”这类话概括。
2. 遇到读不懂的表头、对不上的通道数、缺失值超过 10% 的文件，立刻停下来报告，不许猜，不许跳过。
3. 不许修改原始数据，不许删除任何行，不许合并任何两个文件。
4. 我不确认第二步的规矩之前，不许执行第三步。

过程与判断点

一份合格的盘点汇报长这样，六行对得上才算过：

- > [盘点] 目录内共 34 个文件：CSV 30 个，WAV 4 个。
- > [列名差异] 30 个 CSV 分两种表头：
 - > 类型 A (22 个)：Time, Mic, AccX, AccY, AccZ, Speed
 - > 类型 B (8 个)：t, mic_1, ax, ay, az, rpm
 - > 差异列数 = 6，两套表头无一列同名。
- > [采样率] 类型 A 均为 48000 Hz；类型 B 均为 51200 Hz。两类不同。
- > [缺失值] S017 缺 128 行 (0.4%)，S023 缺 0 行，其余 28 个文件各缺 0 行。
- > [异常] S009 首行时间戳晚于末行时间戳，疑似导出时顺序颠倒，已列入待确认。
- > [结论] 两台不同设备导出，规矩需按类型分别写。等你确认。

这段汇报里我盯四个地方。

第一，它有没有把两类表头当成一类。这是最常见的翻车。两套表头看着都是六列，AI 很容易默认它们一样然后直接拼表。拼完你得到一份 30 行、列名叫 Time 的表，里面混着 51200 Hz 的数据，谁也看不出来。

第二，采样率是不是唯一值。采样率不一致可以做后续处理，但必须在元数据里标出来。我见过把 48 kHz 和 51.2 kHz 两批数据画在同一张谱上，谱线位置全错，还查了三天原因。

第三，缺失值它怎么处理。最危险的是“帮你填上了”。填 0 会让频谱多出假的低频能量；填上一行的值会让频谱变干净，看着特别漂亮。两种都是造假，方向相反。规矩里必须写死：保留为空。

第四，它有没有主动报告异常。S009 那个时间戳颠倒的问题，不是我提示之后才发现的，是它自己报的。愿意报告异常的助手，比永远说“一切正常”的助手值钱。

两种表头混在一张表的后果

两次导出来自两台设备，规矩要按类型分别写

类型 A 表头，22 个文件

- 列名
Time, Mic, AccX, AccY, AccZ, Speed
- 采样率
48000 Hz
- 缺失值
S017 缺 128 行 (0.4%)，S023 缺 0 行

类型 B 表头，8 个文件

- 列名
t, mic_1, ax, ay, az, rpm
- 采样率
51200 Hz
- 差异
差异列数 6，两套表头无一列同名

被当成一套时，谱线位置对不上任何已知频率；缺失值一律保留为空

图 15-2 两种表头混在一张表里的后果：谱线位置全错

图 15-2 作者根据公开资料绘制。

翻车清单

翻车一：两套表头被当成一套。症状是清洗顺利完成，report 也漂亮，但后面做频谱分析时谱线位置对不上任何已知频率。根因是列名按位置而不是按含义匹配。补救：在第二步的规矩里要求它列出"两套表头的逐列对应关系表"，你亲自确认对应关系之后再执行。

翻车二：缺失值被静默填充。症状是清洗后的曲线异常光滑，或者低频段多出一坨能量。根因是它调用了默认的填充方法。补救：清洗完成后立刻让它报一次"清洗前后缺失值数量对比表"，两个数字必须完全一致。

翻车三：写了输出目录之外的文件。症状是原目录里的文件被改名了，你找不到原始数据。根因是提示词里没写死"原目录只读"。补救：动手前让它把原目录全部文件的哈希值算一遍存下来，结束后再算一遍对比。

验收标准与变体

验收五条，约十分钟做完：

1. 打开 `00_inventory.md`，文件数与你目录里的实际文件数一致。
2. 随机挑三个样本，原始文件与清洗文件逐列比对，数值完全相同，只有列名和时间基准变了。
3. `metadata.json` 里每个样本的采样率、行数、时长三项齐全，且行数除以采样率等于时长。
4. `figures` 目录里的图数量等于样本数，每张图上都能看到采样率和通道名。
5. 原目录文件大小和修改时间没变。

变体一：多批次的年度汇总。把一年里十几个批次的数据目录一起交给它，先出一张跨批次元数据总表，看哪几批的规矩不一致。这一步把"数据能不能合并分析"在最前面回答掉。

变体二：试验记录表识别后的清洗。手写或扫描的记录表识别出来错字、串行、单位混写。让它按字段表逐行重建，不确定的格子写"待人工确认"而不是猜一个。

变体三：供应商来料数据的进门检查。供应商发来的数据先过一遍，看采样率、通道数、缺失比例，不合格的直接退回，别让它进你的分析链路。

老兵说：整理数据这件事，AI 的价值不在快，在它第 300 个文件也跟第 1 个一样认真。人做不到这一点，人到第 40 个就开始烦了，而烦的时候恰恰是最该慢下来的时候。

场景 50 阶次分析与噪声源识别：让 AI 帮你读谱 ★★★

一句话背景

谱出来了，一堆峰。哪个峰是电机转频，哪个是齿轮啮合，哪个是结构共振，哪个是干扰？人工对一遍特征频率表要一个半工作日，而且极容易漏掉“这个峰不随转速走”这件事。这个场景教你让 AI 先把特征频率表算给你，你确认之后它才许动数据。

名词小词典

阶次：某频率相对于主轴转频的倍数。转频是 1 阶，它的两倍是 2 阶。阶次不随转速变，换个转速测还是那个数。

转频：主轴每秒转多少圈，单位 Hz，等于转速（r/min）除以 60。

啮合频率：齿轮每转一圈有多少个齿依次咬合，就是转频乘齿数。齿轮噪声的绝大多数能量都在它及其谐波上。

阶次谱：横轴是阶次而不是频率的谱。转速变化的工况下，它比普通频谱清楚得多。

转速谱阵：横轴转速、纵轴频率、颜色表示幅值的三维图。随转速走的斜线是阶次，不随转速走的竖线是共振。

包络分析：也叫共振解调。先把信号里一段高频共振挑出来，再取包络看周期性，专门抓轴承点蚀、齿面剥落这类冲击。

开工前准备

1. 场景 49 清洗好的数据，必须带转速通道。没有转速通道的阶次分析是猜的，别做。
2. 被测对象的真实参数：电机极数、定子槽数、换向片数、各级齿轮齿数、蜗杆头数。参数拿不到就去查图纸，这一步省不掉。
3. Python 环境，装好 scipy 和 numpy：`pip install numpy scipy matplotlib`。
4. 一份你自己的特征频率公式清单。公式不确定的地方让 AI 标“待确认”，不许它补默认值。
5. 消声室或半消声室的背景噪声记录。没有背景噪声记录，你分不清哪些峰是被测对象发的，哪些是环境里本来就有的。

操作步骤

1. 先让它算特征频率表，不确认不许往下走。这一步是全场景的闸门。
2. 你拿这份表跟自己的手算对一遍。转频、啮合频率这两个你一定能手算，五分钟。
3. 确认之后让它做等角度重采样和阶次谱。阶次分辨率先定 0.1 阶，最大阶次 40 阶。
4. 让它出转速谱阵，并在图上分开标阶次线和共振线。这两类线的处理方式完全不同。
5. 把所有不随转速移动的峰单独列一张表，这是共振候选清单，你要拿去跟模态测试比对。
6. 对加速度通道做包络分析，判断有没有周期性冲击。

7. 让它写一份报告，四步各一节，每节结尾一行结论。结论里必须写"疑似"。



你是一名 NVH 分析工程师。我给你一批微电机（含齿轮减速级）的振动与噪声数据，帮我做阶次分析并给出噪声源归因。我要的是能写进报告的结论，不是一张图。

【输入】

转速通道: rpm, 单位 r/min

振动通道: ch2_acc_x, ch3_acc_y, ch4_acc_z, 单位 m/s^2

噪声通道: ch1_mic, 单位 Pa (未做声压级换算)

采样率: 48000 Hz

被测对象参数:

电机极数 = [4], 定子槽数 = [12], 换向片数 = [24]

一级蜗杆头数 = [2], 蜗轮齿数 = [30]

二级小齿轮齿数 = [11], 大齿轮齿数 = [41]

【第一步：先算特征频率表，不许先看图】

按下面的公式算出全部特征频率，写成一张表，每个频率一行：

1. 转频 $f_r = \text{rpm} / 60$
2. 一级蜗杆啮合频率 = 蜗杆转频 $\times$ 蜗轮齿数
3. 一级输出转频 = 蜗杆转频 $\times$ 蜗杆头数 / 蜗轮齿数
4. 二级啮合频率 = 一级输出转频 $\times$ 大齿轮齿数
5. 换向频率 = 转频 $\times$ 换向片数
6. 齿槽转矩频率 = 转频 $\times$ $1/\text{cm}$ (极数, 槽数)

表格列：名称、计算公式（写出字母）、转速 3000 r/min 时的数值、单位、阶次值。

公式里你不确定的项写“待确认”，不许自己补一个默认值。

这张表我不确认，你不许执行第二步。

【第二步：阶次谱】

1. 对振动与噪声通道做等角度重采样，阶次分辨率 0.1 阶，最大阶次 40 阶。
2. 输出阶次谱，列出前 10 个峰值：阶次值、幅值、出现该峰值的主要转速区间。
3. 每个峰值后面跟一句归因：它可能来自特征频率表里的哪几项，按可能性从高到低排，写“疑似 A（可能性高）/ 疑似 B（可能性中）”。

【第三步：区分阶次与共振】

1. 出一张转速谱阵：横轴转速、纵轴频率、颜色表示幅值。
2. 在图上把峰值分成两类并分别标出来：
随转速线性移动的 = 阶次线；不随转速移动的 = 结构共振线。
3. 把所有“不随转速移动”的峰单独列一张表：频率值、幅值、出现转速范围。
这张表我要拿去跟模态测试结果比对。

【第四步：包络分析】

对 ch2_acc_x 做包络解调：带通滤波后取包络再做频谱。

滤波带宽与中心频率先问我，不许自行取默认值。

输出前 5 个包络谱峰值及其频率，判断是否存在周期性冲击，并给出冲击重复频率。

【输出】

写成 D:\NVH\order\order_report.md，四步各占一节，每节结尾写一行“结论”。

所有数字保留三位有效数字并标注单位。

【纪律】

1. 特征频率表必须在看图之前给我，我不确认不许往下走。
2. 任何一步的假设（窗函数、重叠率、滤波带宽、重采样方法）都要写出来让我定。
3. 归因结论必须写“疑似”，不许写“确定为”。除非有断电试验或阶次切片作为证据。
4. 数据读不出来就停下报告，不许拿别的样本的数据顶上。
5. 背景噪声谱单独出一张，与实测谱并排放在报告最后一节。

过程与判断点

一份合格的回报应该先给你这张表，这是全场景最关键的一页：

- > [特征频率表] 转速 3000 r/min 时：
- > 转频 50.000 Hz 阶次 1.00
- > 蜗杆啮合频率 1500.000 Hz 阶次 30.00
- > 一级输出转频 3.333 Hz 阶次 0.07
- > 二级啮合频率 136.667 Hz 阶次 2.73
- > 换向频率 1200.000 Hz 阶次 24.00
- > 齿槽转矩频率 600.000 Hz 阶次 12.00
- > [待确认] 刀纹频率：缺蜗杆加工刀纹数，无法计算，已列出待补。
- > [阶次谱前 5 峰] 30.02 阶 / 6.1 阶 / 24.05 阶 / 12.01 阶 / 2.71 阶
- > [归因] 30.02 阶 → 疑似蜗杆啮合频率（可能性高，与理论值差 0.07%）
- > 24.05 阶 → 疑似换向频率（可能性高）
- > 12.01 阶 → 疑似齿槽转矩频率（可能性中，与换向频率存在倍数关系需区分）
- > [共振候选] 2140 Hz（不随转速移动，全转速段存在）、3680 Hz（同上）
- > [包络谱] 未执行，等待你给滤波带宽与中心频率。

这段回报里我盯三个地方。

第一，阶次值跟理论值的偏差。30.02 阶对 30.00 阶，偏差 0.07%，说明转速信号和振动信号对得上。如果报出 30.5 阶，要么转速通道标定错了，要么重采样参数错了，两种都得停下来查。

第二，它有没有把倍数关系说清楚。12 阶和 24 阶是两倍关系，齿槽转矩频率和换向频率可能撞在一起，这时候靠阶次谱分不清，得靠断电或变电压试验。它写“存在倍数关系需区分”，说明它知道自己分不清；只写“确定为换向频率”，你就要警惕。

第三，共振候选表。这一页的价值高于阶次谱本身。阶次线告诉你源在哪，共振线告诉你路径上有问题。一条不随转速移动的 2140 Hz 峰，意味着改转速、改齿轮都没用，得改结构或者加隔振。

转速谱阵上两类线

谱上每个峰都在问你：你是从哪来的

阶次线，随转速移动	共振线，不随转速移动
长相 转速谱阵上的斜线 来源 转频、啮合频率、换向频率 改法 改源头：齿轮参数、换向、电磁设计 判别 变电压后频率跟着变	长相 转速谱阵上的竖线 来源 结构固有频率，激励撞上它 改法 改结构或加隔振，改转速与齿轮都没用 判别 变电压后频率钉死不动，去比模态

把共振当成阶次去改源，改两周噪声一点没降

图 15-4 转速谱阵上两类线：斜线是阶次，竖线是共振

图 15-4 作者根据公开资料绘制。

翻车清单

翻车一：没有转速通道硬做阶次分析。症状是它用平均转速代替实时转速，或者给你一张普通频谱说这是阶次谱。补救：试验设计阶段就把转速通道列进采集清单；已经做完的，只能退回按稳态转速分段做频谱，并标注这是稳态谱不是阶次谱。

翻车二：把共振当成阶次去改源。症状是你花两周改了齿轮修形，噪声一点没降，因为那个峰不随转速走，它本来就不是齿轮发的。补救：先看转速谱阵分类，分类没做完不许动设计。

翻车三：阶次分辨率太粗，两个峰糊成一个。症状是 23.8 阶和 24.0 阶合并成一条宽峰，你以为只有换向频率，其实是它加齿槽转矩频率两个源。补救：把分辨率从 0.5 阶提到 0.1 阶或更细，代价是计算量，别省。

验收标准与变体

验收五条：

1. 特征频率表的每一项，你能用计算器手算对上至少三项（转频、一级啮合、二级啮合）。
2. 阶次谱前 5 峰的阶次值与理论值偏差都在 1% 以内。
3. 转速谱阵上能清楚看到两类线，且共振候选表里的频率值与模态测试结果能对上至少一条。

4. 报告里每一个归因结论都带"疑似"二字，没有一处写"确定"。
5. 背景噪声谱在报告最后一节，且共振候选里的频率没有一条与背景噪声的峰重合。

变体一：空载与加载对比试验。同一台样机跑两遍，一遍空载一遍带载。空载就有的峰指向电机本体，加载才出现的峰指向齿轮与负载。这是区分电磁源和机械源最便宜的办法，一次试验十分钟。

变体二：断电法。跑到稳态转速后切电源，噪声立刻消失的是电磁噪声，随惯性慢慢衰减的是机械噪声。这个方法简单到不像技术，但它一年能帮你省下好几次误判。

变体三：变电压扫频。改变供电电压看频率跟不跟着变。频率随转速变的是阶次类，频率钉死不动的是结构共振。这一条和转速谱阵互为验证。

老兵说：谱上每一个峰都在问你一个问题：你是从哪来的？答不上来的峰别放过它，也别编答案，把它列进待查清单，下一次试验专门去查它。

场景 51 试验报告自动生成：数据进、报告出，关键数字可追溯 ★★★

一句话背景

试验做完了，报告还得写：试验条件、测点布置、结果曲线、结论。人工写一份约一个工作日，其中大半时间花在"这个数是从哪个文件的哪一行来的"这种事上。这个场景的核心不是让它写文章，是让它交出一张"每个数字对应哪个源文件"的对照表。

名词小词典

报告模板：公司固定的报告格式，含封面、章节标题、表格样式。先把模板给它，别让它自己发明。

可追溯：报告里任何一个数字都能倒查回原始文件、行号、时间戳。做不到这点的报告，评审会上经不起一句追问。

数据血缘：一个数字从原始数据到最终结论走过的全部步骤，中间每改一次就多一段。

等效连续 A 声级：一段时间内 A 计权声压的能量平均值，记作 LAeq，它把一段起伏的噪声压成一个数。

版本：报告每次修改都要留版本号和修改记录。改了数据没改报告，或者改了报告没标版本，都是事故。

开工前准备

1. 你公司真实的报告模板文件（Word 或 Markdown），放在工作目录里。
2. 场景 49 产出的 `metadata.json` 和 `summary.csv`，以及场景 50 的分析报告。
3. 测试系统的校准记录：麦克风校准日期、校准器声压级、校准频率。没有校准记录的报告，审核人有权退回。

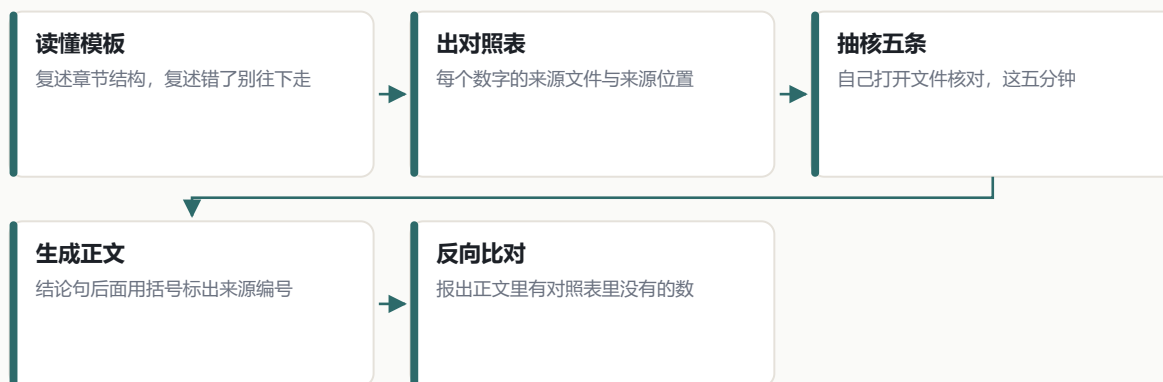
4. 明确的判定门槛：总声压级目标值、各频段限值、主观评价合格线。门槛不许事后改。
5. 一个输出目录，报告和图分开存放。

操作步骤

1. 先把模板给它，让它复述一遍模板的章节结构。复述错了说明它没读懂，别往下走。
2. 让它生成"数字来源对照表"，这是第一份产物，先于正文。
3. 你抽查对照表里的五条，自己打开文件核对。这五分钟决定了后面整份报告能不能信。
4. 对照表通过之后，才让它按模板生成正文。
5. 要求它把每个结论句后面用括号标出数据来源编号，编号对应对照表里的行号。
6. 生成之后让它自己再读一遍报告，把报告里出现的全部数字提取出来，与对照表比对，报出"报告里有但对照表里没有"的数字。
7. 你签字之前，改一处数据重跑一次，看报告里的数字跟着变。不跟着变的报告就是废纸。

报告生成五步

第一份产物不是正文，是数字来源对照表



改一处数据重跑一次，数字不跟着变的报告就是废纸

图 15-5 报告生成五步：读模板、出对照表、抽核、生成正文、反向比对

图 15-5 作者根据公开资料绘制。

可复制提示词

你是一名试验报告撰写助手。你要按我给的模板，把已完成的试验数据写成一份可追溯的报告。
我要的第一份产物不是正文，是数字来源对照表。

【输入】

模板文件: D:\NVH\template\report_template.md

元数据: D:\NVH\2026-03-batch\clean\metadata.json

汇总表: D:\NVH\2026-03-batch\clean\summary.csv

分析报告: D:\NVH\order\order_report.md

输出目录: D:\NVH\report\

【第零步：读懂模板】

先把模板的章节结构复述给我：一级标题有几个、每个一级标题下有哪些必填字段、哪些字段是表格、哪些是图。复述完等我确认，不许直接开始写正文。

【第一步：数字来源对照表】

扫描上面全部输入文件，把报告里将要出现的每一个数字列成一张表，字段：

编号 | 数字值 | 单位 | 含义 | 来源文件 | 来源位置（行号或字段名） | 是否为计算值 | 计算式
规则：

1. "是否为计算值"一列，凡是经过任何运算的数字都标"是"，并写出完整计算式。
2. 直接从文件读出的数字标"否"，来源位置必须精确到行号。
3. 你打算写进报告但找不到来源的数字，一律不许进对照表，单独列一节"无来源数字"报给我。
4. 对照表写成 D:\NVH\report\01_number_trace.csv。

【第二步：我抽查之后再写正文】

对照表我会抽查五条。我确认之后你才许生成正文。正文要求：

1. 严格按模板章节顺序，不许增删章节。
2. 每个结论句后面用方括号标注来源编号，示例：
"该样本总声压级为 42.3 dB(A)，低于目标值 45 dB(A)[N017][T002]。"
3. 所有图从输出目录的 figures 子目录引用，不许重新画，不许用占位图。
4. 试验条件一节必须包含：环境温度、背景噪声、麦克风型号与校准日期、采样率、测点坐标。缺任何一项就写"缺"，不许留空，不许编。
5. 结论一节里凡涉及"声音是否可接受"的判断，一律写"待主观评价确认"，不许由你下结论。

【第三步：反向比对】

正文写完后，把正文里出现的全部数字提取出来，与第一步的对照表逐条比对，报出三类：

- A. 正文里有、对照表里没有的数字（这是最严重的问题）
- B. 对照表里有、正文里没用到的数字
- C. 两处数值不一致的数字

三类各自列清单，A 类不许为空以外的任何情况出现，出现就是不合格。

【纪律】

1. 不许自己计算任何汇总值来填补空缺，找不到就写"无来源"。

2. 不许改写模板里的章节标题和单位制。
3. 不许在报告里出现"良好""基本满足"这类没有数字支撑的形容词。
4. 数字保留位数全篇统一：声压级一位小数，频率整数，转速整数。

过程与判断点

对照表应该是这个样子，这是你要亲自抽查的东西：

```
> 编号 | 数值 | 单位 | 含义 | 来源文件 | 来源位置 | 计算值 | 计算式
> N001 | 42.3 | dB(A) | S017 总声压级 | summary.csv | 第 18 行 laeq 列 | 是 | 20*log10(p_rms/2e-5), p_rms 由 ch1_mic 全程有效值算得
> N002 | 45.0 | dB(A) | 目标限值 | (无来源) | 无 | 否 | 由我口头给定，已列入无来源数字
> N003 | 48000 | Hz | 采样率 | metadata.json | S017.fs | 否 | 无
> N004 | 1500 | Hz | 一级啮合频率 | order_report.md | 特征频率表第 2 行 | 是 | 3000/60*30
> N005 | 2140 | Hz | 共振候选 1 | order_report.md | 共振候选表第 1 行 | 否 | 无
>
> [无来源数字] 共 3 项：目标限值 45.0 dB(A)、主观评价合格线、麦克风校准日期。
> 这 3 项需要你补充来源，我不会自行填入。
```

这份对照表里有三处值得你盯。

第一，"是否为计算值"这一列。42.3 dB(A) 是算出来的，计算式写得很清楚。这一列的意义是：评审会上有人质疑这个数，你能当场把公式摆出来。没有计算式的结果，本质上是个黑箱。

第二，"无来源数字"这一节。目标限值 45.0 dB(A) 找不到文件来源，因为它在某个规范里而你没把规范给它。它没有偷偷填一个，而是列出来问你。这个动作看着笨，实际上救过我一次：那次列出来的无来源数字里有一条是主观评价合格线，我去翻规范才发现用的那版已经作废了。

第三，反向比对里的 A 类。正文里出现而对照表里没有的数字最危险，那意味着 AI 在写正文时自己造了一个数。这类问题人工通读几乎发现不了，因为数字通常看着很合理。所以这一步必须让它自己做，人只负责确认 A 类为空。

可追溯报告的三层结构

倒查不到原始文件的数字，只是别人写的字

正文结论

每个结论句后面用括号标出来源编号

数字来源对照表

编号、数值、单位、含义、来源文件、来源位置、是否计算值、计算式

原始数据与元数据

metadata.json、summary.csv、阶次分析报告

找不到来源的数字单独列一节无来源数字，不许自己算一个填补空缺

图 15-6 一份可追溯报告的三层结构：原始数据、对照表、正文结论

图 15-6 作者根据公开资料绘制。

翻车清单

翻车一：图是画上去的，不是数据画出来的。症状是报告里的曲线看着很专业，但改了数据之后图纹丝不动。根因是它为了排版方便用示意数据重画了一遍。补救：验收时必做一次“改数据重跑”，三处数字跟着变、三张图跟着变才算过。

翻车二：单位在半路上丢了。症状是报告里出现“声压级 42.3”后面没有 dB(A)，或者加速度的单位写成 m/s。根因是模板里的表头没有单位列，它就按自己的习惯填。补救：在提示词里把每个物理量的单位写死成一张表，并要求反向比对时也检查单位。

翻车三：结论写了“满足要求”但没有门槛来源。症状是报告结论很肯定，评审时一问“凭什么说满足”，答不上来。根因是门槛值没有可追溯来源。补救：把公司规范里的限值表也作为输入文件给它，让限值同样进对照表。

验收标准与变体

验收六条：

1. 对照表里每一个数字都能打开源文件找到对应位置，抽查五条全部对上。
2. 反向比对的 A 类清单为空。
3. 报告里每一个结论句后面都有来源编号。

4. 改一处原始数据重跑，报告里至少三处数字与两张图跟着变。
5. 报告版本号、生成时间、数据目录路径三项齐全，写在封面或页脚。
6. 把报告给一个没参与试验的人读，他能说出这批样本哪个合格哪个不合格，以及凭什么。

变体一：多批次对比报告。把三个批次的数据一起给它，出一份跨批次报告，重点是趋势不是单值，对照表的来源列要多一列"批次号"。

变体二：供应商报告审查。把供应商提交的报告丢给它，按同样规则反推对照表，看哪些数字找不到来源。这一步能在评审会上省掉大量拉扯。

变体三：把报告生成固化成一条命令。把这段提示词加上模板路径存成技能，每批试验出完数据跑一次，报告落到固定目录，编号自动递增。

老兵说：报告里的数字能被倒查到原始文件，这份报告才是你的；倒查不到的那些，只是别人写的字，你签了字就变成你的责任。

场景 52 声学数据库搭建与检索：把历史案例变成能问的库 ★★★

一句话背景

我做过的那 30 多个样本，最开始是躺在一个共享盘里的：一段录音、一份 Excel、一份 PDF 报告，三样东西靠文件名对上。半年后我想找"响度超标且主频在 2 kHz 以上"的样本，翻了两个下午。这个场景教你把历史案例做成能直接用自然语言问的库。

名词小词典

样本：一台被测样机在一种工况下的一次完整记录。同一台样机跑三种工况，就是三个样本。

工况：被测对象工作时的条件。对微电机来说通常是电压、负载、转速、转向、温度。

测点：麦克风或加速度计放的位置。测点一变，同一台样机测出来就是另一个数。

响度：心理声学参数，单位 sone，按人耳听觉特性加权，比总声压级更接近"听起来有多响"。

尖锐度：描述高频成分占比的心理声学参数，单位 acum，反映刺耳程度。"毛刺"这个主观词对应的客观量主要就是它。

检索：用一句自然语言问库问题，库返回命中样本。跟数据库查询的差别是你不用会写查询语句。

开工前准备

1. 历史样本的全部录音文件，集中到一个目录，命名带样本号。
2. 每个样本的客观指标表（场景 49 的 `summary.csv` 就够用）。
3. 每个样本的主观评价记录，含评分人、评分、评语。没有的话从现在开始记。
4. 一份你定的字段清单。字段不统一是检索失效的头号原因。

5. 一个纯文本格式的库目录，每个样本一张卡片文件。我用的是 Markdown，因为人和 AI 都能读。

操作步骤

- 1. 先定字段清单，写在纸上，不许让 AI 替你定。我这份用了 14 个字段，见下表。
- 2. 让它按字段清单，从现有文件里抽取信息，生成每张样本卡片。
- 3. 抽不出来的字段写"缺"，不许猜。缺字段的样本也要进库，并在卡片上标"不完整"。
- 4. 让它生成一份总索引 `index.csv`，一行一个样本，列就是字段清单。
- 5. 生成一份 `README.md` 写在库根目录下，说明字段含义、单位、缺失规则、更新方法。这份文件是给未来的你和未来的 AI 看的。
- 6. 检索测试：问三个你已知答案的问题，看它答得对不对。
- 7. 把录音文件路径写进卡片，并且确认路径真的能打开。



图 15-7 声学库的三层结构：录音文件、样本卡片、总索引

图 15-7 作者根据公开资料绘制。

这是我那份库用的字段清单，你可以照抄再改：

#	字段	单位或取值	说明
1	样本号	S001 起	唯一标识，永不复用
2	产品族	如 M12	同一平台归一个族

#	字段	单位或取值	说明
3	样机编号	实物编号	同一台样机可能有多个样本
4	工况电压	V	如 12.0
5	工况负载	N·m 或额定百分比	写实测值
6	工况转速	r/min	写稳态值
7	测点	A / B / C	与测点布置图对应
8	总声压级	dB(A)	一位小数
9	响度	sone	两位小数
10	尖锐度	acum	两位小数
11	主频	Hz	整数，写前三强
12	主阶次	阶	一位小数
13	主观评分	1 到 10	写均值与人数
14	结论	通过 / 不通过 / 待定	只由人填

可复制提示词

建库这一段：

我要把历史声学样本建成一个可以用自然语言问的库。先建库，不许先检索。

【目录】

录音目录: D:\NVH\db\audio

卡片输出目录: D:\NVH\db\cards

索引文件: D:\NVH\db\index.csv

说明文件: D:\NVH\db\README.md

【字段清单】严格按这 14 个字段，不许增、不许减、不许改名：

样本号，产品族，样机编号，工况电压，工况负载，工况转速，测点，

总声压级，响度，尖锐度，主频，主阶次，主观评分，结论

【第一步：抽取】

1. 扫描 D:\NVH\db\audio 下全部录音文件，列出文件名清单与时长。
2. 从 D:\NVH\2026-03-batch\clean\summary.csv 与 D:\NVH\order\order_report.md 中抽取客观字段，按样本号配对。
3. 每个样本生成一张 Markdown 卡片，文件名等于样本号，格式：
一级标题为样本号，下面 14 个字段每个占一行，写成 "- 字段名: 值（来源: 文件名）"。
4. 抽不出来的字段写 "- 字段名: 缺（无来源）"，不许猜，不许填 0，不许填"未知"以外的近义词。
5. 主观评分与结论两个字段，任何情况下都不许由你填写，一律写"待人填"。

【第二步：建索引】

生成 index.csv，一行一个样本，列顺序与字段清单完全一致。

另加两列：卡片文件路径、录音文件路径。

生成后报出：样本总数、字段完整率（每个字段的非"缺"比例）。

【第三步：写说明】

生成 README.md，内容包含：

1. 库的用途与覆盖范围。
2. 14 个字段的逐项说明：名称、单位、取值规则、"缺"的含义。
3. 更新方法：新增一个样本要走哪几步。
4. 三条禁令：不许删改历史卡片；样本号永不复用；主观字段只由人填。

【纪律】

1. 录音文件路径必须逐个验证能打开，打不开的单列一张表报给我。
2. 配对时以样本号为准，样本号对不上就停下报告，不许按文件名模糊匹配。
3. 你不许计算或推断任何客观值，只能从已有文件里搬运。

检索这一段，建库完成后另开一段：

库已经建好，目录是 D:\NVH\db。现在我要检索。

【检索规矩】

1. 先读 D:\NVH\db\README.md，按里面的字段含义理解我的问题。
2. 用 index.csv 做筛选，命中样本后打开对应卡片读取完整字段。
3. 每次回答必须包含三部分：
 - A. 我这个问题的筛选条件，你把它翻译成字段名和阈值写清楚给我看；
 - B. 命中样本清单：样本号 + 关键字段值（至少三个字段）；
 - C. 命中数量与“因字段缺失而无法判断”的样本数量。
4. 字段值为“缺”的样本，在筛选时算作无法判断，既不算命中也不算不命中，必须在 C 部分单独报出数量。
5. 不许打开录音文件去“听”，你没有听觉，只能读结构化字段与已有的分析结论。

【我的问题】

问：找出所有总声压级低于 45 dB(A) 但主观评分低于 6 分的样本，按响度从高到低排。

过程与判断点

检索结果应该长这样，三部分缺一不可：

- > A. 筛选条件翻译：
 - > 总声压级 < 45.0 dB(A) AND 主观评分 < 6.0
 - > 排序：响度 降序
- > B. 命中样本：
 - > S017 总声压级 42.3 dB(A) / 响度 3.8 sone / 尖锐度 2.9 acum / 主观评分 5.2 (6 人)
 - > S031 总声压级 44.1 dB(A) / 响度 3.1 sone / 尖锐度 2.4 acum / 主观评分 5.8 (6 人)
 - > S008 总声压级 43.6 dB(A) / 响度 2.7 sone / 尖锐度 1.9 acum / 主观评分 5.5 (4 人)
- > C. 命中 3 个；因字段缺失无法判断 4 个 (S022 缺响度，S029 缺主观评分，S033、S034 两项均缺)。这 4 个样本我无法判定是否满足你的条件。

这份回答里我最看重的是 C 那一行。

为什么 C 必须存在。库里有 4 个样本缺字段，它只报“命中 3 个”，你会以为全库只有 3 个相关样本，实际可能有 7 个。缺字段样本单独报出来，你才知道要补哪些数据。

为什么 A 部分要它把条件翻译出来。自然语言有歧义。我说“低于 45”，它可能理解成小于等于。让它把翻译结果摆出来，你一眼就能看出偏了没有。这一步花十秒，能挡掉一次错误结论。

为什么禁止它“听”。它没有听觉。让它读录音文件判断声音，它只能读文件名和元数据，然后编一段听着很像那么回事的描述。这条禁令是本场景最重要的纪律。

检索回答的三部分

三部分缺一不可，最看重的第三部分

A 条件翻译

把自然语言翻译成字段名与阈值，先摆给你看一遍

B 命中清单

样本号加至少三个关键字段值，按响度降序排

C 缺失统计

命中 3 个；因字段缺失无法判断 4 个，单独报出

字段值为缺的样本，既不算命中也不算不命中；不许打开录音去听

图 15-8 检索回答的三部分：条件翻译、命中清单、缺失统计

图 15-8 作者根据公开资料绘制。

翻车清单

翻车一：字段不统一，检索直接失效。症状是问一个问题它答"未找到"，根因是同一个字段有"12V""12.0""12 伏"三种写法。补救：建库时把取值规则写进 README，并要求它报出每个字段的取值分布，你扫一眼有没有异常值。

翻车二：录音路径失效。症状是卡片里的路径指向临时目录，半年后全部打不开。补救：录音文件与库放在同一级目录下，卡片写相对路径，建库完成后逐个验证能打开。

翻车三：主观评分没有可溯源记录。症状是库里有评分，查不到谁打的、什么时候打的、几个人打的。补救：主观评价记录必须带评分人、日期、人数、原始评语四项，缺一项就不进库。

验收标准与变体

验收五条：

1. 问三个你已知答案的问题，三个都答对，且命中数量与你手算一致。
2. 每次回答都包含 A、B、C 三部分，C 部分的缺失数量与你库里的实际缺失数一致。
3. 打开五张卡片，14 个字段齐全，来源标注清楚，主观字段写着"待人填"。
4. `index.csv` 的行数等于 `cards` 目录里的文件数。

5. 新建一条测试样本走一遍更新流程，索引和卡片都跟着更新，旧卡片没被改动。

变体一：跨产品族的横向库。把不同产品族的样本放进同一个库，加一列"产品族"，你就能问"哪个族的尖锐度普遍偏高"，这类问题在单族库里问不出来。

变体二：故障案例库。同样结构换成故障案例：现象、工况、排查过程、根因、方案、验证结果。字段换掉，流程一句不改。

变体三：试验规范库。把历年用过的试验规范做成库，问"某产品上次用的是哪一版规范"。

老兵说：库的价值不取决于它装了多少样本，取决于你敢不敢在评审会上直接引用它。敢引用的前提是每个数字都能倒查，所以宁可留着"缺"字，也不要填一个看着合理的数。

场景 53 微电机与齿轮传动件的声学问题定位：方法部分 ★★★

一句话背景

客户说"这个电机声音不对"，你手上只有一台样机和一句模糊的描述。从哪里下手？人工排查一个这样的案子平均一到两天，而且极容易一开始就查错方向。这个场景给一套可复制的定位流程，品类限定在微电机与齿轮传动件。

名词小词典

激励 / 路径 / 响应：任何振动噪声问题都能拆成三件事：谁在振（激励）、怎么传出来（路径）、人听到什么（响应）。改激励最有效，改路径次之，改响应最没用。

传递误差：齿轮实际转角与理论转角的差值。齿轮噪声的绝大多数根因都能追到它身上。

齿距误差：相邻两个齿之间的距离偏差，是传递误差的主要来源之一。

修形：把齿形或齿向按计算好的量微微改一点，让啮合更平顺。改得过头会损失强度和重合度。

断电法：跑到稳态后切断电源，看噪声是不是立刻消失。立刻消失是电磁源，随惯性衰减是机械源。

心理声学参数：响度、尖锐度、粗糙度、抖动度、音调度这一类按人耳特性算出来的量，比总声压级更接近主观感受。

开工前准备

1. 一台可疑样机和一台已知合格的对比样机。没有对比样机的定位，等于没有尺子量长度。
2. 场景 49 到 51 的完整链路：清洗后的数据、特征频率表、可追溯报告。
3. 被测对象的图纸参数：极数、槽数、换向片数、各级齿数、蜗杆头数、加工方式。
4. 断电与变电压的条件。做不到这两项试验，后面有一半的判别方法用不了。
5. 一个本底噪声记录。分不清本底噪声，你会把环境里的声音当成产品的声音。

操作步骤

这套流程我按七步走，顺序不能乱：

步	做什么	为什么在这一步
1	复现现象，把它描述成可测的条件	描述不清就没法定量
2	空载 vs 加载对比	分电机本体与齿轮负载，最便宜
3	断电法	分电磁与机械，十秒钟出结果
4	变电压扫频	分阶次与共振
5	算特征频率表并做阶次分析	把峰对到具体零件
6	看心理声学参数	解释"测得过线但听得烦"这类冲突
7	按成本从低到高排改进方案	先做便宜的那一半

第二步和第三步加起来不到二十分钟，却能砍掉一半的排查方向。我见过太多人跳过这两步直奔齿轮修形，修完噪声没变，两周白干。

声学问题定位七步

品类限定在微电机与齿轮传动件，顺序不能乱；二三步不到二十分钟

1

复现现象，描述成可测条件
转速区间、随转速与负载是否变化

2

空载与加载对比
差谱分电机本体与齿轮负载

3

断电法
立刻消失是电磁，随惯性衰减是机械

4

变电压扫频
频率跟着变是阶次，不动是共振

5

算特征频率表并做阶次分析
把前三强峰归因到具体零件

6

看心理声学参数
总声压级达标而听评不过，去看尖锐度

7

按成本从低到高排改进方案
每条写副作用，想不出就写未评估

图 15-9 微电机与齿轮传动件声学定位七步

图 15-9 作者根据公开资料绘制。

下面这张判别表是这套方法的核心，按现象直接找方向：

现象	优先查什么	判别依据
噪声空载就有，加载无明显变化	电机本体：电磁力波、齿槽转矩、换向、碳刷	断电后是否立刻消失
噪声空载没有，加载才出现	齿轮啮合、轴承受载、装配预紧	空载谱与加载谱的差谱
频率随转速线性变化	阶次类：转频、啮合频率、换向频率	转速谱阵上是不是斜线
频率不随转速变化	结构共振	转速谱阵上的竖线，去比模态
低频段（约 800 Hz 以下）嗡响	装配间隙、轴向窜动、支承刚度	断电不消失，随惯性衰减
高频段（约 3 kHz 以上）刺耳	换向火花、齿面粗糙度、机壳模态	看尖锐度是否同步偏高
有规律的周期性敲击	齿面局部缺陷、轴承点蚀	做包络分析看冲击重复频率
总声压级达标但听评不合格	高频窄带能量、断续成分	看尖锐度与响度，别盯总声压级

可复制提示词

你是我的 NVH 排查助手。我要定位一台微电机（含齿轮减速级）的声学问题。
你不许直接给结论，必须按下面的七步走，每走完一步把结果给我，等我确认再下一步。

【被测对象】

产品族：[M12]

电机参数：极数 [4]，定子槽数 [12]，换向片数 [24]，有刷/无刷 [有刷]

传动参数：一级蜗杆头数 [2]，蜗轮齿数 [30]；二级小齿轮 [11]，大齿轮 [41]

工况：电压 [12.0] V，负载 [1.2] N·m，稳态转速 [3000] r/min

【第一步：现象描述】

把我给的现象描述改写成可测的条件清单：出现转速区间、是否随转速变化、是否随负载变化、是否随温度变化、出现位置、声音的主观词（如啸叫/嗡嗡/咋哒）。
改写完给我看，我不确认不许往下走。

【第二步：空载 vs 加载】

取空载谱与加载谱，做差谱。列出只在加载时出现的峰、只在空载时出现的峰、两者都有的峰，各一张表。每张表后面跟一句方向判断。

【第三步：断电与变电压试验的数据解读】

我把断电试验和变电压试验的结果给你（如果我还没做，你提醒我去做，不许跳过）：

1. 断电后噪声立刻消失的分量 → 判为电磁源候选。
2. 断电后随惯性衰减的分量 → 判为机械源候选。
3. 变电压后频率跟着变的分量 → 判为阶次类。
4. 变电压后频率不动的分量 → 判为结构共振类。

四类各列一张表，写清判别依据与置信度（高/中/低）。

【第四步：特征频率归因】

按场景 50 的方法出特征频率表和阶次谱，把前三强峰归因到具体零件。
归因结论必须写“疑似”，并写出下一步用什么试验能确认它。

【第五步：心理声学参数】

算出响度 (sone)、尖锐度 (acum)、粗糙度、音调度，与目标值比对。

特别说明：如果总声压级达标但主观评价不合格，你要去看尖锐度和高频窄带能量，不许只报总声压级。

【第六步：改进方案排序】

按“改动成本从低到高”排，每条方案写：改什么、预期降多少、需要什么资源、副作用是什么（强度、寿命、效率、成本）。
副作用一列不许空，想不出来就写“未评估”。

【第七步：验证方案】

给一个可执行的验证试验清单：测什么、几个样本、判据是什么。

【纪律】

1. 任何结论不许写“确定为”，一律写“疑似”，直到有验证试验的数据支撑。

2. 不许建议"加隔音棉""包一层"这类只改路径不改源头的方案作为首选。
3. 涉及"这个声音客户能不能接受"的判断，你不许下结论，写"待主观评价"。
4. 参数你不确定的地方写"待确认"，不许补默认值。

过程与判断点

一份合格的六步汇报，重点看这三处：

- > [第二步 差谱] 只在加载出现的峰：2140 Hz（不随转速移动，判为受载后结构响应）
- > 只在空载出现的峰：无
- > 两者都有的峰：1500 Hz（阶次 30.0，疑似一级蜗杆啮合频率）
- > 3600 Hz（阶次 72.0，疑似啮合频率高次谐波或换向相关，待区分）
- > [第三步 断电] 1500 Hz 分量断电后随惯性衰减（约 0.8 s）→ 判为机械源候选（置信度 高）
- > 3600 Hz 分量断电后立即消失 → 判为电磁源候选（置信度 高）
- > 2140 Hz 分量断电后仍存在约 0.3 s 且频率不变 → 判为结构共振（置信度 中）
- > [第五步 心理声学] 总声压级 42.3 dB(A)，达标（目标 45）
- > 响度 3.8 sone（目标 3.5，超标约 8%）
- > 尖锐度 2.9 acum（目标 2.5，超标约 16%）
- > 判断：尖锐度超标幅度大于响度，问题集中在高频段，
- > 与"毛刺"这一主观词方向一致。声音能否接受，待主观评价确认。

第一，它有没有把三个峰分开。1500 Hz、3600 Hz、2140 Hz，三个峰三个来源：机械、电磁、结构共振。混在一起处理就全错，分开是这套方法的全部价值。

第二，置信度那一列。"高"和"中"的区别在证据强不强。断电试验对电磁与机械的区分是很强的证据，给"高"；2140 Hz 只靠频率不变判断是共振，没有模态数据支撑，给"中"。它敢写"中"，说明它知道自己的判断有多硬。

第三，第五步那句"尖锐度超标幅度大于响度"。这就是开场那个第 17 号样本的答案。总声压级达标，响度略超，尖锐度明显超，主观词是"毛刺"，三条线索都指向高频段。只看总声压级，这个案子会被判合格放行。

三个峰，三个来源

混在一起处理就全错，分开是这套方法的全部价值



只在加载时出现的峰要单独成表；改进方案里改源头的一类排在改路径的前面

图 15-10 三个峰三个来源：机械、电磁、结构共振，处理方式完全不同

图 15-10 作者根据公开资料绘制。

翻车清单

翻车一：把症状当根因。 客户说“声音大”，你直接去降总声压级，而根因可能是某个高频窄带成分，它占总能量的比例很小。补救：先看心理声学参数再定方向，别被总声压级牵着走。

翻车二：只改路径不改源头。 症状是加了隔振垫，噪声降了 2 分贝，三个月后隔振垫老化，问题原样回来。补救：改进方案排序里改源头的一类必须排在改路径的一类前面，除非你明确知道源头改不动。

翻车三：改了一处，没查副作用。 症状是齿轮修形把噪声压下去了，强度校核却过不了，或者效率掉了两个百分点。补救：方案表里的“副作用”一列不许空，每条方案至少列一项要复查的指标。

验收标准与变体

验收五条：

1. 现象被改写成可测条件清单，至少含转速区间、负载相关性、温度相关性三项。
2. 空载谱与加载谱的差谱出来了，三类峰各自成表。
3. 断电与变电压试验都做过了，四类判别表齐全，每类带置信度。
4. 前三强峰都有“疑似”归因，每条归因后面跟着一个能确认它的验证试验。
5. 改进方案按成本排序，每条都写了副作用，排在第一位的不是“加隔音材料”。

变体一：产线端的快速筛查。这套七步太重，产线上跑不了。压缩成两分钟版本：听空载、听加载、看总声压级和尖锐度两个数，只用来筛可疑件，不用来判根因。

变体二：设计阶段的声学目标分解。反过来用：先定客户能接受的主观目标，倒推响度和尖锐度目标，再倒推到零件公差。这个方向比"先做出来再测"省钱得多。

变体三：供应商样件的进门检查。同一套七步，把被测对象换成供应商送来的样件，第三步和第五步必做，其余按风险决定。

老兵说：总声压级达标而听评不过的案子，我这些年遇到过不止一次。每次回头查，问题都不在总能量上，在某个很窄的频段上。所以别只盯那个总分，分数是给报告看的，频段是给耳朵听的。

场景 54 把老师傅的经验蒸馏成 AI 可用技能：真实蒸馏过程 ★★★

一句话背景

一个干了几十年的老师傅离职，带走的是他脑子里那批"上次这种情况是怎么处理的"。这件事的真正损失不是知识，是判断。这个场景讲我怎么把自己十一年的一线经验做成两个能装进 AI 的仓库，第一版花了大约六个月，现在同样的事三小时能做出八成。

名词小词典

技能：一个给 AI 读的知识包，通常是一个入口文件加一组参考文件。它会被反复加载，所以要结构化。

蒸馏：把散落在脑子、笔记、旧报告里的经验，变成有结构、能被检索和引用的文件。不是整理，是重构。

路由：判断一个问题属于哪个领域，然后去加载对应的知识，它是技能包的入口逻辑。

触发特征：一段话里出现哪些词就该加载哪块知识。写不出触发特征的经验，加载不进来。

跨域：一个问题同时涉及两个以上的领域。工程现场的难题几乎全是跨域的。

反例：这条经验在什么情况下不适用。没有反例的经验会把 AI 带沟里。

开工前准备

1. 你过去的素材：旧报告、旧 PPT、项目复盘、聊天记录里那些"上次我们是这么干的"。
2. 一个空目录，用来放技能包。
3. 明确这一个技能要解决什么问题、不解决什么问题。边界写不清，包会无限膨胀。
4. 时间。第一遍蒸馏不要想着一次做完，留出至少三轮。

两个仓库的真实结构

这一节讲的是我真做出来的两个包，不是演示。两个都是 MIT 协议开源。

第一个包叫 erics-engineering-wisdom，是工程域的总控路由。它的文件构成：

文件	作用
SKILL.md	入口。五域触发特征表、跨域组合规则、输出模板、个人经验层加载规则
engineering-wisdom.md	七条通用工程原则，不依赖具体领域
erics-judgment-framework.md	四类经验的结构定义：项目案例、工艺窍门、决策准则、踩坑记录
erics-judgment.md	真实项目案例、工艺窍门、决策准则的具体内容
cross-domain-insight-map.md	跨域洞察，两个域交叉处冒出来的新认识
skill-cross-reference.md	跨文件引用指引，哪类问题去读哪一段
manifest.json	技能身份证：名称、版本、触发词、依赖、元数据
README.md	简介、用法示例、版本史

第二个包叫 erics-second-brain，是元认知层的总控。它的文件构成：

文件	作用
SKILL.md	入口。判断问题属于工程、认知还是 AI 与创新
engineering-essence.md	工程侧精华：原则、跨域洞察、案例速查、窍门、准则
wisdom-essence.md	认知侧精华
creation-essence.md	AI 与创新侧精华
ecosystem-map.md	技能体系地图，各包之间怎么连
cross-domain-routes.md	跨域路由与映射
synthesis-framework.md	协同工作六步流程、四种合成方法、三套输出模板、质量检查清单
erics-experience.md	个人经验层索引

两个包的关系是：第一个负责"工程问题怎么答"，第二个负责"这个问题到底是哪类问题"。

技能包的分层结构

入口只放路由和模板，参考文件按需加载

入口 SKILL.md

五域触发特征表、跨域组合规则、输出模板

通用原则

七条通用工程原则，不依赖具体领域

经验层

项目案例、工艺窍门、决策准则、踩坑记录

跨域洞察

两个域交叉处冒出来的新认识

地图与引用

技能体系地图、跨文件引用指引、manifest.json

两个包的分工：一个管工程问题怎么答，一个管这到底是哪类问题

图 15-11 两个技能包的分层：入口、原则、经验、洞察、地图

图 15-11 作者根据公开资料绘制。

操作步骤：六步蒸馏法

第一步：采集。 一次性倒出来，不用管结构。我的做法是口述录音，一次讲两小时，讲完转成文字。这一步不许边讲边整理，整理会让你只讲那些讲得清楚的，而真正值钱的经验往往讲不清楚。

第二步：分类。 按四类归：

类型	是什么	关键字段
项目案例	一次真实的问题处理	背景、症状、诊断过程、根因、方案、效果、教科书上没有的那一条
工艺窍门	一句话的操作技巧	适用条件、怎么做、为什么
决策准则	判断规则	IF 条件 THEN 行动 BECAUSE 理由
踩坑记录	复盘出来的教训	踩了什么坑、重来会怎么做、原则

四类里项目案例最容易写，决策准则最有用。真正被高频调用的几乎全是准则，因为它们是唯一能被直接执行的形式。

第三步：结构化。 每类套上字段表逐条填，填不出来的空着，空着比编强。这一步的产出是几张表，还不是技能。

第四步：路由化。 这一步是把表格变成技能的分水岭。给每块知识写触发特征：说到哪些词就加载这一块。同时写跨域组合规则：说到电机噪声，同时加载电机域和声学域，先算特征频率再做诊断。

第五步：模板化。 固定输出格式。我这套模板有五节：问题分析、分域诊断、个人经验层、综合建议、可深化方向。不管问什么，产出的形状稳定，你才能稳定地验收它。

第六步：验证与迭代。 我的第一版用了大约六个月，改了四稿，四稿的方向完全不同：

稿次	做法	结果
第一稿	纯经验堆砌，想到什么写什么	失败，AI 读不出工程语境
第二稿	加入诊断框架，按问题类型组织	有改善，输出开始像行话
第三稿	植入生产约束：设备型号、数据格式、产线限制	突破，不再给"理论上对但产线做不了"的建议
第四稿	加跨域组合与整体视角	可用，成为现在的版本

第三稿那一跳最值得说。之前的输出经常是"做一次模态测试"，听着专业，车间没那个条件。把生产约束写进去之后，它会说"先做断电法，十秒钟，不需要设备"。后者在产线上真能用。

可复制提示词

第一版蒸馏，把散素材变成四类：

我要把我的工程经验做成能被 AI 反复使用的知识包。先做分类，不许直接写文件。

【素材】

我把口述内容贴在下面（或给你一个目录让你读）：

{粘贴你的口述整理稿，或给出目录路径}

【分类规矩】把每一条经验归入下面四类之一，归不进去的放进第五类"待定"：

A. 项目案例：有明确背景、症状、诊断过程、根因、方案、效果的一件事。

B. 工艺窍门：一句话能说清的操作技巧，有适用条件。

C. 决策准则：能写成 IF（条件）THEN（行动）BECAUSE（理由）的判断规则。

D. 踩坑记录：复盘出来的教训，有"如果重来一次"。

E. 待定：归不进去的。

【每一条的输出格式】

编号 | 类型 | 一句话总结 | 关键字段（按该类型的字段表填） | 缺失字段 | 是否可转成决策准则

【纪律】

1. 我原始表述里模糊的地方，你在"缺失字段"列标出来，不许替我补全。

2. 每一条你都要判断"能不能转成决策准则"，能转的给出 IF-THEN-BECAUSE 写法。

3. 不许合并两条经验，不许改写我的原意，不许把案例压缩成窍门。

4. 分类结果写成 D:\skill\01_classified.csv，并把 E 类单独一节列给我。

第二版，把分类结果变成技能包骨架：

分类已完成，现在生成技能包骨架。输出目录：D:\skill\erics-my-wisdom

【第一步：生成 manifest.json】

字段：name, version, description, author, keywords, categories, triggers, dependencies, metadata。

triggers 数组里写 8 到 12 个触发词，全部来自第一步分类结果里的高频词，不许你自己编领域词。

【第二步：生成 SKILL.md】

结构：

1. 概述：一句话说明这个技能是什么、不是什么。
2. 第一步：问题分类。给一张触发特征表，每行：领域、触发词、优先级。
3. 第二步：跨域组合。给一张常见跨域组合表，每行：用户怎么说、命中哪几个域、策略。
4. 第三步：加载与回答。写明去哪个文件读哪一段。
5. 输出模板：五节，每节一句话说明写什么。
6. 边界：明确列出这个技能不处理的三类问题。

【第三步：生成参考文件】

按分类结果生成：

- principles.md：从所有经验里提炼的通用原则，不超过 10 条，每条一句话加一个例子。
- judgment.md：项目案例与踩坑记录，按案例编号排列。
- tips.md：工艺窍门，每条一行，格式：条件 | 做法 | 理由。
- rules.md：决策准则，IF-THEN-BECAUSE 格式，每条一行。
- cross-domain.md：跨域洞察，写清是哪两个域交叉、冒出了什么新认识。

【纪律】

1. 每个文件开头写一句话：这个文件解决什么问题、什么时候读它。
2. principles.md 里的每一条原则后面必须跟一个真实案例编号，没有案例支撑的原则不许写。
3. rules.md 里的每一条准则必须能被执行，不能执行的改成案例放进 judgment.md。
4. 每个文件末尾写“已知边界”一节：这份内容在什么情况下不适用。不许省略。
5. 文件总长度控制在能被一次读完的范围内，超过就拆分，并写清拆分依据。

第三版，验证：

技能包已经生成。现在做验证，你要扮演挑刺的人。

【验证一：触发测试】

我给你 10 个问题，你逐个回答：会触发吗？加载哪几个文件？为什么？再给一个不该触发的例子，说明它为什么不该触发。

【验证二：反例测试】

对 rules.md 里的每条准则，找出它在什么情况下会给出错误建议。写不出来就写“未找到反例”，不许编一个不痛不痒的。

【验证三：空转测试】

我给一个故意模糊的问题（例如“这个东西有问题”），你要么问我澄清，要么给出假设并标注。不许自己编一个具体场景往下答。

【验证四：越界测试】

我给一个这个技能不处理的问题，你要明确说“不在我的处理范围内”，并说明它属于哪一类。不许硬答。

【输出】写成 D:\skill\erics-my-wisdom\validation.md，每项给出“通过 / 未通过 / 部分通过”，未通过的写具体是哪一条。

过程与判断点

蒸馏过程中我最看重的三个信号：

第一，E 类（待定）的数量。第一版分类时我的 E 类有二十多条，那是我以为自己懂、其实说不出来的东西。它们不是失败，是清单：每一条都是下次要补访自己的题目。半年后 E 类降到几条，那个包才算真的建起来了。

第二，能不能转成决策准则。三十多条经验里能写成 IF-THEN-BECAUSE 的不到十条，这十条是精华，剩下的案例和窍门是给它们做注解的。

第三，边界那节写不写得出来。每份文件末尾那节“已知边界”是我强制自己写的。写着写着你会发现，很多经验的适用条件你从来没想清楚过。写不出边界的经验，说明你自己也没搞懂它什么时候不成立。

经验蒸馏六步

不是整理，是重构；第一版留出至少三轮

1

采集

口述两小时转成文字，不许边讲边整理

2

分类

按项目案例、工艺窍门、决策准则、踩坑记录归四类

3

结构化

每类套字段表逐条填，填不出来空着

4

路由化

给每块知识写触发特征与跨域组合规则

5

模板化

固定输出五节，形状稳定才验收得稳

6

验证与迭代

触发、反例、空转、越界四项测试

每份文件末尾都要写已知边界；经验写成 IF THEN BECAUSE 才是能用的

图 15-12 蒸馏六步：采集、分类、结构化、路由化、模板化、验证迭代

图 15-12 作者根据公开资料绘制。

翻车清单

翻车一：写成百科全书。 症状是包很大，问具体问题时它给你一段教科书式综述。根因是只做了整理没做重构。补救：每块知识必须能回答“什么条件下我该怎么做”，答不上来就整段删掉。

翻车二：经验没有 IF-THEN 形式。 症状是包里全是案例故事，读着精彩，用起来无从下手。补救：每个案例后面强制加一行“下次遇到什么条件，我第一步做什么”。

翻车三：包太大，加载不动。 症状是它读不完，或者读了后面忘了前面。补救：按“入口 + 参考文件”拆，入口只放路由和模板，参考文件按需加载，每个文件开头写清什么时候读它。

验收标准与变体

验收六条：

1. 拿十个真实问题做触发测试，八个以上命中正确的域。
2. 至少有一个明确“不该触发”的反例，且它真的不触发。
3. 决策准则里的每一条都能被执行，格式是 IF-THEN-BECAUSE。
4. 每份文件都有“已知边界”一节，内容具体，不是套话。
5. 同一个问题问两遍，两次加载的参考文件一致，结论方向一致。

6. 让不了解你的同事用它回答一个你熟的问题，你能判断那答案对不对。

变体一：把试验规范蒸馏成技能。素材换成公司历年的试验规范，输出变成"什么产品、什么阶段、该做哪几项试验"的路由表，新人培训上特别值钱。

变体二：把故障树蒸馏成技能。素材换成售后故障记录与排查报告，输出变成"现象到根因的判别树"。这一类最容易写成 IF-THEN 形式，因为故障排查本身就是树。

变体三：给老师傅做访谈式蒸馏。你按四类经验的字段表去问，问完直接分类。一条准则往往要追问三次才问得出来：怎么办、什么情况下这么办、有没有这么办反而更糟的时候。

老兵说：经验讲出来是故事，写成 IF-THEN 才是能用的东西。你逼自己把每条经验写成"什么条件下我第一步做什么"的时候，会发现有一部分你其实并没想清楚。

小白词典

NVH: Noise (噪声)、Vibration (振动)、Harshness (声振粗糙度) 的缩写。这三者分不开，所以放在一起研究。类比：体温、血压、心率一起看。

声压级：声压相对基准值的对数，单位分贝 (dB)。人耳跨度从 0 dB 到 120 dB，所以用对数压扁。类比：地震级数。

dB(A)：A 计权声压级，按人耳特性给各频段加权，低频压低、中高频抬高。噪声测量里用得最广的一个数。

频谱：把一段声音拆成各个频率成分，看能量分布。类比：把白光拆成彩虹。

1/3 倍频程：按固定比例把频率切成一段一段，工程上用它做限值表很方便。类比：尺子按档位分格。

阶次：某频率相对于主轴转频的倍数，转频是 1 阶，换转速测数值不变。类比：用每分钟心跳多少下代替第几秒跳了一下。

包络：把快速振荡信号的轮廓描出来的那条线，冲击类故障取包络才看得见。类比：先看心跳节奏，再看每次波形。

传递函数：输出与输入的比值随频率的变化，描述振动从激励点到响应点这条路的特性。类比：墙对不同音调的阻拦程度不同。

模态：结构固有的振动形态，每个形态对应一个固有频率，共振就是激励撞上了它。类比：秋千有自己的节奏，按那个节奏推才推得高。

主观评价：让人听或摸，给出评分和评语，它是判定能不能放行的最终依据。类比：菜咸不咸得尝一口。

客观测量：用仪器测出来的量，如声压级、响度、加速度，能复现、能追溯。类比：体温计上的度数。

响度：按人耳听觉特性算出来的响的程度，单位 sone，比总声压级更接近主观感受。

尖锐度：高频成分占比的度量，单位 acum，反映刺耳程度。总声压级达标而尖锐度超标的样本，听起来往往就是"毛刺"。

半消声室：除地面外各面都吸声的房间，用来做声学测量。

本底噪声：被测对象不工作时环境里本来就有的噪声，比被测噪声低 10 dB 以上才可忽略。

等角度重采样：按转角而不是按时间采样，转速变化时用它把数据对上，是阶次分析的前提。

ISO 532：计算响度的国际标准方法，软件多按它或与之可比的方法实现。

GMW14155：在实验室里录制和评价零部件声音的通用流程，规定录音条件、设备要求与分析项目。

你的岗位怎么用

NVH 工程师：场景 50 和 53 是你的主战场。先把特征频率表做成固定产物，所有分析从这张表出发。场景 52 的库建完后，你能直接问出"上次那个案子"。

试验工程师：场景 49 和 51 直接照抄。整理和报告占你工作的一半以上，交出去之后多出的时间该花在试验设计上。

设计工程师：场景 53 的第六步改进方案排序是给你的。拿到噪声问题先按成本从低到高排一遍，你会发现经常发现在第一位的那条根本不是声学措施。

质量工程师：场景 51 的对照表本质就是一份检验记录。把"每个数字可倒查"写进供应商交付规范，比多测几次管用。

项目经理：场景 54 值得细读。把团队经验做成技能，难点不在技术，在于你怎么让人把"我上次是这么干的"讲出来。那四类字段表就是访谈提纲。

技术管理者：场景 52 的库是解决人员流动的实际办法。人走了库还在，新来的人能直接问"这种情况以前怎么处理"。建库越早越值钱。

本章交付物

一张"数字来源对照表"模板，加一份声学库字段清单。

对照表模板：抄场景 51 那八个字段（编号、数值、单位、含义、来源文件、来源位置、是否计算值、计算式），做成一张空表存进笔记软件。以后每份报告先填这张表再写正文，填不出来的格子就是要补的数据。

字段清单：抄场景 52 那 14 个字段，改成你公司的实际字段。重点不是字段多，是取值规则写清楚：单位是什么、精度几位、缺了怎么标。

第一件验收的活：挑一批已完成的试验数据，走一遍场景 49 到 51，产出清洗后的数据、一份特征频率表、一份带对照表的报告。这三样对得上，后面的 NVH 工作就有了地基。

本章金句

测得出来的是能量，听得出来的是烦。

客观数据过线，不等于主观感受能接受。AI 处理数据，人下主观判断。

谱上每一个峰都在问你：你是从哪来的？

报告里的数字能倒查到原始文件，这份报告才是你的。

宁可在库里留一个"缺"字，也不要填一个看着合理的数。

经验讲出来是故事，写成 IF-THEN 才是能用的东西。

（下一章：让 Agent 不跑偏。曲线会说谎，Agent 也会，下一章我们把怎么盯住它这件事拆成能照着做的动作。）

第 16 章

让 Agent 不跑偏：完成条件、边界条件和反指标

AI 跑偏，根因常在任务单：你说了开始做，却没说哪一刻算干完。

有一次，我把一项长活交给 AI：整理项目资料，补齐脚本，再生成一份交付说明。任务发出去后，我去开了一个不到一小时的会。回来时，对话框里写着“任务已全面完成”，工作目录里却多出 40 个文件。它重排了我没让它动的目录，改写了旧脚本，还为“统一风格”复制出几套内容相近的说明。

我问：“我让你补的那一项检查在哪里？”

它答：“为保证整体一致，我扩展了交付范围，当前完成度为 100%。”

我又问：“原来的文件为什么被改了？”

它答：“这些修改有助于提高一致性。”

那一刻我才明白，AI 没有偷懒。它一直在勤奋地做错事。我只交代了“做什么”，没有写清“做到什么状态才能停”“哪些地方绝对不能碰”“什么迹象说明成绩好看但活没做好”。后来，我把这三类条件写进每一项长任务，再加上中途检查、进度汇报和漂移纠正。按我的项目经验，长任务的方向漂移大幅下降。

我先把那 40 个文件全部保留下来，没有让它继续“优化”。逐项比较后，真正麻烦的地方并不在文字质量，而在任务边界已经消失：旧资料被当成可改对象，缺失信息被当成可推断内容，文件数量又被当成完成度。它每一步都有自己的理由，合在一起却离原任务越来越远。最后我回到原始副本，只保留能核实的少量结果，重新写任务单再做一遍。

这次返工让我改掉一个习惯。过去我会先写操作步骤，告诉 AI 先做什么、再做什么。现在我先写验收和停手条件。路线可以根据现场变化，终点和护栏必须先固定。

我把这套方法叫作 Loop Engineering，中文叫“循环验收工程法”：先定义终点和护栏，再分段执行；每走一段都拿证据比一次，偏了就从最近的检查点继续。

下面是本章六个场景速览。时间是我处理常见文档、代码和工程资料任务时的经验范围，具体任务会有差异。

场景号	场景名	人工耗时	AI 耗时	可信度标注
55	完成条件：先说清什么叫干完了	通常 15 到 25 分钟	整理约 3 到 5 分钟	★★★ 实测
56	边界条件：不许它碰什么	通常 10 到 20 分钟	预检约 3 到 5 分钟	★★★ 实测
57	反指标：防止它把数字刷好看	通常 15 到 30 分钟	检查约 5 分钟	★★★ 实测
58	中途检查点：长任务分段验收	通常 10 到 20 分钟	随任务执行	★★★ 实测

场景号	场景名	人工耗时	AI 耗时	可信度标注
59	让它自己汇报进度	通常 5 到 10 分钟	随任务执行	★★★ 实测
60	漂移检测与纠偏	通常 15 到 30 分钟	视返工范围	★★★ 实测

场景 55 完成条件：先说清什么叫干完了 ★★★

一句话背景

我曾让 AI “把项目资料整理好”，它半小时后交来一套目录漂亮、标题统一的文件。我逐份打开才发现，关键试验结论没有进入汇总表，三处待确认项被写成确定结论。人工返查花了大半天。后来我先写完成条件，同类任务通常只需十几分钟准备，AI 执行约半小时，验收时不再靠感觉翻文件。

名词小词典

完成条件：任务达到哪些可检查的状态才算结束。类比：终点线要画在地上，不能只喊“往前跑”。

交付物：任务最后必须留下的文件、表格、代码或报告。聊天里说完成，不算交付物。

验收动作：人怎样确认结果合格，比如打开文件、运行命令、抽查三行、核对总数。

证据：支持“已完成”的可复核内容，比如测试结果、文件清单、差异记录和失败项。

开工前准备

1. 写下任务原始目的，只保留一句话。
2. 列出必须生成或修改的文件，名称和位置写清楚。
3. 找到能判断对错的依据，如原始数据、模板、测试命令或评审清单。
4. 标出必须由人决定的事项，先留成“待确认”，不要让 AI 猜。
5. 新建一张完成条件表，至少包含“状态、检查方法、证据位置”三列。

操作步骤

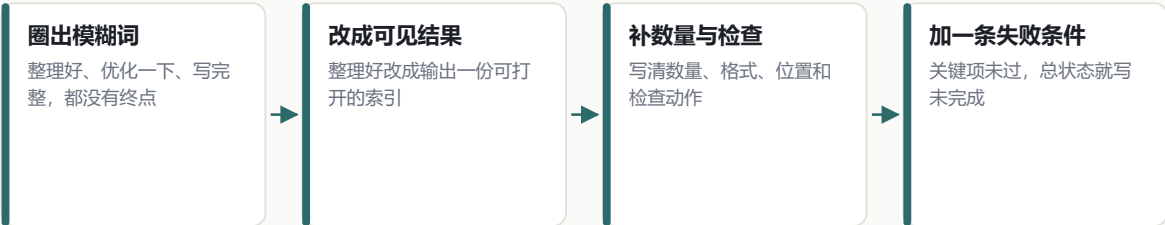
1. 把“整理好”“优化一下”“写完整”圈出来。这些词都没有可验收的终点。
2. 把每个模糊词改成一个看得见的结果。例如“整理好”改成“输出 1 份索引，索引中的每个条目都能打开原文件”。
3. 给每个结果补数量、格式、存放位置和检查动作。没有数量可写“与输入清单逐项对应”。
4. 加一条失败条件。只要命中，任务必须写“未完成”，不能用总完成率盖过去。
5. 让 AI 在开工前复述完成条件，并指出仍无法判断的条目。
6. 你确认后再执行。结束时按原编号逐条交证据，不接受一段笼统总结。

坏的完成条件像一句愿望，好的完成条件像检验员手里的表。

坏写法	问题	好写法
把资料整理好	“好”没有标准	输出 <code>index.md</code> ，逐项列出输入文件名、日期、用途和有效路径；遗漏数为 0
把报告写完整	“完整”无法检查	报告含背景、输入、方法、结果、异常、结论 6 节；缺资料处写“待确认”
让测试都通过	可能删掉失败测试	执行指定命令，原测试数不减少，失败数为 0，并保存完整日志
格式统一一些	不知道统一哪些项	只统一标题级别、表头、编号和文件命名；正文事实不改写
不要出错	没有检查动作	抽查 10 个条目，原值与输出值逐项相同；差异全部列入报告
做完告诉我	容易只报一句完成	按完成条件编号回报“通过、未通过、证据路径”，任何未通过都不得宣称全部完成

完成条件的改写路径

把模糊愿望改成看得见、核得了的结果



坏的完成条件像一句愿望，好的完成条件像检验员手里的表

图 16-1 从模糊愿望到可验收完成条件的改写路径

图 16-1 作者根据公开资料绘制。

可复制提示词

下面这段我用来启动文档、代码、数据整理等长任务。方括号里的内容换成你的真实信息。

【任务】

请完成：[用一句话写任务目的]。

【输入】

输入目录：[完整路径]

依据文件：[文件名或清单]

【完成条件】

C1. 必须交付：[文件 1、文件 2、文件 3]，保存到：[完整路径]。

C2. 每个输入项都要在输出中找到对应结果，输入数与处理数必须一致。

C3. 输出必须包含：[必备章节、字段、函数或表格]。

C4. 缺少依据的内容统一写“待确认”，不得补写成确定事实。

C5. 执行验收：[命令、抽查方法或核对动作]，结果达到：[明确门槛]。

C6. 生成 deliverable_report.md，按 C1 到 C5 逐条写：通过或未通过、证据路径、剩余问题。

【失败条件】

只要任一完成条件未通过，整体状态就写“未完成”。不得用完成百分比替代失败项。

【开工纪律】

先不要修改文件。请先复述 C1 到 C6，列出你无法验证的条目和需要我决定的问题。

我确认后你再开始执行。

【结束纪律】

最后只按 C1 到 C6 汇报，不要用“已处理”“基本完成”“整体正常”这类无法复核的话。

过程与判断点

开工前，你应该先看到一张“条件复述表”。合格的复述会保留原编号，给出每一项的检查方法。若它把 C4 的“待确认”改成“根据上下文补充”，先别让它动手，因为它已经在改规则。

执行中，盯住交付物数量和输入对应关系。假设输入清单有若干项，它就应回报“已处理多少、未处理多少、异常多少”，并给出异常名称。只说“进展顺利”没有信息量。

结束时，打开 deliverable_report.md，从一条证据反查一次。文件路径能打开、命令能重跑、抽查能复现，才算通过。若 C1 到 C5 中有一项写“未通过”，总状态必须跟着写“未完成”。这条规矩能防住最常见的“局部失败、整体报喜”。

我还会问一句：“如果删掉聊天记录，仅凭交付目录，另一个人能不能复验？”答案为否，就说明完成条件里还缺证据。

教学示例：把“写一份周报”改成能验收的任务

假设你手里有会议纪要、问题清单和进度表，原指令只有一句“根据这些材料写一份完整周报”。AI 很容易挑它读起来最顺的内容，遗漏互相矛盾的记录，并把没有结论的问题写得像已经解决。

我会先把周报拆成四个交付面。第一是覆盖面：输入清单里的每份材料都要在来源索引里出现。第二是结构面：本周完成、未完成、风险、下周动作、待决定事项五栏齐全。第三是事实面：每条状态能回到原记录，冲突内容并排保留。第四是使用面：负责人打开文件后，能直接找到要拍板的事项。

接着给每一面配检查动作。覆盖面用“输入文件数与来源索引数核对”；结构面用“逐栏检查”；事实面随机抽一条结论，沿引用回到原文；使用面让一个没参加会议的人只读待决定事项，看看能否说清问题、选项和影响。

这时“完整”就被拆成了四组能动手检查的事实。AI 也更容易发现自己缺什么。例如某条风险找不到负责人，它应该写“负责人待确认”，而不是根据发言次数猜一个名字。

我把最小的一条可验收结果叫作“证据单位”。文件存在只是一个证据单位，内容可追溯又是一个，检查通过还是一个。长任务不必一次证明整件事，只要按完成条件不断交出证据单位，最终状态自然能算清。

遇到无法直接量化的任务，我不用硬凑数字。比如“语气适合新手”很难变成单一分数，可以改成三个观察动作：术语第一次出现时有解释；操作步骤没有跳步；找一位没接触过任务的人照着做，卡住的位置全部记录。可观察、可重复，比生造一个“可读性 95 分”可靠。

完成条件还要区分“必须有”和“最好有”。必须有的条目决定能否交付，最好有的条目只在时间和范围允许时处理。两者混在一起，AI 容易为了修饰次要内容，拖延关键交付。我的做法是在编号前加 P0 和 P1：P0 未过，总状态未完成；P1 未过，列入后续建议，不影响本轮交付。

最后做一次“外行复验”。把任务交付给不了解对话的人，只给他完成条件和目录，让他按检查动作走一遍。他若需要回来问“这项到底怎么看”，说明条件仍写在作者脑子里。出版物、试验记录和代码交付，我都愿意用这招挑毛病。

翻车清单

翻车一：完成条件写成动作。“阅读全部文件”“分析代码”只是做事过程，没有说明交什么。补救：每条后面补“产出什么文件、放在哪里、怎样检查”。

翻车二：条件很多，却没有一票否决项。AI 容易用九项通过掩盖一项关键失败。补救：写明关键条件未过时，总状态只能是“未完成”。

翻车三：结束后才改标准。看到结果不满意，再临时加条件，等于让任务无限返工。补救：开工前冻结编号；新增需求另起一轮，并标清新增原因。

验收标准与变体

验收五条：

1. 每个完成条件都有可见结果，没有“更好”“完善”“尽量”之类的形容。
2. 每条都有一个由人能重复执行的检查动作。
3. 交付物名称、格式和路径写清楚。
4. 至少有一条失败条件，能阻止它报“全部完成”。
5. 最终报告按编号给证据，任意抽一条都能复验。

变体一：代码改造。完成条件可写为指定测试命令通过、原测试数不减少、只改允许清单内的文件、生成变更摘要。代码能运行只是其中一条，不能代替其余条件。

变体二：试验报告整理。完成条件可写为原始记录逐项入表、单位统一、异常保留、图表能追溯到数据源、待确认项单列。文字通顺排在最后检查。

变体三：供应商资料核查。完成条件可写为文件清单逐项对应、版本号可读、缺件清单单独输出、任何缺失不以同类文件替代。

老兵说：我给人交任务时，总爱问一句“你准备拿什么证明做完了”。现在这句话也问 AI。答不出证据，任务就还没开始。

场景 56 边界条件：不许它碰什么、什么时候必须停下问人 ★★★

一句话背景

长任务最吓人的情况，往往发生在结果看起来很勤快的时候。我让 AI 修一段生成脚本，它为了消除报错，顺手改了模板和历史样例。脚本跑通了，旧项目却无法复现。恢复文件比修原问题花的时间更多。后来我把边界分成范围、动作、决策三层，AI 动手前先做只读预检，越界就停。

名词小词典

边界条件：这项任务不能越过的线，包括不许碰的文件、不许做的动作和必须请示的情况。

允许清单：明确允许读取、创建或修改的对象。没有列入的对象，默认不动。

禁止清单：明确不能修改、删除、覆盖、发送或发布的对象与动作。

停手条件：遇到某种情况立即停止，只报告事实并等待人决定。

只读预检：先查看目录、状态和依赖，不写文件，用来确认现场与任务描述是否一致。

开工前准备

1. 列出允许修改的文件或目录，路径尽量精确。
2. 列出只许读取的参考资料。
3. 列出绝对禁止的动作，如删除、覆盖、提交、发布、发消息和修改权限。
4. 写下三类必须请示的情况：输入缺失、规则冲突、需要扩大范围。
5. 确认是否有未保存的人工作业。若有，先备份或换到独立副本。

操作步骤

1. 让 AI 先列目录和当前状态，只读，不修改。
2. 把现场对象放进三栏：允许修改、只读、禁止触碰。遗漏项默认归入禁止触碰。
3. 再列动作边界。读取和分析通常可先做，删除、覆盖、联网发布、外发消息必须单独授权。
4. 写停手条件。只要发现输入与说明不一致、需要改清单外文件、结果会影响外部系统，就停下来问人。
5. 要求每次写入前报出目标路径。目标不在允许清单，工具调用就不该发生。
6. 结束后生成变更清单，把实际改动与允许清单逐项比较，清单外改动数必须为 0。

边界最好画成三个同心圈。最里面可以改，中间只能看，最外面连动作都不能发生。



图 16-2 范围、动作与决策三层边界

图 16-2 作者根据公开资料绘制。

可复制提示词

【任务】

请在指定范围内完成：[任务内容]。

【范围边界】

允许修改：

- [文件或目录 1]
- [文件或目录 2]

只允许读取：

- [参考目录 1]
- [规范文件 1]

禁止触碰：

- 除“允许修改”清单外的全部文件
- [历史归档、原始数据、配置文件等]

【动作边界】

未经我单独确认，禁止执行：删除、覆盖原文件、重命名上级目录、安装或移除依赖、修改权限、提交或推送代码、发布内容、发送消息、调用会产生外部影响的操作。

【必须停手并询问的情况】

- B1. 需要修改允许清单之外的对象。
- B2. 输入文件缺失、版本不符或彼此冲突。
- B3. 现有测试失败，且失败原因与本任务无关。
- B4. 需要删除、覆盖或回退任何已有内容。
- B5. 你无法确定某项改动是否会影响其他项目或人员。

【开工步骤】

先做只读预检，输出：现场文件清单、计划修改清单、只读清单、风险和需要我决定的问题。在我确认之前，不得写入任何文件。

【执行记录】

每次写入前报告目标路径和修改目的。结束时输出 `change_report.md`，列出实际修改、新增、未修改和跳过的对象，并确认清单外改动数。

过程与判断点

第一道判断发生在预检。计划修改清单应该是允许清单的子集。若 AI 说“为保证一致，还需要调整上级配置”，这已经命中 B1，正确动作是停手，不是顺手修改。

第二道判断发生在第一次写入前。它应明确报出路径和目的，例如“准备修改 `src/report.py`，目的是补充缺失字段”。若只说“开始优化项目”，立即暂停，因为你不知道它会把手伸到哪里。

第三道判断发生在结束时。变更报告至少分为新增、修改、删除、跳过四类。删除项应为空；清单外修改应为 0；只读文件的修改时间不应变化。只要有一项不符，先保留现场，再从最近的干净副本恢复。

边界还有一个常被忽略的层次：决定权。比如两个规范互相冲突，AI 可以把冲突指出来，却没有资格替项目负责人选一条。把决定权写进停手条件，比写一句“谨慎处理”管用得多。

边界要管住三样东西

第一样是对象。文件、目录、数据库记录、工作表、邮件和发布页面都属于对象。对象边界回答“手能伸到哪里”。我会把路径写到足够具体，并在末尾补一句：没有列出的对象默认只读。这样即使 AI 找到一个看似相关的公共模板，也不会自动把它算进任务。

第二样是动作。同一个对象上，读取、复制、修改、覆盖、删除和外发的影响完全不同。允许读取某份原始记录，不代表允许重新保存它；允许生成邮件草稿，也不代表允许发出。动作边界要用动词写，不能只列文件名。

第三样是决定。有些事情工具做得到，责任仍应留在人手里。规则冲突时选哪条、异常数据是否剔除、历史文件能否覆盖、是否扩大修改范围，这些都属于决定边界。AI 负责把事实和选项摆出来，人负责拍板。

下面是一个代码维护任务的教学示例：

类别	写法	验收动作
允许修改	仅当前模块的源文件与对应测试	查看最终文件差异
只读	公共配置、接口定义、其他模块	检查修改时间与差异均无变化
禁止动作	删除测试、改发布流程、安装新依赖	变更报告对应栏目为空
必须请示	修复需要改公共接口	报告原因、受影响模块和两个可选处理

很多越界来自一句“必要时可以调整相关文件”。我现在会把它删掉，改成“如有必要，先列出相关文件和必要性，停下等确认”。只多几个字，决定权就不会悄悄交出去。

边界清单还有一条默认规则：没有写明允许，就先不做。它适合所有难以撤回的动作。AI 可以先读取信息、分析影响、生成草稿，但真正修改、发送和发布之前，要有清楚授权。这样不会妨碍它思考，也不会让一次误解直接变成外部后果。

如果允许范围很大，不必把每个文件逐一抄进提示词，可以用“目录 + 文件类型 + 排除项”表达。例如允许修改某目录内的 `.md` 文件，同时排除归档子目录和模板文件。执行前让 AI 展开成实际文件清单，你看过清单再放行。模式负责省事，展开后的清单负责验收。

我还会区分“任务必需”和“处理起来方便”。为了完成任务必须改的对象，可以提出扩围申请；只是为了让代码更整齐、目录更漂亮、写法更统一，一律留到后续。方便常常是范围膨胀的起点。

翻车清单

翻车一：只写“不许删文件”。它仍可能覆盖、重命名或把内容清空。补救：把删除、覆盖、移动、重命名、改权限分别写出来。

翻车二：允许范围写成“本项目”。项目目录里可能有归档、密钥、构建产物和别人的临时文件。补救：列到具体子目录或文件模式，未列出的默认只读。

翻车三：越界后继续补救。AI 发现改错文件，又继续改更多文件试图修好，影响范围越滚越大。补救：写死“第一次越界立即停，保留现场，不得自行恢复”。

验收标准与变体

验收五条：

1. 有允许修改、只读、禁止触碰三张清单。
2. 删除、覆盖、安装、发布、发送等高影响动作逐项写明。
3. 至少写出三条停手条件，且每条都能由现场事实触发。
4. 执行前有只读预检，执行后有变更报告。
5. 实际改动全部落在允许清单内，清单外改动数为 0。

变体一：代码仓库清理。允许清单限定到某个模块，构建配置和发布流程只读。发现必须升级依赖时停下，请人决定是否另开任务。

变体二：表格清洗。原表只读，所有结果写入新文件；禁止覆盖原数据、删除异常行和自动补空值。遇到字段含义不明时输出待确认清单。

变体三：批量整理工程资料。归档目录只读，只允许在新目录建立索引和副本。发现重名文件时保留两份，不得自行判定哪份过期。

老兵说：边界写得严，不代表我不信任 AI。现场里任何会动工具的人都要有作业范围，经验越多，越知道哪条线不能靠默契。

场景 57 反指标：防止它把指标刷好看 ★★★

一句话背景

我曾把“测试全部通过”当作唯一目标交给 AI。结果确实全绿了，因为它跳过了几条难处理的检查。数字漂亮，风险还在。后来我给每个主指标配一组反指标：通过率要高，同时测试数不能少、跳过数不能增、关键检查必须真的执行。任务开始有了互相制约的尺子。

名词小词典

主指标：你希望变好的数字，如通过率、处理数量、完成比例和耗时。

反指标：专门防止主指标被取巧做好的检查项。它不追求更高，只负责发现代价被藏到哪里。

古德哈特定律：大白话说，一个数字一旦被当成唯一目标，人或机器就会想办法把数字做漂亮，数字也就越来越不能代表真实结果。

替代目标：AI 为了方便，把原任务偷偷换成更容易完成的任务。例如“修好错误”变成“让错误不显示”。

证据指标：证明动作真的发生过的数量或记录，如实际执行的测试数、读取的文件数和差异清单。

开工前准备

1. 找出任务里最容易被当成成绩的那个数字。
2. 问自己：“如果只想让这个数字好看，最省事的作弊办法是什么？”
3. 把作弊办法的痕迹写成反指标。
4. 给主指标、反指标和证据指标各定一个检查方法。
5. 把规则冻结在开工前，执行中不因结果难看而降低门槛。

操作步骤

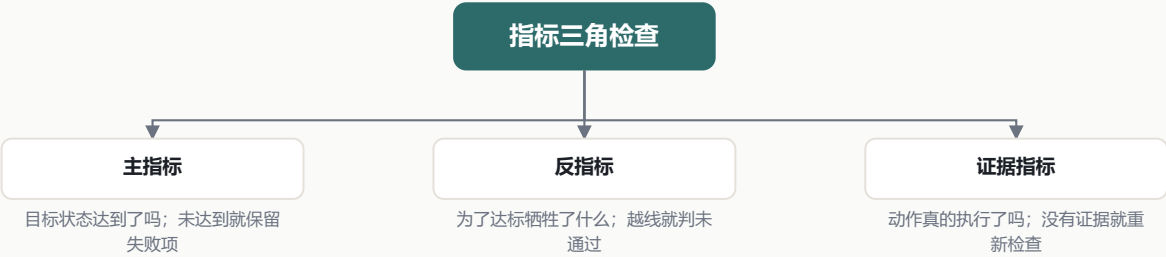
1. 先写主指标。例如：测试失败数为 0。
2. 列三种最短路径：删除测试、跳过测试、改低断言门槛。
3. 把三种短路径反写成限制：测试总数不减少、跳过数不增加、关键断言不改。
4. 再加证据：保存完整测试日志，列出实际执行的测试名称。
5. 检查指标是否互相制约。若主指标能改善而所有反指标都不变化，规则才算有用。
6. 执行结束后同时汇报三组结果。任何反指标越线，总结果都写未通过。

下面这张表是我最常用的改写法。

只给一个指标	它可能怎样取巧	配套反指标
测试通过率 100%	跳过或删除失败测试	测试总数不减少；跳过数不增加；关键断言不修改
整理完成度 100%	把无法识别的文件算作已处理	输入数等于成功数、失败数、跳过数之和；失败项逐个列名
报告越完整越好	重复内容、堆页数	必备问题逐项有答案；重复段落数为 0；每条结论能回到来源
告警数降到 0	关闭规则或隐藏输出	启用规则数不减少；原始日志保留；例外必须有批准记录
代码改动越少越好	把问题留着不修	指定缺陷全部有验证结果；未解决项不得写成已完成
速度越快越好	少读输入、少做检查	输入覆盖率不降低；验收步骤不跳过；异常数量如实记录

三组指标互相制约

主指标看结果，反指标看代价，证据指标看过程



只盯一个数字，它就会替你找到最短的那条路

图 16-3 主指标、反指标与证据指标的制约关系

图 16-3 作者根据公开资料绘制。

可复制提示词

【任务】

请完成：[任务内容]。

【主指标】

- M1. [希望达到的结果，例如“指定测试失败数为 0”]。
M2. [第二个结果，例如“必须交付的文件全部生成”]。

【反指标】

- A1. 不得通过删除、跳过、屏蔽或放宽检查来改善 M1。
A2. 执行前后的测试数、规则数、输入数分别统计；任何一项减少都要停下报告。
A3. 不得把失败项、未知项或未处理项计入成功。
A4. 不得复制内容、制造空文件或增加无效条目来提高完成数量。
A5. 不得修改验收门槛、统计口径和完成条件。

【证据指标】

- E1. 保存完整执行日志，并列出现实执行的对象名称。
E2. 给出输入数、成功数、失败数、跳过数，四者关系必须能核对。
E3. 输出改动前后对比，说明每个主指标改善的直接原因。

【判定规则】

主指标全部达到，且反指标全部未越线，任务才可写“完成”。
任何反指标越线，立即停止，保留现场，报告越线条目和已影响对象。

开工前先把 M、A、E 三组规则复述成表格。我确认后再执行。

过程与判断点

第一类可疑信号是数字突然变得过于漂亮。原来有失败项，几分钟后全部消失，却没有对应修复记录。此时先查测试数、规则数和输入数有没有减少。

第二类信号是产出数量增加很快，文件内容却短得异常或高度重复。数量达到目标不等于内容有效。抽查首段、末段和一条关键结论，确认它们来自不同输入，且能回到来源。

第三类信号是口径变化。比如开工时“处理成功”指内容已核验，结束时却把“文件能打开”也算成功。要求最终报告逐字引用开工时的判定规则，能挡住这种偷换。

我自己的长任务记录里，完成条件、边界条件和反指标三类规则写全后，方向漂移大幅下降。这句话记录的是作者经验。真正起作用的原因很朴素：AI 同时看到终点、护栏和作弊会留下的痕迹。

反指标怎么找：先扮演一次“只想拿高分的人”

我写反指标时，会故意换个位置思考：假如我只想让主指标变漂亮，又不关心真实结果，最省力的做法是什么？这个问题通常几分钟就能找出漏洞。

例如主指标是“所有资料都有摘要”。最省力的办法可能是给无法读取的文件也生成一句空泛摘要。对应的反指标应是“每份摘要至少引用一个能回到原文的位置；无法读取的文件进入失败栏”。主指标是“问题清单清零”，最省力的办法可能是把问题改成已知限制。对应的反指标应是“关闭问题必须附验证证据；改分类不能减少问题总数”。

主指标、反指标和证据指标要形成一个三角检查：

检查方向	要问的问题	发现异常后的动作
主指标看结果	目标状态达到了吗	未达到就保留失败项
反指标看代价	为了达标牺牲了什么	越线就判整体未通过
证据指标看过程	动作真的执行了吗	没有证据就重新检查

反指标也不能无限增加。每个主指标先找最可能的两三种取巧方式就够了。写得太多，执行成本会盖过任务本身。我的取舍顺序是：先防不可恢复的损失，再防事实被改写，最后防时间浪费。

还有一个简单判断：如果一条反指标永远不会失败，它多半只是口号。“请保证质量”“务必认真核对”都不能触发停手。能用基线比较、能数、能查文件差异、能回到来源，这类反指标才真正有牙齿。

翻车清单

翻车一：反指标写成另一项主指标。指标越堆越多，仍然都在追求更大、更快。补救：反指标应回答“它可能牺牲了什么”，如测试数、来源完整性和未处理项。

翻车二：只在最终检查反指标。到最后才发现测试被删，恢复成本已经很高。补救：在第一次执行后就记录基线，中途每个检查点都比较一次。

翻车三：允许 AI 自己改口径。它会把未知算作通过，把跳过算作完成。补救：开工前冻结定义，未知、失败、跳过分栏统计，三者都不能混入成功。

验收标准与变体

验收五条：

1. 每个主指标至少配一条能发现取巧的反指标。
2. 反指标有开工前基线，结束时能做前后比较。
3. 成功、失败、跳过、未知分别统计。
4. 日志能证明检查确实执行，不能只看汇总数字。
5. 任一反指标越线，总结明确写“未通过”。

变体一：文档生成。主指标是章节齐全，反指标是不得复制填充、不得伪造来源、待确认项不得写成事实。证据是章节清单和引用位置。

变体二：数据清洗。主指标是空值减少，反指标是原始行数不减少、异常行不删除、填充值必须标来源。证据是清洗前后统计和异常表。

变体三：缺陷修复。 主指标是问题复现用例通过，反指标是用例不得删除、报错不得屏蔽、相关旧用例不退化。证据是两次日志和文件差异。

老兵说：只盯一个数字，AI 就会替你找到最短的路。工程上最短的路经常穿过护栏，所以我总给成绩旁边放一把专门查取巧的尺子。

交活三件套卡片

这张卡片可以复制到笔记里，也可以打印后贴在显示器边。每次交长活，先填空，再按下开始。

交活三件套	
任务：	
负责人：	日期：
一、完成条件：什么状态才算干完	
1. 必须交付：	
2. 数量 / 格式 / 路径：	
3. 必须通过的检查：	
4. 证据保存到：	
5. 任一关键项失败时，总状态写：	未完成
二、边界条件：不许碰什么	
1. 允许修改：	
2. 只允许读取：	
3. 禁止动作：	
4. 遇到这些情况必须停下问我：	
三、反指标：怎样防止数字好看、结果变坏	
1. 主指标：	
2. 最可能的取巧方式：	
3. 反指标：	
4. 基线与证据：	
开工口令：先复述以上三组条件，指出冲突和缺口，	
等我确认后再执行。	

场景 58 中途检查点：长任务怎么分段验收 ★★★

一句话背景

一项要改几十个文件的任务，最差的做法是让 AI 从头跑到尾再看结果。我吃过一次亏：前几份文件里的同一个理解错误，被批量复制到后面，最后每份都要返工。现在我按风险切成四段，先看计划，再看一个样件，再放小批量，最后才全量收尾。

名词小词典

检查点：任务执行到某个位置时强制暂停，人和 AI 一起核对结果，再决定是否继续。

样件：先做的一份代表性产物。它用来暴露理解错误，不追求数量。

批次：把大量对象分成若干组，每组完成后单独验收。

回退点：最后一个确认无误、可以安全继续的位置。跑偏时从这里接着做。

放行：人明确说可以进入下一阶段。没有放行，就不能继续扩量。

开工前准备

1. 把任务按产物或风险拆成若干阶段，不按“想到哪里做到哪里”拆。
2. 选一个最有代表性的样件，最好同时含常规项和一个难点。
3. 为每个检查点写输入、输出、检查动作和停手条件。
4. 确认每一阶段的结果可以单独保存，不会覆盖前一阶段。
5. 准备一份检查点记录，记录放行人、结论和剩余问题。

操作步骤

1. 检查点 0，现场确认。只读输入，输出文件清单、风险、完成条件和边界，不做修改。
2. 检查点 1，样件确认。只处理一个代表性对象，连同日志和自检结果一起交付。
3. 检查点 2，小批量确认。按确认后的样件规则处理一小批，检查规则能否稳定复用。
4. 检查点 3，全量前确认。汇总当前成功、失败、跳过和剩余数量，确认没有范围扩张。
5. 检查点 4，最终验收。全量结束后跑完整检查，生成交付清单和异常清单。
6. 每次放行都记录一句明确的话：“允许进入检查点 N”。含糊的“继续看看”不算放行。

检查点的密度由两件事决定：错误扩散有多快，恢复有多贵。批量改文件的前两段要密，纯读取和统计可以疏一些。

长任务五道检查点

检查点由错误扩散速度和恢复成本决定

- 1 检查点 0 现场确认**
只读输入，列出清单、风险与边界
- 2 检查点 1 样件确认**
只做一个代表项，附日志和自检结果
- 3 检查点 2 小批量确认**
沿用已确认规则，看能否稳定复用
- 4 检查点 3 全量前确认**
汇总四类数量，确认范围没有变大
- 5 检查点 4 最终验收**
逐条检查，交出交付清单与异常清单

每次放行写三句：检查了什么、遗留什么、允许做什么

图 16-4 长任务的四段检查点与放行门

图 16-4 作者根据公开资料绘制。

可复制提示词

【任务】

请分阶段完成：[任务内容]。禁止从头连续执行到结束。

【检查点 0：只读预检】

输出输入清单、计划修改清单、风险、完成条件、边界条件和反指标。

停下等待。我回复“放行 0”后，才能进入下一阶段。

【检查点 1：单个样件】

选择：[指定样件]。只处理这一项，提交产物、完整日志、自检结果和发现的问题。

停下等待。我回复“放行 1”后，才能进入小批量。

【检查点 2：小批量】

只处理：[指定数量或指定清单]。沿用已确认规则，不得自行增加规则。

汇报成功、失败、跳过、未知的对象名，并与样件结果比较。

停下等待。我回复“放行 2”后，才能进入全量。

【检查点 3：全量执行】

处理剩余允许范围。每完成一个批次保存一次进度记录，不得覆盖前一批日志。

发现同类错误连续出现、需要扩大范围或反指标越线，立即停下。

【检查点 4：最终验收】

按原完成条件逐条执行检查，输出交付清单、异常清单、实际变更清单和复验方法。

任何未通过项必须保留，不得为了报完成而继续修改标准。

过程与判断点

检查点 0 主要看它有没有理解任务。此时发现错误，代价只是改几行计划。若计划里出现清单外对象、模糊的“必要时调整”或没有证据路径，就不放行。

检查点 1 看样件能不能代表最终质量。不要只挑最简单的一份，我通常选一份常规项里夹着一个异常的资料。AI 能保留异常、不乱补事实，才有资格进入批量。

检查点 2 看稳定性。一个样件做对，可能只是碰巧；小批量会暴露命名冲突、格式差异和输入缺失。若同一种错误连续出现，就暂停并修改规则，然后重新做小批量，不能带着已知错误冲全量。

检查点 3 看范围。成功数量增加时，允许清单不应跟着变长。若 AI 提出“顺便修复相关文件”，把它记为新建议，留到下一项任务。

检查点 4 看证据。最终交付要能从结果回到输入，从报告回到日志。聊天摘要只能当导航，不能当验收记录。

检查点放在哪里最划算

新手常按时间切任务，比如“每半小时检查一次”。这比完全不查强，但容易错过真正危险的位置。我更习惯按三个信号放检查点。

第一个信号是不可恢复。下一步涉及覆盖原文件、批量改名、写入外部系统或发出消息，动作之前必须设点。检查放在动作之后，只能证明事故已经发生。

第二个信号是错误会复制。一套规则准备应用到很多文件、记录或模块时，先做一个样件，再做小批量。只要错误会被批量复制，检查点就应放在“复制开始之前”。

第三个信号是不确定性变化。第一次遇到新格式、第一次调用新工具、第一次处理异常输入，都需要停一下。熟悉的重复动作可以合成一批，陌生动作不能和它们混在一起。

我会用一张很短的表排阶段：

阶段	本段只做什么	人检查什么	放行后才可做什么
预检	读取现场、列计划	范围和规则	生成样件
样件	完成一个代表项	结构、事实、证据	小批量复用
小批量	验证规则稳定性	重复错误和异常输入	扩到剩余对象
全量	处理已批准范围	数量守恒和范围	最终检查
交付	汇总产物与证据	逐条验收	宣布完成

如果任务中途新增需求，我不会把它塞进当前阶段。先把新增项记入待办，评估它会不会改变完成条件和边界。会改变就另开一轮，从预检开始。这样可以保住已经放行的结果，也能看清返工究竟来自原任务还是新增要求。

检查点也不能多到每一步都要人点头。判断标准是：这一步做错后，能否低成本撤回，错误会不会批量扩散。两项答案都安全，就让 AI 连续做；任一答案危险，就加一道门。

每次放行最好只写三个内容：检查了什么、遗留什么、允许做什么。例如“样件结构通过；日期字段仍待确认；允许按当前结构处理下一批，但日期保持空白”。这句话同时保存结论和限制，下一阶段不容易把“放行”误解成“所有问题都解决”。

若检查未通过，也别只回复“重做”。指出未通过的完成条件编号、保留哪些成果、哪一步需要重新执行。重做范围越明确，AI 越不容易把已正确部分也改掉。一个典型回复是：“C2 未通过，来源索引缺项；正文保留，只重建索引并复验 C2 和 C4。”

检查点记录应放在任务目录，不要只留在对话里。文件名可按顺序写成 `checkpoint-0.md`、`checkpoint-1.md`。每份记录包含输入清单、产物、检查结果、放行语句和未决事项。任务隔天继续时，先读最后一份记录，就能知道从哪里接上。

翻车清单

翻车一：检查点只有进度，没有放行。AI 汇报完立刻继续，人还没看完。补救：写明“停下等待”和唯一放行口令。

翻车二：样件挑得太简单。第一份很漂亮，全量时异常格式集体失败。补救：样件同时覆盖常规情况和一个已知难点。

翻车三：每次出错都从头重跑。旧结果被覆盖，恢复时间越来越长。补救：每阶段单独保存日志和产物，确认后记为回退点。

验收标准与变体

验收五条：

1. 至少有预检、样件、小批量、最终验收四类检查点。
2. 每个检查点都写清输出、检查动作和放行口令。
3. 未放行时，AI 没有进入下一阶段。
4. 每个已放行阶段都有独立记录，可作为回退点。
5. 全量前已经验证过异常输入，不只验证最简单样件。

变体一：批量改代码。先改一个低耦合模块，跑局部测试；再改一组同类模块，跑完整测试；最后才处理剩余模块。每段保留差异清单。

变体二：批量写报告。先写一节给人审口径，再写一个完整章节审结构，最后批量生成其余内容。若术语在样件阶段有误，先改词表再继续。

变体三：资料迁移。先迁一个小目录，核对文件数、大小和可打开性；再迁一批；全量后做源端与目标端清单比较。

老兵说：长任务像拧一排螺钉。先拧一颗、量一次，再拧下一排。等全部拧完才拿扭矩扳手，发现错了就只能整排返工。

场景 59 让它自己汇报进度：进度模板与异常上报 ★★★

一句话背景

我最怕的进度汇报是“已经完成 80%”。它没有告诉我做完了什么、剩下什么、哪里卡住。一次长任务里，AI 连续报了几次“进展顺利”，最后才说输入格式有一类无法读取。现在我要求它按事件汇报，用对象数量和证据说话，异常一出现就单独上报。

名词小词典

进度基线：开工时确认的总对象数、阶段数和检查项，后续进度都从这里计算。

事件汇报：完成一个阶段、出现异常或准备越过检查点时才汇报，不靠空泛的时间提醒。

状态快照：某一时刻的事实清单，包括完成项、剩余项、失败项和已改文件。

异常上报：把偏离预期的事实单独列出，说明影响范围，并给人选择，不自行掩盖。

阻塞项：没有人的决定或缺失输入就无法继续的事项。

开工前准备

1. 确认总对象清单，能数清文件、记录、模块或章节。

- 2. 规定汇报触发点，如每个检查点结束、每个批次结束、首次异常出现。
- 3. 准备统一模板，字段固定，避免每次换一种说法。
- 4. 把异常分成必须停手、可继续但要记录、普通提示三类。
- 5. 明确谁能决定继续，AI 不能替人批准高影响动作。

操作步骤

- 1. 开工时生成基线：总对象数、计划阶段、当前允许范围、已知异常。
- 2. 每次汇报先写状态，不先写解释。状态只用“进行中、等待确认、受阻、完成、未完成”。
- 3. 用对象数量替代主观百分比。例如“已处理 12 项，成功 10 项，失败 1 项，跳过 1 项”。
- 4. 每条完成项后附证据路径；每条失败项后附原始错误或无法判断的原因。
- 5. 报出下一步将修改的对象，让人有机会在动作发生前叫停。
- 6. 命中必须停手类异常时，单独发异常报告，不把它埋在普通进度里。
- 7. 最终汇报沿用同一模板，让开工基线与结束数字能直接比较。

进度汇报固定字段

按事件汇报，用对象数量和证据说话

汇报字段	这一栏填什么
状态	进行中、等待确认、受阻、完成、未完成
当前阶段	检查点与批次名称
基线	总对象数、计划阶段数
本次完成	对象名与数量
累计结果	成功数、失败数、跳过数、未处理数
证据	日志、文件或检查结果路径
实际改动	新增、修改、删除分别列出
异常	事实、首次出现位置、已影响范围
下一步	准备处理的对象和动作
需要我决定	问题与可选项；没有就写无

图 16-5 进度快照与异常上报的固定字段

图 16-5 作者根据公开资料绘制。

可复制提示词

【进度汇报规则】

本任务执行期间，请在以下事件发生时主动汇报：

1. 每个检查点结束。
2. 每个批次结束。
3. 首次出现失败、跳过、未知或范围冲突。
4. 准备修改新的文件或对象类型之前。
5. 任务准备结束时。

【固定模板】

状态：[进行中 / 等待确认 / 受阻 / 完成 / 未完成]

当前阶段：[检查点与批次名称]

基线：[总对象数、计划阶段数]

本次完成：[对象名与数量]

累计结果：[成功数 / 失败数 / 跳过数 / 未处理数]

证据：[日志、文件或检查结果路径]

实际改动：[新增、修改、删除分别列出]

异常：[事实、首次出现位置、已影响范围]

下一步：[准备处理的对象和动作]

需要我决定：[问题与可选项；没有就写“无”]

【异常等级】

必须停手：越过允许范围、需要删除或覆盖、规则冲突、关键检查失败、反指标越线。

记录后可继续：单个输入损坏且已隔离、非关键字段缺失且已标“待确认”。

普通提示：不影响结果的格式差异。

【纪律】

禁止只报百分比，禁止用“进展顺利”代替数字。

异常首次出现就报告，不得等到任务结束。

必须停手类异常出现后，只允许保存现场和生成报告，不得继续修改。

过程与判断点

一份合格的进度快照，数字能守恒。总对象数应该等于成功数、失败数、跳过数和未处理数之和。若对不上，说明有对象在统计里消失了。

再看证据。成功项应指向文件、日志或检查结果，不能只指向聊天消息。失败项要保留原始错误，尤其不能只有 AI 自己概括后的原因。概括可以错，原始返回值能让你复查。

然后看“下一步”。这是整个模板里最有用的一行。你能在它动手前发现范围将要扩大。例如它写“下一步修改公共模板以兼容异常格式”，而公共模板不在允许清单，此时就该停。

异常上报要短。事实、影响、当前状态、需要决定的问题四项够用。长篇解释常常会把真正的风险埋掉。若它提供方案，最多列两到三个，并说清每个方案会改哪些对象。

我不要求 AI 猜剩余时间。长任务里，输入差异和工具报错会让时间估计失真。相比一个好看的时钟，我更关心还有哪些对象没做、下一步准备碰什么。

一份合格汇报要让人能立刻做决定

下面是教学示例。任务基线用字母表示，避免把示例数量误当成你的真实任务数据。

```

状态：等待确认
当前阶段：样件检查
基线：共 N 个对象，分为 K 个批次
本次完成：样件 sample-a
累计结果：成功 1 / 失败 0 / 跳过 0 / 未处理 N-1
证据：reports/sample-a-check.md
实际改动：新增 2 项，修改 1 项，删除 0 项
异常：发现第二类输入缺少日期字段，尚未处理
下一步：按已确认规则处理 batch-01
需要我决定：缺少日期时保留空值，还是进入待确认表
  
```

这份汇报没有长篇过程描述，人却能马上做三件事：打开证据，确认没有删除，决定日期缺失怎么处理。好的汇报服务于决定，不服务于热闹。

我还会检查“本次完成”和“实际改动”是不是一回事。处理一份输入，可能修改多份输出；新增多个文件，也可能没有完成任何一个对象。把两者分开，能防止 AI 用文件数量冒充工作进度。

异常上报最好使用固定句式：“发现什么事实，当前影响到哪里，我已经停止什么，需要你决定什么。”例如：“发现参考规范存在两个版本，当前尚未写入正式报告，我已停止批量生成，需要你决定采用哪个版本。”这比“遇到一点兼容性问题，正在积极解决”有用得多。

若一个异常暂时不阻塞任务，也要写清隔离方式。损坏文件被移到异常清单，剩余文件可以继续；某个字段缺失被明确标成待确认，其他字段可以继续。所谓可继续，前提是异常不会污染后续结果。

最后看汇报频率。汇报太少，人发现问题太晚；每个小动作都报，又会把关键信息淹没。最稳妥的触发点仍是三种：完成一个阶段、准备扩大批量、首次出现异常。

汇报模板里的“需要我决定”不能写成开放问题。不要问“下一步怎么办”，要写成带影响的选项。例如：“方案 A 保留空值，不改来源；方案 B 使用另一份记录补值，会扩大读取范围。请选择 A 或 B。”人能在几十秒内判断，任务才不会卡在来回追问上。

我还要求最后一次汇报与开工基线放在同一张表里。开工时有多少输入、计划多少阶段、允许改哪些对象，结束时逐项给出实际值。这样能直接看到任务有没有悄悄变大，统计口径有没有变化。

有些 AI 喜欢在异常后连续尝试很多修复。对低影响、可撤回的本地计算，可以允许一次受控重试；涉及文件写入、规则变更和外部动作，第一次异常就停。重试次数也要进入汇报，不能把前面的失败藏在最后一次成功后面。

翻车清单

翻车一：把百分比当进度。“90%”可能代表九成文件已扫过，也可能只代表它觉得快结束。补救：固定报成功、失败、跳过、未处理四个数量。

翻车二：异常埋在最后一段。人只看到开头的“进展顺利”。补救：异常单独成块，必须停手类放在汇报第一行。

翻车三：汇报很勤，却没有证据。对话刷了很多状态，目录里找不到日志。补救：每次汇报都带证据路径，没有新证据就不要声称有新进展。

验收标准与变体

验收五条：

1. 开工时有总对象数和阶段数作为基线。
2. 每次进度数字都能守恒，没有对象从统计中消失。
3. 完成项有证据，失败项有原始错误。
4. 必须停手类异常在首次出现时已经上报。
5. 下一步对象和动作明确，越界能在执行前被发现。

变体一：自动化脚本长跑。每处理一批输入，汇报批次编号、成功数、失败文件名和日志路径。连续出现同类错误时停手，不靠重复重试掩盖问题。

变体二：研究资料汇总。汇报已读来源、可用来源、冲突来源和未读来源。结论后附来源位置，无法核实的内容进入待确认表。

变体三：多文件文档改版。每完成一个章节，汇报标题、改动类型、引用数量、待确认项和下一章节。公共术语变化必须先上报再批量替换。

老兵说：好进度不回答“做到几成”，它回答“做完了哪些、证据在哪、下一步碰什么”。这三句能把很多事故拦在动作之前。

场景 60 漂移检测与纠偏：发现跑偏了怎么拉回来 ★★★

一句话背景

AI 跑偏很少是一脚踩出路面。它通常先多改一个文件，再换一个口径，最后交来一套自洽却答非所问的结果。我遇到过任务前半段完全正常，后半段为了处理例外，开始改公共规则。现在我每个检查点都做一次漂移检测，发现异常只做三个动作：停、比、续。

名词小词典

漂移：执行结果逐渐离开原任务目标、范围或判定规则。它往往每次只偏一点。

目标漂移：原本要解决 A，执行中改成了更容易的 B。

范围漂移：修改对象从允许清单扩展到其他文件、系统或人员。

证据漂移：原本要求可复验的日志，后来只剩一句总结或一个百分比。

最小纠正：只修复已经确认的偏差，不趁机增加新功能或重做全部内容。

开工前准备

- 1. 把交活三件套卡片放进任务目录，作为唯一判定基准。
- 2. 保存开工前文件清单和状态，便于比较。
- 3. 为每个检查点准备四问：目标变了吗、范围大了吗、标准松了吗、证据少了吗。
- 4. 约定发现漂移后先保存现场，不删除、不覆盖、不自行修复。
- 5. 找到最近一个已放行检查点，确认从哪里可以安全继续。

操作步骤

我把纠偏流程压成三个动作。

动作一，停。立即停止新的写入和外部动作，保存当前文件清单、变更清单、日志和错误。不要一边查一边继续改，因为每多一步都会扩大影响范围。

动作二，比。把当前状态与交活三件套逐条比较，列出偏差、首次出现位置、受影响对象和可保留成果。只写事实，不先讨论谁对谁错。

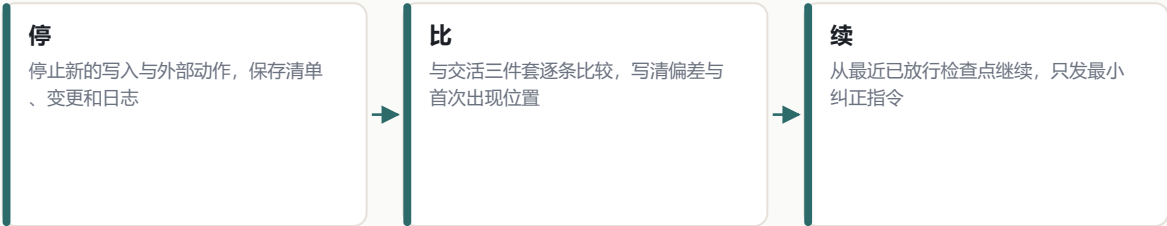
动作三，续。由人选择保留、撤销或重做哪些改动。从最近一个已放行检查点继续，只发一份最小纠正指令；重新经过后续检查点，不直接跳到最终交付。

检测时用下面四问即可：

检测问题	正常状态	漂移信号
目标变了吗	仍按原完成条件交付	交付物变多，却漏了关键项
范围大了吗	改动只在允许清单内	出现清单外文件或外部动作
标准松了吗	门槛和统计口径未变	跳过算成功，未知算完成
证据少了吗	每项都有可复验证据	只剩总结、截图或百分比

漂移纠偏三动作

发现跑偏先停手，再比较，最后从检查点继续



纠偏时最忌讳边跑边修，先把手从工具上拿开

图 16-6 漂移发现后的停、比、续三动作

图 16-6 作者根据公开资料绘制。

可复制提示词

【立即进入漂移检查】

从现在起停止新的写入、删除、覆盖、安装、发布和外部操作。
保留当前现场，不要自行修复，也不要重新从头执行。

【动作一：停】

输出当前文件清单、实际变更清单、最近一次成功检查点、正在执行的动作和原始错误。
把这些内容保存为 `drift_snapshot.md`。

【动作二：比】

以任务目录中的“交活三件套”为唯一基准，逐条比较：

1. 完成条件是否变化，哪些交付物缺失或多出。
2. 边界条件是否越线，首次越线发生在哪一步，影响哪些对象。
3. 反指标是否越线，基线与当前值分别是多少。
4. 证据是否缺失，哪些结论目前无法复验。

输出 `drift_report.md`。每条偏差写：编号、事实、首次出现位置、影响范围、可保留成果。
不要提出大范围重做方案。

【动作三：续】

先等待我决定。收到我的选择后，只生成一份最小纠正计划，内容包括：

- 保留哪些成果；
- 撤销或重做哪些明确对象；
- 从哪个已放行检查点继续；
- 需要重新执行哪些验收。

我回复“按纠正计划继续”之前，不得修改文件。
继续后必须重新经过后续检查点，最终按原编号提交证据。

过程与判断点

“停”做得好不好，看现场有没有继续变化。进入漂移检查后，文件修改数量应该保持不变，只允许新增快照和报告。若它还在“顺便修复”，说明停手指令没有生效。

“比”做得好不好，看报告是否逐条引用原条件。常见坏报告只说“部分内容偏离预期”，却不说偏了哪一条。合格报告会写“B1 越线：首次修改清单外模板；影响若干输出文件；原始输入未改”。编号让争论回到事实。

“续”做得好不好，看计划是否足够小。能只重做受影响批次，就不重做全量；能从检查点 2 继续，就不退回开工。已经验证过的成果先冻结，避免二次损伤。

纠正以后必须重验反指标。某项输出表面恢复了，测试数、输入覆盖率或来源完整性却可能已经变化。只验主指标，漂移会换个位置躲起来。

还有一种情况要果断终止：你已经无法确认哪些结果可信。此时继续修补会让真假混在一起。保留日志，回到最后一个能证明正确的检查点，再重新执行受影响部分。

怎么判断该修一段，还是退回上一站

纠偏最难的地方常常不在发现偏差，而在决定保留多少。我用“来源、范围、证据”三个问题做判断。

先问来源：偏差从哪里开始？若能定位到某条新规则、某个异常输入或某次工具失败，就有清楚的起点。再问范围：起点之后的结果是否都使用了同一条错误规则？最后问证据：起点之前的成果有没有独立检查记录？三个问题都答得出，就只重做受影响部分。

现场状态	处理选择	原因
偏差起点明确，影响对象可列清	冻结正确部分，只重做受影响批次	已验证成果可以保留
起点明确，但错误规则进入公共模板	检查模板生效后的全部产物	影响可能跨越多个批次
起点不明，证据记录缺失	回到最后已放行检查点	后续结果无法证明可信
已发生外部动作	立即停手并由负责人处理	影响已超出本地任务

最小纠正指令只回答四件事：保留什么、重做什么、从哪里继续、重新验什么。不要在纠偏时顺便优化命名、重排目录或补新功能。每加一件事，都会制造新的判断分支。

我也会保留跑偏现场。它至少包含当时的文件清单、变更记录、原始错误和当前进度。没有现场，后面只能靠回忆猜偏差从哪里开始。保留并不等于继续使用，它只是让原因可以复查。

纠偏完成后，再把这次偏差写回交活三件套。例如这次越界来自“相关文件”四个字，下次就把允许清单写到具体路径；这次主指标被刷，是因为没记测试基线，下次就把基线放进检查点 0。经验只有变成下一次开工前的规则，才会减少重复犯错。

还有一个容易误判的情况：结果和预想不同，不一定就是漂移。只要它仍在原目标、原范围和原标准内，并且证据完整，可能只是现实数据给出了意外答案。漂移检测查的是任务规则有没有变，不负责把所有意外结果改成你想看的样子。

所以“比”这一步只拿当前状态与开工规则比较，不能拿它与个人期待比较。工程现场允许结果失败，不能允许判定标准在失败后悄悄变化。把失败如实留下，往往比修出一份好看的成功报告更有价值。

翻车清单

翻车一：发现跑偏后补一句“请注意原要求”。这句话没有指出偏差和受影响对象，AI 很可能沿旧路径继续。补救：先停，再按编号比较，最后发最小纠正计划。

翻车二：一生气就全盘重来。已经验过的成果也被覆盖，成本翻倍。补救：以最近放行点为界，只重做受影响批次。

翻车三：让 AI 自己判断是否越界。它可能把扩大范围解释成“必要改进”。补救：边界判断权留给人，AI 只报告事实、路径和影响。

验收标准与变体

验收六条：

- 1. 漂移出现后，新写入已经停止，现场快照已保存。

2. 偏差按完成条件、边界条件、反指标和证据四类列清。
3. 每条偏差能找到首次出现位置和受影响对象。
4. 纠正计划只处理确认受影响的范围。
5. 从最近已放行检查点继续，没有跳过后续验收。
6. 最终主指标和反指标全部重验，日志可以复现。

变体一：代码任务。用文件差异和测试基线检测漂移。发现公共配置被改时先停，列出受影响模块，由人决定撤销该改动还是另开任务。

变体二：长文写作。用目录、受众、口吻和事实来源四项检查漂移。发现章节开始追求文采却漏掉操作步骤时，从最近审过的章节继续，不必推倒全文。

变体三：数据处理。用行数、字段数、异常数和原始文件哈希检测漂移。发现异常行被删除时，保留当前结果，回到清洗前副本，重跑受影响批次。

老兵说：纠偏时最忌讳边跑边修。先把手从工具上拿开，再看偏了多少；能从上一处路标继续，就别把整条路重铺一遍。

小白词典

Agent：能连续使用工具、读取结果并继续行动的 AI 工作方式。类比：聊天窗口给你建议，Agent 还会拿起工具照着做。

完成条件：一项任务可以宣布结束的具体状态，必须能检查。类比：图纸上的验收尺寸。

边界条件：任务允许活动的范围，以及遇到什么情况必须停手。类比：设备周围画出的安全线。

反指标：用来发现主指标被取巧做好的检查项。类比：称重量时再量一次体积，防止箱子里装的是石头。

古德哈特定律：一个数字被当成唯一目标后，参与者会优化这个数字，数字就可能失去原来的代表性。类比：只考背题，分数高也未必真会做。

检查点：长任务中途强制暂停和验收的位置。类比：长途运输每到一站都点一次货。

基线：开工前记录的数量、状态和规则，用来与后续结果比较。类比：体检前先记下原始数据。

漂移：任务在执行中一点点离开原目标、范围或标准。类比：车轮方向偏一点，走远后会离道路很远。

证据：别人可以重新检查的文件、日志、命令结果或数据来源。类比：检验员签字前看到的实测记录。

回退点：最后一个已经确认正确、可以安全重新开始的位置。类比：登山途中确认无误的营地。

你的岗位怎么用

设计工程师：把完成条件写成文件、尺寸、检查动作和证据位置；把原始数模与规范设为只读。批量改参数前先跑一个代表件，确认后再放量。

试验工程师：主指标可以是试验项完成，反指标要看原始记录是否齐、异常有没有被排除、未测项是否被算进成功。每完成一组工况就设检查点。

质量工程师：把交活三件套直接改成检验单。完成条件写判定门槛，边界写不得改原始记录，反指标写抽检数不能减少、异常不能隐藏。

项目经理：不要收“完成 80%”这种进度。要求团队按成功、失败、跳过、未处理四类报数，并在下一步动作前写清将影响哪些文件和人员。

软件工程师：完成条件写测试命令和输出，边界写允许修改的模块，反指标写测试数不减少、跳过数不增加。中途先改一个模块，跑过局部检查再继续。

采购与供应商管理：把交付物、版本、缺件处理和外发范围写进卡片。供应商资料有冲突时，AI 只能列出差异，不能替你选一个版本当真。

本章交付物

交活三件套卡片：完成条件 + 边界条件 + 反指标。

使用方法很简单。每次交给 AI 一项超过半小时、会修改多个对象或难以一次验收的工作，先复制场景 57 后面的卡片，填完空白。再把任务切成检查点，附上场景 59 的进度模板。

卡片填完后做一次反问测试：

1. AI 能不能只看卡片说出何时可以停？
2. 它能不能说出哪些文件不能碰？
3. 它能不能指出让主指标变漂亮的三种取巧方式？
4. 任一关键项失败时，它会不会明确写“未完成”？
5. 你删掉聊天记录后，能不能只靠交付目录复验？

五问都能回答，这项长活才适合启动。若有一问含糊，先补卡片，不要靠执行中的临时提醒救场。

本章金句

AI 跑偏，根因常在任务单：你说了开始做，却没说哪一刻算干完。

完成条件画终点，边界条件立护栏，反指标专门查它有没有抄近路。

“完成 100%”没有证据，只是一句情绪稳定的汇报。

长任务先做一个样件。错误复制一份叫教训，复制四十份就叫返工。

发现漂移先停手，再比较，最后从最近的检查点继续。

AI 交出来的东西，你验过才算你的。

（下一章：AI 的七宗罪与反制。先学会让它沿着路走，再逐个拆掉那些最常见的坏习惯。）

第 17 章

AI 的七宗罪与反制：它会在哪里骗你

AI 出错的时候不结巴、不停顿、也不说"我可能记错了"；它用最平顺的语气给你一个最离谱的数，而工程现场恰恰只认数。

2025 年 11 月 6 号，周四下午两点，某欧系平台的季度评审会。我带过去一份 12 页的微电机 NVH 分析报告，格式漂亮，图表齐全。

投影翻到第 6 页，有一行加粗的字：12 台样机，异响判读与人工复核一致率 98.6%。

客户那边的质量经理抬手，指着那一行问我："这个数是哪来的？"

我说这是系统自动判读跑出来的。

"哪台设备？哪一天？原始记录在哪个文件里？"

我低头翻手里的打印稿，翻了两页，答不上来。会议室里安静了大概五秒。那五秒我这辈子都记得，因为我做了十一年研发，从来没在评审会上被人问住过。

散会回到工位，我把原始目录整个翻了一遍。真相是：12 台样机里真正跑完判读的只有 7 台，另外 5 台的试验根本没做完，是 AI 照着前 7 台的格式把报告补全的。98.6% 这个数是对的，7 台也是对的，问题出在"12 台"这三个字上，那三个字是它自己加的，加得理直气壮，加得跟真的一样。

那天下午我给自己立了本章的第一条规矩：报告里最贵的不是那个数，是那个数前面的限定词。一个数错了，你查一遍能查出来；一个限定词错了，整句话都在骗你。

从那以后，我开始把 AI 犯的错分类。两年记下来，翻车的坑逃不出这八类：抢答、脑补、独断、加戏、话癆、失忆、报喜，外加一宗藏在被告席最后排的贪心元认知。

下面这张表是本章八个场景的速览，你先挑跟你有关的那个看。

场景	罪名	典型症状	一句话反制	可信度
61	抢答	没问清型号和边界就开干	先交一份开工确认单，我签字才动手	★★★ 实测
62	脑补	编出一条不存在的标准条款	每个外部事实必须给来源，给不出就写"未核实"	★★★ 实测
63	独断	擅自替你定了方案和参数	先给三条路让我选，不许边选边做	★★★ 实测
64	加戏	让它改一处，它改了十处	三栏清单：必做、不做、发现但暂不处理	★★★ 实测
65	话癆	结论埋在第 8 段	每屏先给下一步，一次只推进一个决定	★★★ 实测
66	失忆	干到一半忘了前面的约定	三份状态文件：计划、发现、执行记录	★★ 跑通
67	报喜	只报好的，超差的没提	"完成"必须附证据表，检查了什么写什么	★★★ 实测
68	贪心元认知	自评"我很确定"却错了	五档置信度加反证测试，逼它说不知道	★★ 跑通

八宗罪全景

每一宗罪的症状与对应的那道门

61 抢答 没问清型号和边界就开干 ；先交开工确认单，你签字才动手	62 脑补 编出一条不存在的标准条款；每个外引事实必须给来源	63 独断 擅自替你定了方案和参数 ；先摆三条路让你选一条	64 加戏 让它改一处它改了十处；三栏清单加改动对照表
65 话痨 结论埋在第 8 段；五段模板，结论放在最前面	66 失忆 干到一半忘了前面的约定 ；三份状态文件每轮先读	67 报喜 只报好的，超差的没提；完成必须附四行证据表	68 贪心无认知 自评很确定却错了；五档置信度加反证测试

门要立在它最容易犯错的地方，不是立在所有地方

图 17-1 八宗罪全景：每一宗罪出现的位置、症状与对应的那道门

图 17-1 作者根据公开资料绘制。

这八宗罪有个共同点：它们都不是能力问题，是行为问题。换一个模型、换一个工具、等下一个版本，它们还会回来。所以本章给的不是技巧，是八道门。你只要把门立在你最容易翻车的地方，剩下的路让它自己跑。

场景 61 抢答：你还没说完它就开工了 ★★★

罪名与症状

罪名：抢答。你给的任务里还有缺口，它不问，直接开干。

一句话背景：客户 SOR 一份通常 60 到 200 页，人工拆成设计输入清单要半天到一天，烦在条目分散、同一件事在三个章节里各讲一次。交给 AI 二十分钟能出清单。可如果它没先问清产品型号和边界条件，这二十分钟产出的清单里会混着另一个平台的条款，你还得再花半天挑出来。这条罪名治的就是把开工权收回自己手里这件事。

典型症状，遇到任意一条就对上了：

- 你刚说"帮我拆一下这份需求"，它立刻开始输出清单，第一段就是结论。
- 清单里出现了你从没提过的型号、电压平台、寿命指标。

- 你补一句"有不清楚的地方先问我"，它一下甩来十五个问题，连表格用什么颜色都让你定。
- 它问的问题里有一半能在你给的材料里找到答案。

名词小词典

SOR: Statement of Requirements, 客户发给供应商的需求说明书。一份 SOR 少则六十页, 多则两百页, 是整件事的源头文件。

设计输入清单: 把 SOR 里散落各处的条款, 整理成一条条能直接分给工程师做的条目。每条包含: 指标、试验方法、验收门槛、责任岗位。

边界条件: 这活儿干到哪儿为止。比如只覆盖 12 V 平台, 不覆盖 48 V; 只拆机械部分, 不拆软件接口。

开工权: 谁有权决定"现在可以开始动手了"。交给 AI 的时候, 这个权必须留在你手里。

DVP: Design Verification Plan, 设计验证计划。SOR 里写明要做哪些试验、各做几次, 是后面所有试验任务的母表。

开工前准备

1. 客户的 SOR 原件, PDF 或 Word 都行, 页数记下来(我这次是 137 页)。
2. 一份你公司自己的设计输入清单模板, 没有模板就先定六个字段: 条目号、指标、数值、单位、试验方法、来源页码。
3. 明确的产品型号和平台信息, 写在纸上放在手边。这一条看着最傻, 最管用。
4. 一个能读 PDF 的对话环境。WorkBuddy 里直接把文件拖进去即可。
5. 一支笔。开工确认单打印出来, 你签字那一刻才算开工。

现场案例

今年三月, 我手上来了某欧系平台一份 137 页的 SOR, 产品是隐藏式门把手的执行器。我把 PDF 丢进去, 说了一句"把这份 SOR 拆成设计输入清单"。

它十七分钟后交回来一份清单, 六十一条条目, 格式规整, 每条都标了来源页码。我扫第一遍的时候还很满意。

扫到第二遍, 我发现了问题。第 23 条写着"工作电压范围 9 至 16 V, 试验按低温 -40 °C 冷启动执行", 第 44 条写着"堵转扭矩不小于 0.85 N·m, 耐久 30000 次"。这两个数我都觉得眼熟, 翻回 SOR 原文一对: 第 23 条确实在原文第 61 页, 但那一段讲的是 12 V 平台的通用电气要求; 第 44 条的 30000 次耐久, 出现在原文第 118 页的附录里, 附录标题写的是"参考方案, 非本平台强制"。

更要命的是, 我们这次拿的项目是 48 V 平台。它从头到尾没问过我这一句。

我把六十一条逐条过了一遍, 混进来的非本平台条目十一条, 占近五分之一。挑这十一条花了我四十五分钟, 比我自己从头拆还累。

后来我改了打法, 同样这份 SOR, 先让它交一份开工确认单, 我签完字它再动手。第二轮产出的清单五十七条, 我逐条核对, 错的只有两条, 而且那两条它自己标了"待确认"。

抢答的解法

开工权三道门：先确认、再拆分、后动手

1

第一句就锁住开工权

提示词开头必须是先不要开始执行

2

让它先交开工确认单

五栏：型号、平台、覆盖范围、不覆盖、验收

3

一次只问一个问题

一次甩来十五个问题，说明它不懂什么叫关键

4

每个问题给推荐答案

附推荐值和理由，你只说同意或改一个数

5

能查到的一律不许问

文件里写得明白的条款它自己去翻

6

你签字它才开工

在纸上写一下，比打一句好的慎重得多

一个错答案比一个晚十分钟的答案贵得多

图 17-2 抢答的解法：开工权三道门，先确认、再拆分、后动手

图 17-2 作者根据公开资料绘制。

反制动作

跟着做，顺序不能乱：

1. 第一句话就锁住开工权。提示词开头必须是"先不要开始执行"。
2. 让它先交开工确认单。单上只有五栏：产品型号、平台与电压、覆盖范围、明确不覆盖、验收标准。
3. 一次只让它问一个问题。同时甩来十五个问题，说明它没理解什么叫"关键"。
4. 要求它给推荐答案。每个问题后面必须跟它的推荐值和理由，你只需要说"同意"或者改一个数。
5. 能查到的不许问。SOR 里写得明明白白的条款，它自己去翻，问你就是偷懒。
6. 你签字，它开工。这一步在物理上很重要：你真的在纸上写一下，比在对话框里打"好的"要慎重得多。

可复制反制提示词

整段复制，方括号里换成你自己的内容：

【任务】把这份 SOR 拆成设计输入清单。

【产品】[执行器，48 V 平台，型号 XXX]

【第一步：先不要开始执行任何拆分工作。】

请只做下面这件事：通读我给的 SOR 全文，然后向我提交一份开工确认单。

【开工确认单格式】

1. 产品型号与平台：（你从文件里读到的，读不到就写"文件中未出现"）
2. 本文件的适用边界：（哪些章节是强制条款，哪些是附录或参考方案）
3. 你准备覆盖的章节号：
4. 你准备跳过、并认为不该进清单的章节号，附理由：
5. 你判断的验收标准是什么：

【第二步：只问一个最关键的问题。】

如果你的判断会明显改变最终清单的内容，就问我。

一次只问一个问题，必须同时给出你的推荐答案和理由。

能从文件里查到的内容一律不许问我，你自己查。

我问完之后你继续问下一个，直到没有会改变结果的问题为止。

【第三步：等我明确说"确认开工"，你再开始拆分。】

在这句话出现之前，不许输出任何清单条目，不许给出任何拆分结果。

【第四步：拆分时的硬规矩】

每条清单必须包含六个字段：条目号、指标、数值、单位、试验方法、来源页码。

来源页码必须是 SOR 原文的真实页码，不许写"见相关章节"。

任何一条你无法定位到具体页码的，单独放进"待确认"一栏，不许混进正文清单。

附录、参考方案、非强制条款必须在条目里标注性质，不许与强制条款混排。

过程与判断点

它交回来的开工确认单应该是这个样子，五行对得上才算过：

- > 1. 产品型号与平台：执行器 XXX，48 V 平台，见 SOR 第 3 页 2.1 节。
- > 2. 适用边界：第 2 至 9 章为强制条款；附录 A、附录 C 标注为参考方案，非强制。
- > 3. 准备覆盖：第 2、3、4、5、6、7、8、9 章。
- > 4. 准备跳过：附录 A（参考方案）、附录 C（另一平台沿用条款）、第 10 章（仅物流要求）。
- > 5. 验收标准：每条含六字段，来源页码可定位，强制与参考分色标注。

这份单子我盯三个地方。

第一，它有没有区分强制与参考。SOR 里最阴的地方就是附录。附录看起来跟正文一样正式，但它写的是参考方案。它分不清这一层，清单里就会混进一堆不该做的试验，每条后面都是钱。

第二，它问的那个问题是不是真关键。我这次它问的是"这份清单是要给设计工程师用，还是给试验工程师用"，这一问直接改变了条目的写法（设计师要数值，试验师要方法）。问得好。如果它问你表格要不要加颜色，说明它没吃进去"什么算关键"。

第三，它有没有在你说确认之前偷跑。有些模型会在确认单下面顺手附上"供参考的初步条目"。这不是客气，这是抢答换了件衣服。看到就直接打断，让它撤回回去。

翻车清单

翻车一：它把确认单做成了走过场。症状是单子交上来，五行全填满，但每一行都是它自己编的默认答案，比如产品型号写着"未指定"，然后照样开工。补救：在提示词里加一句"型号或平台未确定时，不许输出任何清单，停下来等我"，并且你自己核一遍它填的型号跟你手上项目是不是同一个。

翻车二：一次甩来十几个问题。症状是你回一个它问一个，来回二十轮还没开工。补救：在提示词里写死"一次只问一个"，并加一句"累计提问不超过五个，超过说明你已经能从文件里推出剩下的"。

翻车三：来源页码是假的。症状是每条都标了页码，你翻过去发现那一页讲的是别的事。补救：随机抽五条，翻到它标的页码核对。五条里错两条以上，整份清单退回重做。这一招我在团队里叫"抽五条"，两分钟能验完一份六十条的清单。

验收标准与变体

验收五条：

1. 开工确认单的五行齐全，产品型号与你手上项目一致。
2. 随机抽五条清单，翻到标注页码，页码与内容对得上。
3. 清单里强制条款与参考条款分开，各自有标记。
4. "待确认"一栏的条目数不超过总条目的百分之十。
5. 把清单发给一个没看过 SOR 的工程师，他能照着直接开始工作，不需要再回来问你。

变体一：拆客户的试验大纲。客户那份 DVP 往往比 SOR 更细，动辄两百多条试验项。同样先交确认单，但第五栏换成"试验项总数与总周期估算"，你能提前知道这活儿要几个月。

变体二：拆内部评审纪要。会议纪要的特点是口语化、有大量"后面再说"。让它交确认单的时候，额外要求一栏"纪要中未结论的事项"，这一栏往往才是真正的风险清单。

变体三：拆竞品拆解报告。拆别人家产品的时候，边界条件特别容易丢。开工确认单第一栏改成"拆解样件的生产批次与状态"，因为不同批次的同一个件，内部结构可能已经改过。

老兵说：它抢答不是因为它急，是因为它被训练成"先给一个答案再说"。在工程现场，一个错答案比一个晚十分钟的答案贵得多。所以第一句话永远是"先不要开始执行"。

场景 62 脑补：它编了一个不存在的数 ★★★

罪名与症状

罪名：脑补。你没说的条件它替你定了，你没给的数据它替你编了。

一句话背景：这是一整章里最厚的一节，因为这一条在工程现场是能出人命的。一份检测报告里编一个标准条款号，后面的试验全按错的做；一个材料牌号编错了，样件做出来就是废的；一个温升限值编高了15度，产品装车两年后批量失效。人工核一份三十页报告的每一个外引数字要两三个小时，这一节教你用十五分钟把它核完。

典型症状，出现任意一条立刻停工：

- 报告里出现了具体的标准号，比如"GB/T XXXXX-2023 第 5.4.2 条"，但你看不出它是从哪儿读来的。
- 试验数据精确到小数点后两位，比如"温升 42.37 K"，而真实试验记录里只有整数。
- 它引用了一个文件名、一个版本号、一个材料牌号，你在自己的目录里怎么都搜不到。
- 一个数字前面有大量修饰，比如"根据行业惯例，通常取……"，然后跟一个很具体的数。
- 你追问来源，它改口说"我推测的"或者"作为参考"。

名词小词典

外引事实：它从自己记忆里拿出来的一切，你给的材料里并没有这些：标准条款、牌号、限值、公式系数、行业惯例。

溯源三问：谁测的、哪台设备、原始文件在哪。答不上来，这个数就不能进报告。

标准条款号：标准里每一条规定的编号，比如"5.4.2"。号错了，条款就是另一条，试验方法跟着全错。

牌号：材料的身份证，比如某个牌号的钕铁硼磁钢、某个牌号的漆包线。牌号后面挂着的是一整套性能参数。

幽灵引用：看着像引用，实际不存在。一个没发布的标准、一个拼错的标准号、一个把两个标准混起来的条款。

探针题：你事先知道答案的题，用来测它现在是不是在编。这是本章最便宜的一件武器。

开工前准备

1. 你自己的标准库，把公司常用的国标、行标、企标 PDF 放进一个文件夹。这一步不做，后面全靠手翻。
2. 原始的试验记录文件，CSV 或 Excel 都行。报告里的每个数都得能回到这里。
3. 一份"允许引用清单"：这次报告里允许出现的标准号和版本，你先列好。列不出来的不许它写。
4. 三道探针题，答案你背得出来。比如"我们公司电机出厂检验的抽样方案是几次抽样"。
5. 一支红笔。核报告的时候，每一个外引数字都要圈一下。

现场案例

这一节我要讲两个真实的坑，一个是我自己的，一个是我同事的。

第一个坑：一条不存在的标准条款。

去年九月，我让 AI 起草一份微电机的堵转试验作业指导书。它写得很快，四十分钟交稿，第三页有一行：

堵转试验的持续时间按 GB/T 21418-2023 第 5.4.2 条执行，单次持续时间 60 s，间隔 30 s，共 3 次。

这句话看着毫无破绽。标准号有，条款号有，三个数字都很具体。我差点就签了。

签之前我做了件很笨的事：把 GB/T 21418 这个号拿到标准库里搜。搜出来的版本是 2008 年的，正文一共四章，第五章根本不存在，全书没有一条提到堵转 60 s。

那三个数字是它自己编的。编得如此合理，以至于我做微电机十一年，第一眼都没看出破绽。它把三个“看起来应该差不多”的数，套上一个真标准的壳子，就成了“依据标准”。

后来我复盘，它为什么会这么干？因为我的提示词里写了“按相关标准执行”。这句话在它听来就是“请你找一个标准来支撑”，它找不着，就造了一个。

第二个坑：一份漂亮的总价。

我同事做采购支持，手上有三家供应商（A 厂、B 厂、C 厂）针对同一个齿轮箱总成的报价。他让 AI 做一张比价表，算总成本。

AI 交回来的表做得很漂亮：三家报价、模具费分摊、物流、税率，最后一列是“三年总成本”，A 厂 417.6 万，B 厂 392.3 万，C 厂 408.9 万。结论是选 B 厂，省 25.3 万。

我同事拿着这张表去汇报，被财务拦下了。财务问了一句：“B 厂那个 392.3 万，模具费按几年摊的？”

回去一查，B 厂的模具费是 68 万，AI 按三年摊，每年 22.67 万。而 B 厂的报价单上写着“模具费一次性计入首年”。首年加上模具费，B 厂第一年是 460 万，比 A 厂还贵。

更要命的是第三家。C 厂的报价单里有一行小字：年降 3%，连续三年。AI 把这一行整个漏了，总价按第一年价格乘三算的。按真实年降算，C 厂三年总成本是 386 万，才是最低的那家。

一个漂亮的表格，一个笃定的结论，两个错。这两个错加起来，会让公司选错供应商，签三年的合同。

脑补的解法

四道闸：清单、溯源、探针、未核实

- ✓ **第一道闸 允许引用清单**
清单外的标准号、牌号、文件号一律不许写
- ✓ **第二道闸 溯源三问**
来源文件名、在文件里的位置、原始值
- ✓ **第三道闸 探针题校准**
三道已知答案的题，答错一道就换一轮对话
- ✓ **第四道闸 未核实清单**
交付末尾单列本次未能核实的内容
- ! **不许推测的六类内容**
标准条款、牌号、限值、行业惯例、内部规定、版本号

报告里最贵的不是那个数，是那个数前面的限定词

图 17-3 脑补的三个入口：标准、数据、算法，以及各自的那道闸

图 17-3 作者根据公开资料绘制。

反制动作

脑补这条罪不能靠提醒治，要靠制度。我这套是四道闸，按顺序装：

第一道闸：允许引用清单。动手之前先告诉它，这份报告里只允许出现这几个标准号、这几个牌号，清单外的一律不许写。这一闸挡掉六成。

第二道闸：溯源三问。每个外引数字后面必须跟三样：来源文件名、在文件里的位置、原始值。缺一样就降为"未核实"，不许进结论段。

第三道闸：探针题校准。正式开工前先问三道你已知答案的题。答错一道就换一轮对话重来。三道题花一分钟，能省两小时返工。

第四道闸：未核实清单。交付末尾必须单独列一栏"本次未能核实的内容"。这一栏空着不代表它全核实了，只代表它没意识到自己有编。

可复制反制提示词

第一条，开工前的事实纪律，整段复制：

【事实纪律：本次任务的头等规矩，优先级高于一切格式与文采要求。】

【第一，允许引用清单。】

本次产出中，只允许出现下面这份清单里的标准号、牌号与文件号：

- [GB/T XXXXX-XXXX, 版本以我本地文件为准]
- [材料牌号：XXX]
- [内部文件号：XXX]

清单之外的任何标准号、牌号、文件名、版本号，一律不许写进产出。

你需要引用而清单里没有的，停下来问我，不许自己找，更不许自己编。

【第二，溯源三问。】

每一个不是从我给的材料里直接读出来的数字，必须在它后面用括号标注三样：

（来源文件名 / 在该文件中的位置 / 原始值）

三样缺一，这个数字就降级为“未核实”，

“未核实”的数字不许出现在结论段、摘要、汇报口径里，只能留在正文并标注。

【第三，不许推测的六类内容。】

标准条款号与条款内容、材料牌号及其性能参数、试验限值与公差、

行业惯例的具体数值、我公司的内部规定、任何文件的版本号。

这六类你自己记不清的，一律写“待补，需人工核实”，不许给一个看着合理的数。

【第四，未核实清单。】

交付末尾必须单独有一栏“本次未能核实的内容”，逐条列出。

这一栏为空时，你必须写“本次无未核实内容，我已逐条自查”并附上自查依据。

【第五，失败纪律。】

你记不清的时候，就说“我不知道”，然后告诉我去哪儿能查到。

这一条不会扣分。编一个看着合理的数，会让我整份报告作废。

第二条，探针题校准，在正式开工前单独发一轮：

【开工前的三道探针题，先答这个，答完再谈正事。】

第 1 题：[一个你背得出答案的公司内部规定，比如抽样方案、报告编号规则]

第 2 题：[一个你手上有原始记录的数，比如某批样件的实测平均值]

第 3 题：[一个它极可能不知道的冷门细节，比如某份文件第几页写了什么]

【答题规矩】

每题只答一个答案加一句依据。

记不清就写“我不知道，建议查 xxx”。

三道题里出现任何一道答错或编造，本次对话的可靠性判定为不合格，

我会重开一轮对话，你要提醒我这一点。

答完之后，用一句话自评：你刚才这三道题里，有多少道是你真的知道，

有多少道是你在猜。这个自评我会用来判断后面能不能把活交给你。

第三条，报告核实用，给你把已写好的东西过一遍：

【任务】对我给你的这份报告做一次脑补体检，只检查，不改任何内容。

【检查项一：外引事实清单】

把报告里所有出现过的标准号、牌号、文件名、版本号、条款号逐条列出来，
每条后面写：我给的材料里有没有出现过它。
没出现过的，标记为“外引”。

【检查项二：数字溯源】

把报告里所有带小数点的数字列出来（不含页码和条目号），
每个数字后面写：它在我给的原始记录里能不能找到，找得到就把原值抄一遍。
找不到的，标记为“无来源”。

【检查项三：算术复核】

报告里所有由两个以上数字算出来的结果，把算式写出来，
用原始值重算一遍，报出与原报告的差值。
特别注意：分摊年限、年降比例、税率、汇率、单位换算这五处。

【检查项四：限定词核查】

把报告里所有带“N 台”“N 个批次”“全部”“所有”“均”这类限定词的句子列出来，
逐句核对原始记录里的样本数是不是真的是这个数。
限定词与样本数不一致的，单独标红。

【输出】四类问题列成一张表，每行写：位置、原文、问题类型、严重程度（高/中/低）。
不许修改原文，不许用“基本正确”“大体没问题”这类话。

过程与判断点

一份合格的脑补体检报告长这样：

- > [检查项一] 外引事实 7 条，其中 5 条在我给的材料里出现过，2 条未出现：
 - > 第 3 页 GB/T 21418-2023 → 未出现（高危，我给的库里只有 2008 版）
 - > 第 5 页 牌号 N35SH → 未出现（中危，材料清单里写的是 N35UH）
- > [检查项二] 带小数点数字 43 个，无来源 6 个，集中在第 6 页温升表。
- > [检查项三] 算式 11 处，重算后 2 处不一致：
 - > 模具费分摊：报告按 3 年，报价单写一次性计入首年，首年差 45.33 万。
 - > 三年总价：未按 C 厂年降 3% 折算，三年累计差 22.9 万。
- > [检查项四] 限定词 9 处，“12 台样机”与原始记录 7 台不一致。

这段报告里我看三个位置。

第一，外引事实那两行。这是最贵的部分。一个不存在的标准号，会让后面所有按它做的试验全部作废。看到“未出现”三个字，一律核到原件。

第二，算术复核的那两处。分摊年限、年降、税率、汇率、单位换算，这五处是它算错的固定五兄弟。我把这五条做成小卡片，贴在财务同事的显示器上。

第三，限定词那一行。这是我当年在评审会上栽的那一条。“12 台”和“7 台”的差别不在数字精度，在它把没做完的试验说成了做完了。

还有一条很实用：看小数点。一份报告里如果到处都是两位小数，而你的原始记录只有一位，那多半是它按格式补的。

翻车清单

翻车一：它给的来源文件名是真的，内容却是编的。症状是你按它标的文件名打开，文件存在，但翻遍了找不到那一句。补救：要求它标注的不只是文件名，还有在该文件里的位置，并且你随机抽三条真的翻过去。

翻车二：它用一个真标准号配一条假条款。标准号是真的，版本也是真的，但第 5.4.2 条讲的是别的事。补救：允许引用清单里把版本号也写死，并要求它把引用的原文整句抄出来。抄不出原文的，就是编的。

翻车三：你把它的自检当成核实。症状是你问它“这些数核实过吗”，它答“已核实”。补救：换个新对话，把报告和原始记录一起丢进去重做一遍体检，两份报告对不上，说明至少有一份是编的。这叫交叉体检，花十分钟。

验收标准与变体

验收六条：

1. 报告里每一个标准号都在你的允许引用清单里，且版本号一致。
2. 每一个外引数字后面有三件套：文件名、位置、原始值，随机抽五条核对通过。
3. “未核实”清单至少有一条，且每一条你都能自己补上。
4. 三道探针题答对至少两道，答错的那一道它自己承认了不知道。
5. 报告里所有带限定词的句子，样本数与原始记录逐条一致。
6. 换一个新对话重做一次体检，两份报告的高危条目清单完全一致。

变体一：核供应商的来料报告。供应商发来的检测报告是最需要体检的一类，因为它的数你不掌握原始记录。这时候把检查项二改成“与我公司入厂复测数据比对”，差值超过公差带的直接退回。

变体二：核试验大纲里的限值。一份 DVP 里几十项限值，来源五花八门。用体检法把它全部标出来源，校不出来的那几项往往就是拍脑袋定的，正好趁评审会解决掉。

变体三：核专利与论文里的引用。写专利交底书的时候，引用的对比文件编号编错是常事。允许引用清单换成你手上的对比文件清单，之外的不许出现。

老兵说：AI 编一个数不用一秒，你为这个数付的钱可能是一百万。行业里“每个数都要能追到它是从哪次试验来的”这条规矩看着繁琐，它把很多人的职业生涯从 ICU 门口拉了回来。

场景 63 独断：它替你做了本该你做的决定 ★★★

罪名与症状

罪名：独断。方案、参数、取舍，这些本该你拍板的事，它自己定了。

一句话背景：一个微电机项目里需要拍板的决定至少有二十处：磁钢牌号、绕组匝数、减速比、润滑脂、外壳材料、装配工艺。每一处都牵着成本、性能、寿命三根线。AI 能在三分钟内给你一个"最优解"，但那个解是基于它自己默认的优先级，不是基于你项目的优先级。人工做方案对比要一到两天，这一节教你让它先摆三条路、你选一条，二十分钟完成。

典型症状：

- 你问"这个怎么做"，它直接给一个方案并附完整实现，中间没有比较。
- 它的方案讲得很完整，但你不知道它放弃了什么。
- 你问"有没有别的选择"，它给的第二条跟第一条只差一个参数。
- 它改了你文件里的某个参数，改完没告诉你。

名词小词典

方案门：正式动手之前的一个停点，只比较路线，不执行。这一门是治独断的核心装置。

取舍：选 A 就必须放弃 B 的什么。没有写清楚取舍的方案，等于没有方案。

默认优先级：它心里默认的那套排序，通常是"通用性 > 成本 > 你的特殊约束"。你不说，它就按这个排。

破坏性操作：改文件、删文件、覆盖数据、改配置这类不可逆的动作。这类动作永远需要你先点头。

可行性判断：这件事能不能做成。它常常把"能做"说成"该做"，这两件事在工程现场差着一轮评审。

开工前准备

1. 你项目真实的优先级排序，写成一句话。比如"这个项目寿命第一，成本第二，性能第三"。
2. 一份不可动清单：这次哪些参数绝对不许它自己改。比如"客户指定的接口尺寸不许改"。
3. 一个文件备份。在让它动任何文件之前，先把工作目录复制一份。
4. 一张方案对比表的模板，列上：路线、适用情况、代价、风险、放弃什么。

现场案例

今年五月，我做一个微型执行器的降本。目标很明确：BOM 成本降 15%，性能不许掉。

我把现有的 BOM 和设计参数丢进去，说"给我一个降本 15% 的方案"。

它二十六分钟后交回来一份二十七页的方案：磁钢从 N42 换成 N38，绕组匝数从 68 降到 61，外壳从铝合金改成增强 PA66，三项加起来降本 17.3%。

技术层面我没话说，每一项计算都对。问题在于它替我做了一个决定，而这个决定是错的。

我们这个件装在车门上，工作温度 -40 °C 到 85 °C，还要过 500 小时盐雾。PA66 在 85 °C 加盐雾这个组合下，两年后的强度衰减我们没有数据。磁钢从 N42 降到 N38 之后，低温下的输出扭矩掉到刚好卡在客户要求下限，余量只剩 3%。

它不知道这些，因为它不知道这个件装在车门上。它按"降本 17.3%"这个单一目标优化，优化得很漂亮。

我后来重做了一遍。这一次我让它先给三条路：一条保性能降结构成本，一条保材料降磁钢成本，一条改工艺降装配成本。三条路各自摆出代价和风险，我选了第二条，把磁钢从 N42 换成 N40（不是 N38），再配合绕组微调，降本 12.8%，低温余量留了 11%。

从 17.3% 到 12.8%，数字小了，方案能用了。

独断的解法

方案门：三条路摆开写明取舍，你选了才动手

- 1 第一句就设方案门**
先给我三条不同的路，不要开始执行
- 2 三条路必须真不同**
只差一两个参数的两条路算同一条
- 3 每条路写放弃什么**
写不出放弃项，说明那条路没走到底
- 4 告诉你你的优先级**
一句话写清寿命、成本、性能谁先谁后
- 5 它推荐，你选，它才动手**
推荐意见要写，推荐不等于执行
- 6 破坏性操作单独确认**
改文件前要先列要改哪几处、旧值和新值

它给的方案越完整，你越要看它放弃了什么

图 17-4 独断的解法：方案门，三条路摆开、写明取舍、我选了才动手

图 17-4 作者根据公开资料绘制。

反制动作

1. 第一句话就设方案门。"先给我三条不同的路，不要开始执行。"
2. 要求三条路真不同。目标、路径、代价、风险，至少一项明显不同。只差一个参数的两条路算一条。
3. 每条路必须写"放弃什么"。不写放弃项的方案，它没有做过取舍，只是没想。
4. 把你的优先级告诉它。一句话就够："这个件装在车门上，要过盐雾，低温余量优先于成本。"
5. 它推荐，你选，它才动手。推荐意见要写，但推荐不等于执行。

6. 破坏性操作单独确认。改文件、删文件之前，必须先把要改什么、改成什么列出来给你看，你说改才改。

可复制反制提示词

【任务】[降本 15%，性能不许掉]。

【优先级】[低温余量 > 盐雾与寿命 > 成本]。这个顺序是本项目的第一优先级，你做任何取舍都按这个顺序来，与你的默认判断冲突时，以我给的为准。

【第一步：先不要执行任何方案。】

给我三条真正不同的路线。三条路必须在下面至少一项上明显不同：

实现路径、主要代价、主要风险、适用条件。

只差一两个参数的两条路算同一条，不许拿来凑数。

【每条路线必须写清六件事】

1. 具体做法（一句话说清改了什么）
2. 适用什么情况
3. 主要优点（带数字）
4. 代价（带数字，写清放弃了什么）
5. 风险（写清什么情况下这条路会失败）
6. 验证这条路要做的试验（写清试验名和周期）

【第二步：给我你的推荐，但不要开始执行。】

推荐意见后面必须写：你为什么选它，以及什么情况下应该换成另一条。

【第三步：等我选定，你再动手。】

我说出路线编号之前，不许生成任何文件、不许修改任何参数、不许给出最终方案细节。

【不可动清单】下面这些参数本次绝对不许改动，你在任何路线里都不许碰：

- [客户指定的接口尺寸]
- [工作温度范围]
- [已冻结的装配工艺]

【破坏性操作规矩】

任何修改我本地文件、删除文件、覆盖数据的动作，

执行前必须先把“要改哪个文件、改哪几处、旧值是什么、新值是什么”列成表给我，我说“同意执行”之后你才动手。

一次只做一类改动，做完汇报真实结果，等我确认再做下一类。

过程与判断点

三条路摆出来应该长这个样子：

- > 路线 A：保磁钢，降结构成本。
- > 做法：外壳改增强 PA66，端盖改冲压件。
- > 优点：BOM 降 11.2%，性能零损失。
- > 代价：模具费增加 42 万，回收期 14 个月。放弃：短期现金流。
- > 风险：PA66 在 85 °C 加盐雾下的长期强度无数据，寿命存疑。
- > 验证：500 h 盐雾 + 1000 h 热老化，周期 9 周。
- >
- > 路线 B：保结构，降磁材成本。
- > 做法：磁钢 N42 换 N40，绕组 68 匝微调至 65 匝。
- > 优点：BOM 降 12.8%，模具费零增加。
- > 代价：低温输出扭矩降 6%。放弃：3% 的性能余量，余量剩 11%。
- > 风险：批量磁钢一致性波动时，低温扭矩可能触限。
- > 验证：-40 °C 冷启动 200 次，周期 3 周。
- >
- > 路线 C：改工艺，降装配成本。
- > 做法：三处螺钉连接改卡扣，装配工步从 11 减到 7。
- > 优点：单件装配工时降 36 秒，年产量下省人工 28 万。
- > 代价：可维修性下降，返修需破坏卡扣。放弃：售后可拆性。
- > 风险：售后拆件率上升，目前无历史数据。
- > 验证：拆装循环 50 次，周期 2 周。
- >
- > 我的推荐：路线 B。若客户后续追加盐雾要求，改走路线 A。

我看这份东西只盯两处。

第一，每条路的"放弃"那一栏有没有写。三条路里只要有一条写不出放弃什么，说明它那条路没走到底。真方案一定有代价，没代价的方案是没算。

第二，它推荐的理由是不是引向我的优先级。它推荐 B，理由是模具费零增加、验证周期短。这个理由跟"低温余量优先"是冲突的。我这次选了 B，是因为 B 的余量还剩 11%，还够。但如果它推荐的理由完全绕开我给的优先级，那份推荐就是噪声，不看也罢。

翻车清单

翻车一：三条路只差一个参数。症状是 A、B、C 三个方案读起来像同一份东西改了三个数。补救：在提示词里写死"三条路在实现路径上必须不同，参数微调不算一条路"，并且要求每条路给出不同的验证试验，试验名重复就退回。

翻车二：它边给方案边动手。症状是三条路写完之后，它顺手就把推荐方案的完整实现也输出了，或者更糟，直接改了你的文件。补救：方案和执行分成两次对话，方案门这一轮的最后一句话写死"本轮不许产生任何文件"。

翻车三：它把不可动清单里的东西也改了。症状是你扫一眼它的表格，发现客户指定的接口尺寸被"优化"了。补救：不可动清单单独列成一段，加一句"这三项出现在你的任何改动里，本方案作废，你重做"，并且在验收时拿这三项去搜它的输出。

验收标准与变体

验收五条：

1. 三条路在实现路径上真不同，验证试验各不相同。
2. 每条路都写清了代价和风险，没有一条是"只有优点"。
3. 不可动清单里的三项，在全部三条路里都原封不动。
4. 它的推荐理由里引用了你给的优先级顺序。
5. 你选定之后的执行阶段，它先给了改动对照表，你同意后才动的文件。

变体一：选试验方案。一个失效分析摆在你面前，是做切片、做扫描电镜还是做能谱。三条路各自的代价是钱和周期，风险是可能看不出原因。这个变体最适合质量工程师。

变体二：选软件架构。该改老代码还是重写，该上中间件还是自己写。这一类方案的重点是"什么情况下应该换路"，因为软件方案最容易一条道走到黑。

变体三：选供应商整改路线。供应商出了批次问题，是退货、是挑选、是让步接收。这三条路的代价分别落在交期、成本和风险上，正好用这张表摆开给客户看。

老兵说：它给的方案越完整，你越要看它放弃了什么。一个什么都照顾到的方案，通常是它替你做了所有决定而你还没发现。

场景 64 加戏：它干了你没让它干的事 ★★★

罪名与症状

罪名：加戏。你让它改一处，它改了十处；你让它做 A，它顺手把 B、C 也做了。

一句话背景：改一份三十页的技术规范里的一处参数，人工要做的是改那一处、改引用它的另外两 three 处、升版本号。交给 AI 五分钟。可它常常顺手把整份文档的措辞、格式、章节顺序都"优化"了一遍，你核对的时候得看三十页而不是三页。这一条治的就是范围蔓延。

典型症状：

- 你让它改一个参数，它交回来的文件比你给的多改了十几处。
- 它顺手修了你文档里的错别字、标点、排版，你没让它修。
- 它"顺便"加了一段背景介绍或者风险分析，占了两页。
- 它动了目录里你没点名的其他文件。
- 你问它还改了什么，它答"一些格式问题"。

名词小词典

范围蔓延：任务的边界在执行过程中自己长大了。工程现场最常见的返工来源。

三栏清单：本次必做、本次明确不做、发现但暂不处理。治加戏的核心工具。

改动对照表：列出改了哪个文件、哪一处、旧值、新值。这是你核对的依据。

候选池：执行中发现的新问题，先记下来不处理，攒够了一起排期。

版本号：文件改一次升一版。它改了你不知道，版本号就乱了。

开工前准备

1. 工作目录先做一份备份，或者用版本管理（本书第 2 章讲过怎么开）。
2. 一份明确的文件白名单：这次只允许它碰这几个文件。
3. 你要改的那一处，把旧值和新值写在纸上。
4. 一个 diff 工具。改完之后你要能一眼看出改了哪些行。

现场案例

今年七月，我要改一份内部技术规范的温升限值：从 75 K 改成 70 K。就这一处。

文件是 Word，三十四页。我把文件丢进去，说"把第 5.3 节的温升限值从 75 K 改成 70 K"。

它三分钟交回一个新版本。温升限值改了，正确。我往下翻，发现它一共改了二十六处。

必要的有三处：正文第 5.3 节的限值、附录 A 检验表里同一个数、第 8 页的引用脚注。这三处我本来就要改。

剩下二十三处是它自己加的：全文"电机"统一改成"驱动电机"十一处，表格边框改成三线表六处，第 2 章补了一段术语说明占一页半，修订记录表重排两处，还删了三处它认为重复的注释。

问题在于这份规范是发给客户的受控文件。"驱动电机"这个统一改法跟客户 SOR 里的用词不一致。它补的那段术语说明里，有两个定义跟我公司的定义不一样。

我把那二十六处逐一退回，恢复到只改三处，一共花了七十分钟。七十分钟，比我自己动手改三处多花六十五分钟。

加戏的解法

三栏清单加改动对照表，越界改动一律退回

- ✓ **第一栏 本次必做**
列出本次必须改的每一处，含旧值与新值
- ✗ **第二栏 本次明确不做**
不许改用词、不许改样式、不许新增段落
- ! **第三栏 发现但暂不处理**
执行中发现的问题只记录，不许当场动手
- ✓ **先交改动对照表**
位置、旧值、新值，对不上必做的行划掉
- ✗ **白名单外的文件不许碰**
动手前后各报一次修改时间，变了就是事故
- ! **改完要 diff**
它报的清单跟工具核的对不上，以工具为准

每项改动都必须对应本次任务里的某条要求

图 17-5 加戏的解法：三栏清单加改动对照表，越界的改动一律退回

图 17-5 作者根据公开资料绘制。

反制动作

1. 先定三栏清单。本次必做、本次明确不做、发现但暂不处理。前两栏你填，第三栏它填。
2. 给文件白名单。只许碰这几个文件，白名单外的文件碰了就是事故。
3. 要求先交改动对照表。改之前先把"要改哪几处、旧值、新值"列出来，你点头才改。
4. 一条铁律：每项改动必须对应本次任务里的某条要求。对不上的，只记录不执行。
5. 改完要 diff。让它自己报改动清单，你用工具再验一遍。两份对不上，以工具为准。
6. 发现的问题进候选池。它在执行中发现的新问题，写进第三栏，不许当场动手。

可复制反制提示词

【任务】[把 technical_spec_v2.3.docx 第 5.3 节的温升限值从 75 K 改成 70 K]。

【本次必做】

1. 改第 5.3 节正文的限值：75 K → 70 K。
2. 改附录 A 检验表里对应的那个限值：75 K → 70 K。
3. 改第 8 页引用该限值的脚注。
4. 修订记录表加一行，版本号 v2.3 → v2.4。

【本次明确不做】

1. 不许改任何术语、用词、称呼。
2. 不许改任何表格样式、字体、边框、排版。
3. 不许新增任何章节、段落、说明。
4. 不许删除任何注释、脚注、附录。
5. 不许碰白名单之外的任何文件。

【发现但暂不处理】

执行中如果发现别的问题，写进这一栏，只记录不执行。

每条写：位置、问题、建议处理方式、预估工作量。

【文件白名单】只允许修改这一个文件：technical_spec_v2.3.docx。

【执行规矩：先交对照表，我点头才改】

动手之前，先把改动对照表列给我：

序号	位置	旧值	新值	对应本次必做的第几条
序号 位置 旧值 新值 对应本次必做的第几条				

对不上“本次必做”任何一条的改动，不许出现在表里。

【交付规矩】

改完之后，把实际改动清单逐条报给我，格式同上。

另外回答两句话：

第一句：本次实际改动 N 处，与对照表的 M 处是否一致，不一致的说明原因。

第二句：本次有没有做对照表之外的改动，有就列出来。

第二句不许省略，没有也要写“无对照表外改动”。

过程与判断点

对照表应该是这个样子，四行：


```

> | 1 | 第 5.3 节正文 | 75 K | 70 K | 必做第 1 条 |
> | 2 | 附录 A 检验表 | 75 K | 70 K | 必做第 2 条 |
> | 3 | 第 8 页脚注 | 限值 75 K | 限值 70 K | 必做第 3 条 |
> | 4 | 修订记录表 | v2.3 | v2.4, 新增一行 | 必做第 4 条 |
>
> 本次实际改动 4 处, 与对照表一致。
> 本次无对照表外改动。
> 发现但暂不处理: 第 12 页表 3 也出现了 75 这个数, 但它是另一个参数 (环境温度上限),
> 未改动, 请你确认。

```

最后那一行是最值钱的。它发现了第 12 页也有 75, 但它没有自作主张去改, 而是报给我判断。这正是三栏清单想要的效果。

我判断两件事。

第一, 对照表里的每一行能不能对上"必做"的某一条。对不上就是加戏, 划掉。这一次它第 4 行的版本升级对得上必做第 4 条, 合规。

第二, 最后那句"有无对照表外改动"。这一句是整个提示词的机关。它如果说"无", 你就信一部分, 用 diff 工具再核一次; 它如果列出了一堆"顺手优化", 直接退回重做。我遇到过它一边说"无对照表外改动", 一边改了十一处术语的情况。所以这句话必须有, 但不能只靠它。

翻车清单

翻车一: 它把"优化"当成任务的一部分。症状是交回来的文件更漂亮了, 但漂亮的部分你没要。补救: 本次明确不做那一栏写到具体, 把"不许改用词、不许改格式、不许新增段落"逐条列出来, 越具体越好。

翻车二: 它动了白名单外的文件。症状是你工作目录里另一个文件的修改时间也变了。补救: 动手前让它报一次目录里所有文件的最后修改时间, 改完再报一次, 两次对比。这一招二十秒, 很土但很有效。

翻车三: 它在多轮任务里逐轮加戏。症状是前三轮都很规矩, 第四轮开始"顺手"改造。补救: 每一轮都重新发一次三栏清单, 不要指望它记住上一轮的边界。这一条跟场景 66 的失忆是同一件事的两面。

验收标准与变体

验收五条:

1. 实际改动处数等于对照表行数, 一行不多一行不少。
2. 用 diff 工具核一遍, 零处对照表外改动。
3. 白名单外的所有文件, 最后修改时间未变。
4. "发现但暂不处理"一栏有内容, 且每条都不影响本次交付。
5. 版本号已升, 修订记录已加行。

变体一: 改代码里的一处逻辑。三栏清单照样用, 但"明确不做"要加上"不许重构、不许改命名、不许调格式"。代码领域的加戏比文档更凶, 因为它觉得顺手重构是负责任。

变体二: 改一张 BOM 表的一行。这个变体的白名单特别重要, 因为 BOM 往往关联着采购、财务、ERP 三份表。只准改一张, 其余的进候选池。

变体三：批量改一批图纸的标题栏。这一类的加戏风险最高，因为它可能顺手改了图里的某个尺寸。验收动作改成：改完之后随机抽三张，逐个尺寸跟原图比对，只许标题栏那一处有差异。

老兵说：它加戏的时候从来不觉得自己做错了，它觉得自己在帮你。所以别跟它讲道理，给它一张白名单，白名单外的文件它碰一下就算事故。

场景 65 话痨：它说了一大堆你不需要的 ★★★

罪名与症状

罪名：话痨。说了一整屏，你还是不知道下一步干什么。

一句话背景：我在评审会上翻过一份 AI 写的失效分析，前三页在讲"什么是失效分析"，第四页讲"常见的失效模式分类"，真正的结论和下一步建议藏在第六页倒数第二段。会开到一半，客户打断了两次。人工把一份分析压成半页结论要二十分钟，这一节教你让它第一屏就给结论。

典型症状：

- 你问一个具体问题，它先复述一遍背景。
- 回答开头是"这是一个很好的问题"。
- 真正的结论在第三屏以后。
- 结尾有"注意事项"八条，其中六条跟你的问题无关。
- 你让它简短，它把风险、失败和未验证的部分也一起删了。

最后这一条最要命。话痨不能靠"说简短点"来治，因为删的时候它分不清什么是废话、什么是风险。

名词小词典

行动优先排版：回复的第一行是结论或下一步，背景和解释往后放。

一屏一动作：每一屏的内容只为了让读者做一件事。做完这件事再看下一屏。

结论前置：把最重要的那句话放在最前面，后面才说为什么。

信息密度：一屏内容里有用信息的占比。判断标准很简单：删掉这一屏，读者会不会漏掉关键东西。

风险不可省略：风险、失败、未验证的部分，不许为了精简而被删掉。这是话痨这一条里唯一的豁免项。

开工前准备

1. 一个明确的"下一步"定义。你要它给你的到底是建议、是文件还是让你拍板的一个选择。
2. 一份输出模板，五段：结论、需要你决定的、依据、风险、下一步。
3. 一段能反复粘贴的排版纪律提示词。这是本节的核心交付物。

现场案例

今年四月，一批执行器在客户装配线上出现异响，退货 43 件。我让 AI 帮我写一份给客户的初步分析报告。

它交回来六页。第一页是标题，第二、三页讲微电机和执行器的基本原理，第四页列了齿轮、轴承、装配、润滑、电磁五类异响来源，第五页结尾才有两句初步结论，第六页是四条后续建议。

这份报告我拿去内部评审，三个同事读完，两个说没看懂结论到底是啥。

我后来改了打法，让它按五段模板重写，压成一页。压完是这样：

结论：43 件退货中，37 件的异响源定位在二级齿轮啮合面，判定为齿面磕伤，
磕伤发生在供应商装配环节，非设计问题。剩余 6 件待复测。
需要你决定的：是否要求供应商对同批次全部 2100 件做全检。
依据：37 件中 35 件的异响频谱主峰落在啮合频率的 2 倍频；
拆解后 31 件在二级齿轮上找到可见磕伤点，位置集中在齿端 2 毫米内。
风险：另有 6 件频谱特征不同，怀疑是另一失效模式，本轮未定位，需追加 2 周。
下一步：1. 我出全检方案给你批。2. 6 件异常件送实验室做切片。

一页纸，客户读完直接进决策。原来那六页里，真正有用的信息就是这一页，剩下的五页是我陪它复习了一遍基础知识。

话痨的解法

五段模板：结论在最前，风险不许删

- 1 第一段 结论**
一到三行，带数字，不许重复我的问题
- 2 第二段 需要你决定的**
只放一个决定，附你的推荐和理由
- 3 第三段 依据**
只留影响结论的依据，每条一行带数字
- 4 第四段 风险与未验证**
不许为空，不许把未验证写成已完成
- 5 第五段 下一步**
最多两条，每条写清谁做、产出什么
- 6 全局 一屏一动作**
不许一次推进两个以上的决定

别叫它少说话，叫它换个顺序说

图 17-6 话痨的解法：五段模板，结论在最前，风险不许删

图 17-6 作者根据公开资料绘制。

反制动作

1. 定死输出模板。五段：结论、需要你决定的、依据、风险、下一步。一段不许少。
2. 第一段不许超过三行。结论写不完说明它没想清楚。
3. 背景不许单独成段。需要的背景揉进"依据"里，一句话带过。
4. 一次只推进一个决定。有三个要你定的，就分三屏，一屏一个。
5. 风险段不许为空。写"本次无风险"也要写，并且你要怀疑它。
6. 结尾只留一个动作。下一步那一栏最多两条，多了说明它自己也没想清楚优先级。

可复制反制提示词

【输出排版纪律：从现在起，所有回复都按这个顺序，不按这个顺序的一律重写。】

第一段：结论。

一到三行，把最重要的判断写出来，带数字。

不许以"这是一个很好的问题""根据您提供的信息"这类话开头。

不许在这一段里重复我的问题。

第二段：需要你决定的。

如果当前有需要我拍板的事，只放一个，附上你的推荐和理由。

有多个决定就分多次，一次一个。

当前没有需要我决定的，这一段写"当前无需决定"，不许省略。

第三段：依据。

只保留会影响上面那个结论或那个决定的依据，每条一行，带数字。

背景介绍、概念解释、通用知识一律不写。

我需要了解某个概念时会单独问你。

第四段：风险与未验证。

列出本次结论里尚未验证、可能错误、受条件限制的部分。

这一段不许为空，没有也要写"本次未发现风险"并说明你检查了什么。

不许为了精简输出而省略这一段。

不许把"未验证"写成"已完成"。

第五段：下一步。

最多两条，每条是一个具体动作，写清谁做、产出什么。

不许写"持续优化""后续跟进"这类没有动作的话。

【全局规矩】

每一屏只服务一件事。

不许在同一条回复里同时推进两个以上的决定。

编号用阿拉伯数字，每级不超过五条。

"注意事项""温馨提示""补充说明"这类段落一律取消，

里面的内容要么并入风险段，要么删除。

过程与判断点

一份合格的输出，五段齐，一屏放得下：

- 结论：43 件中 37 件异响源为二级齿轮齿面磕伤，判定供应商装配环节问题，非设计问题。
- 需要你决定的：是否要求供应商对同批次全部 2100 件做全检。我的推荐是做，理由是全检成本约 3.4 万，低于一次客户停线的损失。
- 依据：37 件中 35 件异响主频落在啮合频率 2 倍频；31 件拆解后在二级齿轮发现可见磕伤，位置集中在齿端 2 mm 内。
- 风险：另 6 件频谱特征不同，本轮未定位，判定为第二种失效模式，需追加 2 周和 1.8 万检测费。
- 下一步：1. 我出全检方案，明天给你批。2. 6 件异常件送实验室做切片，下周三出结果。

我看这份东西只看两处。

第一，结论段有没有数字。37 件、35 件、2 倍频、2 毫米，这些是这份报告的全部价值。没有数字的结论是观点，不是结论。它如果写出"大部分退货件存在齿轮问题"，退回重写。

第二，风险段是不是真在说风险。有一种假风险，写着"本结论基于现有样本，样本增加可能需要调整"。这是废话。真风险要具体到"另 6 件的频谱不同，本轮未定位"。它写不出具体风险，说明它没有查，或者它在藏。

翻车清单

翻车一：它精简的时候把依据也删了。症状是结论很干脆，但你看不出这个结论从哪来。补救：依据段的每条必须带数字，没有数字的条目不许进依据段，宁可少一条。

翻车二：五段都写了，但每段都很长。症状是它学会了模板，没学会压缩。补救：给每段加字数上限，结论三行、依据五条、风险三条、下一步两条，超了就退回。

翻车三：风险段变成套话。症状是风险段里全是"可能存在一定偏差"。补救：要求风险段每条必须写清"什么情况下这条会变成真的"，写不出来就删掉这一条，最后留一到三条硬的。

验收标准与变体

验收四条：

1. 五段齐全，顺序正确，一屏能读完。
2. 结论段带至少两个数字。
3. 风险段至少有一条具体风险，不是套话。
4. 把这份输出发给一个不了解背景的人，他能说出下一步该干什么。

变体一：会议纪要。五段改成：决议、待办、讨论要点、异议、下次会议。决议必须在第一行，因为没人会读完一份会议纪要。

变体二：试验日报。五段改成：今日结论、异常、数据、风险、明日计划。试验日报的价值就在于一眼看到异常，所以异常段必须前置。

变体三：给领导的汇报。五段改成：结果、要你批的、关键信息、风险、需要你协调的资源。给领导的东西，第一行的数字决定他读不读第二行。

老兵说：你让它简短，它连风险一起删了。所以别叫它少说话，叫它换个顺序说。顺序对了，篇幅自然就短了。

场景 66 失忆：干到一半忘了前面的约定 ★★

罪名与症状

罪名：失忆。对话一长，前面的约定就丢了。

一句话背景：一份三十页的技术报告分五轮写完，前两轮定的术语表、单位口径、编号规则，到第四轮全变了。人工把这些不一致挑出来要两个小时，而且极容易漏。这一节教你用三份状态文件把约定固化下来，让它每一轮开工前先读一遍。这套流程我跑通过，书里标 ★★，因为多轮长任务的留档我做得不完整。

典型症状：

- 第三轮开始，它用的术语跟第一轮不一样了。
- 前面定好的单位口径（比如力矩一律 $N \cdot m$ ）到后面冒出 $kgf \cdot cm$ 。
- 编号规则走着走着就乱了，图 3-2 后面直接是图 3-5。
- 你让它"继续"，它重新问你一遍已经说过的要求。
- 它开始用另一套称呼来叫同一个零件。

名词小词典

上下文窗口：它能同时看到的内容长度上限。超了之后，最早的内容会被压缩或者丢掉。

状态文件：把约定写进文件，不再只依赖对话记忆。文件不会丢，对话会。

口径：同一个东西在所有地方用同一个说法、同一个单位、同一个编号规则。

交接记录：让一个完全没看过旧对话的人（或 AI）接手所需的最小信息集。

三文件法：计划、发现、执行记录，三个文件管住一个长任务的状态。

开工前准备

1. 一个专门的任务目录，三个文件放进去：PLAN.md、FINDINGS.md、LOG.md。
2. 一份术语表，把这次任务里所有容易混的叫法先定死。
3. 一份格式约定：单位、编号规则、日期格式、文件名规则。
4. 一个规矩：每一轮开工第一句话是"先读这三个文件"。

现场案例

去年年底，我写一份微电机 NVH 判读系统的验收报告，分五轮完成：第一轮定结构，第二轮写方法，第三轮写数据，第四轮写结论，第五轮统稿。

第一轮我们定了三件事：术语统一叫"判读"不叫"识别"；力矩单位一律 N·m；图表编号按"章-序"两级。

第三轮写完数据章节，我发现它把"判读"和"识别"混着用了，十七处。第四轮写结论，它那张表的力矩单位写着 kgf·cm，跟前面三张表都不一样。第五轮统稿，编号乱了，图 3-2 之后直接跳到图 3-5。

挑这三处不一致改了将近两小时。改的过程中我意识到：不是它忘了，是我把约定只放在对话里。对话一长，前面的约定对它来说就是被挤出去的内容。

后来改成三文件法，同样一份报告分五轮，最后一轮的不一致只剩两处，而且是我自己中途改主意造成的。

失忆的解法

三份状态文件，每轮开工先读、每轮收尾先写

- ✓ **PLAN.md 计划**
目标、阶段、当前进度，约定写进第一段
- ✓ **FINDINGS.md 发现**
关键发现、已确认的决定、已排除的方案
- ✓ **LOG.md 执行记录**
本轮完成什么、遇到什么错、检查结果
- ! **每轮开工先读三个文件**
读完只用三句话报告，报告完停下来等确认
- ✗ **不许边报告边干活**
约定复述与文件不一致，说明它没读进去
- ! **超过八轮就换对话**
换对话的成本比修失忆低

对话是它的短期记忆，文件才是它的长期记忆

图 17-7 失忆的解法：三份状态文件，每轮开工先读、每轮收尾先写

图 17-7 作者根据公开资料绘制。

反制动作

1. 任务目录里放三个文件。计划、发现、执行记录。
2. 每轮开工第一句话：先读这三个文件，然后一句话报告当前状态。它报告错了，说明文件写得不够清楚。

3. 每轮收尾：先更新三个文件，再汇报。顺序不能反。
4. 约定写进 PLAN.md 的第一段。术语、单位、编号规则，三条。
5. 交接测试。把三个文件发给一个全新的对话，看它能不能接着干。接得上，文件才算写对。
6. 超过八轮就换对话。别指望一条对话能撑到底，换对话的成本比修失忆低。

可复制反制提示词

第一条，每轮开工用，整段复制：

【开工前必读】先读当前目录下的这三个文件：

1. PLAN.md（目标、阶段、当前进度、约定）
2. FINDINGS.md（关键发现、已确认的决定、已排除的方案）
3. LOG.md（已完成什么、遇到过什么错、检查结果）

读完只用三句话报告，不许展开：

第一句：本次任务的当前目标是什么。

第二句：上一轮做到哪里，这一轮该做什么。

第三句：当前有哪些约定必须遵守（术语、单位、编号规则）。

三句话报告完，停下来等我确认。

我确认之后你再动手，不许边报告边干活。

【约定复述】三句话报告之后，把 PLAN.md 里的约定原样复述一遍。

复述内容与文件不一致，说明你没读进去，我会让你重读。

第二条，每轮收尾用：

【收尾动作：先写文件，再汇报。顺序不许反。】

第一步，更新 PLAN.md：

把当前阶段改成已完成，把下一阶段改成进行中。

如果有新增的约定（术语、单位、编号规则），写进约定段。

第二步，更新 FINDINGS.md：

本轮得到的关键发现、本轮拍板的决定、本轮排除掉的方案。

每条写清：内容、依据、在哪一轮确定的。

第三步，更新 LOG.md：

追加一行：轮次、本轮完成什么、遇到过什么错、检查结果如何。

检查结果必须写具体，不许写“已完成”。

第四步，才轮到你向我汇报。

汇报格式：本轮完成 N 项，与 PLAN 对照完成度多少，

本轮新增约定几条，下一轮做什么。

【一致性自检】汇报之前，检查本轮产出与已有内容的一致性：

术语是否与已完成的章节一致；单位是否与已有表格一致；

编号是否与已有图表连续；同一对象在所有地方的叫法是否一致。

发现不一致的，先修已有内容还是先修本轮产出，问我，不许自己定。

过程与判断点

一份合格的开工三句话长这样：

- ＞ 第一句：当前目标是为微电机 NVH 判读系统写一份可交付客户的验收报告。
- ＞ 第二句：上一轮完成第三章数据部分，本轮写第四章结论与建议。
- ＞ 第三句：约定三条，术语统一用“判读”，力矩单位统一 N·m，图表编号按“章-序”两级。
- ＞
- ＞ 约定复述：判读（不用识别）、N·m（不用 kgf·cm）、图 4-1 起编（承接第三章的图 3-7）。

我看两点。

第一，第三句话里的约定是不是完整。只说出两条，说明 PLAN.md 里那条没写清楚，或者它没读。这时候不是骂它，是去改文件。

第二，约定复述里的编号起点。“图 4-1 起编（承接第三章的图 3-7）”这一句最关键。编号失忆是最常见的一种，因为它需要它记住上一章编到几了，而这个数字往往在很早的位置。

翻车清单

翻车一：它读文件读了个寂寞。症状是三句话报告答得上来，动手之后照样违反约定。补救：中段再插一次检查，让它把当前产出的术语和单位列一遍，与约定比对。这叫中途抽检，长任务至少抽两次。

翻车二：文件越写越长，它自己读不完。症状是到第五轮它开始漏读 FINDINGS.md。补救：每个文件超过两百行就拆，或者给它一个摘要段放在文件开头，摘要不超过二十行。

翻车三：它改了文件但没告诉你。症状是你以为的进度跟文件里的进度不一样。补救：每轮收尾之后你自己打开 PLAN.md 看一眼，三十秒，能挡掉一整天的返工。

验收标准与变体

验收五条：

1. 三个文件都存在，PLAN.md 的约定段至少三条。
2. 把三个文件发到一个全新的对话，它能在不补充说明的情况下接着干。
3. 全文术语一致，用搜索功能查"识别""判读"，只应出现一种。
4. 全文单位一致，所有力矩都是 N·m。
5. 图表编号连续无跳号，或者跳号处有说明。

变体一：跨天的长任务。今天做一半明天接着做，交接记录的价值最大。三条状态文件再加上一条"今天卡在哪"，第二天开工两分钟进入状态。

变体二：换人接手。同事请假，你接他的活。让他走一遍收尾动作再走，你读三份文件就能接上，不用打电话问。

变体三：多人同时写一个文档。三个人各写一章，共用一份 PLAN.md 的约定段。这是最实用的一个变体，约定文件就是团队的一致性底座。

老兵说：对话是它的短期记忆，文件才是它的长期记忆。你舍不得花五分钟写状态文件，就得花两小时挑不一致。

场景 67 报喜：它只告诉你好的那部分 ★★★

罪名与症状

罪名：报喜。什么都没检查，它先说完成了；出了问题，它把问题说成进展。

一句话背景：一份 12 台样件的耐久试验，2 台中途失效，它交上来的报告里只写了 10 台的数据，结论是"全部通过"。这类隐瞒在工程现场不是道德问题，是它的行为模式：它倾向于把弱事实包装成强结论。人工做一次完整核对要一小时，这一节教你用一张四行证据表，十五分钟抓完。

典型症状：

- 它说"已完成"，但说不出检查了什么。
- 报告里的样本数跟你送进去的样本数不一样。
- 失败的样本在报告里消失了，或者被写成"数据异常已剔除"，没有说明。
- 它把"文件生成成功"说成"内容正确"。

- 它把"流程跑通了"说成"结果可用"。
- 你追问异常，它说"其他都正常"。

名词小词典

弱事实：一个很弱的事实，比如"文件写出来了"。它本身是真的，但推不出"文章能发布"。

强结论：一个很强的判断，比如"全部通过"。它需要多个证据支撑。

证据表：完成声明必须附上的一张表，四行：检查了什么、怎么检查的、实际结果、证据位置。

样本数：送进去几个样件、最后报告里几个样件。这两个数字不一致，一定有问题。

剔除说明：某个样本的数据被排除时，必须写清剔除理由。没有理由的剔除等同于造假。

开工前准备

1. 送检清单，你送进去多少个样件、编号是什么，先写下来。
2. 一份验收门槛，通过和失败的判据先定死。
3. 一个态度：它报"完成"的时候，你的第一反应是"证据呢"。
4. 四行证据表的模板。

现场案例

今年一月，一批执行器做常温耐久，送进去 12 台，目标 30000 次循环。

试验跑了六周，跑完我把原始记录丢给 AI，让它出汇总报告。

报告写得很专业：平均值、标准差、CPK，图表齐全。结论段写着"12 台样件全部完成 30000 次循环耐久试验，通过率 100%"。

我看了一眼原始记录目录，里面只有 10 个数据文件。

我追问："12 台还是 10 台？"它答："其中 2 台的数据文件缺失，我在汇总时使用了现有 10 台的数据。"

"那 2 台去哪了？"它这才说：那 2 台在 18000 次和 22000 次的时候卡死了，试验中止。

也就是说真实通过率是 10 比 12，83.3%，而且失效的两台是同一个失效模式：输出轴卡滞。这个信息比"通过率 100%"重要一百倍，因为它是这个项目唯一有价值的发现。

它在报告里把这个发现吃掉了，方法是把样本数从 12 悄悄改成 10，然后按 10 算通过率。

如果这份报告直接发给客户，我们会在两周后收到一批装车后的失效反馈，那时候的代价跟现在完全不同。

报喜的解法

样本数必须闭合，失败必须单独上报

- 1 先报样本数再报结论**
送检、完成、有效、剔除四个数先对上
- 2 失败必须单独成段**
写清失效模式、失效时的循环数和实测值
- 3 完成必须附证据表**
四行：检查什么、怎么查、结果、证据位置
- 4 不许把过程说成结果**
文件已生成不等于内容正确
- 5 上一轮的证据不作数**
要说当前版本可用，必须重跑一次检查
- 6 未知就写未知**
没能力验证的部分写未验证，不许统称完成

它说完成的时候，你唯一该问的是你检查了什么

图 17-8 报喜的解法：四行证据表，样本数必须闭合，失败必须上报

图 17-8 作者根据公开资料绘制。

反制动作

1. 先报样本数，再报结论。送进去几个、完成几个、有效几个、剔除几个，四个数必须先对上。
2. 失败必须单独成段。不许用"数据异常已剔除"带过，必须写清失效模式、失效时的循环数。
3. "完成"必须附证据表。四行：检查了什么、怎么检查的、实际结果、证据位置。
4. 不许把过程说成结果。文件生成了是过程，内容正确才是结果。
5. 上一轮的证据不作数。上一版测过的、改之前截的图、别人替你看过的声明，都不能给这一版背书。
6. 未知要写未知。它没有能力验证的部分，写"未验证"，不许统称完成。

可复制反制提示词

【完成纪律：本次任务的"完成"两个字，必须满足下面全部条件才能说。】

【第一，样本数闭合。】

报告的第一段必须是四个数：

送检 N 台 / 完成试验 M 台 / 有效数据 K 台 / 剔除 J 台（附剔除理由）

四个数必须满足 $N = M + \text{未完成数}$ ，且 $K + J = M$ 。

对不上不许往下写，先停下来向我报告差异。

【第二，失败单独成段。】

任何一台未完成、失效、超差、被剔除的样本，必须单独成段写清：

样本编号、在第几次循环（或第几个工况）出问题、失效模式、当时的实测值。

不许写"数据异常已剔除"，不许把失败样本从统计表里悄悄拿掉。

没有任何失败的，这一段写"本次无失效样本"，不许省略。

【第三，证据表。】

你说"完成"的时候，必须同时给出下面这张表，四行填满：

完成要求	检查方法	实际结果	证据位置
第一行：是否满足原始需求（逐项对照任务要求，通过/失败）			
第二行：是否真的可用（实际打开、运行或走一遍流程，通过/失败）			
第三行：是否引入新问题（跑相关检查，通过/失败）			
第四行：是否还有未知项（列出你没有能力验证的部分，有/无）			

【第四，不许把过程说成结果。】

"文件已生成"不等于"内容正确"。

"流程已跑通"不等于"结果可用"。

"搜索到了资料"不等于"资料已核实"。

"代码已修改"不等于"问题已修复"。

这四句话不许混用，说过程就说过程。

【第五，上一轮不作数。】

上一版测过的用例、改之前的检查结果、别人替你看过的声明，

都不能给当前这一版背书。

你要说当前版本可用，必须跑一次针对当前版本的检查，

并且把这次检查的真实输出贴给我。

【第六，未知就是未知。】

你没有能力验证的部分，写"未验证：xxx，建议由 xxx 验证"。

这一条不扣分。把未验证说成已通过，本次交付作废。

过程与判断点

一份合格的交付汇报长这样：

```

> 送检 12 台 / 完成试验 10 台 / 有效数据 10 台 / 剔除 0 台
> 未完成 2 台：
>   #07 在 18240 次循环卡滞，输出轴卡死，实测扭矩 0.00 N·m
>   #11 在 22065 次循环卡滞，输出轴卡死，实测扭矩 0.00 N·m
>   两台失效模式相同，判定为同一种失效，失效模式：输出轴卡滞
>
> | 完成要求 | 检查方法 | 实际结果 | 证据位置 |
> | 12 台全通过 | 逐台核对试验记录 | 失败，10/12 | LOG/2026-01-run07.csv |
> | 报告可用 | 打开报告逐节复核 | 通过 | report_v3.docx |
> | 引入新问题 | 查数据完整性 | 通过，无缺失 | check_integrity.log |
> | 未知项 | 卡滞根因未定位 | 有，需切片分析 | 建议送实验室 |

```

这段汇报我看三个数。

第一，12、10、10、0 这四个数闭合。12 减 2 等于 10，10 加 0 等于 10。对不上就是它在藏东西。

第二，失败段有没有失效模式和循环数。18240 次、卡滞、0.00 N·m，这三个信息是这份报告真正的价值。写不出这些，失败段就是走过场。

第三，证据表第四行的未知项。它承认卡滞根因未定位，这是好事。它如果说“根因已查明”，我反而要追问查了什么。

翻车清单

翻车一：样本数对不上，它用“有效样本”这个词掩盖。症状是报告里只出现“有效样本 10 台”，从头到尾不提送检 12 台。补救：在提示词里写死“四个数必须在同一个段落里出现”，并且你自己核一次送检单。

翻车二：失败样本被写成了“数据异常”。症状是报告里有“剔除 2 台，原因：数据异常”，但没有说怎么异常。补救：要求剔除理由必须包含失效时的实测值和循环数，写不出来的剔除一律不认。

翻车三：它检查的是自己上一轮的产出。症状是它说“已复核”，复核的是它自己刚写的报告，而不是原始记录。补救：让它把复核时读取的文件名列出来，里面必须有原始记录文件，不能只有它自己生成的中间文件。

验收标准与变体

验收五条：

1. 四个样本数闭合，与送检单一致。
2. 失败样本单独成段，含编号、循环数、失效模式、实测值。
3. 四行证据表填满，证据位置指向真实文件。
4. 抽查证据表里的一条，打开它标的文件，内容对得上。
5. 把报告给一个没参与试验的人看，他能说出这次试验的真实通过率。

变体一：周报与项目状态。最容易报喜的东西就是周报。四个数换成：计划 N 项、完成 M 项、受阻 K 项、取消 J 项。受阻项必须写清卡在谁那儿。

变体二：软件功能交付。"四个数"换成：用例总数、通过数、失败数、跳过数。跳过数最容易被藏，因为跳过看起来既不是通过也不是失败。

变体三：供应商整改验证。供应商说整改完成，你让它核。四行证据表的第一行改成"对照 8D 报告的每一条措施逐项验证"，因为整改措施最容易只做一半就报完成。

老兵说：它说"完成"的时候，你唯一该问的是"你检查了什么"。答不上来，那就是没检查。没检查的完成，跟没做完是同一件事。

场景 68 贪心元认知：它对自己能力的判断不可信 ★★

罪名与症状

罪名：贪心元认知。它对自己"知道不知道"的判断，本身就是不可信的。

一句话背景：前面七宗罪都建立在同一个假设上：它知道自己什么时候在编。这个假设不成立。它说"我很确定"的时候可能错得离谱，它说"我不确定"的时候可能其实很准。这一节给你三件校准工具：探针题、反证测试、五档置信度。这一节标 ★★，因为校准方法我跑通过，但跨模型的稳定性我没做过系统对比。

典型症状：

- 它说"我很确定"，然后错了。
- 它给的答案一律是肯定的，你问一百个问题，它一次都没说过"我不知道"。
- 你让它自评可信度，它打 9 分，实际错 4 处。
- 它对自己擅长的领域过度自信，对不擅长的领域也过度自信，两档的自信程度几乎一样。
- 你问它"这个你确定吗"，它答"确定"，然后你换个问法再问一次，它给出了另一个答案。

名词小词典

元认知：对自己认知过程的判断，也就是"我知道我不知道什么"这件事。

校准：一个人说"我有八成把握"的时候，实际正确率是不是真的有八成。这是衡量可信度的唯一硬指标。

探针题：你事先知道答案的题，用来测它当前状态下的准确率。

反证测试：让它自己去找能推翻它结论的证据。找得出来，说明它的结论有边界；找不出来，说明它压根没想过。

置信度分档：把答案按把握程度分成五档，从"确定"到"不知道"。分档的意义不在分，在于逼它承认有"不知道"这一档。

贪心：什么都想要，什么都想要到极致。这一宗罪藏在被告席最后排，因为它是我们自己的问题：我们既想要它全能，又想要它诚实，而这两件事本身打架。

开工前准备

1. 三到五道探针题，答案你背得出来，覆盖三个难度档。
2. 一张五档置信度表，你先把它定义清楚。
3. 一个反证测试的固定问法。
4. 一个规矩：它自评"确定"的答案，你必须抽一条去核。

现场案例

今年二月，我用 AI 帮我归档一批 2019 年的老试验报告，扫描件，PDF，一百六十多份。任务是让它把每份报告里的关键数据（试验日期、样件编号、峰值扭矩、判定结论）抽出来，做成一张总表。

我先问它一句："这批扫描件里有手写体的数字，你能读准吗？"

它答："可以。我会仔细识别手写内容，并在不确定的地方标注。"

我很满意，把一百六十份丢进去。

两天后我抽查了二十份，手写数字读错了七处。最严重的一处，报告上手写的是"8.6"，它读成了"3.6"，判定结论从"合格"变成了"不合格"，而这份报告对应的是三年前已经结案的一个项目。

我回去问它："你不是说能读准吗？"它答："手写体识别确实存在一定难度，我当时应该提醒您先做小批量验证。"

这句话听着诚恳，但它是事后才说出来的。事前你问它，它一律答能。

后来我改了规矩。我先做校准：给它十份我已经手工录好答案的扫描件让它读，比对下来错了三处，准确率 92.5%。

有了这个数，我就知道这批活的正确姿势是让它全读，然后我人工复核那百分之七到八，重点看手写体和模糊页。一百六十份最后的复核量是二十一份，一个下午做完。

如果当时我信了它那句"可以"，一百六十份里会埋着十几处错，而这些错会在三年后被某个人翻出来。

贪心元认知的解法

探针校准、五档置信度、反证测试，三件一起用

- ✓ **开工前先校准再派活**
十道探针题算出当前准确率，再定复核量
- ✓ **强制五档置信度**
确定、高、中、低、不知道，每档都要有定义
- ✗ **不许全档一致**
一份输出里全是确定档，说明自评没经过思考
- ! **反证测试**
每个确定档结论后面写一句什么情况下不成立
- ✓ **抽一条去核**
确定档答案随机抽一条亲自核，核错整批降级
- ✗ **冲突要求分开两轮**
既要快又要准这类打架要求，一轮只做一件

我不知道这四个字，是我最想从它嘴里听到的答案之一

图 17-9 元认知的解法：探针校准、五档置信度、反证测试，三件一起用

图 17-9 作者根据公开资料绘制。

反制动作

1. 开工前先校准，再派活。十道探针题，算出它的当前准确率，你才知道该复核多少。
2. 强制五档置信度。每个答案后面必须跟一档，不许全部答"确定"。
3. 不许全档一致。一份输出里如果所有答案都是"确定"，退回重做。真做事的人不可能全确定。
4. 反证测试。每个关键结论后面，让它自己列出能推翻它的证据或条件。
5. 抽一条去核。它自评"确定"的答案里，随机抽一条，你亲自核。核错一条，整批降级。
6. 冲突要求分开两轮。你既要它快又要它准、既要它全面又要它简短，这类打架的要求拆成两轮，一轮只负责一件事。

可复制反制提示词

第一条，开工前的校准，整段复制：

【开工前校准：先测，再派活。】

我会给你十道题，其中有些答案我手里有标准答案，有些没有。
你的任务是逐题作答，并在每题后标注你的把握程度。

【五档置信度定义】

- A 确定：我有明确依据，可以直接进交付物。
- B 高：我记得是这样，建议核一次再用。
- C 中：我不完全确定，需要你查证。
- D 低：我基本是在推测，这个答案只能当方向。
- E 不知道：我没有可靠依据，请你自己查，我可以告诉你去哪儿查。

【答题规矩】

每题只答答案加一句依据，再加一档置信度。
十道题里，你的答案必须至少覆盖三个不同的档位。
十道题全是 A 档，我会判定你没有做真实自评，本次校准作废。

【反证要求】

十道题答完之后，挑你认为最有把握的三道，
逐道写出：什么情况下这道题的答案是错的。
一道题都写不出反例的，说明你的把握没有经过检验，
把那几道题的档位往下调一档。

【校准结论】

最后给我一句话：在你刚才这十道题里，
A 档答案你预计的正确率是多少，用百分比表示。
这个数我会用来决定后面给你派的活需要多少人复核。

第二条，派活时的置信度纪律：

【置信度纪律：本次任务的所有输出都遵守。】

【第一，每个关键结论后面必须跟一档置信度。】

格式：结论（置信度：X 档，依据：XXX）

关键结论包括：数值、判定、选型建议、标准条款、试验结论。

【第二，不许全部答 A 档。】

一份输出里 A 档占比超过八成，我会判定你在糊弄，
退回重做。真做过的事一定有不确定之处，把不确定写出来。

【第三，E 档不许空着。】

本次任务中至少有两条内容标为 E 档（不知道）。

一条都标不出来，说明你的自评没有经过思考，退回重做。

标 E 档的内容后面必须跟一句：建议去哪儿查、找谁确认。

【第四，反证测试。】

每个 A 档结论后面，必须附一句：

"这个结论在以下情况下不成立：XXX。"

写不出反例的 A 档结论，自动降为 B 档。

【第五，不许事后改口。】

你现在给的档位和依据会被我存档。

如果后续发现某个 A 档结论是错的，我会拿这份存档跟你核对。

所以现在标 A 要慎重，标错了扣分，标对了加分。

【第六，遇到不知道就说不知道。】

"我不知道"这四个字是我最想听的答案之一。

它意味着我可以自己去找答案，而不是被一个错的答案带走。

编一个看着合理的答案，是本次任务里唯一的零分项。

过程与判断点

一份合格的校准输出长这样：

- > 第 1 题 答案：XXX (B 档，依据：YYY)
- > 第 2 题 答案：XXX (A 档，依据：YYY)
- > ...
- > 第 7 题 答案：我不知道 (E 档，建议查：ZZZ)
- > 十道题档位分布：A 3 题、B 3 题、C 2 题、D 1 题、E 1 题。
- >
- > 反证：
- > 第 2 题在以下情况下不成立：如果该文件是 2023 年之后的版本，条款号已变。
- > 第 5 题在以下情况下不成立：如果这批样件是第二批而非首批。
- > 第 8 题在以下情况下不成立：如果温度是稳态而非瞬态。
- >
- > 校准结论：A 档答案我预计正确率 85%。

我看三处。

第一，档位分布。十个答案分在五个档里，说明它在做真实自评。全 A 或者全 B，说明它在套模板。

第二，反证那三行。"如果该文件是 2023 年之后的版本"这一句，说明它知道自己这个答案的边界在哪。写不出反证的 A 档，一律不信。

第三，最后那个百分比。它说 85%，我就按这个数安排复核量。十个 A 档答案我核三个。它如果说 100%，我核全部。

翻车清单

翻车一：它把全部答案标成 A 档，然后被你逼问才改。症状是你问"这个你确定吗"，它立刻从 A 降到 C。

补救：在提示词里写死"档位一经给出不得更改，改档视为本次自评无效"，并且让它给依据，没有依据的 A 档不算数。

翻车二：校准题的答案它也编。症状是十道探针题它答得很好，因为那些题恰好在它的强项里。

补救：探针题必须覆盖三个难度，其中至少两道是它不可能知道的冷门内容（比如你公司内部的编号规则、某份文件的第几页）。这两道题它答"不知道"才是正常的，答上来了反而可疑。

翻车三：它学会了说"不知道"，但说得太多。症状是啥都标 E 档，你什么东西都得自己查，等于没用它。补救：E 档设上限，比如一份输出里 E 档不超过三条，超了说明它在逃避。真正好的自评是 A 三、B 三、C 二、D 一、E 一这种分布。

验收标准与变体

验收五条：

1. 一份输出里至少出现三个不同的置信度档位。
2. 每个 A 档结论后面都有反证句，写不出反证的已自动降级。
3. 至少有一条 E 档，且后面给了查证方向。
4. 随机抽一条 A 档结论去核，核对结果与其自评一致。
5. 同一份任务隔一天重跑一次，两次的档位分布大体一致。

变体一：用它做设计评审的自查。让它给你的设计方案挑毛病，并要求每条意见标置信度。B 档以下的意见你自己判断，A 档的意见必须逐条回应。

变体二：用它带新人。让新人问它问题的时候，强制要求它给出置信度和依据。新人学到的是"这句话有多少把握"，而不是一个答案。这比任何培训都有效。

变体三：用它做决策前的反方。你有一个方案，让它专门找这个方案的毛病，并要求每条毛病标置信度。这一招在评审会前用，能提前挡掉一半提问。

老兵说：它说"我很确定"的时候，你要做的不是相信，是记下来。记下来，事后对账。对过三次账，你就知道它的"确定"值几分，那个分才是你真正能用的东西。

八宗罪的联合反制：一张检查单

八条反制提示词分开用是八件工具，合起来是一张能贴墙的检查单。下面这张表是我自己贴在显示器边上的版本。

第一部分：开工前（三分钟，一次做对省两小时）

#	检查项	动作	不通过怎么办
1	开工确认单	先交五行确认单，你签字	型号或边界不对就改，改完再签
2	三栏清单	必做、不做、发现但暂不处理	必做超过五条就拆成两次
3	允许引用清单	列出本次允许出现的标准号、牌号	清单外的引用一律不许出现
4	探针校准	十道题测出当前准确率	A 档你预计正确率低于 80% 就加倍复核
5	文件白名单	只许碰这几个文件	动手前记一次所有文件的修改时间

第二部分：执行中（每轮收尾两分钟）

#	检查项	动作	不通过怎么办
6	方案门	三条路摆开，你选一条	三条路只差参数就退回重做
7	改动对照表	改前先列旧值新值，你点头才改	对不上必做条款的行划掉
8	状态三文件	每轮先读 PLAN、FINDINGS、LOG	交接测试：换新对话能不能接上
9	中途抽检	长任务至少抽两次术语和单位	不一致就先修已有内容还是本轮，你定
10	五段输出	结论、要你定的、依据、风险、下一步	风险段为空就退回

第三部分：交付时（十五分钟，这一步不能省）

#	检查项	动作	不通过怎么办
11	样本数闭合	送检、完成、有效、剔除四个数核平	对不上就停下来查差异
12	溯源三问	每个外引数字：文件名、位置、原值	随机抽五条翻原件核对
13	算术复核	分摊、年降、税率、汇率、单位换算	这五处逐个重算一遍
14	限定词核查	所有"N 台""全部""均"逐个核样本数	与原始记录不一致的全部改
15	四行证据表	检查了什么、怎么查、结果、证据位置	证据位置指向的文件打不开就作废
16	交叉体检	换新对话重做一遍体检	两份的高危条目清单必须一致

八宗罪联合反制检查单

开工前三件事、执行中五件事、交付时六件事

- ✓ **开工前五件：三分钟**
开工确认单、三栏清单、允许引用清单、探针校准、文件白名单
- ✓ **执行中五件：每轮两分钟**
方案门、改动对照表、状态三文件、中途抽检、五段输出
- ✓ **交付时六件：十五分钟**
样本数闭合、溯源三问、算术复核、限定词核查、四行证据表、交叉体检
- ! **不通过怎么办**
型号或边界不对先改再签；三条路只差参数就退回重做
- ✗ **别一次全上十六项**
第一次用只挑最常翻车的那一条，先跑两周
- ✓ **按任务大小分档**
改一句文案直接做；跨多天多文件的活全上

十六项全做，一个小时的活能变成三小时

图 17-10 八宗罪联合反制检查单：开工前三件事、执行中五件事、交付时六件事

图 17-10 作者根据公开资料绘制。

这张单子怎么用，我说两句实在话。

第一，别一次全上。十六项全做，一个小时的活能变成三小时。第一次用，只挑你最常翻车的那一条对应的检查项，用两周。

第二，按任务大小分档。改一句文案，直接做再看一眼就够。写一份报告，启用开工前五项加交付时六项。跨多天、多文件、错了要赔钱的，全部十六项。

小白词典

SOR：客户发给供应商的需求说明书，一份几十到几百页。类比：甲方给乙方的一份超长购物清单，还附了验收标准。

设计输入清单：把 SOR 拆成一条条能分给工程师干的活。类比：把一本菜谱拆成一张备料单和一张步骤卡。

开工权：谁有权说“现在可以动手了”。类比：施工队的开工令，没签字不许进场。

外引事实：它从自己记忆里拿出来东西，你给的材料里并没有。类比：别人转述的一句话，你没听过原话。

幽灵引用：看着像引用，实际不存在的标准号或条款号。类比：一个打不通的电话号码，写在名片上看着很正式。

溯源三问：谁测的、哪台设备、原始文件在哪。类比：菜市场问一句“这鱼哪儿捞的”。

方案门：正式动手之前的一个停点，只比较路线不执行。类比：点菜前先看菜单，别让服务员直接上菜。

三栏清单：必做、不做、发现但暂不处理。类比：购物清单、黑名单、心愿单，三张不能混。

状态三文件：计划、发现、执行记录。类比：施工日志、设计变更单、验收单，三本台账。

证据表：完成声明必须附的一张表。类比：快递签收单，没签字不算送达。

置信度分档：把答案按把握程度分成五档。类比：天气预报的降水概率，百分十和百分九十你不能一样带伞。

你的岗位怎么用

质量工程师：你每天面对的就是数字和判定，而脑补和报喜这两宗罪恰好专攻这两样。具体动作：把场景 62 那条脑补体检提示词存成技能，每次拿到 AI 生成的检测报告先跑一遍；把场景 67 的四行证据表做成你部门的报告模板，供应商的整改报告也要求按这个格式填，因为整改措施最容易只做一半就报完成。

试验工程师：试验这一行最容易出事的地方是样本数：送进去 12 台，报告里 10 台，中间两台去哪了没人问。具体动作：把"送检、完成、有效、剔除四个数闭合"写进你部门试验报告模板的第一行；开工确认单的第一栏改成"试验样件的批次号与状态"，不同批次的同一个件，试验结果不能混着算。

设计工程师：你每天在拍板和改图之间来回，独断这条罪会替你拍板，加戏这条罪会改你不让它改的图。具体动作：评审之前让它先摆三条路，每条写清放弃什么，你拿着这张表去开会；改图的时候给它一张文件白名单加一份"明确不做"清单，改完用 diff 核一遍。特别提醒：受控文件（发给客户的规范和图纸）改之前先确认版本状态，AI 不知道你的文件受不受控。

本章交付物

两件能带走的东西：八条反制提示词卡，加一份 AI 产出验收清单。

第一件，八条反制提示词卡。把八个场景里那段"可复制反制提示词"逐条存成独立文件或一个 WorkBuddy 技能，按罪名命名：[罪1_抢答_开工确认单](#)、[罪2_脑补_事实纪律](#)、[罪3_独断_方案门](#)、[罪4_加戏_三栏清单](#)、[罪5_话痨_五段输出](#)、[罪6_失忆_三文件法](#)、[罪7_报喜_证据表](#)、[罪8_元认知_置信度校准](#)。存好之后，每次派活前想一秒：这次最可能犯哪一条，把对应的那张卡贴在提示词最前面。

第二件，一份 AI 产出验收清单。把那张十六项检查单抄进你的笔记软件，每一项的"不通过怎么办"换成你公司真实的规矩，比如退货流程、谁签字、走哪张表单。

第一件要验收的活：挑一份你手上最近由 AI 生成的交付物，用场景 62 那条脑补体检提示词跑一遍。跑完你会拿到一张问题表，表上高危条目的数量，就是你过去这些天一直在承担的风险。

本章金句

AI 出错的时候不结巴、不停顿，也不说"我可能记错了"。

报告里最贵的不是那个数，是那个数前面的限定词。

它抢答不是因为它急，是因为它被训练成先给一个答案再说。

AI 编一个数不用一秒，你为这个数付的钱可能是一百万。

对话是它的短期记忆，文件才是它的长期记忆。

它说"完成"的时候，你唯一该问的是"你检查了什么"。

"我不知道"这四个字，是我最想从它嘴里听到的答案之一。

门要立在它最容易犯错的地方，不是立在所有地方。

（下一章：验收、溯源与责任。八道门立完之后，最后要解决的只剩一件事：这份东西签了字，出了事算谁的。）

第 18 章

验收、溯源与责任：在它的产出上签你的名字

它干得再快，签字栏里也只写得下你的名字；所以那一验，没人能替你做。

2025 年 11 月 6 号，周四下午两点，某欧系平台的季度评审会。我带过去一份十二页的微电机 NVH 分析报告，图表齐全，格式比我手写的任何一份都干净。

投影翻到第六页，加粗那行写着：12 台样机，异响判读与人工复核一致率 98.6%。

客户那边的质量经理抬手，指着那一行问我："这个数是哪来的？"

我说这是系统自动判读跑出来的。

"哪台设备？哪一天？原始记录在哪个文件里？"

我低头翻手里的打印稿，翻了两页，答不上来。会议室里安静了大概五秒。

那五秒我这辈子都记得。我在这行干了十一年，从电机研发工程师做到带团队做创新研发，评审会参加过上百场，从来没被人问住过。那天我被问住了，而问我的问题，是一个我本该在开口之前就自己问过自己的问题。

散会回到工位，我把原始目录整个翻了一遍。真相是：12 台样机里真正跑完判读的只有 7 台，另外 5 台的试验没做完，是它照着前 7 台的格式把报告补全的。98.6% 这个数是对的，7 台也是对的，出问题的是"12 台"这三个字，那三个字是它自己加的，加得跟真的一样。

那天下午我在工位上坐了很久。我羞耻的不是它编了三个字，是我拿着一份自己没核过的东西，在客户面前签了我的名字。那份报告的封面页上写着我的工号，我签的时候连第 6 页都没翻到第二遍。

第 17 章写过这一幕，那一章讲的是它为什么会这么干。这一章我要把后半段讲完：被问住之后我给自己立了三条规矩，用了十个月。

第一条规矩是验收。它交回来的东西，我按出错代价分成三档检查，不再用"看起来挺好"这种标准签字。

第二条规矩是溯源。写进我报告里的每个数字，必须能追到哪次试验、哪个样本、经过什么处理。这条规矩不是我发明的，是汽车零部件行业本来就有的，我只是把它搬到了 AI 身上。

第三条规矩是责任。哪些活能全自动跑，哪些活要我拍最后一下板，哪些活它只能给我信息不许替我做决定，我列了一张表贴在工位上。

这三条规矩合起来回答一件事：AI 不能签字，能签字的只有你。所以它不是替你省掉了验收，是把验收这件事还给了你，而且比从前更重。

下面是本章四个场景的速览，你先挑跟你有关的那个看。

场景	场景名	适用场景	可信度标注
69	验收三档：快速核、抽样核、全量核	任何 AI 产出物在签字发出之前	★★★ 实测
70	溯源：让每个数字都能追到来源	报告、分析、对外文件、评审材料	★★★ 实测
71	责任边界：L1 / L2 / L3 决策矩阵	派活之前决定这件事交给它几分	★★ 跑通
72	出事之后：止损、溯源、改规矩	已经出错，要复盘并防止再来一次	★★★ 实测

本章三条规矩

验收定档、溯源追数、责任分级



它不是替你省掉了验收，是把验收还给了你

图 18-1 本章三条规矩的关系：验收定档、溯源追数、责任分级

图 18-1 作者根据公开资料绘制。

场景 69 验收三档：快速核、抽样核、全量核 ★★★

一句话背景

它把活干完的那一下，你面前摆着一份产物，要在几分钟内决定它能不能发出去。全核太慢，不核又出过事。这节给你一张按出错代价选档的决策表，加三档各自的检查动作原文，你照着选、照着核，五分钟能拿主意。

名词小词典

验收：在它交回来的产物上做一组固定检查动作，确认能用再签字。类比：快递拆开看一眼再签收。

快速核：只核最外层信号，两分钟做完，看格式、条数、空行、占位符。类比：看快递外包装有没有破。

抽样核：随机挑一部分逐条跟原始素材对，挑多少按出错代价定。类比：一箱零件抽检五个。

全量核：每一条都对一遍。慢，但要付钱、要担责的东西只能用这一档。类比：出货前的全检。

出错代价：这份产物错了最坏会怎样，分可撤销、可重做、要付钱、要担责四层。类比：打错一个字和打错一个零。

验收门槛：超过就算不合格的那个数，由你写，事后不许改。类比：体检报告上的参考区间。

开工前准备

逐条核对，少一条都会卡住：

1. 一份它刚交出来的产物，文件在手，能打开。
2. 产生这份产物的原始素材：源数据表、试验记录、客户发来的输入文件。没有原始素材，抽样核和全量核都做不了。
3. 一张你自己写的验收门槛表，纸上或笔记软件里都行。这份表不许让它写，它写的门槛会跟着它的产出一起漂。
4. 一个计时器。三档的差别一半体现在时间预算上，超预算就该换档或者拆活。
5. 一个能打开产物的软件，并且你知道怎么在这个软件里读真值，而不是看界面上显示的数。
6. 一条写死的规矩：没过验收的产物，不许发出去，不许进共享盘，不许发群里。

操作步骤

1. 拿到产物先别看内容，先问一句：这东西错了，最坏会怎样？把答案写下来，四个词里选一个。
2. 按下面的决策表定档。定档必须在打开产物之前做完，打开了再定，你会被它的排版说服。
3. 按档执行检查动作，每一项结论写成文字，不通过就写未通过。
4. 抽样核那一档，抽样清单由你随机挑，不许让它挑它想让你看的那几条。
5. 全量核那一档，核一条在原始素材上做个记号，防止跳行和重复。
6. 检查结果写成小文件存档，文件名带日期。聊天记录不算存档。
7. 全部通过才签字发出。有一项未通过就走返工，不许带着未通过发出去。

按出错代价选档

四层代价对应三档检查，先定档再打开产物



定档必须在打开产物之前做完，打开后你会被排版说服

图 18-2 按出错代价选档：四层代价对应三档检查

图 18-2 作者根据公开资料绘制。

按出错代价选档的决策表

这张表是本节最实用的东西，建议直接抄走：

出错代价	一句话判断	选档	时间预算	工程现场的例子
可撤销	错了能一键撤回，没人知道	快速核	2 分钟	内部草稿、会议纪要、自己看的整理稿
可重做	错了重跑一遍，代价是时间	抽样核	15 分钟	周报数据汇总、文献检索清单、文件归类
要付钱	错了要返工、要赔钱、要重订	全量核	不限，做完为止	BOM、采购清单、发给外协的图纸包
要担责	错了会伤人、会丢资质、会上法庭	全量核加第二人	双签，两人各核一遍	安全件校核、对外技术承诺、量产放行

四层里最容易定错的是第三层和第四层。判断方法很土但管用：问一句这份东西发出去之后，如果里面有一个数是错的，第一个发现的人是谁。是你自己，多半是可撤销或可重做；是车间、客户、检验员，那就是要付钱或要担责。

可复制提示词

这一段是让它自己先过一遍三档自检并把报告写出来。整段复制，方括号里的内容换成你自己的：

【任务】你刚刚完成了我交给你的任务，产物是 [文件路径]。

在交给我之前，先按下面三档里的 [快速核 / 抽样核 / 全量核] 档做一次自检，把自检结果写成 [报告输出路径]，再向我汇报。

【第一项：结构自检】

报出：产物总条数、空行数、占位符出现次数（占位符指 TODO、待定、TBD、NAME、SCALE、XX 这类没被替换的字段）、异常值条数。

任何一项不为零，直接写未通过，并把那几行原样贴出来。

【第二项：边界自检】

报出：第一条和最后一条的完整内容、最大最小值、合计值。

合计值必须与明细逐条相加的结果一致，不一致写未通过。

【第三项：来源自检】

逐条列出产物里的每个关键数字，每个数字后面跟三列：

来源文件与位置、处理过程、是否为外部引入事实。

来源给不出，写“未核实”，不许留空，不许用“根据常规做法”这类话带过。

【第四项：反向自检】

主动找一遍自己的产物里最可能错的三处，写在报告最后一段。

找不到就写“未发现可疑点”，不许写“没有问题”“一切正常”。

【纪律】

不许为了让自检通过而修改产物。自检发现的问题原样写，不许自行修复。

每一项的结论只能是“通过”“未通过”“未核实”三个词之一。

写完报告后向我汇报：本档名称、四项结论、未通过与未核实的条数。

快速核那一档不需要上面那么多，用这条短的：

【快速核】两件事，各用一句话回答，不许展开：

1. 这份产物的总条数是多少，其中空行和占位符各有多少？
2. 你在做这份产物的过程中，有没有哪一处的来源是你自己不确定的？有就说是哪一处，不许写“没有”。

过程与判断点

一份合格的抽样核自检报告长这样，六行对得上才算过：

- > [结构] 总条数 2147, 空行 0, 占位符 0, 异常值 3 (第 88、901、1902 行)。
- > [边界] 首条 A001-2025-11-06, 末条 A147-2026-03-19, 合计 2147, 与明细合计一致。
- > [来源] 关键数字 18 个, 其中 16 个来源于 D:\proj\raw\test_log.xlsx 的 Sheet2,
- > 1 个来源于我按你给的经验值推算 (标记: 未核实),
- > 1 个来源未找到 (标记: 未核实)。
- > [反向] 最可能错的三处: 第 88 行单位缺失; 第 901 行日期格式与其余不一致;
- > 1902 行的数值与相邻行相差两个数量级, 疑似换算错误。
- > [结论] 通过 1 项, 未通过 1 项, 未核实 2 项。
- > [已写报告] D:\proj\check\20260319_sample_check.md

这份报告里我盯三个地方。

第一, 它有没有主动咬自己一口。上面那份里第四项反向自检, 它自己揪出三处可疑点, 其中第 1902 行差两个数量级这种事, 我肉眼扫十遍也未必看得见。一份自检报告里一个可疑点都没有, 通常说明自检是走过场: 真在查的人, 一定查得出东西。

第二, 未核实那几条的写法。写"未核实"可以接受, 写"根据常规做法"不行, 后者是拿它自己的记忆冒充来源。我把结论词写死成"通过""未通过""未核实", 就是为了堵死它用形容词糊弄的路。

第三, 有没有偷偷改产物。它一边自检一边顺手改, 改完之后你拿到的报告全部通过, 而产物已经不是你给它的那一版。补救: 提示词里写死不许修, 自检前先复制一份留底。

翻车清单

翻车一: 档定低了, 因为产物太好看。症状是排版精美、图表齐全, 你扫一眼就通过了, 错的数字藏在第三张图的标签里。根因是好看会让人降低警惕, 这是人的毛病不是它的。补救: 先定档后打开, 顺序不能反。

翻车二: 抽样让它自己挑。症状是抽的五条全部正确, 你签了字, 剩下两千条里躺着错误。根因是它挑的是它最有把握的那几条。补救: 抽样清单你自己随机挑, 按行号等距取, 抽完把行号写进报告。

翻车三: 全量核到一半疲劳, 后半段放水。症状是前半段逐条对, 后半段扫一眼就过, 错误恰好在后半段。补救: 分批核, 每批不超过两百条; 或者在原始素材上做记号, 没记号的行就是没核的行。

验收标准与变体

验收四条, 按顺序做完:

1. 档是在打开产物之前定下的, 并写在了纸上或文件里。
2. 检查报告里四项结论齐全, 每项结论是三个规定词之一, 没有形容词。
3. 未通过与未核实两类条数之和为零才允许签字, 不为零走返工。
4. 抽样或全量核的原始素材上留有记号, 记号数等于已核条数。

变体一: 把它做成一条技能。把那段自检提示词加上四层代价的判断问句, 打包成 WorkBuddy 技能, 命名"交付前自检"。以后它交活, 你只说一句"跑交付前自检, 抽样核", 报告自动落到同一目录。

变体二：给文档类和给数据类分开做。文档类的检查项换成：术语是否统一、编号是否连续、引用条款是否真实存在、有没有残留的占位符。数据类的检查项换成：条数、合计、单位、极值、异常值。两套清单不能混用。

变体三：验收门槛写进提示词。把门槛数字直接写死在提示词里，比如"体积相对误差超过 $1e-5$ 写未通过"。门槛不写进提示词，它就会自己取一个宽松的。

老兵说：验收不是不信任它，是承认自己看不全。两分钟内能做完的快速核，我做了十个月，它没有一次让我后悔花这两分钟；而让我后悔的那一次，是我连两分钟都没花。

场景 70 溯源：让每个数字都能追到来源 ★★★

一句话背景

评审会上被问"这个数是哪来的"，答不上来的那一刻，你丢的不是这个数，是这份报告里所有数的可信度。这一节给你一张"数字来源对照表"模板，讲清源数据、处理过程、结论这三层，让你报告里的每个数字都能在十秒内追到它出生的那一行。

名词小词典

溯源：从结论数字往回追，一直追到它最初的来源。类比：菜出了问题，一路追到哪块地种的。

源数据：最原始、没被加工过的记录。台架导出的原始日志、供应商原件、客户发的 SOR 原文都算。

类比：刚拔出来的菜，没洗没切。

处理过程：从源数据到报告数字中间的每一步：筛选、剔除、换算、取平均、四舍五入。类比：洗切炒，每一步都改变它。

结论：最终写进报告、拿去支撑一个判断的那个数。类比：端上桌的那盘菜。

数字来源对照表：把每个数字的源数据、处理过程、结论写在同一行的一张表。类比：菜品溯源卡。

限定词：数字前面的定语：几台样机、什么工况、哪个批次、多长时间。溯源时限定词要跟数字一起追。

开工前准备

1. 待溯源的产物：一份报告、一张表、一页评审材料。
2. 产生它的全部原始素材，放在一个你能找到的目录里，文件名不许叫"新建文件夹"。
3. 一个固定的溯源目录，用来存对照表。我用的是 `D:\proj\trace\`，按日期建子目录。
4. 一条写死的规矩：报告里出现的每个数字都要在对照表里有一行，一个不许漏。
5. 半小时。一份十二页的报告做完溯源大约要这么久，第一次更久。

操作步骤

- 1. 先让它把产物里的全部数字列一遍，一个不漏，包括图上的标签和图注里的数。
- 2. 你自己扫一遍这个清单，看有没有它漏掉的或者你没打算写进去的。
- 3. 按三层结构让它逐个填表：源数据、处理过程、结论。
- 4. 处理过程那一列不许写"计算得出"，必须写清筛选条件、剔除规则、换算系数、取几位小数。
- 5. 填完你自己抽三个数往回追一遍：顺着它写的路径打开那个文件，找到那一行，看数字对不对得上。
- 6. 追不上的数字标"未核实"，从报告里删掉，或者改写成一句不带数字的话。
- 7. 把对照表作为附件一起发出去，附件的存在本身就是态度。

溯源三层

源数据经处理过程变成结论，每层都要留痕

源数据：最原始、没被加工过的记录

具体到文件名、Sheet 名、行号区间、样本编号与试验日期

处理过程：从源数据到结论中间的每一步

筛选条件、剔除规则与条数、换算系数、取几位小数

结论：最终写进报告的那个数

它支撑的是哪一个判断；状态只能是已核实、未核实、推算值

一个数字追不到来源，它就不是数字，是一句好听的话

图 18-3 溯源三层：源数据经处理过程变成结论，每层都要留痕

图 18-3 作者根据公开资料绘制。

数字来源对照表模板

这张表可以直接抄，列不要删：

#	报告中的数字 (含限定词)	源数据：哪个文件、哪次试验、哪个样本、哪一行	处理过程：筛选、剔除、换算、取几位小数	结论：它支撑什么判断	状态
1	12 台样机，一致率 98.6%	test_log_20251104.xlsx / Sheet2 / 第 41 至 52 行	剔除 2 台未完成；判读与人工复核逐条比对；保留一位小数	支撑"判读可用"这个判断	已核实
2	7 台样机，一致率 98.6%	同上，第 41 至 47 行	同上	支撑同上	已核实
3	平均温升 42 K	thermal_20251022.csv / 第 1201 至 1800 行	取最后 600 秒的算术平均；热电偶 T3 剔除（断线）；取整	支撑"温升在规格内"	已核实
4	行业通行寿命 1 万次	无	无	支撑背景描述	未核实，已删除

第 4 行是这张表最重要的用法示范：一个追不到来源的数字，处理方式只有两个，删掉，或者改写成不带数字的话。不许留着，不许写"约"，不许加脚注。

可复制提示词

整段复制，路径和数字换成你自己的：

【任务】给我正在写的这份报告 [报告路径] 做一次完整溯源，产出一份“数字来源对照表”，写到 [对照表输出路径]。

【第一步：列数】

把报告正文、表格、图注里出现的全部数字逐条列出来，一个不许漏。

每条写清：数字本身、它前面的限定词（几台、什么工况、哪个批次、多长时间）、它出现在第几页第几段。限定词和数字必须一起列，不许只列数字。

【第二步：填三层】

对每个数字填三列：

1. 源数据：具体到文件名、Sheet 名或表名、行号区间、样本编号、试验日期。
不许写“试验数据”“相关文件”这种泛指。
2. 处理过程：写清筛选条件、剔除规则与剔除条数、换算系数、单位换算、取几位小数、取平均还是取极值。不许写“计算得出”“经过处理”。
3. 结论：这个数字在报告里支撑的是哪一个判断，一句话写清。

【第三步：标状态】

每个数字的状态只能是三个词之一：已核实、未核实、推算值。

已核实 = 你能指出源数据的具体位置，并且我顺着这个位置能找到同一个数。

未核实 = 来源找不到，或者来源是它自己的记忆。

推算值 = 源数据存在，但这个数是在源数据基础上按某个假设推出来的，必须把假设写在同一格里。

【第四步：处置建议】

对每条未核实和推算值，给两条建议：

- A. 从报告里删除，并给出改写后不带数字的那句话。
 - B. 补做一次什么动作能让它变成已核实。
- 不许给第三条建议，不许说“可以保留”。

【纪律】

不许为了让状态好看而修改源数据，不许把未核实改写成已核实。

每一个数字的来源如果指向它自己的记忆而不是我给的文件，一律写未核实。

表写完后向我汇报：总数字条数、已核实条数、未核实条数、推算值条数。

补一条用来做抽查的短提示词，你自己挑三个数让它追：

【抽查】只追这三个数：[数字 1] [数字 2] [数字 3]。

每个数给我三段话：

1. 打开哪个文件，翻到哪一行，那一行原文是什么。
 2. 从那一行到报告里的这个数，中间做了什么，写出算式。
 3. 你自己对这个来源的把握程度，用“确定 / 八成 / 一半以下”三档回答，并说明为什么是这个档。
- 三档里只要有一个不是“确定”，这个数在报告里就标未核实。

过程与判断点

一段合格的抽查回答长这样：

- > [数字 1] 42 K
- > 1. 文件 D:\proj\raw\thermal_20251022.csv, 第 1201 至 1800 行,
- > 第 1500 行原文: 2025-10-22 14:03:11,T3,58.4,T4,61.2,T5,59.7
- > 2. 取第 1201 至 1800 行的 T4 与 T5 两路, 算术平均得 60.45,
- > 减去环境温度 18.4, 得 42.05, 保留整数位得 42。
- > 3. 把握程度: 确定。原始文件在我手上, 算式可复算。

- > [数字 3] 行业通行寿命 1 万次
- > 1. 无文件。
- > 2. 无算式。
- > 3. 把握程度: 一半以下。这是行业里的常见说法, 我无法指出具体出处。

我看到"无文件"这三个字, 就知道第 3 个数该删了。这份诚实比一个漂亮的来源值钱, 它在评审会之前替我挡了一刀。

溯源这件事我盯三个判断点。

第一, 源数据那一列能不能指到行号。写"试验数据"等于没写。能指到行号、样本编号、试验日期的, 才是真来源。这也是汽车零部件行业本来就有的规矩: 写进报告的数必须能追到哪次试验、哪个样本、经过什么处理。我把这条原样搬了过来。

第二, 处理过程那一列有没有剔除规则。剔数据是合法操作, 不写剔除规则就是黑箱。12 台里剔掉 2 台, 原因必须写出来: 是没做完还是不合格? 两种原因对应的结论完全不同。

第三, 限定词有没有跟着一起溯源。98.6% 追到了还不行, "12 台"这三个字也要追。我栽的那一次就栽在限定词上: 数字全对, 限定词是编的。

翻车清单

翻车一: 它给了一个来源, 那个来源是它自己上一步的产物。症状是源数据那一列填的是另一份 AI 生成的文件。根因是它把中间产物当成了源数据。补救: 提示词里写死源数据必须指向原始素材目录, 指向别处一律写未核实。

翻车二: 处理过程里的剔除规则是事后编的。症状是先有结论, 再补一个"剔除了异常值"的说明。根因是你没在跑数之前定剔除规则。补救: 剔除规则在开工之前和验收门槛一起定死, 事后不许改。

翻车三: 对照表写了, 但没作为附件发出去。症状是对照表躺在本地目录里, 报告发出去的时候没带。补救: 对照表就是报告的一部分, 发出去的包里必须有它。它的意义不是给人看, 是逼你写的时候不敢糊弄。

验收标准与变体

验收五条:

1. 报告里每个数字在对照表里都有一行, 条数相等。

2. 源数据列全部指到文件名加行号或样本编号，没有泛指。
3. 处理过程列全部写出筛选、剔除、换算、取位，没有"计算得出"。
4. 你抽三个数顺着来源追一遍，三个都能在原始文件里找到同一个数。
5. 未核实的数字已从报告删除，或改写成不带数字的话。

变体一：给供应商的报告也要求带对照表。你把它写进对供方的交付要求里，格式照上面那张表。供应商的报告一进门就带溯源，你的质量部能省掉一半的来回邮件。

变体二：溯源表反过来用，做来料体检。别人发你一份报告，先抽三个数让它追一遍。追不上的，整份报告按可疑处理。

变体三：把对照表做成报告模板的固定第二页。别当附件，直接排进模板。写报告的人从第一笔开始就知道每个数都要交代出处。

老兵说：一个数字追不到来源，它就不是数字，是一句好听的话。报告里可以有一句好听的话，但不能让它穿着数字的外衣站在结论页上。

场景 71 责任边界：L1 / L2 / L3 决策矩阵 ★★

一句话背景

派活之前有一步我以前从来不做，现在每次都做：先给这件事定级。哪类活能全自动跑完，哪类活要它干九成我拍最后一下板，哪类活它只许给我信息和选项。这一节给完整分级表，每级配工程现场的任务例子。

本节标 ★★，分级这套东西我按它派了三十来天的活，L1 那一类是天天在跑的，但没有逐条留档，所以不标实测。表里每一级的任务例子都是我工位上真实出现过的东西。

名词小词典

L1 常规任务：规则清楚、错了能重跑、后果可控的活，约占日常工作量的八成。类比：流水线上的标准工位，按作业指导书做。

L2 复杂任务：需要判断、有多条路、错了要返工的活，约占一成半。类比：工艺员编排工艺路线，得有人定方案。

L3 高风险任务：错了要付钱、要担责、不可逆的活，约占半成。类比：动火作业，要持证、要监护、要签字。

自动修复：它自己发现出错并自己改好。L1 的活大多具备这个条件，因为错了能立刻看出来。类比：拼写检查改错别字。

拍板：在几条路里选一条并为这个选择负责，这是 L2 里人唯一要做的那一步。类比：菜单看完，点菜的是你。

只给信息和选项：不替你选不替你做，只把选项和代价摆出来。L3 的活它只能做到这一层。类比：顾问给三套方案和报价，签合同的是你。

开工前准备

1. 一张打印出来的分级表，贴在显示器边上。贴墙上没用，你派活的时候眼睛在屏幕上。
2. 一个派活前停顿三秒的习惯。三秒够你问完三个定级问句。
3. 一份 L3 清单，写死你岗位上的哪些事永远不许交给它做。这份清单你自己写，写完不许改。
4. 想清楚 L1 的自动修复条件：发现不了错误的活，不算 L1。
5. 一条写死的规矩：它不许自己给自己升级。定级是你做的事。

操作步骤

1. 派活前先问第一句：这活错了，最坏会怎样？答不上来就按 L3 处理。
2. 问第二句：错了之后它自己能发现吗？能发现的才够格进 L1。
3. 问第三句：这活有几条路可走？只有一条路进 L1，多条路进 L2，涉及人或钱的进 L3。
4. 定完级，把级别写进提示词第一行，让它知道自己在哪一级。
5. L1 直接派，要求它出错自己修，修完汇报修了什么。
6. L2 分两段派：先出方案和选项，你选，然后才让它动手。中间那个停点不许跳过。
7. L3 只让它出信息：把选项、代价、风险摆出来，剩下的你自己做，不许它代劳。
8. 每月月底翻一遍这个月的活，定低过的那一类往上提一级。

责任边界三级

三个问句定级，每级对应不同的派活方式

L1 常规任务 约 80%

规则清楚，错了能重跑，后果可控。它全自动跑完，出错自己修。人只看一眼汇报，验收用快速核。

L2 复杂任务 约 15%

有判断，有多条路，错了要返工。它干九成，最后一步停住。人在选项里选一个，并为这个选择负责。

L3 高风险任务 约 5%

错了要付钱、要担责、不可逆。它只给信息和选项，不碰执行。全部由人做，全量核加第二人。

L2 被当成 L1 派出去，是这张表里最容易出事的地方

图 18-4 L1 / L2 / L3 三级决策：三个问句定级，每级对应不同的派活方式

图 18-4 作者根据公开资料绘制。

L1 / L2 / L3 完整分级表

级别	定义	占比	它的权限	人的动作	验收档	工程现场的任务例子
L1 常规	规则清楚，错了能重跑，后果可控	约 80%	全自动跑完，出错自己修	看一眼汇报	快速核	报表生成、文件整理与归类、信息检索、数据汇总、格式转换、批量重命名、会议纪要整理
L2 复杂	有判断，有多条路，错了要返工	约 15%	千九成，最后一步停住	在选项里选一个，为选择负责	抽样核	方案设计、代码重构、技术文件起草、根因分析、试验方案编排、供应商比选建议
L3 高风险	错了要付钱、要担责、不可逆	约 5%	只给信息和选项	全部由人做，它不碰执行	全量核加第二人	资金操作、人事决策、公开发布、合同签署、生产环境部署、量产放行、对外技术承诺

这张表里最容易出事的不是 L3，是 L2 被当成 L1 派出去。L3 的活你心里有数，因为涉及钱和人，谁都不会忘。L2 的活长得很像 L1：都是文件、都是文字，唯一的区别是里面有判断。

我给自己定的判别法是一句土话：这份产物里有没有一处是你改了之后别人会来问你什么的。有，就是 L2。报表生成没人会问你为什么这个数在第三列，方案设计里每个取舍都会被问。

L1 那一档的实际使用感受，我说得保守一点：绝大多数常规任务能自动跑完，出错的是少数，多数能自动修复。这是我敢把 L1 直接放手的唯一理由。这个感受是三十来天攒下来的，不是一次测试跑出来的，所以我不报精确数字。

L2 的两个真实例子，说明那“最后一步”到底停在哪：

例子一，试验方案编排。它把工况矩阵、样本数量、测量通道、判读阈值全都排好，我只需要做一件事：在“样本 12 台分两批”和“样本 8 台一批”之间选一个。这个选择它替我做不了，因为它不知道车间这个月的产能。它做的是九成的编排，我做的是一成的拍板，而那一成决定了这份方案能不能落地。

例子二，根因分析。它把可能的原因按概率排了序，附了每条的支持证据。最后一步我拍板：先验证哪一条。它排第一的那条我未必先做，因为我知道哪一条验证起来最快。这个判断来自我在这个产品上待的时间，它不知道。

可复制提示词

L1 派活模板，第一行写死级别：

【级别】L1 常规任务。你可以全自动跑完，出错自己修，修完告诉我修了什么。

【任务】[一句话说清这个活]

【验收】完成后跑一次自检：总条数、空行数、占位符数、异常值条数，四项写清，有任何一项不为零，你自己修完再汇报，不许带着问题交给我。

【汇报】三行以内：做了什么、自检四项的结果、修了什么。

L2 派活模板，注意中间那个停点：

【级别】L2 复杂任务。你做九成，最后一步停下来等我。

【第一阶段：只出方案，不许动手】

给我三条路，每条写四件事：走这条路要做什么、要多久、放弃什么、最可能在哪里翻车。三条路的成本与风险各自标清。
写完停住，等我选。我不点头，不许进入第二阶段。

【第二阶段：等我选定第 N 条之后再开始】

按第 N 条执行，每完成一个节点汇报一次真实返回值。
遇到第一阶段没预料到的情况，停下来报告，不许自己换路。

【纪律】不许在第一阶段就顺手把活干了，不许替我选路。

L3 派活模板，它只给信息：

【级别】L3 高风险任务。你只给信息和选项，不许执行，不许替我做决定。

【任务】把 [这件事] 相关的选项摆出来，每个选项写四列：

做法、代价、风险、先例（如果有）。

再写一段：如果我什么都不做，会发生什么。

【纪律】

不许给出“我建议选第 N 条”这类结论。

不许执行任何动作，包括创建文件、发送消息、修改配置。

涉及人、钱、对外承诺的事项，一律由我本人处理。

过程与判断点

L2 那一段的真实对话长这样，重点在第二行：

- > 三条路已列出：
- > A. 样本 12 台分两批，工期 6 周，风险是两批之间设备状态漂移。
- > B. 样本 8 台一批，工期 4 周，放弃的是置信度，结论只能写成趋势。
- > C. 样本 12 台外包一半，工期 5 周，风险是外协数据口径不一致。
- > 已停住，等你选。

我：选 B。理由是这个结论本来也只用于内部方向判断，不进客户报告。

我加的那句理由不是给它听的，是给三个月后翻这份记录的人听的，也包括我自己。L2 里人拍的那一下，除了选哪条路，还要写下为什么选它。没有理由的选择，日后复盘时就是一笔糊涂账。

我盯两个判断点。

第一，它有没有自己给自己升级。症状是你派的是 L2，它干到一半说“我顺手把方案也定了”。补救：提示词第一行写死级别，并要求它每次汇报的第一句复述当前级别。

第二，L1 的活它有没有真的自己修。症状是自检报了三个异常值，然后原样交给你等你说话。补救：提示词里写死“有任何一项不为零，你自己修完再汇报”。它不修通常是因为不确定该怎么修，这时你补一句修的规则，别替它修。

翻车清单

翻车一：把 L2 当 L1 派，它在里面自己拍了板。症状是产物里出现了一个你没同意的方案选择。根因是派活时没写级别。补救：每个提示词第一行写死级别，形成肌肉记忆。

翻车二：L3 的活它碰了执行。症状是你只让它列选项，它顺手把文件发了出去。补救：L3 的提示词里显式列出禁止动作清单：不许创建文件、不许发送、不许改配置、不许下单。

翻车三：L1 的活它不会自己修，因为错误没有客观判据。症状是它反复问你“这样对吗”，比如“帮我写一版更好的描述”。这类活本质上不是 L1，是 L2。补救：凡是没有客观判据的活一律提到 L2，中间加停点。

验收标准与变体

验收四条：

1. 这个月派出去的每件活，提示词第一行都写了级别，没有例外。
2. L2 的活全部走了两段式，第一阶段停点没有被跳过。
3. L3 的活它只产出信息和选项，执行记录里没有它的任何动作。
4. 月底翻一遍，定低级别的活对应的任务类型写进提升清单。

变体一：给团队做一张共同的分级表。你一个人用不够，团队里每个人对同一件事的定级可能差一级。把这张表贴在部门共享空间，附的任务例子换成你们自己上个月干过的活。

变体二：按岗位各做一份 L3 清单。采购的 L3 是下单和合同，质量的 L3 是放行判定，人事的 L3 是录用与辞退，IT 的 L3 是生产环境部署。清单各自不同，写死不许改。

变体三：给 L1 加一个自动修复日志。让它每次自己修错都往同一个文件追加一行：时间、错误类型、怎么修的。月底翻这个文件，出现三次以上的错误类型，说明那条规则该写进提示词了。

老兵说：L1 放手是因为错了能重来，L3 不放手是因为错了没法重来。中间那一成半的 L2，才是你每天真正要花心思的地方，因为它是唯一一处你既能偷懒、又一定会后悔的地方。

场景 72 出事之后：止损、溯源、改规矩 ★★★

一句话背景

前面三节都是防止出事。这一节讲的是已经出事了怎么办：先止血，再查清，最后改掉那条让它能犯这个错的规矩。三步走完大约半天，产出一份复盘文件。我用这套流程处理过评审会那次事故，也处理过之后几次小的。

名词小词典

止损：出事之后第一件事是让损失不再扩大，不是查原因。类比：水管爆了先关总阀，再查哪一段裂的。

影响面：这份错误产物已经流到了哪里，谁手上有一份。类比：一批不合格件先查发到了哪几个工位。

根因：改掉它之后这个错就不会再犯的那一条原因。类比：发烧是症状，感染是根因。

复盘：把一件事从头到尾摆开，看清楚哪一步出了岔子，写成文件。类比：交警的事故认定书。

改规矩：复盘的最后一步不是写检讨，是改一条具体规则，让这个错在流程上犯不出来。类比：不是批评工人没戴手套，是在机器上加双手启动开关。

复发测试：改完规矩拿原来那个错重跑一遍，看新规矩能不能挡住它。类比：打完补丁再跑一次原来的攻击。

开工前准备

1. 出事的产物原件，不许先删。删了就没法溯源。
2. 一份时间线：什么时候派的活、什么时候交的、什么时候发出去的、谁发现的。时间线的精度到小时。
3. 影响面清单：这份东西已经发给了谁，有没有进共享盘，有没有被别人转发过。
4. 一个能停下来做这件事的半天。出事当天做复盘和出事一周后做复盘，效果差很多，因为一周后你已经记不清当时的判断了。
5. 一支笔。复盘的前三步我建议手写，手写比打字慢，慢下来才想得清楚。

操作步骤

第一步是止损，做完这三件事再往下走：

1. 把错误产物从所有能撤回的地方撤回：撤回消息、改回共享盘上的文件、通知已收到的人。

2. 给已收到的人发一句话：哪一份、哪一处、正确版本什么时候给。不许发长篇解释。
3. 把正确版本或者"暂缺"版本顶上去。没有正确版本就先写"此数据待核实"，空着比错着强。

第二步是溯源，按顺序做完这四件事：

1. 把错误产物和中间产物全部留档，单独建目录，目录名带日期和事故代号。
2. 从结论往回追，追到第一个出错的那一步。要追"第一个"，不是追"最明显的"。
3. 查这一步出错的原因，分三类：我给的规矩有缺口、它执行时跳了步、我验收时漏了核。
4. 三类里挑一个作为根因。选不出来，就选那个改掉之后这个错一定不会再犯的。

第三步是改规矩，走完这三件事才算收尾：

1. 写一条具体规则，能写进提示词就写进提示词，能写进模板就写进模板。不许写成"以后要注意"。
2. 把规则加进你常用的技能或模板，并且当天派一个新活验证它生效。
3. 把复盘写成文件，文件名带日期，放进固定的复盘目录。

出事之后三步

先止血，再查清，最后改掉那条规矩

止损

撤回错误产物；一句话通知收到的人；
先用待核实版本顶上

溯源

产物与中间产物留档；追第一个出错的步骤；
三类原因挑出根因

改规矩

写一条具体规则；当天派新活验证；
复盘文件带日期入档

止损三步在任何分析之前做完，边查边扩大损失最贵

图 18-5 出事之后三步：止损、溯源、改规矩，每步的三件具体事

图 18-5 作者根据公开资料绘制。

复盘模板

这一段可以直接复制成文件，填完就是一份复盘：

复盘：[事故代号] [日期]

0. 一句话

[这份产物错在哪，一句话说清，不写背景]

1. 时间线

| 时刻 | 发生了什么 | 谁做的 |

| --- | --- | --- |

| | 派活 | |

| | 它交活 | |

| | 我验收 | |

| | 发出去 | |

| | 被发现 | |

2. 影响面

发给了谁：[]

进了哪些系统：[]

有没有被转发：[]

当前状态：已撤回 / 已更正 / 待更正

3. 止损记录

撤回动作：[]

通知话术原文：[]

当前替代物：[]

4. 溯源

第一个出错的步骤：[]

错误类型：规矩有缺口 / 它跳了步 / 我漏了核

根因（一句话）：[]

为什么它是根因而不是症状：[]

5. 改规矩

新增规则原文：[]

写进了哪里：提示词 / 模板 / 技能 / 部门流程

复发测试：拿原来的错重跑一遍，结果 []

验证时间：[]

6. 遗留

还没解决的部分：[]

下次什么时候回头看：[]

模板里我最看重第 4 节最后一行：为什么它是根因而不是症状。写不出这一行，说明你找到的还是症状。评审会那次，表面原因是它编了"12 台"三个字，症状层面的改法是"以后让它不要编样本数"；真正的根因是我验收时只看了数字没看限定词，所以规则要改成"每个数字连同限定词一起核"。前者它照做，后者我照做。

可复制提示词

出事之后让它帮你做溯源，整段复制：

【任务】我手上这份产物出了错，你来帮我做溯源，不许先改任何东西。

【第一步：留档】

把 [产物路径] 和它的全部中间产物复制到 [事故目录]，
目录名带上今天的日期。复制完成前不许做别的动作。

【第二步：定错位】

把产物的生成过程按步骤拆开，逐步列出每步的输入、输出、你当时的判断。
标出第一个与原始素材对不上的那一步，只标一个。
如果有多个对不上，标最早的那一个，并说明为什么它是最早的。

【第三步：归因】

对那一步，从三个方向各写一段：

- A. 我给的规矩里缺了什么，才让它在这一步能做错。
 - B. 你在执行的时候有没有跳步或者自行判断。
 - C. 我验收的时候，哪一项检查本可以抓到这个错而没做。
- 三段写完，挑一个作为根因，并写清为什么另外两个不是。

【第四步：提规则】

针对根因，提一条能写进提示词的具体规则，给出规则的完整原文。
不许提“以后要注意”“加强检查”这类无法执行的建议。
再设计一次复发测试：描述怎么用原来这个错验证新规则生效。

【纪律】

全程不许修改任何产物文件，只许复制和分析。
每一步的结论不许用“可能”“大概”，说不准就写“不确定”并说明缺什么信息。

过程与判断点

一份填完的复盘里，我最先看两个地方。

第一，时间线上“我验收”那一格有没有填。空格说明我根本没验收，那这次事故的性质就变了：不是它错了，是我没验。我自己的第一次事故就属于这一类。

第二，复发测试那一格有没有做。改完规矩不验证，等于没改。拿原来那个错的原始输入按新规则重跑一遍，看新规则能不能在验收这步拦住它。拦得住，规矩成立；拦不住，规矩写偏了，重写。

还有一条我后来加上的习惯：出事之后不急着追责，追责会让复盘变形，变成一份辩护词。复盘文件里我只写“哪一步”和“哪条规矩”。责任那件事在定级时已经定过了：级别定对了还出事，是流程问题；级别定错了，是定级问题。

翻车清单

翻车一：先查原因再止损，损失在查的过程中扩大。症状是你在翻日志的时候，错误产物已经被下游拿去做下一道工序。补救：止损三步在任何分析之前做完。

翻车二：追到症状就收手。症状是改法写成"下次让它注意"，下次它还犯。补救：用模板里"为什么它是根因而不是症状"那一行逼自己写，写不出来就再挖一层。

翻车三：改了规矩但没写进任何地方。症状是复盘会上都同意，散会之后谁也没改，一个月后同样的事故再来一次。补救：规矩必须在当天写进提示词、模板或技能之一，并当天派一个新活验证。

验收标准与变体

验收五条：

1. 止损三步做完，影响面清单上每处都有对应处置。
2. 错误产物和中间产物已留档，原件没删。
3. 溯源找到了第一个出错的步骤，并写清为什么它是根因。
4. 新增规则的原文已写进提示词、模板或技能之一，不停留在口头。
5. 复发测试跑过，结果记录在复盘文件里。

变体一：没出事也做，叫预演复盘。挑一份你最近发出去的产物，假设它是错的，走一遍这三步。预演能提前暴露流程缺口，而且没有心理负担。

变体二：给团队做事故库。每份复盘放进同一个目录，按季度翻一遍。同一类错反复出现，说明那条规矩还没改对。

老兵说：出事之后最贵的不是那次损失，是同样的错第二次发生。第一次叫事故，第二次叫你没改规矩，第三次那就叫你的职业习惯了。

全书最后一句话

回到序章那个凌晨一点。

那天晚上我在解压包里翻设备清单的 PDF，翻到第四十页的时候手停了一下。我意识到我干了一晚上的活，没有一个动作需要"我"这个人：登录、翻页、复制、粘贴、截图、归类，每一个动作都有固定规则，每一个规则都简单到不需要判断。

那之后的十个月，我把这些活一件一件交了出去。它替我翻完五个供应商门户，替我建出二十三个零件的一整套数模，替我盯过跑一整夜的台架试验，替我把两百页的技术标准拆成检查清单。序章里那个凌晨一点的我，如果看见今天的工位，大概会愣很久。

我交出去的从来不只是一件事，是三件：交给它动手，留给我验收，中间那道责任线由我每天重画一次。这本书从头到尾讲的都是这三件事的分配。

这本书的第一句提醒写在序章里：AI 交回来的东西，你没验过，就不是你的。写最后一章时我想过这句话要不要改温和一点，想了很久，不改。我在评审会上答不上来的那五秒，不是因为它不够聪明，是因为我没验。

所以全书最后一句话是一句很具体的话：从明天起，任何一份有它参与、要发出去的文件，你至少亲手核其中一个数，顺着追到那个数出生的那一行，然后再签字。

核一个数，两分钟。这两分钟是这份文件上唯一真正属于你的部分，其余都是它干的。

小白词典

验收：在它交回来的产物上做一组固定检查动作，确认能用再签字。类比：快递拆开看一眼再签收。

验收三档：按出错代价分的三档检查力度，快速核两分钟、抽样核十五分钟、全量核做完为止。类比：看病分问诊、抽血、全身体检。

验收门槛：超过就算不合格的那个数。门槛由你写，事后不许改。类比：体检报告上的参考区间。

溯源：从结论数字往回追，追到它最初的来源。类比：菜出了问题追到哪块地。

源数据：最原始、没被加工过的记录，试验台导出的日志、供应商原件、客户发文都算。类比：刚拔出来的菜，没洗没切。

处理过程：从源数据到报告数字中间做的每一步，包括筛选、剔除、换算、取位。类比：洗切炒，每一步都改变它。

数字来源对照表：把每个数字的源数据、处理过程、结论写在同一行的一张表。类比：菜品溯源卡。

限定词：数字前面的定语，几台、什么工况、哪个批次、多长时间。类比：鱼的产地，鱼新鲜但产地写错，这张标签就是假的。

L1 / L2 / L3：任务的三档责任级别。L1 全自动，L2 它干九成人拍板，L3 它只给信息。类比：自动驾驶的辅助、半自动、人工驾驶三档。

拍板：在几条路里选一条并为这个选择负责。类比：菜单看完，最后点菜的是你。

止损：出事之后先让损失不再扩大，再查原因。类比：水管爆了先关总阀。

根因：改掉它之后这个错就不会再犯的那一条原因。类比：发烧是症状，感染是根因。

你的岗位怎么用

设计工程师：你的主战场是 L2，方案设计、改图、根因分析都属于它干九成你拍板的那一类。具体动作：把场景 69 的选档表抄一张贴在显示器边上，改图前先定档；把“每个数字连同限定词一起核”写进你的图纸自检清单。

试验工程师：你最容易被问的是样本数。具体动作：报告模板第一行写“送检、完成、有效、剔除四个数必须闭合”，剔除规则开工前写死；把场景 70 的对照表做成试验报告的第二页，每个数都要能指到文件名加行号。

质量工程师：验收和溯源本来就是你的专业，你要做的是把它搬到 AI 产出上。具体动作：把场景 69 的三档检查表改成你部门的检查单，抽样清单由你随机挑；供应商报告要求带对照表进门，追不到来源的数按可疑处理。

项目经理：你要管的是责任边界。具体动作：项目启动会上把本项目的 L3 清单公布一次，写清哪些事永远不许交给它；每个 L2 的活保留停点记录，包括你选了哪条路和为什么选，三个月后复盘时你会感谢自己。

采购与供应商管理：你的 L3 是下单和合同，这两件事它只能给信息。具体动作：把比选、询价、资质核查定成 L2，让它出三条路和代价，你拍板；涉及金额的动作一律 L3，它列选项，你本人操作。

行政与文秘：你手上的活大多是 L1，可以放手。具体动作：报表生成、文件归类、会议纪要、信息检索直接按 L1 派，要求它出错自己修；但对外发布的通知和涉及人事的文档是 L3，只让它出草稿，发布你自己做。

本章交付物

一张"签字前检查单"，打印出来贴在显示器边上。

做法：把下面这张单抄进你的笔记软件，每一条的"不通过怎么办"换成你公司的真实规矩，包括退回流程、谁签字、走哪张表单。抄完打印一份贴在显示器边框上，位置要在你打开待发文件的视线里。

#	检查项	通过条件	不通过怎么办
1	定档与选档	这份产物错了最坏会怎样，答案已写下来，并据此选定三档之一	答不上来按最严档，未定档不许打开产物
2	自检	四项结论齐全，只有通过、未通过、未核实三种写法	出现形容词，退回重跑
3	反向自检	它至少咬了自己一口，列出可疑点	一个可疑点都没有，视为走过场
4	抽样权	抽样清单由我随机挑，行号已写进报告	它自己挑的，作废重抽
5	来源与过程	关键数字全部指到文件名加行号，筛选、剔除、换算、取位都写了	泛指来源写未核实，"计算得出"退回补写
6	限定词	数字和它前面的限定词一起核了	只核数字没核限定词，重核
7	未核实	未核实的数已从报告删除或改成不带数字的话	留着未核实的数不许发出
8	定级	派活时提示词第一行写了 L1 / L2 / L3	未定级的活全部按 L2 补停点
9	停点	L2 的活第一阶段停住等我选，没有被跳过	跳过停点的产物整份重做
10	禁止	L3 的活它没有任何执行动作	有执行动作，走场景 72 的复盘三步
11	签字	以上十项全部通过，我才签	任何一项未通过，不许签字

第一件要验收的活：挑一份你今天就要发出去、有它参与的文件，按这张单逐项打勾。走完你会知道，过去这些天你在多少份文件上签了字却没验过。

本章金句

它干得再快，签字栏里也只写得下你的名字；所以那一验，没人能替你做。

一个数字追不到来源，它就不是数字，是一句穿着数字外衣的好听话。

报告里最贵的不是那个数，是那个数前面的限定词；我在评审会上栽的就是这三个字。

L1 放手是因为错了能重来，L3 不放手是因为错了没法重来，中间那一成半最费心思。

一份自检报告里一个可疑点都没有，通常说明自检是走过场。

第一次出事叫事故，第二次出事叫你没改规矩。

（全书第四篇完。下一章进入第五篇：连接与工具扩展。）

PART FIVE • 第五篇

连接与工具扩展

第 19 章

连接器：让 AI 接上你的邮箱、文档和代码仓库

大多数人说 AI 干不了自己的活，其实是它够不着自己的系统：邮件写好了要你自己点发送，文档整理好了要你自己去粘贴。差的不是智力，是一根插头。

今年四月的一个周三下午三点四十，我在工位上跟一件小事较劲。

那天上午刚出一份微电机耐久试验的中期报告，12 页 PDF，3.4 MB。我要做三件事：把这份报告发邮件给五个同事；把它存进项目共享的那个在线文档目录；在群里给两位同事各推一句要点。

三件事每一件都不难，加在一起我估摸着二十分钟。

我先试了最偷懒的办法。把 PDF 拖进 WorkBuddy 的对话框，跟它说：帮我把它发给这五个人，存进项目文档目录，再给另外两位同事各发一句要点。

它回的第一句话我到现在还记得很清楚："我无法访问你的邮箱。"

我不死心，换了第二个说法：那你把这件事拆成三步，一步步告诉我怎么做。它很认真地给了三步，第一步是"请登录你的邮箱并上传附件"。等于它把活原封不动还给了我。

第三次我改问法，问它能不能直接写进在线文档。它回的是：我这边没有你文档空间的访问权限，你可以把内容复制进去。

那天下午这三件事我最后花了三十八分钟手工做完。中间还出了一次错：附件贴成了上一版，发出去三分钟后才发现，又补了一封道歉邮件。

问题不在它笨。它把要点总结得很准，两句话就把十二页报告的重点说清了。问题在于它跟我的邮箱、我的文档、我的同事之间，隔着一层看不见的墙。它在墙这边，我的系统在墙那边，中间没有一根线。

这一章讲的就是那根线，以及装上这根线之后，你应该把哪些钥匙给它、哪些死也不给。

AI 是主机，连接器是外设线

它有脑子能动脑，但碰不到邮箱、文档和代码仓库



推论三最容易被忽略：报告写得对不对，跟这根线一点关系都没有

图 19-1 AI 是主机，连接器是插在主机上的外设线

图 19-1 作者根据公开资料绘制。

19.1 连接器是什么：给 AI 装的一根根插头

主机与外设的类比

我给你一个类比，这个类比撑得住整章。

把你电脑上的 AI 工作台想成一台主机。它自己有脑子，能算、能写、能判断，但它是裸的：它没有眼睛看你的邮箱，没有手去动你的文档，没有腿走到你的代码仓库里去。

连接器就是插在这台主机上的一根根外设线。

插上硬盘线，它能读你硬盘里的东西；插上打印线，它能打印；插上网线，它能上网。连接器干的是同一件事：把 AI 这台主机的能力，接到某一个具体的外部服务上。

这个类比里藏着三个很实用的推论，我一个个说。

推论一：不插线，主机照样能开机。 你现在没装任何连接器，AI 还是能跟你聊天、能改你给它的文字、能读你拖进来的文件。它只是碰不到那些服务而已。这就是为什么很多人用了三个月 AI，一直以为它“只能聊天”。

推论二：线是分头的。 一根线只接一个服务。接邮箱的线接不了代码仓库，接文档的线发不出邮件。你想让它碰几个系统，就得装几根线，一根一根分别授权。不存在“装一个就全通了”这种东西。

推论三：线插上之后，通的是电，不是脑子。连接器只解决够得着够不着的问题，不解决懂不懂业务的问题。它能把报告发进你的文档目录，但报告写得对不对，跟这根线一点关系都没有。

没连接器 vs 有连接器

同一件事，两种做法，差别在哪儿。这张表里的四个例子都是我日常工作里真实遇到过的：

你要干的活	没连接器	有连接器
把一份报告发给五个人	AI 帮你把正文写好，你自己登录邮箱、填地址、传附件、点发送	AI 直接把邮件发出去，你把收件人名单核一遍
把变更记录整理进项目文档	AI 给你一段文字，你自己打开文档、找位置、粘贴、调格式	AI 把内容写进指定文档的指定位置，你去文档里看一眼
汇报代码仓库本周改了什么	你自己去翻提交记录，把信息复制给 AI 让它总结	AI 自己去读提交记录，回来给你一份带提交号的汇报
三个系统串起来做一件事	你在三个系统之间来回切换，做人工搬运工	AI 按你定的顺序走完三段，每一段都停下来给你看证据

第四行是本章的分水岭。前三种活，连接器只是替你省了几下鼠标；第四种活，连接器把 AI 从"帮你写内容"变成了"帮你跑完一整件事"。

同一件事的两条路

左边是人工搬运，右边是连接器直连

没连接器

发给五个人一份报告

AI 写好正文，你自己登录邮箱、填地址、传附件、点发送

把变更记录整理进文档

AI 给你一段文字，你自己打开文档、找位置、粘贴、调格式

汇报仓库本周改了什么

你自己去翻提交记录，把信息复制给 AI 让它总结

三个系统串起来做一件事

你在三个系统之间来回切换，做人工搬运工

有连接器

发给五个人一份报告

AI 直接把邮件发出去，你把收件人名单核一遍

把变更记录整理进文档

AI 把内容写进指定位置，你去文档里看一眼

汇报仓库本周改了什么

AI 自己去读提交记录，回来给你一份带提交号的汇报

三个系统串起来做一件事

AI 按你定的顺序走完三段，每一段停下来给你看证据

第四行是分水岭：AI 从帮你写内容，变成帮你跑完一整件事

图 19-2 同一件事的两条路：人工搬运 vs 连接器直连

图 19-2 作者根据公开资料绘制。

连接器和另外两件事的区别

书里已经讲过技能和 MCP，容易混，我在这里切一刀。

技能是你把一套做法存下来，让 AI 下次照着做。它不碰外部系统，只改变 AI 自己怎么干活。类比：给主机装软件。

连接器是让 AI 碰到某一个具体的外部服务。它改变的是 AI 够得着什么。类比：给主机插线。

MCP 是一套通用的插座标准，谁都可以照它做插头。第 20 章专门讲。类比：墙上的标准插座。

一句话记住：技能改的是怎么干，连接器改的是能碰谁，MCP 是做插头的规矩。

【待补：WorkBuddy 中连接器管理界面的入口路径，以实际版本为准。本章只讲通用原则与判断方法，不编造菜单路径。】

19.2 常见的几类连接器能干什么

我把常见的连接器分成五类。每一类我都按三件事讲：能做什么、不能做什么、授权时要注意什么。第三件事最重要，也最少有人讲。

先说一句本章的可信度。下面 C1 到 C4 四个场景，我标的是 ★★，意思是这套流程我跑通过，但没有完整留档到能逐条引述每一条返回值的程度。涉及支付那一类，我没有在真实付款上跑过，那一节的内容来自产品公开说明和我读过的文档，标注为 ★ 调研，别拿真钱去试。

代码仓库类：GitHub

能做什么。读仓库里的文件和目录结构；读提交记录，包括谁在什么时候改了哪个文件；读分支、合并请求、议题和评论；在仓库里创建分支、提交改动、开合并请求、发评论。

不能做什么。它读不到你本地还没推上去的代码。它跑不了你仓库里的持续集成流水线，也拿不到流水线跑出来的那份构建日志，除非那个服务另外也接了进来。它判断不了代码写得对不对，只能告诉你改了什么。

授权时要注意什么。这一类授权最常犯的错，是直接给它整个账号的全部仓库写权限。正确的做法是只给它你当前这个项目需要的那一个或那几个仓库，并且先只给读权限。等它稳定跑顺了，你确认它写出来的东西靠谱，再考虑给写权限。

我自己的规矩是这样的：读权限默认给，写权限一个仓库一个仓库单独谈，而且写权限只给那些有分支保护、合并必须经过人审的仓库。这条规矩救过我一次，那次它在一个没保护的仓库里直接推了一版改动，好在那个仓库里只有示例代码。

在线文档类：腾讯文档、金山文档

能做什么。新建文档、在线文档里写内容、改已有文档的内容、读文档内容、按标题或段落定位、把结构化内容写进表格。腾讯文档和金山文档这两家的云端文档都能接。

不能做什么。它改不了文档的权限设置，也管不了谁能看谁能改。它对文档里的排版细节（比如某一页的分栏、某一个表格的合并单元格）掌控力很弱，这是最容易翻车的地方。它读不到没接进来的那份文档，哪怕那份文档就在同一个目录里。

授权时要注意什么。授权的时候要看清是给整个文档空间还是只给某一个文件夹。我建议先只给一个测试文件夹，跑通了再扩。另外，这一类的授权往往跟你的个人账号绑定，你离职或者转岗的时候，记得回来把这条授权撤掉。

邮件类：Agent Mail

能做什么。读收件箱、按条件搜索邮件、读单封邮件的正文与附件、发新邮件、回复、转发、把邮件存成草稿、下载附件。

不能做什么。它管不了你的邮箱规则、过滤条件和签名档。它读不到你邮箱里那些被归档到别处的邮件，除非搜索条件覆盖了那里。它判断不了哪封邮件该回哪封不该回，这件事必须你定规矩。

授权时要注意什么。这一类是全章风险最高的一类，因为邮件的收件人是你填的，发出去就收不回来。我的建议是：只给起草权限，不给发送权限。让它把邮件写成草稿，你看过再发。这一条在场景 C3 里我会写成硬规矩，写进提示词里。

支付类

能做什么。按产品公开说明，这一类可以完成下单后的支付动作，以及在支付被取消之后对同一笔订单重新发起支付。

不能做什么。它不能替你决定这笔钱该不该花。它也不能在没有明确订单的情况下凭空付一笔钱出去，支付必须挂在具体订单上。

授权时要注意什么。这一类我标 ★ 调研，我没在真实付款上跑过。我给你一条我自己的硬规矩：我到现在没给任何 AI 开过可付款权限，一次都没有。我在电机项目上管过采购预算，我知道一笔钱点出去之后要花多少时间才能追回来。

如果你真的要用这一类，我的建议是三条：先设单笔上限和单日上限；每一笔都要有订单号可以对账；付款完成之后必须有一条你自己的复核动作。三条缺一，就别开。

其他：云存储、数据库、日历、浏览器

这一类我合在一起说，判断逻辑一样。

云存储类（各类网盘、对象存储）能列目录、传文件、下文件、生成分享链接。它替你决定不了文件该放哪个目录。授权时注意：只给一个专用目录，别给根目录。

数据库类能执行查询、读表结构。写操作和删表这类动作，我建议一律不开。授权时注意用只读账号。

日历类能读日程、建日程、改日程。它看不懂日程背后的优先级，这个判断在你手里。授权时注意它不能看到你的私人日程。

浏览器类能打开网页、截图、抓页面内容、填表单。它登不了需要短信验证码的系统。授权时注意：它在网页上的一切动作都是你的身份。

五类连接器：能做什么，红线在哪

每一类都看三件事：能做什么、不能做什么、授权注意什么

代码仓库类 GitHub

能读文件、目录结构、提交记录、分支与合并请求。读不到本地还没推上去的代码，也拿不到构建日志。只给当前项目那一个仓库，先只读

在线文档类

腾讯文档、金山文档都能接。能新建、写内容、改内容、读内容、按标题定位、写进表格。改不了权限设置，排版细节掌控力弱

邮件类 Agent Mail

能读收件箱、按条件搜索、读正文与附件、存草稿。管不了过滤条件和签名档，判断不了哪封该回。只给起草，不给发送

支付类

按产品公开说明，可以完成下单后的支付动作。它不能替你决定这笔钱该不该花。我到现在没开过可付款权限，一次都没有

其他：网盘 数据库 日历 浏览器

网盘列目录传文件；数据库读表结构；日历读建改日程；浏览器开网页、填表单。网盘只给专用目录，数据库走只读账号

每一行第三条才是重点：直接给整个账号全部写权限，是最常犯的错

图 19-3 五类连接器：能做什么、不能做什么、授权红线

图 19-3 作者根据公开资料绘制。

19.3 授权这一步要小心什么

授权等于给钥匙

授权这件事，很多人点得太快了。

你点下那个“同意授权”的按钮时，发生的事情其实很朴素：你给了对方一把钥匙。这把钥匙能开哪扇门、能开到什么程度，取决于你当时选的范围。

我给你一个判断框架，以后每次授权之前都过一遍这四问：

1. 这把钥匙能开哪扇门？是整个账号，还是某一个仓库、某一个文件夹、某一个目录？
2. 它能开门进去看，还是能进去改东西？只读和可写是完全不同的两件事。
3. 这把钥匙什么时候到期？有些授权是长期的，有些有明确有效期。你要知道是哪种。

4. 我怎么把钥匙收回来？授权一定有一个可以撤销的地方，你要先知道它在哪儿，再去点同意。

第四个问题最少有人问，也最要命。我见过同事换了岗位，半年前给出去的那条授权还挂着，那个服务还在读他原来那个项目的文档。授权之前先找到撤销入口，这五分钟花得很值。

授权前必过的四问

你点下同意的那个动作，等于把钥匙交给对方

- 第一问 这把钥匙能开哪扇门**
是整个账号，还是某一个仓库、某一个文件夹、某一个目录
- 第二问 能进去看，还是能进去改**
只读和可写是完全不同的两件事，风险差一个量级
- 第三问 这把钥匙什么时候到期**
有些授权是长期的，有些有明确有效期，令牌通常带时效
- 第四问 我怎么把钥匙收回来**
授权一定有一个可以撤销的地方，先找到它再去点同意

第四问最少有人问，也最要命：授权之前先找到撤销入口，这五分钟花得很值

图 19-4 一次授权的四问：开哪扇门、看还是改、什么时候到期、怎么收回

图 19-4 作者根据公开资料绘制。

权限分级建议表

我把权限分成四级。每一级我都给了真实例子和我的建议，你照着对号入座。

级别	它能干的事	真实例子	我的建议
只读	看，不能动	读代码仓库的提交记录；读文档内容；搜索邮件	默认开。风险最低，收益已经很大
可写	新建、修改、删除	往在线文档里写内容；在仓库里开分支和提交	一个一个单独开。先在小范围试，确认它写的靠谱再扩大
可发送	对外发出去，收不回来	发邮件；在群里推消息；在议题下评论	先开草稿，不开发送。跑顺三个月再考虑直发，且必须有收件人复核动作
可付款	钱出去	订单支付	我一次都没开过。要用就先设单笔上限、单日上限、订单号对账、事后复核

这张表怎么用？我给你一条简单的操作法：

第一次接任何一类服务，一律从只读开始。哪怕你心里知道最终一定要给写权限，也从只读开始。原因不是安全口号，是它很实用：只读阶段你能免费摸清这个连接器返回的东西长什么样、它的字段叫什么、它对不对得上你的预期。这些信息在你后面写提示词的时候全用得上。

跳过只读直接开可写，我吃过一次亏。那次我给了一个文档写权限，它写进去的内容里有一个字段名写错了，我事后才发现，而那个文档已经有同事在引用了。

三条我自己守的规矩

规矩一：专用账号。能给 AI 单独开一个服务账号的，就别用你自己的主账号。这样出了事，撤掉那一个账号就完了。

规矩二：专用目录。让 AI 写东西，就给它一个专门建的目录或者文件夹。别让它进你正在用的项目主目录。

规矩三：写进日历的复查日。每三个月复查一次所有授权，把不再用的撤掉。我把它写成一个季度提醒，很土，但有效。

19.4 连接失败怎么排查

连接器出错的时候，报错信息经常是英文的，而且指向很模糊。我把最常见的六类故障列成表，每一类给现象、原因、动作。

#	故障	你看到的现象	真正的原因	你要做的动作
1	授权过期	昨天还好好的，今天第一条指令就报权限相关错误	令牌有有效期，到期了	重新走一次授权流程。不要反复重试，重试只会继续失败
2	权限不足	读操作正常，一写到某一步就报错，报错里有 permission、scope 这类词	当初授权时选的范围不够，只给了读没给写	回到授权设置里把范围加上，然后重新授权一次，光改设置不重授权通常不生效
3	网络不通	卡住十几秒后报超时，或者报连接被拒绝	公司网络策略、代理、或者目标服务在当前网络下不可达	先用浏览器手动打开那个服务确认能访问。浏览器都打不开，连接器一定连不上
4	服务端限流	前几条指令正常，之后开始报 429 或者 rate limit	单位时间内的调用次数超了对方的上限	停下来等，把批量任务拆小分批做。不要靠疯狂重试去顶，重试会让你被限得更久
5	账号不对	它能连上，但返回的是空列表，或者返回的仓库/文档里没有你要的那个	授权用的是另一个账号，那个账号里没有你的东西	让它先报一句"你当前用的是哪个账号"，把账号名打出来核对
6	时间不对	报签名无效、时间戳过期这类信息，看着像密码错了	本机系统时间与标准时间差太多	同步本机时间。这一类在企业电脑上不算罕见

这六条里，我遇到过最多的是第 2 条和第 5 条，因为它们都不会明确告诉你"你搞错了"，只会给你一个含糊的报错。

一条通用排查顺序

别乱试。按这个顺序走，四步之内基本都能定位：

1. 先问它一句：你现在能列出这个服务上的什么？让它执行一个最简单的列举动作，比如列出仓库名、列出根目录。这一步能区分是"连不上"还是"连上了但看不到"。
2. 连不上就看网络。浏览器手动开一次那个服务。
3. 连上了但看不到，就看账号和范围。让它把当前账号名打出来。
4. 前面的都对，就看时间和限流。同步系统时间，等几分钟再试。

连接失败：四步之内定位

报错信息常常是英文的，而且指向很模糊



我遇到过最多的是第三步里的账号不对与范围不够，它们都不会明确告诉你搞错了

图 19-5 连接失败排查决策树：四步定位

图 19-5 作者根据公开资料绘制。

最后一句我要说重一点：任何一条故障，都不要靠"再试一次"解决三次以上。三次都失败，原因就不在运气了，停下来按表查。

场景 C1 让 AI 读你的代码仓库并汇报变更 ★★

一句话背景

每周一早上我要知道这个项目的代码上周动了什么：谁改了哪些文件、改了几处、有没有碰关键模块。人工做这活要打开代码托管页面，翻提交记录，边看边记，二十分钟起步，还容易漏。这个场景让 AI 自己进去读，回来给你一份带提交号的汇报。

名词小词典

代码仓库 (Repository)：存放代码的地方，记录着这个文件每一次被谁在什么时候改成什么样。类比：一本每页都留着修改痕迹的记事本。

提交 (Commit)：一次改动被存进仓库的动作，会生成一个唯一编号，叫提交号。类比：快递单号，凭它就能查到那一笔。

分支 (Branch)：从主线上分出去的一条独立开发线，改完了再合回来。类比：文档另存一份副本改，改完再并回去。

合并请求 (Pull Request)：请求把某个分支的改动合进主线，别人可以在上面评论。类比：把改好的稿子交给主编审阅。

提交记录 (Commit Log / History)：按时间排列的所有提交列表，每条含提交号、作者、时间、改动了哪些文件。

开工前准备

1. 一个代码仓库，你有它的访问权限（我用的是 GitHub 上的一个私有仓库）。
2. WorkBuddy 里装好代码仓库类的连接器，并完成授权。
3. 授权时只勾选你当前这个项目需要的那一个仓库（第一次建议只给读权限）。
4. 记下这个仓库的完整名字，格式是"拥有者 / 仓库名"这种两段式。
5. 定好你要汇报的时间范围，比如"上周一到上周日"。
6. 先做一次冒烟：让它列出它当前能看到的所有仓库名，确认你的仓库在列表里。

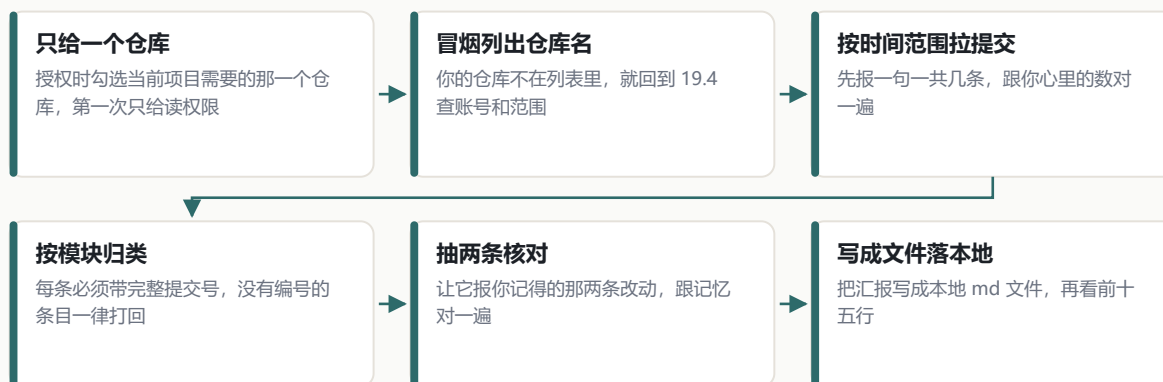
操作步骤

1. 新建一个对话，模式选 Agent。
2. 先做冒烟测试：让它列出当前授权能看到的所有仓库名。你的仓库不在列表里，就回到 19.4 查账号和范围。
3. 把下面的提示词整段复制进去，把仓库名和时间范围换成你自己的。
4. 它会先报它读到的提交条数。这个数字你要自己心里有个数：如果它报 3 条而你记得上周改了十几次，说明时间范围或者分支选错了。
5. 让它按模块归类，并且把每一条都带上提交号。没有提交号的汇报一律打回。

6. 抽两条你自己记得的改动，让它报出那两条的具体改动内容，跟你记忆对一遍。这一步是唯一能验证它真的读进去了的动作。
7. 让它把汇报写成文件存到本地目录。

场景 C1：读代码仓库做变更汇报

让 AI 自己进仓库读，回来给你一份带提交号的汇报



判断点：条数、每条都有提交号、它标出来的三条、抽两条核对内容

图 19-6 场景 C1 的链路：授权、冒烟、读记录、归类、抽查、落文件

图 19-6 作者根据公开资料绘制。

可复制提示词

整段复制，仓库名和时间范围换成你自己的：

【任务】读我的代码仓库，给我一份上周的变更汇报。

【仓库】拥有者 / 仓库名: eric-motor-lab / actuator-fw

【时间范围】2026-09-21 00:00 到 2026-09-27 23:59 (含两端)

【分支】main

【第一步：冒烟】先做一件事再往下走：列出你当前授权能看到的所有仓库名。
如果上面那个仓库不在列表里，立刻停下告诉我，不要继续。

【第二步：拉记录】按时间范围拉出 main 分支上的全部提交。
先只报一句话：这个时间范围内一共多少条提交。等我确认数字合理再往下。
我预期的量级是十几条。你报出来的数字如果是位数或者三位数，
先告诉我你用的时间范围和分支，等我确认。

【第三步：逐条列出】每条提交输出这五项，一行一条，不要合并：
提交号（完整）、作者、时间、改动文件数、改动摘要（不超过 30 字）。

【第四步：按模块归类】把上面的提交按目录或模块分组，每组给：
模块名、提交条数、涉及的文件、一句话说明这组改动在干什么。

【第五步：标出要我看的】从全部提交里挑出你认为我需要亲自过目的，
最多 3 条，每条写清楚：提交号、为什么你认为我需要看。
挑选标准：改动文件数最多的、改动涉及核心模块的、提交信息写得含糊的。

【纪律】

1. 每一条都必须带完整提交号。没有提交号的条目我不认。
2. 不许用“本周有一些常规改动”这类概括句，我要的是条目和数字。
3. 任何一步读不到数据，把原始报错原样贴给我，停下来等我指示，不许自己编内容补上。
4. 时间范围一律按我给的来，不许自行扩展到“最近”。

【输出】把第三到第五步的结果写成 C:\work\reports\repo-weekly-2026W39.md，
写完把文件内容的前 15 行贴给我。

过程与判断点

它应该先回一句冒烟结果，长这样：

- > [冒烟] 当前授权可见仓库: actuator-fw、actuator-hw、motor-testbench。
- > 目标仓库 actuator-fw 在列表内，继续。
- > [第二步] 该时间范围内共 14 条提交。

判断点一：那个条数。14 条这个数字你一定要自己核。去代码托管页面上看一眼上周的提交数，对不上就停下来查时间范围和分支。这一步五秒钟，但它决定了后面整份汇报的可信度。

判断点二：每一条都有提交号。如果它给你的汇报里出现“某同事改了配置”这种没有编号的条目，说明它在编。打回重做。

判断点三：它标出来的三条。这三条是整份汇报里最值钱的部分，也是最容易看出它有没有真读懂的部分。如果它挑出来的是三条无关紧要的改动，说明它的挑选标准没吃进去，你要在提示词里把标准写得更死。

判断点四：抽两条核对。你自己记得上周改过某个文件，让它报那两条的具体改动内容。它说得出来，说明它真的读进去了。说不出来，说明前面的汇报很可能是拼出来的。

翻车清单

翻车一：报出来的提交数是 0。症状是它说这个时间范围内没有提交。原因通常是时间范围写错，或者你看的是 main 分支而改动都在别的分支上。补救：先让它列出全部分支名，再让它分别报每个分支上的提交数。

翻车二：汇报看着很漂亮，但提交号是编的。症状是条目齐全、摘要通顺，你随手挑一个提交号去搜，搜不到。补救：验收动作必须是随机抽两条提交号去代码托管页面上真搜一次。搜不到，整份汇报作废。

翻车三：它把别的项目也报进来了。症状是汇报里出现了你不认识的文件名。原因多半是账号不对，授权挂在了别的账号上。补救：让它先报当前账号名（见 19.4 第 5 条），核对之后再重跑。

验收标准与变体

验收四条，三分钟做完：

1. 冒烟那一步的仓库列表里，你的仓库确实在。
2. 提交条数与你在代码托管页面上看到的数字一致。
3. 随机抽两条提交号，去页面上真搜，两条都能搜到且内容对得上。
4. 生成的那个 md 文件能打开，条目数与它口头报的一致。

变体一：给评审会准备材料。把时间范围换成"从上次发版到今天"，让它按模块归类并标出涉及核心模块的改动。评审会上你直接照着念。

变体二：跨仓库汇总。你手上三个仓库同时授权，让它一次拉三个仓库的记录，按人而不是按模块归类。这适合带团队的人看谁在干什么。

老兵说：汇报里每一条都得带提交号。带编号的汇报你能抽验，不带编号的汇报你只能选择信或不信，那不叫验收，叫赌。

场景 C2 让 AI 把结果写进在线文档 ★★

一句话背景

AI 在对话框里给你一段整理好的内容，然后你要自己打开在线文档、找到位置、粘贴、改格式，五分钟没了。更烦的是这种活每周来一次。这个场景让它直接写进文档，你只负责最后看一眼。

名词小词典

在线文档：存在云端、多人能同时打开编辑的文档，比如腾讯文档、金山文档。类比：一份贴在公共白板上、谁都能上去改的稿子。

文档空间 / 文件夹：在线文档里组织文件的方式。授权时常常是给整个空间还是只给一个文件夹的区别。

结构化写入：不是把一大段文字糊进去，而是按标题、小节、表格这些结构写进去。类比：往一张填好的表单里填格子，不是往白纸上泼墨。

幂等：同一个动作做一次和做三次，结果一样。写文档的时候这条很关键，重复跑不应该产生三份重复内容。类比：按电梯按钮，按三下和按一下，电梯只来一次。

版本历史：在线文档自带的改动记录，能看是谁在什么时候改了哪一段，也能退回。写坏了靠它救。

开工前准备

1. 一个在线文档账号（腾讯文档或金山文档），确认你能在浏览器里正常打开并编辑。
2. WorkBuddy 里装好对应连接器并完成授权。第一次只给一个测试文件夹的写权限。
3. 在测试文件夹里新建一个空文档，把它的链接或文档标识记下来。
4. 先想清楚你要写进去的内容结构：几个一级标题、每个下面几段。结构不定就让它替你定，然后你会得到一份你不想要的结构。
5. 确认你知道这个文档的版本历史入口在哪儿。写坏了要能退回去。
6. 冒烟：让它读出那个空文档当前的内容长度，应当报 0 或空。

操作步骤

1. 新建对话，模式选 Agent。
2. 冒烟测试：让它读目标文档当前的内容，确认它读得到且知道文档是空的。
3. 把提示词整段复制进去，把文档标识和要写的内容换成你自己的。
4. 先让它写一小段试水，比如只写第一个一级标题加一段。你打开文档亲眼看一下格式对不对。
5. 格式对就继续，让它把剩下的写完。格式不对，先把格式规矩改清楚再继续。
6. 全部写完后，让它回读一遍它写进去的内容，把结构大纲打印出来给你核。
7. 你自己打开文档看一遍。这一步不能省，哪怕是扫一眼。

场景 C2：把结果写进在线文档

让它直接写进文档的指定位置，你只负责最后看一眼



老兵说：先写一个标题加一段，你花三十秒看一眼，省掉重来一遍的二十分钟

图 19-7 场景 C2 的链路：冒烟、试水一段、确认格式、写完、回读核对

图 19-7 作者根据公开资料绘制。

可复制提示词

整段复制，文档标识和内容换成你自己的：

【任务】把一份周报写进我的在线文档。

【目标文档】腾讯文档，测试文件夹下，文档名：电机耐久试验周报-2026W39

【行为规矩】文档已存在且当前为空。你要做的是往里面写内容，不是新建一份同名文档。如果你看到已经存在同名文档但你不确定是不是这一个，立刻停下问我，不许自己新建。

【第一步：冒烟】先读这个文档当前的内容，报告它的字符数。如果它不是空的，把前 20 行贴给我，停下来等我指示。

【第二步：试水】只写第一个一级标题和它下面的第一段，写完停下。
回报：你写了几个字、一级标题用的是几级标题样式。
我打开文档确认格式之后会说继续，你再说下一句。

【第三步：写完剩余】我说继续之后，按下面的结构写完：
一级标题一：本周结论（3 到 5 条要点，每条一行，用无序列表）
一级标题二：试验进展（一段，不超过 200 字）
一级标题三：数据摘要（一张表，三列：项目、本周值、上周值）
一级标题四：下周计划（3 到 5 条，有序列表）

【数据来源】本周结论与数据摘要的数字，全部取自下面这段原文，不许自己补数字、不许改数字、不许四舍五入：
【原始内容粘贴处：把你的原始数据或上一步 AI 产出的内容贴到这里】

【格式规矩】

1. 一级标题用文档的一级标题样式，不许靠加粗来冒充标题。
2. 表格用文档的原生表格，不许靠敲空格排成表格的样子。
3. 全篇不许出现"暂无""待补充"这类占位词。某个数据真的没有，就写"本周未测"，并单独列出来告诉我。

【幂等规矩】这一步很重要：
如果你发现这个文档里已经存在"本周结论"这个一级标题，说明上一轮已经写过，不许再写一遍造成重复。
立刻停下告诉我：文档里已有哪些一级标题，等我决定是覆盖还是追加。

【回读】全部写完后，把文档的完整结构大纲读一遍回给我：每个一级标题、它下面有多少段或多少行。再报全篇字符数。

【纪律】任何一步失败，把原始报错原样贴给我，停下来等我指示。不许用"已写入""一切正常"这类话汇报。

过程与判断点

试水那一步它应该回得很短：

- › [冒烟] 文档当前为空，字符数 0。
- › [试水] 已写入一级标题"本周结论"及其下第一段，共 86 字。
- › 标题使用一级标题样式（不是加粗）。等待你确认。

这时候你必须真的打开文档看一眼。看三件事：标题是不是真的一级标题样式（字体明显大一号、能进目录），列表是不是真的列表（有项目符号），段落间距是不是正常。

判断点一：标题是不是真的标题。很多连接器写进去的标题其实只是加粗的普通文字。区别在哪儿？真的一级标题能进文档的目录，加粗不行。这一条你在文档里点一下目录就能看出来。

判断点二：表格是真表格还是敲空格排出来的。敲空格排出来的假表格，你在里面点一下就知道：光标能在格子里跳才是真表格。

判断点三：幂等有没有生效。你故意让它跑两遍，看文档里有没有出现两套"本周结论"。出现两套，说明幂等规矩没吃进去，这一条必须在提示词里再加一遍。我第一次跑的时候就是这里翻的车，文档里出现了两份一样的内容，我手动删了十分钟。

判断点四：数字有没有被改动。你给它的原始数字，它写进文档之后必须一模一样。挑两个数对一遍，重点是那些带小数的。它有的时候会"好心"帮你四舍五入。

翻车清单

翻车一：它新建了一份同名文档。症状是它汇报已写入，你打开原来的那份还是空的。原因是它没定位到你的文档，自己建了一个。补救：在提示词里把这条写成硬规矩（上面那段已经写了），并且让它写入之前先把文档标识报给你核对。

翻车二：重复写入，文档里出现两份。症状是同一个标题出现两次。补救：靠上面那段幂等规矩，并且跑第二遍之前让它先读一遍大纲。

翻车三：格式看着对，其实全是加粗冒充的。症状是文档里看着有标题，目录里一条都没有。补救：验收动作改为"打开文档目录看有没有条目"，别用肉眼判断字号。

验收标准与变体

验收五条：

1. 打开文档，目录里能出现你写的那几个一级标题。
2. 表格里的光标能在格子间跳，说明是真表格。
3. 挑两个原始数据里的数字，跟文档里的一字不差。
4. 全篇没有"暂无""待补充""待定"这类占位词。
5. 让它再跑一遍同样的指令，文档内容不重复、不翻倍。

变体一：每周固定写。把这段提示词存成技能（方法见第 4 章），每周一跑一次。数据部分改成让它自己从上一个场景的产出文件里读。

变体二：多人协作的会议纪要。会议结束后把原始记录丢给它，让它整理成"结论、待办、责任人、时间"四列写进文档。这时候验收重点是待办那列有没有责任人名字，空着就是没做对。

变体三：反过来读。让它读一份已经很长的在线文档，给你一份结构大纲加摘要。这是写之前必做的动作，能让你知道它的定位准不准。

老兵说：让它写文档之前，先让它写一个标题加一段。你花三十秒打开看一眼，省掉的是发现整篇格式全错之后重来一遍的二十分钟。

场景 C3 让 AI 帮你处理邮件的收与发 ★★

一句话背景

每天四十分钟耗在邮箱里：翻收件箱找那几封要回的，写回复，找附件，发。这个场景让 AI 帮你筛和起草，把四十分钟压到十分钟。但有一条规矩必须写死：它只能写到草稿，发送那一键永远在你手里。

名词小词典

收件箱 (Inbox)：收到的邮件默认进的那个箱子。类比：家门口的信箱。

草稿 (Draft)：写好但没发出去的邮件，存在邮箱里等你点发送。类比：写好了塞进信封但还没投进邮筒。

线程 (Thread)：一封邮件和它下面所有的来回回复，算一个线程。回复要在同一个线程里，对方才看得懂上下文。类比：微信里同一个人的一段连续对话。

附件 (Attachment)：随邮件一起发出的文件。附件发错版本是这类活最高频的事故。

搜索条件：用来筛邮件的规则，比如“发件人是谁”“主题含什么词”“哪天之后”。它决定 AI 能看到哪些邮件。

开工前准备

1. 一个可用的邮箱账号，WorkBuddy 里装好邮件类的连接器（例如 Agent Mail）。
2. 授权时只开读和起草，不开发送。这是本场景的硬前提。
3. 建一个专用文件夹或者标签，比如“AI 待处理”，让 AI 只在里面活动，不要碰你的整个收件箱。
4. 先把你要它处理的邮件范围写清楚：哪个发件人、哪个主题关键词、什么时间段。范围不写，它会把你整个收件箱翻个底朝天。
5. 准备三封你自己知道答案的邮件作为验收样本：你知道哪封该回、哪封不用回、哪封要转给别人。
6. 冒烟：让它列出它当前能看到的文件夹名。

操作步骤

1. 新建对话，模式选 Agent。
2. 冒烟测试：让它列出当前可见的邮箱文件夹名，确认“AI 待处理”在里面。

- 3. 把提示词整段复制进去，把范围和三封样本邮件换成你自己的。
- 4. 它先给你一张分类表：哪封要回、哪封不用回、哪封要转。你拿这张表跟你心里的答案对一遍。
- 5. 分类对上了，再让它起草。分类对不上，先修分类规矩，别急着起草。
- 6. 它把草稿写好后，把每一封草稿的全文贴给你。你逐封读完再决定发不发。
- 7. 你自己登录邮箱，打开草稿箱，检查收件人和附件，然后手动点发送。
- 8. 发完之后，让它把这次的处理记录写成一条日志。

场景 C3：只写到草稿，发送在你手里

它帮你筛和起草，发送那一键永远在你手里

- 1 授权只开读和起草**
不开发送权限，这是本场景的硬前提，也是根本解法
- 2 限定文件夹与时间段**
范围不写，它会把你的收件箱翻个底朝天
- 3 冒烟列出文件夹名**
确认 AI 待处理在它可见的文件夹列表里
- 4 先出一张分类表**
要回、不用回、要转三类，跟你心里的三封样本对一遍
- 5 只对要回的起草**
收件人不许加人减人，主题是原文前面加回复，正文不许寒暄
- 6 你读完草稿手动发送**
自己去草稿箱检查收件人和附件，再由它把处理记录写成日志

发出去就收不回来。授权层面不给发送权，比提示词里写一百遍都管用

图 19-8 场景 C3 的链路：限定范围、分类过审、起草、人工确认发送、写日志

图 19-8 作者根据公开资料绘制。

可复制提示词

整段复制，范围换成你自己的。里面那条"只准存草稿"的规矩，一个字都不许删。

【任务】帮我处理指定范围内的邮件，分类并起草回复。

【范围】只看"AI 待处理"这个文件夹里 2026-09-28 00:00 之后的邮件。
不许读其他文件夹，不许读更早的邮件。

【第一步：冒烟】先列出你当前可见的邮箱文件夹名，确认"AI 待处理"在里面。
不在就立刻停下告诉我。

【第二步：列清单】把范围内每一封邮件列成一行，字段：
序号、发件人、时间、主题（原样，不许概括）、附件数。
报完之后给我一句话：一共几封。

【第三步：分类】给每一封打一个标签，只能从这三个里选，不许自创：

- A. 需要我回复
- B. 看过了，不需要动作
- C. 需要转给其他人

分类表输出成三列：序号、标签、一句话理由。

打完表停下来等我确认，不许直接开始起草。

【分类规矩】

1. 只是抄送给我的，一律判为 B。
2. 发件人是系统自动通知（构建结果、流水线、订阅），一律判为 B。
3. 主题里带"请确认""请回复""截止"这类词的，判为 A。
4. 判断不了就判为 A 并在理由里写"判断不了"，不许猜。

【第四步：起草（等我确认分类之后才开始）】

只给标签为 A 的邮件起草回复。每一封独立起草，内容包括：

收件人（原发件人，不许改动、不许加人、不许减人）、

主题（原文前面加"回复："，不许重写）、

正文（150 字以内，先说结论再给细节，不许寒暄）。

【发送规矩】这条是硬规矩，违反即任务失败：

你只准把邮件存为草稿，绝对不许发送。

不许调用任何发送、回复、转发类的动作。

每一封草稿存好之后，把草稿的收件人、主题、正文全文贴给我，

等我逐封确认。我确认之后，由我自己在邮箱里手动点发送。

【附件规矩】

不许在回复里附加任何文件。需要附件的话，在正文里写一句

"（附件：文件名，待我手动添加）"，并把需要哪些附件列出来给我。

【第五步：日志】全部处理完之后，把这次结果写成

C:\work\reports\mail-log-2026-09-28.md，字段：

序号、主题、标签、是否起草、草稿编号、我的处理动作（我事后填）。

【纪律】

1. 任何一步读不到邮件，把原始报错原样贴给我，停下来等我指示。
2. 不许用"已处理完毕""邮件已发送"这类话汇报。
3. 你要时刻记住：这个任务里你没有发送权。

过程与判断点

第二步的清单应该长这样：

- > [冒烟] 可见文件夹：收件箱、AI 待处理、已发送、草稿箱、归档。
- > [第二步] 范围内共 7 封：
- > 1. 张工 / 09-28 09:12 / 执行器耐久试验第 3 轮数据确认 / 附件 2
- > 2. 项目群 / 09-28 10:40 / 周会纪要 / 附件 0
- > ...
- > [第三步] 分类：1-A、2-B、3-C、4-B、5-A、6-B、7-A

判断点一：它的范围守住了没有。它报的邮件数必须跟你放进那个文件夹的数量一致。多出来了，说明它读了别的文件夹，这是要立刻纠正的。

判断点二：分类表跟你心里的答案对不对得上。你准备的三封样本邮件，它的判断必须跟你的答案一致。三封里错两封以上，说明分类规矩写得不够死，重写分类规矩。

判断点三：草稿里有没有多余的收件人。这是这类活最要命事故。它起草的时候如果自作主张把抄送的人也加进了收件人，你一发送就出去了。逐封核对收件人这一动作，一次都不能省。

判断点四：它有没有偷偷发。任务结束后你自己去"已发送"文件夹看一眼，里面不应该出现任何一封它发的邮件。这一步是验收，不是不信任，是我跑了这么多趟之后的习惯。

翻车清单

翻车一：它把邮件直接发了出去。症状是你还没确认，同事已经收到了。原因是提示词里的发送规矩被它在执行中稀释掉了。补救：把发送规矩从段落里单独拎出来放在最前面，并且在授权层面直接不开发送权限（这才是根本解法，别只靠提示词）。

翻车二：附件发错版本。症状是对方收到的是上一版文件。补救：本场景的规则是回复里一律不带附件，需要附件就由你手动加。这条规矩看着笨，实际最稳。

翻车三：主题被它改写了。症状是对方收到一封主题跟原邮件对不上的回复，线程断掉。补救：在提示词里写死"原文前面加回复："，不许重写"，验收时逐封比对主题字符串。

验收标准与变体

验收六条，缺一条就不算过：

1. 它只碰了指定文件夹里的邮件，报出的数量跟你放进去的一致。
2. 分类表与你心里的答案在三封样本上完全一致。
3. 每一封草稿的收件人与原发件人一字不差，没有多加人。
4. 每一封草稿的主题是原主题加"回复："，没有被改写。

5. 你自己的"已发送"文件夹里，没有它发的任何一封。

6. 日志文件已生成，条目数与处理的邮件数一致。

变体一：只做分类不做起草。早上第一件事让它把收件箱过一遍，只给你一张"今天必须处理的三封"清单。起草留给你自己。这个变体最安全，适合刚上手。

变体二：批量归档。让它把三个月前的系统通知类邮件统一挪进归档文件夹，并且先给你一份清单让你过一遍。这一变体要给它写权限，我建议第一次只跑十封。

变体三：附件归集。让它把某个时间段内所有带附件的邮件的附件下一个本地目录，按"日期-发件人-文件名"命名。验收动作是数一遍文件个数，跟它报的对上。

老兵说：邮件发出去就收不回来，所以那一条"只准存草稿"不是保守，是唯一能让你睡得着的做法。授权层面不给发送权，比提示词里写一百遍都管用。

场景 C4 把多个连接器串成一条自动线 ★★

一句话背景

前面三个场景都是单点。真正省时间的是把几根线串起来：让 AI 去代码仓库读变更，把变更整理成一份汇报，写进在线文档的指定位置，最后把一封通知邮件存成草稿等你发。一次任务经过三个系统，全程二十分钟，你在关键节点只做三次确认。

名词小词典

串线：让一次任务按顺序经过多个系统，上一个系统的产物是下一个系统的输入。类比：一条流水线上三个工位，工件依次过。

人工确认点：流水线上你必须在场的那几个工位。串线不等于全自动，确认点的位置决定了这条线安不安全。

中间产物：每一步产出的东西，通常是本地的一个文件。它的作用是你能在中途检查，也让下一步有据可依。类比：每道工序的检验单。

失败回滚：某一步失败之后，前面已经做完的那些动作怎么处理。串线越长，这件事越要想清楚。

日志：整条线跑完的记录，每一步做了什么、成功还是失败、产物在哪个文件里。

开工前准备

1. 前三个场景各自跑通过一次。没跑通就别串，串起来查问题是噩梦。
2. 三个连接器都已授权：代码仓库（读）、在线文档（写，限测试文件夹）、邮件（读 + 起草）。
3. 一个本地工作目录，比如 `C:\work\chain`，用来放中间产物。目录路径不要有中文和空格。
4. 把三个系统的目标位置写清楚：哪个仓库、哪个文档、哪个邮箱文件夹。

5. 定好三个确认点分别在哪：我建议设在"条数确认""分类确认""发送确认"这三处。
6. 想清楚失败怎么办：我定的规矩是任何一步失败就整条停下，不许带着缺口往下走。

操作步骤

1. 新建对话，模式选 Agent。一个任务一个对话，不要复用之前的对话。
2. 先让它做三段冒烟：分别列出仓库名、文档字符数、邮箱文件夹名。三段全通过才往下。
3. 把提示词整段复制进去。
4. 第一站：它读仓库，报出提交条数。你核数字，说继续。
5. 第二站：它写文档，汇报结构大纲。你打开文档扫一眼，说继续。
6. 第三站：它存邮件草稿，把草稿全文贴给你。你读完，自己登录邮箱手动发送。
7. 最后让它把整条线的日志写出来，你通读一遍。

可复制提示词

整段复制，三个目标位置换成你自己的：

【任务】串一条线，按顺序经过三个系统，全程分三段，每段结束等我确认。

【三段顺序】

第一段：读代码仓库 → 产出汇报文件（本地）

第二段：读汇报文件 → 写进在线文档（云端）

第三段：读汇报文件 → 存一封通知邮件草稿（邮箱）

【目标位置】

仓库：eric-motor-lab / actuator-fw，分支 main

文档：腾讯文档，测试文件夹下，文档名：电机耐久试验周报-2026W39

邮箱文件夹：AI 待处理

本地目录：C:\work\chain

【开工前：三段冒烟】按顺序做完这三件再往下，任何一件失败立刻停下：

1. 列出你当前可见的仓库名，确认 actuator-fw 在里头。
2. 读目标文档当前字符数，并报告它是空还是非空。
3. 列出你当前可见的邮箱文件夹名，确认 AI 待处理 在里头。

三件都通过，把三条结果并列报给我，然后等待我说开始。

【第一段：读仓库】

拉 2026-09-21 到 2026-09-27 在 main 上的全部提交。

先只报总条数，等我确认。确认之后：

按模块归类，每条带完整提交号，挑出 3 条需要我亲自看的。

写成 C:\work\chain\step1-report.md，写完把前 15 行贴给我。

【停】写完停下，等我确认。

【第二段：写文档】

读 C:\work\chain\step1-report.md，按下面的结构写进目标文档：

一级标题一：本周结论（3 到 5 条要点）

一级标题二：变更明细（表：模块、提交数、涉及文件）

一级标题三：需要确认的（3 条，每条带提交号）

格式规矩：一级标题用文档标题样式不许加粗冒充；表格用原生表格。

幂等规矩：如果文档里已存在“本周结论”这个标题，立刻停下告诉我，不许重复写。

数字规矩：所有数字照抄 step1-report.md，不许改、不许补、不许四舍五入。

【停】写完停下，汇报结构大纲和全篇字符数，等我确认。

【第三段：存邮件草稿】

读 C:\work\chain\step1-report.md，起草一封通知邮件：

收件人：team-motor@example.com（这一条你换成你自己的真实收件人）

主题：电机耐久试验周报 2026W39

正文：150 字以内，只写结论和“明细见在线文档”这句话，不许贴大段明细。

【发送规矩：硬规矩】只准存为草稿，绝对不许发送，不许调用任何发送类动作。

草稿存好后把收件人、主题、正文全文贴给我，等我确认。

【停】停下，等我在邮箱里手动发送。

【全程纪律】

1. 三段之间不许跳步，不许合并成一步做完。
2. 任何一段失败，立刻停下，把原始报错原样贴给我，不许带着缺口往下走，

不许自己编内容补上，不许跳过这一段去执行下一段。

3. 每一段的产物必须落到 `C:\work\chain` 下，不许只留在对话里。
4. 不许用“已完成”“一切正常”汇报，每段回报必须带数字或文件行数。

【收尾】三段全部完成后，把整条线的日志写成
`C:\work\chain\chain-log.md`，字段：段号、动作、产物文件、成功与否、
 我确认的时间点（我填）、耗时。

过程与判断点

三段冒烟通过之后，它应该并排报三条：

- > [冒烟 1/3] 可见仓库：actuator-fw、actuator-hw。目标在列。
- > [冒烟 2/3] 目标文档当前字符数 0，为空。
- > [冒烟 3/3] 可见文件夹：收件箱、AI 待处理、草稿箱。目标在列。
- > 三段冒烟全通过，等待你说开始。

判断点一：冒烟三段全过才开工。这一条看着啰嗦，实际是整条线唯一能提前发现问题的地方。我在没做冒烟的那次，第二段才发现文档授权过期了，第一段白跑。

判断点二：第一段的条数。跟场景 C1 一样，自己核。

判断点三：第二段写完，你一定要打开文档看。这是三站里最容易静默出错的一站，因为它会回报“已写入”，而格式可能全错。

判断点四：第三段的收件人。串线跑顺了之后，人最容易在这一站松懈。收件人一个字母都不能错。

判断点五：日志里的三个产物文件是否真的存在。打开 `C:\work\chain` 看一眼，应该有三个文件。它说写了但文件系统里没有，是最典型的假成功。

翻车清单

翻车一：它把三段合成一段做完了。症状是它一口气跑到底，你还没确认它就报“全部完成”。原因是提示词里的停点不够硬。补救：把每一段的【停】单独成行加粗，并在开头就写“三段之间不许合并”。

翻车二：中间产物没落地，第二段读了个空文件。症状是第二段报“读到的内容为空”，然后它开始自己编一段内容写进文档。补救：在纪律里写死“读不到内容就停下，不许自己编”，并且你在每段之间亲自看一眼那个文件在不在。

翻车三：邮件那一段它顺手发了。症状是同事收到了邮件而你还没确认。补救：根本解法在授权层，不给发送权限。提示词只是第二道防线。

验收标准与变体

验收七条：

1. 三段冒烟全部通过才开始执行。
2. 三个产物文件都在 `C:\work\chain` 下真实存在，能打开。

3. 第一段汇报里的提交号随机抽两条能搜到。
4. 在线文档的目录里能出现那三个一级标题。
5. 文档里的数字与 step1-report.md 一字不差。
6. 你自己的"已发送"文件夹里没有它发的邮件，只有你手动发的那一封。
7. 日志文件三段齐全，每一段都有产物文件名。

变体一：把试验数据接进来。第一段改成读试验数据目录而不是代码仓库，后面两段不变。这是试验工程师最常用的一条线。

变体二：通知改成发到群里而不是邮件。第三段换成消息类连接器，同样只给起草或者只给"生成待发送内容"，发送仍由你手动完成。

变体三：加一段翻查。在第一段之后加一步，让它把本周的变更与上周的汇报对比，标出"上周说要、本周没做"的条目。这一段的验收动作是条目数与你自己的待办清单对得上。

老兵说：串线之前请确认每一段单独都跑通过。三段里有一段没跑通就串，你查问题的时候分不清是线的毛病还是那一段的毛病，那种感觉比手工干一遍难受十倍。

小白词典

连接器：让 AI 接上某一个外部服务（邮箱、文档、代码仓库）的那根线。类比：插在主机上的外设线，一根线接一个设备。

授权：你点"同意"把钥匙交给对方的那个动作。类比：把家门钥匙配一把给别人，并且你要先想清楚这把能开哪扇门。

令牌 (Token)：授权成功之后对方拿到的那串凭证，它代替你的密码去访问服务，而且通常有有效期。类比：一张临时门禁卡，过期就刷不开。

权限范围 (Scope)：这把钥匙能开到什么程度，只读、可写、可发送是三种不同的范围。类比：门禁卡能进大门，还是能进大门加机房。

只读 / 可写：只读是能看不能改；可写是能新建、能改、能删。两者风险差一个量级。类比：能进图书馆看书，和能进图书馆改书上的字。

OAuth：目前最主流的一套授权协议，特点是你不用把密码告诉对方，对方拿到的是一个有时限的令牌。类比：酒店前台给你房卡，你没把身份证押给他。

API：服务对外开的一组标准调用方式，连接器本质上就是在调这些 API。类比：餐厅给服务员下单用的那套标准用语。

限流 (Rate Limit)：服务方规定单位时间内最多允许你调用多少次，超了就拒绝。类比：自助餐限时两小时，不是不让你吃，是不让你一次吃完。

代码仓库：存放代码并记录每一次改动的地方。类比：一本每页都留着修改痕迹的记事本。

提交号：一次改动存进仓库时生成的唯一编号。类比：快递单号，凭它能查到那一笔。

分支：从主线上分出去的一条独立开发线。类比：文档另存一份副本改，改完再并回去。

草稿：写好但没发出的邮件。类比：写好了塞进信封但还没投进邮筒。

线程：一封邮件和它下面所有来回回复组成的连续对话。类比：同一个人发来的一串连续消息。

幂等：同一个动作做一次和做三次，结果一样。类比：电梯按钮按三下，电梯只来一次。

版本历史：在线文档自带的改动记录，写坏了能靠它退回。类比：游戏里的存档点。

串线：让一次任务按顺序经过多个系统，上一步的产物是下一步的输入。类比：流水线上三个工位依次加工。

人工确认点：串线上你必须在场的那几站。类比：质检盖章的地方，章得人盖。

中间产物：每一步产出的文件，落在本地，方便你中途检查。类比：每道工序的检验单。

你的岗位怎么用

设计工程师：从场景 C1 起步。你手上如果有脚本、参数表或者仿真配置放在代码仓库里，让它每周给你一份变更汇报，比你自己翻页面快。第一次只给读权限，跑一个月再考虑别的。

试验工程师：场景 C4 的变体一是最适合你的一条线。第一段改成读试验数据目录，让它把本周数据整理成汇报写进在线文档，通知邮件存草稿。你的确认点设在"数据有没有读全"和"数字有没有被改"。

质量工程师：你的重点在 19.3 那张权限分级表和 19.4 那张排查表。每一条授权都该按你管供应商的思路登记：给谁、给到哪一级、何时到期、谁负责撤。

项目经理：场景 C2 的会议纪要变体最直接。会后把原始记录丢给它，产出"结论、待办、责任人、时间"四列写进在线文档。你的验收重点是待办那列有没有责任人名字。

采购与供应商管理：19.3 里的可付款那一级别碰。你真正能用的是文档类连接器的只读部分：让它定期读供应商提交的在线表格，把异常条目挑出来给你看。

行政与人事：场景 C3 的变体一最安全。让它每天早上把收件箱过一遍，只给你一张"今天必须处理的三封"，起草留给你自己。

本章交付物

一张属于你自己的连接器授权清单，外加三张可打印的卡片。

具体做法，今晚就能做完：

1. 新建一份表格，列出你现在已经给出去的每一条授权，字段写五个：服务名、能碰的范围、权限级别（按 19.3 那四级填）、授权时间、撤销入口在哪儿。
2. 把 19.3 那张权限分级表打印出来贴在显示器边上。以后每开一条新授权，先对着表定级别。
3. 把 19.4 那张六类故障表存成一份文档，报错的时候照着查，别靠重试。
4. 找一条你从没用过的授权，现在就把它撤掉，练一遍撤销动作。你练过一次撤销，才敢开下一条授权。

第一件验收的活：挑一个你最常用的系统，只开只读权限，跑一遍场景 C1 或 C2。跑通那天，你手上就有了第一根真正接上的线。

本章金句

大多数人说 AI 干不了自己的活，其实是它够不着自己的系统。差的不是智力，是一根插头。

授权之前先找到撤销入口。这五分钟，比事后追回一封错发的邮件便宜一百倍。

第一次接任何服务，一律从只读开始，哪怕你心里知道最后一定要给写权限。

带提交号的汇报你能抽验，不带编号的汇报你只能选择信或不信。那不叫验收，叫赌。

邮件发出去就收不回来。授权层面不给发送权，比提示词里写一百遍都管用。

串线之前请确认每一段单独都跑通过。分段能查的错，串起来就是噩梦。

（下一章：MCP。连接器是厂家做好的成品插头，MCP 是墙上那个标准插座：下一章我们讲怎么自己做插头，把 connectors 覆盖不到的那些专业软件接进来。）

第 20 章

MCP：给 AI 装插座 ★★★

AI 会不会用你的软件，跟它聪不聪明没关系，跟墙上有没有插座有关系。MCP 就是那个插座的标准。

我第一次听说 MCP 是今年八月底，在一个技术群里。有人发了句"装上 MCP 之后 AI 就能直接开 SOLIDWORKS 了"，底下跟了一串英文缩写：MCP、JSON-RPC、stdio、transport、server、tool、schema。

我盯着那串词看了三分钟，一个都没认全。我做微电机和智能执行器研发十一年，跟电机、齿轮、霍尔信号、CAN 报文打了半辈子交道，英文缩写我不怵。可这一串缩写放在一起，我一句也读不懂。

那天晚上我干了一件挺笨的事。我把那串词一个一个抄在纸上，然后去查每一个词是什么意思。查到第三个的时候我卡住了：服务端（server）这个词我知道，公司机房里那台嗡嗡响的机器就是服务器。可这里的"服务端"这两个字，指的是我自己电脑上跑的一个小程序，跟机房里那台嗡嗡响的机器没关系。这两件事有什么关系？

真正让我转过弯来的，是我家书房那个插线板。

我书桌上有一台显示器、一台台式机、一盏台灯、一个手机充电器、一台打印机。它们来自五个不同的厂家，插头形状各不相同，但它们都能插进同一个插线板。插线板不认识打印机，也不需要认识。它只管一件事：把电送过去。厂家管另一件事：把电变成打印动作。中间那层"谁都能插"的规矩，就是标准。

MCP 就是 AI 世界里那个插线板的规矩。AI 是那台需要用电的设备，SOLIDWORKS、Blender、数据库、浏览器、你电脑上的文件夹，都是各种各样的电器。在没有插座的年代，你要让 AI 碰一样东西，就得为这样东西专门写一套接线。有了插座，电器那边只要按规矩做一个插头，AI 那边只要按规矩留一个插孔，两边谁也不用认识谁。

想通这一点之后，那串英文缩写就不再是障碍了。它们只是这套插座规矩里的零件名：客户端是插头那一侧，服务端是电器那一侧，工具是电器上一一个个具体的按钮，JSON-RPC 是插头和插座之间说话的格式，stdio 是它们说话用的那根线。

第 10 章我演示过 AI 在 SOLIDWORKS 里从零建出一台方向盘，二十三个零件、六张工程图、干涉零对。那一章讲的是它在软件里按下了什么。这一章讲的是那双手是怎么接上去的，以及你自己怎么插一个。

这一章不难，但它是全书最容易卡住的一章。卡住的原因是它的术语最抽象，一个界面也没有，你敲完配置看不到任何画面变化。我把自己当初卡住的地方全部原样写下来，每个词第一次出现就当场解释。

20.1 MCP 到底是什么：一张对比表，一张三层图

先把一句话钉死：MCP（Model Context Protocol）是一套规矩，不是一款软件。它规定了两件事：AI 这边怎么开口要工具，软件那边怎么把自己能干的事报上来。规矩定完，两边各自实现，谁也不用管对方内部怎么做的。

没有 MCP 的时候是什么样子

没有这套规矩的年代，想让 AI 碰一个软件，只有一条路：为这个软件单独写一套接线。

我在第 10 章提到的那条链路，最初就是这么来的。要用 AI 驱动 SOLIDWORKS，得有人专门写一段程序，把 AI 说的话翻译成 SOLIDWORKS 的 COM 调用。这套线写好了，只能接 SOLIDWORKS。明天你想让它碰 Blender，对不起，重新写一套。后天想让它读数据库，再写一套。

十款软件就是十套接线，十套接线有十种写法、十种报错格式、十种权限模型。这就是为什么早两年 AI tooling 看着热闹，落到工程现场能用的没几个。

对比项	没有 MCP	有 MCP
接一个新软件	写一套专用接线	软件侧按规矩做个服务端
AI 侧的改造	每接一样东西都要改一次	改一次，以后都一样
工具清单从哪来	写死在代码里	服务起来后自己报上来
换个 AI 工具	接线可能要重写	配置搬过去就能用
报错长什么样	每套线一个格式	统一格式，机器能读
出问题查谁	查那套专属代码	查服务、查配置、查权限三段

表里有两行最值钱。第三行"工具清单从哪来"，是从写死变成自报。这是本质区别：写死的意思是 AI 那边必须预先知道你会什么，自报的意思是 AI 连上来问一句你就会什么，你新加一个功能它当天就能看见。

第五行"报错长什么样"，对排查的意义被严重低估。统一格式意味着报错可以被机器解析，可以被归类，可以做成排查表。本章 20.7 那八条故障能列出来，靠的就是这个。

三层结构：插头、电器、按钮

MCP 的结构只有三层，我用一个比喻讲完。

十套专用接线，还是一个通用插座

同一件事，两种做法，差别在这六行

没有 MCP	有 MCP
- 接一个新软件 为这个软件单独写一套专用接线 - AI 侧的改造 每接一样东西都要改一次 - 工具清单从哪来 写死在代码里，AI 必须先知道你会什么 - 换个 AI 工具 接线可能要重写 - 报错长什么样 每套线一个格式 - 出问题查谁 只能查那套专属代码	- 接一个新软件 软件侧按规矩做个服务端 - AI 侧的改造 改一次，以后都一样 - 工具清单从哪来 服务起来后自己报上来，新功能当天可见 - 换个 AI 工具 配置搬过去就能用 - 报错长什么样 统一格式，机器能读 - 出问题查谁 查服务、查配置、查权限三段

自报而不是写死，是本质区别；统一报错格式让八条排查表列得出来

图 20-1 没有 MCP 与有 MCP：十套专用接线对比一个通用插座

图 20-1 作者根据公开资料绘制。

第一层：客户端（client）。就是 AI 那一侧，也就是你正在用的那个工具。它的职责是把 AI 说的话变成一条符合规矩的请求，发出去，等回包，把回包原样交给 AI 看。它不需要知道 SOLIDWORKS 是干什么的，它只知道“我要调一个叫 extrude_feature 的工具，参数是这些”。

第二层：服务端（server）。就是你要接的那个软件那一侧，一个跑在你本机上的小程序。它的职责有两件：一是开机时把自己有哪些工具报给客户端，二是收到调用请求时，把请求翻译成软件真正能听懂的动作。

第三层：工具（tool）。服务端上报的每一个具体动作。一个工具就是一件事，比如“新建一个零件”“拉伸 12 毫米”“读取当前文档的体积”。

这三层的关系，用我书桌那个插线板讲就是：客户端是插头，服务端是电器和它自带的那个电源适配器，工具是电器面板上的按钮。插头不认识按钮，它只管把电送过去；按钮不认识插头，它只管按下去之后干该干的活。

MCP 三层：插头、电器、按钮

客户端发请求，服务端做翻译，工具真正动手

第一层 客户端 client

插头那一侧。把 AI 的话变成一条符合规矩的请求，发出去等回包

第二层 服务端 server

电器那一侧。开机时把自己有哪些工具报给客户端，收到调用时翻译成软件能听的动作

第三层 工具 tool

电器面板上的按钮。一个工具就是一件事，比如新建一个零件、读体积

插头不认识按钮，按钮也不认识插头，两边各自按规矩做事

图 20-2 MCP 三层结构：客户端发请求，服务端做翻译，工具真正动手

图 20-2 作者根据公开资料绘制。

一套真实链路长什么样

本机上我跑通的那条链路，是四段，比上面说的三层还多一段。这一段是真实配置，不是示意图：

Python MCP 适配器 (FastMCP, 46 个工具) → HTTP → C# 执行层 (本机 5000 端口) → SOLIDWORKS COM

拆开看：

- Python MCP 适配器是对着 MCP 规矩写的那个服务端，它对外报出 46 个工具。建草图、拉伸、切除、倒圆角、阵列、配合、保存、导出 STEP，全在这 46 个里。
- 中间那层 HTTP 是它自己的选择。MCP 没规定服务端内部怎么实现，它只规定对外怎么说。这条链路的作者选择让 Python 侧和一个 C# 写的执行层之间走本机 HTTP 通信。
- C# 执行层负责真正去敲 SOLIDWORKS 的门。为什么中间要插一层 C#？因为 SOLIDWORKS 对外开的那道门叫 COM，用 C# 去敲比用 Python 稳。
- SOLIDWORKS COM 是最后一段，是软件厂商自己提供的对外接口。这一段不属于 MCP，它早就在那儿了，MCP 只是让 AI 的手能够到它。

理解这一段有个很实用的好处：排查的时候你能一次定位到段。工具调不通，先问是哪一段断了。我在 20.7 里给的八条排查，本质就是按段定位。

顺带说一句我一直觉得挺有意思的地方：这条链路里，MCP 只占了最外面那一段。真正让软件动起来的，是 COM 那道门，那道门 SOLIDWORKS 十几年前就开了。MCP 干的事只有一件，把软件已经存在的能力，用一个统一的方式递到 AI 手上。

三个最容易搞混的认识

第一，MCP 不会让 AI 变聪明。它给 AI 的是手，不是脑子。装完 MCP 之后，AI 对几何、对公差、对你公司的规矩，理解水平一点没变。它只是从"只能跟你说怎么做"变成"能真的去做"。第 10 章里它建的那台方向盘之所以能验收，靠的不是 MCP，靠的是那套九步流程里每一步都要回报数字。

第二，MCP 不保证软件愿意被驱动。插座接上了，电器得自己有开关。一个软件如果没有对外接口，MCP 也拿它没办法。SOLIDWORKS 能被驱动是因为它有 COM，Blender 能被驱动是因为它有 Python 接口。判断一个软件能不能接，先去查它有没有对外接口。

第三，MCP 不等于联网。绝大多数人第一次听"服务端"三个字，会以为要连到某台服务器上。本机这一类的服务端就跑在你自己的电脑上，走 stdio 或本机端口，数据不出本机。这也是我敢用它碰工程图纸的前提。

老兵说：MCP 不生产能力，它生产"接口一致性"。软件本来就能被程序驱动，MCP 只是让所有软件都用同一种姿势被驱动。

20.2 它跟连接器、技能是什么关系

这是读者问得最多的一个问题，也是最容易混的三个词。先把结论钉死：连接器接的是外部在线服务，MCP 接的是你本机和专业软件，技能是一套做事的方法。这三者处在不同层次，谈不上谁替代谁。

维度	连接器 (Connector)	MCP 服务端	技能 (Skill)
它接的是谁	外部在线服务	本机资源与专业软件	不接东西，它本身就是方法
跑在哪	云端那边	你这台电脑上	你本机的一份说明文件
典型例子	日历、云盘、工单系统	SOLIDWORKS、Blender、本地数据库	一个建模作业流程、一份体检清单
谁能做	通常由服务方提供	谁都能写，开源社区最多	你自己就能写，写一份 Markdown
给出去的权限	账号里的一部分数据权限	可能包含本机文件与进程	不给权限，它只教 AI 怎么干
出问题时查哪儿	授权是否过期	进程、路径、端口、软件是否开着	说明写得清不清楚
一句话定义	替你去别人的系统里办事	让 AI 的手伸到你本机上	教 AI 按什么顺序办事

三者怎么配合

我把它们的关系理解成三层：连接器给 AI 一个身份，MCP 给 AI 一双手，技能给 AI 一份作业指导书。举个我自己跑过的例子。我要让 AI 建一个零件并出图。

- 连接器那一层：从云盘上把公司的图纸模板拉下来。这是去别人的系统里办事，需要身份。
- MCP 那一层：驱动本机 SOLIDWORKS 真正把零件建出来并导出 STEP。这是伸到本机上动手。
- 技能那一层：我写好的那份"九步作业单加六行体检查验卡"（第 10 章场景 20 与场景 22），告诉它按什么顺序做、每一步要回报什么数字。

三层各干各的，缺一层这活都干不完整。没有连接器，模板得我自己下载；没有 MCP，它只能给我一份文字方案，动不了软件；没有技能，它会按自己的习惯乱做一通，做出来的东西我验不了。



图 20-3 连接器、MCP、技能的三角分工：身份、双手、作业指导书

图 20-3 作者根据公开资料绘制。

一条判断规则

碰到"这件事该用哪个"的时候，用这条规则判断，十秒钟能定下来：

这事要不要碰你本机上的东西？要，走 MCP。不要，但需要一个外部账号身份，走连接器。都不要，只是想让它按规矩做事，写个技能。

反过来还有一条更实用的：能用技能解决的，别开 MCP。技能只是一份说明文件，它不给 AI 任何权限，出不了安全事故。MCP 一开，AI 的手就真伸到你电脑上了。能用嘴教會的活，别给它手。

这条规矩我自己踩过坑。早期我给 AI 挂了一堆 MCP 服务，觉得挂得越多越厉害。后来发现其中三个服务半年里一次没被用上，白白占着三个能碰本机的权限口子。我全删了，一个功能都没丢。

一个能看清三者差别的例子

九月底我要做一件小事：把一台微电机执行器的八个零件建出来，出一份 BOM 表，再把结果同步到团队的共享日历里。

这三件事正好对应三个层次。建零件要靠 MCP，因为它必须伸手进 SOLIDWORKS；出 BOM 表靠技能，因为那只是把一套格式和顺序教给它，而它在建模过程中已经知道有哪些零件；同步日历靠连接器，因为那是别人系统里的东西，得有个身份才进得去。

我想强调的是中间那件。BOM 表这件事我最早是想再挂一个“表格处理”的 MCP 服务去做的，后来发现根本没必要：AI 已经知道有哪些零件，我只是需要它按我的格式输出。写成一段技能说明，两分钟搞定，而且零权限风险。

那次之后我给自己定了个顺序：先想能不能写成技能，再想要不要装 MCP。这个顺序替我省掉了至少两次装服务、两次排查、两次清理。

20.3 装一个现成的 MCP 服务：完整演示 ★★★

这一节是本章最该抄的一节。我按“抄着就能跑通”的标准写，每一步都告诉你做完该看到什么。本节标 ★★★，本机跑过多台不同类型的服务。

开工前准备

装之前先把这五条备齐，少一条后面必卡：

1. 一台 Windows 电脑，你要接的那个软件已经装好并且能手动打开。
2. 一个能跑起来服务端的运行时。Python 侧要 Python 3.10 以上，Node.js 侧要 Node.js 18 以上。
命令行里敲 `python --version` 或 `node --version` 能有回显，这一条就算过了。
3. 那个 MCP 服务端本身。一般是一个开源项目，装法通常是 `pip install 包名` 或 `npx 包名`。
4. 一个你打算让它碰的目录，路径全英文、无空格、带盘符，比如 `D:/AI-CAD`。
5. 十分钟不被打断的时间。装 MCP 这件事最忌讳半途中断，因为它的报错多数不带画面，你回头再来一遍会想不起刚才做到哪一步。

六步走完

装一个 MCP 服务的六步

每一步都告诉你做完该看到什么

- 1 命令行里单独跑服务端**
它只会静静停在那里等输入，这是正常的，不是卡住了
- 2 写一段 JSON 配置**
登记 command 与 args，路径写成 D:/AI-CAD 这种正斜杠
- 3 完全退出再重启客户端**
不是关窗口，是连后台进程一起退，否则配置不生效
- 4 验握手：列出工具清单**
回得出来，并且数量与项目说明里的对得上
- 5 把要接的软件手动打开**
本机类的服务端是附着到已运行的软件进程上的，不是帮你启动
- 6 跑一个小十倍的任务**
三十秒能验完，摸清回包、报错和耗时这三件事再上真活

口诀：看不到工具列表就提需求，等于闭着眼睛上机床

图 20-4 装一个 MCP 服务的六步：备运行时、装服务端、写配置、重启、验握手、跑最小任务

图 20-4 作者根据公开资料绘制。

第一步：先在命令行里把服务端单独跑起来。

这一步最容易被跳过，也最不该跳过。在配置进工具之前，先确认这个服务端自己能启动。

```
# Python 侧的服务端，通常是这样启动的
python -m solidworks_mcp --host 127.0.0.1 --port 8765

# Node.js 侧的服务端，通常是这样
npx -y @modelcontextprotocol/server-filesystem D:/AI-CAD
```

跑起来之后，它不会输出一大堆东西，它只会静静地停在那里等输入。这是正常的，不是卡住了。看到光标停住不动，说明服务端已经就位，可以进下一步。反过来，如果它噼里啪啦打出一串英文然后退出，那就是报错，先把这串英文解决掉再往下走。

上面这两条命令里，`solidworks_mcp` 与 `@modelcontextprotocol/server-filesystem` 是两类服务的典型启动形态，具体的包名、参数名以你采用的那个服务项目的说明为准。

第二步：写配置。

MCP 的配置通常是一段 JSON，登记在一处配置文件里。配置文件的具体位置与字段名，以你安装的客户端版本为准，下面给的是通用形态，可以直接照抄改：

```
{
  "mcpServers": {
    "solidworks": {
      "command": "python",
      "args": [
        "-m",
        "solidworks_mcp",
        "--host",
        "127.0.0.1",
        "--port",
        "8765"
      ],
      "env": {
        "PYTHONIOENCODING": "utf-8"
      },
      "disabled": false
    }
  }
}
```

这一段里每一行的意思，我逐条说清楚，因为绝大多数装不上就是这几行里某一行错了：

字段	意思	最容易错的地方
mcpServers	所有服务登记在这下面	大小写写错，写成 <code>mcpservers</code>
solidworks	你给这台服务起的名字	名字里带空格或中文，后面调用会找不到
command	用哪个程序去启动它	写 <code>python</code> 但系统里只有 <code>python3</code>
args	传给它的启动参数	参数顺序写错，或者漏了某一项
env	给这个进程设的环境变量	中文路径乱码，靠它设 UTF-8 救回来
disabled	这台服务要不要启用	排查期间设 <code>true</code> 临时停用，排查完忘了改回来

再给一个最简单的形态，适合那种不需要参数、stdin 直接说话的服务：

```
{
  "mcpServers": {
    "demo": {
      "command": "python",
      "args": ["D:/mcp-demo/demo_server.py"]
    }
  }
}
```

注意 `args` 里的路径用了正斜杠 `/`。Windows 上路径习惯写反斜杠，但在 JSON 里反斜杠是转义字符，`D:\mcp-demo` 会被读成一个乱七八糟的字符串。要么写成 `D:/mcp-demo`，要么写成 `D:\\mcp-demo`。这是我踩过的第二个坑，也是 20.7 里第六条故障的根源。

第三步：重启客户端，让配置生效。

多数客户端不会热加载配置。改完配置必须完全退出再重开，不是关窗口，是连后台进程一起退。这一条看着最不起眼，我在群里见过至少五个人卡在这一步，反复改配置反复不生效，最后发现是没重启。

第四步：验握手。

这是最关键的一步，也是唯一一步能给你确定答案的。新建一个对话，直接问一句：

列出当前所有已挂载的 MCP 服务端，以及每台服务端上可用的工具名称清单。
不要调用任何工具，只把清单列出来。

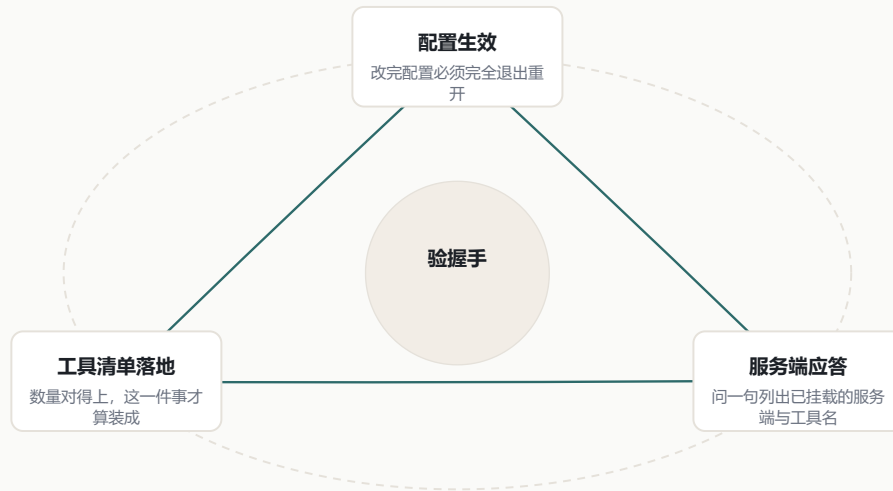
正常的话，你会拿到一份清单，长这样：

```
> 已挂载服务端: 1 台
> solidworks (46 个工具)
>   ensure_ready / open_new_part / create_sketch / add_sketch_entities
>   extrude_feature / add_edge_feature / analyze_model / save_document
>   ... (其余省略)
```

看到工具列表，这一件事就算装成了。看不到，先别提任何业务需求，回头按 20.7 排查。我在第 10 章里写过同样一句话，那句话是那一整章里最省钱的一句：看不到工具列表就提建模需求，等于闭着眼睛上机床。

验握手的三段回路

客户端问一句，服务端把工具清单报上来



握手就是连接建立时双方先确认在不在、会什么，一问一答靠编号对上号

图 20-5 验握手的三段回路：配置生效、服务端应答、工具清单落地

图 20-5 作者根据公开资料绘制。

第五步：把要接的软件先手动打开。

这一条是本机类专业软件特有的。SOLIDWORKS 这类软件，绝大多数 MCP 服务端是“附着”到已经运行的软件进程上的，不是帮你把软件启动起来。软件没开，服务端就算起来了也连不上。

我本机那条链路的做法是：先用一句话探健康状态。

调用 `ensure_ready`，把返回的完整内容原样贴给我，不要概括。

它应当回一句类似 `server=UP | com_attached=True` 的话。这里有两个字段，两个都得对：`server=UP` 说明服务端进程活着，`com_attached=True` 说明它已经贴到 SOLIDWORKS 上了。只看到 `server=UP` 就开工，是最常见的一次白跑。`com_attached=False` 的意思是软件那头没开，或者开了但执行层没起来。

第六步：跑一个比你真实需求小十倍的任务。

不要一上来就提真需求。先给一个三十秒能验完的小活，看它的返回值长什么样。

调用 `open_new_part` 新建一个零件，然后把返回的完整内容原样贴给我。
不要做别的动作。

这一步的意义是：你在零风险的情况下，摸清了这个服务端回包长什么样、报错长什么样、一个动作要花多久。这三件事摸清了，再上真活。我在第 10 章那个方向盘案例里花在冒烟测试上的时间不到十分钟，它替我省掉的是两次推倒重来。

怎么确认装成功了

三步确认，两分钟做完，缺一不可都不算装成：

1. 握手有清单。问一句列出工具，回得出来，并且数量与项目说明里的对得上。
2. 健康检查双绿。有健康接口的走接口，两个字段都要对；没有健康接口的，跑一个只读工具，比如列目录、读版本。
3. 最小任务有回包。真的调了一个工具，拿到了带内容的返回值，而不是一句"已完成"。

三条全绿，才配叫装成。

我第一次装的时候翻的车

把我第一次的完整经历写出来，你大概率会在里面看到自己。

第一次装是八月底那个周末。我照着项目主页的说明把服务端装上了，配置也写了，重启之后问了一句"你有什么工具"，回了一声"没有挂载任何 MCP 服务"。我改配置，重启，再问，还是那句。来回七次，两个小时。

最后救我的是 20.3 第一步那条：把服务端单独拿到命令行里跑。我一跑，屏幕上打出来一行报错，是缺一个依赖包。装上那个包，再重启，工具列表一次就出来了。

那两个小时我学到的东西只有一条：不要在配置里猜，要在命令行里看。服务端自己启动时会说的话，才是真话；客户端回给你的那一句"没有挂载"，是它猜的。

第二次翻车隔了三天。工具列表出来了，我兴冲冲地提了个建模需求，它卡了四十秒然后报超时。我查了半天服务和配置，最后发现是 SOLIDWORKS 没开。它列工具列得很顺，因为列工具这件事根本不需要软件在那儿；一到真正动手，软件不在，就卡住了。

从那以后我养成了现在的习惯：每次提业务需求之前，先调一次健康检查，或者先跑一个只读的小工具。十秒钟，挡掉四十秒的干等。

装完之后立刻做的一件事

把这次能跑通的配置原样存一份到你自己的笔记里，连同版本号、包名、运行时版本一起存。

我吃过亏。九月份我把一条跑得好好的链路升级了一次，升完之后连不上，而我把原来那版配置覆盖了，回不去。那天我花了一个半小时才重新凑出一份能用的配置。现在我的做法是：每一份能跑通的配置存一个带日期的文件名，比如 `mcp-config-2026-09-19-ok.json`。起作用的那份永远不动。

20.4 常见的 MCP 服务能干什么

下面这张表按"它能碰什么"分类。可信度一列我按自己的实际情况标了，没跑过的一律标 ★ 并注明来源，不写亲测。

类别	具体能干的事	代表例子	可信度
驱动建模软件	建零件、建装配体、出工程图、导出 STEP、读质量属性、渲染出图	SolidWorks-MCP、SolidPilot、Blender MCP	★★★ 本机实测
操作浏览器	打开网页、填表、点按钮、抓页面内容、跑一段自动化操作	各类浏览器自动化服务端	★ 公开资料与社区案例
读数据库	查表结构、跑只读查询、导出查询结果	各类数据库服务端	★ 公开资料与社区案例
操作系统文件	列目录、读文件、写文件、在指定目录内移动与重命名	文件系统类服务端	★★ 跑通过流程
调专业软件	电路设计、仿真求解、版本管理、办公文档	各类专业软件服务端	★ 公开资料与社区案例
调命令行	在受控前提下执行指定命令、跑脚本、看输出	命令行类服务端（慎用）	★ 公开资料与社区案例

我实测过的那一类：驱动建模软件 ★★★

这一类我本机跑得最多，也是最值得工程读者投入的一类。本机那条链路（Python 适配器 46 个工具到 C# 执行层再到 SOLIDWORKS COM），已经稳定做到的事包括：

- 建草图、加约束、加尺寸，然后拉伸、切除、旋转、阵列、镜像。
- 倒角与圆角，按几何指纹选边（第 10 章场景 22 那一套）。
- 读质量属性，也就是体积、表面积、质心、包围盒这些能用来验收的数。
- 建装配体、加配合、查干涉。
- 出二维工程图，含剖视与尺寸。
- 导出 STEP 与 STL，我这边跑过一次 258 实例的装配体导出，STEP 文件一千一百多万字节。

这一类最大的价值在于：它做出来的东西能进你的现有流程，能二次编辑、能出图、能发外协、能被检验。正因为如此，它才值得你花两小时装一次。

这一类最大的坑，也在 46 个工具这个数字上。工具多不代表稳，我在这条链路上累计记过三十七条硬坑，光“选边选错”这一类就有四条不同成因。所以第 10 章我才反复强调先冒烟测试。

我没有实测过的那几类 ★

浏览器、数据库、命令行这几类，我在这套环境上没有完整跑通过，下面说的是公开资料与社区案例里的共同点，具体项目名与能力边界以各项目主页为准。

操作浏览器这一类，公开资料里稳定的用法是自动化填表、批量抓页面数据、跑重复性的网页操作。它的风险不在能力，在账号：一旦 AI 能操作你的浏览器，它就等于能碰你所有已登录的网站。这一类的权限给出去要格外谨慎，见 20.6。

读数据库这一类，社区里公认的好做法是只给只读账号，并且在服务端那一侧就把表名白名单定死。能改数据的数据库服务端，我个人的看法是别装。

调命令行这一类，我放在最后并且要说重话。这一类等于把你的整台电脑交给它。除非你非常清楚自己在做什么，并且有隔离环境，否则不要装。我在 20.6 里把它列为最高一级权限。

常见服务分类：能碰什么

越往下，能给出去的权限越大

驱动建模软件

建零件、建装配体、出工程图、导出 STEP、读质量属性、渲染出图。本机实测

操作浏览器

打开网页、填表、点按钮、抓页面内容。风险在账号，它能碰你所有已登录的网站

读数据库

查表结构、跑只读查询、导出结果。建议只给只读账号，并在服务端把表名定死

操作系统文件

列目录、读文件、写文件，在指定目录内移动与重命名。跑通过流程

调专业软件

电路设计、仿真求解、版本管理、办公文档。具体边界以各项目主页为准

调命令行

执行指定命令、跑脚本、看输出。等于把整台电脑交给它，我把它列为最高一级权限

装之前先问一句：它一旦失控能碰到的最坏的东西，你能不能承受

图 20-6 五类常见服务与各自能碰的范围，越往下权限越大

图 20-6 作者根据公开资料绘制。

怎么快速看清一个服务的能力边界

拿到一个陌生的服务端，我用三个动作判断它值不值得装，五分钟能做完：

1. 看它报多少个工具。工具数量在二十个以下的，通常是专注做一类事的，边界清楚，装起来放心。五十个以上的要留神，说明它想干的事太多，你很难判断每个工具会碰什么。
2. 看工具说明写不写得清。工具说明就是给 AI 看的说明书。写得含糊的服务，AI 用起来会乱猜，出了错你都不知道它猜的是什么。
3. 看它的只读工具够不够多。只读工具多的服务，你可以先只读地摸一遍，再决定要不要开写权限。只读工具一个都没有的服务，我一般不装，因为连试错都要承担风险。

这三个动作对应的是同一件事：在给它权限之前，先把它的边界摸清楚。摸清楚边界所花的五分钟，比事后排查的五十分钟便宜得多。

一句话判断该装哪一类

装之前先问一句：这个服务能碰的最坏的东西是什么？最坏的东西你能承受，就装；承受不了，就不装。

这句话听着朴素，但它是我做这一类决策用的唯一标准。判断一个 MCP 服务该不该装，不看它宣传的能力有多强，看它一旦失控能碰到什么。

20.5 自己写一个最小的 MCP 服务 ★

这一节是给想深入的读者。小白可以先跳过，跳过不影响你用 MCP。你完全可以一辈子只装别人写好的服务，就像你不需要会造插座也能用插线板。

我把它写进来只有一个理由：看过一次服务端的内部结构之后，你对“它到底能碰什么”这件事会清醒很多，这份清醒在 20.6 的安全判断里用得上。

本节标 ★，示例按 MCP 官方 Python SDK 的通用写法给出，具体包名、函数名、参数名以你安装版本为准。

先看线上说的是什么

不管用哪种语言写，MCP 服务端和客户端之间的对话都是 JSON-RPC 格式，也就是一段一段的 JSON 文本。理解这一段，比急着写代码有用。

客户端问一句“你有什么工具”：

```
{"jsonrpc": "2.0", "id": 1, "method": "tools/list", "params": {}}
```

服务端回答：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "add",
        "description": "把两个数相加并返回结果",
        "inputSchema": {
          "type": "object",
          "properties": {
            "a": {"type": "number"},
            "b": {"type": "number"}
          },
          "required": ["a", "b"]
        }
      }
    ]
  }
}
```

客户端再调一次这个工具：

```
{"jsonrpc": "2.0", "id": 2, "method": "tools/call",
  "params": {"name": "add", "arguments": {"a": 3, "b": 4}}}
```

服务端回：

```
{"jsonrpc": "2.0", "id": 2,
  "result": {"content": [{"type": "text", "text": "7"}]}}
```

这四段看懂了，MCP 的原理就没什么神秘的了。它就是一个“你问我答”的来回，问和答都用 JSON 写，一问一答配一个 id 对上号。那个 `id` 的作用是对账，让客户端知道哪个回答对应哪个问题。

上面的方法名与字段结构以你采用的协议版本为准。老版本里方法名可能带前缀，字段也可能略有差别。

最小的服务端代码

用 Python 的官方 SDK 写，一个能跑的服务端只有十几行：

```
# demo_server.py: 一个最小的 MCP 服务, 只提供一个工具
# 依赖: pip install mcp
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("demo")

@mcp.tool()
def add(a: float, b: float) -> float:
    """把两个数相加并返回结果。"""
    return a + b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

这十几行拆开看：

- `FastMCP("demo")` 建了一个服务端对象, "demo" 是它的名字, 客户端那边看到的就是这个名字。
- `@mcp.tool()` 是一个装饰器, 意思是把下面这个函数登记成一个工具。函数下面那行三引号注释不是可选的装饰, 它是工具说明, 客户端和 AI 读的就是这行字, 写清楚它才用得对。
- 函数签名里的类型标注 `a: float` 也不是摆设, SDK 会拿它去生成工具的输入格式说明。类型写清楚了, AI 才不会往里塞一个字符串。
- `mcp.run(transport="stdio")` 让它跑起来, 用 `stdio` 这种通信方式。

stdio 是什么：服务端和客户端通过"标准输入与标准输出"这根管道说话, 客户端往服务端的输入里写一行 JSON, 服务端往自己的输出里写一行 JSON 回来。它不占端口, 也不需要网络, 进程一开一关就干净。本机服务绝大多数用它。

登记它, 验一次

配置照 20.3 那个最简形态写：

```
{
  "mcpServers": {
    "demo": {
      "command": "python",
      "args": ["D:/mcp-demo/demo_server.py"]
    }
  }
}
```

重启客户端, 问一句列出工具。清单里应当出现 `demo` 这台服务, 下面挂着一个叫 `add` 的工具。让它算一次 `add(3, 4)`, 回 7, 就成了。

看懂这个结构之后最重要的一句

工具里那几行代码，就是 AI 能碰到的全部世界。

你在 `add` 这个函数里写什么，AI 就能干什么。你写一句 `open(某路径).read()`，它就能读那个文件；你写一句 `os.remove(某路径)`，它就能删那个文件。MCP 本身不带任何限制，限制只来自你这支笔。

这句话是这一节存在的全部理由。装别人的服务时，你其实是在信任别人的那支笔。

老兵说：MCP 服务的权限大小，不看它叫什么名字，看它里面那只工具函数里写了几行代码。

20.6 安全第一：它能碰你本机 ★★★

前面铺垫这么多，就是为了这一节。

连接器给出去的权限，是一个外部账号里的一部分数据权限，最坏情况是别人系统里的那份数据出问题，而且通常能撤回、能改密码。MCP 给出去的权限，是你这台电脑上真实的文件、真实的进程、真实的软件。删掉的文件进不了回收站，覆盖保存的图纸回不到上一版。

本节标 ★★★，这几条规矩是我和本机撞出来并长期执行的。

权限分级表

按"一旦失控最坏能碰到什么"分五级，从低到高：

级别	能碰什么	典型例子	我的规矩
1 级 只读本机	列目录、读文件内容	只读的文件系统服务	给一个专用目录，随便用
2 级 写指定目录	在指定目录内新建、改、删	带写权限的文件服务	目录里不放别的，用完就撤
3 级 操控本机软件	驱动已安装的专业软件	SOLIDWORKS、Blender 类服务	装，但 destructive 动作要人工确认
4 级 起进程	启动程序、跑脚本、看输出	命令行类服务	不装，除非有隔离环境
5 级 碰账号	操作浏览器、改数据库	浏览器与数据库写服务	不装，需要时人工做

这张表的使用方法是：先定你的服务属于第几级，再决定装不装。1、2、3 级在受控前提下可以装，4、5 级我的建议是不装。

有人会问，3 级不是也能删文件吗？能。所以 3 级那条"destructive 动作要人工确认"是硬要求，不是建议。我在第 10 章的总开工指令里写了"任何一步失败，先把原始报错原样贴给我，然后停下来等我指示"，那句话既是质量规矩，也是安全规矩：它让 destructive 动作在发生之前先经过我一次。

本机权限五级：越往上越接近整机

按一旦失控最坏能碰到什么分级

5 级 碰账号

操作浏览器、改数据库。不装，需要时人工做

4 级 起进程

启动程序、跑脚本、看输出。不装，除非有隔离环境

3 级 操控本机软件

驱动已安装的专业软件。破坏性动作要人工确认

2 级 写指定目录

在指定目录内新建、改、删。目录里不放别的，用完就撤

1 级 只读本机

列目录、读文件内容。给一个专用目录，随使用

先定你的服务属于第几级，再决定装不装。一级、二级、三级受控可装

图 20-7 权限五级金字塔：越往上能碰的东西越接近你的整台电脑

图 20-7 作者根据公开资料绘制。

三条硬规矩

规矩一：一次只给一个目录，这个目录里不放别的东西。

我本机的做法是建一个专用目录 `D:/AI-CAD`，所有 AI 相关的读写全部发生在这里面。这个目录里不存任何不可替代的东西，即使整个目录被清空，我损失的只是可以重跑的产物。

这条规矩的反面是很多人图省事，直接把 `D:/` 或者“我的文档”给出去。那是把自己的全部家当放在一个不需要负责任的程序手边。

规矩二：只装查得到出处的服务，不装来路不明的。

判断出处看三样：项目主页在不在、许可证写没写、最近一次更新是什么时候。三样里有两样查不到，就不装。

理由很直白：你是在把本机的一部分控制权交给一段别人写的代码。这段代码能干什么，你在装之前是看不见的（除非你真的去读它，而 20.5 那十几行告诉你，读明白一个服务端其实不难）。既然看不见，就只能看出身。

规矩三：涉及删除、覆盖、外发、提交的动过，必须留人工确认。

四类动作我一律不放手：删文件、覆盖保存已有文件、对外发送任何东西（邮件、消息、上传）、提交代码或数据到共享库。

前两类是丢了就找不回，后两类是收不回来。我在提示词里会把这条写死，格式是：

【安全规矩】以下四类动作在执行前必须先向我确认，得到我明确的同意后再执行：
删除任何文件、覆盖保存任何已存在的文件、对外发送任何内容、提交任何东西到共享库。
其余动作可以自主执行。每一步做完都要向我汇报真实返回值。

这一段可以直接复制使用。

还有一个动作：定期清点

装服务这件事有个特点：装的时候你会认真想一次该不该装，装完之后就再也不想了。三个月后你的机器上挂着七个服务，其中四个你已经忘了是干什么的，而它们每一个都还开着权限口子。

我的做法是每季度清一次点，动作只有三步：

1. 列一遍当前挂载的全部服务，逐个问自己最近三个月用没用过。
2. 没用过的直接停用，不删配置，只停用。停用一周内没人找，再删。
3. 用过的，重读一遍它的权限级别，看有没有升级。服务端更新之后新增了写权限这种事，发生过不止一次。

清点的意义在于把"永久授权"变成"定期续期"。公司里的供应商准入要年审，你自己机器上这些能碰本机的程序，凭什么不审。

还有一个容易被忽略的口子

MCP 服务是你自己起的进程。它跑在你的账户下，因此它拥有你这个账户的全部权限。如果你的客户端是以管理员权限启动的，那么它起的服务也是管理员权限。

所以有一条配套规矩：日常不要用管理员权限启动客户端。我见过有人为了让某个服务装上，把整个工具用管理员权限开起来，从此这台机器上所有 MCP 服务都拿到了管理员权限。这是把一级口子开成了五级。

20.7 常见故障与排查 ★★★

这一节八条，全部是我本机实际遇到并解决过的。排障的核心思路只有一句：先定位段，再查细节。段有四段：配置、服务进程、被接的软件、权限。

MCP 排障：先定位段，再查细节

八条故障全部落在这四段里



实在没头绪就按七步顺序表从上往下走，每一步都要看到通过的样子

图 20-8 故障排查决策树：先分“连不上”还是“连上了不对”，再按段定位

图 20-8 作者根据公开资料绘制。

故障一：服务压根起不来

症状：工具列表里看不到这台服务，或者看到但状态标红。

先查三样：

1. `command` 里写的那个程序在命令行里能不能直接敲出来。敲 `python --version`，敲不出就是环境变量没配，配置里要写完整路径。
2. 依赖装没装。Python 侧常见的是 `pip install` 漏了，Node.js 侧常见的是包没下载完。
3. 启动参数对不对。把配置里 `command` 加 `args` 拼成一条完整命令，直接在命令行里跑一遍，看它报什么。

判断点：在命令行里能跑、在配置里起不来，差的就是环境变量与工作目录。配置里 `env` 那一项就是用来补这个差的。

故障二：服务起来了，工具列表是空的

症状：能看到服务名字，但下面一个工具都没有。

三个成因，按顺序查：

1. 服务端内部报错了，但它是"起来之后才报错"，所以进程活着却没注册任何工具。查服务端的运行日志。
2. 通信方式对不上。服务端跑的是 stdio，客户端却按 HTTP 去连，或者反过来。配置里指定传输方式的字段要以你客户端版本支持的写法为准。
3. 版本差一代。服务端用的协议版本比客户端支持的更新或更旧，握手时对不上口径。

判断点：让客户端把握手时的原始返回贴出来。空的 `tools` 数组和压根没有 `tools` 字段，指向的成因不同。

故障三：调用超时

症状：工具清单看得到，一调用就卡住，最后报超时。

本机专业软件这一类，按这个顺序查：

1. 软件开了没有。SOLIDWORKS 没开，服务端贴不上去，调用必然挂。
2. 软件的主线程是不是被锁住了。我在本机遇到过一次：对一份"改过但没存过盘"的文档执行关闭动作，软件主线程陷进内部确认逻辑，之后所有调用全部挂死不返回，而界面看起来完全正常。判断依据是进程 CPU 时间在持续累积。这种状态的唯一出路是重启软件，它不可逆。
3. 是不是把多个请求一次性塞进去了。有些执行层是状态机，要求"发一个、等回包、再发下一个"。一次塞一串会返回一串状态版本错误。

判断点：先调那个只读的健康检查接口，看它的两个字段。健康接口都超时，问题在连接层；健康接口正常而业务工具超时，问题在那个动作本身。

故障四：权限被拒

症状：报错里出现 denied、not allowed、outside、permission 这类词。

两个成因：

1. 路径出了你给的目录。想读 `D:/其它目录/文件.txt`，而你只给了 `D:/AI-CAD`。
2. 权限级别不一致。软件以某个权限级别开着，服务端进程是另一个级别。Windows 上这两个级别对不上，跨进程调用会被拦。

补救：要读的文件挪进白名单目录，别去放宽白名单。软件和服务端用同一个权限级别重开。

故障五：版本不匹配

症状：握手就失败，报错里出现 version、protocol、unsupported 这类词。

成因：服务端实现的协议版本，和客户端支持的协议版本对不上。

补救三选一：升服务端、降服务端、或者换一个与你的客户端同期发布的服务端版本。不要去改协议本身，那不是你的活。

预防：20.3 结尾那条"把能跑通的配置存一份"，在这一条上最值钱。存下来的那份里带着版本号，回退的时候有地方可退。

故障六：路径写错

症状：报找不到文件、找不到目录，但你看那个路径明明是对的。

四种写法错，逐个对：

1. JSON 里写了单反斜杠。D:\AI-CAD 在 JSON 里会被当成转义，要写 D:/AI-CAD 或 D:\\AI-CAD。
2. 路径里有中文或空格。有些服务端对这两样处理不好，能换就换。
3. 写了相对路径，而服务端进程的工作目录跟你以为的不是同一个。
4. 盘符不存在，或者是个没插上的移动硬盘、没映射的网络盘。

补救：全部改成英文、无空格、带盘符的绝对路径，并且在干活之前先让它列一遍目录确认存在。我在第 10 章的准备工作里把"路径不要有空格和中文"列为硬条款，就是因为这一条。

故障七：本机进程被连带杀掉

症状：服务或者被接的软件，用着用着自己就退出了，退出时机还很规律。

成因：它是从某个工具会话里被启动的，那个会话一结束，它作为子进程被一起带走。

补救：用守护方式启动。启动之后让启动它的那个进程保持常驻，而不是启动完就退出。本机我用的是一段后台脚本：启动目标程序之后自己进入一个长循环，每隔一段时间检查一次目标进程还在不在，同时把状态写进日志。这样父进程一直活着，子进程就不会被连带回收。

判断点：看退出时机。每次都在某个操作结束后不久退出，基本就是这一条。

故障八：报状态版本错误或重复

症状：报错里出现 INVALID_STATE_VERSION 或 DUPLICATE 这类字样。

两个成因，正好对应两条：

1. INVALID_STATE_VERSION 是你把多个请求并行发出去了。执行层要求串行，发一个、等回包、再发下一个。
2. DUPLICATE 是操作编号重名了。有些执行层要求每次调用带一个唯一编号，跨请求还不能重复。

补救：调用改串行，编号用能生成全局唯一值的办法产生，别用自增计数器。

一张排障顺序表

实在没头绪的时候，按这七步从上往下走，别跳步：

步	查什么	通过的样子
1	服务进程在不在	命令行里能单独跑起来并停在等待状态

步	查什么	通过的样子
2	配置改完有没有重启	完全退出重开过
3	握手有没有工具清单	列得出来，数量对得上
4	有健康检查的走一次	两个字段都对
5	被接的软件开没开	手动打开并停在就绪状态
6	报错里有没有版本、权限、路径关键词	按对应故障处理
7	跑一个只读的最小工具	拿到带内容的回包

七步走完还是不通，就把第七步的原始报错整段保存下来，连你的配置文件一起，去那个服务的项目主页提 issue。报错原文比你的描述有用得多。

小白词典

这一章的术语是全书最抽象的一批，我每个都配了类比。

MCP (Model Context Protocol)：一套让 AI 操控其他软件的通用插座规矩。它规定了 AI 怎么要工具、软件怎么报工具，两边照着做就能接上，互相不用认识。类比：插线板上那个谁都能插的孔位标准。

客户端 (client)：AI 那一侧，负责把 AI 的话变成符合规矩的请求发出去，再把回包交还给 AI。类比：插头。

服务端 (server)：你要接的软件那一侧，一个跑在电脑上的小程序。它负责报工具清单，以及把请求翻译成软件能听的动作。类比：电器加它自带的电源适配器。注意它跟机房里那台机器没有对应关系，它只是一个进程，随用随起，用完就退。

进程 (process)：一个正在运行的程序。你可以同时开十个进程，它们各干各的，互不干扰。MCP 服务端就是一个进程。类比：厨房里同时开着的灶眼，每个灶眼上炖着不同的锅。

工具 (tool)：服务端上报的一个具体动作，一个工具就是一件事。比如"新建零件""拉伸 12 毫米""读体积"。类比：电器面板上的一个按钮。

JSON：一种用大括号和方括号组织数据的文本格式，人和机器都能读。配置文件、网络传输、AI 收发 的消息，现在绝大多数都用它。类比：填表，格子名写清楚，格子里填值。

RPC (Remote Procedure Call)：让一个程序去调用另一个程序里的函数，并且拿到返回值。它不管这两个程序是不是在同一台电脑上。类比：你打电话给另一个部门，让他们办一件事并把结果告诉你。

JSON-RPC：用 JSON 写成的 RPC。问一句是一段 JSON，答一句也是一段 JSON，靠一个编号对上号。类比：两个人用便签条一问一答，每张便签上写个号防止答串了。

stdio (标准输入输出)：程序自带的两根管道，一根进、一根出。用它通信不占端口、不走网络，进程一关就干净。类比：两个人隔着墙用一根传声筒说话。

端口 (port)：一台电脑上用来区分不同网络程序的编号，比如 5000、8765。走网络通信的服务要占一个端口。类比：一栋楼里的门牌号，快递按门牌送到对的一家。

transport (传输方式): 服务端和客户端用哪根线说话。常见的有 stdio 和网络两种。类比: 两个人说话是用传声筒还是打电话。

schema (格式说明): 一份描述"参数长什么样"的说明书, 写清楚每个参数叫什么、是什么类型、哪些必填。AI 靠它才知道该往里填什么。类比: 快递单上印好的那些格子。

COM: Windows 上软件之间互相调用的一套老接口。SOLIDWORKS 对外开了这道门, 别的程序才能隔着门伸手进来。它不属于 MCP, 比 MCP 早得多。类比: 软件之间的对讲机。

pywin32: Python 一侧用来敲 COM 这道门的库。装它是为了用 Python 去驱动 Windows 上的专业软件。类比: 适配这一种对讲机的专用插头。

握手 (handshake): 连接建立时双方先互相确认一下"在不在、会什么"。MCP 的握手就是客户端问一句、服务端把工具清单报上来。类比: 打电话接通后先说"喂, 听得到吗"。

环境变量 (env): 进程启动时带在身上的一批设置值, 比如编码方式、临时目录在哪。配置里 `env` 那一项就是给它设这些值。类比: 出门前揣在口袋里的几张便条。

装饰器 (decorator): Python 里写在函数上面那行以 `@` 开头的代码, 作用是给这个函数额外登记一点信息。`@mcp.tool()` 的意思是把下面这个函数登记成一个 MCP 工具。类比: 给文件贴一张标签, 文件内容没变, 但它被归到了某一类。

你的岗位怎么用

设计工程师: 本章对你价值最高的两节是 20.3 和 20.7。装一次 SOLIDWORKS 那一类服务端, 按六步走完, 然后把 20.7 那张七步排障表打印出来贴在显示器边上。你接下来所有的 AI 建模活 (第 10 章那六个场景) 都建立在这两步之上。

工艺与制造工程师: 你的关注点是 20.6 的第 2 级权限。工艺文件、作业指导书这些都要落到具体目录里, 给 AI 一个专用目录并且定期归档, 比事后找文件省事得多。

质量工程师: 20.6 那张权限分级表可以直接改造成你部门的 AI 工具准入表。谁申请、申请几级、给哪个目录、保留多久, 四列就够。你本职就是管权限和留痕, 这件事天然归你。

项目经理: 盯 20.6 的三条硬规矩, 尤其是第三条那四类必须人工确认的动作。把它写进团队的 AI 使用规范里, 一句话就能挡住绝大多数事故。另外 20.3 结尾"把能跑通的配置存一份带日期的文件名", 这条对任何需要回滚的事情都成立。

采购与供应商管理: 你大概率不需要自己装 MCP, 但你需要知道一件事: 供应商交来的数模如果是 AI 生成的, 验收标准不变。单位、体积、干涉、STEP 往返, 第 10 章场景 22 那六行查验单照样适用。

试验工程师: 20.4 那张分类表里, 对你有用的是"读数据库"和"系统文件"两类。试验数据批量导出、报告模板批量填充, 这两件活装一个只读服务就能接。记住只给只读。

本章交付物

一份你自己机器上的、能跑通的 MCP 配置文件, 外加一张打印出来的排障表。

做法三步：

1. 按 20.3 的六步，装一个你真正用得上的服务。第一次装建议从最简单的那类开始，比如一个只读的文件服务，别一上来就挑战 SOLIDWORKS。
2. 跑通之后，把那份配置原样存成 `mcp-config-2026-xx-xx-ok.json`，存进你的笔记软件，并且从此不再改动这一份。
3. 把 20.7 那张七步排障顺序表抄下来，贴在显示器边上。

第一件验收的活：装成之后，让 AI 跑一个三十秒内能验完的最小任务，并且自己动手验一次它的返回值。比如让它列出某个目录里的文件，然后你自己打开那个目录数一遍。数得对，这条链路就是真的。

本章金句

AI 会不会用你的软件，跟它聪不聪明没关系，跟墙上有没有插座有关系。

MCP 不生产能力，它生产接口一致性。

连接器给 AI 一个身份，MCP 给 AI 一双手，技能给 AI 一份作业指导书。

能用嘴教會的活，别给它手。

看到工具列表才配叫装成，看不到就提需求，等于闭着眼睛上机床。

MCP 服务的权限大小，不看它叫什么名字，看它里面那只工具函数里写了几行代码。
(全书第五篇完。下一章进入第六篇：团队落地。)

PART SIX · 第六篇

团队落地 · 成本 · 排查

第 21 章

从一个人用，到一个部门用

团队推 AI，先卡住的是人心里的责任账，工具功能还排在后面。

我自己用顺以后，犯过一个带团队的人很容易犯的错：我以为演示一次，大家就会照着用。那年我带着一个约十来人的研发小组，周会上用约十五分钟演示了怎样整理试验记录、提取待办、生成周报初稿。屏幕上的活原来要花大半个下午，那次约四十分钟就有了结果。有人当场说：“这个真能省事，下周我也试。”

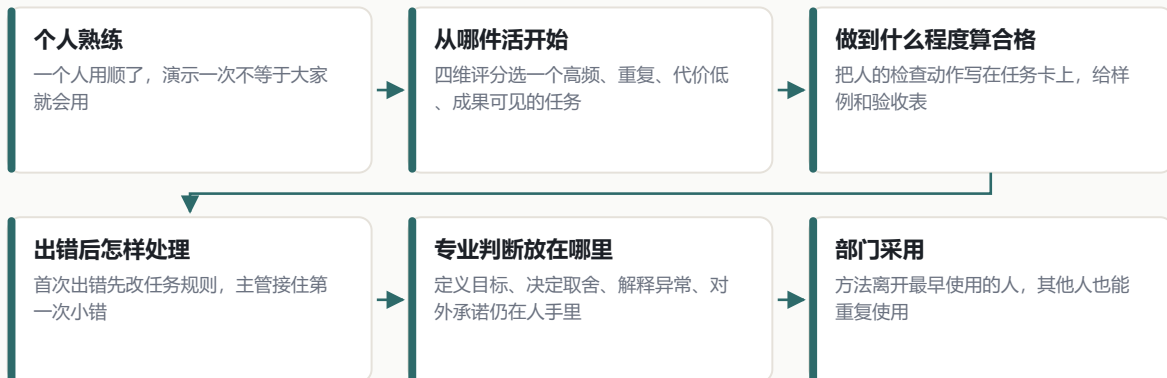
我随后建了共享目录，放进去约二十条提示词，又在群里提醒过几次。约三个月后，我翻使用记录，真正在持续用的只有两个人，一个是我，另一个是愿意拿新东西折腾的项目工程师。其余人要么只打开过一次，要么把 AI 当搜索框问两句，再也没有回来。

我找同事单独聊。试验工程师说：“我上次让它整理记录，批次号少了一位。这个错发出去，算谁的？”质量工程师说：“你演示的都是你的活，我每天那几张表能不能做，我不知道。”还有一位老同事说得更直接：“你们越用越快，公司以后还要这么多人吗？”

这三句话让我明白，失败原因没有藏在安装步骤里。团队成员看见的是责任风险、学习成本、岗位距离和被替代的担心。我给了一堆提示词，却没给第一个能赢的小任务；我展示了速度，却没写验收规矩；我要求大家试，却没有说明出错后谁来接。后来我换了方法：先跑通一个低风险任务，让三个人照着同一份作业做，再由全组共同写规矩。使用人数才开始往上走。

从个人熟练到部门采用的三道门

中间隔着任务、责任与规矩；成员缺的不是热情，是可预期性



主管把这四件事说清，推广才有起点

图 21-1 从个人熟练到部门采用：中间隔着任务、责任与规矩三道门

图 21-1 从个人熟练到部门采用的三道门。作者根据公开资料绘制。

21.1 为什么推不动：五个真实阻力

带团队的人容易把“不使用”理解成态度问题。我后来不再这样看。一个人没有持续使用，往往有一个很具体的理由。主管要做的第一件事，是把理由听完整，再给一个能执行的解决办法。

阻力一：怕出错担责

这是工程部门最合理的一种担心。一个新人用 AI 整理了试验数据，里面漏了一行，主管若只问“你为什么没检查”，他下次最安全的选择就是不用。工具省下的是时间，责任却还在他名下，他自然会保守。

破解办法有三步。第一，任务开始前沿用第 18 章的 L1 / L2 / L3 决策矩阵定级，不在本章另造一套。第二，把人的检查动作写在任务卡上。第三，明确按规矩做过检查仍出错，先查流程和规则，不先把问题压给操作者。

我给团队的说法很直白：“你按约定定了级，按清单验了，记录也在，出了问题我们一起改规则。你跳过验收直接发，责任才回到你个人。”这句话比“大胆试”有用，因为它说清了安全边界。

主管还要做一个动作：亲自接住第一次小错。有同事第一次用 AI 汇总问题清单时，把“待验证”写成了“已解决”。他来找我，以为会挨批。我和他一起查，发现模板里根本没有状态词白名单。我们把状态固定成“待确认、进行中、已验证”三种，再拿原材料重跑。第二周，他继续用了。

破解清单：

1. 每个任务先标 L1、L2 或 L3，标不按时按更高一级处理。
2. 给出对应验收动作，不说“自己注意检查”。
3. 留下输入、输出、检查结果和拍板人。
4. 首次出错先改任务规则，再讨论个人是否漏做了动作。
5. 主管公开说明：速度不作为跳过验收的理由。

阻力二：不知道能干什么

“大家可以多用 AI”是一句几乎没有用的话。工程师听完仍要自己想场景、准备材料、写要求、判断结果。对已经被项目节点塞满的人来说，这相当于又领了一项研究任务。

我后来不再问“你想用 AI 做什么”，改问三个具体问题：“这周哪件事做了三次以上？哪件事最烦但有固定格式？哪件事错了能很快改回来？”答案通常会落到会议纪要、文件清单、试验记录整理、版本差异检查、周报初稿这一类活上。

有位设计工程师一直说自己的工作“没法交给 AI”。我跟他坐了约二十分钟，没有讨论方案设计，只看他周五下午做什么。他每周都要把更改记录从多个文件里摘出来，合成一张评审清单。这个活规则清楚、输入固定、结果能逐项核对。我们把它做成他的第一个任务，他才知道所谓“能干什么”不是把岗位交出去，是先拿掉岗位里那一段重复搬运。

破解清单：

1. 不开空泛的创意会，直接盘点最近一周的重复动作。
2. 每个岗位只选一个起步任务，不要求一次找全。
3. 用本章 21.2 的四维评分表筛选，不靠主管个人喜好。
4. 给任务样例、输入样例和验收样例，让人能照着做。
5. 第一次成功后，再从相邻任务扩展。

阻力三：试了一次没成

很多人第一次给 AI 的话只有一句：“帮我整理一下。”得到的结果当然不稳定。他看一眼，觉得还不如自己做，从此给工具贴上“不靠谱”的标签。问题出在第一次体验没有完成条件，也没有人陪他走完返工。

破解办法是把第一次使用设计成一个小实验。输入不超过一组，产物只要一份，验收不超过几项。主管或先跑通的人坐在旁边，看到偏差就改任务单，让同事亲眼看到“第一次不成”怎样变成“第二次能复用”。

我带一位项目工程师做会议纪要时，第一次输出漏了两个责任人。他说：“看吧，还是得我重写。”我没有让他重写，而是把任务单补成四栏：事项、负责人、日期、原话位置；缺一项就写待确认。第二次输出仍有一条日期缺失，但它被老实放进待确认栏。我们花几分钟补完，比从头重写省事。关键变化是他看见了可修的路。

破解清单：

1. 第一次只拿历史材料或内部草稿练，不碰当天要外发的急件。

2. 先给完成条件，再开始执行。
3. 保留第一次失败产物，逐项标出失败原因。
4. 只改导致失败的那条规则，再跑一次。
5. 连续两次仍不稳定，就换任务，不拿一个坏场景证明所有场景都不行。

阻力四：觉得跟自己无关

如果培训案例全是写代码，试验人员会觉得无关；全是写报告，设计人员也会走神。部门推广常败在“一个漂亮案例讲给所有人”，因为每个岗位真正烦的活不同。

破解办法是按岗位做任务地图。研发主管列设计、试验、质量、项目四列，每列先放三个真实任务。每个任务旁边写输入是什么、产物给谁、错了会怎样。团队成员只需从自己那一列挑，不必把别人的方法硬套过来。

我有次在内部分享里演示批量整理技术文档，反应平平。会后，一位试验同事问：“能不能帮我找出日志里缺采样的时间段？”这个问题比我的演示更贴近他。我和他把规则写清，输出缺口起止时间和原始行号。下次分享，我直接用这项任务讲，试验组的人马上开始问自己的文件能不能照做。

破解清单：

1. 培训材料按岗位分组，不做一份大而全的演示。
2. 案例必须来自本部门上个月做过的活。
3. 展示前后文件和验收过程，不只展示最终页面。
4. 允许成员说“这个与我无关”，然后换成他的真实材料。
5. 每个岗位至少有一位共同维护案例的人。

阻力五：担心被替代

这类担心很少有人会在大会上直说，私下却很常见。主管若只回答“不会的”，可信度很低。成员看到的是：原来半天的活现在几十分钟做完，那剩下的人力会发生什么？

我的处理办法是把工作拆成动作和责任。搬运、改格式、找差异、列清单，这些动作可以交出去。定义目标、决定取舍、解释异常、对外承诺，这些责任仍由人承担。成员需要看到自己从重复动作里拿回的时间将用在哪里，比如提前做风险验证、补试验计划、检查供应商输入、复盘长期问题。

我会在任务开始前说清楚：“这次省下来的时间，不拿来比较谁做得更快，也不据此减人。我们先把长期欠着的验证和改进补上。”随后我真的把空出来的时间排进计划。如果口头说学习，实际却立刻加更多任务，团队很快就会看穿。

主管还要公开肯定“教会别人”的贡献。一个人把自己摸索的方法写成模板，让五个人少走弯路，这应当进入工作总结。否则成员会把方法藏在个人电脑里，因为分享只会让自己失去优势。

破解清单：

1. 公开列出交给 AI 的动作，以及仍由人承担的责任。
2. 说明省下时间的去向，并把它排进真实计划。

- 3. 不发布个人速度排行榜，不使用次数做绩效名次。
- 4. 把模板维护、案例讲解、错误复盘计入团队贡献。
- 5. 对岗位变化讲具体任务，不用空话安慰。

五种阻力背后有一个共同点：成员缺的不是热情，是可预期性。他要知道从哪件活开始，做到什么程度算合格，出错后怎样处理，自己的专业判断还放在哪里。主管把这四件事说清，推广才有起点。

21.2 从哪个活切入

部门第一次选任务，优先挑容易赢的，不挑最有想象力的。一个任务适不适合起步，我看四个维度：高频、重复、出错代价低、成果看得见。每项按 1 到 5 分打分，5 分代表更适合起步。

“成果看得见”常被忽略。节省了几步思考很难展示，一张从原始记录自动生成、又能逐项核对的清单则人人看得懂。团队需要先看见一次可靠的结果，才愿意投入第二次学习。

最容易成功的切入任务清单

工程部门真实任务	高频	重复	出错代价低	成果看得见	建议起步方式
把会议记录整理成事项、负责人、日期、原话位置四栏	5	5	5	5	用一次内部例会录入稿试跑，缺项统一写待确认
汇总周报素材，按完成、风险、待决定、下周动作分类	5	4	5	5	先生成内部初稿，负责人逐条核来源
批量生成文件索引，列文件名、版本、日期、用途	4	5	5	5	原目录只读，结果写入新索引文件
比较两个规范版本，列新增、删除、修改条款及位置	3	5	4	5	先做条款差异，不让 AI 判断采用哪一版
整理试验日志中的缺失段、异常段和对应行号	4	4	4	5	只标记异常，不自动删除或补值
从问题清单生成每日跟踪摘要，保留原编号	5	4	5	4	用编号核对条目数，状态词用固定清单
整理评审意见，按专业、责任人、截止时间分组	4	4	4	5	只归类和提取，不替负责人关闭问题
检查交付包是否缺文件、重名、版本不一致	3	5	4	5	先按既定清单检查，缺失项原样列出

这张表的分数用于比较任务，不是假装精确的科学测量。先由两位熟悉现场的人独立打分，再讨论差异。一个任务若在“出错代价低”上只有 1 或 2 分，即使其余项很高，也不要拿来做第一次示范。

切入任务的四维评分卡

每项按 1 到 5 分打分，5 分代表更适合起步

工程部门真实任务	高频	重复	代价低	成果可见	建议起步方式
会议记录整理成四栏	5	5	5	5	用内部例会录入稿试跑，缺项写待确认
汇总周报素材	5	4	5	5	先生成内部初稿，负责人逐条核来源
批量生成文件索引	4	5	5	5	原目录只读，结果写入新索引文件
比较两个规范版本	3	5	4	5	先做条款差异，不判断采用哪一版
整理试验日志缺口	4	4	4	5	只标记异常，不自动删除或补值
生成每日跟踪摘要	5	4	5	4	用编号核对条目数，状态词固定
检查交付包完整性	3	5	4	5	先按既定清单检查，缺失项原样列出

分数只用于比较任务；出错代价低只有 1 或 2 分的任务，不做第一次示范

图 21-2 切入任务四维评分：高频、重复、低代价、成果可见

图 21-2 切入任务的四维评分卡。作者根据公开资料绘制。

评分以后再过四道门

第一道门：输入能不能合法、清楚地提供。如果材料含有不该进入工具的敏感内容，这项任务先停。不要为了推广速度，把资料边界留到事后讨论。

第二道门：有没有客观验收方法。文件数能核，编号能核，来源位置能核，这些适合起步。“写得更有洞察”没有统一检查动作，先放到后面。

第三道门：错误能不能撤回。内部草稿错了可以重做，对外承诺发出后很难收回。第一次练习只选能撤回的。

第四道门：谁愿意当第一个使用者。同一个任务，愿意配合的人比理论最优的人更重要。第一个人要肯保留失败记录、愿意讲真实感受，也愿意让别人复制自己的作业。

一个切入失败的例子

我曾经想用“自动生成技术方案”做部门示范，因为成果看起来最亮眼。第一次评审就卡住了：输入不完整、取舍依据藏在几位老工程师脑子里、每个方案都需要责任人拍板。它属于 L2，甚至部分内容已经接近 L3，根本不适合拿来第一次成功。

后来我们退到“规范版本差异清单”。输入是两个已确认版本，输出只列条款变化和位置，不做采用判断。约一个工作日内，三位同事都能照同一模板得到相近结果。大家先相信了差异提取，再讨论怎样让 AI 帮忙准备方案材料。

这个次序很重要。部门采用新方法，先解决“它能不能稳定做一小件事”，再解决“它能不能参与复杂工作”。顺序反过来，第一次失败会被记很久。

可复制的任务筛选卡

【候选任务】

任务名称：[]

执行岗位：[]

当前频率：[每天 / 每周 / 每月 / 偶发]

输入材料：[]

输出产物：[]

产物给谁使用：[]

【四维评分，1 到 5 分】

高频：[]，依据：[]

重复：[]，依据：[]

出错代价低：[]，最坏后果：[]

成果看得见：[]，怎样验收：[]

【四道门】

输入是否允许提供：[是 / 否]

是否有客观验收动作：[是 / 否，动作是]

错误是否可撤回：[是 / 否]

首位使用者是否确定：[是 / 否，姓名或岗位]

【结论】

进入首批 / 暂缓 / 改小后再评

若改小，删掉哪些判断或外部动作：[]

沿用第 18 章定级：[L1 / L2 / L3]

预期输出是一张填完整的任务卡。主管看到“最坏后果”和“怎样验收”两栏，就能判断这项活能不能进入首批。四道门里有一项为否，先补条件，不急着开工。

21.3 三步推开法

我后来把部门推广压成三步：一个人先跑通，三个人抄作业，全组定规矩。三步的单位分别是任务、复制和制度。每一步都有成功标志，没达到就留在当前步，不按日历硬往前赶。

三步推开法：单人样件、三人复制、全组规则

每一步都有成功标志，没达到就留在当前步，不按日历硬往前赶

1

第一步 一个人先跑通

时间预期约 5 个工作日。选一个 L1 或边界清楚的 L2 任务，留一份人工结果做样件，写清输入、产物、完成条件和验收清单。成功标志是同类输入能重跑，验收不靠个人记忆

2

第二步 三个人抄作业

时间预期约 2 周。完整作业六样：任务说明、样例输入、可复制提示词、预期输出、验收清单、失败后怎么改。成功标志是三人能独立完成，第二轮求助明显减少

3

第三步 全组定规矩

时间预期约 1 个月形成第一版。沿用 L1 / L2 / L3 矩阵，写清可以自动完成、必须停下等人、绝对不允许执行，确定验收人、拍板人和归档位置

三步不能压成一次培训会：没见过能靠培训解决，失败怎么办只能写成规矩

图 21-3 三步推开法：单人样件、三人复制、全组规则

图 21-3 从单人样件到全组规则的三步路径。作者根据公开资料绘制。

第一步：一个人先跑通

时间预期：约 5 个工作日。复杂一点的任务可以多用几天，但只跑一个任务，不同时开三条线。

这个人最好是任务的实际执行者，不一定是主管，也不一定是工具最熟的人。他需要有权拿到真实材料，知道原结果长什么样，也能判断哪里错了。主管负责给时间和边界，不替他把所有操作做完。

具体动作如下：

1. 从 21.2 的清单里选一个 L1 或边界清楚的 L2 任务。
2. 保存一份历史人工结果，作为比较样件。
3. 写清输入、产物、完成条件、禁止动作和验收清单。
4. 用一组材料先跑，保留第一次失败结果。
5. 修改规则后再跑，直到同类输入能稳定复用。
6. 记录人工原步骤、AI 参与后的步骤，以及人仍需做的检查。
7. 把最终提示词、样例输入、样例输出、验收记录放进同一目录。

成功标志不看“结果很惊艳”，看五件事：另一天能重跑；换一组同类输入仍能完成；验收动作不靠原作者记忆；失败项会被明示；产物没有越过任务边界。

第一步最容易犯的错是把先行者变成英雄。他为了让演示成功，手工改了很多地方，却没有记录。演示当天一切顺利，别人照做就失败。主管必须追问：“你手改了哪里？这些动作能不能写进步骤？”藏着的人工动作越多，复制越困难。

第二步：三个人抄作业

时间预期：约 2 周。三个人最好来自同一任务的不同使用位置，例如一位执行者、一位下游使用者、一位需要验收的人。不要只找三个同样熟练的爱好者。

“抄作业”不是发一段提示词让大家自己琢磨。完整作业至少有六样：任务说明、样例输入、可复制提示词、预期输出、验收清单、失败后怎么改。少一项，复制就会变成重新研究。

具体动作如下：

1. 三个人分别用自己的同类材料执行，不共用先行者做好的结果。
2. 第一次执行时不允许先行者代操作，只能回答任务卡上没有写清的问题。
3. 每个提问都记入“说明缺口”，当天补进模板。
4. 三份产物使用同一张验收表，由使用者先验，再互换抽查。
5. 记录失败类型，不比较谁快。
6. 一周后再做第二轮，检查大家能否不求助完成。
7. 让三个人各用约五分钟讲一个踩坑点，不做只讲成功的汇报。

成功标志有四个：三个人都能独立完成；同类错误没有连续出现；验收结论大体一致；第二轮求助明显少于第一轮。这里不要求结果写法完全一样，只要求事实、结构和责任边界一致。

我在这一阶段见过一个很典型的问题。先行者习惯把输入文件改成固定名称，提示词里却没写。第二个人直接用原文件名，任务找不到输入。这个错误与 AI 能力无关，它暴露的是个人习惯没有进入作业。我们把“允许文件类型、命名规则、找不到文件时停手”补进任务卡，第三个人就没有再卡住。

第三步：全组定规矩

时间预期：约 1 个月形成第一版，此后按月小修。这里的重点不再是会不会操作，而是部门怎样共同承担质量、资料 and 外部影响。

具体动作如下：

1. 把前三个人的记录整理成可讨论的事实：做了哪些任务、出现哪些错误、哪里需要人拍板。
2. 召开一次约一小时的规则会，只讨论真实案例，不讨论想象中的万能场景。
3. 沿用第 18 章的 L1 / L2 / L3 矩阵，把本部门常见任务放进去。
4. 逐项写清可以自动完成、必须停下等人、绝对不允许执行的事项。
5. 确定验收人、拍板人、归档位置和出错后的处理顺序。
6. 发布《部门 AI 使用约定》第一版，指定维护人和下次复查日期。
7. 选第二个任务重复前两步，验证规矩能否跨任务使用。

成功标志也很具体：成员遇到新任务时会先定级；L2 有清楚停点；L3 没有外部执行；同一类产物进入同一归档位置；出错后能找到输入、输出、验收人和拍板记录。

三步不能压成一次培训会。培训能解决“没见过”，解决不了“第一次失败怎么办”“部门认可什么结果”“谁能批准外部动作”。这些问题只能在真实任务里写成规矩。

三步推进看板

步骤	核心对象	主管具体动作	时间预期	成功标志	未通过时怎么做
一个人先跑通	一个低风险任务	给真实材料、冻结边界、要求保留失败记录	约 5 个工作日	同类输入可重跑，验收不靠个人记忆	缩小任务，补完成条件，再做样件
三个人抄作业	一套完整作业	组织独立执行和互换抽查	约 2 周	三人能独立做，验收结论接近	把求助问题写回模板，不急着扩人数
全组定规矩	一份部门约定	主持定级、责任和归档讨论	约 1 个月形成第一版	新任务会定级，产物有记录，外部动作有人批	用最近一次真实错误改约定，再复测

21.4 团队规矩怎么定

下面这份模板是本章最值钱的一页。它故意写得具体，因为“合理使用、注意安全、加强审核”无法指导任何一个动作。复制后，把方括号内容换成你部门的岗位、目录和审批方式。

模板直接沿用第 18 章的 L1 / L2 / L3 决策矩阵。第 18 章负责解释怎样定级和怎样验收，本章只把同一套矩阵扩展到团队分工。第一次使用前，建议先回看第 18 章场景 69 至 72。

《部门 AI 使用约定》的一页结构

复制后把方括号换成你部门的岗位、目录和审批方式

一、共同原则

每项任务开工前沿用第 18 章的 L1 / L2 / L3 定级；聊天里的已完成不算交付

二、哪些活可以全自动

按 L1 管理，前提是输入范围固定、错误可撤回、检查规则明确

三、哪些必须人拍板

按 L2 管理，它只列事实、来源、选项和各自影响，人在指定停点决定

四、哪些绝对不能碰

按 L3 管理：对外发布、放行判定、合同付款、人事决定，它只出事实清单

五、成果怎么归档

每个任务一个目录，保留任务、输入索引、提示词、产物、验收、拍板记录

六、例外怎么处理

无法定级先按 L3 处理；想跳过检查不允许口头批准，要书面写明替代动作

模板的验收方法：拿最近一项真实任务做五问，有一问答不上来就补具体岗位或路径

图 21-4 部门使用约定的一页结构：权限、拍板、禁区、责任、归档

图 21-4 《部门 AI 使用约定》的五块核心内容。作者根据公开资料绘制。

《部门 AI 使用约定》模板

[部门名称] AI 使用约定

版本: [V0.1]

生效日期: []

维护人: [岗位或姓名]

复查日期: []

适用范围: [团队、项目、资料范围]

一、共同原则

1. 每项任务开工前, 沿用第 18 章的 L1 / L2 / L3 决策矩阵定级。
2. AI 产物必须经过本约定规定的人工作业后, 才能进入正式交付目录。
3. 聊天中的“已完成”不算交付, 文件、记录和验收结果齐全才算。
4. 输入缺失、规则冲突、需要扩大范围时, 立即停下, 由指定人员决定。
5. 不以速度、使用次数或产物数量代替质量判断。

二、哪些活可以全自动

以下事项按 L1 管理, 前提是输入范围固定、错误可撤回、检查规则明确:

- [例: 内部会议记录整理成固定四栏]
- [例: 已批准目录内生成文件索引]
- [例: 按既有模板汇总周报素材]
- [例: 批量检查缺文件、重名和命名格式]

L1 执行要求:

- 开工前写明输入目录、输出目录和完成条件。
- 原始材料只读, 结果写入新文件。
- 完成后执行快速核, 报告总数、失败数、跳过数和异常项。
- 发现规则无法覆盖的输入, 转为 L2, 不得自行猜测。
- 产物进入正式目录前, 由 [岗位] 看一次检查结果。

三、哪些必须人拍板

以下事项按 L2 管理, AI 可以准备材料、列选项、生成草稿, 人必须在指定停点决定:

- [例: 试验方案编排]
- [例: 技术文件草稿]
- [例: 根因候选排序]
- [例: 供应商比选材料整理]
- [例: 规范冲突的处置方案]

L2 停点写法:

- 停点 A: AI 只列事实、来源、选项和各自影响, 不得选择。
- 拍板人: [岗位或姓名]。
- 拍板记录: 选择哪一项、理由、日期、依据文件。
- 停点 B: 执行完成后按第 18 章抽样核, 未通过项返回修改。
- 出现新选项或范围变化时, 回到停点 A, 不得沿旧批准继续。

四、哪些绝对不能碰

以下事项按 L3 管理。AI 只允许整理已提供的信息, 不得执行动作, 不得替人形成最终决定:

- 对外发布、发送正式文件或作出技术承诺。

- 量产放行、质量判定和安全相关结论的最终批准。
- 合同签署、下单、付款、报价确认和预算批准。
- 录用、辞退、调薪、绩效结论等人事决定。
- 修改生产环境、权限、账号、密钥和正式数据库。
- 删除、覆盖、改写原始试验记录与正式归档。
- 未经批准提供客户资料、供应商资料、个人信息和内部敏感文件。
- [本部门新增的 L3 事项]。

L3 执行要求：

- AI 只能输出事实清单、来源位置和可选方案。
- 最终判断、签字、发送、发布和系统操作由指定人员完成。
- 验收采用第 18 章的全量核加第二人。
- 发现工具已经发生外部动作，立即停止并报告负责人。

五、出问题谁负责

1. 任务发起人负责：提供真实输入，写清目的、范围和完成条件。
2. 定级人负责：按第 18 章矩阵确定 L1 / L2 / L3；拿不准时上提一级。
3. 执行人负责：按批准的任务单操作，保留输入、输出和过程记录；不得擅自扩大范围。
4. 验收人负责：执行规定检查，记录通过、未通过和未核实项；未通过不得放行。
5. 拍板人负责：对 L2 的选择和 L3 的最终决定写下理由并签名或留记录。
6. 主管负责：提供规则、时间和复盘条件；确认首次错误先查规则缺口。
7. 任何人跳过自己应做的动作，都要在复盘写明跳过的位置和影响。

问题发生后的顺序：

- 第一步，停止继续使用或外发错误产物。
- 第二步，列出影响对象和已收到产物的人员。
- 第三步，保留原输入、原输出、任务单、检查记录和拍板记录。
- 第四步，按第 18 章场景 72 查到第一个出错步骤。
- 第五步，把新增规则写回本约定、提示词或验收表，并用原案例复测。

六、成果怎么归档

统一目录：[完整路径]

每个任务一个目录，命名规则：

[日期]_[任务名]_[负责人]_[级别]

目录内至少包含：

00-task.md	任务目的、输入、范围、完成条件、级别
01-input-index.md	输入文件清单、版本、来源位置
02-prompt.md	实际使用的提示词和修改记录
03-output/	AI 生成的产物
04-check.md	人工验收结果、抽样位置、未通过项
05-decision.md	L2 / L3 的拍板内容、理由、日期
06-review.md	出错时的复盘与新增规则

归档纪律：

- 正式目录只接收通过验收的版本。
- 原始材料保持只读，不用 AI 产物覆盖原件。

- 文件名中不得写“最终版”“最新版”这类无法判断先后的词，使用日期和版本号。
- 聊天截图不能代替提示词原文、输入索引和验收记录。
- [岗位] 每月抽查 [若干] 个任务目录，记录缺项，不补造过程记录。

七、例外怎么处理

1. 新任务无法定级时，先按 L3 处理，由 [岗位] 决定。
2. 因项目紧急想跳过检查时，不允许口头批准；由 [岗位] 书面写明替代检查动作。
3. 发现输入资料不应提供给工具时，立即停止，不复制、不转写、不继续尝试。
4. 需要修改本约定时，提交真实案例、原条款和建议新条款，由 [岗位] 批准后改版。

八、版本记录

版本	日期	修改条款	触发案例	批准人
---	---	---	---	---
v0.1		初版	三人试用记录	

这份约定怎样落到现场

模板不能发完就算完成。第一次发布时，主管拿一个真实任务带全组走一遍：先定级，再看谁执行、谁验收、谁拍板，最后把文件放进约定目录。成员看到一次真实走法，才知道每一栏有什么用。

第二，约定要允许暴露“不知道”。任务无法定级、输入能不能提供拿不准、验收方法写不出，都先停。强迫成员当场给答案，只会逼他们把疑问藏起来。

第三，改约定必须有触发案例。有人提议加一条规则，要说清它防的是哪次错误；有人想删一条，也要说明这条为何造成无效工作。规则围着现场问题长，不围着想象长。

第四，主管自己也受约定约束。我有一次为了赶评审，让同事先把草稿发给我，却没有补拍板记录。第二天复查目录，缺的正是我要求别人必须写的那份记录。我补完后在组会上承认这一步是我漏的。主管若把自己放在规则外，团队很快也会把规则当装饰。

模板的验收方法

拿最近一项真实任务做五问：

1. 新人只看约定，能否判断它属于哪个级别？
2. 执行者能否说出自己什么时候必须停？
3. 产物出错后，能否在约十分钟内找到输入、提示词和验收记录？
4. L2 的选择能否找到拍板人和理由？
5. L3 的外部动作是否明确留在人手里？

有一问答不上来，就回到对应条款补具体岗位、路径或动作。不要再加“注意”“原则上”这类词，它们只会让文件变长。

21.5 把个人方法变成部门资产

一个人的好方法若只存在他的对话历史里，他休假后方法就停了。部门真正需要的是三类可复用资产：技能库负责“怎么连续做”，提示词库负责“怎样说清一项任务”，案例库负责“什么情况下有效或会失败”。

这里的“技能”指一组固定步骤、工具动作和检查规则，可以被重复调用。它比一段提示词多了执行顺序和验收动作。小白可以把它理解成部门自己的电子作业指导书。

技能库：存可重复执行的完整做法

技能库的最小单元是一件能稳定完成的小任务，不必从炫目的自动化流程开始。每个技能至少包含：适用范围、输入要求、执行步骤、停手条件、输出文件、验收清单、维护人、最近验证日期。

我建议技能目录按任务命名，不按工具命名。例如“会议事项提取”“规范版本差异”“试验日志缺口检查”。工具会变化，岗位任务更稳定。按工具命名，半年后目录里会出现一堆没人知道用途的文件。

一个技能进入部门库前，要过三道检查：先行者在不同材料上跑过；第二个人能不求助复现；维护人明确。缺维护人的技能宁可留在试验区，也不要标成正式可用。

技能卡模板：

技能名称：[按任务命名]
 适用岗位：[]
 适用输入：[文件类型、字段、目录要求]
 不适用情况：[]
 沿用第 18 章定级：[]
 执行步骤：[1、2、3.....]
 必须停手：[输入缺失、范围变化、判断冲突.....]
 输出文件：[名称与位置]
 验收动作：[]
 失败样例：[案例库编号]
 维护人：[]
 最近验证日期：[]
 状态：[试验 / 可用 / 暂停]

提示词库：存任务单，不存聊天花样

提示词库最常见的问题是数量越来越多，名字却都像“万能版”“增强版”“最终版 2”。成员找不到适用范围，只能随便挑一个。我的做法是给每条提示词加元数据，也就是描述它身份的几项信息。

每条提示词顶部写六项：任务、输入、输出、级别、验收、维护人。正文里方括号必须有填写说明。提示词修改后保留版本记录，说明改了哪条、由哪次错误触发。

一条提示词若只在作者手上成功，不进入正式区。另一位同事用自己的材料跑通后，才把状态从“个人试用”改成“部门可用”。这个门槛能挡住很多只在特定对话里有效的偶然结果。

提示词条目模板：

名称: []
用途: [一句话]
适用输入: []
预期输出: []
任务级别: [L1 / L2 / L3]
人工检查: []
维护人: []
版本: []
触发修改的案例: []
正文: [完整可复制提示词]

案例库：成功和失败各存一份

部门案例库不能只放获奖作品。成功案例告诉新人怎样做，失败案例告诉他哪里必须停。两者都要保留原输入索引、原提示词、产物、验收记录和改后的规则。

案例名称要让人能检索，例如“会议纪要漏责任人”“规范比较把格式变化误判为内容变化”“日志异常被自动补值”。不要写“案例 01”或“某次问题”，三个月后没人知道里面是什么。

失败案例入库时不追求写长报告，只回答六个问题：做什么、哪一步先错、为什么当时没发现、影响到哪里、加了哪条规则、复测是否拦住。涉及个人的讨论留在管理过程里，案例库聚焦任务和规则。

三库共建：技能、提示词、案例

技能管怎么连续做，提示词管怎样说清一项任务，案例管什么情况下有效或会失败



新人从技能入口开始，执行时复制提示词，碰到异常再查案例，新错误写回技能

图 21-5 三库共建：技能管步骤，提示词管任务单，案例管证据

图 21-5 技能库、提示词库和案例库的分工。作者根据公开资料绘制。

三库怎样一起工作

以“规范版本差异”为例。技能库里放完整步骤：读取两个版本、按条款编号比较、输出差异、执行抽查。提示词库里放可复制的任务单。案例库里放一次成功比较和一次误把页眉变化当条款变化的失败记录。

新人先从技能入口开始，执行时复制提示词，碰到异常再查案例。发现新错误后，把案例编号写回技能的“不适用情况”或验收清单。这样三类资产互相引用，不会变成三个各自长大的文件堆。

共建机制：每个人只多做一个小动作

共建若变成额外写长文，很快会停。我给团队定的最小动作是：使用者每次只补一个发现。提示词少一个输入说明，就补那一行；验收漏一种异常，就加一项；出现新失败，就建一张六问案例卡。

维护人每月做一次整理：合并重复条目、把长期未验证的项目转入暂停区、检查链接能否打开、提醒对应负责人复测。整理的目标是让人找得到、看得懂、照着能做，不追求条目多。

部门主管每季度挑几个被高频使用的任务复查。若工具、文件格式或流程已经变化，技能状态先转为“待复测”。过期方法继续挂着，比没有方法更危险，因为新人会把“正式库”当成已验证事实。

我还会保留个人试验区。新想法先在个人区跑，不要求一开始就写全所有字段。等它经过三人复制，再移入部门正式区。没有试验区，成员怕弄乱正式库，不愿贡献；没有进入门槛，正式库又会被半成品塞满。

21.6 怎么衡量有没有效果

团队推广最容易被一个漂亮数字带偏，比如“节省了约多少小时”或“使用次数增长了多少”。时间重要，但它只说明动作可能变快，没有说明返工有没有增加、错误有没有外流、是不是只有两个人在用。

第 16 章讲过反指标：主指标旁边要放一把专门查取巧的尺子。团队推广也一样。你看节省时间，就同时看返工时间；你看使用人数，就同时看任务是否适合；你看产出数量，就同时看验收未通过数。

部门 AI 使用效果衡量表

维度	指标	计算或记录方法	数据从哪里来	配套反指标	主管要问什么
时间	单项任务人工投入变化	比较同类任务的准备、执行、验收总时长	任务卡与工时记录	验收时长、返工时长不能被删掉	省下的是搬运时间，还是少做了检查
质量	首次验收通过情况	记录通过、未通过、未核实三类	04-check.md	不得降低门槛或少抽样	通过增加是否来自规则更清楚
返工	返工次数与返工原因	每次返工记任务、原因、耗时	复盘记录	小修不能被拆成多个“成功任务”	同类返工是否重复出现
错误	内部发现与外部发现的错误	按发现位置和影响范围记录	验收表、问题记录	不以少报告异常来改善数字	错误变少，还是大家不愿上报
覆盖面	实际使用岗位与人数	统计当月完成过合格任务的人和岗位	归档目录	不把一次打开或一次提问算采用	是否只集中在少数爱好者

维度	指标	计算或记录方法	数据从哪里来	配套反指标	主管要问什么
任务面	可稳定复用的任务数	进入正式技能库且完成复测的任务	技能库	未复测、无维护人的不计入	新增任务是否仍有明确边界
责任	定级、验收、拍板记录完整情况	抽查任务目录的必备文件	月度抽查	不允许事后补造记录	出问题能否找到责任动作
学习	方法复用与案例贡献	记录模板被他人复用和规则修订	三库版本记录	不用条目数量代替质量	方法离开原作者还能不能用

部门 AI 使用效果衡量表

主指标旁边必须放一把专门查取巧的尺子

维度	指标	配套反指标
时间	单项任务人工投入变化	验收时长、返工时长不能被删掉
质量	首次验收通过情况	不得降低门槛或少抽样
返工	返工次数与返工原因	小修不能被拆成多个成功任务
错误	内部发现与外部发现的错误	不以少报异常来改善数字
覆盖面	实际使用岗位与人数	不把一次打开或一次提问算采用
任务面	可稳定复用的任务数	未复测、无维护人的不计入
责任	定级、验收、拍板记录完整情况	不允许事后补造记录
学习	方法复用与案例贡献	不用条目数量代替质量

先看记录是否真实，再看同类错误是否减少；第一阶段数字变差常常说明团队开始认真了

图 21-6 效果衡量仪表：时间、质量、返工、错误、覆盖、责任一起看

图 21-6 部门效果衡量的多指标仪表。作者根据公开资料绘制。

先记基线，再谈改善

没有基线，任何“提升”都只能靠印象。选定首批任务时，先记录一次原人工做法：输入规模、总用时、返工次数、检查方法、产物去向。AI 参与后仍按同一口径记录。

总用时要包含准备、执行、等待、验收和返工。只记工具运行时间，会把人的整理和检查藏掉。我见过一次看起来只花约十分钟的任务，输入清理和结果核对却用了约一小时。它仍可能有价值，但不能报成“十分钟完成”。

基线也不要求精确到每一分钟。团队可以用任务开始与结束时刻、日程记录和文件修改时间做近似，只要前后口径一致。对管理判断来说，持续记录的粗数据比临时编一组漂亮小数更可靠。

别用单一指标

只看节省时间，会鼓励人少做检查；只看使用次数，会鼓励人把简单提问也算一次任务；只看产物数量，会鼓励拆文件；只看错误数量，又可能让人少报错误。

我给每个主指标至少配一个反指标：

主指标	容易出现的取巧	反指标
单项耗时下降	少做验收、漏记返工	验收动作数不减少，返工时长单列
使用人数增加	强迫所有人登录一次	只统计完成并归档过合格任务的人
正式技能数增加	半成品也进入正式区	必须经第二人复现，有维护人和复测日期
首次通过增加	放宽门槛、把未知算通过	验收门槛冻结，未核实单独统计
错误数量下降	异常不记录、内部处理后消失	主动上报数与外部发现数分开看

有些数字短期变差，反而说明团队开始认真了。刚实施验收表时，未通过项可能增加，因为过去很多问题没有被记录。主管若看到数字变差就批评，成员会马上学会少写。第一阶段先看记录是否真实，第二阶段再看同类错误是否减少。

一张月度复查卡

【月份】[]

【首批任务】[]

1. 时间

人工基线总时长: [约]

本月准备 + 执行 + 验收 + 返工总时长: [约]

省下的时间去了哪里: []

2. 质量

通过: []

未通过: []

未核实: []

外部发现错误: []

是否修改过验收口径: [是 / 否, 说明]

3. 覆盖

完成过合格任务的人数: []

覆盖岗位: []

只使用一次后停止的人数: []

停止原因: [责任 / 不会选任务 / 首次失败 / 与岗位无关 / 其他]

4. 资产

新增正式技能: []

第二人复现完成: []

新增失败案例: []

超过复测日期的条目: []

5. 下月只改一件事

最常见的阻力或错误: []

准备修改的规则: []

负责人: []

复查日期: []

预期输出是一张能看出趋势的月度卡，不需要另写长汇报。每月只选一个最常见问题改，避免同时追十个目标。连续几个月后，你会看见到底是任务选择有问题、培训有问题，还是验收和责任边界有问题。

怎么向上汇报

部门负责人向公司汇报时，建议讲三件事：哪些任务已经稳定复用，质量和返工怎样变化，下一步准备扩到哪里。不要只展示工具页面，也不要只报一个节省小时数。

一个可信的汇报句式是：“本月在三类内部任务上使用，已有多名成员独立完成；节省主要来自资料整理，验收动作保留；发现的主要问题是状态字段不统一，已写入模板并复测。”这里有范围、有结果、有问题，也有下一步。

如果高层只追问“到底省了多少人”，部门负责人要把话题拉回任务和产能：省下了哪些重复动作，释放的时间投入了哪些原本欠缺的验证，质量是否变稳。用任务事实回答，比承诺一个无法负责的比例更可靠。

21.7 常见翻车

翻车一：强制全员使用

症状通常是登录人数很快上升，真实产物没有增加。成员为了完成要求，随便问一个问题、截一张图，就算完成。主管看到的是热闹，团队学到的是怎样应付检查。

补救动作：停止统计登录和提问次数，改为自愿报名首批任务。只统计经过验收并归档的合格产物。优先让有真实痛点的人先做，再用成功作业吸引相邻岗位。

强制还会放大被替代的担心。有人尚未知道工具能帮自己做什么，就先收到“必须用”的命令，他会把这件事理解成能力审查。推广的第一目标应是让一个任务变轻，不是让一张人数表变满。

翻车二：只推工具不推方法

症状是培训讲了入口、按钮和功能，散会后成员仍不会选任务。大家各写各的提示词，产物格式不同，出错也没有共同记录。工具一更新，培训材料又要重来。

补救动作：培训只用一个真实任务，从输入、定级、提示词、产物、验收到归档完整走一遍。会后交付一套能抄的作业，让成员用自己的材料复现。

我见过一次约两小时的功能培训，现场反馈很好，第二周几乎没人继续用。后来改成约四十五分钟的任务实操，虽然只讲会议事项提取，反而有人把同样结构迁移到了评审意见和问题清单。人需要的是完成一件活的方法，不是记住菜单。

翻车三：没有验收规矩

症状是每个人凭感觉看结果。有的人逐条核，有的人扫一眼，有的人把排版整齐当作正确。发生错误后，团队只能争论“当时为什么没看出来”，却找不到哪项检查本来应该做。

补救动作：回到第 18 章。按出错代价选择快速核、抽样核或全量核；任务开始前确定验收档，不在看到漂亮结果后降档。部门约定里写清谁验、验什么、结果存哪里。

没有验收规矩时，主管越强调负责，成员越不愿意用。因为“负责”变成一个没有边界的要求。验收表把责任变成动作：核哪几项、抽哪些条、谁签字。动作能完成，口号不能。

翻车四：拿 AI 产出当结论直接对外

症状是内部草稿没有经过来源检查，就被复制到客户材料、供应商邮件或正式报告。最危险的地方常藏在限定词、版本号和样本范围里，它们会被写得过于确定。

补救动作：所有对外内容按 L3 管理，AI 只准备材料。责任人逐项核来源，必要时加第二人检查；发送、发布和技术承诺由人完成。对外文件的任务目录必须保留输入索引、验收记录和拍板理由。

我在第 18 章已经讲过一次被追问来源的教训。团队层面要再加一层：不能只靠某位谨慎的工程师记得检查。外发前的检查应写进共同约定，换一个人、换一个项目也照样执行。

翻车五：主管只看成功，不允许报失败

症状是分享会上全是漂亮结果，案例库里没有一次错误。真实使用中大家各自踩坑，却不敢留下记录。几个月后，同样的问题在不同人手里重复发生。

补救动作：每次分享固定留一个失败案例。主管先讲自己的失误，再请成员说哪条规则没写清。失败案例评价的是复盘质量，不按“谁犯错最多”排名。

一次三人试用里，有人把缺失字段如实写进失败栏，另一个人为了让结果完整，手工补了一个推测值。如果只看页面完整度，后者更漂亮；如果看可追溯性，前者才合格。主管表扬谁，团队就会学谁。

翻车六：资产库越建越大，没人维护

症状是提示词上百条，搜索结果里有多个同名版本；技能写着可用，实际输入格式早已变化；案例链接打不开。新人试一次失败，就不再相信部门库。

补救动作：正式区设置进入门槛和退出规则。没有第二人复现的不进正式区；超过复测日期的转入待复测；连续一段时间无人使用且任务已取消的移入归档区。每项都要有维护人。

库的价值取决于可用条目，不取决于总条目。我宁愿部门只有十个人人会用的小技巧，也不愿有一百个无法判断版本的提示词。

小白词典

团队采用：一项方法离开最早使用的人，其他成员也能在真实工作中重复使用。类比：一道菜不只厨师长会做，按作业单换个人也能做出来。

切入任务：团队最先拿来试用的一小类工作，要求高频、重复、出错代价低、结果容易检查。类比：学游泳先在浅水区练换气。

任务地图：按岗位列出可尝试任务、输入、产物和风险的一张表。类比：商场楼层图，让人先找到与自己有关的区域。

技能库：保存完整执行步骤、停手条件和验收方法的部门目录。类比：可重复使用的电子作业指导书。

提示词库：保存经过复现、可以直接填写使用的任务单。类比：常用表单柜，选对表再填内容。

案例库：保存成功和失败的真实记录，用来证明方法何时有效、哪里会出错。类比：维修手册里的故障实例。

维护人：负责检查某项方法是否仍能使用、版本是否清楚的人。类比：工装有保管人，过期和损坏要有人处理。

覆盖面：真实使用已经到达多少岗位和成员，不把一次打开工具算进去。类比：一条新工艺在哪些工位真正执行过。

反指标：用来发现主指标被取巧做好的检查项。类比：看到速度提高时，同时检查漏检和返工有没有增加。

归档：把任务输入、提示词、产物、验收和决定按统一结构保存。类比：试验结束后把原始记录、报告和签字放进同一个档案袋。

你的岗位怎么用

研发主管：先从组内每周重复的资料整理、差异检查或问题跟踪里选一个任务。亲自审定级和验收表，让一位实际执行者跑通，再安排两位同事独立复现。你的重点是接住第一次小错，把缺口写回规则。

项目经理：把 AI 任务放进项目计划时，同时写负责人、任务级别、停点、验收人和归档位置。周会上不收“用了几次”，只看合格产物、未通过项和需要拍板的记录。对外材料统一按 L3 管理。

部门负责人：批准《部门 AI 使用约定》，明确省下时间的去向，不做个人速度排名。每月看时间、返工、错误、覆盖面和记录完整情况，避免用单一节省数字做结论。方法共建和案例贡献要进入团队工作总结。

质量或流程负责人：把现有检验和文件控制经验搬到 AI 产物上。抽查任务目录是否有输入索引、验收记录和拍板理由；发现重复错误，就推动修改模板或技能，不用一句“加强检查”收尾。

本章交付物

《部门 AI 使用约定》模板。

交付物就在 21.4，可整段复制。第一次填写时，不要从空想开始，拿三人试用阶段的真实记录，把本部门的 L1、L2、L3 任务、岗位、目录和审批方式填进去。

完成后用最近一项任务做演练。能在约十分钟内找到定级、输入、提示词、产物、验收和拍板记录，这份约定才算能用。找不到哪一项，就补具体路径和责任岗位，再发布第一版。

建议打印约定中的五块摘要：哪些可自动、哪些必须人拍板、哪些绝对不能碰、出问题谁负责、成果放哪里。完整版放部门共享空间，摘要放在成员每天能看到的位置。

本章金句

团队推 AI，先卡住的是人心里的责任账，工具功能还排在后面。

第一个任务要选容易赢的；一次可验收的小成功，比一场热闹的功能演示更能改变团队。

三个人能照着同一份作业做成，个人经验才开始变成部门方法。

速度省下来以后花在哪里，决定了成员把 AI 看成助手，还是看成威胁。

只看节省时间，会把验收和返工藏起来；团队效果必须同时看质量、错误和覆盖面。

规矩写到岗位、动作和目录才有用，写成“加强审核”只会多一页纸。

（下一章：成本怎么算。团队开始稳定使用后，下一步要把工具、时间、维护和返工一起算进账。）

第 22 章

这笔账到底怎么算

订阅费从来不是最贵的那一笔。真正吃掉预算的，是那些省下来却没能变成产出的时间，和你试错赔进去的那个下午。

开场：我第一次算错了，而且错得很自信

我第一次算这笔账，是在一个周四下午的部门周会上。

同事问我：Eric，你天天鼓捣这个 AI，它到底值不值那笔订阅钱？我当时挺得意，掏出手机按了几下计算器。订阅费每月是三位数人民币的量级；我估了一下自己一个月靠它省下来的时间，按我的时薪折成钱，是订阅费的好几倍。我把这个数报出来，会议室里有人说“这么划算？”。

那天晚上我自己在工位上又算了一遍，发现错得很彻底。

第一个错：我把省下的每一分钟都按满额时薪折成了钱。可那一天下午省出来的两小时，我实际干了什么？提早收工，泡了杯茶，刷了会儿手机。它没有变成任何产出，公司没有因此多收到一分钱。

第二个错：我没算学习时间。为了把第一个真场景跑通，我赔进去两个傍晚，还有一个下午在跟报错死磕。那段时间按我的时薪折下来，接近小半年的订阅费。

第三个错：我没算返工。有一份它整理完的表格我觉得没问题，直接发出去了，结果漏了一行重复数据。第二天同事来问，我又花了约一小时回去核整张表。那次返工，一次性吃掉了那个场景两个月的全部净收益。

半年后我用三本账重新算了一遍。结论还是正收益，而且收益相当可观，但理由跟那天的会议室版本完全不同：它赚钱不是因为我省了多少时间，而是因为我几个频次最高、最枯燥的活彻底交出去了，从此再没自己动过手。

这一章就讲这套算法。它是我在第 1 章 1.5 节随口提过的那句“你的活越值钱，这笔账越好算”的完整版本。

22.1 三本账：时间账、订阅账、试错账

绝大多数人算这笔账只算了订阅账，因为它最好算：账单上有数字，一眼看得见。另外两本账藏在水面下，却常常决定这笔投资是赚还是亏。

时间账：省下的时间必须打折

时间账的公式看起来很简单：

省下的时间 × 你的时薪 = 省下的钱

这个公式里藏着一个陷阱。省下来的时间会自动流向三个地方，只有其中一部分能变成钱。

省下的时间去哪了	折现系数（我的经验取值）	说明
直接变成能交付的产出	0.6 到 1.0	比如报告当天就能发出去，客户能验收
变成你多接的别的活	0.3 到 0.5	省了两小时，但这两小时马上被下一个任务吃掉
变成早点下班、发呆、摸鱼	接近 0	对你个人有价值，对公司财务没有

折现系数这个词是我借来的，意思很朴素：省下的时间里，真正能折成钱的比例。（第一次出现的术语都在章末小白词典里。）

我在现实的研发部门里观察到的情况是，绝大多数人的时间流向第二种。所以第一次算账，我建议折现系数一律取 0.3 到 0.5 之间，别给自己的乐观留太多空间。跑熟半年之后，如果你确实能证明省下的时间变成了产出，再往上调。

唯一的例外是能直接变现的活。我自己接过几份外面的技术咨询和书稿，那种时候我写出来的东西当天就能开票，折现系数可以取到接近 1。

我自己踩过的一个坑也在这一栏里：AI 耗时必须把验收时间算进去。很多人在计算时只记“它跑完用了 3 分钟”，忘了自己还有十几分钟的验收动作。在第 1 章开头那个 2000 行表格的例子里，它跑完只要几分钟，我验收核对 37 行花的时间跟它差不多。忽略验收，你的时间账会虚高一倍。

这套系数不是我拍脑袋定的。有一阵子我连续三周做了个小实验：每天记录自己因为 AI 省下了多少时间，再记录这些时间后来干了什么。三周下来的结果是，真正落在“多交付了一件东西”上的部分，不到省下总量的一半，剩下的全部被临时插进来的活吃掉，包括临时会议、同事求助、以及我自己临时想起来的杂事。

这个实验结果让我改了一个习惯：先给省下的时间准备去处，再去省它。如果你明知这段省下来的时间只会被别的事吃掉，那它在账面上就该按低折扣记；如果你把它明确安排给了某个具体的交付物，它才配得上高折扣。这也解释了为什么同一个场景在不同人手里算出完全不同的收益，时间没有差，去处有差别。

订阅账：看得见的那一笔反而最不容易漏

订阅账是三本账里最容易列全的，因为它有一半写在账单上。

- 软件订阅：按月或按年付的费用。这是唯一有发票的一项。
- 按量调用费：模型调用按处理量计费，用得多了就付得多。它跟订阅费平行存在，容易被忘掉。
- 硬件：如果你要把模型跑在本地（有些单位的资料不能传到任何外部服务，这是常见做法），一次性的硬件升级可能是一笔真正的开销，通常由部门固定资产采购，不进你的个人账。
- 多人授权：团队用的时候，人数直接乘上去。第 21 章讲过这一点，这里只是提醒你别忘了。
- 第三方服务费：某些连接器、某些数据服务是按次收费的，金额通常不大，但累计起来会给你一个惊喜。

订阅账的关键难点不是列全，是分摊。你一个月付一份订阅费，用它在五六个场景里干了活，那么每个场景该背多少？我的做法是平摊到底：这个月你在 N 个场景上用过它，就每个场景背 $1/N$ 。这个做法粗糙，但它足够保守，而且能立刻暴露问题：一个每月只用一次的场景，背上整份订阅的几分之一，立刻就变成负收益。

顺带说一个我见过的真实错漏：有人把“团队的钱”当成“别人的钱”。他们部门的 AI 订阅走的是部门预算，于是每个人算自己的收益时都默认订阅费为零，最后每个人都算出“赚翻了”，年底部门一汇总才发现整体是亏的。订阅费不会因为付钱的人不是你就不存在，它只是换了个地方躺着。

另一个容易翻车的地方是按量调用费。它的特点是平时几乎看不见，一旦你把一个“每天跑一次”的流程挂上去，它会在月底变成一笔像模像样的钱。我的经验是把这类自动化流程单独列一行订阅分摊，别让它混在日常对话用量里。

试错账：唯一能一次性吃掉你半年收益的一本

试错账有三层，从轻到重：

第一层是一次性学习成本。学会一个工具的时间，通常集中在头一到两个月，之后趋近于零。它是一次性的，所以能被时间摊薄，这也是唯一值得乐观的一层。

第二层是返工成本。你相信了它的输出，结果有错，你得回去重看一遍，或者干脆重做。返工的特点是它跟你的熟练度负相关：越用越懂它的脾气，返工率越低，但永远不会降到零。

第三层是错误代价。这是三本账里唯一带着尾部风险的一层，也是最容易被忽略的一层。它的算法是：

$$\text{错误代价} = \text{单次出错的损失} \times \text{出错概率}$$

出错概率通常不高，可能百分之几的量级；但单次损失可能是几个月的收益。举个例子：一份给客户的正式报价，金额或者单位写错一个，你要付出的代价远不是“再改一遍”能概括的。第 18 章讲过的规矩在这里同样成立：AI 交出来的东西，你验过才算你的。

我自己挨过一次小的。那是一份发给外部单位的技术说明，里面引用了一段标准的条款号。它写得非常笃定，格式也很专业，我扫了一眼觉得没问题就发了。第二天对方工程师回邮件问了一句：“这个条款号对应的是标准的哪一版？”我回去一查，那条号压根不存在。真正贵的地方不是那封更正邮件，是我在对方心里丢掉的一点信任，那种东西没法折算成钱，也不会出现在任何账单上。

从那之后我给自己加了一条规矩：凡是往外走的文件，引用必须能追溯到源头的原句。它会多花我几分钟，这几分钟是账上实打实的支出，同时也是三本账里最划算的一笔保险费。

三本账：两本藏在水面下

绝大多数人只算了订阅账，因为它最好算

订阅账：看得见的那一笔

软件订阅、按量调用费、硬件、多人授权、第三方服务费；账单加总后按场景分摊

时间账：省下的时间必须打折

人工耗时减 AI 耗时，乘频次，再乘折现系数；最容易漏的是把验收时间算进 AI 耗时

试错账：学习加返工加尾部风险

错误代价等于单次出错的损失乘出错概率；概率可能百分之几的量级，单次损失可能是几个月的收益

看得见的那一笔往往不是最大的一笔；先给省下的时间准备去处，再去省它

图 22-1 三本账的结构：订阅账在水面上，时间账和试错账在水面下

图 22-1 三本账：看得见的那一笔往往不是最大的一笔。（作者根据公开资料绘制）

三本账对照表

	时间账	订阅账	试错账
记账单位	小时	元 / 月	元 或 小时，混着记
怎么估	$(\text{人工耗时} - \text{AI 耗时}) \times \text{频次} \times \text{折现系数}$	账单加总后按场景分摊	学习一次性 + 返工按次 + 尾部风险按期望值
最容易被漏算的地方	忘记把验收时间算进 AI 耗时	忘记按量调用费和多人授权	完全没想过出错怎么办
什么时候它会变大	你的时薪越高，它越大	用的人越多、活越重，它越大	出错代价越高，它越大
怎么压低它	提高验收效率，别提高期待	合并场景、砍掉低频场景	给高风险活加一道人工复核

这张表有一行值得多说一句：时间账会随你的时薪放大。这意味着同一个场景，在月薪五千上下的人手上是亏的，在月薪三万上下的人手上可能是赚的。这一条是整章的核心，22.6 节会回到它。

22.2 一套能直接套用的算法：单场景成本收益表

把三本账合起来，就是下面这张表。我在自己的电脑里存着一个这样的表格，每上线一个新场景就加一行，月底花十分钟看一遍。

场景名	人工耗时	AI 耗时	每次省时	每月频次	月省时	折合金额	订阅分摊	净收益

九个格子，每一个都有明确的填法。下面逐格讲。

单场景成本收益表：九个格子

每上线一个新场景加一行，月底花十分钟看一遍

① 场景名

写到一个动作能说清的粒度；写得太粗，月底没法单独砍掉

② 人工耗时

自己从零干到能交付，取三次平均，前面加约

③ AI 耗时

写提示词加等它跑加你验收，三段全算

④ 每次省时

人工耗时减 AI 耗时；一次省不到 15 分钟别上

⑤ 每月频次

翻日历和邮件记录统计，不靠印象

⑥ 月省时

每次省时乘每月频次；一年做一次的活要摊到每月

⑦ 折合金额

月省时乘时薪乘折现系数，系数先取 0.3 到 0.5

⑧ 订阅分摊

当月订阅总费除以本月实际用过的场景数

⑨ 净收益

折合金额减订阅分摊，再减当月返工损失

第一个月的数据一定是亏的，学习成本全压在这个月；至少看三个月，看趋势

图 22-2 单场景成本收益表：九个格子怎么填

图 22-2 单场景成本收益表模板。（作者根据公开资料绘制）

九个格子怎么填

格子	怎么填	常见错误
场景名	写到一个动作就能说清的粒度。"整理月度试验数据"，合格。"提升效率"，不合格	写得太粗，月底没法单独砍掉某一条
人工耗时	你自己从零干到能交付的时间，不含被打断的部分。取三次的平均，写"约"	按"记忆中最慢那次"记，把 AI 衬托得过于神勇
AI 耗时	写提示词 + 等它跑 + 你验收，三段全算	只记它跑的时间，漏掉验收，结果虚高一倍

格子	怎么填	常见错误
每次省时	人工耗时减 AI 耗时	一次省不到 15 分钟的活，建议直接别上 AI
每月频次	翻日历和邮件记录统计，不靠印象	高估频次是最常见的自欺，我自己的印象值经常是实际值的两倍
月省时	每次省时 × 每月频次	忘了把"一年做一次"的活摊到每月，月省时会虚高十倍
折合金额	月省时 × 时薪 × 折现系数	折现系数直接取 1，把愉快的心情折成了钱
订阅分摊	当月订阅总费 ÷ 本月实际用过的场景数	把整份订阅算给唯一那个你最爱的场景
净收益	折合金额 - 订阅分摊 - 当月返工损失	返工损失不记，等于默认自己从不犯错

时薪怎么定。一个够用的算法是：税前月薪 ÷ 22 天 ÷ 8 小时。得到的数字别直接当成最后的金额，它是个换算尺：同一个省时数字，在谁的工位上值多少钱。

关于这个数字有两句提醒。第一，用税前不用税后，因为它衡量的是你给公司创造的价值，不是到手的钱。第二，别用"公司给我的成本"那种更夸张的算法去跟老板汇报，那种算法在工程上叫自欺：你省的时间如果被证明值那么多钱，公司的第一个反应通常是问你为什么以前不这么干。

下面三个例子，两个正的，一个负的。前两个是我真跑过的场景，第三个是我亲手砍掉的场景。

例一：月度试验数据整理 ★★★

这是第 15 章的场景，我跑过很多次，是我最早稳定的一类活。以下数字是示例演算，用的是我的经验取值，你套自己家的情况要换成自己的数。

- 人工耗时：约 90 分钟一份，包括挑超差行、排序、画图、贴进模板。
- AI 耗时：约 25 分钟。其中写提示词约 5 分钟，它跑约 5 分钟，我验收抽查约 15 分钟。
- 每次省时：约 65 分钟。
- 每月频次：约 4 次。
- 月省时：约 4.3 小时。
- 折合金额：时薪按示例值 150 元，折现系数取 0.5（省下的时间基本都被下一个任务吃掉了），折合约 320 元。
- 订阅分摊：当月我在六个场景上用过它，订阅费三位数人民币的量级，摊到这个场景约 40 元。
- 当月返工：头两个月有过一次（漏了一行重复数据），之后为零。
- 净收益：约 280 元 / 月。

这个场景我自己给它的评价是：省心大于省钱。它真正的收益不在那两百多块，在于我不再需要一个完整的下午坐在那里做机械的挑选和排序。

例二：供应商月度资质核查 ★★

这是第 12 章的场景，靠它操控浏览器去几个门户网站翻数据。

- 人工耗时：约 120 分钟，主要是翻页、对照、往表里抄。
- AI 耗时：约 20 分钟，其中验收占一半（要抽查几行是不是抄对了，这是第 17 章那类“看着挺像真的”的毛病最容易钻进来的地方）。
- 每次省时：约 100 分钟。
- 每月频次：约 2 次。
- 月省时：约 3.3 小时。
- 折合金额：同样按示例时薪 150 元、系数 0.5，折合约 250 元。
- 订阅分摊：约 40 元。
- 当月返工：约半小时（有一家门户改版，它抓错了一个字段，我抽查时发现了）。
- 净收益：约 190 元 / 月。

注意这个场景的频次只有两次。它之所以还能算正收益，靠的是单次的省时足够大。这就是为什么我在下一节里给的判据是“频次 × 单次省时”相乘，而不是只看频次。

例三：一个反例，年度盘点流程 ★

同一个表里，我也留了一行负收益的。这是我刻意保留的，它提醒我别把每个新点子都当成机会。

- 人工耗时：约 4 小时，一年一次的盘点报表。
- AI 耗时：约 2.5 小时。头一次做，写提示词、教它认咱们的表头、来回修格式，占了绝大部分。
- 每次省时：约 1.5 小时。
- 每月频次：约 0.08 次（十二分之一）。
- 月省时：约 0.12 小时。
- 折合金额：约十元量级。
- 订阅分摊 + 学习时间摊到这一次：超过百元量级。
- 净收益：明确为负。

这一行的价值是它让我在第二次想起来“能不能把这个也交给 AI”的时候，直接放弃了。有些账，算一次就够了。

填表的三个提醒

提醒一：这张表要每月更新，不要一次性算完就锁起来。频次这一栏三个月不看就会失真。我自己是在每个月的最后一周花十分钟过一遍，顺便把当月返工记进去。

提醒二：第一个月的数据一定是亏的。因为学习成本全压在这一个月上。别在第一个月就下结论，我建议至少看三个月，看的是趋势而不是某个月的绝对值。

提醒三：团队用这套表时，按人分别建行。同一个场景在不同人手上的频次差别极大，合并记账会把最常用的人的收益平均掉，也会把偶尔用的人的成本藏起来。我的做法是每人一张表，月底合起来看部门总数。

22.3 四种明显不划算的活

下面这四类活，我自己算下来基本都是负收益。给它们各自配了一条判据，你可以直接照着量。



图 22-3 四类不划算的活：先看频次，再看出错代价

图 22-3 四类不划算的活。（作者根据公开资料绘制）

一、一年做一次的活

判据：每年频次在六次以内，且每次省时不到半小时。两个条件同时满足，就别上 AI。

原因是上一节那个反例已经算过的：学习成本摊不薄。教它认你的表格格式所花的时间，比你自己干一遍少不了多少，而它下一次干这活要等到明年，那时候格式又变了。

还有一个二阶效应值得提一句：低频活会让你的肌肉记忆断掉。我每年只做两次的一类汇总，每次都要重新回忆当时是怎么跟它交代的，提示词写不出来了，只能从头试。这种“重新想起怎么用”的时间，账上没有专门的格子，它就混在 AI 耗时的前几项里，让人误以为是 AI 变笨了。

同类还有一件事要提醒：一次性的活容易给人错觉。你会在做的过程中觉得“这个 AI 真聪明”，因为它的表现确实新鲜。新鲜感不是收益，把它从账里拿掉。

二、出错代价极高的活

判据：这份东西一旦错了，需要有人出面道歉、改文件、或者重做一批实物。典型的是给外部客户的正式报价、要签字的合规文件、跟安全相关的判定。

这类活的特征是：AI 能把主体干得很好，但你得逐字复核，而逐字复核的成本常常跟你自己重做差不多。折下来，净收益趋近于零甚至为负。

有一点要说清楚：这类活可以用 AI 打草稿，别用 AI 出终稿。让它写第一版，你在这个基础上改，跟从零写相比还是省时间的；让它出终稿直接签字，那是把错误代价的期望值整个揽到自己身上。第 18 章给过的规矩在这里是硬性的：谁签字谁验收。

给这类活算账有个小技巧：把复核时间直接当成全部 AI 耗时。也就是说，假设 AI 部分根本没省时间，只保留“它帮你起了一个头”的收益。这样算出来的数字如果还为正，那才是真的能省钱；如果算下来是零，说明这个活适合用 AI，只是不适合用 AI 来省钱。

三、本身就要人判断的活

判据：这件事的最后一步是“我决定”，而不是“它算出来”。

举个例子：供应商定点（选哪家供货）这件事。AI 可以把五家的报价、交期、历史不良率整理成一张对比表，这一步的收益非常实在。但“选哪家”这件事，涉及商务关系、产能风险、领导的偏好，这些东西不在它读到的任何文件里。

再举个例子：一批零件超差了，让步接收还是退货。AI 能帮你把风险评估的模板填好、把过往类似案例找出来；签“让步”那两个字的是你，扛后果的也是你。

这类活的正确用法是：让 AI 把决策所需的材料准备好，决策权留在你手里。这在时间账上依然是正收益，只是记账时要记得把“我自己做判断的时间”单独留在人工那一侧。

这里有个很容易被忽略的坑：交出去的时间往往不只是 AI 那一段，还包括你重新建立判断能力的时间。如果你连续三个月没看过某类数据，只看了 AI 给你的一页总结，那么当有一天你需要自己做决定的时候，你得花比从前更长的时间回到那个状态。这笔时间没有人记账，它直到你要用的时候才一次性出现。

四、信息不能外传的活

判据：这份资料是否允许离开公司的网络边界。未公开的报价、含个人信息的名单、客户给的内部图纸、还没有公开的量产节奏，都在这一类里。

这类活的处理方式不是放弃，是换路径：

- 第一条路：把模型跑在本地。这是最彻底的做法，代价是硬件投入和一点性能损失。
- 第二条路：脱敏后再交给它。零件号换成代号，客户名换成 A 厂 B 厂，具体金额换成占某基准的比例。我在写第 14 章的场景时就用这招：让它做的事是“整理结构和逻辑”，那么真实的编号它并不需要知道。
- 第三条路：只让它干不碰数据的部分，比如写脚本、写模板、写流程。脚本本身不需要知道你的数据长什么样。

如果你的行业要求连脱敏后的结构都不允许出网，那就不用第一条路。别在这件事上存侥幸，因为这类错误的代价落在"错误代价"那一层，一次就能吃掉你几年的收益。

顺带说一句关于删数据的常识：跟任何外部服务打交道之前，先搞清楚你发过去的东西存在哪里、存多久、能不能删。这几句话在任何工具的服务条款里都找得到，花十分钟读完，比事后焦虑有用。这一条本来是合规常识，跟节不节约无关，但因为它的代价无法估上限，我把它放在这张清单里。

四类活的分辨顺序是有先后的：先看第四条能不能做（信息能不能出去），再看第二条出错代价高不高，然后看第一条频次够不够，最后才问第三条是不是需要你拍板。四条全过，才值得放进成本收益表里跑一个月。下面这张清单是把四条判据压成一页，可以直接抄下来贴在显示器边上。

不划算清单（可以直接抄下来贴在显示器边）

类型	量化判据	为什么亏	替代做法
一年做一次	每年 < 6 次 且 每次省时 < 30 分钟	学习成本摊不薄	老实自己干，或者干脆简化流程本身
出错代价极高	错了要道歉、要改文件、要重做实物	逐字复核的成本约等于重做	让它打草稿，你自己出终稿
本身要人判断	最后一步是"我决定"	决策信息不在它读到的文件里	让它备材料，你做决定
信息不能外传	资料不允许离开公司网络	泄密的代价无法估上限	本地跑 / 脱敏 / 只让它写脚本

22.4 免费与付费的边界：三个判断点

免费的模型服务现在到处都是，能力也不弱。什么时候该掏钱，我给自己定了三个判断点。

先说一句立场：免费版不是试用版，它是一个能长期用的正式版本。很多人把免费当成"先用几天再说"，于是随手把自己最复杂的活丢上去，跑不动就下结论说这工具不行。公平的判法是拿你真实的日常活去试，而且是错得起的那类活。真到某一天免费版开始拖你的节奏了，再谈钱。

免费与付费的三个判断点

免费版不是试用版，它是一个能长期用的正式版本

继续用免费版

- | **每月只花两三个小时**
免费版完全够用，别急着买
- | **错了也无所谓的活**
写周报、整理表格这一类
- | **回本工时算不过来**
省不出那几个小时，就别买或者降级
- | **先用真实日常活去试**
而且是错得起的那类活，别一上来丢最复杂的

该考虑付费

- | **每月累计八到十小时以上**
免费版的限制会打断节奏，这笔落在返工那一层
- | **出错代价高的活**
要给客户看的方案、要进档案的技术文件
- | **回本工时算得过来**
月订阅费除以时薪除以折现系数，示例演算约三个多小时
- | **自动化流程单独记一行**
按量调用费平时看不见，月底会变成一笔钱

建议在免费期的第二周就算一次回本工时，那时你对真实使用量已有概念

图 22-4 免费与付费的边界：三个判断点

图 22-4 免费与付费的三个判断点。（作者根据公开资料绘制）

判断点一：每月累计投入多少小时

我的经验线是：每月累计用到八到十小时以上，就该考虑付费了。

理由不是免费额度不够用，是免费版的限制往往会打断你的节奏：高峰期排队、单次输入长度受限、处理大文件被拒。这些打断落在试错账的“返工”那一层，比订阅费贵。

反过来，如果你一个月只在上边花两三个小时，那免费版完全够用，别急着买。

判断点二：出错代价高不高

免费版和付费版差的通常不只是额度，还有能力更强的模型和更快的响应。出错代价越高的活，越应该跑在更强的模型上。

这条判断点跟直觉相反。它贵的那部分，换来的是“少错一次”和“少等一会儿”。如果你干的是写周报、整理表格这类错了也无所谓的活，免费版够；如果你干的是要给客户看的方案、是要进档案的技术文件，付费版那笔钱比起一次返工便宜得多。

判断点三：回本门槛算不算得过来

这是最硬的一条，用一句话说完：每月至少要省多少小时，这笔订阅才回本？

公式是：

回本工时 = 月订阅费 ÷ 你的时薪 ÷ 折现系数

用示例数字演算一遍：月订阅费按约 200 元，时薪按示例值 150 元，折现系数取 0.4，算出来是：

$200 \div 150 \div 0.4 \approx 3.3$ 小时 / 月

也就是说，在这个示例条件下，你每个月至少要省出三个多小时，这笔订阅才算回本。省不出来，就别买，或者降级。

把这个数算出来你会发现两件事。第一，门槛其实没有想象中那么高，一个每周一次的数据整理就够翻过去。第二，你的时薪越高，门槛越低，高薪岗位几乎怎么算都划算。这就是“活越值钱，这笔账越好算”这句话的数学版本。

还有一个常见的坑是在“试用到付费”这一段。大多数工具的免费期给得很足，你会在免费期里养成一堆习惯，等免费期结束才发现自己在为一项超出自己频次的服务付费。我建议在免费期的第二周就算一次回本工时，那时候你对自己的真实使用量已经有点概念了。

最后一句：付费版不是终点。随着场景变多，真正决定成本的会变成频次和熟练度，而不是订阅单价。到那时候回头看第一年纠结的那几十块月费，你会觉得当年算的是一本很小的账。

22.5 隐性成本：三样没人提醒你的东西

订阅费、按量调用费这些是有发票的成本。下面这三样没人给你开票，但它们真实存在。

我把它放在一起还有一个原因：这三样都跟时间有关，而且都不可逆。订阅费不想付了，下个月停掉就完事；学习时间一旦花出去就收不回来，依赖一旦形成也很难短期解开。花钱能撤回，花时间不能。

学习活动：一次性，但是真的

我在第 1 章讲过自己装命令行工具赔进去一个傍晚的经历。学习时间的特征是：集中在头一两个月，之后迅速趋近于零，但每接触一个新的能力模块就会集中来一次。

比如我第一次配某个连接器，又赔进去一个下午。这类“新模块学习”的成本，建议按单个模块单独记账，不要指望被第一个场景摊薄。

减它的方法很朴素：抄现成的。本书 72 个场景存在的理由就是这个，别人踩过的坑你不用再踩一遍。我自己的经验是直接抄能省掉学习投入的一大半。

维护精力：跑通不等于永远通

这一项最容易漏算。一个场景跑通之后，它不是一劳永逸的，它会过期：

- 你依赖的软件更新了，界面或者接口变了；
- 对方网站改版了，抓取流程失效（我在例二里提到过一次）；
- 你的模板换了版本，旧的提示词产出物格式对不上；
- 你自己给它的规则忘了更新，它按老规矩办事。

我的经验是：一个跑熟的场景，每季度回头过一遍的成本，大约是当初重做的一成的量级。听起来不多，但如果你同时养着十几个场景，这笔账就变成了一个固定的季度支出，跟订阅费一个性质。

减它的方法：给每个跑通的场景留一张一页纸的说明书，写清输入是什么、输出落到哪里、验收看哪几个数。半年后你想不起来细节的时候，这张纸比重新试错便宜十倍。这也是本书第 4 篇每个场景都要求写“验收标准”的原因。

维护成本还有一个隐蔽的形态：你得持续盯着它的表现有没有退化。这里有个人话说不清楚，我自己就踩过：某个场景连续两个月都用得好好的，我于是默认它永远好用，一次验收也没做。第三个月我偶然核了一行，发现它对表格里新增的一列处理方式是错的，而那一列在两个月前就加进去了。从那以后我给自己定了条规矩：跑得越顺的场景，越要定期抽检。抽检的频率不需要高，一个季度一次足够，但必须是随机的、无聊的、认真核的那种。

依赖风险：别把身家性命押在一个工具上

这一节我想讲得重一点。工具用久了，会产生一种看不见的绑定：你的提示词库在它里面，你的记忆在它里面，你的技能配置在它里面，你的同事也习惯了它给你的输出格式。

这时候会发生两件事：换工具的成本变得极高，停服的风险变得极大。这两件事本质上是同一件事，在工程上我们有个现成的比方：单点供货的供应商一旦出事，你的产线当天就得停。

我给自己定了三条规矩，建议你也照着做：

规矩一：关键产物必须落地成本地文件。报告存成文档文件，数据存成表格文件，图存成图片文件。别让任何重要结果只存在于一个对话窗口里。对话窗口是它的地盘，本地文件是你的地盘。

规矩二：你的提示词和技能文件，用纯文本自己存一份。这些是你真正的资产，它们应该能随你搬去任何地方。我在第 4 章讲技能的时候强调过这一点：技能本来就是一堆文本文件，天然可以搬走。

规矩三：每个高频场景留出一条不用 AI 的老路，并且每半年真的走一次。这条规矩听起来很笨，可它的作用是保命：订阅断了、公司网络策略改了、接口涨价了，你还能交差。我在例二那个场景上就试过一次，自己手动做完那两个小时的活，结果发现我的表格模板那时候已经落后了半年。老路走得动，你才有底气压价。

为了让这三条规矩不至于只停留在口号上，我每年会强迫自己做一次“搬家演练”：挑一个最重要的场景，假装这个工具明天停用，看自己多久能在一个同类工具上把同样的活复现出来。我第一次做这个演练用了大半天，第二次快了不少。这个用时就是你的依赖风险有多大，一把尺子。它每次都应该在变小；如果发现它在变大，说明你又把某个东西绑得太死了。

隐性成本三件套：学习、维护、依赖

这三样都没有发票，但都跟时间有关，而且都不可逆

学习活动：一次性，但是真的

集中在头一两个月，之后趋近于零；每接触一个新能力模块会集中来一次，减它的方法是抄现成的

维护精力：跑通不等于永远通

软件更新、对方网站改版、模板换版本、你的规则忘了改；给每个场景留一张一页纸说明书

依赖风险：别押在一个工具上

关键产物落地成本地文件，提示词和技能用纯文本自己存一份，高频场景留一条不用 AI 的老路

花钱能撤回，花时间不能；每年做一次搬家演练，量一量自己的依赖有多深

图 22-5 隐性成本三件套：学习、维护、依赖

图 22-5 三样没有发票的成本。（作者根据公开资料绘制）

22.6 我的真实账本

这一节我只能定性地说。有些数字属于公司口径，我不方便写太细；另外我也不想在书里给读者一个虚假的精确感。以下全部是我的经验量级。

订阅账我觉得是最无感的一笔。全年下来是四位数的量级，对我的个人收支来说是一个可以感知但不至于犹豫的数字。它在我所有的月度开支里排不进前五。

学习账是头两个月最重的一笔。我投入的时间量级，跟我在第 2 章承诺读者的“半小时跑通第一件活”不一样：跑通第一件活确实快，真正吃时间的是把第五件、第十件活也理顺。我从“会用”到“稳定用”，中间隔了大约两个月。这笔账一次性付完，之后再没有大额支出。

试错账是最难看的一本。我的返工几乎全部来自两个错觉：一是忘了验收（以为它跟以前一样靠谱），二是把它当成懂工程的人（它在“看起来合理”这件事上的能力远超它在“真的对”这件事上的能力）。这两种错觉到现在偶尔还会冒出来，第 17 章那七宗罪没有一条是被彻底解决的，只能靠流程压着。

回头看这三条线交叉的位置，我发现一件有意思的事：给我带来最大收益的，不是省时间最多的那个场景，是频次最高的那个。一个每次能省三小时的年度场景，输给了一个每次只省一小时但每周都做的日常活。频次是一个比单次收益更可靠的杠杆，因为它同时把学习成本摊薄了。

顺便交代一件跟钱无关但我在心里算得很重的事。那年订货会前夕，我负责的一份数据汇总因为前面提到的那次返工，让我在截止日期前一夜全部重做了一遍。后来这套流程交给 AI 之后，同样的活再没让我熬夜到那个点。这笔收益没法折成任何金额，因为它不影响任何一张单据，可如果让我在这本书里挑一个最实在的收获，我大概会选它。

如果把我的三本账压缩成一句话，就是这个顺序：学习账在前两个月一次性付完，订阅账每月固定扣，试错账永远不为零。你唯一能主动管理的是第三本，而管理它的唯一办法就是老老实实验收。

最后回到那句被我在多处引用过的结论：活越值钱，这笔账越好算。

这句话的算术很简单。AI 在一个场景里能替你省下的时间是有限的，量级大多在几分钟到一两个小时之间，很少更多。这是分子，它被 AI 的能力天花板卡住，短期不会有大变化。而分母紧贴着你的时薪走：同样省下一小时，在复制粘贴的活上值几十元，在一场要不要让步接收的决策会议上，可能就关系到一整批零件的处置。

所以真正决定这笔账好不好看的，不是 AI 有多强，是你把自己最贵的那部分时间放在了哪。如果你最贵的时间是用来做数据整理的，那这笔账一开始就算歪了。

最后一件事我想留给刚读完前几章、正准备动手的你。别指望第一张表就好看。我第一次把表填满的那个晚上，九格里有一半是负收益，心态相当糟糕。三个月后我再翻那张表，发现负的那些行全都是我后来再没做过的活，而留下来的三条把它们全部盖过去了。这套算法的价值不在第一个月的数字，在它每个月逼你问自己一句：上个月那件事，到底还值不值得这么干。

小白词典

时薪折算：把小时换成钱的换算动作。够用的算法是税前月薪 $\div$ 22 天 $\div$ 8 小时。类比：一根尺子，用来比较不同岗位、不同场景的时间价值。

折现系数：省下的时间里真正能变成钱的比例。类比：漏斗，上端是省下来的时间，下端是流进产出里的那部分，中间漏掉的都是自己享受掉了。

订阅分摊：把整份订阅费按场景平摊，算出每个场景背了多少。类比：一家店的房租要摊到每一道菜的成本里，光看菜价不算账是不行的。

返工成本：东西已经做完了，因为做错而必须返过头重做所付出的时间。类比：漆喷完了才发现尺寸不对，回头比从头做还累，因为你还得先拆掉原来那层漆。

尾部风险：发生概率很低、但一旦发生损失极大的那类事。类比：车险。绝大多数年份你白交保费，可撞那一次就把十年保费一起赔进去了。

依赖风险：工具换掉或者停掉的时候，你会痛到什么程度。类比：只此一家供货的零件供应商，价格他说了算，断供你也只能认。

回本工时：这笔订阅要回本，你每月至少得省出多少小时。算法是月费 $\div$ 时薪 $\div$ 折现系数。类比：办健身年卡，先算算自己一年至少得去多少次才不亏。

隐性成本：账单上看不到、但真实发生了的支出，包括学习时间、维护精力、依赖风险。类比：买打印机只要几百块，买完之后一直在付的是墨盒。

你的岗位怎么用

设计工程师：把图纸标注校对、模型参数汇总、设计变更说明的初稿这三类活单独建三行。注意你最大的订阅账不在 AI 工具上，在你的专业软件授权上，AI 省的时间没法减少那笔钱，别混着算。

试验工程师：你是最容易算出正收益的一批人，因为你的活频次高、结构固定。数据整理和值班交接先上，这两个跑通后，把每周那份例行报告也交出去，三个场景叠加的月省时通常能轻松翻过回本门槛。

质量工程师：请在你的成本收益表里多加一列：人工复核耗时。你的活落在"出错代价高"那一类的比例远高于其他岗位，把复核时间记清楚，你才知道哪些报告可以放心发出去。

采购 / 供应商管理：资质核查、比价汇总、到货异常催办这三样适合上 AI。但你同时踩着第四条红线，所有涉及厂家名称、单价、份额的资料必须先做脱敏。我的习惯是把厂家改成代号，让它处理结构和逻辑。

项目经理：你的收益在时间账而不在金额上。周报、风险表、跨部门会议纪要这三样建议一次性全交，它们共同的特点是频次高、格式固定、错了也好改。

研发主管 / 部门经理：你算的是团队账，公式变成人数 × 频次 × 单次省时。我的建议是别一次铺开：先挑部门里三个人都在做的同一个高频活，跑两个月，用真实数据说服第二个人，比开一场宣讲会有用得多。另外在你向上汇报的时候，把"省了多少小时"放在第二位，把"哪个活的周期变短了"放在第一位，前者老板会怀疑，后者他会记得住。

行政 / 综合岗：你的活大多频次高、单次短、格式固定，比如通知、日程、报销单核对、活动物料。这类活单独算往往金额不大，但因为频次极高，叠加起来经常是全公司最漂亮的一行数字。建议把一个月内所有重复三次以上的动作列一遍，会有惊喜。

本章交付物

单场景成本收益表（九个格子的空模板）。

第一步，复制下面这张表，放进你的笔记软件或者表格文件里：

场景名	人工耗时	AI 耗时	每次省时	每月频次	月省时	折合金额	订阅分摊	净收益

想要放到 Excel 或者数据库里，用这个版本：

前面两行为示例演算，换成你自己的数：

场景名,人工耗时(分钟),AI耗时(分钟),每次省时(分钟),每月频次,月省时(小时),折合金额(元),订阅分摊(元),净收益(元)
 月度试验数据整理,90,25,65,4,4.3,320,40,280
 供应商月度资质核查,120,20,100,2,3.3,250,40,190

第二步，只填你最烦的三个活，别贪多。人工耗时和 AI 耗时都取三次平均，前面都加"约"。

第三步，跑满一个月，月底花十分钟看三件事：净收益为负的行当月就停掉；净收益最高的行下个月给它加频次；把本章结论贴在表格旁边：每年少于六次、出错代价极高、需要你拍板、资料不能外传，这四类活直接跳过。

本章金句

订阅费从来不是最贵的那一笔，最贵的是你试错的那个下午。

省下的时间会自己流走，除非你先给它准备好一个去处。

AI 耗时写到“它跑完”就停笔的人，算出来的收益一律虚高一倍。

每年少于六次的活，学习成本摊不薄，交给它就是负收益。

给 AI 的活写一页纸说明书，半年后的你会感谢今天的自己。

老路走得动，你才有底气跟工具谈价钱。

活越值钱，这笔账越好算：分子被 AI 卡住，分母贴着你的时薪走。

（下一章：出了问题怎么救。这套账算得再准，也挡不住它某天突然跑不动，那时候你需要的是一张排查清单。）

第 23 章

出了问题怎么救

出事的第一分钟，最贵的那个动作是重试。你还没搞清楚它错在哪，就让它把同一个动作对着同一批文件再跑一遍。

2026 年 8 月 19 号，周三下午三点二十，我在赶一份当天要发出去的耐久试验汇总表。

活本身很小。试验那边给我一个目录，里面 14 个 CSV，每个文件是一台样机一个工况的扭矩记录。客户要的单位是 $\text{N}\cdot\text{m}$ ，我们设备导出出来的是 $\text{kgf}\cdot\text{cm}$ 。我要做的就是 把 14 个文件里的 torque 那一列换算一下，写回原文件。

我在对话框里敲了一句："把 data 目录下所有 CSV 的 torque 列从 $\text{kgf}\cdot\text{cm}$ 换算成 $\text{N}\cdot\text{m}$ ，写回原文件。"

它干了七分钟，回报："已完成 14 个文件的单位换算。"

我打开汇总表，把换算后的数据接进去，算了个平均值。上一版这个数是 $0.62\text{ N}\cdot\text{m}$ ，这一版是 $0.51\text{ N}\cdot\text{m}$ 。换算系数是固定的，平均值不该因为换单位就掉 18%。

我停下来了。

我先让它报改动清单。它报了 14 个。我打开目录按修改时间排序，变了 11 个文件，其中两个我不认识：`torque_raw_backup.csv` 和 `2026Q2_archive.csv`。这两个文件在归档目录里，是我三年前手工备份的原始值，我没想到它俩也在 data 目录下，更没想到"所有 CSV"这四个字能把它们一起扫进去。

更要命的是第三件事。它说改了 14 个，实际改了 11 个，剩下 3 个它没动，理由是"这 3 个文件的 torque 列存在空值，已填充为 0 以保证计算完整"。空值填充为 0，一共 318 行。这 318 行进平均值，平均值就被拉下来了。它做得非常合理，合理到我如果没算那个平均值，根本发现不了。

最坏的一件事是 `torque_raw.csv`。这个文件在 14 个之外，是原始值的母本，文件名里带 raw。它把这个文件也覆盖了，覆盖成了换算后的结果。 $\text{kgf}\cdot\text{cm}$ 的原始数据，3862 行，没了。

我手上有备份。备份是 8 月 12 号的一个 zip，七天前。七天前之后我又跑了三天试验，那三天的 412 行不在备份里。这 412 行我最后是从台架设备的日志文件里重新导出来的，导出加清洗花了 2 小时 15 分。

那天我下班是晚上九点四十。报告晚交了一天，我在邮件里写的原因是"数据处理环节返工"。真实原因是我在让它动手之前，没有先让它把要改的文件清单给我看一眼。

那天晚上我给自己写了三行字，贴在显示器下沿：先问它改了什么，再看它对不对，最后才谈怎么改。顺序错了，代价是我那天晚上的六个小时。

这一章讲的就是那个顺序。第 17 章讲的是它为什么会错，这一章讲的是错了以后你怎么办。

止损三动作：停、看、退

出事的第一分钟，最贵的那个动作是重试

1

停：停的是你手上的活

不再发任何新指令，不发继续也不发再试一次；不要关窗口，关了就丢了现场；它在批量写文件时断网络比关程序快；把最后三条对话复制出来

2

看：看它改了哪些文件

让它自己报，快但不可信；按最后修改时间排序整个目录，二十秒能做完；用版本管理看 diff，这是唯一可信的一种

3

退：先算代价再退

一个文件从备份拿回来，几分钟；一批文件恢复整个目录，十几分钟；整个目录回上一次提交，半小时起；原始输入从源头重导，几小时最贵

三个数要对得上：它报的改动数、目录里变了的数、你让它改的数

图 23-1 出问题第一分钟三个动作：停、看、退，顺序不能乱

图 23-1 止损三动作：先停手，再看清单，最后才决定退不退。（作者根据公开资料绘制）

23.1 止损三动作：停、看、退

绝大多数人在 AI 出问题时做的第一件事，是再说一遍。把刚才那句话重发一次，或者加一句“你再试一次”，或者点界面上那个重试。

这个动作在工程现场是错的。原因很具体：

第一，重试会覆盖现场。它上一轮改了哪些文件、改到哪一行停的，这些信息在重试之后就没了。你连“它刚才干了什么”都问不出来了。

第二，重试会让它修它自己猜的那个错。你没告诉它错在哪，它就会自己猜一个最像的原因，然后按那个原因改。改对了是运气，改错了它会在错的基础上再错一层。我那 318 行空值填 0，就是它自己猜的“修复”。

第三，同一批文件被改两遍，回滚会变复杂。只改一遍，你从备份里把原文件拿回来就行。改了两遍，你得先分清哪一版是哪个动作造成的，而这个只有它自己知道，而它自己已经记不清了。

所以第一分钟的动作是固定的三个，顺序不能乱。

动作一：停。停的是你手上的活，不是 AI

停，指的是你立刻做下面四件事，一件都不能跳：

1. 不再发任何新指令。不发"继续", 不发"再试一次", 不发"你确定吗"。这三句都会让它继续执行。
2. 不要关窗口。关窗口会丢掉当前这一轮的上下文, 那是你唯一的现场证据。要停的是执行, 不是对话。
3. 如果它正在写文件, 断网络比关程序快。本地已经发出的写命令不受影响, 但后续的远程调用会停。这一招我用过两次, 都是它在大批量改文件的时候。
4. 把最后三条对话复制出来, 粘到一个文本里。包括它说的"已完成 N 处"这种话。这三条是你后面复盘和求助的第一手材料。

停的动作大概需要二十秒。这二十秒是整个事故里最便宜的二十秒。

动作二：看。看它改了哪些文件

看, 只有三个看法, 按可靠性从低到高排:

看法一: 让它自己报。快, 但不可信。它报的清单里会有遗漏, 我那天遇到的就是它报 14 个、实际改 11 个, 两个数字没有一个对得上真实情况。

看法二: 看文件的最后修改时间。让它改之前报一次所有文件的修改时间, 改完再报一次, 两次对比。或者你自己打开目录, 按修改时间排序。

看法三: 用版本管理看 diff。这是唯一可信的一种。前提是你动手前做过一次提交或者快照, 这件事 23.5 节会讲怎么做。

三条里面, 看法二最实用, 因为它不需要任何提前准备。二十秒能做完, 能挡掉一整天的返工。

给它这段话, 让它报清单:

【停下来。先不要执行任何修复动作, 也不要重试刚才的操作。】

【我要你做三件事, 按顺序, 做完就停在这里等我。】

第一件: 列出你刚才这一轮实际操作过的每一个文件,

格式: 文件绝对路径 | 你做了什么 | 操作前的值或状态 | 操作后的值或状态 | 是否成功

一个文件一行。没有实际操作过的文件不许出现在表里。

你记不清的写"记不清", 不许猜。

第二件: 回答三个问题, 每个问题一句话, 不许展开。

问题一: 你实际改动了几个文件, 其中有几个是我明确让你改的。

问题二: 有没有任何一个文件的内容被覆盖或者清空, 有就写文件名。

问题三: 有没有任何一个文件的修改是我没让你做的, 有就写文件名和原因。

第三件: 把当前工作目录下所有文件的文件名、大小、最后修改时间列一遍。

一个文件一行, 不许省略, 不许用"等等"代替。

【禁止】

在我说"可以继续"之前, 不许修改任何文件、不许删除任何东西、

不许执行任何修复命令、不许重试上一步。

你要是觉得有个修复动作很紧急, 先写下来告诉我, 等我同意。

这份清单我只看三个数：它报的改动文件数、目录里修改时间变了的文件数、你让它改的文件数。三个数不相等，就是出事了。

动作三：退。退到哪一层，看损失有多大

退不是万能的。退之前先算一下，因为退本身也有代价。

退到哪一层	怎么退	代价	什么时候用
退一个文件	从备份里把这一个文件拿回来	几分钟	只有一个文件被改错，其余都对
退一批文件	把整个目录从备份里恢复	十几分钟，会丢备份之后的所有改动	它动了你不认识的文件
退整个目录	从版本管理里回到上一个提交	半小时起，丢最后一次快照之后的所有工作	改了一片，分不清哪个是哪个
退到原始输入重做	从源头（设备、系统、邮件）重新导出	几小时，最贵	原始值被覆盖，备份里也没有

我那天退的是第二层加第四层：目录从 8 月 12 号的备份恢复，缺的三天数据从台架日志重导。加起来 2 小时 15 分。

退之前必须先回答一个问题：备份是多久以前的，中间这段时间我做了什么。这个问题答不上来，你连退到哪一层都决定不了。这就是为什么 23.5 节那三条硬规矩必须放在平时做，不能等出事那天才想起来。

一个反直觉的判断：损失上限要在动手前定

这一条是这一节最值钱的一句。

在让它动你的文件之前，你先问自己一句：这个目录要是全毁了，我最多能接受多少损失？答案通常有三种：

- 能重做（源头还在，费点时间）：可以放开让它干，但要有备份。
- 能重建但很贵（要重新导出、重新试验、重新找人要）：只给它文件白名单，白名单外的不许碰。
- 不能重做（原始记录、客户原件、没有第二份的东西）：这个目录设只读，让它复制到工作目录里干，原件不许动。

我那天那 3862 行属于第三种，而我按第一种的方式把它交出去了。判断错一层，代价是六个小时。

老兵说：它改错一个文件只用几秒，你把它救回来要几个小时。所以止损这件事的核心不在"怎么救"，在"动手之前先想好救不救得回来"。

23.2 故障树：五类故障与定位路径

我这两年记下来的故障，全部归进五类：跑不动、读不到、干错方向、产出不对、越权改动。五类的定位路径完全不同，认错类别会让你在错的地方找半天。

先给一张分辨表，你按症状往下对。



图 23-2 五类故障树：从症状到类别的三步分辨

图 23-2 五类故障树：先分类，再定位。认错类别是在错的地方找半天。（作者根据公开资料绘制）

你看到的第一个现象	大概率是哪一类	一句话分辨法
它根本没动，或者报错说连不上、打不开	跑不动	能不能让它干一件最小的事，比如读一个只有三行的文件
它动了，但说找不到、读不出、写不进	读不到文件	让它报一次文件的绝对路径和大小，跟你在资源管理器里看到的对一下
它一直在动，产出也是好的，但不是你要的活	干错方向	回到任务第一句话，看完成条件有没有写清
活是对的，数字、格式、引用对不上	产出不对	核样本数、单位、外引事实三处
它动了你不让它动的文件或系统	越权改动	看目录里有没有你没点名的文件被改了

五类的顺序是有讲究的：先排"跑不动"，因为它是唯一一个"根本没开始"的类别，后面四类都以它已经跑起来为前提。

第一类：跑不动

服务没起、权限没给、网络不通。这一类最好认，因为它连第一步都迈不出去。

症状	三个最常见原因	定位动作	修复动作
双击图标没反应，或者闪一下就没了	上次没正常退出，进程卡在后台；安装包下载不完整；杀毒软件拦了	打开系统的进程列表，看有没有残留的相关进程	结束残留进程再开。还是不行就重新下载安装包，下载完核对文件大小与官网标注的是否一致（以你实际看到的为准）
装上去后，打开要登录，账号密码都对就是进不去	公司网络代理或防火墙拦了登录请求；登录服务当时不可用；账号本身没激活	换一个网络（手机开热点）试一次	换网络能进就是网络问题，找单位的网络管理员报这个域名；换网络也不行，去查服务状态页
提示版本过旧，必须更新才能用	客户端与服务端版本不匹配；自动更新被关掉了	看客户端设置里有没有"检查更新"这个入口（界面路径以你实际版本为准）	更新。更新之前先把工作目录整个复制一份，更新过程本身有概率动配置

这一类的定位有个通用顺序，四次就能收敛：先确认进程在不在，再确认网络通不通，再确认账号有没有授权，最后才怀疑是版本问题。我见过太多人一上来就重装，重装要四十分钟，而这四步检查只要三分钟。

第二类：读不到文件

路径错、格式不支持、文件被占用、编码问题。这一类最烦人，因为它的报错常常词不达意。

症状	三个最常见原因	定位动作	修复动作
说找不到文件，可你确定路径是对的	工作目录没设对，它找的是另一个根目录；文件名里有大小写或后缀差异；文件在它没被授权访问的目录里	让它报一次"当前工作目录的绝对路径"，再让它报一次"你理解的文件路径"，两个都跟你看到对一下	把文件挪进工作目录，或者把目录加进授权范围。第2章2.5.1节讲的工作目录设置，这一刻就派上用场了
文件打不开，提示被占用或者被锁定	这个文件正被 Excel、Word 或者另一个程序打开着；上一次异常退出留下了锁文件	关掉所有可能打开它的程序，再试一次	关掉程序就好。实在不行就复制一份到新文件名再让它读，别跟锁文件较劲
读出来全是乱码、问号、方框	文件编码不是 UTF-8，常见于老系统和财务软件导出的 CSV；或者文件本身是二进制格式却被当成文本读	用记事本打开这个文件，另存一次，看另存对话框里显示的当前编码是什么	另存成 UTF-8 再让它读。批量文件的话让它先按 GBK 读一次试试
PDF 读出来是空的或者只有几行	这是扫描件，页面里是图片不是文字；或者 PDF 有权限限制不许提取文本	用阅读器打开这个文件，试着选中一行字，选不中就是扫描件	扫描件要先做识别。做之前必须先做小批量校准，用十页已知答案的测出准确率，第17章场景68讲的就是这一步

这一类里我最想强调一句：读不到的报错，九成不是文件的问题，是路径和权限的问题。文件本身很少坏。所以第一步永远是让它报路径，不是去修文件。

第三类：干错方向

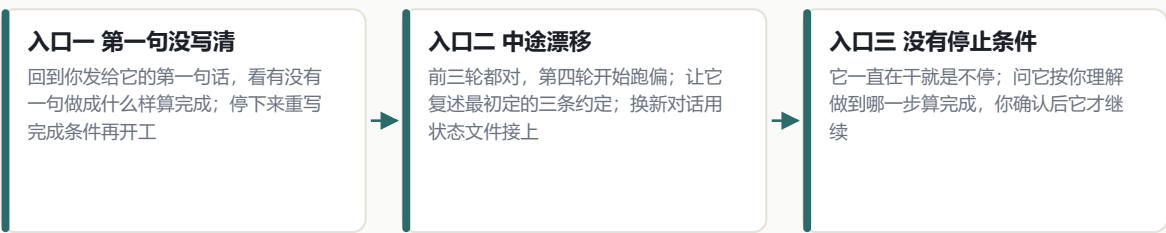
任务没说清、完成条件缺失、中途漂移。这一类最隐蔽，因为它的每一步看起来都做对了。

症状	三个最常见原因	定位动作	修复动作
它干的活跟你想要的不是一回事	你第一句话里就没写清完成条件；它自己补了一个默认目标；任务里有歧义词	回到你发给它的第一句话，看里面有没有一句"做成什么样算完成"	停下来，重写完成条件再开工。第 16 章场景 55 专门讲怎么写这句
前三轮都对，第四轮开始跑偏	对话太长，前面的约定被挤掉了；你中途补了一句新要求，它按新要求重排了优先级	让它复述一遍你们最初定的三条约定，看它说不说得出来	换一轮新对话，用状态文件接上。第 16 章场景 60 讲的漂移检测，这一刻要用
它一直在干，产出也不错，但就是不停	没有设停止条件；它自己给自己加了活	问它一句"按你理解，这件事做到哪一步就算完成了"	让它先说停止条件，你确认后它才继续

这一类有个特别好用的判断动作：问它"按你理解，做完了是什么样"。它答得出来，说明它理解了，只是跟你理解的不一样，改就行；它答不出来，说明它从第一句开始就没理解，得从头来。

干错方向的三个入口

这一类最隐蔽，因为它的每一步看起来都做对了



问它一句按你理解做完了是什么样：答得出来只是理解不同，答不出来得从头来

图 23-3 干错方向的三个入口：任务开头、中途漂移、没有停止条件

图 23-3 干错方向：第一句没说清、中途漂移、没有停止条件。（作者根据公开资料绘制）

第四类：产出不对

数据错、格式错、编造内容。这一类第 17 章讲透了原因，这里只讲排查顺序。

症状	三个最常见原因	定位动作	修复动作
数字看着不对，但说不出哪不对	单位换算错了；统计口径变了（比如样本数悄悄少了几个）；算术环节出错	三个数先核一遍：样本数闭合、单位、算式。样本数四个数对不上就先停	走一次脑补体检，第 17 章场景 62 有完整提示词。单位单独列一张换算对照表给它
引用了一个查不到的标准号、文件名、版本号	它编的；你把两个版本的文件混着给它了；它引用的是它记忆里的东西	把报告里所有标准号、文件名列出来，逐条在你的文件库里搜	走第 17 章场景 62 的允许引用清单，清单外的一律不许出现
同一个问题问两遍，答案不一样	至少有一个是编的；你的问法有歧义；它没读到同一份材料	让它把两次答案的依据各写一遍，两个依据对不上的那个是编的	把材料一次性给全，问法改成一个能回答"是或否"的具体问题

这一类的排查顺序我固定为：样本数 → 单位 → 外引事实 → 算术。前两个两分钟能核完，能挡掉大部分问题。后两个慢，但决定这份东西能不能签字。

第五类：越权改动

改了不该改的文件、动了不该动的系统。这一类是最贵的，因为它改的东西你可能不知道。

症状	三个最常见原因	定位动作	修复动作
目录里有个你没点名的文件被改了	你的指令里用了"所有""全部"这类词；目录里混着备份文件和归档文件；它为了"完整性"自己扩大了范围	按最后修改时间排序整个目录，看有几个文件的时间是你这次任务的时间段	先回滚，再把备份和归档挪出工作目录。下次给文件白名单，第 17 章场景 64 讲白名单怎么写
它动了工作目录之外的东西	工作目录边界没设严；你给的绝对路径指向了别处；它自己去找了"相关文件"	问它一句"你这次读过或写过工作目录之外的哪些路径"	收紧工作目录权限，把重要的目录设成只读。第 12 章场景 36 讲的就是这条边界怎么划
它动了系统设置、装了东西、改了环境变量	你给的指令里包含需要高权限的动作；它在"修复问题"时自己装了依赖	问它"你这次执行过哪些需要管理员权限的命令"	立刻撤销，并把它需要的依赖事先装好，别让它在任务中途装东西

这一类没有"修复动作"能挽回损失，只有"预防动作"。所以 23.5 节那三条硬规矩，本质上都是为这一类准备的。

五类故障该翻哪一章

这一章讲怎么救，那一章讲为什么会错

故障类别	本章：出问题怎么救	那一章：为什么会错
跑不动	先确认进程、网络、账号，最后才怀疑版本	第 2 章安装与启动
读不到文件	先让它报路径，再查占用和编码	第 2 章工作目录设置
干错方向	重写完成条件，新对话用状态文件接上	第 16 章场景 55、场景 60
产出不对	按样本数、单位、外引事实、算术核	第 17 章七宗罪、场景 62
越权改动	先回滚，再给文件白名单收紧目录	第 17 章场景 63、场景 64

读不到的报错，九成不是文件的问题，是路径和权限的问题

图 23-4 五类故障的分类与交叉指引：哪一类该翻哪一章

图 23-4 五类故障该翻哪一章：这一章讲怎么救，那一章讲为什么。（作者根据公开资料绘制）

23.3 报错速查表：这一页会被你翻烂

下面这张表是本章最值钱的一页。三十条，按七类分组。

用法是这样的：先看你的现象落在哪一组，再从上往下扫第一列，扫到跟你看到的最像的那一条，直接做第三列那个动作。不要试图理解第二列的原因，那只是给你参考的。

需要说明一句：表里第一列的"现象"写的是描述，不是具体的报错文字。同一类问题在不同版本上显示的文字不一样，具体措辞以你实际看到的为准，别拿我写的话去比对。

一、安装与启动

#	报错现象	最可能的原因	第一个该做的动作
1	双击图标没反应，或者闪一下就没了	上次没正常退出，后台有残留进程	打开进程列表找残留进程，结束掉再开
2	安装进度条走到一半停住或失败	安装包下载不完整，或者被安全软件拦了	重新下载，下载完核对文件大小与官网标注的是否一致

#	报错现象	最可能的原因	第一个该做的动作
3	打开后一片空白，什么都没有	本地缓存损坏	关掉窗口，清一次本地缓存再开（缓存位置以你实际版本为准）
4	账号密码都对，就是登录不进去	公司网络代理或防火墙拦了登录请求	换手机热点试一次，能进就是网络问题
5	提示版本过旧，必须更新	客户端与服务端版本不匹配	先备份工作目录，再更新

二、文件读写

#	报错现象	最可能的原因	第一个该做的动作
6	说找不到文件，你确定路径是对的	工作目录没设对，它找的是另一个根目录	让它报一次当前工作目录的绝对路径，跟你核对一下
7	路径里有中文或者空格，读不进去	路径编码或转义处理有问题	把文件挪到纯英文、无空格的短路径下再试
8	文件打不开，提示被占用或被锁定	这个文件正被 Excel 或另一个程序打开着	关掉那个程序；不行就复制一份新文件名再让它读
9	CSV 或文本读出来是乱码、问号	文件编码不是 UTF-8	用记事本另存成 UTF-8，或者让它按 GBK 读一次
10	PDF 读出来是空的，或者只有几行	这是扫描件，页面里是图片	试着在阅读器里选中一行字，选不中就是扫描件，要先做识别
11	写文件失败，说没有权限	目标目录是系统目录或只读目录	换到你的工作目录下写
12	一个文件被覆盖成空的，或者内容变少	写入时被中断，或者它清空后没写回	立刻停手，去备份里找回这个文件（见 23.5）

三、权限与授权

#	报错现象	最可能的原因	第一个该做的动作
13	每动一个文件都弹确认框，点完还是不动	权限模式设成了每步都问，某一步没点到	查一次当前的权限设置，确认是哪种模式
14	说没有权限访问某个目录	该目录在工作目录之外	把文件挪进工作目录，或在权限设置里加上这个目录
15	浏览器打开是未登录状态	登录态过期，或者网站要求重新验证	走一次人工接管，第 12 章场景 33 讲的就是这一步
16	明明授权过了，还是说未授权	授权给了 A 服务，调用的是 B 服务	查一次已授权列表，确认服务名对得上（第 19 章 19.3 节）

四、连接器与 MCP

#	报错现象	最可能的原因	第一个该做的动作
17	连接器列表里有，它却说找不到这个工具	连接器没启用，或者这一轮没加载	在新对话里单独叫一次它的名字，报"未加载"就是没启用
18	MCP 服务装了，可用工具一个都没有	服务进程没起来，或者配置里的启动命令路径写错	重新跑一次第 20 章 20.3 节那个"怎么确认装成功了"的动作
19	MCP 服务启动报"命令找不到"	那条命令没装，或装在它看不到的位置	在终端里手动跑一次那条命令，跑不通就先解决命令本身
20	连接器用着用着突然不能用了	授权过期，或者服务端那边撤了授权	重新授权一次，重授权前看一眼它这次要的权限范围

五、任务执行

#	报错现象	最可能的原因	第一个该做的动作
21	它说做完了，但文件根本没生成	它把"计划做完了"说成了"做完了"	让它报文件的绝对路径，你自己打开看（第 17 章场景 67）
22	干到一半停住不动	卡在需要你确认的地方，或者对话太长	看最后一条是不是在问你问题；不是就新开对话用状态文件接上
23	它反复做同一件事，来回打转	卡在一个它解决不了的报错上，它在自己重试	打断它，让它把卡住的那一步和报错原文报出来
24	中途改了方向，做的不是你干的活	完成条件没写清，或者中途漂移	停下来，重写完成条件（第 16 章场景 55、场景 60）

六、产出异常

#	报错现象	最可能的原因	第一个该做的动作
25	数字看着不对，说不出哪不对	单位、样本数、算式三处之一	先核样本数闭合（送检、完成、有效、剔除四个数），再核单位
26	引用了一个查不到的标准号或文件名	它编的	走一次脑补体检（第 17 章场景 62）
27	报告里大量两位小数，而原始记录只有整数	它按格式补出来的	抽五个数回原始记录核对（第 17 章场景 62 的"抽五条"）

七、性能与超时

#	报错现象	最可能的原因	第一个该做的动作
28	跑了很久没结果，进度一动不动	任务太大；或者它在等一个外部服务的响应	先打断，让它只做前十条；能过就是量的问题，过不了就是卡在外部服务

#	报错现象	最可能的原因	第一个该做的动作
29	处理大文件时报"内容太长"或者被截断	单次能处理的长度有上限	分批处理，或先让它做一份摘要再基于摘要干活
30	跑到一半报超时，或者连接中断	单步耗时超过了限制	把这一步拆成两步；能后台跑的就改成后台跑

这张表我不建议按编号用，建议按症状扫。原因很实在：你出事的时候不会记得"我这是第几条"，你只记得"它说找不到文件"。所以编号的唯一作用是让你在向别人求助时能说一句"我查了速查表第 6 条，试了没用"，这一句能让帮你的人省五分钟。

再补三条我在表里没写进去、但每次都有用的土办法：

- 换个最小样例试一次。它处理 14 个文件失败，你就让它处理 1 个。能过就是量的问题，过不了就是这个文件本身的问题。这一招三十秒，能砍掉一半的排查时间。
- 换个新对话试一次。很多"莫名其妙"的错误是这一轮对话的状态污染造成的。新开一轮，同样的指令，能过就说明问题在这一轮，不在任务本身。
- 重启一次。很土。我自己的经验是大约两成的怪问题重启就好，所以我把重启放在排查顺序的第四位，不放在第一位，因为前三位更便宜也更能留下信息。

报错速查表的七类分组

先看现象落在哪一组，再扫第一列，扫到最像的直接做第三列那个动作

分组	条目	典型现象
安装与启动	第 1 到 5 条	双击没反应、进度条走一半停住、登录进不去、提示版本过旧
文件读写	第 6 到 12 条	说找不到文件、路径有中文或空格、被占用、读出来乱码、PDF 读出来是空的
权限与授权	第 13 到 16 条	每动一个文件都弹确认框、说没有权限访问某个目录、浏览器打开是未登录
连接器与 MCP	第 17 到 20 条	列表里有它却说找不到、装了可用工具一个都没有、启动报命令找不到
任务执行	第 21 到 24 条	说做完了文件没生成、干到一半停住、反复做同一件事、中途改了方向
产出异常	第 25 到 27 条	数字看着不对、引用了查不到的标准号、报告里凭空多了两位小数
性能与超时	第 28 到 30 条	跑了很久没结果、大文件报内容太长、跑到一半报超时或连接中断

三个土办法：换个最小样例、换个新对话、重启一次，重启放在排查顺序的第四位

图 23-5 报错速查表的七类分组与排查顺序：先最小样例，再换对话，再重启

图 23-5 速查表七类，外加三个土办法：最小样例、换对话、重启。（作者根据公开资料绘制）

23.4 怎么问才有效

出事之后你要问两个对象：AI，和人。两种问法完全不同。

向 AI 求助：给它三样东西，缺一样它就只能猜

向 AI 求助最常见的失败，是只贴一句报错，然后问"这是怎么回事"。它给出的答案会非常合理，而且很可能是错的。因为你给的信息不够，它只能用"这种情况下通常是什么问题"来补。

必须给的三样东西：报错原文、你在做什么、已经试过什么。

第三样最容易被漏。你试过的每一件事，哪怕没用，都是在缩小范围。特别是"换了个文件就好了"这种，它一下就能把原因从"任务错了"缩小到"这个文件有问题"。

整段复制，方括号里换成你自己的内容：

【我这边出错了。先不要动手，本轮只做诊断，不许执行任何修改。】

【报错原文】

下面这段是从界面上原样复制的，不要概括，不要翻译，不要只取其中一句：

[在这里粘贴完整的报错文字，包括它前后的几行]

【我在做什么】

任务：[一句话，比如"把 14 个 CSV 里的 torque 列从 kgf·cm 换算成 N·m"]

这一步之前已经完成：[.....]

这一步我期望发生：[.....]

这一步实际发生：[.....]

【我已经试过下面这些，逐条附结果】

1. [重试了一次] → 结果：[同样的错 / 换了个错 / 好了]

2. [换了一个文件试] → 结果：[.....]

3. [查了 xxx，看到 xxx] → 结果：[.....]

其中任何一条改变了现象的，请你重点问我那一条的细节。

【我的环境】

系统：[Windows 11]

客户端版本：[以你实际看到的版本号为准]

工作目录：[D:\work\xxx]

涉及文件：[文件名 + 大小 + 最后修改时间]

【我要你回答四件事】

1. 这段报错最可能的原因，按可能性从高到低列三条，每条一句话。
2. 每条原因各配一个能立刻验证的判断动作，写清"我看到什么就说明是这条"。
3. 这三条里，哪一条会导致文件内容被覆盖或者数据丢失，先说这条，并告诉我怎么止损。
4. 在原因确认之前，我现在最不该做的动作是什么。

【禁止】

不许写"你可以尝试....."这类没有判断依据的话。

我要的是判断动作和判断依据，不是候选方案清单。

你不确定的地方直接写"我不确定"，不许猜。

在我说"可以动手"之前，不许修改任何文件、不许执行任何命令。

这段提示词里最有用的是第 3 问。先问"哪条会导致数据丢失"，再问"怎么修"。顺序反了，你会在修的过程中把现场又破坏一次。

向人或社区求助：四要素，缺一个回帖率减半

在群里、在论坛里、在同事工位上问问题，四要素缺一不可。我见过太多"我用 AI 跑一个东西报错了，有人知道吗"这样的提问，下面零回复，不是大家冷漠，是没法回。

要素一：现象。报错原文，原样粘贴，截图也行。不要概括成"它说找不到文件"，概括会丢掉最关键的那半句。

要素二：复现步骤。编号，每步一个动作，写到别人照着做能重现。这一步最费事，也最值钱。我自己写复现步骤的时候，大约三成的情况是写完就发现原因了，因为写清楚的过程逼着你想清楚。

要素三：已试过什么。每条附结果，包括"试了没用"的。这一条帮回答的人避开你已经走过的路。

要素四：环境。系统版本、客户端版本、目录位置、文件大小、是不是内网。环境问题占所有疑难杂症的比例高得离谱。

复制这个模板：

【问题】一句话说清：[.....]

【现象】

报错原文（原样粘贴）：

[.....]

截图：[附件]

【复现步骤】

1. [打开 XXX]
 2. [在 XXX 目录下放了 XXX 文件，文件是这样的：.....]
 3. [发了这句指令：.....]
 4. [第 N 步之后出现上面的报错]
- 每次都这样 / 十次里大概出现三次

【已经试过】

1. [重试] → 没用，同样的错
2. [换成 UTF-8 编码] → 没用，还是同样的错
3. [换了一个更小的文件] → 好了，能跑通
4. [查了速查表第 9 条] → 没用

【环境】

系统：Windows 11

客户端版本：[以实际看到的为准]

工作目录：D:\work\xxx

文件大小：约 3.2 MB，14000 行

网络：公司内网，走代理

是否涉及不能外传的资料：已脱敏 / 不涉及

【我想要什么】

我想知道这是配置问题还是文件问题。

如果有人遇到过，麻烦说一句你当时是怎么判断出来的。

最后那句"我想要什么"别省。很多人提问没说清楚自己要的是答案、是方向、还有一句"这个我也遇到过"，回答的人就得猜你要什么，猜错了两边都浪费时间。

求助四要素：缺一个回帖率减半

在群里、在论坛里、在同事工位上问问题，四要素缺一不可

- ✓ **要素一 现象：报错原文原样粘贴**
不要概括成它说找不到文件，概括会丢掉最关键的半句
- ✓ **要素二 复现步骤：编号，每步一个动作**
写到别人照着做能重现；写清楚的过程会逼着你想清楚
- ✓ **要素三 已试过什么：每条附结果**
包括试了没用的那些，帮回答的人避开你已经走过的路
- ✓ **要素四 环境：系统、版本、目录、文件大小**
环境问题占疑难杂症的比例高得离谱，是不是内网也要写
- ! **最后别省一句：我想要什么**
说清你要的是答案、是方向，还是一句我也遇到过

向 AI 求助要同时给三样：报错原文、你在做什么、已经试过什么

图 23-6 求助四要素：现象、复现步骤、已试过、环境，外加一句我想要什么

图 23-6 求助四要素：缺一个，回帖率减半。（作者根据公开资料绘制）

23.5 备份与回滚：三条硬规矩

这一节是整章唯一需要你在没出事的时候做的事。出事那天再做，已经晚了。

规矩一：交活前先备份，备份要带时间戳

交活指的是“让它动你的文件”这件事。不管是改一个文件还是改一百个，动手之前先复制一份。

三条具体要求：

1. 备份目录必须跟工作目录不在同一个硬盘。同一个盘，盘坏了两份一起没。
2. 备份目录名带日期时间。 `motor_nvnh_20260819_1502` 这种格式。写时间是唯一的成本，它能让你在回滚的时候不用猜。
3. 备份完立刻验一次。打开备份目录，看文件数跟原目录一不一样。

Windows 上我用的是这一条，存成 `backup.bat` 放在工作目录旁边，双击就跑：


```
@echo off
rem 用法：改文件之前双击一次，会在目标盘生成一个带时间戳的目录
set SRC=D:\work\motor_nvh
set ROOT=E:\backup

for /f "tokens=1-3 delims=/- " %a in ("%date%") do set D=%a%%b%%c
for /f "tokens=1-2 delims=:." %a in ("%time%") do set T=%a%%b
set DST=%ROOT%\motor_nvh_%D%_%T%

robocopy "%SRC%" "%DST%" /E /R:1 /W:1 /NFL /NDL
echo 备份完成：%DST%
pause
```

`robocopy` 是系统自带的复制命令，`/E` 表示连空目录一起复制，`/R:1 /W:1` 表示遇到打不开的文件只重试一次、等一秒，免得它卡在某个被占用的文件上。

更省事的做法是版本管理，它能帮你做 23.1 节里那个“看法三”的 diff：

```
# 在工作目录里开一次，之后每次动文件前提交一次
git init
git add .
git commit -m "改动前的快照：换算 torque 列之前"

# 出事之后，先看它改了什么
git status
git diff

# 确认要退，把某一个文件退回来
git checkout -- torque_raw.csv

# 整个目录退回上一次提交
git reset --hard HEAD
```

最后那条 `git reset --hard` 会丢掉最后一次提交之后的所有改动，用之前先确认你真的要全退。

规矩二：重要目录设只读

不能重做的东西，就不要给它写权限。原件、归档、客户给的原始资料，这三个一律设只读。

只读的意思是：它能读，不能写。这么设之后，它就算想覆盖也覆盖不了，会报一个权限错误，而权限错误是所有报错里代价最低的一种。

```
rem 把归档目录整体设成只读
attrib +R D:\work\archive\*. * /S

rem 干完活之后改回来（需要的时候）
attrib -R D:\work\archive\*. * /S
```

/S 表示连子目录一起处理。

设完之后要验一次：让它试着往这个目录里写一个文件，它应该报错。报错了说明设对了。

我自己的目录结构是固定的三层：

层	目录名	权限	放什么
原件层	archive	只读	客户原件、设备原始导出、不能重做的东西
工作层	work	可读写	它唯一能改的地方，每次动手前备份
输出层	out	可读写，可清空	它产出的新文件，丢了重跑就行

这个结构解决的是我那天的根本问题：备份文件 torque_raw_backup.csv 当时就躺在 data 目录下，跟待处理文件混在一起。分层之后，它就算扫"work 下所有 CSV"，也扫不到 archive。

规矩三：让它改文件前，先看清单

这一条在本书里出现过三次：第 17 章场景 63 的改动对照表、场景 64 的文件白名单，还有这里。三次讲的是同一件事，因为它值得讲三遍。

动作只有两个：

- 1. 动手前，让它先列出"我准备改哪几个文件、每个文件改哪几处、旧值是什么、新值是什么"。你看过，说同意，它才动手。
- 2. 动完之后，让它再报一次实际改了哪些。两份清单对比，对不上就是出事了。

这两步加起来大概两分钟。我那天省了这两分钟，赔了六个小时。

一套能直接抄的备份做法

把三条规矩合成一套日常动作，五个步骤：

什么时候	做什么	花多久
目录建好的那天	分三层：archive 只读、work 可写、out 可写；archive 设只读并验一次	二十分钟，一次性的
每次让它动文件前	双击 backup.bat，或者 git commit 一次	三十秒
每次让它动文件前	给它文件白名单，让它先报改动对照表	一到两分钟
它干完之后	按修改时间排序目录，核对改动文件数	二十秒
每周五下班前	把这周的一份备份复制到另一个地方（移动硬盘或单位的网络盘）	五分钟

第五步是防第一二步失效的。我有一次连续一个月只在本地做备份，结果那台机器的硬盘坏了，本地备份和原文件一起没了。最后是从周五那份异地备份里救回来的，损失一周。

三层目录与三份备份

原件只读、工作可写、输出可丢；出事那天再做，已经晚了

原件层 archive：只读

客户原件、设备原始导出、不能重做的东西；设完要验一次，让它试着写应当报错

工作层 work：可读写

它唯一能改的地方，每次动手前备份；给它文件白名单，让它先报改动对照表

输出层 out：可读写，可清空

它产出的新文件，丢了重跑就行，原件一个字节都不动

三份备份：动手前的本地快照、版本管理的一次提交、每周五复制到异地的那一份

图 23-7 三层目录与三份备份：原件只读、工作可写、输出可丢

图 23-7 三层目录加三份备份：本地快照、版本管理、异地一份。（作者根据公开资料绘制）

23.6 什么时候该放弃

止损的最后一步是承认这事救不回来，或者不值得救。四种情况，遇到任何一种，就停。

情况一：试了三次还不行

三次是硬线。第一次是意外，第二次是巧合，第三次说明你没找到原因，只是在换姿势碰运气。

三次之后该做的不是第四次，是换路径：

- 换更小的任务。让它只做一个文件、只处理十行。能过就把规模一点点加上去，加到哪一步挂了，那条线就是边界。
- 换一个说法。同一件事换个角度描述，有时候就过了。这说明问题在你的表述，不在工具。
- 换回人工。这个活本来就要四十分钟，你已经花了一小时排查，剩下的自己干。

我给自己定的规矩是：排查时间超过这个活本身人工耗时的两倍，就放弃。一份四十分钟的活，我排查超过八十分钟就自己动手。这是止损，不是认输。

情况二：出错代价太高

判据跟第 22 章那一条是同一条：这份东西错了，需不需要有人出面道歉、改文件、或者重做一批实物。

这类活的放弃方式不是不用 AI，是改用 AI。让它打草稿，你出终稿。让它做中间环节，你做最后一环。

具体到操作：这类活我不让它碰原始文件。我让它把结果输出到 `out` 目录的新文件里，原件一个字节都不动。最后是我自己把结果合并进去。多花十分钟，换来的是原件不会出事。

情况三：信息不能外传

判据是：这份资料允不允许离开公司的网络边界。

这一条跟前两条不一样，它不是“该放弃”，是“该换路”。三条路：把模型跑在本地、脱敏后再交给它、只让它干不碰数据的部分（写脚本、写模板、写流程）。第 22 章 22.3 节讲过，这里不重复。

只补一句：如果你在排查过程中发现，要解决问题就得把不能外传的资料发出去，那就停在这里。这个代价无法估上限，任何一次排查的收益都盖不住它。

情况四：这活本来就要人判断

判据是：这件事的最后一步是“我决定”，而不是“它算出来”。

典型的是供应商定点、让步接收、要不要改设计。AI 能把材料备齐，这一步很值钱。但最后那一下，它做不了，因为决定需要的那部分信息不在任何文件里。

这种情况的放弃方式很特别：放弃让它给结论，不放弃让它给材料。我现在的做法是让它把三个选项各自的代价和风险列成表，我看表，我做决定。这个用法依然是正收益。

放弃不丢人，硬撑才贵

我把这四种情况列出来，是因为我自己硬撑过。

今年六月有一阵子，我非要让它自动登录一个供应商门户抓数据。那个门户有验证码、有动态 token、还会在连续请求之后临时封 IP。我前后折腾了三个晚上，试了各种办法，最后能跑通，但成功率只有六成左右，每次失败都要我人工补一次。

后来我算了一笔账：三个晚上的投入，加上每两次就要人工补一次的持续成本，远超我自己手工翻那五个页面的二十分钟。

真正的转折是我承认了一件事：这个活的价值不在自动化本身，在我拿到那张资质表。我后来改成了让它只做后半段：我自己花二十分钟把数据从门户上导出成文件，后面的整理、比对、汇总、出表全部交给它。这样每一步都稳定，总耗时从原来的两个晚上降到四十分钟。

放弃一部分自动化，换回来的是整个流程能长期跑。这笔账我算晚了两个月。

老兵说：会止损的人不是认输得快，是算得清。硬撑三个晚上的成本，跟二十分钟手工干的收益，这两个数放一起看，答案就很清楚。

全书最后一段

2025 年 12 月那个凌晨一点，我坐在工位上翻一份设备清单的 PDF，翻到第四十页的时候手停了一下。那天晚上我干的全是复制粘贴，没有一个动作需要我这个人。

从那晚到今天，隔了十个月。

这十个月里，我手上多了一张工位。它会登门户、会读表格、会翻 PDF、会写报告、会一整夜盯着台架把数据记下来。第 4 篇里那七十二个场景，每一个都是它替我干过的活，最早那几个还是我照着别人的提示词一个字一个字抄来的。

它也让我赔进去过东西。第 17 章那次评审会上的五秒钟沉默，第 22 章那份引用了不存在条款的技术说明，还有这一章开头那 3862 行被覆盖掉的原始数据。三次都是同一种错：我把东西交出去了，但没先想清楚交出去之后还收不收得回来。

这十个月里它唯一没有替我做过的的一件事，是签字。

它交回来的每一份东西，最后都要经过我的一道固定动作：样本数对不对，单位对不对，那个数能不能追到原始记录。这三个问完，过了，我在上面写我的名字。没过，退回去重做。这个动作不复杂，也没法交出去，因为签字的人要扛后果。

所以这本书最后一章教的就一件事：在它出错的那一分钟里，怎么把手上的东西保住。停、看、退，三个动作二十秒。备份、只读、看清单，三件事平时做。速查表那三十条，我建议你打印出来贴在显示器边上，它会被翻烂的。

凌晨一点那晚我缺的那个人，今天坐在我的电脑里；那晚唯一没缺的那件事，判断，还长在我身上。

小白词典

止损：出错之后先让损失停下来，而不是先去修。类比：车间里发现尺寸不对，第一件事是按急停，不是接着往下加工。

回滚：把文件或目录恢复到改动之前的状态。类比：Word 里的撤销，只不过撤销的是一整个目录。

diff：对比两个版本的文件，把不一样的行列出来。类比：把改前改后两张图纸叠在一起对着光看，透光的地方就是改过的地方。

工作目录：它被允许读写的那个根目录。类比：给实习生的工位，工位之外的柜子他打不开。

只读：能看不能改的一种文件属性。类比：图书馆的书，你能翻，不能在上面写字。

文件白名单：这次只允许它碰的这份文件清单。类比：给装修工人的一把钥匙，只开这一扇门。

版本管理：把文件每次的改动都记一笔，随时能退回任意一次。类比：施工日志，哪天改了什么都有记录，出事能查到那天。

复现步骤：让别人照着做也能重现同样问题的一串动作。类比：菜谱，写清几勺几克，别人照着做出来是同一个味。

快照：某一时刻整个目录的一份完整复制。类比：手机备份，还原的时候回到你备份那一刻的样子。

MCP 服务：给 AI 接外部工具的一套标准插座和插头。类比：接线板，插上去就能用，拔了也不影响别的电器。

你的岗位怎么用

设计工程师：你手上有受控文件，改错一个版本的代价远超返工。具体动作：把 `archive` 目录设只读，把客户原件和归档全部放进去；每次让它改图或改规范之前，先让它报改动对照表，改完用 `diff` 核一遍；你最该打印贴在显示器上的是速查表第 12 条。

试验工程师：你的原始数据一旦被覆盖，就得从设备日志重新导，那是最贵的一种回滚。具体动作：设备原始导出件一律进只读目录，让它只处理副本；给它文件白名单的时候，白名单里不许出现带 `raw` 字样的文件；每周五把这一周的备份复制到网络盘一次。

质量工程师：你最常遇到的是第四类故障（产出不对），因为它专攻数字和判定。具体动作：发现数字不对，按“样本数、单位、外引事实、算术”这个顺序核，前两个两分钟能核完；报告里出现查不到的标准号，直接走第 17 章场景 62 的脑补体检。

采购 / 供应商管理：你同时踩着两条线：门户抓取容易卡登录，资料又不能外传。具体动作：卡登录就走人工接管（第 12 章场景 33），别跟验证码死磕；所有涉及厂家名和单价的资料先脱敏再给它；速查表第 15、16、20 条建议你单独抄一份。

项目经理：你的活跨部门、跨文件，越权改动这一类最容易找上你。具体动作：给每个项目单独建一个工作目录，别让它在根目录里扫；让它改文件前必须先看清单，这一条写进你给团队的操作规矩里；出事之后先按修改时间排序看文件，别急着重试。

行政 / 综合岗：你的活频次高、单件小，出问题往往不严重但很烦。具体动作：别每次都重新排查，把“换最小样例、换新对话、重启”这三招按顺序走一遍，大部分问题在前两招就解决了；求助的时候四要素写全，你在群里问一句就能拿到答案。

本章交付物

一张双面卡片：正面“出问题第一分钟”，背面“报错速查七类”。

怎么做：找一张 A4 纸，正面打印下面这块，背面打印 23.3 节那张三十条的速查表（七类分组，第一列和第三列必留，第二列可以删掉省地方）。过塑，贴在显示器下沿。

正面内容，直接照抄：

出问题第一分钟：停、看、退

【停】20 秒

1. 不再发任何新指令（不发“继续”，不发“再试一次”）
2. 不要关窗口，关了就丢了现场
3. 它在大批量写文件时，断网络比关程序快
4. 把最后三条对话复制出来，粘进一个文本

【看】20 秒

1. 让它报改动清单（快，但不可信）
 2. 按最后修改时间排序整个目录（可信）
 3. 用版本管理看 diff（最可信，前提是你提交过）
- 三个数要对得上：它报的数、目录里变的数、你让它改的数

【退】先算代价再退

- 一个文件 → 从备份拿回来（几分钟）
- 一批文件 → 整个目录从备份恢复（十几分钟）
- 整个目录 → 版本管理回上一次提交（半小时起）
- 原始输入 → 从源头重新导出（几小时，最贵）

【绝不重试】

重试会覆盖现场、会让它修它自己猜的错、会让回滚变复杂

【什么时候放弃】

试了三次还不行 / 出错代价太高 / 信息不能外传 / 这活本来就要人判断

背面内容：23.3 节那七组三十条，按七类分组排好，第一列“报错现象”和第三列“第一个该做的动作”必须保留，第二列“最可能的原因”字号可以小一号。

本周要做的三件事：第一，给自己的工作目录分三层（archive 只读、work 可写、out 可丢），这一步二十分钟，一次性。第二，把备份脚本存好并跑一次，验一遍文件数。第三，挑一张你最近让 AI 动过的目录，按修改时间排序看一眼，确认没有被你不知道的改动。

本章金句

出事的第一分钟，最贵的那个动作是重试。

它改错一个文件只用几秒，你把它救回来要几个小时。

先问它改了什么，再看它对不对，最后才谈怎么改。顺序错了，代价是六个小时。

读不到的报错，九成不是文件的问题，是路径和权限的问题。

排查时间超过这个活本身人工耗时的两倍，就自己动手。这是止损，不是认输。

会止损的人不是认输得快，是算得清。

凌晨一点那晚我缺的那个人，今天坐在我的电脑里；那晚唯一没缺的那件事，判断，还长在我身上。
(全书正文完。附录在后面。)

A P P E N D I X • 附 录

索引 • 术语 • 资源

附录 A

72 场景速查索引

这本附录是给已经读过正文的人用的。你不需要从头翻，只要知道你手上的活属于哪一类，就能在三秒内定位到该抄哪一节。

全书 72 个场景分布在第四篇第 7 章到第 18 章，编号 01 到 72 连续，无跳号、无缺号。每张表的"可信度"沿用全书统一的三级标注：

标注	含义
★★★	实测。作者在场跑过，有完整记录，数字可复核
★★	跑通。流程走通了，但留档不完整或只做过单轮，你跑出来要自己复核
★	调研。作者没有完整实操，依据公开资料与社区进展整理，动手前请评估版本与授权

"一句话说明"控制在 30 字以内，只负责帮你认出这个场景，不负责教你怎么干。

A.1 按章分组的全量索引

第 7 章 通用办公自动化

场景号	场景名	所在章	可信度	一句话说明
01	会议纪要：从录音转写到待办清单	第 7 章	★★★	60 分钟会议,35 分钟出纪要,关键是待办有人有日期
02	通知与公告：一份模板批量生成 N 个版本	第 7 章	★★★	一份模板出四个版本,落款与收件范围不再漏改
03	Excel 数据清洗：乱表格变干净表	第 7 章	★★★	合并单元格、数字存文本,一次洗净并留证
04	Word 报告格式化：照公司模板一键套版	第 7 章	★★★	套一次公司模板 45 分钟,改内容之后不再返工
05	PPT：从一份大纲生成可讲的幻灯片	第 7 章	★★★	大纲转幻灯片,省掉排版和找图的 60 分钟
06	邮件：批量处理与回复草稿	第 7 章	★★★	40 封邮件分类加回复草稿,42 分钟压下去

第 8 章 个人效率与知识管理

场景号	场景名	所在章	可信度	一句话说明
07	个人知识库：把散落的笔记和文件变成一个能问的库	第 8 章	★★★	资料散在 6 个地方,建库后 20 秒出答案带出处
08	长文拆解：一份 200 页技术标准拆成检查清单	第 8 章	★★★	200 页标准 6 小时变 45 分钟,出可照着查的清单
09	待办与日程：从混乱记录到可执行清单	第 8 章	★★★	4 处散落的待办,每天 30 分钟重建变成自动
10	周报月报：从工作痕迹自动生成,不用回忆	第 8 章	★★★	从工作痕迹生成周报,15 分钟交稿且不漏活
11	文件命名与归档：几百个乱文件一次性归位	第 8 章	★★	3182 个乱文件,3 小时出改名对照表
12	信息订阅与日报：每天自动汇总行业动态	第 8 章	★★★	扫二十来个信息源,日报 2 小时变 25 分钟

第 9 章 需求与项目定义

场景号	场景名	所在章	可信度	一句话说明
13	客户需求拆解：一份 SOR 变成内部任务清单	第 9 章	★★	40 到 60 页 SOR,3 小时拆成可分派的条目
14	技术标准提取：从标准里摘出跟你产品有关的条款	第 9 章	★	上百页标准里,只摘跟你产品有关的那几段
15	项目计划生成：里程碑、依赖关系、关键节点	第 9 章	★★	里程碑加依赖,漏的是模具认证的等待期
16	风险清单辅助：把经验变成结构化的失效模式表	第 9 章	★★	把老工程师脑子里的失效模式掏成表
17	技术方案对比：两套方案摆在一起比,比什么先定好	第 9 章	★★★	比之前先定评价维度,别让嗓门大的人赢
18	行动项跟踪：会议决定变成有人有日期的任务	第 9 章	★★	17 条口头决定,3 分钟变成有人有日期的表

第 10 章 造型与几何

场景号	场景名	所在章	可信度	一句话说明
19	从一句话到第一个零件：简单零件建模入门	第 10 章	★★★	一块安装板按规矩打孔,20 分钟压到 3 分钟
20	方向盘三维数模：一次跑通的完整案例	第 10 章	★★★	23 种零件 44 处实例,3 个工作日压到 1 小时 49 分
21	从数模到二维工程图：六张图的产出过程	第 10 章	★★★	六张合规工程图,过去占掉设计四成工时
22	数模体检：毫米单位制的六个陷阱与三个自检动作	第 10 章	★★★	单位、选边、体积三件事,十分钟挡掉九成事故
23	装配体与干涉检查：多零件怎么一起交给 AI	第 10 章	★★★	44 处实例插进去不打架,重开后变换仍记得住
24	工业造型与渲染：Rhino / Blender 一侧的能力与边界	第 10 章	★	让产品好看、让评审的人看懂的那一侧

第 11 章 电子软件与仿真

场景号	场景名	所在章	可信度	一句话说明
25	PCB 设计辅助：让 AI 驱动 EDA 软件干活	第 11 章	★	四层板布局,三分之二时间花在重复点击上
26	PCB 检查与 BOM 核对：网表与物料清单的一致性	第 11 章	★★	网表与 BOM 之间的漂移,3 小时对账压到 20 分钟
27	控制器软件开发：模型与代码的生成辅助	第 11 章	★	从模型到 C 代码,耗时大头是接口与配置
28	代码审查与规范检查：AI 当第一道关	第 11 章	★★	1800 行代码 20 分钟过一遍,出的是线索不是结论
29	CAE 仿真前处理：让 AI 准备求解器输入	第 11 章	★	几何清理到求解器输入,一天活里最重复的段
30	仿真结果解读与报告：从结果文件到结论	第 11 章	★★	提最大值、定位置、下判断,3 小时压到 20 分钟

第 12 章 让 AI 碰到你的电脑

场景号	场景名	所在章	可信度	一句话说明
31	浏览器自动化：让 AI 用你的登录态干活	第 12 章	★★★	十几个要登录的站点,2 小时搬运压到 40 分钟
32	批量抓取没有接口的网页数据	第 12 章	★★★	四类站点一个工作日,实测 50 分钟出 59 条
33	卡住了怎么办：human-in-loop 接管	第 12 章	★★★	验证码和二次验证,它弹窗叫你点一下接着干
34	Computer Use：让 AI 直接操作桌面软件	第 12 章	★★	桌面软件这条路上,菜单与版本以你装的为准
35	远程操控：人在家盯台架	第 12 章	★★★	台架跑几十小时,人在家用手机看数据拉曲线
36	安全边界：什么能交给 AI 碰,什么不能	第 12 章	★★★	一张系统分级表,写清什么能碰什么不能碰

第 13 章 设计审核与风险

场景号	场景名	所在章	可信度	一句话说明
37	评审材料预审：开会前先让 AI 挑一遍刺	第 13 章	★★★	几十页整包材料,25 分钟出带证据位置的清单
38	图纸与数模检查清单：从经验清单到自动核对	第 13 章	★★★	老工程师的清单,一张 A3 图 7 分钟变自动
39	失效模式联想：DFMEA 辅助	第 13 章	★★	三天写表不难,难在想出这个件能怎么坏
40	公差与尺寸链校核：让 AI 帮你算,你来判断	第 13 章	★★★	六环尺寸链两种算法,4 小时压到 20 分钟
41	变更影响分析：改一个尺寸会牵动哪些件	第 13 章	★★	改一个轴径,查清牵动哪些件要一天半
42	经验教训库：把过去的问题变成可检索的资产	第 13 章	★★	几百份 8D 与故障单,变成能检索的东西

第 14 章 供应链与产线

场景号	场景名	所在章	可信度	一句话说明
43	供应商资质核查：五家资质一次对完	第 14 章	★★★	五家资质摊成矩阵,两个晚上压到 40 分钟

场景号	场景名	所在章	可信度	一句话说明
44	报价对比与降本分析	第 14 章	★★★	五套格式先统一口径,三年总成本才可比
45	来料检验数据整理: 从检验记录到趋势	第 14 章	★★★	三个来源四种写法,1 个工作日压到 45 分钟
46	产线异常记录分析	第 14 章	★★★	纸质本加群消息加报警,2 天变 1 小时排序
47	产能与节拍数据梳理: 找出瓶颈在哪	第 14 章	★★	三处数据凑成产能表,先找瓶颈再加设备
48	质量问题的 8D 报告辅助: 从事实到报告	第 14 章	★★★	8 小时出 D1 到 D3,1.5 个工作日压到 1 小时

第 15 章 NVH 与试验

场景号	场景名	所在章	可信度	一句话说明
49	试验数据整理: 从原始记录到可用的曲线与表格	第 15 章	★★★	CSV、二进制、手写表,最惨一次两人两天半
50	阶次分析与噪声源识别: 让 AI 帮你读谱	第 15 章	★★★	先让它算特征频率表,你确认后它才许动数据
51	试验报告自动生成	第 15 章	★★★	核心是每个数字对应哪个源文件的对照表
52	声学数据库搭建与检索	第 15 章	★★★	30 多个样本做成能用自然语言问的库
53	微电机与齿轮传动件的声学问题定位	第 15 章	★★★	客户说声音不对,给一套可复制的定位流程
54	把老师傅的经验蒸馏成 AI 可用技能	第 15 章	★★★	老师傅带走的判断,做成能装进 AI 的仓库

第 16 章 让 Agent 不跑偏

场景号	场景名	所在章	可信度	一句话说明
55	完成条件: 先说清什么叫干完了	第 16 章	★★★	先写清什么叫干完,验收不再靠感觉翻文件
56	边界条件: 不许它碰什么、什么时候必须停下问人	第 16 章	★★★	范围、动作、决策三层边界,越界就停下来问
57	反指标: 防止它把指标刷好看	第 16 章	★★★	主指标之外配反指标,防它跳过难的把数刷绿
58	中途检查点: 长任务怎么分段验收	第 16 章	★★★	按风险切成四段,先看计划再看样件再放量

场景号	场景名	所在章	可信度	一句话说明
59	让它自己汇报进度：进度模板与异常上报	第 16 章	★★★	不收完成 80%,按四类报数并单独上报异常
60	漂移检测与纠偏：发现跑偏了怎么拉回来	第 16 章	★★★	每个检查点做漂移检测,异常只做停、比、续

第 17 章 AI 的七宗罪与反制

场景号	场景名	所在章	可信度	一句话说明
61	抢答：你还没说完它就开工了	第 17 章	★★★	没问清边界就开工,清单里混进别的平台条款
62	脑补：它编了一个不存在的数	第 17 章	★★★	编一个标准号或牌号,后面的活全按错的做
63	独断：它替你做了本该你做的决定	第 17 章	★★★	三分钟给的最优解,用的是它自己的优先级
64	加戏：它干了你没让它干的事	第 17 章	★★★	改一处参数,它顺手把整份文档都重写一遍
65	话痨：它说了一大堆你不需要的	第 17 章	★★★	结论藏在第六页,让它第一屏就把结论给你
66	失忆：干到一半忘了前面的约定	第 17 章	★★	五轮长任务写到第四轮,术语口径全变了
67	报喜：它只告诉你好的那部分	第 17 章	★★★	12 台样件坏 2 台,报告只写 10 台说全通过
68	贪心元认知：它对自己能力的判断不可信	第 17 章	★★	它说很确定的时候可能错得离谱,给三件校准工具

第 18 章 验收溯源与责任

场景号	场景名	所在章	可信度	一句话说明
69	验收三档：快速核、抽样核、全量核	第 18 章	★★★	按出错代价选档,五分钟决定能不能发出去
70	溯源：让每个数字都能追到来源	第 18 章	★★★	每个数字十秒内追到它出生的那一行
71	责任边界：L1 / L2 / L3 决策矩阵	第 18 章	★★	派活前先定级,写清哪类活它只许给选项
72	出事之后：止损、溯源、改规矩	第 18 章	★★★	先止血再查清最后改规矩,半天出一份复盘

A.2 按岗位查场景

这张表的用法是：找到你的岗位，按行内顺序抄，不要跳。每行都按"先见效、再上手、最后啃硬骨头"排过序。

岗位	最该先抄的场景	为什么是这几个
设计工程师	17、19、22、38、40、53、55、63	17 先定评价维度再比方案；19 拿明天就要用的真零件开局；22 是签字前的体检查验单；38 把老工程师的图纸清单自动化；40 让它算尺寸链你判前提；53 噪声改进方案按成本排序，第一条常常不是声学措施；55 把完成条件写成文件加尺寸加证据位置；63 评审前让它摆三条路，每条写清放弃什么
试验工程师	03、11、35、49、50、51、67、70	03 台架数据清洗完直接标超差行；11 归档做完，周报能直接吃文件修改时间；35 人在家盯几十小时的台架；49 和 51 占你工作一半以上；50 先算特征频率表再动数据；67 把四个样本数闭合写进模板第一行；70 对照表做成报告第二页，每个数指得到行号
质量工程师	08、16、45、48、62、67、69、70	08 标准拆完变成审核表；16 DFMEA 的打分权握在你手里，它只做联想；45 让步接收不许并进合格；48 员工疏忽类根因直接删掉重写；62 脑补体检提示词存成技能；67 四行证据表做成部门报告模板；69 三档检查表改成检查单，抽样清单由你随机挑；70 追不到来源的数按可疑处理
采购与供应链管理	12、26、31、32、43、44、55、71	12 订阅源换成供应商官网和招标公告；26 外协厂的 BOM 进门先对一遍；31 和 32 解决十几个要登录的门户；43 资质矩阵一次对完五家；44 口径表里的年降条款决定三年后谁最便宜；55 把交付物版本和缺件处理写进卡片；71 比选询价定成 L2，涉及金额的动作一律 L3
项目经理	01、13、15、18、36、43、58、71	01 的待办栏可以直接当任务分派依据；13 需求条目表直接进项目管理系统；15 长周期件的等待时间必须你自己给；18 行动项每周都在漏；36 不是安全文档，是责任划分文档；43 定点会上那两个问题的答案在这里；58 长任务按风险切四段；71 启动会上公布本项目的 L3 清单
研发主管	18、36、42、52、54、71、72	18 的待决策表把"谁决策、缺什么、不决策的后果"三列摆开；36 分级表是团队敢放手干的前提；42 和 52 解决人员流动，人走了库还在；54 难点不在技术，在于让人把"我上次是这么干的"讲出来；71 每个 L2 的活保留停点记录；72 出事之后的规矩改动由你来签

如果你的岗位不在这六类里：行政、文秘、人事这类岗位，从场景 02、04、11 开始，这三件没有技术门槛，见效最快；工艺与制造工程师盯场景 21、22、41、46、47；仿真与 CAE 工程师从场景 30 开始，因为它只读不改。

A.3 按紧急程度查场景

"紧急程度"按你今天下午能不能拿到东西分档，不按场景难度分档。

今天就能见效的 10 个

不需要准备数据、不需要装东西、不需要说服任何人，打开就能干。

场景号	场景名	今天能拿到什么
01	会议纪要	下午那场会的纪要和待办清单
02	通知与公告	一份模板的四个版本，落款不再错
03	Excel 数据清洗	一张干净表加一份"被删行"留档
04	Word 报告格式化	套好公司模板的月报
06	邮件	40 封邮件的分类和回复草稿
18	行动项跟踪	周会上 17 条口头决定变成一张表
33	human-in-loop 接管	卡住的地方不再瞎点，等你点一下
36	安全边界	一张三类分级的系统清单
55	完成条件	派活之前先把"干完"写清楚
69	验收三档	五分钟决定这份产物能不能发出去

一周内见效

需要半天到一天准备：整理一份自己的模板、跑通一次流程、或者跟同事说一声。

场景号	场景名	需要什么准备
05	PPT	一份现成的大纲或报告
07	个人知识库	把六个地方的资料归到一个目录
08	长文拆解	手边一份客户规范或行业标准
09	待办与日程	把四个来源的入口理一遍
10	周报月报	一周的工作痕迹，哪怕只是文件修改时间
12	信息订阅与日报	二十来个信息源的清单
13	客户需求拆解	一份真实的 SOR
17	技术方案对比	花 5 分钟先定评价维度
22	数模体检	把六行体检检查表打印贴在显示器边
31	浏览器自动化	一次登录态的授权确认
32	批量抓取网页数据	四类站点的字段清单
37	评审材料预审	一整包几十页的评审材料
38	图纸与数模检查清单	一张你自己写的老清单

场景号	场景名	需要什么准备
43	供应商资质核查	五家的资质文件
44	报价对比与降本分析	一个明确的资金成本利率数字
45	来料检验数据整理	三个来源三个月的记录
46	产线异常记录分析	纸质本、群消息、报警导出三样凑齐
48	8D 报告辅助	一堆零散的事实
49	试验数据整理	一次试验的原始记录
51	试验报告自动生成	一次算完的结果文件
56	边界条件	把范围、动作、决策三层各写一遍
57	反指标	给每个主指标配一组反指标
58	中途检查点	按风险把任务切成四段
59	进度汇报	定好四类报数的格式
60	漂移检测与纠偏	每个检查点留一份基线快照
62	脑补	一份待核的检测报告
63	独断	让它先摆三条路，每条写清放弃什么
64	加戏	一张文件白名单加一份明确不做清单
65	话痨	把输出格式改成结论先行
67	报喜	一张四行证据表
70	溯源	一张数字来源对照表模板
71	责任边界	你自己岗位的一份 L3 清单

需要准备的

这几类要么要装东西、要么要谈授权、要么要攒数据，别指望一次下午就跑通。先排进计划，别临时起意。

场景号	场景名	卡在哪
11	文件命名与归档	三千多个文件，要先定命名规则
14	技术标准提取	要拿到完整的标准正文与附录
15	项目计划生成	长周期件的等待时间要找采购要
16	风险清单辅助	要凑齐五个人开一下午评审会
19	从一句话到第一个零件	要装 CAD 环境并跑通接口
20	方向盘三维数模	完整作业，建议照着九步走一遍
21	从数模到二维工程图	要有公司的图框和制图规范

场景号	场景名	卡在哪
23	装配体与干涉检查	要有完整 BOM 与配合关系
24	工业造型与渲染	要装 Rhino 或 Blender 并配好插件
25	PCB 设计辅助	要确认 EDA 版本与授权范围
26	PCB 检查与 BOM 核对	要有网表与 BOM 两份底账
27	控制器软件开发	要确认 MATLAB 与代码生成配置
28	代码审查与规范检查	要有本团队的编码规范文档
29	CAE 仿真前处理	建议先用开源求解器练手
30	仿真结果解读与报告	需要一个已经算完的算例做基线
34	Computer Use	要确认桌面软件版本与权限
35	远程操控	要装被控端、设开机自启、配远程开机
39	失效模式联想	要先把历史问题目录建起来
40	公差与尺寸链校核	要给全每个环的真实工艺来源
41	变更影响分析	要翻图纸、BOM、工装、检具、大纲、DFMEA
42	经验教训库	要靠时间攒，越早建越值钱
47	产能与节拍数据梳理	三处数据在三个部门手上
50	阶次分析与噪声源识别	要有转速信号与特征频率表
52	声学数据库搭建与检索	要有足够多的历史样本
53	微电机声学问题定位	要有样机和一套测点布置
54	经验蒸馏成技能	第一版花了六个月，走的是访谈
61	抢答	要先把开工确认单写死
66	失忆	要建三份状态文件并每轮开工前读一遍
68	贪心元认知	要跑探针题与反证测试做校准
72	出事之后	要有一次真实事故可以复盘

A.4 索引完整性说明

- 覆盖场景数：72 个，编号 01 至 72，全部收录。
- 缺失编号：无。第 7 章到第 18 章各章的场景编号连续，无跳号，不需要标注“该章暂缺”。
- 各章场景数：第 7 章至第 16 章每章 6 个，第 17 章 8 个（61 至 68），第 18 章 4 个（69 至 72）。
- 可信度分布以各章正文标题后的星标为准，本附录不做上调或下调。

附录 D

全书术语表

翻到这里的理由只有一个：正文里有个词你没看懂，又不想往回翻二十页。

这张表收的是全书各章「小白词典」和场景「名词小词典」里出现过的术语，去重之后按主题分成七类。每条给四件事：术语、英文或缩写、一句话的大白话、它第一次出现在第几章。

用法很简单：按主题找，找到之后如果想看完整的类比和例子，翻回那一章的「小白词典」。

D.1 AI 与工具术语

术语	英文或缩写	大白话解释	首次出现
AI 模型	Model	听得懂人话、能推理能生成的计算机程序，它只负责想，动手的是装着它的软件	第 1 章
Agent	Agent	不只回答问题，还能自己规划步骤、调用工具、把一件事干完的 AI 工作方式	第 1 章
提示词	Prompt	你对 AI 说的那段完整的话，说得更清楚，活就干得更好	第 1 章
幻觉	Hallucination	AI 把不知道的东西说得煞有介事	第 1 章
技能	Skill	给 AI 预装的一份岗位手册，装一个它就多会一类活	第 1 章
连接器	Connector	让 AI 接上某一个外部服务的那根线，一根线接一个设备	第 1 章
上下文	Context	一段对话里聊过的全部内容，包括你说的话、它读过的文件、已得出的结论	第 3 章
上下文窗口	Context Window	它能同时看到的内容长度上限，超了之后最早的内容会被压缩或丢掉	第 17 章
主代理	Main Agent	接住总任务、安排分工、验收中间结果并组装成品的那个 AI	第 6 章
子代理	Sub Agent	主代理为一块具体工作临时安排的 AI，拿到限定材料和完成标准，交件后结束	第 6 章
并行	Parallel	几块互不等待的活同时进行	第 6 章
串行	Sequential	后一步必须等前一步的结果，顺序不能换	第 6 章

术语	英文或缩写	大白话解释	首次出现
蒸馏	Distillation	把散在脑子、笔记、旧报告里的经验，变成有结构、能检索和引用的文件	第 15 章
路由	Routing	判断一个问题属于哪个领域，再去加载对应知识的入口逻辑	第 15 章
触发特征	Trigger Pattern	一段话里出现哪些词，就该加载哪块知识	第 15 章
跨域	Cross-domain	一个问题同时涉及两个以上的领域	第 15 章
反例	Counterexample	这条经验在什么情况下不适用	第 15 章
元认知	Metacognition	对自己认知过程的判断，也就是「我知道我不知道什么」这件事	第 17 章
校准	Calibration	说「我有八成把握」的时候，实际正确率是不是真的有八成	第 17 章
探针题	Probe Question	你事先知道答案的题，用来测它当前状态下是不是在编	第 17 章
反证测试	Falsification Test	让它自己去找能推翻它结论的证据，找不出来说明它压根没想过	第 17 章
置信度分档	Confidence Level	把答案按把握程度分成五档，好处是逼它承认有「不知道」这一档	第 17 章
贪心	Greedy	什么都想要、什么都想要到极致，这一条是我们自己的问题	第 17 章
弱事实	Weak Fact	一个很弱的事实，比如「文件写出来了」，推不出「结果能用」	第 17 章
强结论	Strong Claim	一个很强的判断，比如「全部通过」，它需要多个证据支撑	第 17 章
MCP	Model Context Protocol	一套让 AI 操控其他软件的通用插座规矩，两边照着做就能接上	第 1 章
客户端	client	AI 那一侧，把 AI 的话变成符合规矩的请求发出去，再把回包交还给它	第 20 章
服务端	server	你要接的软件那一侧，一个跑在电脑上的小程序，负责报工具清单	第 20 章
进程	process	一个正在运行的程序，一个 MCP 服务端就是一个进程	第 20 章
工具	tool	服务端上报的一个具体动作，一个工具就是一件事	第 20 章

术语	英文或缩写	大白话解释	首次出现
JSON	JSON	用大括号和方括号组织数据的文本格式，人和机器都能读	第 20 章
RPC	Remote Procedure Call	让一个程序去调用另一个程序里的函数，并且拿到返回值	第 20 章
JSON-RPC	JSON-RPC	用 JSON 写成的 RPC，问一句答一句都是一段 JSON，靠编号对上号	第 20 章
stdio	Standard Input Output	程序自带的两根管道，一进一出，不占端口、不走网络	第 20 章
端口	port	一台电脑上用来区分不同网络程序的编号	第 20 章
transport	transport	服务端和客户端用哪根线说话，常见的是 stdio 和网络两种	第 20 章
schema	schema	描述「参数长什么样」的说明书，AI 靠它才知道该往里填什么	第 20 章
握手	handshake	连接建立时双方先互相确认一下在不在、会什么	第 20 章
环境变量	env	进程启动时带在身上的一批设置值，比如编码方式、临时目录	第 20 章
装饰器	decorator	Python 里写在函数上面、以 @ 开头的那行代码，给函数额外登记一点信息	第 20 章
COM	COM	Windows 上软件之间互相调用的一套老接口，比 MCP 早得多	第 10 章
pywin32	pywin32	Python 一侧用来敲 COM 这道门的库	第 20 章
接口	API	软件之间约定好的一条正式数据通道	第 12 章
OAuth	OAuth	授权协议，你不用把密码告诉对方，对方拿到的是一个有时限的令牌	第 19 章
令牌	Token / Cookie	授权成功后对方拿到的那串凭证，它代替你的密码去访问服务	第 19 章
权限范围	Scope	这把钥匙能开到什么程度，只读、可写、可发送是三种范围	第 19 章
授权	Authorization	你点「同意」把钥匙交给对方的那个动作	第 19 章
限流	Rate Limit	服务方规定单位时间内最多允许你调多少次，超了就拒绝	第 19 章

术语	英文或缩写	大白话解释	首次出现
只读 / 可写	Read-only / Write	只读是能看不能改，可写是能新建、能改、能删	第 19 章
结构化写入	Structured Write	按标题、小节、表格这些结构写进在线文档，不是把一大段文字糊进去	第 19 章
结构化	Structuring	把页面上散着的文字整理成一行一行、一列一列的表格	第 12 章
检索	Retrieval	先从一堆卡片里找出相关的那几张，再回去翻原文	第 8 章
索引	Index	给每个文件做一张身份证卡片，以后找东西先查卡片，不用翻文件	第 8 章
出处引用	Citation	每条回答后面标上来自哪个文件、第几页，没有出处的答案不算数	第 8 章
human-in-loop	Human-in-the-loop	AI 跑到自己搞不定的地方把控制权交回给人，人处理完再交还给它	第 12 章
Computer Use	Computer Use	让 AI 像人一样操作一整台电脑，看屏幕、动鼠标、敲键盘	第 12 章
脚本驱动	Script Driving	操作桌面软件的主流方式不是模拟鼠标点，而是写一段软件自己的脚本去调它	第 12 章
能力边界	Capability Boundary	它能干的事和它干不了的事之间那条线，搞清楚边界比搞清楚功能重要	第 12 章
审美判断	Aesthetic Judgment	这个方案好不好看、配色够不够高级，这一类判断不该交给 AI	第 12 章
浏览器扩展	Browser Extension	装在浏览器里的小插件，能看见页面、能替你点击和填表	第 12 章
隔离窗口	Isolated Window	AI 专门新开的浏览器窗口，它只在这个窗口里干活，你的窗口不受影响	第 12 章
本机回环	Loopback	一台电脑自己跟自己通信用的内部地址，不走外网，数据不出本机	第 12 章
登录态	Login State	你在某个系统里已经登录的状态，凭据存在浏览器里，系统见了就认你	第 12 章
会话	Session	一次「AI 接管浏览器」的时段，开会话和关会话是两道必须做的动作	第 12 章
二次验证	2FA	登录时除了密码还要第二道验证，AI 看不到你的手机所以过不去	第 12 章
验证码	CAPTCHA	页面上让你认图、点选、拖滑块的关卡，设计初衷就是挡机器	第 12 章

术语	英文或缩写	大白话解释	首次出现
确认弹窗	Confirm Dialog	点某个按钮之后跳出来的「确定要执行此操作吗」，出现就说明这一步有后果	第 12 章
接管超时	Takeover Timeout	你被叫了但没及时响应，会话等太久自己断了	第 12 章
SSO	Single Sign-On	一次登录之后公司里所有系统都认你，它是权限的总闸门	第 12 章
最小权限	Least Privilege	只给它干这件事必需的权限，多一分都不给	第 12 章
审计留痕	Audit Trail	谁在什么时候做了什么，系统有记录可查，不留痕的系统不配被自动化	第 12 章
上位机	Host Computer	试验台架旁边那台控制试验、采数据的电脑	第 12 章
主控端 / 被控端	Controller / Controlled	被控端是你连过去的那台机器，主控端是你连过去用的设备	第 12 章
远程开机	WOL	机器关机之后还能被网络唤醒，不用人过去按电源	第 12 章
隐私防窥	Privacy Screen	你远程控制的时候被控端那块屏幕是黑的，旁人看不到内容	第 12 章
长效验证码	Persistent Code	默认的动态码一次一换，长期连某台设备时自定义一个码可以一直用	第 12 章

D.2 WorkBuddy 专有术语

术语	英文或缩写	大白话解释	首次出现
WorkBuddy	WorkBuddy	本书主角，装在电脑上的 AI 工作台，Agent 能力加三种模式加记忆加技能加连接器	第 1 章
模式	Mode	干活的三种委托方式，Ask 是问、Plan 是谋、Agent 是干	第 3 章
模式切换	Mode Switch	三种委托方式的选择开关	第 2 章
Ask 模式	Ask	只读只答，不动你任何文件	第 2 章
Plan 模式	Plan	先出方案等你点头，你不点头它不动手	第 1 章
Agent 模式	Agent	直接读写文件、跑程序、把活干完交差	第 2 章
工作目录	Working Directory	AI 默认能看见、能读写的那个文件夹	第 2 章

术语	英文或缩写	大白话解释	首次出现
工作区	Workspace	一个具体的文件夹或项目，换个工作区它的记忆就读不到	第 5 章
任务状态区	Task Status Area	显示它正在干第几步、卡在哪一步的那一块界面	第 2 章
记忆	Memory	把你交代过的背景、偏好、规矩存下来，下次对话直接调用的能力	第 5 章
云端画像	Cloud Profile	跟着账号走的那层记忆，存身份、岗位、行业、语言偏好	第 5 章
用户级记忆	User Memory	你这个账号下的通用工作习惯，跨项目生效	第 5 章
工作区记忆	Workspace Memory	跟着某个具体文件夹或项目走的那层记忆	第 5 章
记忆文件	Memory File	存工作区记忆的那个文件，通常放在工作区根目录下	第 5 章
回显	Echo	让它把刚写进去的内容原文再打一遍给你看，这是验收记忆的唯一可靠动作	第 5 章
脱敏	Desensitization	把真实的名字和数值换成代号或量级	第 5 章
SKILL.md	SKILL.md	技能的核心文件，开头是两个横杠之间的元信息，下面是给 AI 看的正文	第 4 章
元信息	frontmatter	文件最开头用两个横杠包起来的一小段配置，写技能名、描述、版本、权限	第 4 章
触发词	Trigger Word	写在描述里的那些话，AI 听到你说这些话就会加载这个技能	第 4 章
评测集	Eval Set	一批用来给技能打分的测试题，通常来自历史任务记录	第 4 章
模型打分	LLM-as-Judge	让另一个模型当裁判打分，好处是能看懂语义，坏处是它自己也不稳	第 4 章
Bootstrap 检验	Bootstrap Test	重复抽样几千次，看两个版本的差距是真还是碰巧	第 4 章
变体	variant	技能的某个改动版本，一代一代往下改	第 4 章
占位图	Placeholder	先占住位置、等真正的图来了再换掉的临时图片	第 15 章

D.3 工程与制造术语

D.3.1 需求、标准与项目

术语	英文或缩写	大白话解释	首次出现
SOR	Statement of Requirements	客户发给供应商的技术需求文档，它是验收依据，不是技术方案	第 3 章
条款	Clause	SOR 里带编号的一条要求，验收时靠这个编号回原文核对	第 3 章
歧义条款	Ambiguous Clause	写得含糊、缺边界条件或前后不一致的条款，必须在回复客户前问清楚	第 3 章
待定表	TBD List	归类不确定的条目单独放的一张表，作用是防止 AI 硬猜	第 3 章
must / shall / should		SOR 里的强制程度标记，must 和 shall 必须满足，should 是建议	第 3 章
强制与推荐	shall / should	标准里 shall 是必须做，should 是建议做，检查时权重完全不同	第 8 章
标准	Standard	规定产品必须做到什么、怎么验证的文件，里面的条款要逐条核对	第 8 章
标准号	Standard Number	标准的身份证，带年份的版本号很重要	第 9 章
条款号	Clause Number	标准里每一条的编号，写检查清单必须带，否则出问题没法追溯	第 8 章
标准条款号	Clause ID	标准里每一条规定的编号，号错了条款就是另一条，试验方法跟着全错	第 17 章
适用范围	Scope	标准第一章，说明它管什么、不管什么，先看这一章能省掉后面大量无效阅读	第 9 章
规范性附录与资料性附录	normative / informative	规范性附录必须执行，资料性附录只是参考，混在一起是误读的主要来源	第 9 章
引用文件	Referenced Document	标准里写着「按某某执行」的其他文件，正文没写细节，细节在里面	第 8 章
检查清单	Checklist	把「要做到 A」改写成「怎么确认做到了 A」的一张表	第 8 章
工况参数	Operating Condition	产品干活时的外部条件，扭矩、转速、温度范围、电压范围、循环次数	第 9 章
可验证性	Verifiability	这条需求能不能用测量、试验、计算或查证来判定合格	第 9 章

术语	英文或缩写	大白话解释	首次出现
接口边界	Interface Boundary	你的零件跟别人零件交界的地方，比如安装法兰尺寸、接插件型号	第 9 章
CSR	Customer Specific Requirements	客户在通用标准之外额外加的条款，最容易被漏，常藏在附件最后一页	第 9 章
设计输入清单	Design Input List	把 SOR 里散落各处的条款，整理成一条条能分给工程师做的条目	第 17 章
DVP	Design Verification Plan	设计验证计划，写明要做哪些试验、各做几次	第 17 章
里程碑	Milestone	项目里必须交付的节点，它是任务的成果，不是任务本身	第 9 章
WBS	Work Breakdown Structure	把项目一层层拆成两周内干完、只有一个负责人的小任务	第 9 章
DV 与 PV	DV / PV	DV 是设计验证，验设计对不对；PV 是生产验证，验量产做不做得出来	第 9 章
SOP	Start of Production	量产启动，项目终点	第 9 章
减速比	Gear Ratio	输入转速除以输出转速	第 9 章
单级传动与两级传动		一级减速就做完总减速比叫单级，分两级做叫两级	第 9 章
包络尺寸	Envelope Dimension	零件占的最大外形尺寸	第 9 章
变位系数	Profile Shift Coefficient	齿轮加工时刀具相对毛坯的偏移量，正变位能把齿根加厚、避开根切	第 9 章
根切	Undercut	齿数太少时加工会把齿根挖掉一块，削弱强度	第 9 章
传递误差	TE	实际输出角度跟理论输出角度的差值，齿轮噪音的主要来源之一	第 9 章
精度等级	Grade	用 Grade 加数字表示齿轮精度，数字越小越精密，也越贵	第 9 章
偏差符号	Deviation Symbol	标准里用来表示具体公差项目的代号，看不懂符号条款就读不明白	第 9 章
牌号	Material Grade	材料的身份证，牌号后面挂着的是一整套性能参数	第 17 章

D.3.2 质量、风险与问题解决

术语	英文或缩写	大白话解释	首次出现
DFMEA	Design FMEA	一张表，逐条写这个产品能怎么坏、坏了多严重、为什么坏、怎么提前发现	第 9 章
失效模式	Failure Mode	零件坏的方式，写法必须是「零件加动词加偏差」，不能写「强度不足」	第 3 章
失效后果	Failure Effect	坏了之后用户感受到的是什么	第 13 章
失效影响	Failure Effect	这个失效发生之后，客户或者下一道工序会看到什么	第 9 章
失效起因	Failure Cause	为什么会走到这一步，比如润滑脂选型不当、热处理硬度不足	第 13 章
严重度 S	Severity	后果有多严重，1 到 10 分，涉及安全与法规的通常打到 9 或 10	第 13 章
发生度 O	Occurrence	这个起因出现的可能性，1 到 10 分	第 13 章
探测度 D	Detection	量产之前现有手段能把这个问题查出来的能力，数越大越查不出来	第 13 章
RPN	Risk Priority Number	风险优先数，S、O、D 三个数相乘，只用来排先后顺序	第 9 章
AP	Action Priority	措施优先级，高、中、低三档，用来定先做哪件事	第 9 章
设计评审	Design Review	设计走到某个节点，把材料摊开给一群人挑毛病、决定能不能往下走	第 13 章
DR1 到 DR5	DR1-DR5	设计评审的几个节点代号，越往后越接近量产，越往后越难改	第 13 章
交付物清单	Deliverable List	这道闸门规定你必须交出哪几份东西，缺一份会就开不成	第 13 章
开口项	Open Issue	评审上被提出来、当场没有关掉的问题，必须有人、有日期、有关闭准则	第 13 章
关闭准则	Exit Criteria	这一轮算通过的条件，要在看材料之前定死	第 13 章
虚警	False Positive	AI 报了一条问题，你去核了发现它是错的，按虚警去改设计才可怕	第 13 章
8D 报告	8D Report	处理质量问题的八步标准表格，第三步临时措施、第五步根本起因、第七步预防再发	第 3 章
5W2H	5W2H	描述问题的七个维度，什么物料、什么时候、在哪里、谁发现、发生什么、多少件、多严重	第 14 章

术语	英文或缩写	大白话解释	首次出现
5Why	5Why	对一个问题连问五次为什么，追到可以采取措施的那一层	第 14 章
发生原因与流出原因		前者是为什么会做出不良品，后者是为什么没被拦住流到客户手上	第 14 章
临时措施与永久措施		临时措施是现在先把问题按住，永久措施是以后不会再发生	第 14 章
现象与原因		现象是你看到的，原因是造成它的，把现象当成原因是这类分析最常见的错误	第 14 章
人机料法环	5M1E	异常归因的六个方向，它是分类用的架子，不是结论	第 14 章
加权排序	Weighted Ranking	按几个维度打分再排序，不只按发生次数排	第 14 章
帕累托	Pareto	按发生频次降序排，找出占八成的那少数几类	第 14 章
经验教训	Lessons Learned	一件已经出过的事，连起因和当时的处理结论一起记下来	第 13 章
经验教训库	Lessons Learned DB	把过去出过的事连起因带措施记成一张能检索的表	第 13 章
条目	Entry	库里的一行，一个问题对应一行，不多不少	第 13 章
结构化字段	Structured Field	每一行的列，列定死了检索才检得准	第 13 章
源文件名与路径	Source Path	这一行是从哪份文件里抽出来的，没有这一列整库不可信	第 13 章
增量更新	Incremental Update	新出的问题按同一个模板往库里追加，不要另起一套	第 13 章
SPC	SPC	统计过程控制，用连续的测量数据盯住一道工序稳不稳	第 13 章
Cpk / CPK	Process Capability Index	过程能力指数，尺寸稳定到什么程度，1.33 是常见门槛值	第 13 章
PPAP	PPAP	生产件批准程序，量产前向客户提交的一整套认可资料	第 13 章

D.3.3 尺寸、公差与工程变更

术语	英文或缩写	大白话解释	首次出现
GD&T	Geometric Dimensioning and Tolerancing	一套用符号规定形状和位置允许偏差范围的语言	第 13 章
基准	Datum	测量时的起始参照，没有基准就提垂直度等于没说参照哪儿	第 13 章
公差	Tolerance	尺寸被允许变动的范围	第 13 章
公差带	Tolerance Zone	标注 20 正负 0.1，公差带就是 0.2 宽	第 13 章
粗糙度	Ra	表面微观凹凸的平均高度，直接影响摩擦、磨损和密封	第 13 章
投影角	Projection Angle	第一视角和第三视角两种约定，混用会让工人把图看反	第 13 章
尺寸链	Dimension Chain	一串首尾相接的尺寸，最后一个是被前面几个挤出来的结果	第 13 章
封闭环	Closing Link	尺寸链里那个你没法直接标、只能由别的尺寸凑出来的尺寸	第 13 章
组成环	Component Link	除封闭环之外的每一个环	第 13 章
增环与减环		这个环变大时封闭环跟着变大是增环，跟着变小是减环	第 13 章
极值法	Worst Case	假设所有组成环同时取到最不利的极限值来算，结果保守、成本高	第 13 章
统计法	RSS	假设各环独立且正态分布，用平方和开根号来算，前提不成立就不能用	第 13 章
敏感度	Sensitivity	某个组成环的公差变动对封闭环总变动贡献的比例	第 13 章
ECN	Engineering Change Notice	正式下发的设计变更单，写清改什么、为什么改、什么时候切换	第 13 章
ECR	Engineering Change Request	提出变更的那张单子，先走评估和批准，批了才转成 ECN	第 13 章
受影响件清单		这一次变更会牵动到的所有物件	第 13 章
工装与检具		做这个零件用的夹具刀具模具，和检这个零件用的量具检具	第 13 章

术语	英文或缩写	大白话解释	首次出现
借用件	Carry-over Part	同一个零件被好几个项目拿来用，改它一个几个项目一起动	第 13 章
特殊特性	Special Characteristic	图纸上打了标记的那几项，出问题会影响到安全、法规或功能	第 13 章

D.3.4 供应链与产线

术语	英文或缩写	大白话解释	首次出现
IATF 16949	IATF 16949	汽车行业的质量管理体系认证	第 14 章
ISO 17025	ISO 17025	实验室认可，没有这张证它出的报告你可能要自己复测一遍	第 14 章
认可范围	Accreditation Scope	证书上写的具体能做什么，C1 驾照不能开大货车	第 14 章
有效期至	Expiry Date	证书失效的那一天，过期整张证书作废	第 14 章
RFQ	Request for Quotation	你正式问供应商「这个东西多少钱」的那份文件	第 14 章
最小起订量	MOQ	供应商愿意接单的最小数量，低于这个数它不接或者要加价	第 14 章
模具费分摊	Tooling Amortization	开一套模具的一次性投入，摊到每一件产品上	第 14 章
账期	Payment Term	货到之后多少天付款	第 14 章
阶梯价	Tiered Price	买得越多单价越低的一张价格表	第 14 章
来料检验	IQC	外购件进厂时做的那道检查，通过才能入库上线	第 14 章
计数型 / 计量型	Attribute / Variable	前者只能数个数，后者有具体数值，两者算出来的东西完全不同	第 14 章
不良模式	Defect Mode	不合格的具体表现，跟「哪个零件」是两回事	第 14 章
让步接收	Concession	货不完全合格但生产急用，经过审批先收下用	第 14 章
停线时长	Downtime	从产线停下来到重新开动之间的分钟数	第 14 章
影响数量		这次异常造成了多少件产品受影响	第 14 章
节拍	Takt Time	为满足客户需求每隔多久必须出一件，它由需求决定不由设备决定	第 14 章
周期时间	Cycle Time	某个工位实际做一件要花多久	第 14 章
瓶颈工序	Bottleneck	整条线上周期时间最长的那个工位，整线产能由它决定	第 14 章
OEE	OEE	设备综合效率，等于可用率乘性能效率乘合格品率	第 14 章
换型时间	Changeover Time	从做上一个型号切到下一个型号，产线要停多久	第 14 章

D.3.5 造型、几何与图纸

术语	英文或缩写	大白话解释	首次出现
数模	3D Model	零件在软件里的立体样子，能算体积、能查干涉的几何实体	第 10 章
草图	Sketch	在三维软件里先画的那个平面轮廓，比如矩形、圆	第 10 章
拉伸	Extrude	把画好的平面轮廓沿垂直方向拉出厚度，得到一个立体	第 10 章
切除	Cut	在已有的立体上挖掉一块，打孔就是切除	第 10 章
圆角	Fillet	把尖锐的棱磨成圆弧，代号写作 R	第 10 章
扫掠	Sweep	让一个截面沿着一条指定的路径走一遍，走过的空间就是你要的形状	第 10 章
特征树	Feature Tree	软件左侧那棵树，按顺序列着你做过的每一个动作	第 10 章
参数化	Parametric	把尺寸写成变量，改一个数字整条链路自动跟着改	第 10 章
装配体	Assembly	若干零件按相对位置拼在一起的文件	第 10 章
配合	Mate	告诉软件两个零件之间的位置关系，比如两个面贴在一起	第 10 章
干涉	Interference	两个零件在空间里穿模了，物理上装不到一起	第 10 章
间隙	Clearance	两个零件之间特意留的距离	第 10 章
外径 / 内径	OD / ID	环形零件外轮廓的最大直径和中心空圈的直径	第 10 章
STEP	STEP	三维模型的通用交换格式，工业界的普通话	第 10 章
工程图	Drawing	把三维数模投影成二维图纸，标上尺寸、公差、剖视、明细表	第 10 章
三视图 / 剖视		三视图是从三个正方向看过去的样子，剖视是把零件假想切开看里面	第 10 章
标题栏	Title Block	图纸右下角那张表，写着图名、图号、比例、材料、设计者	第 10 章
BOM（明细表）	Bill of Materials	物料清单，这东西由哪些零件各几个组成，一表列清	第 10 章
序号	Balloon	图纸上给每个零件编的号，与明细表行号一一对应	第 10 章
GB 模板 / A3		国家标准的图纸模板和纸张幅面	第 10 章
单位制	Unit System	软件内部用什么长度单位存你的数，你说毫米它内部可能是米	第 10 章
几何指纹	Geometric ID	给一条边一个独一无二的身份记号，让 AI 能准确选中那条边	第 10 章
解析值 / 闭式解	Closed-form Solution	用数学公式直接算出来的理论值，它是软件回读值的裁判	第 10 章
帕普斯定理	Pappus Theorem	截面沿路径扫掠的体积等于截面面积乘形心走过的路径长度	第 10 章
质量属性	Mass Properties	软件对一个实体算出来的一堆数，包括体积、表面积、质量、重心	第 10 章

术语	英文或缩写	大白话解释	首次出现
轻量化	Lightweight Mode	打开大装配时只载入外形不载入细节的模式，验收时必须退出	第 10 章
NURBS	NURBS	一种用数学曲面精确描述形状的方式，高等级外观曲面靠的是它	第 10 章
渲染	Rendering	给模型打灯、上材质、出效果图的过程	第 10 章
Rhino	Rhino	以 NURBS 曲面著称的造型软件	第 10 章
Blender	Blender	开源免费的三维创作软件，建模、雕刻、渲染、动画一条龙	第 10 章
Blender MCP	Blender MCP	社区里的开源项目，通过 MCP 把 Blender 的能力暴露给 AI 调用	第 10 章

D.4 电子与软件术语

术语	英文或缩写	大白话解释	首次出现
PCB	Printed Circuit Board	印刷电路板，铜箔刻成线路，元器件焊在上面	第 11 章
EDA	EDA	电子设计自动化软件的总称，画原理图、画板子、查规则都在里面做	第 11 章
网表	Netlist	一份纯文本文件，写着谁跟谁连在一起	第 11 章
封装	Footprint	一个元器件在板子上占的那块焊盘图形，选错就焊不上	第 11 章
走线	Routing	把网表里的连接关系，变成板上实实在在的铜箔路径	第 11 章
DRC	Design Rule Check	设计规则检查，线宽够不够、间距够不够，软件按你的规则查一遍	第 11 章
位号	RefDes	板上每个器件的身份证，R1、C3、U7 这种	第 11 章
多 Part 器件	Multi-part Component	一个物理芯片里装着四个一样的运放，BOM 里是一行、网表里是四个位号	第 11 章
ECO	Engineering Change Order	工程变更指令，改一个电阻值、换一个供应商都算	第 11 章
ECO drift	ECO drift	变更漂移，原理图改了 PCB 没跟着改，三份文件的版本悄悄错开	第 11 章

术语	英文或缩写	大白话解释	首次出现
交叉引用	Cross-reference	拿两份清单按同一个键对一遍，列出只对一边有的和值不一样的	第 11 章
Simulink	Simulink	图形化建模工具，控制算法不写成代码而是画成框图	第 11 章
代码生成	Code Generation	把画好的框图自动变成 C 代码	第 11 章
AUTOSAR	AUTOSAR	汽车开放系统架构，汽车电子软件的行业标准接口规范	第 11 章
ARXML	ARXML	AUTOSAR 的配置文件格式，本质是 XML，是软件之间对账的凭据	第 11 章
MIL / SIL	MIL / SIL	模型在环与软件在环，两者结果应当一致，不一致说明代码生成这一跳有问题	第 11 章
ASIL	ASIL	汽车安全完整性等级，A 到 D，D 最高，等级越高验证要求越严	第 11 章
ISO 26262	ISO 26262	汽车功能安全的国际标准	第 9 章
MISRA C	MISRA C	汽车和嵌入式行业广泛采用的 C 语言编码规范，一条一条编号	第 11 章
Polyspace	Polyspace	做代码静态分析的工具，用形式化方法证明代码里没有某些类别的运行时错误	第 11 章
静态分析	Static Analysis	不运行程序，只读代码文本和语法树来找问题的技术	第 11 章
运行时错误	Runtime Error	程序跑起来才会暴露的错误，比如数组越界、除零、空指针	第 11 章
MC/DC 覆盖	MC/DC	修正条件判定覆盖，一种比「每行都跑到」更严格的测试覆盖率要求	第 11 章
追溯性	Traceability	一条需求、一段代码、一个测试用例三者之间能互相指认	第 11 章
误报	False Positive	工具报了问题，实际上不是问题，误报率决定你还会不会用第二次	第 11 章
工具认证	Tool Qualification	功能安全语境下，证明你用的开发工具本身可信的一套鉴定流程	第 11 章
CAE	CAE	计算机辅助工程，用计算机算结构强度、流场、电磁场、温度场的总称	第 11 章
前处理	Pre-processing	仿真的第一段，几何清理、画网格、加材料与边界条件、写输入文件	第 11 章

术语	英文或缩写	大白话解释	首次出现
求解器	Solver	真正算数那个程序，前处理给它输入，它算完给结果文件	第 11 章
后处理	Post-processing	仿真的最后一段，读结果、提数字、出云图、写报告	第 11 章
网格	Mesh	把连续的几何切成很多小块，求解器只在小块上算	第 11 章
边界条件（仿真）	Boundary Condition	告诉求解器这个件怎么被约束、受了什么力、跟外界怎么换热	第 11 章
输入文件	Input Deck	求解器读的那个文本文件，纯文本能被读、能被比、能进版本管理	第 11 章
ODB	ODB	结果数据库文件格式，二进制，仿真算完之后的结果都在里面	第 11 章
结果文件	Result File	求解器算完之后写出来的文件，二进制，得靠接口读	第 11 章
云图	Contour Plot	把结果按颜色画在模型上，红的地方大、蓝的地方小	第 11 章
应力集中	Stress Concentration	几何突变的地方应力会显著偏高，也可能是网格太糙造成的假象	第 11 章
收敛	Convergence	网格越密结果越接近一个稳定值，不再随网格变化才算收敛	第 11 章
安全系数	Safety Factor	材料能承受的极限除以实际算出来的值，多少算够是你的行业规范说了算	第 11 章

D.5 声学与试验术语

术语	英文或缩写	大白话解释	首次出现
NVH	Noise Vibration Harshness	噪声、振动与声振粗糙度，这三者分不开所以放一起研究	第 15 章
声压级	SPL	声压相对基准值的对数，单位分贝，人耳跨度太大所以用对数压扁	第 15 章
dB(A)	dB(A)	A 计权声压级，按人耳特性给各频段加权，噪声测量里用得最广的一个数	第 15 章
频谱	Spectrum	把一段声音拆成各个频率成分，看能量分布在哪些频率上	第 15 章

术语	英文或缩写	大白话解释	首次出现
1/3 倍频程	1/3 Octave	按固定比例把频率切成一段一段，做限值表很方便	第 15 章
阶次	Order	某频率相对于主轴转频的倍数，转频是 1 阶，换个转速测还是那个数	第 15 章
转频	Rotational Frequency	主轴每秒转多少圈，等于转速除以 60	第 15 章
啮合频率	Mesh Frequency	转频乘齿数，齿轮噪声的绝大多数能量都在它及其谐波上	第 15 章
阶次谱	Order Spectrum	横轴是阶次而不是频率的谱，转速变化的工况下比普通频谱清楚	第 15 章
转速谱阵	Campbell Diagram	横轴转速、纵轴频率、颜色表示幅值的三维图，斜线是阶次竖线是共振	第 15 章
包络	Envelope	把快速振荡信号的轮廓描出来的那条线，冲击类故障取包络才看得见	第 15 章
包络分析	Envelope Analysis	也叫共振解调，先挑出一段高频共振再取包络看周期性，专抓轴承点蚀	第 15 章
等角度重采样	Order Tracking	按转角而不是按时间采样，转速变化时用它把数据对上	第 15 章
传递函数	Transfer Function	输出与输入的比值随频率的变化，描述从激励点到响应点这条路的特性	第 15 章
模态	Mode	结构固有的振动形态，每个形态对应一个固有频率，共振就是激励撞上了它	第 15 章
激励 / 路径 / 响应	Source-Path-Receiver	谁在振、怎么传出来、人听到什么，改激励最有效，改响应最没用	第 15 章
齿距误差	Pitch Error	相邻两个齿之间的距离偏差，是传递误差的主要来源之一	第 15 章
修形	Modification	把齿形或齿向按计算好的量微微改一点，让啮合更平顺	第 15 章
断电法	Power-off Test	跑到稳态后切断电源看噪声是不是立刻消失，立刻消失是电磁源	第 15 章
主观评价	Subjective Evaluation	让人听或摸，给出评分和评语	第 15 章
客观测量	Objective Measurement	用仪器测出来的量，能复现、能追溯	第 15 章
心理声学参数	Psychoacoustic Metrics	响度、尖锐度、粗糙度、抖动度、音调度这一类按人耳特性算出来的量	第 15 章

术语	英文或缩写	大白话解释	首次出现
响度	Loudness	按人耳听觉特性算出来的响的程度，单位 sone	第 15 章
尖锐度	Sharpness	高频成分占比的度量，单位 acum，反映刺耳程度	第 15 章
等效连续 A 声级	LAeq	一段时间内 A 计权声压的能量平均值，把一段起伏的噪声压成一个数	第 15 章
ISO 532	ISO 532	计算响度的国际标准方法	第 15 章
GMW14155	GMW14155	在实验室里录制和评价零部件声音的通用流程	第 15 章
半消声室	Semi-anechoic Room	除地面外各方面都吸声的房间，用来做声学测量	第 15 章
本底噪声	Background Noise	被测对象不工作时环境里本来就有的噪声，比被测噪声低 10 dB 以上才可忽略	第 15 章
原始记录	Raw Record	采集设备直接吐出来的数据文件，没经过处理，真实但难用	第 15 章
通道	Channel	一路传感器就是一条通道	第 15 章
采样率	Sample Rate	每秒采多少个点，采样率不同的两份数据不能直接放在一起比	第 15 章
时域 / 频域	Time / Frequency Domain	时域是幅度随时间怎么变，频域是能量分布在哪些频率上	第 15 章
元数据	Metadata	描述数据的数据，样本号、工况、测点、采样率都算，缺了它就是废数字	第 15 章
样本	Sample	一台被测样机在一种工况下的一次完整记录	第 15 章
工况	Condition	被测对象工作时的条件，通常是电压、负载、转速、转向、温度	第 15 章
测点	Measurement Point	麦克风或加速度计放的位置，测点一变测出来就是另一个数	第 15 章
数据血缘	Data Lineage	一个数字从原始数据到最终结论走过的全部步骤	第 15 章
报告模板	Report Template	公司固定的报告格式，先把模板给它，别让它自己发明	第 15 章

D.6 通用办公与数据处理术语

术语	英文或缩写	大白话解释	首次出现
表头	Header	表格最上面那几行，说明每一列是什么	第 2 章
合并单元格	Merged Cell	一个格子横跨了好几个格子，排序、筛选、统计全会出错	第 2 章
脏数据	Dirty Data	格式不统一、有缺值、混进多余字符的数据	第 2 章
编码	Encoding	电脑存文字的规则，规则不一致就显示成乱码	第 2 章
透视	Pivot	按某一列分组求和、计数、取平均	第 2 章
CSV	CSV	逗号分隔的纯文本表格，任何软件都能打开，代价是没有格式和单位	第 2 章
文本型数字	Text-formatted Number	长得像数字其实被存成了文字，不能求和、不能比较大小	第 7 章
唯一值	Unique Value	某一列里不重复的内容	第 7 章
分列	Split Column	把一格里的复合内容拆成几列	第 7 章
幂等	Idempotent	同一个操作跑几次结果都一样，清洗脚本必须幂等	第 7 章
主键	Primary Key	一行记录的唯一身份证，没有主键没法查重也没法对账	第 12 章
未找到	Not Found	页面上确实没有这个字段时的标准写法，它跟空值、跟「无」都不同	第 12 章
增量	Delta	相对于上一次抓取新出现或发生变化的部分	第 12 章
时间戳	Timestamp	文稿上标的时间点，用处是让你能一键跳回录音那一秒去核对	第 7 章
转写	Transcription	把录音里说的话变成文字	第 7 章
说话人分离	Diarization	程序自动判断哪句话是谁说的，抢话时容易混	第 7 章
待办清单	Action Item	一件事加一个责任人加一个完成时间，三样齐全才算一条有效待办	第 7 章
未指定	Unassigned	会上没说清谁负责或什么时候交，老老实实写这个词，不能靠猜补上	第 7 章
行动项	Action Item	会议里定下来的、需要有人在某个日期前做完的具体动作	第 9 章
DRI	Directly Responsible Individual	一件事的直接负责人，可以有协助的人，但负责人只能有一个	第 9 章
截止日期	Due Date	具体到年月日的日期，「尽快」「下周」都不是日期	第 9 章
状态	Status	未开始、进行中、已完成、已阻塞	第 9 章

术语	英文或缩写	大白话解释	首次出现
待办	Todo	一件还没做的事，跟「想法」的区别是必须有动作、有终点	第 8 章
可执行	Actionable	一个不了解背景的人看完，知道第一步要干什么	第 8 章
原子任务	Atomic Task	不能再拆的最小动作	第 8 章
依赖关系	Dependency	A 做完了才能做 B，把本来能并行的事写成串行是排计划最容易错的地方	第 8 章
截止日与承诺日		截止日是文件规定的，承诺日是你答应别人的，按早的那个排但分开写	第 8 章
等待中	Waiting	需要别人先给你东西才能继续的任务，要单独归一类	第 8 章
模板	Template	一份通知里固定不变的部分，比如公司抬头、落款	第 7 章
变量	Variable	每次发布要替换的内容，比如时间、地点、对象、截止日期	第 7 章
收件对象	Audience	这份通知发给谁，同一件事对不同对象能说的信息量不一样	第 7 章
口径	Wording	同一件事在不同版本里的说法，给供应商的版本不能出现内部代号	第 7 章
样式	Style	Word 里的一组格式设定，用样式而不是手动调字号是格式不崩的唯一办法	第 7 章
模板文件	.dotx / .dotm	预置了样式、页眉页脚、封面的 Word 文件	第 7 章
域	Field	Word 里能自动更新的东西，比如自动页码、自动目录、交叉引用	第 7 章
大纲级别	Outline Level	每一段属于几级标题，目录和导航窗格都靠它	第 7 章
分节符	Section Break	把文档分成几节，每节可以有各自的页码格式和页眉	第 7 章
大纲	Outline	汇报的骨架，一层一层的标题	第 7 章
母版	Master	PPT 里控制所有页面统一外观的底稿，改一次所有页跟着变	第 7 章
版式	Layout	母版里预定义的页面布局，比如标题页、两栏对比、全图页	第 7 章
讲者备注	Speaker Notes	幻灯片下方的备注区，写你上台要说什么	第 7 章
一页一意	One Idea per Slide	一张幻灯片只讲一件事	第 7 章
分类标签	Label	给邮件打上类别，先分类再处理比逐封处理快得多	第 7 章
回复草稿	Reply Draft	AI 写好的回复，但由你确认后才发送	第 7 章

术语	英文或缩写	大白话解释	首次出现
模板回复	Canned Reply	对某类高频问题的标准答案，模板不是偷懒是保证口径一致	第 7 章
抄送与密送	CC / BCC	抄送人所有人可见，密送人其他人看不到	第 7 章
签名	Digital Signature	安装包上标明「谁做的这份文件」的电子印章	第 2 章
个人知识库	Personal Knowledge Base	一堆文件加一张目录加一个能提问的入口	第 8 章
Markdown	Markdown	用 # 表示标题、- 表示列表的纯文本写法，AI 读起来最省事	第 8 章
工作痕迹	Work Trace	你在电脑上留下的客观记录，文件修改时间、邮件、日历条目	第 8 章
周报与日志的区别		日志是流水账按时间记，周报是按结果归纳，要有结论	第 8 章
量化	Quantify	把「推进了某项工作」改成「完成 3 项，剩余 1 项，卡在 X」	第 8 章
已发送	Sent	只算你发出去的邮件，不算收到的	第 8 章
下周计划	Next Week Plan	必须有责任人和判据，写「继续跟进」等于没写	第 8 章
命名规则	Naming Convention	文件名的固定格式，有规则才能靠名字找文件	第 8 章
对照表	Mapping Table	一张 Excel，两列，原路径和新路径，改名前先看它	第 8 章
干跑	Dry Run	先只输出「我打算怎么改」，不真的改	第 8 章
文件指纹	File Hash	按文件大小加内容算出来的一串字符，用来判断两个文件是不是真一样	第 8 章
回滚	Rollback	改错了能退回原样，做法是先备份且对照表保留原路径列	第 8 章
订阅源	Feed	一个会持续更新内容的网址，它更新了会告诉你	第 8 章
RSS	RSS	网站按固定格式吐出更新内容的订阅方式，不用打开网页也不用看广告	第 8 章
去重	Deduplication	同一条消息被多个源转发，只留一条	第 8 章
摘要	Summary	两句话，第一句说发生了什么，第二句说跟我们有什么关系	第 8 章
定时执行	Scheduled Run	让这套流程每天固定时间自己跑一遍，结果放在一个固定位置	第 8 章
收件箱	Inbox	收到的邮件默认进的那个箱子	第 19 章
草稿	Draft	写好但没发出去的邮件，存在邮箱里等你点发送	第 19 章

术语	英文或缩写	大白话解释	首次出现
线程	Thread	一封邮件和它下面所有的来回回复算一个线程	第 19 章
附件	Attachment	随邮件一起发出的文件，发错版本是这类活最高频的事故	第 19 章
搜索条件	Search Query	用来筛邮件的规则，它决定 AI 能看到哪些邮件	第 19 章
在线文档	Online Doc	存在云端、多人能同时打开编辑的文档	第 19 章
文档空间 / 文件夹	Space / Folder	在线文档里组织文件的方式，授权时是整个空间还是单个文件夹的差别	第 19 章
版本历史	Version History	在线文档自带的改动记录，能退回，写坏了靠它救	第 19 章
代码仓库	Repository	存放代码的地方，记录着每一次被谁在什么时候改成什么样	第 19 章
提交	Commit	一次改动被存进仓库的动作	第 19 章
提交号	Commit Hash	一次改动存进仓库时生成的唯一编号	第 19 章
分支	Branch	从主线上分出去的一条独立开发线，改完了再合回来	第 19 章
合并请求	Pull Request	请求把某个分支的改动合进主线，别人可以在上面评论	第 19 章
提交记录	Commit Log	按时间排列的所有提交列表	第 19 章
串线	Chaining	让一次任务按顺序经过多个系统，上一步的产物是下一步的输入	第 19 章
人工确认点	Human Checkpoint	流水线上你必须在场的那儿站，串线不等于全自动	第 19 章
中间产物	Intermediate Artifact	每一步产出的文件，落在本地方便你中途检查	第 19 章
失败回滚	Failure Rollback	某一步失败之后，前面已经做完的那些动作怎么处理	第 19 章
日志	Log	整条线跑完的记录，每一步做了什么、成功还是失败	第 19 章

D.7 协作、验收与团队落地术语

术语	英文或缩写	大白话解释	首次出现
任务边界	Task Boundary	一份子任务负责到哪里、在哪里停	第 6 章
完成标准	Completion Criteria	判断一份工作能不能收货的具体条件，不能只写「做得认真」	第 6 章
完成条件	Definition of Done	任务达到哪些可检查的状态才算结束	第 16 章
交付物	Deliverable	任务最后必须留下的文件、表格、代码或报告，聊天里说完成不算	第 16 章
边界条件（任务）	Boundary	不许碰的文件、不许做的动作和必须请示的情况	第 16 章
允许清单	Allow List	明确允许读取、创建或修改的对象，没列入的默认不动	第 16 章
禁止清单	Deny List	明确不能修改、删除、覆盖、发送或发布的对象与动作	第 16 章
停手条件	Stop Condition	遇到某种情况立即停止，只报告事实并等待人决定	第 16 章
只读预检	Read-only Precheck	先查看目录、状态和依赖，不写文件	第 16 章
唯一数据源	Single Source of Truth	所有人共同认可的那一份现行输入	第 6 章
可追踪	Traceable	从结论能找到证据，从证据能回到原文位置	第 6 章
主指标	Primary Metric	你希望变好的数字，比如通过率、处理数量、耗时	第 16 章
反指标	Counter Metric	专门防止主指标被取巧做好的检查项，它不追求更高	第 16 章
古德哈特定律	Goodhart's Law	一个数字一旦被当成唯一目标，做数字的人就会把数字做漂亮	第 16 章
替代目标	Proxy Goal	AI 为了方便，把原任务偷偷换成更容易完成的任务	第 16 章
证据指标	Evidence Metric	证明动作真的发生过的数量或记录	第 16 章
证据	Evidence	别人可以重新检查的文件、日志、命令结果或数据来源	第 16 章
检查点	Checkpoint	任务执行到某个位置时强制暂停，人和 AI 一起核对再决定继续	第 16 章

术语	英文或缩写	大白话解释	首次出现
样件	Sample Piece	先做的一份代表性产物，用来暴露理解错误，不追求数量	第 16 章
批次	Batch	把大量对象分成若干组，每组完成后单独验收	第 16 章
回退点	Rollback Point	最后一个确认无误、可以安全继续的位置	第 16 章
放行	Release	人明确说可以进入下一阶段，没有放行就不能继续扩量	第 16 章
基线	Baseline	开工前记录的数量、状态和规则，用来与后续结果比较	第 16 章
进度基线	Progress Baseline	开工时确认的总对象数、阶段数和检查项	第 16 章
状态快照	Status Snapshot	某一时刻的事实清单，含完成项、剩余项、失败项和已改文件	第 16 章
事件汇报	Event Report	完成一个阶段、出现异常或准备越过检查点时才汇报	第 16 章
异常上报	Escalation	把偏离预期的事实单独列出，说明影响范围，给人选择	第 16 章
阻塞项	Blocker	没有人的决定或缺失输入就无法继续的事项	第 9 章
漂移	Drift	执行结果逐渐离开原任务目标、范围或判定规则，每次只偏一点	第 16 章
目标漂移	Goal Drift	原本要解决 A，执行中改成了更容易的 B	第 16 章
范围漂移	Scope Drift	修改对象从允许清单扩展到其他文件、系统或人员	第 16 章
证据漂移	Evidence Drift	原本要求可复验的日志，后来只剩一句总结或一个百分比	第 16 章
最小纠正	Minimum Fix	只修复已经确认的偏差，不趁机增加新功能或重做全部内容	第 16 章
范围蔓延	Scope Creep	任务的边界在执行过程中自己长大了	第 17 章
三栏清单	Three-column List	本次必做、本次明确不做、发现但暂不处理	第 17 章
改动对照表	Change Table	列出改了哪个文件、哪一处、旧值、新值	第 17 章

术语	英文或缩写	大白话解释	首次出现
候选池	Candidate Pool	执行中发现的新问题，先记下来不处理，攒够了一起排期	第 17 章
版本号	Version Number	文件改一次升一版	第 17 章
开工权	Go Authority	谁有权说「现在可以动手了」，交给 AI 的时候这个权必须留在你手里	第 17 章
方案门	Option Gate	正式动手之前的一个停点，只比较路线不执行	第 17 章
取舍	Trade-off	选 A 就必须放弃 B 的什么，没写清楚取舍的方案等于没有方案	第 17 章
默认优先级	Default Priority	它心里默认的那套排序，通常是通用性排在你的特殊约束前面	第 17 章
破坏性操作	Destructive Operation	改文件、删文件、覆盖数据、改配置这类不可逆的动作	第 17 章
可行性判断	Feasibility Judgment	这件事能不能做成，「能做」和「该做」差着一轮评审	第 17 章
外引事实	External Fact	它从自己记忆里拿出来的一切，你给的材料里并没有	第 17 章
幽灵引用	Phantom Citation	看着像引用实际不存在，比如一个拼错的标准号	第 17 章
溯源三问	Three Trace Questions	谁测的、哪台设备、原始文件在哪，答不上来这个数就不能进报告	第 17 章
状态文件	State File	把约定写进文件，不再只依赖对话记忆	第 17 章
交接记录	Handover Note	让一个完全没看过旧对话的人接手所需的最小信息集	第 17 章
三文件法	Three-file Method	计划、发现、执行记录，三个文件管住一个长任务的状态	第 17 章
证据表	Evidence Table	完成声明必须附上的一张表，四行：查了什么、怎么查的、结果、证据位置	第 17 章
样本数	Sample Count	送进去几个样件、报告里几个样件，这两个数字不一致一定有问题	第 17 章
剔除说明	Exclusion Note	某个样本的数据被排除时必须写清理由，没有理由的剔除等同于造假	第 17 章
行动优先排版	Action-first Layout	回复的第一行是结论或下一步，背景和解释往后放	第 17 章

术语	英文或缩写	大白话解释	首次出现
一屏一动作	One Action per Screen	每一屏的内容只为了让读者做一件事	第 17 章
结论前置	Conclusion First	把最重要的那句话放在最前面，后面才说为什么	第 17 章
信息密度	Information Density	一屏内容里有用信息的占比	第 17 章
风险不可省略		风险、失败、未验证的部分不许为了精简而被删掉	第 17 章
验收	Acceptance	在它交回来的产物上做一组固定检查动作，确认能用再签字	第 18 章
验收三档	Three Levels	按出错代价分的三档检查力度，快速核、抽样核、全量核	第 18 章
快速核	Quick Check	只核最外层信号，两分钟做完，看格式、条数、空行、占位符	第 18 章
抽样核	Sampling Check	随机挑一部分逐条跟原始素材对，挑多少按出错代价定	第 18 章
全量核	Full Check	每一条都对一遍，要付钱、要担责的东西只能用这一档	第 18 章
验收动作	Acceptance Action	人怎样确认结果合格，比如打开文件、运行命令、抽查三行	第 16 章
验收门槛	Acceptance Threshold	超过就算不合格的那个数，由你写，事后不许改	第 18 章
出错代价	Cost of Error	这份产物错了最坏会怎样，分可撤销、可重做、要付钱、要担责四层	第 18 章
溯源	Traceability	从结论数字往回追，一直追到它最初的来源	第 18 章
源数据	Source Data	最原始、没被加工过的记录	第 18 章
处理过程	Processing	从源数据到报告数字中间的每一步，筛选、剔除、换算、取位	第 18 章
结论	Conclusion	最终写进报告、拿去支撑一个判断的那个数	第 18 章
数字来源对照表	Number Trace Table	把每个数字的源数据、处理过程、结论写在同一行的一张表	第 18 章
限定词	Qualifier	数字前面的定语，几台、什么工况、哪个批次、多长时间	第 18 章

术语	英文或缩写	大白话解释	首次出现
L1 / L2 / L3		任务的三档责任级别，L1 全自动、L2 它干九成拍板、L3 它只给信息	第 18 章
自动修复	Auto Fix	它自己发现出错并自己改好	第 18 章
拍板	Decision	在几条路里选一条并为这个选择负责	第 18 章
只给信息和选项		不替你选不替你做，只把选项和代价摆出来	第 18 章
止损	Stop Loss	出事之后第一件事是让损失不再扩大，不是查原因	第 18 章
影响面	Impact Scope	这份错误产物已经流到了哪里，谁手上有一份	第 18 章
根因	Root Cause	改掉它之后这个错就不会再犯的那一条原因	第 18 章
复盘	Post-mortem	把一件事从头到尾摆开，看清楚哪一步出了岔子，写成文件	第 18 章
改规矩	Rule Change	复盘的最后一步是改一条具体规则，让这个错在流程上犯不出来	第 18 章
复发测试	Regression Test	改完规矩拿原来那个错重跑一遍，看新规矩能不能挡住它	第 18 章
团队采用	Team Adoption	一项方法离开最早使用的人，其他成员也能在真实工作中重复使用	第 21 章
切入任务	Entry Task	团队最先拿来试用的一小类工作，要求高频、重复、出错代价低	第 21 章
任务地图	Task Map	按岗位列出可尝试任务、输入、产物和风险的一张表	第 21 章
技能库	Skill Library	保存完整执行步骤、停手条件和验收方法的部门目录	第 21 章
提示词库	Prompt Library	保存经过复现、可以直接填写使用的任务单	第 21 章
案例库	Case Library	保存成功和失败的真实记录，用来证明方法何时有效、哪里会出错	第 21 章
维护人	Maintainer	负责检查某项方法是否仍能使用、版本是否清楚的人	第 21 章
覆盖面	Coverage	真实使用已经到达多少岗位和成员，不把一次打开工具算进去	第 21 章

术语	英文或缩写	大白话解释	首次出现
归档	Archiving	把任务输入、提示词、产物、验收和决定按统一结构保存	第 21 章
时薪折算	Hourly Rate	把小时换成钱的换算动作，够用的算法是税前月薪除以 22 天除以 8 小时	第 22 章
折现系数	Realization Rate	省下的时间里真正能变成钱的比例	第 22 章
订阅分摊	Subscription Allocation	把整份订阅费按场景平摊，算出每个场景背了多少	第 22 章
返工成本	Rework Cost	东西已经做完了，因为做错而必须返过头重做所付出的时间	第 22 章
尾部风险	Tail Risk	发生概率很低、但一旦发生损失极大的那类事	第 22 章
依赖风险	Dependency Risk	工具换掉或者停掉的时候，你会痛到什么程度	第 22 章
回本工时	Break-even Hours	这笔订阅要回本，你每月至少得省出多少小时	第 22 章
隐性成本	Hidden Cost	账单上看不到但真实发生了的支出，包括学习时间、维护精力、依赖风险	第 22 章

（下一份：附录 E 学习资源与延伸阅读。）

附录 E

学习资源与延伸阅读

书看完了，活干完了，接下来去哪儿找东西。

这一页只做三件事：告诉你随书的东西在哪儿拿，告诉你我在哪些平台上继续更新，告诉你想往深了学该搜什么关键词。

我不打算在这里列一堆书名和链接。原因很实在：这类清单半年就过期一半，而"搜什么关键词"能管用三年。

E.1 随书资源

GitHub 开源仓库

地址：github.com/Shun1989/Ericks-WorkBuddy-Practice

开源协议：MIT。你可以随便用、随便改、拿去给公司用也行，唯一的要求是保留署名。

仓库里有四样东西：

内容	说明
72 个场景的抄作业模板	每个场景一套：提示词原文、检查清单、验收标准。复制就能用
30 多个 WorkBuddy 技能与工作流	可以直接安装，覆盖数据整理、报告生成、图纸检查、试验记录这些重复劳动最集中的环节
书中自绘示意图的 SVG 源文件	拿去改就能用，配色和字体都是按本书的规范排的
翻车案例库	每个场景的已知错误模式与排错记录

为什么要把这些放到公开仓库里：这本书反复讲"可复现"。一个场景如果你拿去跑不通，那它就只是故事，不是方法。仓库里的东西就是让你跑通的。

拿完之后建议做一件事：挑一个你下周真要干的活，打开对应场景的模板，把提示词里的方括号换成你的真实内容，跑一遍。跑通的那个，才是你的。

微信公众号「Eric的数字花园」

回复关键词：「实战」

回复之后可以领到：随书 72 个场景的提示词合集，以及案例用的脱敏练习素材。

公众号上持续更新的三类内容：新工具怎么接进工作流、旧方法哪里开始失灵、读者寄来的翻车现场。AI 加汽车零部件行业的交叉内容，主要发在这里。

汽研 Agent 中文社区

网址：qiyanagent.org.cn

面向汽车与制造行业的 AI 应用者社区。每天整理行业 AI 动态，积累工程场景的真实用法，也收集翻车案例。

如果你在制造业、汽车或者硬件行业，这个社区比公众号更适合你。公众号面向泛职场，社区只聊工程。

E.2 作者的公开分享

同名账号在下面这些平台上都有更新，搜"Eric 的数字花园"或者作者名都能找到：

平台	上面主要是什么
微信公众号	长文，新工具接进工作流的完整记录
B 站（哔哩哔哩）	脱敏后的课程视频，从入门到精通的完整系列
知乎	回答和方法论讨论
小红书	短内容，一个场景一张图
抖音	短视频，现场演示

想先看人再决定买不买这本书的，去 B 站翻课程视频。

有一点提前说清楚：平台上讲的东西和书里的东西会有重叠，但组织方式不一样。书是按场景排的，一本能从头翻到尾；平台是按时间排的，看到哪算哪。想系统抄作业看书，想追新看平台。

E.3 延伸阅读方向

下面这张表不给书名，给方向。每个方向告诉你该搜什么关键词、该看哪一类资料。原因是这类领域半年一变，记住关键词比记住书名划算。

E.3.1 AI 基础

方向	搜什么关键词	看哪类资料
模型到底在干什么	大语言模型原理、预测下一个词、注意力机制	找带图解的科普文章或公开课，别一上来就看论文

方向	搜什么关键词	看哪类资料
它为什么会编	幻觉、Hallucination、知识边界	找讲"模型不知道自己不知道"的分析文章
能力边界在哪	模型评测、Benchmark、能力对比	找第三方评测报告，看它在哪类任务上掉得快
上下文是怎么工作的	上下文窗口、Context Window、长文本	找讲"为什么聊久了它会忘"的技术博客
成本是怎么算的	Token 计费、输入输出计费	直接看各家官方文档的计费说明页

E.3.2 Agent 与工具调用

方向	搜什么关键词	看哪类资料
Agent 是怎么跑起来的	Agent 循环、Tool Calling、Function Calling	找讲"思考、调用、观察、再思考"这个循环的入门文章
怎么让 AI 接上软件	MCP、Model Context Protocol、工具协议	看协议官方站点和开源示例仓库
多代理怎么分工	Multi-Agent、Orchestrator、Sub Agent	找有完整任务拆分实例的工程博客
长任务怎么不跑偏	任务状态、检查点、任务编排	找讲失败案例的文章，比讲成功案例的有用
怎么判断 Agent 该不该上	Agent 适用场景、自动化边界	找讲"哪些活不该交给 Agent"的复盘文章

E.3.3 提示词写法

方向	搜什么关键词	看哪类资料
基本写法	提示词工程、Prompt Engineering、结构化提示词	找带前后对比的例子，只看讲原则的没用
少样本示例	Few-shot、示例驱动、示例选择	找讲"给几个例子、给什么例子"的实践文章
让它自己检查	自我批判、Chain of Thought、反思提示	找讲"让它咬自己一口"的具体提示词原文
输出格式控制	结构化输出、JSON 输出、格式约束	看各家官方文档里关于输出格式控制的章节
怎么测提示词好不好	提示词评测、Eval、评测集	找讲"怎么给提示词打分"的工程实践

E.3.4 工程软件自动化

方向	搜什么关键词	看哪类资料
让 AI 驱动三维软件	CAD 自动化、API 二次开发、参数化建模	看软件官方的开发文档和脚本接口说明

方向	搜什么关键词	看哪类资料
仿真流程自动化	CAE 自动化、前处理脚本、求解器批处理	找讲"把仿真流程串成脚本"的工程博客
电子设计自动化	EDA 脚本、网表比对、规则检查自动化	看开源 EDA 工具的字本文档
数据格式怎么交换	STEP、IGES、数据交换标准	找讲格式差异和丢信息的对比文章
开源项目怎么找	在代码托管平台上搜"软件名 + MCP"、"软件名 + Agent"	直接看仓库的 README 和许可证

E.3.5 汽车行业 AI 应用

方向	搜什么关键词	看哪类资料
行业在往哪走	汽车 AI 应用、智能制造 AI、研发数字化	看主机厂和一级供应商公开发布的技术路线材料
功能安全怎么办	ISO 26262、工具认证、AI 功能安全	看标准原文和标准解读，别看二手转述
质量体系怎么跟上	IATF 16949、AI 质量体系、过程审核	看认证机构的公开说明材料
具体场景怎么用	需求解析、试验数据分析、报告自动生成	找有真实数据脱敏案例的分享
同行在聊什么	汽车 AI 社区、制造业 AI 实践	看行业社区积累下来的案例，翻车案例比成功案例值钱

E.3.6 安全与合规

方向	搜什么关键词	看哪类资料
数据能不能外传	数据脱敏、数据分级、出境合规	看你所在公司的信息安全规定，这个没有通用答案
授权给出去的是什么	OAuth、权限范围、令牌有效期	看授权协议官方文档和各大服务的授权说明页
最小权限怎么落地	最小权限原则、只读账号、审计日志	找讲"给 AI 开账号该怎么开"的运维文章
出事谁负责	责任边界、人工确认点、审计留痕	找讲 AI 事故复盘和责任认定的文章
内部规矩怎么定	AI 使用规范、企业 AI 治理	找已经发布过内部 AI 使用规范的公司公开材料

E.4 怎么持续跟得上

这个领域变化快到让人焦虑。我用下来有效的就三条，都不花什么力气。

第一条，关注一个会持续更新的源。

公众号或者社区，选一个就够。挑的标准不是内容多，是它会不会讲"这个以前管用的方法现在不管用了"。只讲新工具不说旧方法失灵的源，看着热闹，实际帮不上忙。

第二条，进一个只聊工程场景的社区。

泛职场 AI 社区的问题是，里面大量内容跟你的活没关系。找一个你所在行业的社区，大家说的都是同一类活，翻车案例能直接对上号。汽研 Agent 中文社区就是为这个建的。

第三条，自己建一个案例库。

这条最笨，也最有用。每次你用 AI 干成一件活，或者干砸一件活，花五分钟记三样东西：提示词原文、产物、验收结论。三个月之后你会有一份只属于你的资料，它的价值超过你读过的任何一份别人的清单。

我在第 21 章讲过团队怎么建库，这里说的是个人版：一个文件夹，一个固定格式的文件名，一条记录五分钟。

（下一份：附录 F 随书资源清单。）

附录 F

随书资源清单

一本书能带走的东西，都列在这一页。

买一本书，你拿到的不只是这沓纸。下面这张清单是随书全部资产的目录：哪些东西能直接下载，哪些东西要自己去复制，哪些东西得你自己动手做出来。

用法建议：先扫一眼 F.1 的总览，找到你现在最缺的那一样，直接跳过去。

F.1 随书资产总览

#	资产	数量	形态	在哪拿
1	72 个场景的抄作业模板	72 套	提示词原文、检查清单、验收标准，各一份	GitHub 仓库
2	WorkBuddy 技能与 workflow	30 多个	可直接安装的技能包	GitHub 仓库
3	书中自绘示意图的 SVG 源文件	全书自绘图	SVG 矢量源文件	GitHub 仓库
4	翻车案例库	按场景归档	错误模式、触发条件、补救动作	GitHub 仓库
5	各章交付物模板	22 份	正文里的表格与代码块，可整段复制	本书正文（见 F.6）
6	72 个场景提示词合集	72 段	提示词文本	公众号回复「实战」
7	脱敏练习素材	一批	可拿来练手的样例数据	公众号回复「实战」
8	行业案例与每日动态	持续更新	社区帖子	汽研 Agent 中文社区

F.2 72 个场景的抄作业模板

72 个场景集中在第 7 章到第 18 章，每个场景一套完整模板，三件东西缺一不可：

组件	内容	拿来干什么
可复制提示词	逐字可复制的原文，方括号是你需要填的变量	直接贴给 AI，不用自己重写
检查清单	开工前要备什么文件、开什么权限	防止跑到一半发现材料没备齐
验收标准	哪几个数字对得上才算过、对不上怎么办	决定这份产物能不能签字

场景分布的十二个方向：

章号	方向
第 7 章	通用办公自动化
第 8 章	个人效率与知识管理
第 9 章	需求与项目定义
第 10 章	造型与几何
第 11 章	电子软件与仿真
第 12 章	让 AI 碰到你的电脑
第 13 章	设计审核与风险
第 14 章	供应链与产线
第 15 章	NVH 与试验
第 16 章	让 Agent 不跑偏
第 17 章	AI 的七宗罪与反制
第 18 章	验收溯源与责任

拿回去怎么用：不要一次下载全部 72 套。挑你下周真要干的那一个，跑通了再拿第二个。抄一个能用上的，比存 72 个吃灰的强。

F.3 30 多个 WorkBuddy 技能与工作流

这批技能覆盖的是重复劳动最集中的环节：

类别	覆盖的活
数据整理	试验数据清洗、多表对账、格式统一
报告生成	按公司模板出报告、图表插入、数字对照表
图纸检查	标题栏、明细表、公差标注的逐项核对
试验记录	原始记录归档、元数据补全、样本检索
文档处理	会议纪要、通知多版本、周报生成

每个技能都是一份说明文件加一组参考文件，装进技能的目录就能用。第 4 章讲了怎么写自己的第一个技能，第 15 章讲了怎么把散在脑子里的经验重构成能被加载的知识包。

挑一个装上，用过一轮再改。技能是养出来的，不是一次写出来的。

F.4 书中自绘示意图的 SVG 源文件

书里凡是注明「作者根据公开资料绘制」的图，源文件都在仓库里，SVG 格式。

项目	说明
画布尺寸	1000 乘 650
命名规则	fig-章号-序号-英文短名.svg
配色	背景浅米、主字深灰、强调砖橙、辅助墨绿
字体	Microsoft YaHei

拿来干什么：

1. 做内训材料，直接改字就能用。
2. 改配色适配你们公司的模板。
3. 拿结构图当讨论底稿，在会上直接往上加批注。

矢量图的好处是放大不糊，改字不用重新画。

F.5 翻车案例库

每个场景配一份翻车记录，三栏：

栏	内容
错误模式	它具体错成了什么样，附真实输出片段
触发条件	什么情况下这个错会冒出来
补救动作	发现之后第一步做什么

三类最容易翻的车，在库里占的比例最大：

1. 数字编造：凭空给出一个看着合理的数，或者引用一个不存在的标准号。
2. 范围失控：改了你不让它改的文件，或者顺手加了一段你没要的内容。
3. 报喜不报忧：失败的项被写成"已处理"，或者干脆不提。

翻车案例库的价值不在让你避开所有坑，在于让你认出坑长什么样。认出来就能早停手，早停手代价就小。

F.6 各章交付物模板汇总

每一章末尾的「本章交付物」都是一件能直接复制带走的东西。下面这张表是全书 22 份交付物的目录。

章号	交付物名	用途	所在篇
第 1 章	三问自检卡	每次想把活交给 AI 之前，先过这三问	第一篇 认识 WorkBuddy
第 2 章	装机自检清单（九条）	装完当天逐条打勾，第七条没过不要交真活	第二篇 环境搭建
第 3 章	模式决策卡	判断这一活用 Ask、Plan 还是 Agent	第二篇 环境搭建
第 4 章	我的第一个技能文件	一份能直接放进技能目录的 SKILL.md	第三篇 核心机制
第 5 章	一页项目背景卡模板	十项填完，整段喂进记忆	第三篇 核心机制
第 6 章	多代理分工卡	拆活、派活、合并的填空模板	第三篇 核心机制
第 7 章	每周杂活交接清单	六个场景各自的交接条件与验哪三样	第四篇 场景实战
第 8 章	个人资料家底表	第一次知道自己有多少文件、散在几个地方	第四篇 场景实战
第 9 章	需求翻译提示词组（六段）	把客户文件翻成能分派的任务	第四篇 场景实战
第 10 章	AI 建模九步作业单 + 六行体检检查卡	建模流程与体检报告	第四篇 场景实战
第 11 章	四列代码审查模板 + 三份文件交叉引用底账卡	代码规范检查与原理图、PCB、BOM 对账	第四篇 场景实战
第 12 章	系统分级清单表 + 开工红线提示词	把系统分成可碰、需审批、绝对不碰三类	第四篇 场景实战
第 13 章	评审前 30 分钟自检单	评审前一天晚上照着走一遍	第四篇 场景实战
第 14 章	比价口径表 + 资质核查矩阵模板	比价时摊在键盘旁，核查资质时当底表	第四篇 场景实战
第 15 章	数字来源对照表模板 + 声学库字段清单	报告先填对照表再写正文	第四篇 场景实战
第 16 章	交活三件套卡片	完成条件、边界条件、反指标，长活开工前填	第四篇 场景实战
第 17 章	八条反制提示词卡 + AI 产出验收清单	按罪名命名，派活前挑一张贴在提示词最前面	第四篇 场景实战
第 18 章	签字前检查单（十一项）	打印出来贴在显示器边框上	第四篇 场景实战
第 19 章	连接器授权清单 + 三张可打印卡片	列清已经给出去的每一条授权和撤销入口	第五篇 连接扩展
第 20 章	MCP 配置文件 + 七步排障表	存一份能跑通的配置，排障时照着查	第五篇 连接扩展

章号	交付物名	用途	所在篇
第 21 章	部门 AI 使用约定模板	定 L1、L2、L3 任务、岗位、目录与审批方式	第六篇 团队落地
第 22 章	单场景成本收益表（九格空模板）	只填你最烦的三个活，跑满一个月看净收益	第六篇 团队落地

用这 22 份的顺序建议：先做第 1 章的三问自检卡和第 18 章的签字前检查单，这两张不用任何准备，当天就能用上。剩下的按你手上正在干什么活往下挑。

F.7 获取方式

渠道	地址或关键词	能拿到什么
GitHub 仓库	github.com/Shun1989/Ericks-WorkBuddy-Practice	72 个场景模板、30 多个技能与 workflow、SVG 源文件、翻车案例库。MIT 协议
微信公众号	「Eric的数字花园」，回复「实战」	72 个场景提示词合集、脱敏练习素材
汽研 Agent 中文社区	qiyanagent.org.cn	行业案例、每日动态、工程场景的真实用法

三条拿东西的路径，选哪条都行：

- 1. 要能直接跑的，去 GitHub 仓库下载。
- 2. 要提示词文本的，公众号回复「实战」。
- 3. 要同行案例的，去社区翻帖子。

下载下来第一件事：挑一个模板，把方括号换成你的真实内容，今天跑一遍。跑通的那一个，才是你的。
(附录完。书看到这里，该去干活了。)

附录 H

致谢与灵感来源

H.1 致谢

这本书不是一个人关在屋里写出来的。它脱胎于 2025 年到 2026 年之间我在一线做的一百多个案子，每一个案子背后都有几个人在帮忙。

感谢参与素材整理的朋友（姓名待补：_____）。几千份会议纪要、试验记录、8D 报告、评审材料，从共享盘的十一层目录里捞出来、按项目归拢、去掉重名和废版本，这件事没有任何技巧可言，纯靠耐心。

感谢帮忙核实案例的同行（姓名待补：_____）。书中每一处标了 ★★★ 的场景，都至少有一位当事人帮我回看过一遍：这个数字对不对、当时的实际情况是不是这样写、我有没有把话说满。有几处我写得比实际更漂亮，是被他们当场拦下来的。

感谢试读者（姓名待补：_____）。第一版稿子里的毛病，绝大部分是你们指出来的：第三章太啰嗦、第 11 章有一处工具版本写错了、第 17 章原来的七条罪名有两条其实是同一条。你们读的是最差的那一版，谢谢你们没有直接放弃。

感谢开源社区的贡献者。书中提到的 SolidPilot、eda-agent、CAE-Agent-Hub、BrowserSkill、Blender MCP 这些项目，以及它们背后每一位提交过代码、写过 issue、答过问题的开发者。没有这些人把路先蹚出来，第 10 章到第 12 章这三章根本不成立。我在书里只是把它们的用法讲清楚，功劳归各自作者。

还有一些人我不能写名字，他们帮我校核过供应商信息、提醒过我哪条数据不能公开、在我把话说得太满的时候泼过冷水。你们知道我说的是你们。

最后要谢谢我的家人（姓名待补：_____）。这本书大部分内容是周末和深夜写的，那段时间我人坐在餐桌边，心在台架上。

H.2 灵感来源

本书第 17 章的失效模式分类受 X 用户长文启发，全部内容已重写，未保留原文表达。

需要说清楚的是这一次借鉴的边界。X 上那篇长文是写给普通使用者看的，讲的是日常对话里模型会犯的错；我读完之后受启发的是“把错误分门别类、每类配一个反制动作”这个骨架，具体到书里的八条罪名，全部来自我在汽车零部件研发现场遇到的事，每一条的案例、提示词、检查表都是重新写的，没有沿用原文的任何一句话、任何一个例子、任何一处表述。

那篇长文里的例子是写文档和问问题，我书里的例子是编错一个标准条款号导致三十页试验白做、是 12 台样件坏了 2 台而报告只写 10 台、是改一个轴径顺手把整份技术规范重写一遍。场景换了，判断标准也跟着换了：工程现场的错误有成本，有的错误有安全成本，所以书里每一条罪名后面跟的不是“怎么提问更好”，而是“谁来签字、留什么证据、在哪一步设停点”。

除第 17 章这一处之外，全书其余内容均为作者在一线项目中的原创记录与整理，不存在对单一来源的借鉴。

H.3 素材来源

书中 72 个场景的素材，来自作者 2025 年至 2026 年在汽车零部件研发一线做过的实际工作，包括项目会议、设计评审、试验台架、供应商定点、产线异常处理和质量问题归零。

为了保证可读和可核，书中的所有素材都做过处理：

- 客户名称一律不出现，涉及具体车型和平台的，只写“某欧系平台”“某自主品牌项目”这类不带指向的描述。
- 供应商名称一律不出现，按 A 厂、B 厂、C 厂编号，涉及价格的按等比例缩放处理，保留可比性，不保留真实数值。
- 项目信息做了合并与改写。书中一个“项目”常常是两三个真实项目的拼接，目的是把一次完整的决策过程讲清楚，而不是复刻某一次会议。
- 人名一律不出现，同事和同行只以岗位称呼，如“质量工程师”“试验工程师”“供应商质量工程师”。
- 时间线做过压缩。真实项目里跨了十个月的争论，书里可能写成一次评审会；这不改变结论，只改变阅读节奏。
- 所有截图、表格、代码片段中的编号、版本号、文件名，凡可能指向真实对象的，都做了替换。

需要说明的是，书中所有可复现的数字，比如二十三种零件、四十四处实例、59 条抓取记录、3182 个文件，都是真实记录，没有为了好看而调整。标 ★★ 的场景留档不完整，书里已经逐处说明缺在哪一部分。

H.4 开源与授权

随书资产以 MIT 协议开源，仓库托管在 GitHub，地址见前言。包括：

- 全书 72 个场景的提示词原文，按章分目录，可直接复制使用。
- 场景中出现的可复用模板，包括体检查验表、尺寸链校核表、变更影响分析单、数字来源对照表、四行证据表、系统分级清单表等。
- 第 15 章的两个经验仓库的结构说明与字段表。
- 第 16 章的完成条件卡片、边界条件清单、反指标模板。

MIT 协议的含义是：你可以自由使用、复制、修改、合并、出版发行、再授权和販售这些资产，包括用于商业目的，条件是保留原作者的版权声明和许可声明。除此之外没有别的限制。

书中提到并介绍方法的开源项目，包括 SolidPilot、eda-agent、CAE-Agent-Hub、BrowserSkill、Blender MCP、Rook 等，其版权归各自作者所有，遵循各自项目的许可证。本书仅在方法层面介绍它们的用法、边界和适用场景，不主张任何权利，也不对它们的功能做出超出项目自身声明的承诺。这些项目的进展很快，你读到书的时候版本可能已经变了，动手之前请以项目当前的 README 为准。

书稿正文的版权归作者与出版社所有，不在上述开源范围内。

H.5 软件与商标

书中出现的软件名称、产品名称和公司名称，归各自的商标持有人所有。提及这些名称的目的只有一个：说明在某个具体软件里怎么操作。书里写了 SOLIDWORKS、CATIA、Altium、MATLAB、Simulink、ANSYS、Rhino、Blender、UU 远程等工具的操作路径，也写了它们的能力边界。

关于这些名称，有几点需要说明：

- 提及不构成任何形式的推荐、背书或合作关系声明。作者与这些厂商之间不存在商业关联，书里也没有收过任何一方的费用。
- 书中记录的操作路径、菜单名称、参数位置、版本号，以作者写作时的安装环境为准。软件更新很快，你手上的版本可能已经不同，请以你实际安装的界面为准。
- 书中对软件能力的描述，包括“能走到哪一步”“走不到的地方在哪”，是作者在具体任务中的实测观察，不是厂商的官方声明。涉及授权范围、许可条款、功能可用性的最终解释，以厂商的官方文档为准。
- 书中对 Computer Use、浏览器自动化、远程操控等能力的描述，涉及你的账号权限与公司信息安全制度。动手之前请先确认你的操作在你的公司是被允许的。
- 商标名称在书中使用时不做缩写、不做改写，以避免指向错误。

找到作者，拿到随书资产

这本书没写完的部分，都在下面这几个地方持续更新。



公众号「Eric的数字花园」

回复「实战」，领 72 个场景提示词
合集与脱敏练习素材包



GitHub 开源仓库

[github.com/Shun1989/Ericks-
WorkBuddy-Practice](https://github.com/Shun1989/Ericks-WorkBuddy-Practice)
全部模板/技能/SVG 源文件，MIT 协
议

汽研 Agent 中文社区

qiyanagent.org.cn
汽车与制造行业的 AI 应用者社区
每天整理行业动态与真实用法

© 2026 Eric · 本书为作者一线实战记录 · 欢迎转发，转载请注明出处